

Instruction Sets, Programs, and Proofs

Instruction Sets, Programs, and Proofs

Semantics, Equivalence, and Optimization

Michael J. Bommarito II

BOMMARITO PRESS

2026

Instruction Sets, Programs, and Proofs: Semantics, Equivalence, and Optimization

Copyright © 2026 Michael J. Bommarito II. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior written permission of the publisher, except for brief quotations in reviews.

First Edition.

Published by Bommarito Press.

Portions of this book were drafted and edited with the assistance of AI tools. All facts, citations, and final language were reviewed and verified by the author.

Printed in the United States of America.

Preface

“[T]his arithmetic by 0 and 1 is found to contain the mystery.”

—Gottfried Wilhelm Leibniz, “Explanation of Binary Arithmetic” (Leibniz
1703)

Leibniz found mystery in an exact rule. With only zero and one, he could represent numbers and calculate with them. Three centuries later, the same two marks describe the words stored in a processor, the instructions that transform them, and the proofs that say those transformations are correct. The mystery did not disappear as the rules became precise. Precision gave it somewhere to live.

This book begins with a modest question: what does it take to establish a claim about machine code? The question leads farther than its size suggests. A compiler replaces one instruction sequence with another. A programmer says that a sequence is the shortest possible. An instruction-set manual prints a table found by exhaustive search. Each statement concerns a finite machine, yet words such as *every*, *equivalent*, and *shortest* reach beyond the examples anyone can inspect.

That reach is one source of the subject’s beauty. A short program can still denote a function on every possible input state in its finite domain. A finite formula can ask whether any of those inputs makes the program fail. A finite refutation can sometimes rule out every cheaper program in a declared language. Small written objects can carry universal claims about machines. This book studies when they really do.

Testing remains indispensable. It can expose an error in a proposed program. It cannot by itself show that two programs agree on every input, or that no cheaper program exists. For those claims we need mathematics, and we need to know exactly what the machine evidence establishes.

The chapters first build a machine without choosing a commercial instruction set. They develop words, registers, memory, machine state, operand forms, and instructions as functions. This foundation lets us compare features often grouped under the labels reduced instruction set computer (RISC) and complex instruction set computer (CISC): explicit and implicit operands, load-store rules and memory operands, fixed and variable encodings, condition flags, and instruction extensions. RISC-V and x86-64 then become sustained contrasting companions. Optional vector extensions appear only when the scalar model’s boundary is itself the subject.

After that foundation, the book turns to equivalence, lower bounds, cost models, proof certificates, and exhaustive search. The deeper subject is the passage from an executable example to a statement that ranges over every allowed case. No particular instruction set owns that passage.

Readers should know how programs and processors work. Experience with a compiler, assembler, or systems code is enough preparation. No prior work with satisfiability solvers, proof assistants, or proof certificates is assumed. The mathematics uses functions, finite sets, elementary algebra, logic, and proof by contradiction. Each piece of notation arrives with the problem that needs it.

The intended reader is willing to stop at a small example and ask what has actually been defined. That habit matters more than prior formal training. A compiler engineer may recognize every instruction in an example yet still need to state which flags a rewrite preserves. A verification student may know the solver vocabulary yet still need to follow an effective address into memory. The book brings those two kinds of care together.

It can be read in three ways. For a first pass, execute the Ao examples by hand, follow the paired RV64 and x86-64 comparisons, and attempt the exercises marked by a concrete verb such as Execute, Break, or Transfer. For a course in semantics, add the definitions and reader proofs, requiring each answer to name its state and observation. For a course or reading group in machine-assisted proof, reproduce the artifact routes and their negative controls. The paths differ in depth, but none should reverse the dependency order: words before states, states before steps, steps before programs, and program meaning before optimization.

Examples use small widths because a reader should be able to inspect every case that carries the explanation. The real architectures retain their real 64-bit widths where the rule depends on them. Moving between those scales is part of the method. A width-four table reveals a carry boundary; algebra says which positive widths share it; a bit-vector route checks a declared machine instance. The text identifies which step supplies which reach.

Figures and colored panels are working parts of the exposition. Definitions fix objects. Key-idea panels state distinctions worth carrying forward. Warnings isolate attractive mistakes. Proof kits expose the shape of an argument before its details. Artifact panels bind a claim to evidence. A reader should be able to cover any panel and recover its role from the surrounding prose; the panel organizes the explanation but never replaces it.

Two rules govern the book.

First, scope is part of every technical claim. A statement about an instruction sequence must name the word width, allowed instructions, operand forms, constants, and cost model that give the statement its meaning. Change one of them and the answer may change. Scope is not fine print around a theorem. It is part of the theorem.

Second, every main theorem has a reader proof and a machine proof. The reader proof is small enough to reconstruct with pencil and paper. It may use an

invariant, a case split, or a replay. The machine proof runs the complete stated instance against a stored artifact. One supplies understanding; the other supplies coverage. A solver output is not an explanation, and an explanation is not a complete check of a large search space.

The evidence is recorded in objects. Each object joins a statement to its scope, artifact, checking route, and evidence status. A negative control sends a nearby false claim or damaged artifact through the same route. The checker must reject it. This modest demand catches a surprisingly empty ceremony: a checker that reports success no matter what it receives.

Different objects justify different kinds of belief. Some rely on a solver verdict. Some carry a certificate that a separate program checks. Others reproduce a finite calculation or enter a proof-assistant kernel. The text keeps these routes distinct. A bounded calculation does not become a theorem for every width, and a solver verdict does not become a kernel proof because both happen to end successfully.

The exercises are part of the development. They ask the reader to reproduce a result, damage an artifact, alter a machine, find a boundary, or carry an argument one step farther. At several points the exercises stop where the present proof library stops. An honest boundary is not a defect in the subject. It is where the next piece of mathematics begins.

The accompanying repository contains the object ledger, evidence artifacts, checkers, and build instructions. The mathematics can be studied without running the software. Running it reveals another part of the beauty: a claim about every input eventually becomes particular bytes, read by a particular program, with a particular way to fail.

The Introduction starts with fixed-width addition and asks which surrounding state gives that operation its meaning. Chapter 1 then constructs one machine word. From that smallest exact object, the book builds state, instructions, memory, programs, two real architectures, and finally the claims we can prove.

Contents

Preface	v
Introduction	xvii
Calculation by rule	xvii
A machine small enough to finish	xx
Two architectural companions	xx
From execution to claims	xxii
Reader proof and machine proof	xxvi
The book's four-part route	xxvii
How to study a chapter	xxviii
Axeyum's role	xxix
The object ledger	xxx
Exercises	xxxi
 I Constructing an Instruction Set	 1
1 Words and Their Meanings	3
1.1 Binary place value	3
1.2 Writing fixed-width patterns	5
1.3 Words as residue classes	6
1.4 Bitwise logic	10
1.5 Unsigned and signed readings	15
1.6 Changing widths	18
1.7 Words and bytes	20
1.8 Several arithmetic contracts	21
1.9 Width as an engineering budget	23
1.10 RV64 and x86-64 questions	24
1.11 What the word model fixes	25
1.12 Implementation boundary	25
1.13 Exercises	27

2	State	29
2.1	State replaces relevant history	30
2.2	Ao machine state	31
2.3	Architectural and physical state	35
2.4	State as a software contract	38
2.5	Observations	39
2.6	Reachability and invariants	44
2.7	RV64 and x86-64 state	45
2.8	Implementation boundary	47
2.9	What the state model fixes	48
2.10	Exercises	49
3	Instructions and Operands	53
3.1	Instructions as state transitions	53
3.2	From spelling to action	55
3.3	Operand forms	59
3.4	The single-step relation	65
3.5	Implicit and explicit operands	67
3.6	Instruction sequences	69
3.7	Instruction descriptions and tools	71
3.8	Implementation boundary	75
3.9	Exercises	76
4	Memory	81
4.1	Memory as a finite byte map	81
4.2	Stores and loads	86
4.3	Effective addresses and valid ranges	90
4.4	Aliasing and dependence	93
4.5	Address translation	95
4.6	Memory in RV64 and x86-64	98
4.7	The costs of memory	99
4.8	Implementation boundary	103
4.9	Exercises	105
5	Programs and Control	109
5.1	Programs and instruction bytes	109
5.2	Execution traces	113
5.3	Branches and control flow	116
5.4	Four outcomes of bounded execution	119
5.5	Replacement in context	124
5.6	Control contracts and costs	126
5.7	Implementation boundary	129
5.8	A complete Part I trace	131
5.9	Exercises	131

II	Reading Two Real Instruction Sets	135
6	Encoding and Decoding	137
6.1	Why instructions need encodings	137
6.2	Four distinct objects	139
6.3	Capacity of an instruction word	142
6.4	Codes, boundaries, and ambiguity	144
6.5	The Ao encoding	145
6.6	RV64I fixed-width fields	148
6.7	Encoding and linking	152
6.8	x86-64 variable-length encoding	153
6.9	Three decoder shapes	155
6.10	Decoder correctness	156
6.11	The cost of decoding	158
6.12	Axeyum route	160
6.13	Where the model stops	161
6.14	Exercises	162
7	Data Movement and Address Formation	167
7.1	Why machines move data	167
7.2	Values, locations, and transfers	169
7.3	Explicit movement in Ao	174
7.4	Effective-address calculation	176
7.5	Data movement in RV64I	181
7.6	Data movement in x86-64	182
7.7	Relating movement operations	184
7.8	The cost of data movement	186
7.9	Axeyum route	189
7.10	Where the model stops	190
7.11	Exercises	190
8	Arithmetic, Logic, and Condition State	195
8.1	Why machines record arithmetic conditions	195
8.2	Four questions about addition	197
8.3	From a one-bit adder to a circuit	201
8.4	The signed-overflow rule	204
8.5	Subtraction, borrow, and comparison	206
8.6	Bitwise logic	208
8.7	Multiplication and division	211
8.8	Condition state in three machines	212
8.9	Carry chains	215
8.10	Condition liveness	216
8.11	Policies for out-of-range results	218
8.12	Axeyum route	221

8.13	Arithmetic beyond integer words	223
8.14	Where the model stops	224
8.15	Exercises	225
9	Control Transfer	229
9.1	Why programs choose	229
9.2	Fallthrough, target, and link	232
9.3	Choice in the transition relation	233
9.4	Control transfer in Ao	235
9.5	Control transfer in RV64I	237
9.6	Control transfer in x86-64	239
9.7	The cost of control layout	241
9.8	Direct and indirect transfers	242
9.9	Control-flow graphs	247
9.10	One loop in three machines	251
9.11	From traces to control-flow claims	253
9.12	Axeyum route	254
9.13	Obligations of a branch proof	256
9.14	Where the model stops	257
9.15	Exercises	257
10	Procedures, Stacks, and ABIs	263
10.1	Why procedures require agreements	263
10.2	The procedure contract	265
10.3	Stacks and frame invariants	267
10.4	Procedures in Ao	270
10.5	Procedures in RV64I	271
10.6	Procedures in x86-64	274
10.7	Argument classification	277
10.8	One leaf contract in three machines	280
10.9	Nested calls and frame preservation	281
10.10	Protecting return control	283
10.11	Observing ABI conformance	285
10.12	Binary boundaries	286
10.13	Axeyum route	288
10.14	Where the model stops	289
10.15	Exercises	290
III	Proving Claims About Programs	295
11	Equivalence, Observation, and Refinement	297
11.1	Why program agreement matters	297
11.2	Programs produce behaviors	299

11.3	Preconditions and the promised domain	301
11.4	A hierarchy of agreement	303
11.5	Observing real instructions	306
11.6	Refinement across state spaces	307
11.7	Contexts expose forgotten state	310
11.8	Counterexamples as traces	311
11.9	Traps and undefined domains	313
11.10	Composing claims	315
11.11	Checking finite equivalence claims	316
11.12	Translation validation	317
11.13	Axeyum route	318
11.14	Where the model stops	320
11.15	Exercises	320
12	Relating Different Instruction Sets	325
12.1	Why translation needs a relation	325
12.2	Cross-machine state relations	330
12.3	Memory correspondence	334
12.4	Forward simulation	336
12.5	Absolute value on three machines	338
12.6	Relating and forgetting condition state	343
12.7	Fault and outcome correspondence	343
12.8	Axeyum route	345
12.9	Where the model stops	348
12.10	Exercises	349
13	Program Languages, Costs, and Minimality	353
13.1	From sequences to complete searches	353
13.2	Declare the candidate language	357
13.3	Symmetry and resources	362
13.4	The equivalence contract	365
13.5	Cost models	366
13.6	Matching upper and lower bounds	371
13.7	An Ao lower bound	371
13.8	Enumeration as evidence	374
13.9	Negative controls for search	380
13.10	RV64 and x86-64 search spaces	380
13.11	Axeyum route	382
13.12	Where the model stops	384
13.13	Exercises	384

14	Evidence That Can Fail	391
14.1	Why solvers produce proofs	391
14.2	Six forms of support	395
14.3	Binding claims to checkers	398
14.4	Evidence manifests	399
14.5	Negative controls	402
14.6	Checker design	405
14.7	A vacuous proof file	406
14.8	Formula generation	408
14.9	Independent checking	408
14.10	Kernel reconstruction	409
14.11	Production and retention costs	412
14.12	Resource failures	413
14.13	Axeyum route	416
14.14	Where the model stops	418
14.15	Exercises	419
IV	Synthesis and Boundaries	425
15	One Algorithm, Three Machines	427
15.1	The reduction problem	427
15.2	The algorithmic contract	433
15.3	The language-independent loop	436
15.4	The Ao implementation	440
15.5	The RV64I implementation	442
15.6	The x86-64 implementation	444
15.7	Bytes and machine boundaries	446
15.8	Local simulation lemmas	450
15.9	Relating three traces	453
15.10	A concrete execution	455
15.11	Breaking the proof	457
15.12	Costs after correctness	459
15.13	Axeyum route	463
15.14	Where the model stops	470
15.15	Exercises	471
16	The Edge of the Model	477
16.1	Models as instruments	477
16.2	Floating-point state	480
16.3	Vector state	483
16.4	Concurrency and event structures	485
16.5	Privilege and virtual memory	488
16.6	Self-modifying code	490

16.7	Timing and leakage	492
16.8	Verified compilation	495
16.9	Extending the model	498
16.10	Three boundary failures	500
16.11	Questions for every extension	501
16.12	Axeyum beyond the scalar core	502
16.13	Choosing the next boundary	502
16.14	Exercises	503
Acknowledgments		507
About the Author		509
Sources		511
Glossary		521
Index		527
Colophon		531

Introduction

Put 15 in a four-bit register and add 1. The register now contains 0. Put the same two values in wider registers and the result may be 16. Ask an operation to keep a carry bit, and another part of the state changes as well. Ask it to store the result, and addresses, byte order, and possible failure enter the story. The written symbol (+) did not change. The machine around it did.

An instruction set makes these choices precise. It says which values exist, where they live, how instruction bytes are decoded, which parts of state an operation reads and writes, how control moves, and what happens when execution cannot continue normally. A processor implements those rules. An assembler gives them a readable spelling. A calling convention adds agreements between separately written programs. These layers meet in ordinary code, but they are not the same layer.

This book takes them apart and puts them back together.

Calculation by rule

The wish to make calculation mechanical is older than the electronic computer. In 1703, Gottfried Wilhelm Leibniz described an arithmetic that used only the characters 0 and 1. He showed addition, subtraction, multiplication, and division in that notation. The paper did not describe electronic bits or a modern processor. It established the narrower fact we need here: two written characters suffice for a positional arithmetic. (Leibniz 1703)

The distinction matters because a symbol is not yet a physical state. Ink can form a 0. A relay can be open. A voltage can lie below a threshold. A magnetic region can point in one direction. Each medium can support two distinguishable conditions, but the conditions acquire the names 0 and 1 through an interpretation imposed by a designer. Chapter 1 studies the mathematical pattern. Chapters 2 and 4 ask what a machine must preserve for that pattern to remain usable as state and memory.

In 1936, Alan Turing went in the other direction. He did not begin with a particular relay, tube, or numeral system. He described a person following a finite table of rules while reading and writing symbols on a divided tape. That spare

construction made a procedure into a mathematical object. A finite description could govern an execution that had no fixed length in advance. (Turing 1937a)

Electronic builders then faced a different problem: how should a physical machine hold the numbers and orders that control its own work? The 1945 Electronic Discrete Variable Automatic Computer (EDVAC) draft divided the machine into arithmetic, control, memory, input, and output parts. Its memory held words available to automatic control, and the report treated binary digits as a practical choice for electronic construction. (Neumann 1945) The report did not create every element of the stored-program computer, and its authorship history is contested. It does show the abstractions being assembled into an engineering description.

By 1964, the relation between the description and the machine had become an economic commitment. The System/360 architects designed one machine-language interface for six IBM models whose performance differed by a factor of 50. They identified compatibility across the family as a design objective while also serving commercial, scientific, real-time, and logical work. (Amdahl et al. 1964) A customer could buy a larger implementation without treating every existing program as scrap. The instruction set had become a durable boundary between investment in software and change beneath it.

That boundary is one meaning of **computer architecture**: the features of a computer visible to the programs that use it, separated from the engineering choices used to realize those features. A register can remain visible across many processor designs. One design may use a simple pipeline; another may break the same instruction into internal operations and execute them out of order. If both preserve the specified architectural effect, the same machine program can run on both.

The architectural boundary buys freedom on each side. Hardware designers can change circuits without asking every application developer to rewrite a program. Software developers can target a specified interface without choosing a particular cache or execution unit. The price is constraint. Once customers depend on a behavior, changing it may cost more than preserving an awkward rule for another generation. Compatibility is therefore neither clutter nor purity. It is a technical property with owners, beneficiaries, and migration costs.

The present problem is not that those contracts have disappeared. It is that our claims about programs now travel through more layers. A compiler rewrites machine code. A **superoptimizer** is a tool that searches instruction sequences for a program that improves a declared cost. A binary translator moves work between architectures. A solver reports that no counterexample exists. The word **counterexample** means an input that exposes a difference between the claim and the computed behavior. A proof checker receives a certificate produced by yet another program. At every boundary, a correct calculation can answer the wrong question.

The practical cost appears at several places. A compiler developer spends time defining which rewrites preserve the language and target-machine rules.

A processor vendor spends design and validation effort on every architectural case software may exercise. A cloud operator pays for instruction throughput, memory traffic, and energy, not for the elegance of the instruction spelling. A formal-methods team pays to build semantics and proof infrastructure before it can retire a class of tests. These costs use different units. Combining them into one vague claim of efficiency conceals the decision.

The superoptimizer made that tension explicit. Massalin searched instruction sequences for the smallest implementation of a function in 1987. (Massalin 1987) Later systems made the search broader and more systematic. **Superoptimization** is this search for an optimal program inside a declared program language, from small peephole replacements to parallel hierarchical search. (Bansal and Aiken 2006; Lu and Bodík 2025) The modern question is no longer only whether a machine can discover good code. It is what the word *good* means, whether the search covered the declared language, and what evidence a skeptical reader can check afterward.

The word *optimal* has the same problem. It may mean fewest instructions, fewest encoded bytes, least estimated latency on one processor, lowest energy under a measurement protocol, or lowest monetary cost for a workload. Each choice orders programs differently. A proof can establish a minimum only after the candidate programs and the ordering have been fixed.

The book's central question

What must be defined, executed, related, and checked before a statement about machine code deserves belief?

This question includes ordinary correctness before it reaches optimization. If two programs are said to agree, we need their state, inputs, outcomes, and observation. If one is said to refine the other, we need the relation between different machine states. If one is said to be minimal, we need the candidate language and cost. The proof machinery comes last because it can only preserve the meaning supplied to it.

A stack of interpretations

The same physical event can be described at several levels. A circuit crosses a voltage threshold. The designer interprets the resulting stable condition as a bit. A group of bits is interpreted as an instruction word. A decoder interprets fields as an operation and operands. The operational semantics interprets that decoded instruction as a state transition. A program interprets many transitions as a computation. No level is made false by the level below it, but each connection carries assumptions that must be stated.

A machine small enough to finish

We begin with an abstract instruction set called Ao. It has fixed-width words, eight registers, byte-addressed memory, a program counter, four condition bits, and normal or exceptional outcomes. Its instructions move data, perform integer arithmetic and logic, access memory, compare values, branch, and halt.

Ao is deliberately artificial. No hardware vendor must preserve its past, and no software ecosystem depends on its encodings. We can therefore print its whole relevant state and give every core instruction a complete rule. When an example behaves unexpectedly, there is nowhere for a hidden architectural custom to hide.

The point is not to design an ideal processor. Ao gives us a common language for questions. What is a word? What counts as state? Does an operand name a value, a location, or both? When do two addresses overlap? Does a branch read registers directly or read condition state written earlier? What does it mean for an execution to end?

After answering each question in Ao, we ask it again of two real instruction sets: x86-64 and RISC-V. Their differences then become informative rather than bewildering. We already know the role that needs to be filled. We can inspect how each architecture fills it.

Why invent Ao instead of starting with a real manual?

A real manual must serve compatibility, implementation, and reference needs. Its definitions are distributed across chapters, modes, tables, and exception rules. Ao lets the reader see one complete semantic dependency chain first. The real manuals then become sources we can interrogate with precise questions, not forests in which we hope to recognize familiar mnemonics.

Ao also makes mistakes cheap. We can reverse byte order, omit a condition write, or move a branch target by four bytes and immediately ask which claim breaks. Those mutations become negative controls for the later executable model. A teaching machine should make both the right rule and its nearest wrong neighbors easy to inspect.

Two architectural companions

The reduced-instruction movement arose from a concrete engineering argument, not from a rule that fewer mnemonics must be better. During the 1970s and early 1980s, researchers asked whether complex instructions justified the control hardware, design time, and chip area they consumed. The Berkeley RISC I project used the growing capacity of very-large-scale integration (VLSI) to build a small processor around simple instructions, a large register file, and pipelined execution.

Its 1981 paper reported the instruction set together with the chip resources used to realize it.(Patterson and Séquin 1981)

The opposing category was less tidy. Existing machines had acquired rich instructions for many reasons: compact code, expensive memory, convenient assembly programming, compatibility, and the use of microcode to implement a complex visible operation with simpler internal steps. Calling all of them *complex instruction set computers* turned a varied history into the other side of a debate. The label can still point to useful tendencies. It cannot replace the rule for one instruction.

Technology also moved beneath the argument. Transistors became more abundant. Caches reduced the apparent cost of some memory accesses. Compilers improved. Modern x86-64 processors may translate a visible instruction into internal operations, while RISC-V permits optional extensions and variable-length encodings. The old labels no longer predict a pipeline by themselves. They remain useful when attached to a named choice, such as whether arithmetic can read memory directly or whether an instruction has a fixed base length.

RISC-V and x86-64 are often placed on opposite sides of a reduced-versus-complex instruction-set contrast. The labels are historically useful, but they cannot supply an instruction's meaning. They do not tell us which register changes or how many bytes an encoding occupies. Nor do they say whether arithmetic writes condition flags, which forms access memory, or what a call does before an application binary interface (ABI) gives the stack a role.

The book compares those dimensions directly. A base RISC-V instruction can name a destination register separately from two sources. A selected x86-64 arithmetic form can use its destination as an input and also change condition state. RISC-V commonly expresses memory traffic through load and store instructions. Selected x86-64 arithmetic forms can name memory as an operand. The base encoding structures differ. So do some of the special rules attached to registers and operand widths.

None of these observations declares a winner. Each one creates a semantic obligation. We have to record the right state, decode the right instruction form, and compare the right outcomes. The labels RISC and CISC can summarize a family resemblance after that work. They cannot perform it.

The real architectures in this book are also bounded slices. The name x86-64 will never silently mean every historical mode, optional extension, privileged operation, and processor timing table. The name RISC-V will never silently mean every standard or proposed extension. Each load-bearing example will name the instruction forms, state, source revision, privilege assumptions, and ABI convention it uses.

Those boundaries reflect two different economic histories. The Intel manuals describe a programming environment accumulated across a long line of compatible processors. The current manual set separates basic architecture, instruction reference, system programming, and model-specific state because no small chapter

could responsibly make all of those layers implicit. (Intel Corporation 2026a; Intel Corporation 2026b) Software compatibility gives that accumulated interface value. It also makes removal and redesign expensive.

RISC-V began with a chance to choose a new base. Its specification states that the ISA was designed for research and education and for use as a free, open industry standard. It separates small base ISAs from optional extensions and tries not to require one processor implementation style or implementation technology. (RISC-V International 2026e) Openness changes who may implement the standard. It does not make every implementation, core, or software toolchain identical. The word **toolchain** means a connected set of programs that turns source code into a runnable program. It includes the compiler, assembler, and the program that joins separately built parts. Compatibility still depends on a pinned base, extensions, privilege rules, platform, and ABI.

The comparison therefore has an economic dimension without becoming a market contest. One architecture carries a large installed software inheritance. The other makes a stable specification available for many implementations and custom extensions. Both must spend engineering effort on conformance, toolchains, operating systems, and application software. This book compares semantic features, then asks what each feature requires from those actors.

Family labels are not causal explanations

Do not explain an effect by saying only that an architecture is RISC or CISC. Name the actual dimension: operand role, instruction length, memory form, condition state, address calculation, call mechanism, or extension boundary. The label may summarize history after the rule is known.

The sustained comparison has a second purpose. An abstraction that seems natural after reading only one architecture may merely reproduce that architecture's habits. Ao places condition bits in state, but RV64 branch predicates remind us that control need not consume a separately stored flag. Ao uses fixed four-byte encodings, but x86-64 reminds us that instruction length can itself be a decoded result. The companions pressure-test the foundation.

From execution to claims

Suppose two short programs leave the same value in their designated output register. Are they equivalent?

Perhaps. One may also change a scratch register. One may change condition state that a later branch reads. One may touch memory, trap on an address, or stop after a different number of instruction bytes. Agreement on the selected output is a fact. Equivalence is a claim whose observation and preconditions must say whether the other differences matter.

The book therefore constructs state before it studies optimization. A local rewrite is not a relation between two lines of assembly alone. It is a relation between their behaviors for every allowed starting state. If the rewrite crosses architectures, equality is usually the wrong shape. The two machines have different register names and condition state. We instead relate their inputs and show that their executions produce related outputs.

The difference between a test and a proof begins here. Running both programs on ten inputs may reveal an error. Passing ten inputs does not settle a claim about every allowed state. For small domains we may enumerate all cases. For larger finite domains we may ask a solver for a counterexample. If none exists, we must still ask what formula the solver received and which semantics produced it. We must also ask what evidence an independent program can inspect and reject when damaged.

The machinery is valuable only after the statement is right. A perfect proof of a formula about the wrong flags or a reversed byte order proves the wrong thing with unusual confidence.

Consider two four-bit programs. Program P adds 1 to register r_0 and halts. Program Q adds 1 to r_0 , clears r_1 , and halts. If the only reported output is r_0 , the programs agree on every input. If the caller may inspect r_1 , they do not. Neither conclusion is a correction to the other. They answer different questions because they use different observations.

Now give addition a carry output. On an initial $r_0 = 15$, both programs leave 0 in r_0 , but a version that forgets to update carry no longer agrees with one that follows the full addition rule. A test that starts at $r_0 = 3$ will miss the defect. The boundary input 15 exposes it. This small example contains the structure of much larger verification tasks: declare the input set, execute a complete state transition, select an observation, and quantify over every allowed start.

The order matters. If we first notice that the printed values of r_0 match, we may be tempted to name the programs equivalent and repair the definition afterward. The book uses the reverse discipline. We decide what a client may observe, then ask whether the two executions agree under that observation. A claim is not a favorable output with a grander name.

Observation relative to a client

An observation is a declared function that extracts the parts of a machine outcome a particular client may distinguish. Two outcomes agree for that client when the observation returns the same result for both. Changing the client may change the observation and therefore change the claim.

Figure 1 shows why a final green check is not enough. The checker may faithfully validate an artifact for a generated problem. The generator may faithfully encode executable semantics. Those semantics may still omit a flag that the original

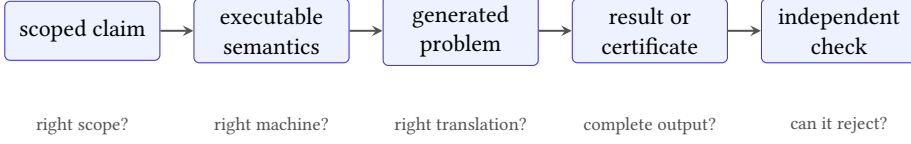


Figure 1: A machine-backed claim crosses several trust boundaries. Each arrow needs its own explanation and failure test.

claim meant to preserve. Assurance accumulates only when every connection is identified.

A correct proof of the wrong byte order

Suppose a generator reverses the weights used to join four memory bytes. A solver and certificate checker may correctly establish every property of that generated function. The result still does not prove a claim about Ao loads. The term **little-endian** means that the least-valued byte goes at the lowest address. The failure lies before the solver, at the connection between the printed semantics and the generated problem.

Four kinds of machine claim

One sequence on the page can support several kinds of sentence. Their quantifiers determine what evidence can settle them. Let S be the allowed starting states, let $P(s)$ be the outcome of running program P from state s , and let O be the observation chosen by the client.

The first sentence asks for a witness:

$$\exists s \in S. O(P(s)) = 7.$$

One execution can settle it. If we exhibit an allowed state and replay the program to the observed value 7, the witness establishes existence. Testing another thousand states does not change the logical shape.

The second sentence asks for correctness over the whole input set:

$$\forall s \in S. O(P(s)) = f(s).$$

No selected example settles this statement unless S contains only that example. A proof needs a reason that covers every allowed state. For a small finite set, complete enumeration may provide that coverage. For a larger set, an algebraic argument, induction, invariant, or checked refutation of the negation may do the work.

The third sentence compares programs:

$$\forall s \in S. O(P(s)) = O(Q(s)).$$

The equation states observational equivalence under the chosen start set and observation. The equation does not compare source text, instruction count, execution time, or unobserved registers. Enlarging O strengthens the claim because the programs must agree on more. Shrinking S weakens it because fewer starts remain to be checked.

The fourth sentence compares one program with an entire language of cheaper candidates. If $L_{<c}$ contains every allowed program whose declared cost is less than c , **minimality** is the absence of a cheaper equivalent. It has the form

$$\neg \exists Q \in L_{<c}. \forall s \in S. O(P(s)) = O(Q(s)).$$

The statement rules out an equivalent cheaper program. A search that fails to find one is not enough by itself. We must know that the generator represented every member of $L_{<c}$, that the semantics and observation match the printed claim, and that the final evidence rules out the resulting existential formula.

Claim	Smallest decisive evidence	Common mistake
Existence	One valid witness and replay	Treating many failures as nonexistence
Universal correctness	An argument or check covering all allowed starts	Treating a test suite as the domain
Equivalence	All starts under one declared observation	Comparing only the intended output
Minimality	An upper-bound witness plus exclusion of every cheaper candidate	Reporting that a search found nothing

Table 1: The quantifier determines what kind of evidence can settle a claim.

Table 1 is a compact map for the rest of the book. The important distinction is not whether a solver was used. It is whether the evidence matches the logical burden of the sentence.

Change one quantifier

Take the claim “program P returns o for every four-bit input.” Replace “every” with “some.” Then replace the output o with “the same output as Q .” Write the three formulas. Name the smallest decisive evidence for each one. Give one test result that would leave each claim unsettled.

Reader proof and machine proof

Every load-bearing theorem in this book needs two forms of support. The reader proof exposes the reason. It may use a width-four example, an invariant, a case split, a short simulation, or a contradiction that can be reconstructed at a blackboard. The machine proof covers the declared finite space through an artifact and a checker that can reject damaged evidence.

Neither can replace the other. A solver log does not explain why a branch rewrite works. A persuasive hand calculation does not cover every 64-bit input. The hand argument tells us what the encoding should preserve. The machine route tells us whether the industrial-size instance contains an exception.

Computations keep a separate name. An exhaustive run can be valuable and reproducible without carrying a proof certificate. The book states which kind of evidence it has rather than promoting every successful command to the same rank.

We also make the checker lose. A nearby false claim, mutated instruction, damaged trace, or truncated certificate must be rejected for the expected reason. This negative control cannot prove the original statement. It can show that the route distinguishes at least one precise error from success.

Why separate production from checking? Discovery and verification often have different costs. Finding a program may require exploring millions of candidates. Checking the program only requires executing its few instructions and comparing the result with the specification. Finding a contradiction in a large Boolean formula may consume substantial solver time. Checking a well-formed proof certificate can follow a narrower sequence of local steps. The expensive search is then repeatable without requiring every reader to trust the search program's heuristics.

The separation resembles manufacturing inspection, but the analogy stops at an important boundary. A physical inspection samples or measures an object that may vary from the next object produced. A proof checker reads a finite symbolic artifact under fixed rules. If it checks every required inference, its coverage is determined by the proof system, not by a sampling rate. The remaining risks lie in the checker, the proof system, the formula generator, and the connection between that formula and the machine claim.

This division also changes engineering economics. A team can spend heavily on one proof-producing tool while keeping the trusted checker small enough to review, test, and reproduce elsewhere. The smaller trusted base does not make the whole route correct by decree. It gives reviewers fewer lines and rules on which the final acceptance depends. Chapter 14 will put exact trust classes on these routes.

The two proofs also fail differently. A reader argument may omit a case or use an unstated premise. A machine route may encode the wrong operation, search an incomplete candidate language, accept a damaged certificate, or lose the provenance of its inputs. Placing them beside each other creates useful friction. The hand proof tells us which invariant and controls to expect. The machine result tells us where the friendly miniature stopped covering the real finite domain.

Three recurring proof shapes

Word and state laws often use a case split or direct algebra. Program equivalence often uses an invariant or simulation across steps. Minimality usually argues by contradiction: assume a cheaper candidate exists, encode that existential claim, and refute every candidate in the declared language. The shape of the proof follows the quantifier in the claim.

Checking is not free certainty

A small checker can still implement the wrong proof rule. A valid certificate can still concern a formula generated from the wrong instruction semantics. The value of independent checking is narrower: it removes the need to trust the producer's search procedure for the inferences recorded in the artifact. Controls and semantic review must cover the remaining connections.

The book's four-part route

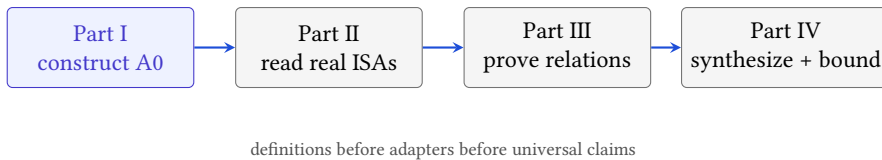


Figure 2: The curricular order follows the dependency order of the eventual proofs. Later parts may not bypass a missing semantic layer.

Part I constructs Ao. Chapter 1 begins with words and the several meanings a fixed pattern can receive. Chapters 2 through 5 add state, operands, instructions, memory, programs, and control. By the end, the reader can execute a small Ao program by hand and say exactly why each state change occurs.

Part II keeps one concept fixed while placing RISC-V and x86-64 beside it. Encoding and decoding come first because instruction bytes and instruction meaning are different objects. Data movement, addresses, arithmetic, condition state, control transfer, procedures, stacks, and ABIs follow. The comparison stays paired: we do not teach one architecture as normal and the other as a collection of exceptions.

Part III asks what can be proved about programs. It develops observation, equivalence, refinement, cross-ISA simulation, and candidate program languages. It then asks about cost and **minimality**, the claim that no cheaper candidate exists. The proof system grows from the semantics already on the page.

Part IV carries one scalar algorithm through Ao, RV64, and x86-64. The final chapter then marks the edge of the model: floating point, packed and vector state, concurrency, privilege, virtual memory, self-modifying code, timing, and leakage all require more structure than the scalar sequential core.

The order is an argument. First say what a machine step means. Then compare real steps. Then relate programs. Then ask a finite artifact to support a universal sentence.

The chapters are not independent essays. A byte-order proof in Chapter 4 depends on the word decomposition in Chapter 1. A trace in Chapter 5 depends on the complete step in Chapter 3. A cross-ISA simulation in Chapter 12 will depend on both real-ISA adapters from Part II. A minimality statement in Chapter 13 will depend on all of those semantics plus a candidate language and cost. The dependency chain prevents a solver formula from quietly becoming a machine theorem.

How to study a chapter

The book is meant to be used with pencil, terminal, and source manual near one another. New material normally arrives in five passes. First comes an object in ordinary language: a word, state, operand, instruction, trace, or relation. Next comes a mathematical definition precise enough to settle boundary cases. Then a small instance is executed or proved by hand. A real-architecture comparison asks which parts survive contact with RV64 and x86-64. Only then does an Axyum route enlarge or check the claim.

Do not skip the hand execution because the code can run it. For a transition, write the old state on the left and the new state on the right. Circle every component the instruction reads. Mark every component it writes. State why the untouched components remain unchanged. This practice catches omitted state before a solver turns the omission into thousands of flawless but irrelevant cases.

Proofs deserve a similar first pass. Before following symbols, identify the quantifier. Is the claim about one execution, every word of a fixed width, every state satisfying a premise, or every candidate below a cost bound? Then identify the proposed reason: decomposition, invariant, simulation, or contradiction. The details become easier to judge once the burden is visible.

A six-question reading routine

For each load-bearing claim, ask:

1. What are the objects, and are their widths and state components fixed?
2. Which starting objects are allowed?
3. What outcome or observation is compared?

4. Is the claim about one case, a finite domain, or an unbounded family?
5. What is the human-scale reason it should hold?
6. What artifact, checker, and failing control cover the machine route?

If one answer is missing, pause there. Later machinery cannot supply an earlier definition.

Exercises carry part of the exposition. The early ones rehearse a definition on a new value. Middle exercises alter a premise or observation and ask which conclusion survives. Later ones ask for a proof, a counterexample, or a small checker. Some deliberately present an attractive false statement. Finding the smallest state that breaks it is often more instructive than reading another warning about hidden assumptions.

The figures also have jobs beyond decoration. A state diagram inventories what must be preserved. An encoding diagram separates bit fields from their decoded roles. A control-flow graph turns a program counter update into a visible fork. When a figure seems obvious, try to redraw it from the adjacent definition. Any edge or field that cannot be justified is a useful question for the text or model.

Readers may choose different depths without changing the logical order. A first course can execute Ao examples, study the paired ISA cases, and complete selected reader proofs. A verification course can add every artifact and negative control. An experienced compiler engineer may move quickly through basic arithmetic but should resist skipping the observation and state relations: those are where familiar optimization claims acquire exact scope.

Axeyum's role

Axeyum is the evidence stack used by the accompanying repository. It supplies bit-vector expressions, solver routes, certificate machinery, and other formal infrastructure. This book asks it to grow new architectural layers: Ao execution, bounded real-ISA slices, decoders, state relations, and replay. The software is introduced when a mathematical need appears rather than in a detached command tour.

The first use is deliberately modest. A reader constructs one fixed-width word, evaluates an operation, and asks whether a proposed identity has a counterexample. The next layer constructs complete Ao states and observations. Later layers execute instructions and traces, relate states from different machines, generate solver problems, and check evidence. Each addition should reuse the semantics already exercised by hand.

The four questions to ask of an Axeyum result

What exact statement entered the route? Which executable semantics generated it? What artifact or model came back? Which independent program checked or replayed the result, and what nearby false input did that route reject?

The chapter sequence is not an Axeyum reference manual. Commands and interfaces may change; the evidentiary questions should not. The repository guide pins the checkout, build steps, route, and limitations used by each machine-backed object. A reader interested only in the mathematics can follow the reader proofs. A reader interested in trust can reproduce the machine route and attack its controls.

At the current boundary, Axeyum has useful formal substrate but not the Ao, RV64, x86-64, and cross-ISA packages the full book requires. Planned examples remain labeled as obligations until those packages and their controls run. This prevents tutorial prose from becoming accidental evidence.

The object ledger

The project keeps a machine-readable record for its definitions, claims, and unresolved obligations. Chapters take scope and evidence descriptions from that record rather than inventing stronger wording in prose. Evidence files carry raw digests. Checkers return failure when a claim, artifact, or control does not meet its contract.

The ledger is not the subject of the first chapters. A reader should meet a word before meeting its object record. They should understand a state transition before meeting its formula. The record keeps the book honest when a claim moves between prose, executable semantics, solver input, and checked result.

Three distinctions in that record matter throughout the book. A definition fixes vocabulary and creates an implementation obligation; it does not become truer because software happens to represent it. A theorem makes a universal claim and needs an argument that covers its stated domain. A computation reports the result of a reproducible finite procedure. It may settle a bounded question, find a witness, or expose a counterexample without pretending to be a proof of something larger.

The printed artifact panels bind those distinctions to concrete evidence. An identifier points to the record. The scope names widths, instruction forms, starting states, observations, and cost assumptions. The route names the generator or executor and the checker. The limitation says what the route does not establish. A digest detects a changed file; it does not establish that the file expresses the intended machine. That connection still needs semantic review and controls.

Every chapter makes the same promise

The ordinary prose will say what an object means and why a claim should hold. The formal statement will fix its scope. When machine evidence exists, the artifact route will say how to reproduce and challenge it. When a required route does not yet exist, the book will name the missing work instead of borrowing certainty from a nearby calculation.

This separation also lets the manuscript improve without rewriting its own history. A missing executable layer can later be implemented and checked. A computation can later gain a certificate. A false conjecture can be replaced by its smallest counterexample. The prose must then change where the evidence changes, but the reader never has to infer that a successful command silently upgraded the kind of claim being made.

Exercises

These exercises establish habits used throughout the book. Each answer should name the object, scope, evidence, and nearest excluded case. A favorable output alone is not a complete answer.

Reproduce: follow the interpretation stack

Write the four-bit pattern 1011 on paper. Give it three interpretations: an unsigned integer, a signed two's-complement integer, and an instruction field whose value selects operation 11. Which facts belong to the pattern itself? Which enter only with an interpretation? Then name one physical medium that could store the pattern and the threshold or distinction needed to recover each bit.

Break: enlarge the observation

Program P adds 1 to r_0 . Program Q adds 1 to r_0 and clears r_1 . Begin with an observation that returns only r_0 , then add r_1 , the carry bit, and the number of executed instructions one at a time. For each observation, decide whether equivalence survives. Give the smallest starting state that separates the programs when it does not.

✂ Classify the claim

Classify each sentence as existence, universal correctness, equivalence, minimality, or none of those:

- (a) This sequence returns 7 on input 3.
- (b) Some three-instruction sequence returns 7 on input 3.
- (c) These sequences agree on all 8-bit inputs.
- (d) No two-instruction sequence computes the function on all 8-bit inputs.
- (e) This sequence ran faster in one benchmark trial.

State the additional scope that sentence (e) needs before it can support a comparison.

✂ Attack: the checker that always accepts

A checker reads a certificate file and exits successfully without inspecting its contents. The correct certificate passes. Explain why that positive result provides no useful evidence about the checker. Design one negative control, state the expected failure, and explain what remains untested even if the control fails as expected.

✂ Transfer: separate architecture from implementation

Choose one instruction from a processor manual. List its architectural inputs, outputs, and exceptional cases. Then list three implementation choices the manual permits a processor designer to change without changing that visible effect. If the manual does not make the separation clear, record the exact question you would need another source to answer.

✂ Generalize: price a compatibility choice

Suppose a vendor wants to remove one rarely used instruction. Name the parties who might bear the migration cost: processor designers, compiler teams, operating-system developers, library vendors, application owners, and users with old binaries. Choose two parties and give a concrete unit for each cost. Do not decide whether removal is good until the measurement and compatibility scope are stated.

At present, the mature solver machinery is ahead of the architectural layers this revised book requires. The missing work is stated plainly. Axeyum must gain reusable Ao state and execution, source-pinned RV64 and x86-64 slices, decoders, and cross-machine state relations before the later chapters can claim semantics-backed ISA proofs. A handwritten bit-vector identity is not a substitute for those layers.

That boundary is a useful place to begin. We know the machine we need to construct, the two real systems against which it must answer, and the kinds of evidence that would let a skeptical reader check the result. Chapter 1 starts with the smallest object any of them can hold: one fixed-width word.

PART I

Constructing an Instruction Set

Words and Their Meanings

The pattern 11111111 can be read as 255, as -1, as eight Boolean values, as a mask, or as one byte of an instruction. The pattern does not confess which reading is intended. The operation or convention supplies it.

That distinction is the first foundation of instruction semantics. A register stores a fixed number of bits. An operation supplies a reading and a rule for producing another fixed pattern. We begin with the finite set on which all those rules act.

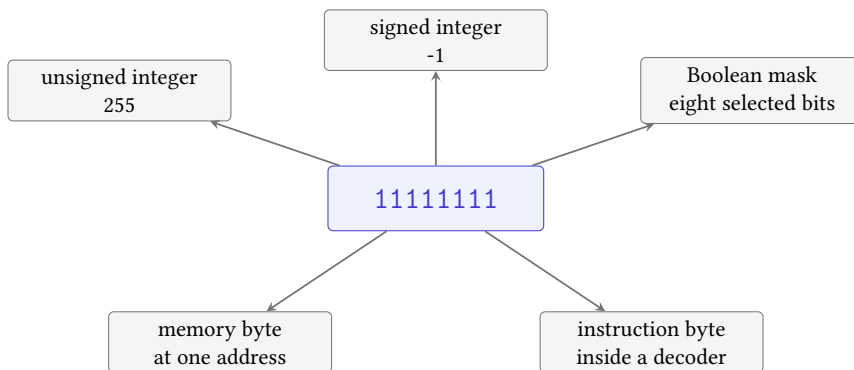


Figure 1.1: A stored pattern acquires a meaning only through a declared reading or use. The arrows are interpretations, not changes to the eight bits.

Figure 1.1 warns against a common shortcut. We may say that a register “contains -1” when the signed reading is already clear. The machine stores the pattern. The signed operation, comparison, conversion, or printing convention supplies the -1. This distinction may feel fussy on one byte. It becomes decisive when two instructions preserve the same bits but give them different roles.

1.1 Binary place value

1.1.1 Reusable positional rules

A positional numeral assigns a weight to each place. In decimal, those weights are powers of ten. In binary, they are powers of two. The gain is not merely a shorter alphabet. One small collection of carrying and place-value rules works at every position, so a finite procedure can act on numerals much longer than the examples used to teach it.

Mechanical and electronic computers inherited that advantage. A circuit can store two stable alternatives and combine them with Boolean operations. A register groups a fixed number of those positions. The instruction set then defines what operations may do to the group: add it, shift it, compare it, split it into bytes, or treat it as an address.

Binary storage did not remove interpretation. It multiplied the useful interpretations of one pattern. The same positions can support unsigned arithmetic, two's-complement arithmetic, logical masks, encodings, and pieces of larger structures. Modern verification must therefore bind the pattern to the intended width and operation before it can establish a claim.

Place value is already an algorithmic idea

The written digits are finite data. A rule for their weights tells us how to interpret every numeral of the admitted length. Machine words keep that compact bargain and add a fixed boundary: beyond the highest position, the operation must say what happens.

1.1.2 Unique positional values

Let a width- w pattern have bits $b_{w-1} \dots b_1 b_0$, where every b_i is either 0 or 1. Its unsigned positional value is

$$U_w(b_{w-1} \dots b_0) = \sum_{i=0}^{w-1} b_i 2^i.$$

The rightmost bit has weight 1, the next has weight 2, and position i has weight 2^i . At width four,

$$U_4(1011) = 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11.$$

This equation is a reading of the pattern. It is not a claim that the marks on the page contain the number 11 without a convention.

The sum always lies in the natural representative range. Its smallest value is zero. The largest value has a one in every position.

$$1 + 2 + 4 + \dots + 2^{w-1} = 2^w - 1.$$

The identity follows by doubling the sum and subtracting the original:

$$2(1 + 2 + \cdots + 2^{w-1}) - (1 + 2 + \cdots + 2^{w-1}) = 2^w - 1.$$

Thus a width- w pattern never reads outside $0, \dots, 2^w - 1$, and every possible word representative has room in that interval.

The converse is constructive. Given a representative $x < 2^w$, divide it by 2. The remainder is bit b_0 , and the quotient contains the remaining high bits. Divide the quotient by 2 again to obtain b_1 . Repeating the step w times produces the pattern. For 11, the successive quotient-remainder pairs are

$$11 = 2 \cdot 5 + 1, \quad 5 = 2 \cdot 2 + 1, \quad 2 = 2 \cdot 1 + 0, \quad 1 = 2 \cdot 0 + 1.$$

Reading the remainders from last to first gives 1011.

No second width- w pattern can have the same unsigned value. Suppose two patterns first differ at their highest differing position j . One has a one there and the other has a zero. The weight 2^j exceeds the sum of all lower weights,

$$1 + 2 + \cdots + 2^{j-1} = 2^j - 1.$$

No arrangement of lower bits can cancel that difference. The two positional sums must be unequal.

Existence and uniqueness are separate

Repeated division shows that every natural representative below 2^w has a width- w binary pattern. The highest-differing-position argument shows that it has at most one. Together they establish a one-to-one correspondence between the 2^w patterns and the natural representatives $0, \dots, 2^w - 1$.

This correspondence is the hinge between two useful views. The bit view makes positions, masks, and encodings visible. The residue view makes boundary arithmetic concise. We may move between them because split-by-division and join-by-place-value undo each other. Later proof routes must preserve that hinge; a solver formula that numbers the bits in the opposite direction describes a different reading.

1.2 Writing fixed-width patterns

Binary notation writes one symbol for every bit, which makes bit positions visible and long words tiresome. Hexadecimal notation groups four bits into one digit. The digits 0 through 9 keep their usual values; A through F stand for 10 through 15. Thus

$$1111 = 0xf, \quad 1010\ 0110 = 0xa6.$$

The prefix 0x tells us that the following digits are hexadecimal. It does not tell us the width. The written numeral 0xff may be an 8-bit word, the low byte of a 32-bit word, or an integer literal that an assembler will later fit into an instruction field.

A value is not yet a width

The numeral 255 names a mathematical integer. A word names an element of a fixed finite set. Before applying a machine operation, state the width and the rule that maps the numeral into that set.

Leading zeroes carry this missing information when the context has not already fixed it. The patterns 1111 and 00001111 have the same unsigned reading, but they are different-width words. They offer different high bits, different points where arithmetic returns to zero, and different results under a width-sensitive operation such as bitwise not.

Hexadecimal is useful because the grouping is exact. Each hexadecimal digit has sixteen possible values, and four bits have $2^4 = 16$ possible patterns. The mapping is one-to-one:

hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
value	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

To convert binary to hexadecimal, group from the low end in blocks of four. To convert back, replace each digit with its four-bit pattern. No arithmetic meaning is lost, provided the width or leading group is retained.

Decimal notation behaves differently. The value 255 has a short decimal spelling, but its decimal digits do not align with fixed groups of bits. Converting between decimal and binary uses repeated division or a weighted sum. Converting between hexadecimal and binary is a local substitution at each group. This is why architecture manuals, debuggers, masks, addresses, and instruction dumps commonly use hexadecimal: it is compact while leaving bit boundaries recoverable.

The notation still does not choose an interpretation. The spelling 0xfffffffff can denote the unsigned value $2^{32} - 1$, signed -1, four bytes, a mask, or part of a wider object. Hexadecimal makes the pattern easier to inspect. It does not decide what an operation means.

1.3 Words as residue classes

Begin with a four-bit counter. It visits the patterns for 0 through 15. Add one to 15 and the counter returns to 0 because there is no fifth bit in which to store 16. This is **wraparound**: arithmetic returned from the largest word to zero. Add two to 15 and the counter stops at 1.

A0 word

For $w \geq 1$, a **word of width** w is a residue modulo 2^w . We use the natural representatives $0, 1, \dots, 2^w - 1$. Addition, subtraction, and multiplication take the ordinary integer result and keep its remainder modulo 2^w .

Object `A0.def.word` records this carrier and its readings. The quotient notation says that integers differing by a multiple of 2^w represent the same stored word. At width four, 16 and 0 therefore represent the same word, as do 17 and 1.

The reader proof of wraparound follows from division with remainder. For any width-four word x , ordinary addition gives $x + 1$. If $x < 15$, that number already lies between 1 and 15, so taking the remainder changes nothing. If $x = 15$, then $x + 1 = 16 = 1 \cdot 16 + 0$, whose remainder after division by 16 is 0. The apparent exceptional case is simply the boundary of the finite carrier.

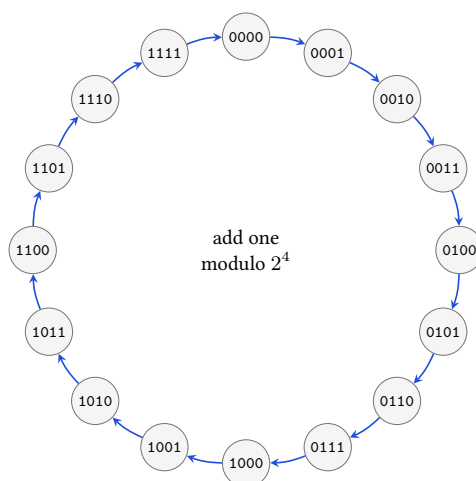


Figure 1.2: Addition by one visits every width-four word and returns to zero after 1111. The circular layout shows the modular rule; it is not a picture of storage or time.

The cycle in Figure 1.2 gives the right shape but not the definition. A diagram might suggest that the counter physically travels around a ring. It does not. The remainder operation supplies a successor for each of the sixteen words, including the edge case 1111. The circle is useful because the last arrow is governed by the same rule as every other arrow.

There are exactly 2^w words of width w . A unary instruction on one word is therefore a function on a finite set. At width 64 the set is too large to list usefully, but it still has a declared edge. One finite rule gives an answer for every one of those inputs.

A proof from one decomposition

To show that adding one wraps exactly at the largest width- w word, split the domain into two cases. If $0 \leq x < 2^w - 1$, then $x + 1 < 2^w$, so its remainder is itself. If $x = 2^w - 1$, then $x + 1 = 2^w$, whose remainder is zero. The two cases cover every natural representative, and the boundary appears in only the second case.

This argument is more useful than listing all inputs. It identifies the load-bearing fact: every representative except the largest has room for one more, while the largest is one less than the modulus. The same split works at width 8 or width 64 without asking us to inspect 2^{64} rows.

1.3.1 Congruence and representatives

The notation is older than the electronic computer. In the opening sections of *Disquisitiones Arithmeticae*, Gauss gave a name and a compact symbol to the relation between integers that leave the same remainder. He wrote that one number is congruent to another for a modulus when their difference is divisible by that modulus. (Gauss 1801) Our special choice of modulus, 2^w , comes from the number of patterns in a width- w word.

Write $a \equiv b \pmod{2^w}$ when $a - b$ is a multiple of 2^w . The two integers then name the same width- w word. At width four, $-1 \equiv 15 \pmod{16}$ and $17 \equiv 1 \pmod{16}$. Choosing the natural representative gives a convenient stored spelling; it does not erase the other integers in the same residue class.

Congruence is preserved by addition and multiplication. If a and b name the same word, then adding the same integer c to both produces another pair whose difference is still a multiple of 2^w . Multiplying by c does the same. This is why modular operations are well-defined on words rather than on one accidental choice of integer representative.

The phrase **well-defined** carries a precise burden. Suppose a word is written once by representative a and again by representative a' , with $a \equiv a' \pmod{2^w}$. Suppose the same is true of b and b' . An operation on words is well-defined only if choosing the primed spellings cannot change the resulting residue. For addition,

$$(a + b) - (a' + b') = (a - a') + (b - b').$$

Both terms on the right are multiples of 2^w , so their sum is too. Hence $a + b \equiv a' + b' \pmod{2^w}$. For multiplication, expand

$$ab - a'b' = a(b - b') + b'(a - a').$$

Again, each term is a multiple of the modulus. Addition and multiplication therefore act on the residue classes themselves, not merely on the natural representatives we print.

This small proof prevents a large ambiguity. We often compute with the representative 15 for a width-four word. A signed argument may instead use -1. The two calculations may take different paths through the integers, but an operation defined on the modular carrier must return the same stored word. If it does not, the purported word operation has accidentally depended on a reading.

An **equivalence relation** is a relation that is reflexive, symmetric, and transitive. Congruence has all three properties. Every integer is congruent to itself; reversing a divisible difference preserves divisibility; and adding two divisible differences gives a divisible difference. The relation partitions all integers into 2^w nonoverlapping classes. A word is one of those classes. Its natural representative is a convenient name for the class, not its entire mathematical identity.

Why the quotient has exactly 2^w words

Division with remainder assigns every integer exactly one remainder in $0, \dots, 2^w - 1$. Integers with the same remainder are congruent, and integers with different remainders cannot differ by a multiple of 2^w . Each remainder therefore names one class, no two names collide, and every integer belongs to one of the 2^w classes.

1.3.2 Width and the number of patterns

The cardinality 2^w gives width an information meaning before any numeric reading is chosen. One bit can distinguish two alternatives. Two bits can distinguish four. Appending one independent bit doubles the number of possible patterns because each old pattern can be paired with either 0 or 1.

If a field must distinguish N alternatives, its width must satisfy $2^w \geq N$. The smallest sufficient width is

$$w = \lceil \log_2 N \rceil.$$

The ceiling means the least whole number not below the logarithm. Five operation codes need at least three bits: two bits distinguish only four cases, while three bits distinguish eight. The three unused patterns do not vanish. The instruction-set design must reserve them, declare them illegal, or assign them later.

This counting lower bound is independent of the chosen code. No clever assignment can put five distinct operations into four patterns without making two operations share a spelling. A variable-length code can change the question by allowing different numbers of bits, but it must then provide a rule that says where one code ends. Chapter 6 will apply that trade to fixed and variable instruction encodings.

Unused patterns have economic and semantic value. Reserving them can leave room for later extensions. Declaring them illegal can detect damaged or unsupported input. Assigning every pattern can make a decoder total, but it

removes that particular source of extension space. The count alone does not choose among these policies. It makes their capacity cost visible.

The same argument applies to data. An unsigned field for the values 0 through 999 needs ten bits because $2^9 = 512 < 1000 \leq 1024 = 2^{10}$. Sixteen such fields occupy 160 payload bits before alignment or metadata. Rounding every field to 16 bits simplifies many machine operations but raises the payload to 256 bits. Packing saves 96 bits per record while introducing extraction, insertion, and possibly cross-byte access work.

Capacity is not physical reliability

Counting patterns says how many alternatives a width can name. It does not say how a circuit stores them or how reliably a receiver distinguishes them. Voltage ranges, timing, noise, error detection, and physical energy belong to the realization of the bit abstraction. Chapter 2 introduces that boundary; later chapters ask which physical effects remain invisible to architectural semantics and which become observable faults or timing behavior.

Subtraction needs no separate underflow object in the carrier. At width four, $0 - 1 = -1 \equiv 15 \pmod{16}$, so the stored result is 1111. An instruction may also record a borrow or carry convention, but that condition is additional state about the mathematical subtraction. The word result itself comes from the same remainder rule.

The carrier and the condition answer different questions

Modular arithmetic always supplies a width- w result. Carry, borrow, and signed overflow report how an integer reading crossed a chosen boundary. They do not repair or replace the stored word.

Binary arithmetic before electronic computers

In 1703, Leibniz described arithmetic using only zero and one and connected the spare notation with rules for calculation. His machine was not a modern instruction set, and the essay does not supply our semantics. It records an older encounter with the same compression: a tiny alphabet can support a complete arithmetic once its rules are fixed. (Leibniz 1703)

1.4 Bitwise logic

The algebra and the machinery met in stages. Boole's nineteenth-century work treated logical classes through algebraic operations. It was not a manual for binary

computers. In 1938, Claude Shannon used a two-valued symbolic calculus to analyze relay contacts and switching networks. A closed or open path could be represented symbolically, and algebraic manipulation could test equivalence or seek a circuit with fewer contacts. (Boole 1854; Shannon 1938)

That bridge matters because it separates three objects that are easy to collapse: a logical value, a physical switching condition, and a stored bit. The algebra describes relations among two values. An engineering design says which physical conditions realize them. A word arranges several such positions and gives each a weight or role.

For $0 \leq i < w$, bit i of a word x is

$$(x \text{ div } 2^i) \bmod 2.$$

Bit zero is the least significant bit. Bit $w - 1$ is the most significant. Bitwise and, or, exclusive or, and not apply one Boolean operation at every position.

The three binary operations have four possible input pairs. The table gives all results:

a	b	$a \text{ and } b$	$a \text{ or } b$	$a \text{ xor } b$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Bitwise not needs one input: it exchanges 0 and 1. Because the table lists every possible input pair, it is both an executable definition and a tiny exhaustive proof resource. The xor column shows directly that $a \text{ xor } 0 = a$ and $a \text{ xor } a = 0$ for either bit.

For example,

$$1010 \text{ and } 1100 = 1000, \quad 1010 \text{ xor } 1100 = 0110.$$

Each output position can be checked without consulting another position.

Applying a Boolean law to every position lifts it to words. If $x \text{ xor } m$ toggles the positions selected by mask m , a second xor with m restores each position. Selected bits toggle twice. The positions not selected agree with xor zero twice. Thus

$$(x \text{ xor } m) \text{ xor } m = x.$$

The proof is local to one position, yet it covers words of every width. This is our first bitwise proof that scales without enumerating every word.

Bitwise operations become useful when one word carries several independent choices. A **mask** is a word used to select positions. If a mask has a one at position i , then and keeps bit i of the other operand; if the mask has a zero there, and clears it. With width eight,

$$10110110 \text{ and } 00001111 = 00000110.$$

The mask keeps the low four positions. Using or with 10000000 sets the high position without deciding the other seven. Using xor with the same mask toggles that position. These are three different state transformations even when a particular input happens to make two results agree.

Reading a field

Suppose bits 4 through 6 hold a three-bit field inside an eight-bit word x . First shift x right by four positions. Then apply the mask 00000111. The shift moves the desired field to positions 0 through 2; the mask clears everything above it. Each step has a separate purpose, which makes a wrong shift distance or a wrong mask easy to attack.

The extraction formula follows from positional division. Write

$$x = q2^{p+\ell} + f2^p + r,$$

where $0 \leq f < 2^\ell$ is the desired length- ℓ field at position p , and $0 \leq r < 2^p$ contains the lower positions. Whole-number division by 2^p discards r :

$$x \text{ div } 2^p = q2^\ell + f.$$

Taking the remainder modulo 2^ℓ then discards the higher part and leaves f . Hence

$$\text{extract}(x, p, \ell) = (x \text{ div } 2^p) \bmod 2^\ell.$$

The shift-and-mask method is the same calculation in word operations. Logical right shift performs the division, and an all-ones mask of length ℓ performs the remainder.

Insertion needs two protections. It must prevent old field bits from surviving under new zeroes, and it must prevent an oversized new value from reaching neighboring positions. Let

$$M_{p,\ell} = (2^\ell - 1)2^p$$

be the mask whose ones occupy exactly the destination field. Then

$$\begin{aligned} \text{insert}(x, y, p, \ell) &= (x \text{ and not } M_{p,\ell}) \\ &\quad \text{or}((y \text{ and } (2^\ell - 1))2^p). \end{aligned}$$

The first line clears the destination in x . The second keeps only the admitted low bits of y and moves them into place. Since the two pieces have no common one-position, or combines them without a carry.

Two laws state the intended frame precisely. Extracting the inserted field returns $y \bmod 2^\ell$. Extracting any position outside that field returns the corresponding bit of x . The first is a result law; the second is a frame law saying what the operation leaves unchanged. Chapter 3 will require the same two-part discipline for instructions.

Clearing and masking do different work

Clearing only the old field does not constrain an oversized new value. Masking only the new value does not remove old one-bits where the new field contains zeroes. A correct insertion needs both steps.

A logical left shift by k multiplies by 2^k and then reduces modulo 2^w . A logical right shift divides by 2^k and keeps the whole-number quotient. Thus 1100 shifted logically right by one becomes 0110. An arithmetic right shift has a different purpose: it repeats the high bit under the two's-complement reading, so the same input becomes 1110. The operation, not the stored pattern, chooses the behavior.

We should not read meaning directly from bits. A high bit of one does not announce that a value is negative. A signed operation treats it as a sign; an unsigned operation does not.

A shift count also needs a boundary rule. What happens when $k \geq w$? Different languages and instruction forms may mask the count, reject it, or define a zero result for a logical shift. Ao chooses a total rule. A logical shift by $k \geq w$ returns zero. An arithmetic right shift by $k \geq w$ returns all zeroes when the source high bit is zero and all ones when it is one. A rotation uses the effective count $k \bmod w$. These are Ao teaching rules, not claims about every language or processor.

The total rule matters for proofs and implementations. A function that is undefined at large counts cannot support a universal theorem over all natural counts. A function that silently masks a count proves a different theorem from one that returns zero. The eventual Axeyum operation must implement the rule stated here and reject a control that substitutes either neighboring policy.

1.4.1 Shifts and rotations

For $0 \leq k < w$, a logical left shift has the word-level definition

$$\text{shl}_w(x, k) = (x2^k) \bmod 2^w.$$

The low k result positions become zero. Source bits that move beyond position $w - 1$ are discarded. The operation is therefore multiplication by a power of two on the modular carrier, but it need not preserve the ordinary integer product. At width eight, shifting 11100001 left by three gives 00001000. The mathematical product is larger than the unsigned range; only its low eight bits remain.

A logical right shift uses whole-number division:

$$\text{lshr}_w(x, k) = x \div 2^k.$$

The high k result positions become zero, and the low k source bits are discarded. This operation is not an inverse of left shift on every word. Once a discarded bit is gone, the other shift cannot recover it. The identity

$$\text{lshr}_w(\text{shl}_w(x, k), k) = x$$

holds only when the top k bits of x were zero. The premise names the information that the left shift would otherwise lose.

Arithmetic right shift fills the new high positions with copies of the old high bit. Under the two's-complement reading, it computes floor division by 2^k . This differs from truncation toward zero for a negative odd value. At width four, 1101 has signed reading -3. Arithmetic right shift by one gives 1110, whose signed reading is -2. The result is $\lfloor -3/2 \rfloor$, not -1. An optimizer that replaces signed division with a right shift must account for the language's rounding rule.

A **rotation** moves bits around a fixed cycle instead of discarding them. Consider a left rotation by k . Split x into the bits that remain after the left shift and the high bits that cross the boundary:

$$\text{rol}_w(x, k) = \text{shl}_w(x, k) \text{ or } \text{lshr}_w(x, w - k),$$

for $0 < k < w$, with rotation by zero defined as the identity. At width eight, rotating 11100001 left by three gives 00001111: the three high ones return in the low positions. Unlike a shift, a rotation is a permutation of positions. Rotating left by k and then right by k returns every word.

Why a rotation loses no bit

The source positions 0 through $w - k - 1$ move upward by k . The source positions $w - k$ through $w - 1$ wrap to positions 0 through $k - 1$. The two destination ranges are **disjoint**, meaning that they share no position. Together they cover all w positions. Every source position has one destination, and every destination has one source. Reversing the movement gives the opposite rotation.

Masks and shifts also support field insertion. To replace a field of length ℓ beginning at position p , first clear those positions in the old word. Then keep only the low ℓ bits of the new field, shift them left by p , and combine the two pieces with or. The range premise $0 \leq p$, $0 \leq \ell$, and $p + \ell \leq w$ is part of the operation. Without it, the field may cross the word boundary or ask for a negative position.

Extract, change, and insert one field

Treat 10110110 as four two-bit fields. Extract the field in positions 4 and 5. Replace it with 01 while preserving the other six positions. Write the clear mask, shifted new field, and final word. Then change one mask bit and show which supposedly preserved position is damaged.

1.5 Unsigned and signed readings

The unsigned reading of a word is its representative between 0 and $2^w - 1$. The signed two's-complement reading agrees with that value when the high bit is zero. When the high bit is one, it subtracts 2^w .

At width four, 0111 has unsigned and signed value 7. 1111 has unsigned value 15 and signed value $15 - 16 = -1$. No bit has moved. We changed the map from the stored word to an integer.

The high-bit-zero half contains 0 through $2^{w-1} - 1$. The high-bit-one half contains the representatives 2^{w-1} through $2^w - 1$; subtracting 2^w maps them to -2^{w-1} through -1 . The signed range is therefore

$$-2^{w-1}, \dots, 2^{w-1} - 1.$$

At width four it is -8 through 7. Adding 0001 to 0111 produces 1000. Its unsigned reading is 8; its signed reading is -8. The stored addition is well-defined. Whether the mathematical interpretation crossed a signed or unsigned boundary is a separate fact that later condition-state rules may record. It is imprecise to say that 1000 simply is negative. It has signed reading -8 and unsigned reading 8.

The modular carrier also explains negation. An additive **inverse** is a value that adds to the original value to produce zero. The inverse of word x is the word $-x \bmod 2^w$. At the bit level, invert every position and add one:

$$-x \equiv (2^w - 1 - x) + 1 \equiv \text{not } x + 1 \pmod{2^w}.$$

The all-ones word represents $2^w - 1$, so subtracting x from it flips each bit. The final one completes the modular inverse. For width four, 0011 becomes 1100 after inversion and 1101 after adding one. The signed readings are 3 and -3.

The word **nonnegative** means zero or positive. This rule is not a trick pasted onto binary notation. It is the ordinary additive inverse in the residue system. The same width- w adder can add unsigned representatives or two's-complement readings because both use the same stored modular addition. The interpretation of carry and signed overflow changes; the low w sum bits do not.

Two words are their own additive inverses. Zero plainly satisfies $-0 = 0$. The word 2^{w-1} does too, since doubling it gives 2^w , which is zero modulo 2^w . Under the signed reading this second fixed point is the most negative value. At width four, negating 1000 returns 1000. There is no stored pattern for +8. A claim that signed negation always produces the mathematical opposite therefore needs a premise excluding the most negative input, or it needs a wider result.

The signed range is asymmetric

A width- w two's-complement word represents one more negative integer than positive integers. The range begins at -2^{w-1} and ends at $2^{w-1} - 1$. Negating the lower endpoint cannot produce its positive opposite at the same width.

Two's complement was not the only possible signed reading. A **sign-magnitude** convention reserves one bit for the sign and uses the remaining bits for the magnitude. A **ones'-complement** convention forms a negative pattern by inverting every bit of the positive pattern. Both admit separate positive-zero and negative-zero patterns. Historical computers used these and other numeric organizations. For example, the UNIVAC 1108 programmer's reference defines a 36-bit signed word by ones' complement and explicitly gives all-zero and all-one patterns for +0 and -0. (Sperry Rand Corporation 1970) The manual makes representation an architectural rule a program must know, not a hidden fact about a circuit. The eventual prevalence of two's complement should not be retold as if the bits demanded it.

The engineering attraction is visible in the mathematics we have already built. Modular addition serves signed and unsigned readings, zero has one pattern, and sign extension has a simple rule. Those benefits do not make every operation interpretation-free. Comparisons, division, right shifts, conversions, and overflow tests still need the signed or unsigned contract.

Two boundary facts deserve separate names. An unsigned addition produces a **carry out** when the ordinary nonnegative sum is at least 2^w . A signed addition has **signed overflow** when the sum of the two signed readings lies outside -2^{w-1} through $2^{w-1} - 1$. At width four, 1111 plus 0001 has a carry out: 15 plus 1 is 16. Under the signed reading, -1 plus 1 is 0, so there is no signed overflow. By contrast, 0111 plus 0001 has no carry out, but 7 plus 1 lies outside the signed range. The stored result 1000 is the same kind of word in both calculations; the two conditions answer different questions about it.

One result, two overflow questions

Do not use *overflow* without saying whether the claim concerns the unsigned carry boundary or the signed range. The conditions can disagree on the same stored addition.

The signed-overflow rule can be read directly from signs. Adding operands with different signed signs cannot overflow: their mathematical sum lies between them. Adding two nonnegative values overflows exactly when the stored result has a high bit of one. Adding two negative values overflows exactly when the

stored result has a high bit of zero. Chapter 3 will turn these observations into explicit Ao condition-state updates rather than leaving them as prose.

1.5.1 When modular and signed addition agree

Let $S_w(x)$ denote the two's-complement reading of word x . Consider any two words x and y . Modular addition first forms

$$z = (x + y) \bmod 2^w.$$

If the mathematical signed sum $S_w(x) + S_w(y)$ lies inside the signed range, then

$$S_w(z) = S_w(x) + S_w(y).$$

In this exact sense, the shared word adder computes signed addition. Outside the range, the word result still exists. Its signed reading is the mathematical sum shifted by 2^w into the interval that fits the width.

The proof can be organized by the two readings. Each unsigned representative equals its signed reading or its signed reading plus 2^w . There are therefore integers $e_x, e_y \in \{0, 1\}$ such that

$$x = S_w(x) + e_x 2^w, \quad y = S_w(y) + e_y 2^w.$$

Adding gives

$$x + y = S_w(x) + S_w(y) + (e_x + e_y)2^w.$$

The last term vanishes when we take the residue modulo 2^w . The stored sum is congruent to the mathematical signed sum. If that sum already lies in the signed range, it is the unique signed reading for the result. If it lies outside, congruence alone cannot make the out-of-range integer fit the width; the reading returns the congruent integer that does fit.

This proof explains both the power and the limit of the shared carrier. We do not need a second addition operation for signed low bits. We do need a separate condition to say whether reading those bits as the mathematical sum is valid.

The familiar sign rule now follows. Two operands with different signs have a sum between them, so their sum remains inside the signed range. Two nonnegative operands overflow only if the result crosses the upper endpoint and reappears with a high bit of one. Two negative operands overflow only if the result crosses the lower endpoint and reappears with a high bit of zero.

A condition is a theorem about readings

The word adder always returns $(x + y) \bmod 2^w$. The signed-overflow condition is true exactly when the signed reading of that word differs from the

ordinary sum of the operand readings. The condition does not alter the result. It reports whether one interpretation law survived the modular operation.

Subtraction uses the same structure because $x - y = x + (-y)$ in the modular carrier. Its signed-overflow test must still follow the subtraction readings, not a memorized addition slogan. Subtracting a negative value from a nonnegative value can cross the upper endpoint; subtracting a nonnegative value from a negative value can cross the lower endpoint. Chapter 8 will derive the complete condition formulas beside the machine flags that record them.

Two readings, one pattern

List all sixteen width-four patterns. Write the unsigned and signed reading of each. Identify the point at which the signed order wraps from 7 to -8.

1.6 Changing widths

Moving a value between widths requires another rule. Truncating from width w to a smaller width v keeps the remainder modulo 2^v . Truncating 00010001 to four bits gives 0001; the discarded high bit cannot be recovered from the result. Zero extension to a larger width inserts zeroes above the old high bit. Sign extension repeats the old high bit so that the two's-complement integer reading remains the same.

For 1111, zero extension from four bits to eight gives 00001111, whose signed reading is 15. Sign extension gives 11111111, whose signed reading remains -1. An instruction that merely says it produces eight bits has not yet told us which value-preservation rule it uses.

The preservation arguments are short. Zero extension adds only zero coefficients above the old high bit, so the unsigned sum of bit weights does not change. For sign extension, a nonnegative source also gains only zeroes. A negative source of width w , with unsigned representative u , becomes the wider unsigned representative $u + 2^v - 2^w$ at width v . Subtracting 2^v gives $u - 2^w$, the original signed reading.

This distinction will return in both companion architectures. Their registers may hold 64-bit words while selected operations produce narrower results. The semantics must say whether upper bits are preserved, cleared, or supplied by sign extension.

The words *preserves the value* are incomplete until the reading is named. Zero extension preserves the unsigned reading. Sign extension preserves the signed reading. Neither operation promises to preserve both. For a source whose high bit is zero, they happen to produce the same wider pattern; that agreement on half the inputs is not a reason to treat the operations as interchangeable.

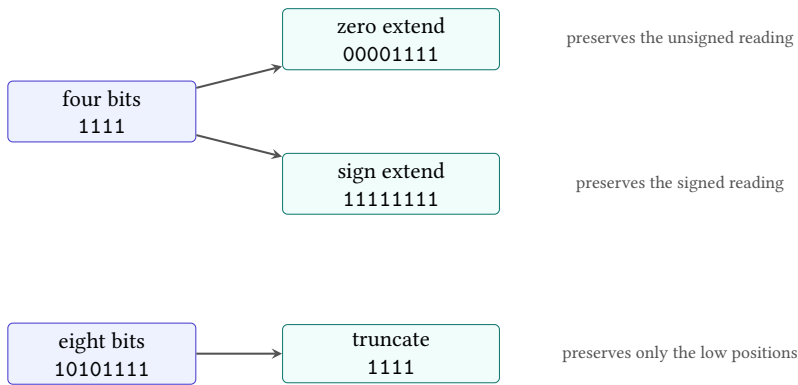


Figure 1.3: Zero extension, sign extension, and truncation are three different maps. Their names identify both the bit movement and the reading or positions they preserve.

Why sign extension preserves a negative value

Let a negative width- w word have unsigned representative u . Its signed reading is $u - 2^w$. Extending it to width $v > w$ fills the new high positions with ones, adding $2^v - 2^w$ to the unsigned representative. The wider signed reading subtracts 2^v :

$$(u + 2^v - 2^w) - 2^v = u - 2^w.$$

The two large terms cancel, leaving the original signed integer.

Truncation has no such general preservation law. It is a many-to-one map: every pair of source representatives that differs by a multiple of 2^v has the same width- v result. A later proof may still use truncation under a premise that the discarded bits are known to be zero, or known to repeat the sign. The premise, not the word *truncate*, earns the preservation claim.

Some compositions do have exact laws. Truncating a zero-extended word back to its source width returns the original pattern. The same holds after sign extension because both extensions preserve every original low position. Extending after truncation does not generally return the original wider word: the discarded positions are unavailable, so the extension rule must invent them from zero or from the retained sign bit.

These one-way identities are useful controls. A checker should accept truncate-after-extend for every source word and reject the reverse identity on a witness with nonzero discarded positions. The pair teaches more than two successful examples because it marks exactly where information is lost.

1.7 Words and bytes

A byte is a word of width eight. Memory will store bytes at addresses, so a wider word needs a declared relation to a sequence of bytes.

Little-endian split and join

Let the word width be a positive multiple of eight. Byte i is

$$b_i = (x \operatorname{div} 2^{8i}) \bmod 2^8.$$

A **little-endian** byte sequence puts the lowest-weight byte first in increasing address order: $b_0, b_1, \dots$. Joining the sequence computes

$$\sum_i b_i 2^{8i}$$

at the original width.

Object `A0.def.byte` records this relation. It does not yet define memory. It says how one word can be taken apart and put back together.

The reader proof of the round trip is positional. Each b_i selects exactly bits $8i$ through $8i + 7$. Multiplying by 2^{8i} returns those bits to their original positions. Different bytes occupy **disjoint** positions, meaning that no position belongs to two bytes. Their sum reconstructs the original word.

For two bytes, write $x = b_0 + 2^8 b_1$, where both byte values lie between 0 and $2^8 - 1$. Splitting extracts $b_0 = x \bmod 2^8$. Dividing x by 2^8 discards that low remainder and exposes b_1 . Joining returns $b_0 + 2^8 b_1 = x$. For more bytes, the same quotient-and-remainder step repeats. Nothing in this argument depends on the hexadecimal spelling of x .

The general proof can be written as repeated division. Let the word contain n bytes, so its width is $8n$. Division by 2^8 gives a quotient and a remainder. Both are unique.

$$x = 2^8 q_1 + b_0, \quad 0 \leq b_0 < 2^8.$$

Apply the same step to q_1 , then to the next quotient:

$$q_1 = 2^8 q_2 + b_1, \quad q_2 = 2^8 q_3 + b_2, \quad \dots \quad q_{n-1} = 2^8 q_n + b_{n-1}.$$

Because $x < 2^{8n}$, the final quotient q_n is zero. Substituting each equation into the previous one gives

$$x = b_0 + b_1 2^8 + b_2 2^{16} + \dots + b_{n-1} 2^{8(n-1)}.$$

The right side is exactly the join formula. Splitting and then joining returns the original word.

The reverse direction matters too. Begin with any n -byte sequence and join it. The resulting sum is less than 2^{8n} , because its largest value occurs when every byte is 255. Splitting at position i divides away the lower i byte weights and takes the remainder modulo 256. All higher terms are multiples of 256, so only b_i remains. Joining and then splitting returns the original sequence.

Split and join form a bijection

Split-after-join is the identity on byte sequences, and join-after-split is the identity on words whose width is a positive multiple of eight. The two functions undo each other. They establish a one-to-one correspondence between width- $8n$ words and sequences of n bytes. Endianness chooses which sequence position meets the lowest address; it does not change the number of possible words.

The counting agrees with the algebra. There are 256 choices for each of n bytes, hence $256^n = (2^8)^n = 2^{8n}$ sequences. That is exactly the number of width- $8n$ words. Equal counts alone would not prove that our particular split and join are inverse. The quotient-and-remainder argument does.

A byte round trip with unequal digits

Split 0x00a17c03 into four little-endian bytes. Join them in the proper order, then join the reversed list. Unequal bytes make an order error visible; a test word such as 0x00000000 would let the wrong rule pass.

For 0x12345678, the increasing-address byte sequence is 78, 56, 34, 12. If we instead attach increasing addresses to the list 12, 34, 56, 78, the same little-endian join produces 0x78563412. The numeric word did not reverse by itself. We changed the relation between addresses and place values.

Width four was useful for the reader proofs because all sixteen cases fit on a page. A complete Ao state has width equal to a positive multiple of eight. Here lies the first firm boundary between a proof miniature and the executable teaching machine.

1.8 Several arithmetic contracts

The modular carrier guarantees a stored result for addition, subtraction, and multiplication. A programming interface may expose that result in several ways. These choices are contracts above the carrier, not rival accounts of what remainder means.

Modular arithmetic returns the low w bits. Crossing the unsigned range wraps around. Counters, checksums, hashing, cryptographic operations, and address calculations often use this behavior deliberately.

Checked arithmetic returns the mathematical result when it fits and otherwise reports failure, raises an exception, traps, or produces a separate status. The failure is part of the outcome.

Saturating arithmetic returns the nearest endpoint when the mathematical result lies outside the chosen range. Adding 20 to an unsigned eight-bit value 250 returns 255 rather than 14. Image, audio, control, and vector operations may prefer a clipped boundary to wraparound.

The three contracts agree on inputs whose mathematical result lies in range. Agreement there does not make them equivalent. Width-eight addition of 250 and 20 separates all three: modular addition returns 14, checked addition reports the out-of-range event, and unsigned saturation returns 255. A test suite containing only small operands cannot tell which contract was implemented.

These choices appear side by side in current software interfaces. Rust’s fixed-width integer types provide named method families such as `wrapping_add`, `checked_add`, `overflowing_add`, `strict_add`, and `saturating_add`. (Rust Project Developers 2026) The names make the boundary policy part of the operation a programmer selects. A proof that replaces one method with another must cover the out-of-range inputs where their results or outcomes differ.

Signed arithmetic adds another distinction. The modular word result still exists, but a language may refuse to assign a normal signed result when the mathematical sum lies outside the signed range. A compiler cannot infer a language-level rewrite merely from the fact that the target adder produces low bits. It must preserve the source language’s rule for the exceptional case as well as the machine’s visible flags, traps, or lack of them.

The carrier does not choose the error policy

Modulo 2^w defines the stored low bits. A checked, trapping, flag-setting, or saturating interface adds an observation about crossing a declared range. State both layers before proving that an arithmetic replacement is correct.

This distinction also explains why “overflow” cannot name one universal event. Unsigned carry asks whether a nonnegative result crossed 2^w . Signed overflow asks whether a signed result left a different interval. Saturation asks which endpoint should replace an out-of-range result. A language-level exception asks whether ordinary execution continues at all. The same adder inputs can lead these observers to different conclusions.

1.9 Width as an engineering budget

A wider word admits more patterns, but those patterns are not free. A billion eight-bit values contain one billion payload bytes. A billion 64-bit values contain eight billion payload bytes. Ignoring metadata, alignment, and compression, the width choice changes the payload from about 1 GB to about 8 GB. Reading the wider representation also moves eight times as many payload bits through a memory interface.

That arithmetic does not prove that the narrow representation is faster or cheaper in every system. A processor may widen narrow values before calculation. Alignment may add unused space. Conversion instructions may cost time. Compression may remove repeated high bits. The point is narrower: representation width places a first-order bound on storage and transfer, then the surrounding machine changes the realized cost.

Width also determines parallel capacity. A 256-bit vector register can hold thirty-two eight-bit lanes, sixteen 16-bit lanes, eight 32-bit lanes, or four 64-bit lanes. More lanes may increase the number of independent operations per instruction, while narrower lanes also reduce each result's range. Whether the trade is acceptable depends on the data, the arithmetic contract, and the cost of conversion or saturation.

Addresses create a different pressure. Their width limits the directly named address space, but widening every address also enlarges pointers stored in tables and object structures. An architecture and its software environment therefore balance address reach, register rules, instruction encoding, memory footprint, and compatibility with existing programs. “Use the widest integer” is not an architecture principle. It is one choice among competing costs.

The present industrial question is often not which single width a machine has. Modern architectures support several operand widths and impose rules for how they meet a wider register. In RV64I, instructions with a *W* suffix compute a 32-bit result and sign-extend it into the 64-bit destination. (RISC-V International 2026e) In 64-bit mode, selected x86-64 writes to a 32-bit general-purpose destination clear the upper half of the corresponding 64-bit register. (Intel Corporation 2026a) Both rules give later instructions a known upper half, but they establish different known patterns.

The shift count is another width contract. In the teaching slice, `SRAI rd, rs1, shamt` uses a six-bit immediate amount in RV64I. The x86-64 form `SAR r64, CL` masks the supplied count to six low bits, and its flag effects depend on the effective count. (RISC-V International 2026e; Intel Corporation 2026b) A compiler, decoder, or proof model must preserve those exact rules. Replacing them with the vague sentence “shift by k ” loses behavior at the boundary where verification is most useful.

The narrow-result pair is also concrete. `ADDIW rd, rs1, imm` computes a 32-bit result and sign-extends it into the RV64 destination. The x86-64 form

MOV r32, r/m32 writes a 32-bit general-purpose destination and clears the upper 32 bits of its 64-bit register. The same low pattern with bit 31 set therefore produces different upper halves. (RISC-V International 2026e; Intel Corporation 2026a; Intel Corporation 2026b)

Table 1.1: Three present width rules that answer different questions.

Rule	What it fixes	What it does not settle
RV64I *w result	low 32-bit computation and sign-extended destination	source-language overflow policy
x86-64 32-bit register write	upper half of the 64-bit register becomes zero	signed reading of the low half
Shift-count rule	effective count admitted by one instruction form	whether the right shift is logical or arithmetic

These are economic rules as well as mathematical ones. Known upper bits can remove a separate extension instruction. Narrow lanes can increase packed throughput and reduce traffic. Stable legacy behavior lets old binaries keep running, while it also constrains new implementations and compiler back ends. The proof obligation begins only after the chosen benefit and the exact rule have names.

1.10 RV64 and x86-64 questions

Both later companion slices contain fixed-width integer values and access byte-addressed memory. That common description does not settle how a given instruction reads a word, changes its width, or treats its upper bits. Those rules belong to the selected instruction form.

The question for RV64 and x86-64 is therefore not whether either machine uses binary. It is which width each form reads, which width it writes, which integer reading its conditions use, and how its bytes meet memory. We will pin the exact forms and source revisions before answering those questions as architectural facts.

The paired questions can now be stated without slogans:

Table 1.2: Word-level questions carried from Ao to each real instruction form.

Question	RV64 slice	x86-64 slice
Source width	named operand form	named operand form
Destination width	register or memory rule	register, subregister, or memory rule
Integer reading	operation and condition	operation and condition
Upper bits	explicit width rule	selected subregister rule
Byte order	declared memory premise	declared memory premise

This table is a checklist, not a set of answers. Saying that one architecture has 64-bit registers does not settle the result of a narrower write. We need the exact instruction form and the governing manual revision. Part II will supply those source-pinned answers after Ao has given us the questions in a stable order.

1.11 What the word model fixes

A word begins as one member of a finite carrier. Its width fixes the number of patterns and the modulus for stored arithmetic. Positional expansion relates each pattern to one natural representative. A signed reading maps the same carrier to a different interval without moving a bit.

Operations then state what they preserve. Boolean operations act independently at each position. Shifts move bits across a boundary and may discard them. Rotations permute all positions. Extension chooses a reading to preserve. Truncation keeps low positions and generally loses information. Split and join relate one byte-multiple word to an ordered byte sequence.

None of these definitions supplies a physical register or memory cell. None chooses a language's failure policy. None says that two machine instructions with similar printed operands have the same effects. Those are later layers. The word model instead gives them a common exact vocabulary.

The distinctions also organize proof. A residue law may use congruence. A bitwise law may prove one position and lift it to every position. A width-change law names the reading it preserves. A split/join law uses quotient and remainder. A claim about cost states whether it counts bits, bytes, lanes, instructions, traffic, or time. Choosing the proof shape after choosing the claim is part of the method, not decoration around it.

One pattern can support many meanings without ambiguity

Ambiguity enters only when a claim omits its reading, width, operation, or observation. Once those are declared, the same underlying pattern can safely serve as an unsigned value, signed value, mask, field collection, byte, or encoding fragment. The multiplicity is a source of expressive power because the boundaries are explicit.

1.12 Implementation boundary

The definitions in this chapter are book-level foundations. Axeyum already has fixed-width bit-vector terms and solver routes, but the redesigned book does not treat the former width-eight formulas as its word foundation. Object `OP.a0.word-`package records the actual missing layer: one reusable Ao package for words,

readings, width changes, and byte split and join, with interfaces suitable for execution and proof.

The controls should attack the rules we have just used. A wrong modulus should fail at wraparound. A wrong sign extension should fail on a source whose high bit is one. Reversed byte weights should fail on a word with unequal bytes. A damaged round trip should return a concrete witness rather than a success message.

An introductory Axeyum workflow will eventually make this obligation concrete. The reader will construct a fixed-width term, state a relation such as split followed by join, ask for a counterexample within the declared width, and replay any returned model through executable Ao operations. If the solver reports that no counterexample exists, the route must still identify the generated formula and the independent checking boundary. Axeyum's current bit-vector machinery is useful substrate for that route; it is not yet the Ao word package named by this chapter.

The first Python-facing lesson should use a small, typed surface. Construct a word by giving its width and natural representative. Ask for its unsigned and signed readings. Apply one width change or byte split. The bound Rust type should reject a value outside the constructor contract rather than silently choosing a width. Python may format binary and hexadecimal views, but Rust must own the carrier and operations that later checks reuse.

The proof-facing lesson begins from the same objects. It quantifies over a declared width, applies the executable operation or its matched formula encoding, and requests either a separating input or evidence that the finite claim has no separating input. Replaying a returned word must reproduce both its stored pattern and the observations that made it a counterexample.

This progression keeps three activities distinct: calculating one example, searching the whole finite domain, and checking the evidence returned by that search. The commands may be close together in a notebook. Their claims are not interchangeable.

What Axeyum will add

The mathematics on these pages defines the intended word operations. The planned Axeyum package will add executable operations, formula generation, counterexample replay, checked evidence for selected finite claims, and negative controls. It will not turn an unstated width convention into a theorem.

Until that package exists, this chapter has reader proofs and exact definitions but no generated machine-proof box. A solver result about one handwritten formula would not establish the width-parametric library on which later Ao state and instruction semantics must depend.

One word now has a carrier, bit operations, several readings, width changes, and a relation to bytes. It still has no location. We do not know which register holds it, which memory address names one of its bytes, which instruction comes next, or whether execution has stopped. Chapter 2 constructs the state in which those questions have answers.

1.13 Exercises

The exercises move from calculation to proof and transfer. A complete answer states the width and reading before using a result. For every false claim, prefer a smallest or boundary witness that explains the failure.

1. **Execute.** Compute every width-four sum $x + 1$. Explain the wraparound once by enumeration and once by the remainder definition.
2. **Explain.** Find two width-four additions with the same stored result but different signed or unsigned interpretations.
3. **Execute.** Convert $0x9d$ to eight binary digits. Use masks and shifts to extract its low three bits and its high three bits.
4. **Execute.** Zero-extend and sign-extend each width-four pattern whose high bit is one. State which integer reading each operation preserves.
5. **Execute.** For every pair among 0111 , 1111 , and 0001 , compute the width-four sum, unsigned carry out, and signed overflow. Explain each disagreement between the two conditions.
6. **Transfer.** Split $0x89abcdef$ into little-endian bytes and join them again. Then reverse the byte list and calculate the different word it represents.
7. **Prove.** Prove that splitting and joining a two-byte word returns the original word. Identify the exact step that also works for any positive byte count.
8. **Break.** Design a false join rule by changing one exponent. Give a word whose round trip exposes the error. This will become the negative control for the future Ao word package.
9. **Prove.** Show that bitwise exclusive or with a fixed mask is its own inverse. State why the argument works independently at every bit position.
10. **Break.** Give a pair of different width-eight words that truncate to the same width-four word. Then state a premise under which truncation would preserve the unsigned reading.

11. **Prove.** Use the highest-differing-position argument to prove that two different width- w patterns cannot have the same unsigned reading. Explain why counting 2^w patterns and 2^w values would be insufficient without an existence or injectivity argument.
12. **Execute.** Rotate 10010110 left by 1, 3, and 8 positions. For each result, rotate right by the same effective count. State the boundary rule you used for the count 8.
13. **Break.** Find the smallest width-four word that disproves

$$\text{lshr}_4(\text{shl}_4(x, 1), 1) = x.$$

Then state and prove a premise that repairs the identity.

14. **Prove.** Derive the field-extraction formula from $x = q2^{p+\ell} + f2^p + r$. Identify the bounds on f and r that make the two division steps valid.
15. **Break.** Implement field insertion on paper while omitting the old-field clear step. Give an input on which a requested zero bit remains one. Repeat while omitting the mask on the new field, and show damage to a neighbor.
16. **Explain.** Add 250 and 20 at width eight under modular, checked, and unsigned-saturating contracts. Name the observation that distinguishes each pair of contracts.
17. **Prove.** Show that the width-four word 1000 is its own additive inverse. Generalize the result to width w , then explain why it is the only nonzero word with that property.
18. **Design.** A format needs 19 operation codes and 5 register names. Find the smallest fixed width for each field. Count the unused patterns. Give one reason to reserve an unused pattern and one reason to reject it.
19. **Economics.** A table holds 50 million values known to lie between 0 and 1000. Compare the raw payload sizes for 10-bit packed fields, 16-bit fields, and 64-bit fields. Then list two machine effects that could keep raw bit savings from becoming equal runtime savings.
20. **Transfer.** From the pinned RV64I manual, state what ADDIW does to the upper 32 destination bits. From the pinned Intel manual, select one 32-bit general-purpose register write and state its upper-bit rule in 64-bit mode. Give one input for which the resulting 64-bit patterns differ.
21. **Audit.** Write a claim that split followed by join is the identity for four bytes. List the scope, formula, expected negative control, and replay observation that an eventual Axeyum artifact must bind. Explain why a solver result for a handwritten formula alone would not establish the Ao package.

State

Pause a machine between two instructions. The arithmetic has stopped, but its result is only one part of what remains. Registers contain words. Memory contains bytes. A program counter identifies the next instruction. Condition bits remember facts that a later branch may use. The machine is either able to continue, halted, or trapped for a reason.

Leave one of these parts out and two programs may appear to agree only because we forgot where to look. We will therefore build the complete state first. For a particular question, we may then state which parts of that state matter.

A stored program needs a place to resume

The 1945 EDVAC report divided a proposed computing system into organs for arithmetic, control, memory, input, and output. Its terminology is historical, but the design problem remains current: after one operation, the machine must retain enough information to determine the next. Modern architectural state is not copied from that report; it is one descendant of the need the report made explicit.(Neumann 1945)

The important word is *enough*. We could describe every charge, latch, queue entry, and temperature in a physical processor. Such a description would be too large for our purpose and still incomplete. We could describe only the destination register. That would be small, but too weak to predict a branch or a memory access. A state model earns its size by the questions it can answer.

State is therefore a boundary around relevant history. Once the state is fixed, the next architectural step should not need a secret account of how the machine arrived there. Two executions that reach the same state face the same successors under the same program. Their earlier paths may differ, but the transition rule receives the same input.

State sufficiency

A state description is sufficient for a transition model when the modeled next-step possibilities depend only on that state and the declared external inputs, not on an unrecorded execution history.

State sufficiency is a modeling requirement, not a physical claim that history vanishes entirely. A cache or branch predictor can retain effects of earlier execution. Ao omits them, so Ao cannot answer a question whose next result depends on those effects. A timing model may restore them as state. The right snapshot changes with the promised observation.

2.1 State replaces relevant history

Suppose a machine has accepted a finite history h of instructions and external inputs. Let $\sigma(h)$ be the state description retained after that history. The description is sufficient only if equal descriptions make the histories interchangeable for the next modeled step. In symbols, if

$$\sigma(h_1) = \sigma(h_2),$$

then every declared next input must permit the same modeled successors and outputs after h_1 and h_2 . The state need not reconstruct either history. It must retain every fact from those histories that can still affect the promised future.

This condition is sometimes called a Markov condition: once the present state is known, the transition rule does not consult an additional past. We need the condition, not the borrowed name. Ao will be deterministic once its program and external setting are fixed, so one state and one legal instruction select one next state. A later weak-memory model may use a relation with several possible successors. State sufficiency applies to both forms.

Here is the smallest useful failure. Imagine a device with one visible bit (b) and one hidden mode (m). Its next operation toggles (b) when ($m=1$) and preserves (b) when ($m=0$). If our proposed state records only (b), the two physical situations $((b,m)=(0,0))$ and $((0,1))$ collapse to the same description. Give each the command step. One remains at 0 and the other becomes 1. The proposed state was not sufficient because the transition still depended on a fact we called history and then failed to record.

We can repair the model in either of two honest ways. Add (m) to the state, or narrow the promised transition model so that mode never affects a result. Calling the mode internal does not repair anything. An omitted component is safe only when the questions and rules cannot expose it.

This gives state a precise economy. Too little state makes future behavior depend on an unnamed past. More state than the question needs makes every

definition and proof carry distinctions that never matter. The useful state is not the smallest tuple in the abstract. It is a sufficient description for the declared transitions and observations.

2.1.1 A stored bit is a physical distinction

Chapter 1 treated a bit as one of two mathematical symbols. A processor must hold that distinction while time passes. The minimum physical idea is feedback: part of a circuit's present output helps determine its later input. With suitable gain and timing, the circuit can have two stable regions. A small disturbance within one region settles back there; a strong enough write signal moves the circuit into the other region.

The voltage is not itself the bit. A decoding rule maps one permitted region to 0 and the other to 1. Voltages between the regions, transitions that have not settled, noise margins, setup times, and failures of stable sampling belong to the physical contract. The architectural model deliberately replaces that contract with a word that is already either 0 or 1.

One abstract storage cell makes the replacement visible. Let $q_t \in \{0, 1\}$ be the retained value before a clock event, let d_t be a proposed new value, and let e_t be a write-enable bit. Define

$$q_{t+1} = \begin{cases} d_t, & e_t = 1, \\ q_t, & e_t = 0. \end{cases}$$

When writing is disabled, the old output returns as the new value. The cell therefore remembers. When writing is enabled, the proposed input replaces the old value. A register applies this rule to several cells under a common control. A register file adds a rule that selects which register receives the write.

This equation is a functional model, not a transistor diagram. It assumes the write and sampling conditions succeed. That boundary is valuable: instruction semantics can prove which word a register should contain without proving the analog behavior of every storage cell. A circuit-correctness or fault model would have to reconnect those levels.

The stored-program machine joins two kinds of structure. Combinational logic maps present inputs to outputs. Storage feeds selected outputs into a later moment. The first gives a function; the second lets repeated applications of functions form an execution. State is where time enters the model without requiring the complete past to enter with it.

2.2 Ao machine state

The abstract machine used in this book is Ao. Its word width w is a positive multiple of eight. The width-four patterns in Chapter 1 remain useful on a blackboard, but

they are not complete Ao machine states. Full Ao begins at one byte because its memory is byte-addressed.

A0 machine state

An Ao **machine state** is

$$S = (R, M, pc, Z, N, C, V, O).$$

The map R assigns a w -bit word to each of the eight registers $r_0, \dots, r_7$. The finite map M assigns bytes to valid data addresses. The w -bit word pc is the program counter. The bits Z, N, C, V record zero, negative, carry, and overflow conditions. The outcome O is running, halted, or trapped with a reason.

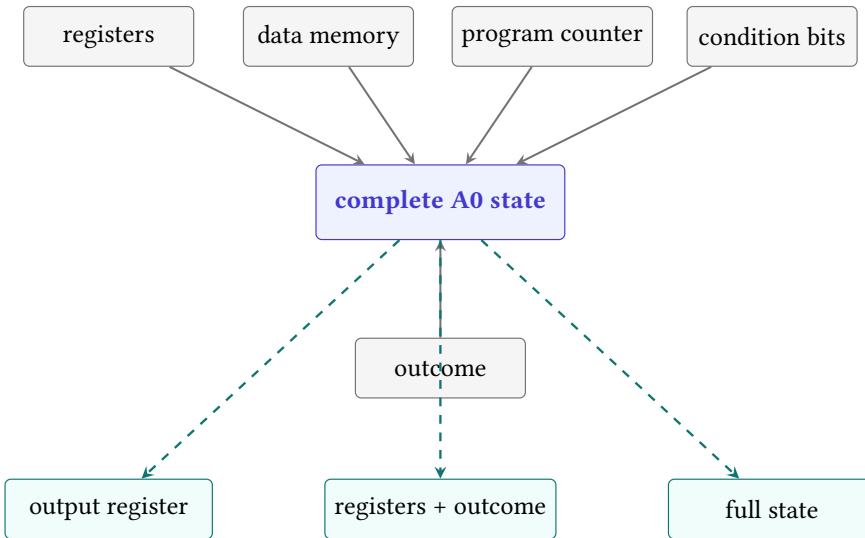


Figure 2.1: Ao gathers every modeled component into one complete state. An observation projects that state to the information a claim exposes.

The tuple is not a bag of unrelated fields. Its components constrain one another through instruction rules. A running state permits a step; a halted or trapped state does not. The program counter supplies the next fetch address. Register contents may supply operands or addresses. Condition bits may supply a branch predicate. Memory supplies bytes only at addresses in its finite domain. Chapter 3 will define the transformations that preserve these connections.

Object `A0.def.state` records this contract. Every register is ordinary: Ao has no hardwired zero register and no implicit stack pointer. The condition bits

are named state, not comments about the previous line. If an instruction writes one of them, a later instruction can observe that write.

Program code is not part of M . An Ao program supplies a separate, finite map of immutable instruction bytes and an entry address. This choice rules out self-modifying code in the first model. It also prevents a data store from silently changing the next instruction. Chapter 5 will add the program to the execution setting; for now we are describing the snapshot on which one step acts.

Separating code from data is a modeling decision, not a claim about all real machines. It gives the first Ao model a stable program while we learn to reason about data memory. A theorem under this separation says nothing about self-modifying programs unless a later model adds code memory that instructions can write and repairs every affected definition.

The outcome needs three cases. A running state may take a step. A halted state has finished by executing `halt`. A trapped state records that execution could not continue under an Ao rule, such as an invalid memory range. Halt and trap both have no successor, but they do not mean the same thing. A program that finishes and a program that fails are not interchangeable merely because both stop.

Three stopped descriptions

Suppose the last destination contains 42 in three states. In the first, the outcome is halted. In the second, an invalid load has trapped. In the third, the outcome remains running but no further step has been requested. Looking at the destination alone collapses three different execution situations into one number. A claim about successful completion must include the outcome.

The reason attached to a trap also matters when a claim distinguishes failure modes. Ao will name reasons such as illegal encoding, incomplete fetch, and invalid data range. We may later choose an observation that retains only the fact of trapping, but the complete state does not discard the reason first.

2.2.1 Well-formed states and equality

Not every tuple of mathematical objects is an Ao state. The width must be a positive multiple of eight. Each register value and the program counter must fit that width. Every memory value must be a byte, and every memory key must be a width- w address. Condition fields are bits. The outcome must be one of the declared cases, with a recognized reason when it is trapped.

The tuple notation is a Cartesian product with a well-formed subset. If $\mathcal{W}_w$ is the set of width- w words, $\mathcal{M}_w$ is the set of admitted finite byte maps, and $\mathcal{T}$ is the set of trap reasons, then the unconstrained shape is

$$(\mathcal{W}_w)^8 \times \mathcal{M}_w \times \mathcal{W}_w \times \{0, 1\}^4 \times (\{\text{running, halted}\} \cup \mathcal{T}).$$

The machine definition then removes malformed members. For example, a trapped outcome must carry one recognized reason, while a running outcome carries no trap reason. Thinking in products helps us inventory the fields. Thinking in the well-formedness invariant—the condition every legal state must maintain—stops us from mistaking every product-shaped tuple for a legal state.

A constructor should enforce these requirements once. Later instruction rules can then assume they receive a well-formed state and must return another one. Without that invariant, every transition theorem carries distracting side cases about a nine-bit value in an eight-bit register or a memory entry that is not a byte.

Two Ao states are equal exactly when their corresponding components are equal: the same register map, memory map, program counter, four conditions, and outcome. This one-component-at-a-time rule gives both proof directions. To establish state equality, establish every component. To refute it, one unequal component is enough.

Even before memory and trap reasons enter, the state space is large. Count only eight registers, the program counter, four condition bits, and the three coarse outcome classes. At width w , these fields admit

$$3 \cdot 2^{9w+4}$$

assignments. For $w = 64$, that is $3 \cdot 2^{580}$, more than 10^{175} . This number does not measure the states reachable by one program, and it omits memory. It measures the cost of replacing a universal argument with an undirected list of examples. No test campaign can visit the raw product one member at a time.

The count also explains why a clean state definition helps a solver. An instruction usually reads and writes only a few components. A proof can keep the untouched components symbolic and use the frame condition instead of enumerating their values. Structure, not sampling, makes the state space manageable.

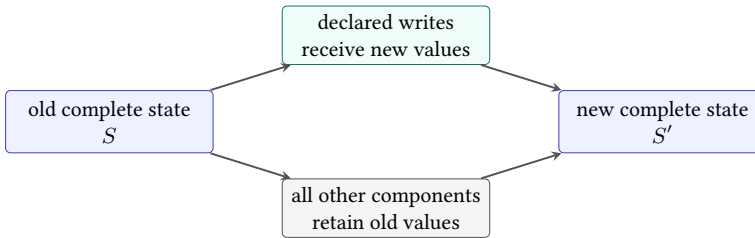


Figure 2.2: A complete transition combines declared new values with a frame condition that preserves every component outside the write set.

The preservation branch in Figure 2.2 is a **frame condition**. It says what the instruction leaves alone. A state diff is a convenient way to display the other branch, but a diff is complete only when it is interpreted against an old state and a rule that preserves everything omitted from it.

We can write the rule without listing every field in every instruction. Let C be the set of state components and let $W \subseteq C$ be an instruction's declared write set. For each $c \in W$, the instruction gives a new-value expression $u_c(S)$. The complete update is

$$S'[c] = \begin{cases} u_c(S), & c \in W, \\ S[c], & c \notin W. \end{cases}$$

The second line is load-bearing. Without it, an instruction description that mentions only r_0 says nothing about memory, conditions, the program counter, or the outcome. Readers often supply preservation from habit. A checker must receive it as part of the semantics.

The frame law has a short reader proof. Choose any component $c \notin W$. The definition selects its second branch, so $S'[c] = S[c]$. The choice of c was arbitrary; therefore every component outside W is preserved. The proof is almost small enough to miss. Its universal conclusion is exactly what a destination-only test fails to establish.

Now attack it. Mutate an addition rule so that it writes the correct sum to r_0 and also clears one memory byte. Every test that inspects only the sum still passes. The write-set check fails because memory changed outside the declared set. A negative control at this layer should make precisely that mutation and require rejection.

A destination-only test is therefore weak. It checks one changed component without checking conditions, control, memory, outcome, or the frame. A complete-state checker can still print a compact diff for the reader. It must derive that diff from states whose equality and preservation it actually checks. Compact presentation and complete checking can therefore coexist without weakening the state contract.

2.3 Architectural and physical state

The tuple above is an **architectural state**: it contains the parts that Ao instructions are defined to read or change. A physical processor needs much more. It may contain pipeline registers, cache tags, branch predictor entries, queues, and circuitry that has no name in an assembly program.

Amdahl, Blaauw, and Brooks made the architecture boundary explicit while describing IBM System/360 in 1964. They separated the attributes visible to a programmer from the organization and physical implementation of a particular machine. That separation allowed members of one family to differ internally while preserving a common software contract. It did not make implementation irrelevant. It located which differences compatible software was entitled to ignore. (Amdahl et al. 1964)

Whether those parts belong in a model depends on the question. If the question is whether two additions leave the same Ao state, cache replacement policy is

irrelevant because Ao has no cache state. If the question is how long the two additions take on a particular processor, the Ao tuple is insufficient. A smaller model is not false merely because it omits timing; it is false when it is used to answer a timing question.

One practical test sharpens the word *architectural*. Ask whether the declared instruction semantics can read the component, write it, or expose it through a modeled outcome. If yes, the component belongs in the state for that semantic question. If it affects only how a particular implementation reaches the result, it may remain outside a functional model. The answer can change when the question changes: cache contents are outside Ao functional equivalence and inside many timing or leakage models.

An omitted component is a theorem premise

Every omitted kind of state limits the claims that follow. Leaving caches out does not assert that caches do not exist. It asserts that the present observation and transition rules do not speak about their effects.

The distinction cuts both ways. Different internal executions may have the same architectural result. The processor is free to use either route if its contract permits both. Yet equal values in one destination register need not mean equal architectural results. Memory, flags, the program counter, and the outcome can still differ.

Consider two paused Ao states. Both have 7 in r_0 , 12 in r_1 , the same memory, and a running outcome. In the first state $pc = 8$; in the second $pc = 12$. If we record only r_0 , the difference disappears. At the next step it may decide which instruction executes. The missing component was not minor. It was merely unobserved.

State completeness is therefore relative to a declared machine contract. The Ao tuple is complete for Ao because every Ao instruction effect must appear in that tuple or its outcome. It is not complete for RV64, x86-64, or a physical processor merely because it is written with eight fields. Each later slice must perform its own inventory.

2.3.1 Many physical representations

An implementation need not store each architectural register in one fixed physical cell. An out-of-order processor may map one architectural name to different physical registers at successive moments. It may hold speculative results that have not yet become architectural results. It may keep several instructions in flight although the ISA describes their effects in program order.

The mapping and speculative values are implementation state. They matter to correctness because the processor must eventually present an allowed architectural

result. They do not become extra programmer-visible registers merely because the implementation uses them.

Suppose architectural register r_1 currently maps to physical location p_7 . A speculative instruction computes a candidate replacement in p_{12} . Before the instruction commits, the architectural reading still comes from the old committed mapping. At commit, the implementation may change the mapping to p_{12} . Two different physical descriptions therefore represent the same architectural state before commit, and a controlled change of representation realizes one architectural transition.

Another relation connects these levels. Let $\alpha(P) = S$ map a physical implementation state P to its represented architectural state S . A correct implementation need not take one physical step for each architectural step. It must make its longer physical evolution agree with the architectural transition at the declared comparison points.

The map can be many-to-one. Cache contents, queue positions, predictor tables, and unused physical registers may differ while α returns the same Ao state. That freedom permits implementation techniques. It also marks the boundary of a functional proof: equal $\alpha(P)$ values do not imply equal timing, power, or speculative leakage.

Exceptions make the comparison point especially important. If an instruction traps, the architecture specifies which earlier effects are visible and which later effects are not. An implementation may have computed later candidates, but it cannot expose them as committed architectural updates when the contract forbids that exposure. Chapter 16 will revisit speculation and leakage. Here we retain the simpler fact that internal work and architectural state change are different events.

2.3.2 Initial states are premises

Every execution theorem begins somewhere. Ao takes an initial state from a declared set. The set may fix the entry program counter, argument registers, selected memory bytes, and a running outcome while leaving scratch registers unrestricted. Nothing in the word *initial* makes an omitted component zero.

Real systems reach an entry state through earlier mechanisms: reset, firmware, a loader, an operating system, a caller, or a restored checkpoint. Those mechanisms have their own contracts. The Ao theorem may abstract them into a premise, but it must not borrow properties they have not promised.

This matters whenever a program reads before writing. If scratch register r_4 is unrestricted at entry and the first instruction adds it to the result, the program's behavior depends on r_4 . Testing from an all-zero snapshot hides that dependence. An honest repair writes a starting value to r_4 , adds a premise fixing it, or proves that no read occurs before a write.

The three repairs answer different questions. Writing the starting value changes the program. Adding a premise narrows the admitted environment. Proving write-

before-read establishes that the initial value is irrelevant. One cannot be substituted for another after a counterexample appears.

Reset state also differs from ordinary start state. A hardware reset may fix some architectural fields and leave others with values the architecture does not promise. A procedure entry normally inherits most state from its caller. A resumed task receives values from a saved context. Later chapters will name these environments precisely. For now, the lesson is enough: construction of the initial-state set is part of the claim.

2.3.3 Ownership of state

Ao describes one scalar machine. Its eight registers and condition bits belong to that one execution, and its finite memory is part of the same state. This simple ownership rule lets us write one tuple. Real systems divide state among hardware threads, processes, the operating system, devices, and shared memory.

The inventory alone does not settle that division. Two hardware threads may have separate integer registers and program counters while reaching a shared memory system. A process may see an address space assembled by system software. A device may change memory without executing an Ao instruction. Each case changes which actor can produce the next state.

Ownership matters to a frame claim. Saying that an instruction preserves “memory” is clear in Ao because no concurrent actor changes it during the step. In a shared system, the instruction may issue no write while another actor changes a location. A proof must distinguish preservation by this actor from global equality of the memory snapshot.

We therefore keep Ao single-threaded and closed during one step. External input, device action, interrupt delivery, and competing memory actions are not secret components of its state. They are excluded behaviors. Chapter 16 will add selected events and owners when it introduces concurrency and weak memory.

The exclusion is not merely convenient. It makes every state transition name one responsible rule. Once several actors may step, the semantics must say which interleavings or partial orders are permitted. A larger tuple without an ownership model would still leave the next-state question unanswered.

2.4 State as a software contract

Architectural state persists beyond one instruction. When an operating system stops one runnable task and starts another, it must eventually restore every state component that the first task is entitled to observe. A debugger needs enough of the same contract to suspend and resume execution. A virtual-machine monitor must preserve the guest-visible state across a transition. A checkpoint intended for later replay must bind both values and the model that gives those values meaning.

This turns architectural growth into systems work. A new visible register or mode may require storage in a task record, save-and-restore logic, debugger formats, crash dumps, virtual-machine state, migration checks, and tests for old software. The relevant costs have concrete units: bytes reserved per saved context, bytes transferred during a save, instructions or cycles spent away from the workload, and engineering effort in every layer that handles the state. The cost is not proof that an extension is undesirable. It is part of the extension's contract.

Intel's current manuals make the layering visible. The general-purpose registers, instruction pointer, and flags belong to the basic programming environment. Floating-point and vector facilities add further register state. Intel's older floating-point and vector save area occupies a fixed 512-byte region. The later extended-state facility organizes enabled features into separately identified state components and an extended region. Software can discover and manage those components rather than pretending that every processor has one timeless context shape. (Intel Corporation 2026a)

That example does not say that every context switch copies 512 bytes, nor that all supported components are enabled. Implementations can avoid unnecessary work, and operating systems choose policies. The architectural point is narrower: once a task can observe a component, suspension and restoration must not silently substitute another task's value.

Migration sharpens the same obligation. Moving an execution to another machine requires a destination that can represent the source's exposed state or an explicit translation that preserves the promised behavior. A byte dump is not enough if the destination assigns different meaning to a field. We call an encoding used for storage or transfer a **serialization**. State transfer depends on semantics, not only that encoding.

Formal verification pays another state cost. If a component can affect the property, the model must represent it or justify its abstraction. Each independent bit can double a raw finite state space. Proof methods survive by using preserved conditions, symbolic values, decomposition, and observations that forget irrelevant differences. Deleting a difficult component without proving it irrelevant is not abstraction. It is a change of theorem.

2.5 Observations

Now consider a less severe difference. Program p leaves 7 in r_0 and 19 in scratch register r_3 . Program q also leaves 7 in r_0 , but leaves 20 in r_3 . A caller that receives only r_0 may be unable to tell. A comparison of all registers can tell at once.

Observation

An **observation** is a declared function from a complete state to the information exposed by a claim. Two states agree under observation A when

$$A(S_1) = A(S_2).$$

In 1956 Edward Moore approached finite sequential machines from the position of an experimenter who could supply inputs and watch outputs. Two internal states were distinguishable when some experiment produced outcomes that depended on which state had been present. The question was operational: a hidden difference mattered when a finite interaction could expose it. (Moore 1956)

Our observation function is deliberately simpler than Moore's adaptive experiments. It inspects one complete Ao state at a declared boundary. Later, a surrounding program can turn a hidden flag or register difference into a visible result. The shared lesson is that a state difference and a detectable state difference are not the same statement. Detection depends on the permitted experiment or observation.

Object `A0.def.observat ion` fixes this role. An observation may select one register, all registers, a range of memory, the outcome, or a declared combination. The complete state still exists. The observation says which differences the claim exposes.

Figure 2.1 shows three observations, ordered from narrow to broad. This ordering is not always a straight line. One observation might expose a memory range and hide all registers; another might expose two registers and no memory. Neither necessarily contains the other. The useful relation is whether one observation can be derived from another. If it can, the first reveals no distinction that the second has already forgotten.

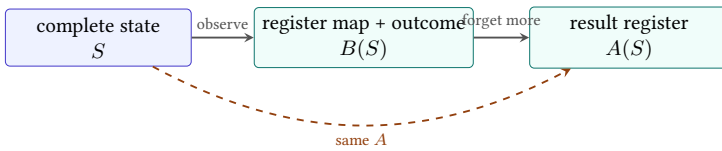


Figure 2.3: A narrower observation can factor through a broader one. Once the broader result is known, an additional projection obtains the narrower result without consulting the complete state again.

In the diagram, $A = f \circ B$ for a function f that selects the result register. Agreement under B then implies agreement under A : equal broad observations remain equal after applying f . The reverse may fail because f discards the other registers and the outcome.

Factorization gives an information order without counting output fields. Say that B is at least as informative as A when some function f makes $A = f \circ B$. Then every distinction made by A is also made by B . The proof is the equality calculation above. Two observations can be equally informative even when they encode their answers differently, so this order is best understood up to reversible recodings.

This view handles observations that are not simple projections. One may return the parity of a register, whether an address lies in a permitted range, or the coarse outcome stopped for either halt or trap. Listing more fields is not the definition of broader. The factorization test asks whether one answer can be computed from the other without reopening the complete state.

An observation partitions the state space into groups that it cannot distinguish. Every state with the same observed result belongs to the same group, even if hidden components differ. A broader observation usually splits some of those groups. Full-state observation leaves only groups of equal states.

Define $S_1 \sim_A S_2$ when $A(S_1) = A(S_2)$. This is an equivalence relation on the state space:

- reflexivity holds because $A(S) = A(S)$;
- symmetry holds because equality of observed values is symmetric; and
- transitivity—two successive agreements give an agreement between the endpoints—holds because $A(S_1) = A(S_2)$ and $A(S_2) = A(S_3)$ imply $A(S_1) = A(S_3)$.

The equivalence classes are exactly the groups in the partition. Observation does not merely discard a few printed columns. It replaces the complete state space with a quotient whose members are the distinctions the claim retains.

The partition view explains why an observation is neither correct nor incorrect by itself. It is adequate or inadequate for a context. If a caller can read only the result register and outcome, grouping states that differ solely in a dead scratch register may be appropriate. If a later branch reads a condition bit, an observation that groups different condition values is too coarse for replacement before that branch.

Observation fixes what counts as a difference

Choose the observation from the contract of the surrounding context. Then ask whether the programs agree under it. Choosing the projection after seeing a counterexample changes the question instead of answering it.

2.5.1 Continuations expose hidden differences

Take two running states with the same registers, memory, program counter, and outcome. Let only the zero condition differ. An observation of r_0 groups them

together. Now place both states before a conditional instruction that writes 1 to r_0 when the zero condition is set and 2 otherwise. After that **continuation**—a permitted sequence that runs after the boundary—the same result-register observation separates them.

The continuation is a Moore-style distinguishing experiment inside a program. The initial observation could not see the condition directly. The permitted continuation converted it into a visible register difference. Calling the condition hidden was therefore a statement about one observation at one boundary, not a promise that no future context could detect it.

Let K denote a continuation and let A be the final observation. The experiment observes

$$A(K(S)).$$

Two states are indistinguishable to a set of contexts only if this value agrees for every admitted K . The empty continuation recovers direct observation. Longer such sequences may expose differences in flags, scratch registers, memory, control location, or outcome.

This does not force every replacement claim to compare complete state. A procedure contract may promise that a scratch register is dead when the procedure returns. The admitted callers then exclude later sequences that read it before overwriting it. The contract, not our preference, determines the context set.

The continuation view also explains when states may be merged in a smaller machine. A proposed group must be stable under every admitted input: members of one group must produce matching visible outputs and move into the same next group. Otherwise one step separates states that the smaller model declared identical.

The proof is by attack. Suppose S_1 and S_2 are merged, but some input sends them to states in different groups or produces different visible outputs. Apply that input. The reduced model begins from one claimed state and would need two incompatible answers. The merge was not a valid state abstraction.

This stability condition is stronger than noticing that two snapshots happen to print the same value today. It asks whether the chosen equivalence survives the transition rules. Chapter 11 will turn that idea into program equivalence and refinement. Here it tells us why a state model cannot be reduced solely by deleting columns from a trace.

The scratch-register example gives a small proof of an important limit. Let $A(S) = R[r_0]$. Both final states map to 7, so they agree under A . Let $B(S) = R$, the complete register map. The first map sends r_3 to 19 and the second sends it to 20, so $B(S_p) \neq B(S_q)$. Agreement after applying one observation does not imply equality of the underlying states.

The converse direction is safe. If two complete states are equal, every function maps them to equal observations. Observation can forget a difference; it cannot create one between equal inputs.

Equality survives every observation

Assume $S_1 = S_2$. An observation A is a function, so replacing an input by an equal input cannot change its output. Therefore $A(S_1) = A(S_2)$. The reverse implication needs more: it holds only when A is **injective**, meaning that two different admitted states cannot have the same observed value. A projection to one register is not injective because many complete states share that register value.

The tiny argument will later carry a large burden. Full-state equivalence implies agreement under every declared observation. Observational equivalence does not imply full-state equality unless the observation retains enough information. When a proof uses the reverse direction, it must establish that extra premise rather than smuggle it in through the word *equivalent*.

The freedom is useful and dangerous. We must choose the observation as part of the claim, before seeing which components a proposed rewrite happens to disturb. Otherwise any difference can be dismissed after the fact by choosing a function that forgets it. That would not establish equivalence. It would only tailor the question to the answer.

We also need the observation to be stable across both programs. Comparing program p under one observation with program q under another can make any two results appear equal by choosing constant functions. One claim uses one declared observation, applied to both sides.

Design the smallest honest observation

A routine receives two words, uses r_3 as scratch space, writes its answer to r_0 , and may trap on an invalid input. A caller may later read r_0 and distinguish normal return from a trap, but it promises not to read r_3 . Write an observation that exposes exactly those facts. Then remove one component and describe the false replacement it would permit.

One result, three questions

Suppose two runs finish with the same r_0 , different C , and different outcomes: one halted and one trapped. They agree under the observation $A(S) = R[r_0]$. They disagree under an observation of $(R[r_0], C)$, and they disagree under any observation that includes the outcome. All three statements are exact. Only the first would be useful to a context that cannot inspect the carry bit or distinguish success from failure.

2.5.2 Initial-state premises

A program theorem rarely concerns every well-formed state. It may require a running outcome, a particular entry program counter, valid input addresses, or argument words in selected registers. These facts form the **initial-state premise**. They identify the states from which the claim promises behavior.

The premise differs from the observation. A premise restricts which inputs must be considered. An observation selects which output differences count. Weakening the premise asks the program to handle more starting states. Broadening the observation asks it to preserve more visible output state. Both changes make a claim harder, but they act at opposite ends of execution.

One address premise prevents an expected trap

Suppose a routine loads four bytes beginning at the address in r_1 . A premise that all four addresses exist removes invalid-range inputs from the claim. Without that premise, a correct theorem must include trapped outcomes or establish some other reason the range is valid. Hiding the outcome in the final observation would not make the load succeed.

Premises must be stated before testing candidate programs. Otherwise a failed input can be excluded merely because it is inconvenient, just as an output difference can be hidden by choosing a narrow observation afterward. Later objects will bind the input premise and output observation beside the program and machine model so neither can drift during search or replay.

For cross-ISA work, the premise may be relational. It can say that an Ao register and an RV64 register represent the same argument while their unused registers differ. The output observation then asks whether the related runs produce corresponding results and outcomes. Equality of the raw state tuples is neither expected nor required. What matters is the declared correspondence at entry, at matched control points, and at the observed exit.

2.6 Reachability and invariants

A state becomes useful when a rule can move from it. For one decoded instruction i , a deterministic rule has the shape

$$T_i : S \mapsto S'.$$

The arrow says that the complete old state is the input and the complete new state is the output. It does not say that every mathematical tuple admits a step. Ao accepts only well-formed running states whose instruction and addresses satisfy the relevant rules. Ao permits no successor after either halt or trap.

We could model failure with a partial function that simply has no result. Ao instead records a trapped outcome with a reason when a running instruction encounters a declared failure. This preserves a difference that later claims may observe. After the trap is recorded, no further Ao step follows. A rule that silently drops the state would lose the evidence needed to distinguish failure from successful halt.

A transition relation has the broader shape $S \ T_i \ S'$. One old state may relate to no successor, one successor, or several successors. A deterministic function is the special case with at most one successor for each admitted input. Relations become necessary when the model deliberately keeps a choice open, as later concurrency and weak-memory models may do. The state inventory still has to be sufficient: the permitted successor set must not depend on an unnamed past.

Deterministic does not mean easy to predict. A 64-bit state function can be computationally expensive, and the initial state may be unknown. Conversely, a relation with several outcomes can support a strong theorem if every permitted outcome satisfies the property. The mathematical shape tells us how many successors the semantics permits, not how convenient analysis will be.

Fix an initial-state set I . A state is **reachable** when zero or more legal steps lead to it from a member of I . Reachability is narrower than well-formedness. A tuple may satisfy every field rule yet never arise from the selected program and entry states.

This distinction supports induction over execution. Suppose a property $P(S)$ holds for every initial state and every legal step preserves it:

$$P(S) \wedge S \ T \ S' \implies P(S').$$

Then P holds for every reachable state. The reader proof follows the length of the path. It holds at length zero by the initial premise. If it holds after n steps, preservation gives it after step $n + 1$. The argument does not inspect every reachable state separately.

Such a preserved property is an **invariant**. Well-formedness is the first one we need: constructors establish it, and every instruction rule must preserve it. Later preserved properties will connect a loop counter to consumed memory, or an Ao state to an RV64 state at matching control points. The word earns its keep only when the initial case and preservation step are both present.

Reachability can justify a smaller proof domain, but only after it is proved. Declaring an awkward state unreachable is not a premise-free shortcut. The initial set, transition rules, and invariant must rule it out. This is another form of the chapter's central discipline: a missing state needs an argument, not an omission.

2.7 RV64 and x86-64 state

Ao now gives us an inventory to carry to a real instruction set. We ask which registers exist, what the program counter denotes, which condition state an instruction

can read or write, how memory enters the model, and which exceptional outcomes the selected instruction forms admit.

For the later RV64 slice, the integer register named $x0$ needs special treatment. A model that stores and later returns an arbitrary word from $x0$ would describe a different machine. The slice must also name its program counter, ordinary integer registers, memory assumptions, and relevant exceptional outcomes. In base RV64I, the integer register file has 32 width-64 registers; reads of $x0$ return zero and attempted writes are discarded. The architecture also has a program counter, although ordinary code does not treat it as another integer register. (RISC-V International 2026e)

For the later x86-64 slice, the model must name the selected general-purpose registers, RIP, memory, and exactly which RFLAGS bits matter to the included forms. An arithmetic replacement that preserves its destination but changes a later branch condition is not full-state equality. It may be valid under a narrower observation, but only if the surrounding program is unable to rely on the omitted flag.

In 64-bit mode, the basic Intel programming environment exposes 16 general-purpose registers, the 64-bit instruction pointer RIP, and the RFLAGS register, among other state. That sentence is still broader than our teaching slice. An x86 instruction may preserve, define, clear, set, or leave individual flag bits undefined. The slice must take the rule from each included instruction form rather than model RFLAGS as one ordinary destination word. (Intel Corporation 2026a; Intel Corporation 2026b)

Table 2.1: A relation must say which parts of the three inventories correspond.

Machine	Integer and control state	Conditions, memory, and outcome
A0	eight ordinary width- w registers and pc	four condition bits, finite byte memory, and running, halt, or named trap
RV64 slice	mapped subset of 32 width-64 registers, special $x0$, and program counter	no base integer condition-code register; related memory and selected result
Intel slice	mapped subset of 16 general-purpose registers and RIP	selected RFLAGS bits, related memory, and selected result

The table is a proof checklist, not a claim that the rows are identical. A relation may connect $A0$ r_0 to RV64 $x10$ and x86-64 RAX, while allowing unrelated scratch registers to differ. It may connect three control locations that use different numeric addresses. It must also say what happens to $A0$ condition bits when the RV64 segment has no corresponding flag state.

These are state questions, not verdicts about RISC and CISC. One architecture has a special integer-register rule; the other example carries selected condition

bits. The exact slices and source revisions must be fixed before either description supports an architectural claim.

The comparison also explains why cross-ISA states will not be equal. Ao has eight ordinary registers and four named condition bits. RV64 has a larger integer register file with a special zero-register rule in the planned slice. The selected x86-64 slice has its own register names, partial-register rules, and condition state. Later we will relate states by what they represent, not by demanding identical tuples.

A state relation is not an observation

An observation extracts information from one machine state. A state relation connects states from two possibly different machines. Cross-ISA proofs need both: a relation for corresponding inputs and intermediate states, and an observation for the outputs the claim exposes.

2.8 Implementation boundary

We can now say what an executable Ao state package must do. It must construct the entire tuple, enforce the width and memory conditions, represent all three outcomes, and apply observations without changing the state they inspect. Object `OP.a0.state-memory` records that implementation obligation for Axeyum.

The controls follow from the reader examples. A projection checker must reject a claim that two unequal full states are equal merely because r_0 matches. An outcome test must distinguish halt from trap. A state constructor must reject a width that is not a positive multiple of eight. A memory test must not allow an address outside the finite map.

The first Axeyum lesson at this layer should remain small. Construct two Ao states that differ only in r_3 . Apply an output-register observation and a full-register observation. The first results should agree; the second should produce the differing register as a witness. Mutate the projection so that it silently omits a requested component. The negative control must reject the mutated route.

The serialized state should be canonical enough for an artifact digest to bind one meaning. Register order, address order, word width, outcome tags, and trap reasons must not depend on a hash-map iteration order or a Python display choice. A reader-facing table may choose a pleasant order, but replay should parse one explicit representation and reject duplicates, missing fields, and out-of-width values.

Canonical means that one admitted state has one accepted byte encoding. It is stronger than merely having a parser. If two field orders or two number spellings represent the same state, a digest of the bytes can change while the meaning does not. If one byte sequence admits two parses, the digest does not even select one state.

Let E encode an Ao state and D decode admitted bytes. The first reader law is a round trip:

$$D(E(S)) = S.$$

This law implies that encoding is injective. If $E(S_1) = E(S_2)$, apply D to both sides. The round-trip law gives $S_1 = S_2$. Thus equal encoded bytes cannot conceal two different states.

The other direction needs a stated byte grammar. For every canonical encoding b , require $E(D(b)) = b$. A permissive parser may accept noncanonical input for a user interface, but an evidence checker should either reject it or normalize it before hashing. The choice must not depend on which library or language happens to read the artifact.

Useful negative controls follow directly. Swap two register entries, repeat a memory address, omit the outcome, put a nine-bit value in an eight-bit field, or append an unknown field. Each mutation attacks a different ambiguity. The checker must reject the changed artifact rather than silently invent a default state.

Observation descriptions also belong in the artifact. A label such as `result` is insufficient unless it resolves to a declared register, width, and outcome policy. The checker should rebuild the observation from that declaration and apply it to both full states. It should not trust stored observation outputs that could have been edited independently of their source states.

The Python layer can make this lesson direct. It constructs states through the bound Rust types, asks for named observations, and prints both the compact result and any separating component. Rust owns width checking, canonical serialization, projection, and comparison. Python owns presentation and the sequence of questions; it does not carry a second definition of state.

That exercise teaches the division of labor. Executable state constructors make malformed states hard to express. Observation functions state what the claim exposes. A solver may search for two states that a proposed implication handles incorrectly. A replay checks the returned states through the same executable definitions. None of those layers decides which observation is appropriate for the surrounding program; that remains part of the theorem we choose to state.

No such Axeyum package accompanies this draft yet. The tuple and the arguments in this chapter are definitions and reader reasoning; they are not a hidden machine-proof route. That boundary will remain visible until the executable state and its controls exist.

2.9 What the state model fixes

The chapter began with a paused machine and a question about what remains. We can now answer without listing hardware parts by habit. A state is sufficient when equal recorded states make the relevant past interchangeable for the next modeled

behavior. Ao chooses one such state for a scalar teaching machine. Its tuple records words, bytes, control location, conditions, and outcome; its well-formedness rules say which tuples are legal.

The model fixes five distinctions that later proofs will rely on:

- a mathematical bit and the physical regions that realize it are related by a decoding contract, not identity.
- architectural state is the state named by the semantic contract, not every latch and queue in one processor.
- complete state and observed state differ because an observation is a declared function that may forget information.
- a premise restricts admitted initial states, while an observation selects which final differences count.
- a relation connects states of unlike machines, while an observation reads one state.

These distinctions also locate the costs. Physical storage spends area and energy to retain distinctions. System software spends bytes and execution time to preserve visible state. Verification spends representation and proof effort on every component that can affect the claim. An observation can reduce that burden only when the surrounding contract truly cannot expose what it forgets.

One quiet fact connects the layers. The physical machine carries a vast and changing electrical history. The architectural state replaces that history with a finite mathematical object sufficient for the next specified step. That compression is what makes an instruction rule possible.

A state is a still picture. It tells us where every modeled value is, what may happen next, and whether another step is allowed. It does not move itself. Chapter 3 supplies the object that turns one such picture into the next.

2.10 Exercises

1. **Execute.** Write an Ao state of width eight with register $r_i = i$, memory bytes 10, 20, 30, 40 at addresses 0 through 3, $pc = 0$, cleared condition bits, and a running outcome. Evaluate observations that select r_0 , all registers, and (pc, O) .
2. **Break.** Construct two states that agree in every register but differ in memory. Give an observation that hides the difference and one that exposes it. Explain why neither observation changes the states.
3. **Prove.** Show that complete-state equality implies agreement under every observation. Then use a two-element example to show that the converse is false for a non-injective observation.

4. **Transfer.** For a proposed RV64 state and a proposed x86-64 state, list one special component or rule that has no direct Ao counterpart. State the functional question that makes it relevant.
5. **Explain.** A sequence changes C but no register or memory byte. State two replacement claims for which that fact leads to different answers.
6. **Design.** Give one timing question for which Ao architectural state is too small. Name the missing physical information without adding it to Ao.
7. **Execute.** Starting from the state in the first exercise, change only pc , then only O , then one memory byte. For each change, list the observations in Figure 2.1 that can detect it.
8. **Prove.** Let observation A be computable from observation B . Show that agreement under B implies agreement under A . Give an example showing that the reverse implication can fail.
9. **Break.** Define an observation after inspecting two unequal states so that it hides their only difference. Explain why the resulting true equality does not support the original full-state claim.
10. **Transfer.** Sketch a relation between an Ao input state and a future real-ISA input state for a routine with two arguments and one result. Name what the relation deliberately leaves unmatched.
11. **Explain.** The hidden-mode device records only its visible bit. Give two histories that reach visible bit zero and then produce different answers to the same next command. State both honest repairs to its model.
12. **Execute.** For the storage rule $q_{t+1} = d_t$ when $e_t = 1$ and $q_{t+1} = q_t$ otherwise, begin with $q_0 = 0$. Apply the pairs $(d, e) = (1, 0), (1, 1), (0, 0), (0, 1)$. Record every retained value and identify which inputs were ignored.
13. **Prove.** Count the register, program-counter, and condition assignments of Ao at width eight before memory and outcome enter. Then include the three coarse outcome classes. Explain why this is not a reachable-state count for a particular program.
14. **Break.** Remove the preservation branch from the complete-update definition. Give two different next states that both satisfy an instruction description saying only that r_0 receives $r_1 + r_2$.
15. **Audit.** A test compares the declared write components before and after an instruction but never checks components outside the write set. Inject one hidden memory write that the test accepts. State the missing check.

16. **Prove.** Show that agreement under a fixed observation is an equivalence relation. For an observation of the low bit of r_0 , describe one complete equivalence class at width four.
17. **Design.** Give two observations that return different data formats but carry the same information. Supply conversion functions in both directions.
18. **Break.** Find observations A and B such that neither factors through the other. Use one register observation and one memory observation, then exhibit a pair of states separated by each but not the other.
19. **Execute.** Construct two states that differ only in Z . Apply a continuation that copies 1 into r_0 when $Z = 1$ and 2 otherwise. Show the direct and post-continuation results under observation of r_0 .
20. **Prove.** A proposed state abstraction merges two states, but one input gives different visible outputs from them. Prove that no deterministic reduced transition rule can preserve both behaviors from the merged state.
21. **Generalize.** Replace Ao's deterministic step with a relation. Rewrite state sufficiency so that it compares sets of permitted successors. State what remains unchanged in the hidden-mode counterexample.
22. **Prove.** Use induction on path length to show that an initial and step-preserved property holds in every reachable state. Identify the two premises that would be missing from the claim "bad states are unreachable."
23. **Economics.** An extension adds k bytes of task-visible state. For n suspended tasks, express the direct storage increase. List two other costs that cannot be inferred from k alone and name the measurements they would require.
24. **Transfer.** Map Ao r_0, r_1, r_2 to three selected RV64 registers and three selected x86-64 registers. Add program-counter and outcome clauses. State why this relation says nothing yet about instruction behavior.
25. **Prove.** Assume $D(E(S)) = S$ for every admitted state. Prove that E is injective. Then explain why this law alone does not forbid two different byte strings from decoding to the same state.
26. **Audit.** Design five mutations of a serialized Ao state: a duplicate register, a duplicate address, an omitted outcome, an out-of-width word, and an unknown field. State the required checker result for each.
27. **Boundary.** Name one functional, one timing, and one leakage claim that the Ao tuple cannot establish. For each, identify the missing state or physical behavior without expanding Ao itself.

Instructions and Operands

The word `add` does not say enough. It does not name the width, the inputs, the destination, the condition bits, the next program counter, or the cases in which execution cannot continue. It is a useful name for a family of actions, not a complete account of any one action.

To move from one machine state to the next, we need an operation, an operand form, concrete operands, and a rule that exposes every architectural effect. The printed instruction is short. Its meaning cannot be.

An instruction meaning is a complete transition

An instruction does not merely calculate a value. It transforms one declared machine state into another, or produces a declared failure. Every component that can affect later execution belongs in that account.

This view separates three questions that assembly listings often place on one line. Which bytes were fetched? Which instruction instance did the decoder produce? What does that instance do to state? A correct answer to one does not repair an error in another.

3.1 Instructions as state transitions

3.1.1 From operation tables to transition rules

An instruction table is one of computing's durable forms. It gives a short name or bit pattern to an operation and tells a programmer which operands may accompany it. Early manuals needed such tables because a machine could perform only the actions supplied by its hardware. Modern architecture manuals still use them because compatibility requires precise answers about decades of accumulated forms.

The purpose has widened. Compilers use instruction descriptions to select and schedule code. Assemblers turn accepted forms into bytes. Disassemblers try to recover forms from bytes. Emulators perform the effects. Hardware verification

compares an implementation with an architectural account. Binary translators relate one machine's transition to another's. Each consumer needs more than a mnemonic, though not always the same more.

For this book, the central object is the architectural transition. It lies between notation and hardware. It says what state change software is entitled to observe while leaving circuit organization and timing to a later model. That position gives the instruction an unusual reach: a finite rule describes its action on every admitted state.

That compact rule was not obvious at the beginning of stored-program computing. The 1945 EDVAC report called instructions *orders*. Before giving their bit patterns, it asked for their mathematical and logical meaning and their operational significance. It then classified orders partly by where values came from and where results went. The terminology is old; the separation is current. An operation such as addition, the roles that supply its values, and the code that selects the operation are related facts, not one fact. (Neumann 1945)

The physical restrictions of an early machine made each choice expensive. An order had to fit the available storage, control circuits had to recognize it, and data had to reach the correct arithmetic path. Later machines gained more storage and more circuits, but they also acquired a stronger constraint: old programs had to keep their meaning. An instruction table became a promise made across time.

The original System/360 manual shows that promise taking a systematic form. Its five basic formats were called RR, RX, RS, SI, and SS. Their fields named register operands, indexed storage addresses, immediate data, and explicit lengths. A one-halfword instruction named no storage address; longer formats made room for one or two. The format did not merely arrange bits. It organized the routes by which operands reached an operation across a compatible family of implementations. (International Business Machines Corporation 1964; Amdahl et al. 1964)

This history prevents two opposite mistakes. We should not treat today's three-register form as the natural shape of every instruction. We should also not treat a mature instruction with several effects as an arbitrary bag of exceptions. Operand arrangements record engineering choices about storage, circuits, code density, address reach, and compatibility. Once software depends on those choices, changing them has a cost beyond the decoder.

The architectural rule still does not prescribe the control circuit. Wilkes and Stringer described a control unit in which one machine instruction could be realized by a sequence of smaller control operations. That is the root of what came to be called microprogramming. It demonstrates a general point: one instruction transition may be implemented by many internal steps without making those steps visible as architectural instructions. (Wilkes and Stringer 1953)

Some present processors use hardwired control for one form, sequenced control for another, or a mixture that changes between implementations. This book does not infer a current processor's internal method from the apparent complexity of its

assembly. The architectural rule says what must be true at the boundary. Chapter 16 will ask how an implementation can arrive there.

A short operation name can denote a vast table

At width sixty-four, one register addition rule covers more input pairs than could be listed. The rule earns that compression by stating the arithmetic, conditions, failures, and preserved state exactly. A short name without those clauses compresses nothing; it merely postpones the definition.

3.2 From spelling to action

We can now name four objects that a listing tends to collapse:

1. an *assembly statement*, which belongs to a notation accepted by an assembler;
2. an *encoded instruction*, which is a sequence of bytes or bits in an instruction set;
3. a *decoded instruction instance*, which selects an operation and legal concrete operands; and
4. an *instruction transition*, which determines the architectural successor for an admitted state.

The arrows between these objects need not be one-to-one. An assembler may accept aliases or pseudo-instructions. Several encodings may denote closely related operations. A **disassembler**, a tool that turns instruction bytes back into assembly text, must choose one printable spelling. The same decoded instance produces different successor values in different input states. Keeping the objects separate lets us state which arrow a test checks.

Instruction instance

An **instruction instance** is an operation together with one allowed operand form and concrete operands. Its semantics gives the complete state change for a running input state, including the next program counter and any trapped outcome.

Formally, let O be the finite set of Ao operations. Each operation $o \in O$ has a set F_o of admitted operand forms. Each form f has a set $P_{o,f}$ of concrete operand tuples. The decoded instruction space is the tagged union

$$I = \bigcup_{o \in O} \bigcup_{f \in F_o} \{o\} \times \{f\} \times P_{o,f}.$$

The tags matter. The bit pattern for a register number and the bit pattern for an immediate may happen to contain the same numeral; they do not acquire the same role. The form tells the semantics how to interpret each field.

An integer triple without type tags, such as $(3, 1, 2)$, is therefore not yet an addition. It becomes the register instance `add r3, r1, r2` only under the addition operation and its register-register form. Under another form, a field could be an offset, a condition selector, or a reserved value. A typed constructor keeps those alternatives apart before execution begins.

Object `A0.def.instruction` records this state-transformer view. A decoded instruction is not the same object as its printed assembly or its instruction bytes. Assembly is a human-readable notation. Decoding turns bytes into an instruction instance. Semantics says what that instance does. Chapter 6 will study the boundary between the last two.

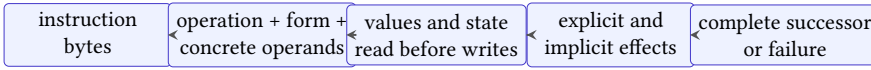


Figure 3.1: One printed instruction conceals several contracts. Bytes select an instance; the instance determines reads and effects; the semantics returns a complete successor or a declared failure.

For deterministic Ao, a fixed instruction instance and running input state select at most one successor. We may therefore present its meaning as a function whose result includes either an ordinary state or a trapped state. Other machine models may use a relation and admit several successors: an value that the model leaves partly unspecified, an external event, or a concurrent action can make more than one next state possible. Calling both accounts semantics does not make their proof obligations identical.

Let S_{run} be the well-formed running Ao states and let S_{stop} contain the declared halted and trapped states. A useful type for the semantics is

$$\text{exec} : I \times S_{\text{run}} \longrightarrow S_{\text{run}} \cup S_{\text{stop}}.$$

This function is total over legal instances and admitted running states: a failed memory access returns a trap rather than falling outside mathematics. The whole-machine step is still partial because stopped states have no next instruction. Total instruction outcomes and partial program continuation are different design choices.

One may instead define a relation $E \subseteq I \times S \times S$. A function gives a convenient executor; a relation can state several allowed successors or other permitted choice. For deterministic Ao they agree when

$$(i, S, S') \in E \quad \text{exactly when} \quad \text{exec}(i, S) = S'.$$

The book uses the function for hand execution and the relational reading when it prepares proof claims. Neither presentation licenses an omitted case.

Deterministic instruction semantics

An instruction semantics is deterministic when one instruction instance and one admitted input state never relate to two different successor states. Existence is separate: a halted state may have no instruction successor, and a partial model may leave some forms outside its domain.

Determinism is useful because a witness state can be replayed without making another choice. It does not imply termination of a program, successful execution, or simplicity of an instruction. A deterministic load may trap on one state and succeed on another. A deterministic variable-length decoder may need several bytes before it selects its one answer.

The separation matters even when the printed text looks familiar. Two assemblers may accept different spellings for the same decoded form. One sequence of bytes may be illegal even though a nearby legal sequence prints a recognizable mnemonic. A decoder can correctly identify an instruction whose transition function is wrong. The book will test each boundary with a different kind of control.

Consider the Ao instance `add r0, r1, r2`. It reads $R[r_1]$ and $R[r_2]$. It adds them modulo 2^w , writes the result to $R[r_0]$, sets Z, N, C, V by the Ao addition rules, and advances pc by four modulo 2^w . Other registers and data memory remain unchanged. Because the instruction has already been decoded, no data-range trap belongs to this particular operation.

Take width eight, $R[r_1] = 255$, and $R[r_2] = 1$. The result in r_0 is 0. The zero bit Z is one, the negative bit N is zero, and the carry bit C is one because the mathematical sum reached 256. Signed overflow V is zero: the signed inputs are -1 and 1, whose mathematical sum fits in the signed width. If $pc = 12$, the next sequential value is 16.

One complete A0 addition

The input facts are width eight, $R[r_1] = 255$, $R[r_2] = 1$, $pc = 12$, and a running outcome. The instruction reads r_1 and r_2 . It writes 0 to r_0 , writes $(Z, N, C, V) = (1, 0, 1, 0)$, advances pc to 16, preserves all other registers and data memory, and leaves the outcome running. Removing any one of those clauses produces an incomplete transition.

Preservation clauses deserve to be explicit. If a rule lists only the fields that change, a reader may infer that every unlisted field stays fixed. An executable semantics must enforce that inference. Otherwise an implementation could change r_7 while still satisfying claimed output conditions that mention only r_0 and the flags.

This worked step shows why a destination value is not the whole result. A rule that writes 0 to r_0 but leaves $C = 0$ has performed a different Ao transition. So has a rule that leaves $pc = 12$.

3.2.1 The complete effect of addition

Let the decoded instruction i be printed as `add rd, ra, rb`, and let S be a well-formed running state. Write

$$x = S.R[r_a], \quad y = S.R[r_b], \quad n = 2^w.$$

The mathematical sum $m = x + y$ lies between 0 and $2n - 2$. Its destination word is the unique remainder

$$z = m \bmod n.$$

The carry bit records whether the mathematical sum crossed the word boundary:

$$C' = [m \geq n].$$

Here the brackets denote one when the proposition is true and zero otherwise. The zero and negative bits inspect the width- w result:

$$Z' = [z = 0], \quad N' = [z \geq 2^{w-1}].$$

The signed-overflow bit V' follows the Ao addition definition from Chapter 1: it is one exactly when equal-sign signed inputs produce a result with the opposite sign. Chapter 8 will derive and compare the arithmetic tests. For this chapter, the important fact is that V' is a declared write even when the destination word already looks right.

Let $p = (S.pc + 4) \bmod n$. The successful successor is the complete update

$$S' = S[R[r_d] \leftarrow z, Z \leftarrow Z', N \leftarrow N', C \leftarrow C', V \leftarrow V', pc \leftarrow p].$$

The update notation inherits Chapter 2's frame law. Every unlisted register, every memory byte, and the running outcome retain their old values. If $r_d = r_a$, the right side still uses the old x , because all right-side expressions are evaluated in S .

Take width four and the instance `add r1, r1, r1`. Suppose the old $R[r_1] = 9$, old $pc = 28$, and old flags are arbitrary. The mathematical sum is 18, so $z = 2$ and $C' = 1$. The result is nonzero and its high bit is zero, so $Z' = 0$ and $N' = 0$. Both signed inputs represent -7, while the result represents 2, so $V' = 1$. The next program counter is $(28 + 4) \bmod 16 = 0$. Aliasing and program-counter wrap both occur in the same small example, yet the rule needs no special assignment order.

A plausible result with three hidden errors

For that width-four state, suppose an executor returns $R[r_1] = 2$ but sets $C = 0$, leaves $V = 0$, and reports $pc = 32$. Its visible arithmetic remainder is correct. Its carry and signed-overflow observations are wrong, and 32 is not even a width-four program-counter word. The result violates both the instruction rule and the state well-formedness invariant. Checking the destination alone accepts all three defects.

This example also separates two kinds of rejection. A checker can reject the result because a declared effect is wrong. It can independently reject it because the alleged state is not in the state space. A robust executor should construct only well-formed output states, while an artifact checker should still defend against malformed serialized claims.

The A0 addition successor is unique

For a fixed legal instance and running state, the two register reads and old program counter are fixed. Division by 2^w gives one remainder z . Each condition-bit predicate therefore has one Boolean value, and modular addition gives one next program counter. The complete-update law fixes every listed component to those values and preserves every unlisted component. No second distinct state can satisfy all of those component equations.

Uniqueness is not the same as correctness of an implementation. The equations show that the specification admits at most one successor. Evidence must still show that an executor returns that successor for the states in its claim. A hand example covers one state; exhaustive enumeration covers a stated finite width; a symbolic proof covers the formula and assumptions accepted by its checking route.

3.3 Operand forms

An operand is often introduced as a place where an instruction gets a value. That account is too narrow. A destination operand names where a value goes. A memory operand also triggers address formation and can trap. A relative target contributes to the next program counter rather than to arithmetic data.

Operand form

An Ao **operand form** states how an operand is obtained, whether the instruction reads or writes it, its width, and any state effect required to use it. Ao has register operands, signed immediate values, base-plus-offset memory locations, and relative branch targets.

Object `A0.def.operand` records these four forms. An Ao immediate comes from the signed eight-bit immediate field and is sign-extended to w bits. A memory address adds that same signed offset to a base register. A relative target adds four times the signed offset to the sequential address $pc + 4$. The multiplication keeps branch targets on four-byte instruction boundaries.

The four forms do not all return the same kind of thing. A source operand is resolved to a value. A destination operand is resolved to a location that can be updated. A branch operand is resolved to a possible next address. Treating all three as words loses the rule that connects a calculated word to a part of state.

Let W be the set of width- w words and L the Ao locations that an instruction may write. Let locations, and A the set of code or data addresses. We can give the operand roles separate functions:

$$\begin{aligned} \text{source} &: P_{\text{src}} \times S \longrightarrow W \cup \text{Trap}, \\ \text{destination} &: P_{\text{dst}} \times S \longrightarrow L \cup \text{Trap}, \\ \text{target} &: P_{\text{target}} \times S \longrightarrow A \cup \text{Trap}. \end{aligned}$$

A register source returns the old register value. A register destination returns the named register location. A memory source first returns an address, checks the required byte range, and then returns the interpreted word. These functions share operand syntax only where the architecture says they do.

This distinction is familiar outside machine semantics. In algebra, an expression denotes a value. In programming languages, an assignable place must also identify where an update will occur. In logic, the assertion after an update must distinguish the new value at that place from every preserved place. Instruction semantics brings those three views together without requiring one overloaded notion of “operand.”

Consider `movi r3, -1`. The immediate source function ignores the register file and sign-extends the encoded eight-bit value to w bits. The destination function identifies r_3 . The operation then writes the source value to that location. By contrast, `load r3, [r1+4]` uses r_1 and the offset to form an address, reads memory through a checked access, and writes the returned value to the same destination location. The destination syntax agrees; the source computation does not.

There is also a useful difference between a location and its present value. If $R[r_3] = 9$, resolving source r_3 gives 9, while resolving destination r_3 gives the register-file position named r_3 . Writing 10 changes the value stored at that position;

Table 3.1: The four Ao operand forms and the work each form contributes.

Form	Value source	State access	Extra rule
register	named register	read or write	width w
immediate	instruction byte	no register read	sign extension
memory	base plus offset	register and bytes	range check
relative tar- get	sequential pc plus offset	program counter	alignment and code range

it does not change which position the operand names. Confusing the two makes read-modify-write forms impossible to state cleanly.

The written operands need not list every state component the operation uses. The Ao add instance prints three registers yet also writes four condition bits and the program counter. These are **implicit effects**: architectural reads or writes fixed by the operation even though they are not printed as operands.

The distinction between a printed operand and a value role prevents another shortcut. A destination may also be a source. An address operand may read a base register, read an offset from the instruction, access memory, and expose a trap. A branch target may consume the current program counter even when the assembly does not print it. Counting commas does not count semantic inputs.

We can classify effects along two axes. An effect is *explicit* when an assembly operand names the affected location or value role. It is *implicit* when the operation fixes that role without printing it. An effect is *data* when it contributes to the main calculated result and *control* when it determines continuation, conditions, or failure. The ordinary next program counter is implicit control state. An x86-64 arithmetic flag write is also implicit even though later instructions can read it as data. These categories describe the interface; they do not rank importance.

Explicit effects

A semantic rule must expose every architectural read, write, program-counter change, and exceptional outcome used by its instruction form.

The explicit-effects principle, object `A0.prin.explicit-effects`, records this rule. It is a discipline for the semantics, not a requirement that assembly syntax spell out every effect.

The frame is the complement of those effects. If $W_i(S)$ is the write footprint of instruction i in state S , then every successful successor S' must satisfy

$$\ell \notin W_i(S) \implies S'(\ell) = S(\ell).$$

This is not shorthand for “the other fields probably stay unchanged.” It is one part of the instruction rule. A state diff is useful to a reader only because this law gives absence from the diff a precise meaning.

3.3.1 Operand validity before arithmetic

An operation name does not accept every operand tuple. A register field must name an available register. A form may require distinct roles, a particular width, or an immediate that fits its encoded field. A memory form may permit only certain base and offset combinations. These restrictions define the set of legal instruction instances.

Keeping that check separate from execution gives illegal forms an honest home. If reserved bytes decode to no instruction, the failure belongs to decoding. If an external API tries to construct an operand tuple that no Ao encoding can denote, the typed constructor should reject it before the step function runs. The execution semantics need not invent behavior for an object the instruction set excludes.

There are three distinct predicates in this neighborhood. A byte sequence can be available for fetching from the current program counter. A fetched sequence can decode to a legal instance. A legal instance can be executable without a state-based trap. For Ao we can write them as

$$\text{canFetch}(P, pc), \quad \text{decodes}(b, i), \quad \text{enabled}(i, S).$$

The first speaks about the program and an address. The second speaks about bytes and the instruction grammar. The third speaks about a decoded instance and running state. A valid decoder must not use a later execution check to give meaning to reserved bytes.

This separation also clarifies partial specifications. If a book defines only register addition, a memory-form byte sequence is outside that small decoder; it is not an addition with an unknown result. If a legal load has an invalid data address, it is inside the instruction set and returns the defined access trap. “Not modeled,” “illegal,” and “trapped” are three different claims.

Legality precedes effect

First establish that the bytes select one admitted instruction instance. Then apply the transition for that instance to state. A semantic equation for a legal addition says nothing about a reserved encoding that resembles it.

This distinction matters in testing. Random instruction bytes exercise the decoder’s acceptance partition. Random legal instruction instances exercise the transition rules. Generating the second set by decoding the first is useful, but it can miss a legal form that the decoder never emits. A complete test plan checks the two domains and their connection. Neither collection can establish coverage of the other merely by producing many successful examples.

3.3.2 Read before commit

An instruction may name the same location in more than one role. In `add r0, r0, r1`, the old value of r_0 is a source and r_0 is also the destination. The semantics must read the source before replacing the destination. Otherwise the result would depend on the order in which a prose description happened to mention its effects.

A clean rule takes a logical snapshot of every required input from the old state. It evaluates register sources, immediate values, addresses, and any required memory bytes against that state. Only after those operations succeed does it commit the destination, implicit state, next program counter, and outcome together.

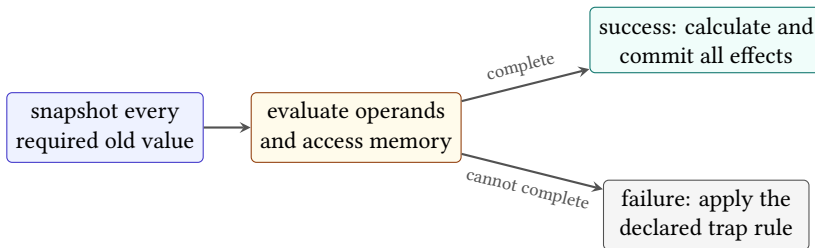


Figure 3.2: Operand evaluation uses the old state. The instruction commits its ordinary effects only after every required operand operation succeeds.

This snapshot is semantic, not a demand that hardware copy the full state. An implementation may forward a result, rename a register, or calculate several pieces in parallel. It implements the rule when its architectural successor is the same as if the inputs came from the old state and the effects were committed according to the declared outcome.

Memory makes failure ordering visible. Suppose an instruction reads a memory source and writes a register destination. If the address or range check fails, the destination write does not occur under the Ao rule. A faulty implementation that clears the destination first and discovers the bad address second has created a successor the semantics excludes.

Prose order is not effect order

The sentence “write the destination and advance the program counter” does not license an implementation to expose the first write when a later required operation fails. A semantic rule must state which effects commit together and which state accompanies each failure.

For instructions with two memory operands or implicit stack effects, the same issue becomes more demanding. The rule must identify every access, their ordering, and the state retained on failure. Ao avoids those forms in its first instruction set,

but the method is already in place for the selected x86-64 cases that combine more work in one instruction.

3.3.3 Evaluation and commit

We can make the snapshot rule exact. For a legal instance i , let $E_i(S)$ evaluate every required source, destination location, address, condition, and target in the old state. Its result is either a bundle q of resolved facts or a trap reason t :

$$E_i : S_{\text{run}} \longrightarrow Q_i \cup \text{TrapReason}.$$

The successful commit function $C_i(S, q)$ begins from the old state S , replaces exactly the declared write locations, and returns a running state. The trap commit $T_i(S, t)$ begins from the same old state and applies the declared trap rule. Thus

$$\text{exec}(i, S) = \begin{cases} C_i(S, q), & E_i(S) = q, \\ T_i(S, t), & E_i(S) = t. \end{cases}$$

The notation does not require a physical evaluation phase followed by a physical commit pulse. It requires the visible result to agree with that logical construction.

For Ao register addition, evaluation returns two old register values, the destination location, and the old program counter. It cannot trap after a legal instance has been supplied. Commit calculates the width- w sum, four condition bits, and $pc + 4$, then updates those locations. For an Ao load, evaluation also forms and checks the byte range. Failure selects the trap commit before the destination receives a value.

Aliased sources still use old values

Let the destination of `add r0, r0, r1` be the same location as its first source. The evaluation function is applied to the old state, so it returns the old values $S.R[r_0]$ and $S.R[r_1]$ before commit is called. Commit then replaces r_0 with their sum. No ordering of assignments inside a prose description can substitute the new r_0 for the captured first source. Therefore operand aliasing changes which old value is read, but it does not make the result depend on an accidental write order.

A failed evaluation cannot expose an ordinary write

Assume $E_i(S) = t$ for a trap reason t . By the definition of exec , the selected branch is $T_i(S, t)$, not $C_i(S, q)$ for any successful bundle q . The ordinary destination and sequential program-counter updates occur only in C_i . They therefore cannot appear in the trapped successor unless the trap rule itself explicitly declares them. This is a proof from the construction, not an appeal to how quickly hardware detects the failure.

The two proofs use the same boundary in different ways. The first protects old inputs from successful writes. The second protects the failure state from writes that belong only to success. A model that uses a mutable state record can implement both laws, but only if it avoids returning a partly changed record when evaluation fails.

Architectural atomicity has this bounded meaning: the specified observer sees the declared success state or the declared trap state, not an arbitrary prefix of the semantic updates. It does not by itself promise an atomic memory operation between threads, immunity from interruption, or absence of speculative internal work. Those claims require larger state and observation models.

3.4 The single-step relation

A running machine does not execute a mnemonic in isolation. It uses the program counter to fetch instruction bytes, decodes those bytes, and applies the decoded instruction's transition.

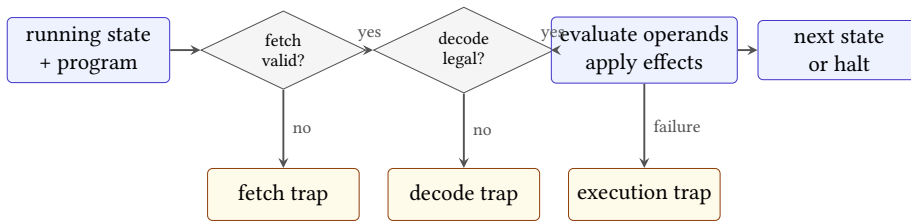


Figure 3.3: One Ao step keeps fetch failure, decode failure, and execution failure distinct. Only the successful path reaches the ordinary next state.

A0 single step

An Ao **single step** takes a running state and a program. It checks the program counter and fetch range, decodes one four-byte instruction, and applies that instruction's complete transition. A fetch or decoding failure produces a trapped state. A halted or trapped state has no successor.

Figure 3.3 is ordered. The machine cannot evaluate an operand from an instruction that did not decode. It cannot decode bytes that were not fetched from a valid code range. If operand evaluation traps, later writes in the instruction do not occur unless the instruction rule explicitly says otherwise. This order is part of the semantics, not an implementation optimization.

Three failures, three debugging questions

A fetch trap asks whether the program counter names available instruction bytes. A decode trap asks whether those bytes form one legal Ao encoding. An execution trap asks whether a legal decoded instruction can complete on this state. Collapsing all three into “bad instruction” hides which rule failed.

Object `A0.def.step` records this relation. The definition names decoding before Chapter 6 teaches the encoding because execution needs the boundary to exist. For hand traces in Part I, we will print both the four bytes and their decoded form so the decoding step is visible without becoming the lesson.

The ordinary next address is $pc + 4$ modulo 2^w . A branch or jump may replace it with a relative target. A trap records its reason and prevents a later instruction from running. These three cases are part of the transition, not annotations added after the register calculation.

Two simple attacks test whether a proposed step rule has hidden omissions. First, change the expected carry bit while leaving the destination correct. A complete-state check must reject the step. Second, leave the program counter unchanged after a normal add. A checker that looks only at r_0 will miss the error; a checker for the whole Ao transition must not.

3.4.1 Read and write footprints

The **read footprint** of an instruction on a state is the architectural information needed to determine its successor. The **write footprint** is the set of components that may differ in that successor. For `add r0, r1, r2`, the read footprint contains the two source registers, the program counter, and enough outcome state to require a running input. Its write footprint contains r_0 , the four condition bits, and the program counter.

Footprints are more exact than lists of printed operands and less cumbersome than repeating the complete transition. They expose data dependence: a later instruction depends on an earlier one when it reads a component the earlier one may write. They also support frame reasoning: everything outside the write footprint must retain its old value unless the failure rule says otherwise.

The footprint can depend on the state. A memory operand’s byte addresses come from register values. A conditional branch reads the predicate needed to choose one path, while the successor address depends on that choice. For a conservative static analysis, one may use a larger footprint that contains every possible address or component. That is safe for detecting possible dependence, but it may report interactions that no admitted state can realize.

Equal reads give equal deterministic effects

Fix one deterministic instruction instance. If two admitted input states agree on its complete read footprint, operand evaluation receives the same values and makes the same success or failure decisions. The instruction then calculates equal values for its write footprint. Components outside that footprint retain their respective old values. The full successors are equal only if the old states also agree on those preserved components.

The last sentence matters. Agreement on what an instruction reads is enough to make its newly calculated effects agree. It is not enough to make complete successor states equal when unrelated old state already differed. Chapter 11 will turn that distinction into observations and state relations.

3.5 Implicit and explicit operands

The two companion architectures make operand roles visible in different ways. A later RV64 integer addition will use a form such as `add rd, rs1, rs2`. The destination name is separate from the two source names. The selected base form does not write an x86-style condition-code register.

A later x86-64 register addition will use a two-operand form such as `add rax, rbx`. The destination `rax` is also the first source. The instruction reads the old values of `rax` and `rbx`, writes the sum to `rax`, and changes selected status flags.

One spelling therefore names three register roles with three operands. The other names three value roles with two operands and adds implicit condition writes. This is a concrete comparison of operand and state design. Calling one RISC and the other CISC may place the examples in historical families; it does not supply either transition rule.

The current RISC-V specification makes the base integer register-register rule unusually direct. `ADD rd, rs1, rs2` reads `rs1` and `rs2`, writes the low integer-register-width (XLEN) bits of their sum to `rd`, and ignores arithmetic overflow. The base form does not update a bank of arithmetic condition flags. The register fields also remain in consistent positions across the base formats, a choice the specification connects to simpler decoding. (RISC-V International 2026e)

The selected Intel rule differs in role as well as spelling. In 64-bit mode, a register-register `ADD r64, r64` reads the old destination and the source, writes the width-limited sum to the destination, and defines the documented arithmetic status flags. Other Intel `ADD` forms admit an immediate or one memory operand, but not two general memory operands. Prefix, encoding, mode, and exception details belong to the exact selected form, not to the mnemonic in isolation. (Intel Corporation 2026b)

Table 3.2: One numerical operation produces two different architectural contracts.

Question	RV64 base ADD	selected x86-64 register ADD
Printed roles	rd, rs1, rs2	destination/source, source
Old register reads	rs1, rs2	destination, source
Register write	rd	destination
Condition writes	none in the base form	documented arithmetic flags
Overflow result	low 64 bits	low 64 bits plus status effects
Control effect	next instruction address	next instruction address

Suppose all named source registers hold corresponding values and both destinations represent corresponding locations. The destination words can agree after addition. Complete states still do not have the same shape. A relation may ignore the x86-64 arithmetic flags, require them to agree with a derived predicate on the RV64 values, or relate them to extra Ao condition state. Each choice supports a different set of surrounding programs.

Here is the reader argument that keeps the extra state honest. Suppose two instructions start from the same state and both leave z in their destination. One leaves $Z = 0$, while the other leaves $Z = 1$. Their output states differ in the Z component. They may agree under an observation of the destination alone, but they are not equal state transformers. A following branch that reads Z can expose the difference.

The comparison also warns against translating syntax directly. To relate the two additions, we must first decide where the three input and output values live, then account for every extra state component. A destination-only observation may permit a relation that a flag-sensitive context would reject. The choice belongs in the claim, not in an informal declaration that both lines are additions.

Instruction forms also differ in where they place constants and addresses. An RV64 immediate form may carry a constant in fixed instruction fields and write a destination distinct from its source. An x86-64 form may allow an immediate or memory operand in a position whose other role can be a register. These are encoding and operand choices. The arithmetic relation can still be the same after widths, source values, destinations, and extra effects are aligned.

Implicit operands deserve the same treatment as implicit conditions. A stack operation may read and write a stack pointer that the assembly does not print. A procedure-return instruction may obtain its target from a fixed register or memory location. Some multiply and divide forms use architecturally selected register pairs. These choices can make a short line depend on substantial state.

Fewer printed operands do not mean fewer inputs

Assembly syntax is a notation optimized for programmers who know the architecture's conventions. It may omit a fixed register, a condition-code write, a stack update, or the ordinary next address. Semantic comparison must restore those facts before counting sources, destinations, or effects.

The comparison also has a physical shadow. A three-register form must identify two old source values and one destination even when the destination register number differs from both sources. A two-address form can reuse one printed field for an old source and the new destination. Those facts influence field allocation, register-file ports, rename information, and code size, but none determines a complete processor design alone. The semantic roles are the stable starting point from which such costs can be measured.

Read sets before arithmetic

For each of these Ao forms—`add r0,r1,r2`, `movi r0,-1`, `load r0,[r1+4]`, and `branch.eq +2`—list the complete read set and write set before calculating any values. Include implicit state and possible outcomes. Which two forms can trap after successful decoding?

3.6 Instruction sequences

If instruction i takes S_0 to a running state S_1 , and instruction j takes S_1 to S_2 , then the sequence $i; j$ has taken S_0 to S_2 . This is ordinary function composition with an outcome check between the functions.

If i halts or traps, there is no S_1 on which j may run. A model that applies j anyway has not found an unusual execution of Ao. It has defined another machine. This small point will become important when bounded execution distinguishes a stopped trace from one that merely reached its step limit.

Memory operands add a second source of interruption. Before an arithmetic operation can use a memory value, the machine must form the address and complete the load. If that access traps, the main arithmetic write does not occur. The order of these effects belongs in the instruction rule.

** Composition needs a running middle state**

Assume instruction i takes S_0 to a running state S_1 , and instruction j takes S_1 to S_2 . Then the two-instruction sequence is defined by applying j to the output of i . If i instead returns a halted or trapped state, the premise needed

to apply j is false. The sequence stops with the first outcome; it does not possess a hidden second transition.

The result is a partial form of composition. Each decoded instruction has a total rule over the running states admitted by its operands: it returns a next state or a trapped state. Program sequencing adds an outcome guard. Later trace definitions will make that guard visible at every adjacent pair.

When all required middle states are running, deterministic composition is associative. Grouping $(i; j); k$ or $i; (j; k)$ produces the same successive states because both groupings apply i , then j , then k . Parentheses organize the argument; they do not change the instruction order. The outcome guard must still be checked after each step.

Footprints explain when order can change. Two instructions with disjoint write footprints are not automatically interchangeable: one may write a component the other reads, both may trap under different premises, or their program-counter effects may select different code. For straight-line local reasoning, a sufficient commutation argument must show that neither write meets the other's reads or writes and that both outcome conditions remain the same in either order.

These footprint intersections are the semantic root of familiar instruction hazards. A read-after-write dependence carries a value forward. A write-after-read dependence can make changing the order destroy an old value before it is consumed. A write-after-write dependence makes final state depend on which write comes last. Hardware may execute internally in another schedule, but retirement must preserve the architectural result required by the machine model.

The statement is deliberately broader than registers. Condition bits, memory bytes, program counters, and outcomes can create the same dependencies. An optimizer that sees only named registers may move an addition across a flag-reading branch or move a load across an overlapping store. The assembly still looks plausible. The state transition has changed.

3.6.1 A sufficient commutation rule

We can state the safe straight-line case more carefully. Temporarily remove the ordinary program-counter update from each local effect and suppose both instructions are known to succeed. Let R_i, W_i and R_j, W_j be read and write footprints that remain valid in either order. Assume

$$W_i \cap (R_j \cup W_j) = \{\} \quad \text{and} \quad W_j \cap (R_i \cup W_i) = \{\}.$$

Then neither instruction changes a value read or written by the other. Their local effects commute.

Independent local effects commute

Take any architectural location ℓ . If neither instruction writes ℓ , both orders preserve its old value. If only i writes it, then j neither reads nor writes that location, and i 's reads are unchanged by j ; both orders leave i 's calculated value there. The case in which only j writes it is symmetric. The premises exclude a location written by both. Thus every location agrees after $i; j$ and $j; i$. Because success was assumed stable in either order, the outcomes agree as well.

The theorem uses stronger premises than “different destinations.” Let i write r_1 and let j read r_1 while writing r_2 . Their destinations differ, but switching them changes the value seen by j . Let two stores write overlapping bytes through different starting addresses. Their printed operands differ, but their write footprints intersect. Let two checked loads use disjoint destinations while one access can trap. Changing their order can change which trap is observed first.

The program counter needs special handling because two consecutive instructions normally occupy different addresses. We do not literally swap two whole step functions while retaining their old fetch addresses. An optimizer constructs a new program whose local data effects occur in a new order and whose control path is repaired. The theorem applies to those local effects; program equivalence must also relate the new instruction locations and final program counter. Chapter 11 will make that observation explicit.

Static tools rarely know the exact state. They use conservative footprints: a set guaranteed to contain every dynamic read or write for states under consideration. Enlarging a footprint can create a false dependence, but shrinking it without proof can miss a real one. This gives analysis a one-sided safety rule. Uncertainty costs optimization opportunities; an unsound omission can cost correctness.

A flag defeats a register-only order test

Suppose an x86-64 addition writes a destination register and arithmetic flags. A following conditional branch reads one of those flags. A tool that records only the printed register operands sees no register flow into the branch and might move another flag-writing instruction between them. The full footprints show a write-then-read dependence on condition state. The numerical addition can remain correct while control changes.

3.7 Instruction descriptions and tools

An instruction definition now serves a chain of tools. An assembler must know accepted operand forms and encodings. A disassembler must map bytes back to

a chosen notation. A compiler must know which operation patterns a form can realize, what operands it accepts, which state it uses and defines, and often how it is scheduled. An emulator needs executable effects. A debugger needs register and control consequences. A validator or formal model needs cases that are complete enough to reject a nearly correct transition.

These consumers overlap, but no single description automatically satisfies them all. A compiler pattern can be adequate for instruction selection while omitting exception detail required by an emulator. An assembler table can encode legal forms without stating their state transformation. A prose manual can be authoritative while remaining awkward to execute. The engineering problem is to keep several representations aligned with the same architectural contract.

The same form therefore appears through several projections. Let D be a complete architectural description. An assembler uses a projection concerned with accepted notation and encoding. A compiler uses one concerned with operations, operand constraints, effects, and cost. An emulator uses one concerned with executable state changes and failures. If $A(D)$, $C(D)$, and $E(D)$ are those projections, consistency does not mean that their files are identical. It means that they agree where their responsibilities overlap.

For example, if the assembler emits bytes for a register-register addition, the decoder used by the emulator must accept those bytes as the corresponding instance. If the compiler says a form defines condition state, its scheduling and live-value analyses, which track values still needed later, must protect later uses of that state. If an emulator says a memory form can trap before its destination write, a validator must not certify a trace that exposes the write on failure. These are commuting diagrams in practical clothing: taking either route across an overlapping contract must reach the same fact.

The table gives a review method. For each instruction-form change, list the consumers, identify the shared facts, and install at least one cross-boundary test. An assemble-then-decode round trip checks one connection. A decoded-step test checks another. A compiler-generated instruction replayed through an independent executor checks a third, provided the test compares the complete declared state rather than only the main result.

Round trips also have limits. Disassembling and reassembling can change an alias while preserving bytes or meaning. Assembling and disassembling through the same mistaken table can reproduce the same mistake. Independence must be named: a second implementation, an official example, a separately generated oracle, or a proof checked by a smaller trusted component. Repetition within one data pipeline is useful regression evidence, not automatic corroboration.

LLVM's TableGen documentation illustrates the scale of one such representation. Its expanded x86 addition record contains explicit input and output operands, assembly syntax, an instruction-selection pattern, encoding facts, and an implicit definition of EFLAGS, among other fields. The short source definition factors

Table 3.3: Different consumers need different projections of one instruction contract.

Consumer	Primary question	Omission that can break it
assembler	Which accepted form emits which bytes?	reserved or constrained field
disassembler	Which instance and spelling fit these bytes?	alias or instruction length
compiler	Which operation, operands, effects, and costs apply?	implicit use or definition
emulator	What complete successor or failure occurs?	frame, trap, or width rule
debugger	Which visible state changed and where does control continue?	implicit state or next address
validator	Which claim and boundary must be checked?	legality or preserved component

shared facts through classes because spelling every field for every instruction would be difficult to maintain. (LLVM Project 2026b)

GCC uses a different machine-description system. Its target files contain instruction patterns, operand constraints, RTL effects, and assembly output templates for the forms the compiler needs. Again, the mnemonic alone is not the useful unit. The compiler needs a relation between an internal operation, admitted operands, target effects, and emitted text. (GNU Compiler Collection Project 2026)

This supplies an honest economic unit. Adding one architectural form can require edits or generated cases across target descriptions, assemblers, disassemblers, emulators, debuggers, operating-system context rules, test corpora, documentation, and formal models. The cost can be counted as affected artifacts, validation cases, implementation area, or support time. It cannot be reduced to one universal dollar figure without a named organization and measurement.

Compatibility makes the cost persistent. A form used in shipped binaries may need support long after the product that introduced it disappears. Removing a form can strand software; changing an implicit effect can invalidate an optimizer or operating-system assumption. Conversely, regular fields and explicit effect descriptions can reduce duplication and make generated tools possible. Instruction-set design therefore allocates future maintenance work as well as present bits.

Security also enters through disagreement between consumers. If a validator, decoder, and processor partition byte strings differently, bytes checked as one instruction stream may execute as another. If a compiler omits an implicit destructive write, it may preserve a value the architecture permits the instruction to destroy. If an emulator commits a destination before a trap, replay diverges from hardware.

The useful control changes one layer at a time and checks that the disagreement is detected at its proper boundary.

Count the maintained contracts

The practical cost of an instruction form is not its mnemonic count. Count the operand cases, effects, encodings, tool consumers, validation controls, and years over which compatibility must be preserved.

3.7.1 Why the single-step boundary helps

The chapter's transition deliberately stops at one architectural instruction. That boundary includes every state component required to determine and record the step. It excludes the time taken, the internal control sequence, overlap with other instructions, and interactions with another processor. The exclusions are not claims that those things do not exist. They state what the present function is allowed to forget.

This is an instance of abstraction from mathematics and science. Choose a set of states, choose an input, and define a rule for the next state. Details may be removed only when their differences cannot alter the declared result. A pipeline stage, cache hit, or renamed physical register can be absent from the architectural state because the instruction rule does not inspect it. If a later question asks about timing or information leakage, the old abstraction may no longer be sufficient.

The boundary also resembles a controlled physical experiment. We prepare an initial condition, apply one intervention, and observe a result. Repeating the experiment over chosen states can expose a defect. A universal semantic claim is stronger: it covers every state in its stated domain. Testing, exhaustive finite enumeration, and symbolic proof offer different routes to that claim, each with a different scope and trust chain.

Four statements should therefore remain separate:

1. the printed rule determines one successor for every admitted input;
2. a hand calculation agrees with the rule for one chosen input;
3. an executor agrees with expected outputs for a tested collection; and
4. checked evidence establishes agreement for a quantified domain.

The first is a specification property. The second is reader evidence. The third is implementation evidence bounded by the test set. The fourth is a proof claim whose formula, assumptions, and checker must be named. None becomes another merely because all four discuss the same instruction.

A useful negative control preserves most of the visible result while changing one excluded possibility into an included error. The wrong-*pc* mutation keeps

arithmetic correct but violates declared control state. The illicit r_7 write keeps the state diff tempting but violates the frame. A reserved encoding keeps the mnemonic-looking fields but violates legality. Together the controls show why the one-step boundary contains more than the main calculation.

This small model will grow in controlled directions. Chapter 4 expands operand evaluation into byte-addressed memory. Chapter 5 repeats the step to form traces. Chapter 6 supplies the exact byte-to-instance map. Chapter 16 adds selected implementation and environment state. Each extension must say which old conclusions survive and which new observations can distinguish states that were formerly treated as the same.

3.8 Implementation boundary

An Axeyum implementation must join the state, decoder, operand rules, and instruction transitions into one reusable Ao step. Object `OP.a0.step` records that obligation. Its controls must include the wrong destination, a missing condition write, a wrong sequential program counter, a wrong branch target, reserved encoding fields, and attempted execution after halt or trap.

The live Axeyum checkout already contains useful lower layers. Its bit-vector terms can express width-limited arithmetic. Its solver and verification crates have transition-system interfaces and bounded and unbounded checking routes. Its LLVM work includes typed scalar-instruction parsing. None of those objects is an Ao machine. The checkout currently lacks the closed Ao instruction type, operand resolver, decoder, complete state executor, and public replay path required here.

That distinction changes the implementation plan. Ao should not begin as a Python dictionary of mnemonics mapped to callbacks. The Rust layer should own the closed operation and operand types, their legality checks, and the deterministic executor. A small PyO3 layer can then construct typed instances, run steps, encode states in the canonical interchange form, and display diffs. Solver formulas should be derived from or checked against that shared semantic core so a counterexample can replay through the same executor.

The first milestone is deliberately narrow: width-eight register addition with a complete state before and after, including flags, outcome, and program counter. The positive case must match the worked example in this chapter. Negative controls must mutate one destination bit, one flag, and the next program counter separately. A second milestone adds typed immediates. Fetch, decode, memory, and control transfer should enter only when each earlier boundary has a working failure control.

The corresponding introductory Axeyum route should begin with execution, not with a solver. A reader should construct one width-eight state and one decoded add, run the public step function, and inspect the complete output. Only then should a relation ask whether a proposed shortcut agrees for every allowed input. A returned counterexample must replay through that same step function. A no-

counterexample result must name the formula and checking route before it can support a machine-proof claim.

The public data model should keep the layers in Figure 3.1 separate. A decoded instance contains an operation and typed operands, not a closure supplied by Python. The Rust semantics matches that closed instruction type and returns a complete state result. Serialization records enough information to reconstruct both objects without accepting a printed mnemonic as authority.

One useful introductory display is a state diff paired with the full-state record. The diff teaches which components changed. The full record lets the checker establish preservation of everything else. If the artifact stores only the diff, a missing illicit write is indistinguishable from a preserved component.

Controls should attack one layer at a time. Change bytes while retaining the printed instance. Change an operand register while retaining the operation. Change one implicit condition while retaining the destination. Corrupt the next program counter while retaining all arithmetic. A route that rejects only the last mutation has not checked decoding or operand binding.

A useful first mutation

Change only the sequential update from $pc + 4$ to $pc + 1$. A test at $pc = 0$ catches the immediate error. A stronger route quantifies over every allowed running state and shows that destination arithmetic cannot conceal the wrong next address. The mutation is useful because it preserves the tempting part of the result.

The implementation does not accompany this draft. The worked addition is a reader execution of the printed Ao definition, not evidence that a general step function has run over every state. When the package exists, its output must agree with these rules and its controls must fail for the stated reasons.

Registers and immediates keep an instruction's values close at hand. A memory operand does not. It turns one step into a claim about addresses, byte order, valid ranges, overlap, and failure. We need to construct that part of the machine before we can safely compose programs that touch it. The next chapter therefore treats a memory access as a checked state movement, not as a bracketed spelling around an address.

3.9 Exercises

1. **Execute.** At width eight, run `add r0, r1, r2` from $R[r_1] = 127$, $R[r_2] = 1$, and $pc = 20$. Compute r_0 , Z , N , C , V , and the next pc .

2. **Break.** Alter exactly one component of your answer to the first exercise. State an observation that would miss the error and the smallest observation that would catch it.
3. **Prove.** Show that if a first instruction traps, the two-step Ao sequence has no state produced by the second instruction. Identify the outcome rule on which the argument depends.
4. **Transfer.** Write the value roles for the three-operand RV64 addition pattern and the two-operand x86-64 pattern. Do not compare their complexity; compare what each form reads and writes.
5. For `movi r3, -1`, state where the immediate originates and how it becomes a width- w word. Explain why it is part of the instruction instance but not a register in the input state.
6. Design a negative control for a step implementation that incorrectly advances pc by one byte instead of four. Give the starting pc , expected result, and bad result.
7. **Execute.** Give four legal instruction bytes and a matching decoded Ao add. Walk through every box in Figure 3.3, including the checks that succeed.
8. **Break.** Keep the decoded operation and all explicit operands correct, but omit one implicit write. Construct a following instruction that can expose the omission.
9. **Prove.** Show that two state transformers equal on every complete running state remain equal after composition with the same deterministic suffix, provided both produce running middle states.
10. **Transfer.** For one paired addition form, state the weakest observation under which it could refine Ao addition and one surrounding context that requires a stronger observation.
11. **Explain.** Give one assembly alias or pseudo-instruction from an architecture you know. Identify the assembly statement, encoded instruction, decoded instance, and transition. Which arrow is not one-to-one?
12. **Classify.** For each of these facts, say whether it belongs to fetch, decode, operand resolution, or commit: an unavailable code byte, a reserved function field, a bad data range, a sign-extended immediate, and a new zero flag.
13. **Design.** Give typed data constructors for Ao register, immediate, memory, and relative-target operands. State one invalid object that an integer tuple without role tags would admit.

14. **Prove.** Let two old states agree on all source locations for a fixed register addition but differ in an unrelated register. Prove that their calculated writes agree and their complete successor states differ.
15. **Break.** Write a faulty executor for `add r0, r0, r1` that updates r_0 before reading it. Choose width-eight inputs that distinguish the faulty rule from old-state evaluation.
16. **Execute.** Resolve the source value and destination location for `movi r3, -1` at widths 8 and 16. Show the bit pattern written in each case.
17. **Execute.** For `load r2, [r1+4]`, list in order the facts that evaluation must produce before successful commit. Do not calculate the loaded value until the address range is known to be valid.
18. **Break.** Construct a proposed state diff that contains the correct destination and flags but silently changes r_7 . State the frame law that rejects it.
19. **Prove.** Using the definition of E_i , C_i , and T_i , prove that a trapped load cannot expose its ordinary destination write under the Ao rule.
20. **Classify.** Explain why a legal load with a bad address is not the same as reserved bytes and neither is the same as a form omitted from a partial teaching model.
21. **History.** Compare one EDVAC order role with one System/360 operand format. Identify a continuity in purpose without claiming that the later format was a direct copy.
22. **Transfer.** Explain how a sequence of internal control actions can implement one architectural instruction without becoming several architectural instructions.
23. **Audit.** From the pinned RISC-V manual, record the explicit sources, destination, overflow rule, and condition-state effects of base integer ADD. Cite the section and revision.
24. **Audit.** From the pinned Intel manual, select exactly one 64-bit ADD form. Record its explicit operands, implicit effects, and exception boundary. Do not generalize from it to every form.
25. **Break.** Give two instructions with different destination registers that do not commute because one reads the other's destination. Calculate a concrete starting state and both final states.
26. **Break.** Give two instructions with disjoint register writes that do not commute because both affect condition state. State which premise of the commutation theorem fails.

27. **Prove.** Reproduce the location-by-location proof that two successful local effects commute under the chapter's footprint premises. Explain why the proof needs stable read sets.
28. **Design.** Extend the commutation claim to two loads that can trap. State additional premises that make the observed outcome independent of order.
29. **Analysis.** Give a dynamic memory footprint for one concrete base-plus-offset access and a conservative static footprint for a range of possible base values. Explain the precision cost of the latter.
30. **Tooling.** Inspect one official LLVM or GCC target description. List five fields or clauses beyond the mnemonic and explain which consumer uses each.
31. **Economics.** Suppose a new instruction form affects an assembler, disassembler, compiler backend, emulator, debugger, and formal model. Propose measurable maintenance units for a release. Do not invent a dollar estimate.
32. **Security.** Design a negative control in which a decoder and a validator disagree about one reserved field. State the dangerous false acceptance and the expected rejection.
33. **Axeyum.** Specify the input and output types for a first public Ao register-addition executor. Include enough state to detect a bad flag and a bad program-counter update.
34. **Axeyum.** Design three independent mutations for that executor and state why one combined failing test would provide weaker diagnostic evidence.
35. **Synthesis.** Write a complete transition contract for one small instruction not used in this chapter. Include legality, old-state reads, ordinary writes, frame, next program counter, and failure behavior.

Memory

Store the 32-bit word `0x12345678` at address 8. What does address 8 contain afterward? The answer is not the whole word. Ao memory gives each address one byte, so the store must distribute the four bytes across four addresses. To define that one instruction, we need an address, a width, a byte order, a valid range, and a rule for failure.

Memory makes order matter twice. The bytes of one value have an order across addresses, and separate accesses have an order through time.

It also makes absence observable. A register read always names one of Ao's eight registers. A memory access can form a word address for which some needed byte does not exist. The semantics must decide that failure before it can state what the instruction writes.

4.1 Memory as a finite byte map

A0 data memory

Ao **data memory** is a finite map from w -bit addresses to bytes. A load reads $w/8$ consecutive addresses and joins their bytes in little-endian order. A store splits a w -bit word and writes its bytes to consecutive addresses in that order.

Object `A0.def.memory` records the complete contract. This is data memory, not the immutable program map. An Ao store cannot overwrite instruction bytes in the first model.

4.1.1 Address separates place from meaning

Early calculating machines had storage, but not every store presented the same abstraction. A value might live in a physical register, a delay line, a drum position, or a location selected by wiring. The stored-program idea made numbered locations serve both instructions and data. Later machines varied the physical technology

while preserving a stable architectural question: what value does a load obtain from a named address?

The early technologies answered the physical question in strikingly different ways. A delay line kept pulses moving through a medium, so access was tied to when a pulse returned. A rotating drum tied access to a position passing a head. A cathode-ray store represented bits by charge patterns on a screen. A magnetic-core array selected small magnetic rings through intersecting wires. These mechanisms differed in retention, access pattern, capacity, speed, destructive reading, and manufacturing labor. No single physical picture is the definition of computer memory.

Tom Kilburn's 1947 storage report is especially useful because it begins from physical obligations rather than a modern array notation. A charge pattern on a cathode-ray tube could retain information for only a short time. The system therefore had to write, sense, and regenerate the pattern often enough to keep the represented digits available. Williams and Kilburn soon reported an experimental electronic computer built to test that storage principle. (Kilburn 1947; F. C. Williams and Kilburn 1948)

Regeneration exposes a truth hidden by $M(a) = b$. A physical state drifts. The system samples a region, decides which symbolic value it represents, and restores a region associated with that value. The abstract byte remains equal before and after refresh even though charges move and circuits act. Equality at the architectural level is therefore an observation relation over changing physical states, not physical immobility.

Magnetic core memory used another physical basis. Several researchers and organizations developed magnetic storage and selection methods, followed by patent disputes. Core planes were also manufactured through exacting manual work, much of it performed by women threading fine wires through small rings. The resulting architecture could present numbered, directly selected words while concealing both magnetization and labor from a load instruction. A history that names only a device inventor misses the production system that made the abstraction available. (Computer History Museum 2026a)

Semiconductor memory changed the scale again. Robert Dennard's 1967 filing described, among other embodiments, a cell with one field-effect transistor and one capacitor. A word line controls access; a bit line carries information to or from stored charge. The charge leaks, so the information must be regenerated periodically. The compact cell helped make high-density dynamic memory possible, but its physics did not abolish refresh. (Dennard 1968)

A stored bit is maintained, not frozen

A physical memory preserves a symbolic distinction by controlling changing matter. Charge leaks, magnetic regions must be selected and sensed, and signals need margins. The architectural byte map hides that work only for claims that do not depend on its timing, energy, reliability, or leakage.

This physical route supplies four operations beneath an abstract read. A cell or group of cells must be selected. Its physical condition must be sensed. A decoder must classify the sensed condition as bits. The result must sometimes be restored because sensing or time disturbed it. A store runs the relation in the other direction: symbolic bits control physical changes that establish a new represented region.

The representation is many-to-one. Many acceptable voltage or magnetic configurations can denote the same byte. Temperature, manufacturing variation, and noise can change the exact physical state while the decoded byte remains stable. If the state crosses a decision boundary, the abstraction fails and a reliability mechanism must correct, report, or expose the error. Chapter 16 will add those mechanisms; Ao assumes the byte supplied by its map is already the architectural value.

4.1.2 From physical cells to reliable bytes

A physical read can return a signal too weak or ambiguous to classify with the required confidence. A stored charge can decay; a particle or electrical disturbance can change a bit; a wire or cell can fail. The architectural map still promises a byte or a declared machine outcome. A real memory system therefore needs a policy for detecting, correcting, retrying, or reporting physical errors.

The simplest detection example is a parity bit. For data bits $d_0, \dots, d_{m-1}$, store an additional bit

$$p = d_0 \text{ xor } d_1 \text{ xor } \dots \text{ xor } d_{m-1}.$$

On a later read, compute the exclusive-or of the data bits again and compare it with p . Flipping one data bit changes the parity and is detected. Flipping two data bits can restore the old parity, so this code does not detect every corruption and does not identify which bit to repair.

Stronger error-correcting codes add enough check information to distinguish selected error patterns. Their guarantees always have a scope: number and kind of errors, codeword arrangement, and assumptions about independent or clustered faults. Saying that memory “has ECC” without that scope does not state what corruption is corrected, reported, or left undetected.

Correction also changes the evidence boundary. A program load can return the declared byte even though a lower layer detected and repaired a physical error. A reliability counter may change, an event may be logged, or an uncorrectable condition may become an architectural exception. Whether those effects belong to

the observed state depends on the claim. Ao observes only the returned byte and its own trap outcome.

This abstraction is economically important. More check bits consume capacity and transfer bandwidth. Encoding, checking, and repair consume circuits, energy, and time. Fewer protections can increase silent-corruption and service cost. The right comparison names a failure model, usable capacity, workload, and reliability target; it does not ask whether redundancy is simply good or bad.

That question contains two separations. First, an address names a place, not a type. The byte at address 8 does not announce whether it belongs to an integer, a character, a pointer, or an instruction. The operation that reads the byte supplies the width and interpretation. Second, architectural memory is not a photograph of the hardware. Caches, translation structures, buses, and storage cells may participate in an access without appearing as separate components in the instruction-set state.

Ao begins below both companion architectures with a finite byte map. The map is deliberately small enough to draw and complete enough to define loads, stores, overlap, and failure. It has no page table, cache, permission bit, memory-mapped device, or concurrent writer. Those omissions let us establish the sequential byte rules first. Chapter 16 will ask what must change when a later model admits the missing mechanisms.

The phrase *random access* also needs restraint. It says that an address can select a location without the program traversing every preceding location. It does not promise that every physical address takes the same time. Bank state, row state, translation, caches, contention, refresh, and devices can make timing vary while the architectural load result remains the same.

The durable abstraction is a numbered place

Memory technology has changed repeatedly. The architectural contract survives by naming locations and the effects of accessing them, rather than requiring software to describe the physical device that retains each bit. That abstraction is powerful precisely because it has a boundary: timing and concurrent visibility need additional rules.

An access of $n = w/8$ bytes beginning at address a is valid only when all addresses $a, a + 1, \dots, a + n - 1$ belong to the finite data-memory range. The first Ao model permits an access to begin at any address; it imposes no data alignment restriction. That is a design choice, not a claim about either companion architecture.

Address arithmetic still uses words. The machine forms a memory address by adding the sign-extended eight-bit offset to a base register modulo 2^w . It then checks the resulting sequence of byte addresses against the finite range. Wraparound arithmetic does not make an out-of-range byte valid.

Address formation and access validity are different rules

Word arithmetic always produces an effective-address word. A memory access succeeds only when every byte in its declared range exists. The first fact does not imply the second.

The requirement that every byte exist prevents a dangerous half-definition. If a four-byte access begins at a valid address but ends beyond memory, Ao does not improvise a shorter access. The requested width belongs to the instruction form. Either the whole access completes or the state records a trap.

4.1.3 Memory equality by address

Let $A_w = \{0, \dots, 2^w - 1\}$ be the word-valued address set and $B = \{0, \dots, 255\}$ the byte set. An Ao memory is a finite partial function whose finite domain is a subset of the address set:

$$\text{dom}(M) \subseteq A_w, \quad M : \text{dom}(M) \longrightarrow B.$$

Some address words may therefore have no data byte. The domain $\text{dom}(M)$ is the finite set on which the function returns a byte. This model distinguishes a stored zero from an absent location: $M(a) = 0$ is a value, while $a \notin \text{dom}(M)$ is a failed lookup.

That distinction is more than mathematical tidiness. A dense host-language array often fills allocated entries with zero. If an implementation silently uses the same zero for every absent address, a load outside Ao memory becomes indistinguishable from a successful load of stored zero. The trap outcome is lost before the instruction rule can inspect it.

Two finite memories are equal when they have the same domain and return the same byte at every address in that domain. Their construction history does not matter. One memory may have reached its present contents through a store; another may have begun with those bytes. If every lookup agrees, the Ao state cannot distinguish them through memory.

An update at an existing address a with byte b produces $M[a \leftarrow b]$, defined by

$$M[a \leftarrow b](d) = \begin{cases} b, & d = a, \\ M(d), & d \neq a. \end{cases}$$

For Ao, this update does not enlarge the domain. A store may replace bytes at present addresses; it may not allocate missing addresses as a side effect. Allocation belongs to a larger memory-management model.

A multi-byte update can be built by applying distinct single-address updates. Because no two byte positions of one valid non-repeating range name the same address, their order does not change the result. This construction gives an

implementation-independent meaning to a store even if a concrete executor copies a slice or updates a persistent map in another way.

Distinct single-byte updates commute

Let $a \neq d$. Compare $M[a \leftarrow x][d \leftarrow y]$ with $M[d \leftarrow y][a \leftarrow x]$ at an arbitrary address q . If $q = a$, both return x , because the update at d does not meet a . If $q = d$, both return y . At every other address, both return $M(q)$. The functions have the same domain and agree at each address, so they are equal.

The proof fails when the two updates name the same address and carry unequal bytes. Then the last update determines the value. This small fact is the algebra beneath overlapping stores later in the chapter.

This address-by-address definition gives memory proofs a reliable shape. To show that two stores produce equal memories, choose an arbitrary address and divide the argument according to whether it lies inside or outside each write footprint. To show that memories differ, one address with unequal bytes is enough. The first task is universal; the second needs a witness.

One byte separates two memories

Let memories M_1 and M_2 share the domain 0 through 15. Suppose they agree everywhere except address 9, where $M_1(9) = 00$ and $M_2(9) = 01$. An eight-bit load at 9 distinguishes them directly. A thirty-two-bit little-endian load at 8 also distinguishes them because the unequal byte receives weight 2^8 .

Preservation claims use the same definition. Saying that a store changes only its requested range means that every address outside that range returns its old byte. The phrase *everything else is unchanged* is therefore not a gesture toward the rest of memory. It abbreviates a quantified clause that a checker must test or derive.

4.2 Stores and loads

Return to 0×12345678 . Its little-endian split is 78, 56, 34, 12. A store at address 8 writes 78 at 8, 56 at 9, 34 at 10, and 12 at 11. A load from address 8 joins those four bytes with weights $2^0, 2^8, 2^{16}, 2^{24}$.

The word has not been numerically reversed. The low byte receives the low address. Endianness describes the relation between byte addresses and place values.

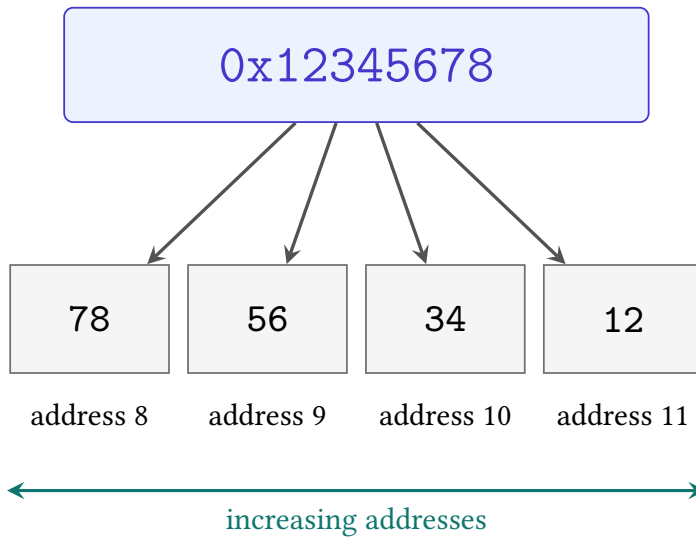


Figure 4.1: A little-endian store places the low-weight byte at the lowest address. The word’s printed hexadecimal order and memory’s address order run in opposite visual directions here.

4.2.1 Access rules give bytes a value

Memory stores bytes. A load instruction decides how many adjacent bytes to read and how to place them in its result. The four locations in Figure 4.2 therefore support several valid readings. An eight-bit load uses one byte. A sixteen-bit load uses the first two. A thirty-two-bit load uses all four. None changes the stored bytes.

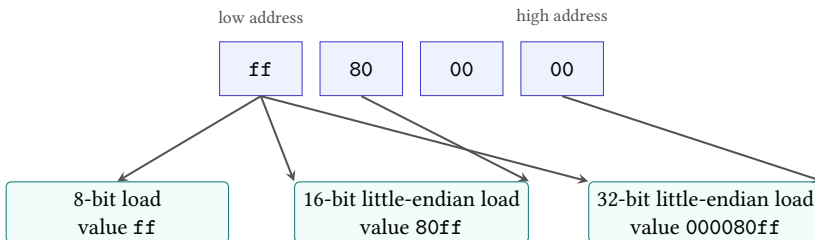


Figure 4.2: A byte sequence does not carry one compulsory width. The selected load form determines the range and the little-endian place values.

The destination width creates a further question. A narrow load into a wider register must say what fills the upper bits. Zero extension fills them with zero. Sign extension repeats the narrow value’s high bit. Loading the byte `ff` into a thirty-

two-bit destination can therefore produce 000000ff or ffffffff. Endianness alone chooses neither result.

Load interpretation

A load interpretation consists of a starting address, a byte count, a byte order, and a rule for placing the assembled value in the destination. The last rule includes any zero extension, sign extension, truncation, or preservation of destination bits.

This definition prevents a common shortcut. Saying that a machine is little-endian settles the weights of bytes within a multi-byte value. It does not settle the access width, destination width, signed extension, alignment, or failure behavior. Two instruction forms can read the same first byte and still produce different register values without disagreeing about byte order.

Stores reverse the construction but have their own width. A byte store writes one low byte even when its source register is wider. A word store writes the declared sequence of bytes. Unwritten neighboring bytes retain their old values. This preservation fact is the memory version of the frame conditions from Chapter 2, and later proofs will rely on it as heavily as on the bytes that change.

The round-trip proof now has a machine premise. Let x be a w -bit word and suppose every address in the $w/8$ -byte range beginning at a is valid. The store writes

$$b_i = (x \text{ div } 2^{8i}) \bmod 2^8$$

at address $a + i$. The following load reads that same b_i and computes

$$\sum_{i=0}^{w/8-1} b_i 2^{8i}.$$

Chapter 1 showed that these disjoint byte positions join to x . No other memory byte enters the sum. The load therefore returns x .

Store then load

The proof has three premises: the store and load use the same width, the same starting address, and a wholly valid range. The store writes byte b_i at $a + i$. The load reads that same location and multiplies the byte by 2^{8i} . Chapter 1's split/join identity reconstructs x . If any premise changes, the conclusion needs a new argument.

There is a reverse round trip. Load a valid n -byte range into a word and store that word back with the same width, address, and byte order. The load constructs $x = \sum_{i=0}^{n-1} M(a+i)2^{8i}$. Splitting x at byte position i returns $M(a+i)$. The store

therefore writes each old byte back to its own address. Its frame preserves every address outside the range, so the complete memory is unchanged.

Load then store preserves ordinary memory

Inside the valid range, uniqueness of base-256 digits makes the stored byte at $a + i$ equal to the byte just loaded from $a + i$. Outside the range, the store frame returns the old byte. The new memory agrees with the old memory at every domain address and has the same domain. Point-by-point equality gives equality of the complete maps.

The qualifier *ordinary memory* matters. A read from a device register may acknowledge an interrupt, consume queued data, or return a changing measurement. Writing the value back may issue a command. The reverse round trip is valid for the side-effect-free Ao map, not for every address that a real system can place in an address space.

The two directions prove different facts. Store then load recovers the word under a no-intervening-write premise. Load then store preserves the memory under a side-effect-free-access premise. A test suite should include both, because a matching pair of faulty split and join functions can pass the first while disagreeing with the declared byte order. A known byte-layout example breaks that shared mistake.

Notice what the proof does not require. It does not require the starting address to be a multiple of four, because the first Ao memory model permits data access whose start is not naturally aligned. It does not require unrelated bytes to have any particular value. It does require the store to be atomic on failure and no intervening write to change the loaded range.

The valid-range premise is load-bearing. Suppose a four-byte store begins at the last address of memory. Only its first target address exists. Ao traps and leaves data memory unchanged; it does not perform a one-byte fragment of the store. A later load cannot appeal to the round-trip argument because the required store never completed.

Partial failure changes the theorem

A faulty implementation might write each valid byte and trap only when it reaches the missing address. That route changes memory before reporting failure. It does not implement Ao's atomic store rule, and a later observation of the valid prefix can expose the difference.

Atomic here describes one transition in the sequential Ao semantics. It means that a failing store has no partly updated successor. It does not claim the stronger hardware property that another processor can never observe pieces of a successful

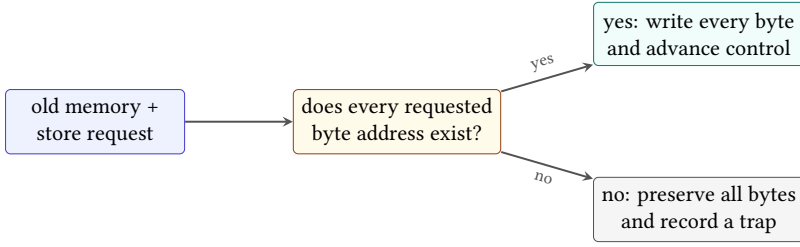


Figure 4.3: Ao checks the complete store range before selecting either a full memory update or a trapped state with memory preserved. There is no partial commit between them.

multi-byte store. Ao has no second processor or concurrent observer. Reusing the word outside that scope would quietly add a theorem the model cannot even state.

The success case has a frame rule. Let A be the set of addresses written by a valid store, let M be the old memory, and let M' be the result. For every address d outside A ,

$$M'(d) = M(d).$$

Inside A , the corresponding source byte determines $M'(d)$. Together, the inside and outside clauses define the entire new finite map. A checker that compares only the requested range verifies the first clause and misses corruption elsewhere.

The trap case has an even stronger frame rule: every memory address retains its old byte. Other state changes also require an explicit decision. Ao records the trapped outcome and does not advance into another instruction. The source and destination registers remain as the step rule states. A real architecture may record exception metadata in additional state; that state is outside Ao rather than implicitly preserved by it.

A control that reverses the weights

A faulty load joins the bytes from addresses 8 through 11 with address 8 at weight 2^{24} instead of 2^0 . It returns 0x78563412, not 0x12345678. The stored bytes are individually correct. The relation between address and weight is wrong.

4.3 Effective addresses and valid ranges

For the Ao operand $[r_1 - 4]$, the base is the word $R[r_1]$, and the offset is the sign extension of the immediate byte fc . The effective address is their width- w sum. Only after forming that word does the machine check whether the whole access range exists.

This order separates two ideas that are easy to blur. Word arithmetic always produces a word, even when it wraps. Memory validity is a predicate on the resulting addresses. If an eight-bit base contains 2 and the offset is -4, the effective address is 254. A four-byte access still traps in a memory whose valid range is 0 through 63.

The address expression also does not identify a fixed location by itself. Changing $R[r_1]$ changes the address denoted by $[r_1 - 4]$. Operand syntax and state work together to select bytes.

The valid range may itself wrap in word arithmetic. A width-eight access of four bytes starting at 254 names the word addresses 254, 255, 0, and 1 if we apply modular addition to each address. Ao does not infer validity from that cycle. Its finite memory domain must contain every address the access rule names. This makes the finite map, rather than the maximum word value, the authority for presence.

Nor must a finite domain be one uninterrupted interval. A model may contain addresses 0 through 31 and 48 through 63 while leaving a hole between them. A four-byte access beginning at 30 then finds its first two bytes and misses its last two. Checking that 30 and 33 fit within the minimum and maximum domain addresses would accept the access incorrectly. The semantics asks membership for each requested address.

That generality is useful even when an implementation later chooses a dense array. The mathematical interface describes which addresses exist. A dense representation can add a lemma that its bounds check is equivalent to per-address membership. A sparse representation can check the map directly. The theorem belongs to the interface; the faster check earns its place by establishing equivalence to that theorem.

Form first, check second

At width eight, evaluate base 250 with offsets 5, 6, and 10. For each result, state the effective-address word and then test a four-byte access against a memory domain containing addresses 0 through 255. Repeat with a domain containing only addresses 0 through 63.

4.3.1 Ranges as sets of byte addresses

For a non-wrapping access, it is convenient to write the requested range as the right-exclusive interval $[a, a + n)$. It contains a through $a + n - 1$ and contains exactly n addresses. Right-exclusive intervals compose cleanly: an access ending at b and one beginning at b are adjacent rather than overlapping.

The notation must not hide word wrap. If $a + n$ is calculated modulo 2^w , the two endpoints alone no longer reveal whether the access crossed the maximum word. A valid-range checker should enumerate the declared byte positions or

use an arithmetic condition that detects overflow before relying on an interval comparison. Testing only that the first and last words belong to a domain can also fail when the finite domain has a hole between them.

Alignment adds an independent predicate. A rule may require $a \bmod n = 0$, a weaker alignment, or none. Passing that test does not make the range present. Failing it may trap even when every byte exists. Thus a complete access rule has at least three stages: form the effective-address word, check address-policy predicates such as alignment, and check every byte needed by the transfer.

This separation becomes useful when relating machines. Ao permits an access without natural alignment that a selected real-ISA environment may reject. The real operation cannot refine the Ao operation on all Ao states without either restricting the input relation to aligned addresses or representing the additional failure in the abstract model. Matching successful examples does not repair the missing case.

4.3.2 Alignment as congruence

An n -byte access is naturally aligned when its starting address satisfies

$$a \equiv 0 \pmod{n}.$$

The condition partitions addresses into residue classes. For a four-byte access, addresses congruent to 0 modulo 4 are naturally aligned; the other three classes are not. Alignment is therefore a number-theoretic property of the start and width, not a visual property of hexadecimal notation.

When n is a power of two, the test has a bit form. If $n = 2^k$, natural alignment means the lowest k address bits are zero. A four-byte access needs two low zeros; an eight-byte access needs three. This equivalence follows because those low bits are exactly the remainder after division by 2^k .

Low zero bits characterize power-of-two alignment

Write $a = q2^k + r$ with $0 \leq r < 2^k$. The lowest k bits encode the unique remainder r . They are all zero exactly when $r = 0$, which is exactly when 2^k divides a . Thus the bit test and congruence test admit the same addresses.

Natural alignment does not guarantee that one lower-level transfer suffices. The memory hierarchy may use a line size, bank boundary, or page size different from the operand width. A naturally aligned eight-byte access can still cross a smaller modeled region boundary; a nonaligned access can remain inside one large line. Each boundary must be named before its crossing has a cost or fault consequence.

For pages of 2^k bytes, an n -byte range beginning at a stays in one page exactly when

$$(a \bmod 2^k) + n \leq 2^k,$$

provided the range itself does not wrap the word address space. This test uses the within-page offset, not natural operand alignment. It explains why a multi-byte access near the end of a page may require two translations even on an architecture that permits the address alignment.

4.4 Aliasing and dependence

Two memory accesses **alias** when their byte ranges overlap in the same state. Equality of their starting addresses is sufficient but not necessary. A four-byte access at a and another at $a + 2$ share addresses $a + 2$ and $a + 3$.

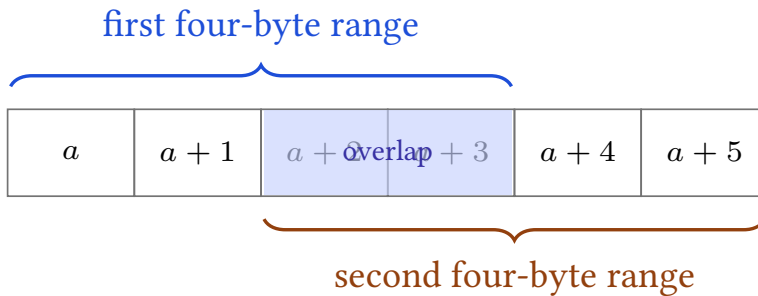


Figure 4.4: Equal starting addresses are not required for aliasing. These four-byte ranges share two addresses.

Let the first store write 0x11111111 at a , and let the second write 0x22222222 at $a + 2$. In that order, the bytes at a through $a + 5$ end as

11 11 22 22 22 22.

Reverse the stores and the middle two bytes become 11. The final memory differs even though each store, considered alone, writes the same source word.

Disjoint stores have a simpler argument. If their address ranges share no byte, the first store changes no address read or written by the second, and the second changes no address written by the first. At every address, either one designated store supplies the final byte or the original byte remains. The final data memories are therefore equal after either order, provided both accesses are valid and neither traps.

Overlap breaks the non-overlap step. A proof that exchanges memory operations must establish non-aliasing, account for the shared bytes, or accept a narrower claim. A register identity alone supplies none of these.

Why disjoint stores commute

Choose any address d . If d lies in the first range, non-overlap says the second store does not change it; the first store supplies the same final byte in either order. The symmetric case handles the second range. If d lies in neither range, both stores preserve its original byte. The cases cover the memory domain, so the final maps are equal. Validity is required so that both orders perform both complete stores.

The address-by-address proof explains exactly why the theorem stops at overlap. An address in both ranges has two possible final writers, and their order decides which one wins. We may recover commutation when the overlapping bytes happen to be equal, but that is a different premise and a narrower theorem.

4.4.1 Loads create data dependence

Place a load beside a store. If their ranges are disjoint and both operations succeed, moving the load before the store preserves the loaded value. The store changes no byte read by the load. If the ranges overlap, the loaded value may contain old bytes in one order and new bytes in the other. This is a memory dependence even when the instructions name different starting addresses or use different widths.

The proof again works address by address. For every byte position read by the load, non-aliasing says the store preserves that memory entry. The join rule therefore receives the same byte at every weight in both executions. Equal input bytes give an equal loaded word. The argument also needs the same outcome in both orders: moving an access across a trap can change whether the other access occurs at all.

Aliasing is state-dependent when addresses come from registers. The printed operands $[r_1]$ and $[r_2 + 4]$ neither guarantee overlap nor rule it out. A compiler may establish non-aliasing from an input contract, an allocation rule, or arithmetic facts about the registers. Without such evidence, treating different syntax as different memory is wishful typesetting.

Widths make the question asymmetric. A one-byte load inside a four-byte store range aliases the store, while a neighboring one-byte load may not. A proof must compare byte footprints rather than the source-language types or the number of registers written. This is why later instruction semantics will record read and write footprints separately.

Memory dependence follows byte footprints

Two accesses interact when the bytes read or written by one meet the bytes written by the other. Mnemonics, operand spellings, and unequal starting addresses are not enough to decide the question.

4.5 Address translation

Ao uses an address word directly as the key of its finite map. Most ordinary programs on current general-purpose systems use a further abstraction. The instruction forms a **virtual address**. Translation and protection state decide whether that address is admitted and, if so, which physical storage location or device it reaches.

This indirection has a history in the cost gap between storage technologies. The Atlas one-level storage system combined a smaller fast core store with a larger drum store. Hardware and supervisory software moved blocks and changed their placement so a programmer could work in one address system instead of manually scheduling every transfer. The published system described both the automatic mechanism and its effect on instruction time. (Kilburn et al. 1962)

Calling this merely “disk used as extra memory” hides the lasting idea. Translation separates a program’s name for a location from present physical placement. A modern operating system can use that separation to isolate processes, share selected pages, map files, delay physical allocation, protect code from writes, and place devices at controlled addresses. Moving an absent page in from storage is one possible policy response, not the definition of virtual memory.

4.5.1 Pages and translation

Suppose a model divides virtual and physical addresses into pages of 2^k bytes. Every virtual address v has a virtual page number and an offset:

$$v = q_v 2^k + o, \quad 0 \leq o < 2^k.$$

A page-table entry can relate q_v to a physical frame number q_p and carry permissions. When the entry is present and permits the requested access, translation produces

$$p = q_p 2^k + o.$$

The offset is preserved. The page number changes.

Take sixteen-byte pages, so $k = 4$. Virtual address 43 decomposes as page 2, offset 11. If virtual page 2 maps to physical frame 7, the physical address is $7 \cdot 16 + 11 = 123$. A four-byte load beginning at 43 requests virtual addresses 43 through 46. They remain inside virtual page 2, so one entry covers the range. A

four-byte load beginning at 46 crosses into virtual page 3 and must justify both translations and permissions.

Checked page translation

A checked translation takes a virtual address, an access kind, and translation state. It either returns a physical address with the required permission or returns a declared fault. A multi-byte access must justify every page and byte that its architecture requires.

Permissions are predicates, not decorations. A page may permit reading but not writing, or data access but not instruction fetch. Presence alone does not authorize the operation. Conversely, an absent entry does not prove that the program made an arithmetic mistake; it says that the present mapping does not supply the requested translation.

Two virtual pages can map to the same physical frame. Then different virtual addresses are aliases even though their numerical ranges are disjoint. A store through one mapping can change a later load through the other. The alias proof must therefore compare translated physical byte footprints, not virtual numbers alone.

The converse also occurs. Two processes can use the same virtual number while their translation roots map it to different physical frames. Equality of the printed address does not establish shared storage across address spaces. The translation context is part of the state needed to answer the question.

Virtual inequality, physical equality

With sixteen-byte pages, map virtual page 2 and virtual page 9 to physical frame 7. Virtual addresses 43 and 155 both have offset 11. Each translates to physical address 123. A byte store through either address is visible through the other, provided their permissions admit the accesses. Comparing only the virtual intervals would miss the alias.

4.5.2 Faults as transitions

If an entry is absent or its permissions reject the access, the processor reports an architectural fault according to the selected environment. System software may terminate the program, create a mapping, copy a shared page, bring data from storage, or take another documented action. Retrying can then produce a different result because the handler changed translation or memory state.

This sequence requires a larger machine than Ao. It includes privilege state, page tables, exception metadata, handler code, and possibly storage I/O. Ao collapses the boundary to one trapped outcome. That simplification supports the byte

proofs in this chapter, but it cannot prove a page-fault handler correct or measure the time required to service a fault.

Translation itself can require memory reads. Page-table entries reside in memory on the selected real systems, and implementations may cache recent translations. One program load can therefore cause implicit reads before the requested data byte is reached. The architectural result can remain one load transition even though the implementation performs several memory-system operations.

The current RISC-V privileged specification defines page-based schemes and distinct page-fault and access-fault outcomes. It also leaves selected priority questions between misalignment and translation faults to the implementation. Intel's system-programming volumes likewise define paging, protection, and page-fault behavior for named modes. A portable claim must use the selected environment's rule rather than inventing one universal order of checks. (RISC-V International 2026f; Intel Corporation 2026c)

Translation is not byte interpretation

Page translation decides which underlying location an address reaches and whether the access is allowed. Endianness decides how bytes in the admitted range contribute to a multi-byte value. Either rule can be correct while the other is wrong.

4.5.3 Memory-mapped devices

An address space can include device registers as well as ordinary memory. A load may receive a device status, consume input, or acknowledge an event. A store may start an operation. Repeating such an access is not automatically safe, and merging two writes can change external behavior.

This is why a range check is not a complete effect rule. The model also needs an address-region classification and the access behavior for that region. Ordinary Ao bytes are stable under repeated reads and change only through Ao stores. A device region may violate both assumptions. Optimizers, emulators, and proof tools must not apply ordinary-memory identities to it without an explicit side-effect or idempotence premise.

The distinction has an economic consequence. Treating every access as a device access would block useful caching and code motion. Treating a device as ordinary memory can lose events or issue commands twice. Region metadata and source-pinned rules allow tools to preserve correctness without discarding all optimization.

4.6 Memory in RV64 and x86-64

The later RV64 slice will use distinct load and store forms to move values between integer registers and memory. Arithmetic in the selected base forms uses registers. An effective address commonly combines one base register with an immediate.

In the base RV64 forms, the effective address is the value in `rs1` plus a sign-extended twelve-bit offset. `LB` and `LBU` read one byte and differ in sign versus zero extension. `LH`, `LW`, and `LD` read wider values in the RV64 environment, while matching stores take low source-register bits. A load whose destination is `x0` must still perform the access and raise required exceptions or device side effects; discarding the returned value does not erase the read. (RISC-V International 2026e)

The RISC-V specification for instructions available to ordinary, non-privileged software guarantees that naturally aligned loads and stores do not raise an address-misaligned exception. For misaligned accesses, the execution environment may provide invisible support or expose a defined failure. Even a successful misaligned access need not have the same atomicity guarantee as a naturally aligned one. Thus “RISC-V permits misalignment” is too broad; the environment and claim must be named.

The later x86-64 slice will include selected arithmetic forms that name a memory source or destination. Such a form can combine address calculation, memory access, and arithmetic in one decoded instruction. Its semantic rule must still expose the address, bytes, value width, state effects, and possible exception.

The selected x86-64 environment uses little-endian byte order for ordinary integer values and supplies several effective-address components through its memory-operand forms. Many ordinary scalar accesses can use addresses that are not naturally aligned, but particular instructions and enabled checks can add alignment requirements. A faithful rule must consult the selected instruction entry, mode, and system settings instead of turning common practice into a universal no-fault promise. (Intel Corporation 2026a; Intel Corporation 2026b)

This gives a more useful RISC/CISC comparison. RV64 base arithmetic keeps its ordinary operands in registers and uses explicit loads and stores for memory traffic. Selected x86-64 arithmetic can admit one memory operand. Both still need byte order, effective-address, translation, permission, alignment, and fault rules. The difference in instruction form changes where those obligations are combined; it does not make memory foundational on only one side.

This comparison holds one question fixed: which included instruction forms may access memory? It does not say that every instruction in one architecture has one character or that a shorter assembly spelling has fewer semantic obligations.

For either slice, a usable memory rule must answer the same checklist. It must name the address inputs, byte range, value construction, alignment premise, successful state change, and failure outcome. The answers may differ. The questions do not.

Table 4.1: Memory questions that syntax alone cannot answer.

Question	Required semantic fact
Where?	effective-address calculation and width
How much?	access width and byte range
Which value?	byte order and extension or truncation
What changes?	destination, memory, program counter, and other state
What can fail?	alignment, range, permission, or selected architectural outcome

Both real architectures live inside systems that add translation and access policy. A program commonly supplies a virtual address. Hardware and operating system state determine whether that address names storage, whether the access is permitted, and which exception follows a failure. Device mappings may give a read or write effects unlike ordinary memory. None of this turns the byte order question into a page-table question; it adds layers around the transfer.

The book's selected scalar slices will make those layers explicit by premise or exclusion. A theorem about an ordinary mapped region should say so. A decoder theorem need not model translation because it concerns bytes already supplied to decoding. A program theorem that loads from memory cannot borrow that simplification without naming how the bytes become available.

This discipline prevents an architectural manual from doing more work than it actually does. The manual specifies instruction behavior and exceptions in a declared mode. An application binary interface may add stack and object-layout conventions. An operating system supplies mappings and policies. A language implementation adds rules about valid objects and undefined behavior. A proof crossing these layers needs a relation between them, not a single citation to the lowest one.

4.7 The costs of memory

Saying that memory is expensive or slow is not yet an engineering claim. A memory system exposes several resources, and an improvement in one can worsen another:

- *capacity* is counted in bytes available at a named level;
- *latency* is time from a specified request to a specified response;
- *bandwidth* is bytes transferred per unit time under stated load;
- *transfer granularity* is the number of bytes moved or reserved for one transaction, line, burst, or page;

- *energy* is energy per access, per transferred bit, or over a named workload; and
- *occupancy and contention* describe how requests consume finite queues, banks, channels, and shared paths.

These quantities are not interchangeable. A system can have high bandwidth for a long stream and still impose large latency on one dependent load. A larger transfer can use available bandwidth efficiently while wasting energy and nearby storage when the program needs one byte. A larger memory can hold a working set while taking longer or more energy to access. Every quantitative claim must name the level, request pattern, unit, and platform.

A small expectation calculation shows why the request distribution matters. Suppose a named level answers in time t_h on a hit. With probability $1-p$ the request hits, while probability p requires an added miss cost t_m . Under this simplified model, the expected access time is

$$T = t_h + p t_m.$$

The equation is a weighted average, not a universal law of caches. It assumes the probabilities and costs are meaningful for the workload, that the added miss cost is measured consistently, and that overlap or queueing has not been silently ignored.

The sensitivity to miss probability is t_m : increasing p by one percentage point adds $0.01t_m$ to the expectation under the model. A rare event can therefore matter when its penalty is large. But an average still hides tails and dependence. A pointer-chasing program may wait for each load, while a streaming program can overlap several requests and approach a bandwidth limit. The same average latency does not predict both throughputs.

Name the unit before comparing

Let $t_h = 4$ ns, $t_m = 80$ ns, and $p = 0.05$ in the simplified expectation model. Compute T . Then double the number of independent outstanding requests without changing these three numbers. Explain why the equation alone cannot determine the new bytes-per-second throughput.

4.7.1 Locality and performance

Programs often reuse recently accessed locations or access neighbors. The first pattern is **temporal locality**; the second is **spatial locality**. A memory hierarchy exploits those regularities by retaining copies near the processor and transferring groups of neighboring bytes. The architectural map can stay unchanged while the time and energy of reaching a byte depend strongly on recent execution.

This is a new kind of state sufficiency. Ao state is sufficient to predict the next architectural value, but not its execution time. Two runs with equal Ao states can

have different cache and translation state and therefore different costs. A timing theorem must enlarge the state or quantify over the hidden conditions. Chapter 16 will construct that boundary explicitly.

The economic unit follows the workload. A compiler transformation may reduce instructions while increasing bytes transferred. A data layout may use more capacity to improve spatial locality. A service operator may pay for provisioned memory capacity, bandwidth, page-fault work, and energy through different parts of a system. “Faster” is not a conclusion until the measured resource and workload are fixed.

One useful byte can cause a larger transfer

Suppose a hierarchy moves a 64-byte line on a miss and the program uses one byte before evicting it. The useful-byte fraction is $1/64$. If the program soon uses all 64 bytes, the same transfer is fully used. The architectural load of the first byte is identical in both cases; the spatial history changes the economic value of the transfer.

Virtual memory adds other units. Page-table storage consumes bytes. A translation miss can cause several dependent memory reads. A page fault can consume handler time and data movement. Larger pages reduce the number of entries needed for a region but can increase internal waste and the amount of data tied to one mapping decision. These are tradeoffs, not arguments for one page size in every system.

4.7.2 Allocation, regions, and lifetimes

The architectural map says which byte addresses are present. An allocator answers a different question: which present ranges have been assigned to which uses? Let an allocation record contain a base address (b), a requested size (n), a reserved size (r), and a lifetime status. A successful request must choose a range that does not overlap any live allocation. It may reserve more than the requested (n) bytes to satisfy alignment or bookkeeping rules. The difference ($r-n$) is one form of **internal fragmentation**. It is capacity paid for but unavailable to the requesting object.

Free ranges can also be separated into pieces. Their total size may exceed a request while no single range is large enough or suitably aligned. That condition is **external fragmentation**. Thus “32 bytes are free” is not enough information to prove that one aligned 32-byte allocation can succeed. The locations, sizes, and alignment of the free ranges matter.

Allocation is therefore a transition over metadata as well as, sometimes, over bytes. If a request succeeds, a free range becomes live and a handle or pointer is returned. If it fails, the failure frame must say whether the byte map and allocation records remain unchanged. When an object is freed, its lifetime ends even if its old

bytes are not immediately cleared. A later allocation can reuse the same addresses. Equal numerical addresses at two times do not prove equal object identity.

This distinction explains why the Ao byte map is necessary but insufficient for a full language-memory theorem. Ao can prove that a checked store changes exactly a stated byte range. It cannot, without allocation metadata, prove that the range belongs to the intended live object. Adding that metadata also adds economic questions: time spent finding a range, bytes lost to fragmentation, bytes used by the allocator itself, and work required to clear, move, or reuse storage. Each question needs its own state and unit.

4.7.3 Validity and memory safety

Suppose a process has a readable page containing two adjacent allocated objects. A load can stay inside the mapped page yet cross from the first object into the second. Translation and permissions approve the bytes. The source-language or allocator contract may still reject the access. The ISA does not automatically know object bounds, lifetimes, ownership, or pointer provenance.

This separates three predicates:

$$\begin{aligned} & mapped(a, n), \\ & permitted(a, n, \text{read}), \\ & validObjectAccess(a, n, \text{program context}). \end{aligned}$$

The first says translations exist. The second says page-level policy admits the architectural read. The third belongs to a richer language and runtime model. The first two do not imply the third.

An access after an object is freed gives the temporal version. Its old virtual address may remain mapped and may even be reused for another object. Hardware can successfully return bytes while the program violates its lifetime rule. Conversely, a language-valid access can fault temporarily if the operating system must establish or restore a page mapping. The layers answer different questions.

Government product-security guidance now treats memory-safety defects as an industrial planning and maintenance concern, including migration, toolchain work, dependencies, and long-lived products. That consequence should not be misread as a claim that one language or hardware feature removes every memory risk. It shows that unchecked object access creates patching, incident, and transition work beyond the cost of the original instruction. (Cybersecurity and Infrastructure Security Agency et al. 2023)

Mapped does not mean safe

Page translation and permissions protect regions at their declared granularity. They do not by themselves prove that a byte belongs to the intended object, that the object is still alive, or that concurrent access is properly synchronized.

4.8 Implementation boundary

The Axyum state and memory package must implement the finite map, valid-range check, atomic load and store outcomes, little-endian split and join, wrapped effective-address calculation, and observations of changed and unchanged bytes. Object `OP.a0.state-memory` records that obligation.

The live Axyum checkout supplies pieces, not this package. Its bit-vector layer can express fixed-width addresses and byte extraction. Solver terms can represent arrays. Verification tests parse selected LLVM loads and stores and exercise checked-memory questions. Those facilities do not yet define Ao's finite domain, byte order, trap frame, canonical state record, or public executor. An SMT array that returns a value at every index is especially not the same as a finite partial map unless a separate presence predicate is modeled and checked.

The Rust core should own that distinction. One possible public shape pairs a canonical ordered map of address words to bytes with constructors that reject out-of-width addresses and values. A checked-range function returns either the exact ordered address vector or a trap reason. Loads and stores consume that result so no later loop can discover an absent byte after beginning an ordinary commit.

Canonical serialization must order entries by the unsigned address value and record the address width and explicit domain. Omitting a zero-valued entry would change the domain and therefore the state. A compact sparse encoding is allowed only when decoding reconstructs exactly which addresses are present, including those whose stored byte is zero.

The PyO3 layer should expose construction, checked access, and display without duplicating split, join, or range validity in Python. A reader can ask for the effective address, ordered footprint, old bytes, new bytes, and outcome. Those fields should be views of the Rust result. If Python independently computes them again, agreement between the display and itself provides no check on the semantic core.

The first milestone should use width eight and a domain small enough for exhaustion. It implements byte load/store, then two-byte and four-byte little-endian access, with holes admitted in the domain. Exhaustive tests can range over every width-eight source value and every starting address in a small fixed domain. Property tests should also vary sparse domains so the dense-array shortcut cannot pass unnoticed.

A later symbolic route can prove the split/join identity over bit vectors and the frame law over declared addresses. It must connect its presence predicate and byte variables to the executable map. A solver result about four isolated byte terms does not establish the executor's address arithmetic, range check, or trap commit. Replay and independent negative controls close those links.

Its negative controls are already suggested by the chapter. Reverse the byte weights. Permit a range whose last byte is absent. Partly modify memory before a

trapped store. Declare two overlapping ranges disjoint. Each mutation must be rejected by the corresponding check.

The introductory Axeyum route should make memory visible rather than burying it in a solver formula. A reader constructs a finite memory map, performs one store, inspects the changed bytes, and loads the word back. The route then asks for a counterexample to the round trip under the printed validity premise. Any model returned by the solver must replay through those executable store and load functions.

The controls should travel through the same route. Reversing the join weights must yield a concrete unequal-byte witness. Removing the last address from the domain must produce a trap and unchanged memory. A partial-write mutation must fail an observation of the valid prefix. These checks test different boundaries; one passing control cannot stand in for the others.

The artifact for one memory step should bind the complete old state, decoded instruction, effective address, requested footprint, outcome, and complete new state. A compact list of changed bytes is useful for display, but the checker must reconstruct the full successor and confirm the frame rule. Otherwise an artifact can omit an illicit change and still look like a plausible diff.

At the Python layer, the introductory exercise should expose values a reader can inspect without duplicating semantics. The reader supplies bytes and an initial state, calls the bound Rust implementation, and receives a structured step record. Python may print the address calculation and memory table. It must not compute the successor again with a second informal implementation and call agreement with itself a check.

The first solver question can stay finite and sharp: for every width-eight word and every valid four-byte starting address in a declared finite domain, does storing and then loading return the word while preserving every byte outside the range? A counterexample must replay through executable memory operations. A no-counterexample result needs an evidence route that covers the whole stated domain. Until that package exists in Axeyum, the object records an implementation obligation rather than a machine-backed result.

Why a memory formula is not yet memory semantics

A formula that expands four byte variables can establish a relation among those variables. It does not by itself establish address formation, finite range checking, atomic failure, or preservation of unrelated bytes. The Ao memory package must connect all of those rules before later instruction proofs may depend on it.

The store/load result in this chapter is a reader proof under a printed premise. It does not stand in for the absent executable memory model. Once that model

exists, the round trip and each bad variant must run through the same memory semantics used by later instruction proofs.

Memory has made sequence visible. Two individually valid stores can leave different final states when their order changes. A list of instructions still does not tell us which one runs next. For that we need a program counter, a program, and a trace through both.

4.9 Exercises

1. **Execute.** Store 0x89abcdef at address 4 in little-endian order. List the four writes and reconstruct the loaded word by hand.
2. **Break.** Reverse the byte weights in the load rule. Give the smallest two-byte word for which the faulty round trip differs from the original, and calculate both results.
3. **Prove.** Prove that two valid stores to disjoint byte ranges commute. State exactly where the argument fails for overlapping ranges.
4. **Transfer.** For one later RV64 load form and one selected x86-64 memory-source form, list the address inputs, value width, destination, and possible exceptional outcome that their pinned semantics must supply.
5. In eight-bit address arithmetic, compute the effective address for base 2 and offset -4. Explain separately why the word result exists and why a four-byte access may still be invalid.
6. Design an atomicity control for a four-byte store whose final address is outside memory. State which bytes a faulty partial store would change and what Ao requires instead.
7. **Execute.** Draw the two ranges in Figure 4.4. Perform the two example stores in both orders and list the final byte at every address from a through $a + 5$.
8. **Break.** Change Ao so a failing store writes its valid prefix. Give a pre-state and an observation that distinguish the faulty result from the required atomic trap.
9. **Prove.** Strengthen the disjoint-store argument to one load and one store: state a non-aliasing premise under which moving the load before the store preserves the loaded value.
10. **Transfer.** Explain why a real-ISA memory form needs both a source-pinned address rule and a source-pinned failure model before it can refine the Ao access, even when a sample address calculation matches.

11. **History.** Compare delay-line, cathode-ray, drum, and magnetic-core storage by selection method and retention. Explain why none supplies the architectural definition of a byte map by itself.
12. **Physics.** Draw the word line, access transistor, capacitor, and bit line of the chapter's dynamic-cell miniature. State what writing, sensing, leakage, and refresh do without claiming that one cell is one architectural byte.
13. **Explain.** Give two distinct physical states that could decode to the same stored bit. State the observation under which they are equal and one timing or reliability observation under which they may differ.
14. **Reliability.** Calculate the parity bit for 10110110. Flip one data bit and show detection. Then flip two bits and exhibit the limit of this parity rule.
15. **Economics.** For a memory with one check bit per eight data bits, calculate raw-bit overhead as a fraction of data capacity. List two costs not captured by that fraction.
16. **Prove.** Show that finite memories with the same domain are equal when they agree at every address. Explain why matching checksums would need an additional collision argument.
17. **Break.** Implement absent memory as an implicit zero. Give one successful zero load and one invalid load that the faulty representation cannot distinguish.
18. **Prove.** Reproduce the address-by-address proof that updates at two distinct addresses commute. Give unequal bytes that refute the conclusion when the addresses are equal.
19. **Execute.** Load four bytes, store the resulting word back with the same rules, and verify every address of the final memory, including two addresses outside the range.
20. **Break.** Construct split and join functions that agree with each other but both use big-endian weights. Show why their store/load round trip passes and why the fixed layout 0x12345678 rejects them for Ao.
21. **Prove.** For $n = 8$, prove that natural alignment is equivalent to the lowest three address bits being zero.
22. **Execute.** With sixteen-byte pages, decide whether four-byte accesses beginning at offsets 0, 12, 13, and 15 remain in one page. Keep this test separate from natural alignment.
23. **Design.** Give a finite memory domain with a hole for which the first and last requested addresses exist but a middle byte does not. Explain why endpoint checking fails.

24. **Transfer.** For one misaligned RV64 load, list the behaviors the pinned ordinary-instruction specification permits the execution environment to select. Do not infer multi-byte atomicity from successful completion.
25. **Audit.** Select one x86-64 scalar memory instruction from the pinned Intel manual. Record its effective-address inputs, width, alignment conditions, destination or source, and named exceptions for one mode.
26. **Execute.** With sixteen-byte pages, translate virtual address 43 when virtual page 2 maps to physical frame 7. Repeat for a four-byte range beginning at virtual address 46 and list every required virtual page.
27. **Break.** Map two different virtual pages to one physical frame. Construct two unequal virtual addresses that reach the same physical byte and show how a store through one affects a load through the other.
28. **Explain.** Give two processes the same virtual address but map it to different physical frames. State which additional state is needed to decide whether the addresses alias.
29. **Design.** Define read, write, and execute permission predicates for one page-table entry. Give one mapping that is present but rejects a store.
30. **Break.** Treat a device status register as ordinary memory. Give one read side effect that makes load-then-store preservation false and one optimization that would become unsafe.
31. **Analysis.** With $t_h = 4$ ns, $t_m = 80$ ns, and miss probabilities 0.01 and 0.05, compute the simplified expected access times. State two workload facts the averages omit.
32. **Economics.** A 64-byte transfer supplies 4 useful bytes for one layout and 48 for another. Compute each useful-byte fraction. Name the capacity and bandwidth costs separately.
33. **Security.** Place two language objects in one readable page and construct a load that is mapped and permitted but crosses the first object's bound. Identify which of the chapter's three predicates rejects it.
34. **Security.** Give an address that remains mapped after an object is freed and reused. Explain why page permission cannot establish the old object's lifetime.
35. **Axeyum.** Specify types for a finite Ao byte map, checked range, ordinary load result, store result, and trap. State how canonical serialization distinguishes absent addresses from stored zero bytes.

36. **Axeyum.** Design independent negative controls for reversed byte weights, a missing middle address, partial store on failure, and corruption outside the write footprint. Name the observation that catches each.
37. **Synthesis.** State a complete memory-access contract that includes effective address, translation context, permission, region kind, byte order, alignment, success effect, trap effect, and frame. Mark which clauses Ao omits by design.

Programs and Control

Write two instructions on adjacent lines. Why should the second run after the first? Position on the page is an assembly convention. The machine follows its program counter. An ordinary instruction advances that counter. A branch may replace the ordinary next address. A trap or halt permits no next step at all.

Control turns a list of instructions into a path through states. That path, not the layout of the listing, is the execution we will reason about.

A program listing is not an execution

A listing arranges instructions for a reader. An execution follows program counter values through complete states. Sequential layout matters only because the instruction semantics assigns it an address rule.

5.1 Programs and instruction bytes

A0 program

An Ao **program** is a finite immutable map of instruction bytes, together with an entry program counter and a declared code-address range. Every Ao instruction occupies four bytes and begins at an address divisible by four.

Object `A0.def.program` records this object. The data memory in machine state and the program's code map are separate. The first Ao model cannot change its own instructions by storing to data memory.

The program counter selects four bytes from the code map. A normal non-control instruction advances *pc* by four modulo 2^w . A fetch traps if *pc* is not divisible by four or if the four-byte range is incomplete. The fact that another instruction happens to be printed below the current one does not repair either failure.

The code range also prevents a valid four-byte pattern from becoming an instruction merely because the bytes exist somewhere. A fetch must begin at a

declared code address, satisfy the four-byte alignment rule, and find all four bytes. Decoding begins only after those conditions hold.

For now, a hand trace will show both bytes and decoded instructions. Chapter 6 will explain how the decoder obtains the instruction instance and why illegal encodings are part of program behavior.

The separation between code and data is a model decision, not a universal law of stored-program machines. Some systems permit stores to locations that can later be fetched as instructions. Loaders relocate code before execution. Run-time tools that join compiled units can select or patch call destinations. Just-in-time compilers create new instruction bytes while a process runs. Debuggers can insert a breakpoint instruction. Each case changes the state needed to predict a later fetch.

Ao begins with an immutable program map because that boundary supports a clean first theorem. Once the program object and initial state are fixed, every fetch reads the same four bytes at a given code address. A store changes data memory only. Trace replay can bind one digest of the program rather than track code writes between steps. These are genuine simplifications, and every proof that uses them must remain scoped to the first model.

Extending Ao with code that stores can change would require at least three choices. First, the state would contain the bytes from which future instructions are fetched. Second, a store would need rules governing whether and when a later fetch sees the change. Third, an evidence artifact would bind each fetched instruction to the code state at that step, not only to an initial program file. Real systems can add cache synchronization, permissions, and coherence obligations beyond those three. Chapter 16 will treat such additions as model extensions rather than silently importing them here.

The program map also differs from a source file and an executable file. Source syntax may expand into several instructions. Assembly labels must be assigned addresses. A tool that joins compiled units may place sections and resolve references. A loader may map selected file bytes into virtual addresses. Ao starts after those choices: its program is already a finite address-to-byte map with a declared entry. Later cross-layer claims will need to prove that the earlier artifact produced that map.

The same file does not imply the same fetched program

Execution depends on the bytes and addresses supplied to instruction fetch. File format, relocation, loading, permissions, and run-time code generation can stand between a named file and those bytes. A reproducible trace should bind the program representation used by its semantics.

This boundary has an economic side. Immutable code is easier to share across processes and easier to hash, cache, and audit. Generated or patched code can

specialize a program to current information, but it adds compilation work, memory use, synchronization, and another validation surface. Neither policy is always superior. The proof obligation follows the chosen life cycle of the instruction bytes.

5.1.1 Stored programs and data-dependent control

Before stored programs, changing a machine's task could mean rewiring, reconfiguring switches, or replacing an external sequence of control media. Placing instructions in addressable storage made the next operation depend on a small part of machine state: the program counter. Conditional transfer then made that counter depend on values produced during the calculation.

The historical gain was not merely convenience. A finite body of code could repeat, select among cases, and call reusable routines. The same bytes could process inputs of many sizes because the path and number of visits were chosen while the program ran. A short program became a description of a potentially long computation.

That foundation remains visible today beneath compilers, operating systems, interpreters, and virtual machines. Optimizers rearrange control while trying to preserve observed behavior. Security mechanisms restrict indirect targets. Debuggers reconstruct paths from partial evidence. Formal verification asks whether every path from an allowed entry respects a contract. Each task begins with the same modest question: which successor states does the program admit?

The idea has more than one historical root. In 1936, Alan Turing described a mathematical machine by a finite table. A row related a machine configuration and scanned symbol to an action, a written symbol, a movement, and another configuration. The table was finite even when the symbolic process it governed did not stop. This was not yet a stored-program architecture or a modern instruction set. It supplied a foundational separation that this chapter still uses: the description of a transition rule can be finite while the sequence generated by repeated application has no fixed length. (Turing 1937b)

Stored-program work in the 1940s joined a different set of practical problems. Instructions had to be represented, placed, selected, and sequenced in an electronic machine. Conditional transfer let a result affect which instruction came next. Program preparation therefore required more than laying arithmetic operations in a row. It required explicit paths, addresses, tests, and joins. No one document or machine owns that entire development; mathematical models, wartime machines, circulated designs, engineering teams, and later working systems contributed different parts.

Goldstine and von Neumann's 1947 planning report used flow diagrams to expose those paths before assigning detailed machine orders. A diagram could show a box of work, a test, and the routes that rejoined or repeated. That notation did not replace the machine's program counter. It gave people a level at which to reason about control before committing every address and instruction. (Goldstine and Neumann 1947)

The distinction between a path description and a machine trace remains load-bearing. A flow diagram or control-flow graph usually includes several possible edges. One execution follows edges selected by one initial state and the values later computed from it. The graph organizes possibilities; the trace records one realized history. Confusing them turns a drawing into false evidence that every edge can occur, or turns one test run into false evidence that an edge not selected in that run cannot occur.

By the 1960s, researchers were asking how control structure could support proof. Böhm and Jacopini studied transformations of flow diagrams and languages formed from composition and iteration. The result became part of a larger structured-programming discussion. It did not show that machine jumps were obsolete, nor that every structured rewrite preserved cost or readable state. Its lasting relevance here is narrower: sequence, selection, and iteration are sufficient building forms for describing broad classes of control while the underlying machine still executes local transfers. (Böhm and Jacopini 1966)

Floyd's 1967 method placed assertions at selected points in a flowchart. The proof had to show that each admitted path carried the assertion at its source to the assertion at its destination. A global claim about all executions was thus reduced to local preservation obligations plus an entry argument. Hoare's 1969 axiomatic account organized related reasoning around a condition required before execution, a program fragment, and a condition promised afterward. Both works made a program more than an object to run: it became an object about which consequences could be derived. (Floyd 1967; Hoare 1969)

These histories meet in the transition relation used by Ao. Turing's finite table motivates a finite rule with potentially unbounded use. Stored-program machines make the selected operation depend on an addressed program and a program counter. Flow diagrams and structured control organize possible paths. Inductive assertions turn visits to chosen control points into proof obligations. Ao is not a reconstruction of any one historical system. It is a small meeting place where those distinctions can be stated without hiding them behind a compiler or operating system.

A finite description can govern unbounded time

A program map is finite, but execution may revisit its addresses without a fixed limit. The mystery is not an infinite program hidden in memory. It is iteration: a finite transition rule applied again to each new state.

5.2 Execution traces

A0 execution trace

An A0 **execution trace** is a finite sequence

$$S_0, S_1, \dots, S_k$$

in which each adjacent pair is linked by one A0 step. A bounded runner reports whether the last state halted, trapped, exhausted the step bound, or remains able to continue.

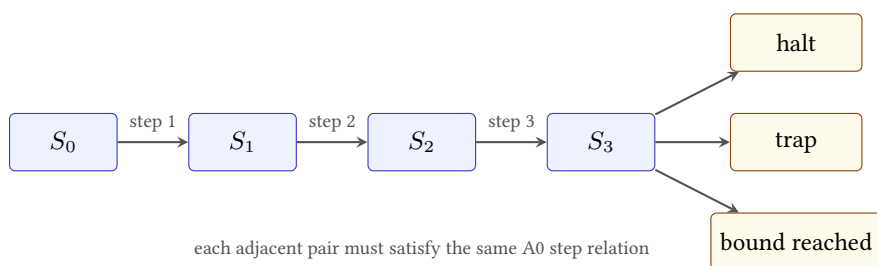


Figure 5.1: A bounded trace is a sequence of checked adjacent steps. Reaching a bound is distinct from reaching a terminal machine outcome.

The diagram's three exits do not mean that one state has three outcomes. A particular run produces one result. The fan-out records the cases the runner must distinguish. If the last state is running when the requested bound is used up, the runner reports that fact without changing the machine outcome to halted or trapped.

Object `A0.def.trace` records this relation. A trace contains its initial state. A three-instruction execution that finishes normally therefore contains four states: the state before the first step and one state after each step.

Consider this width-eight decoded program, with instructions at addresses 0, 4, 8, and 12:

```

movi r1, 5
movi r2, 3
add  r0, r1, r2
halt

```

Listing 5.1: A short A0 program

Begin with $pc = 0$, a running outcome, and arbitrary old values in the three registers. The first step writes 5 to r_1 and sets $pc = 4$. The second writes 3 to r_2 and sets $pc = 8$. The third writes 8 to r_0 , sets $Z = 0$, $N = 0$, $C = 0$, $V = 0$, and

sets $pc = 12$. The fourth changes the outcome to halted. It preserves the registers, memory, program counter, and conditions. There is no fifth step.

The complete four-instruction trace

The trace contains five states, not four. S_0 is the input. S_1 has $r_1 = 5$ and $pc = 4$. S_2 also has $r_2 = 3$ and $pc = 8$. S_3 also has $r_0 = 8$, cleared conditions, and $pc = 12$. S_4 changes only the outcome to halted. Every omitted component is preserved at each named step.

Writing the states rather than only the instruction outputs exposes two common counting errors. The initial state belongs to the trace, and halt is an executed transition rather than the absence of one. The statement “four instructions ran” therefore corresponds to four edges and five vertices.

This trace is straight-line because every non-halt step uses its ordinary next address. The definition still comes from the program counter. That distinction will let a branch choose another path without changing what a trace is.

5.2.1 From one step to many

The instruction semantics defines one edge $S \rightarrow S'$. Program theorems usually concern zero or more edges. Write $S \rightarrow^k T$ when there is a trace of exactly k steps from S to T . The zero-step case relates every state to itself. The successor case composes one step with a k -step execution.

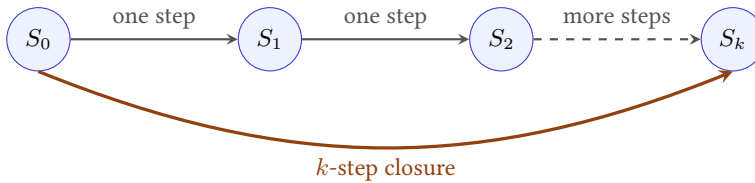


Figure 5.2: Multi-step execution is built from checked adjacent one-step transitions. It does not introduce another instruction semantics.

This definition supplies a composition law. If $S \rightarrow^m T$ and $T \rightarrow^n U$, joining the traces gives $S \rightarrow^{m+n} U$. The shared state T appears once in the combined state sequence even though it closes the first trace and opens the second.

Concatenating execution traces

Take the state sequence from S to T and append the second sequence after omitting its first copy of T . Every adjacent pair comes from one of the original traces, including the join because both use exactly the same state T . The edge count is $m + n$, while the state count is $m + n + 1$.

The same construction can be stated as relation algebra. Let R be the one-step relation. Define R^0 as the identity relation and $R^{k+1} = R^k; R$, where the semicolon means relational composition. Then $S R^k T$ says that some k -edge trace begins at S and ends at T . The relation for any finite number of steps is

$$R^* = \bigcup_{k \geq 0} R^k.$$

The resulting relation is the **reflexive transitive closure** of R . It is reflexive because zero steps leave every state where it began. It is transitive because two finite traces with a shared endpoint can be joined.

The zero-step case is not a technical nuisance. It makes a state reachable from itself, lets an empty program fragment serve as the identity for composition, and gives induction a base case. If a theorem means one or more steps, it should use a different relation, often written R^+ . Silently using R^* can make an alleged progress theorem true by choosing zero steps.

Trace concatenation is associative at the level of endpoints and edges. Given traces of lengths a , b , and c , either order of joining produces a trace of length $a + b + c$. The state sequences also match when each shared endpoint is retained once. The proof does not depend on determinism; it depends only on exact equality at the joins. Determinism becomes relevant when asking whether another trace of the same length could follow different states.

Suppose the one-step relation is deterministic on running states:

$$S \rightarrow T_1 \wedge S \rightarrow T_2 \implies T_1 = T_2.$$

Induction on k then proves that two k -step traces from the same state have the same endpoint and the same state at every index. For the induction step, determinism makes their first successors equal; the induction hypothesis applies to the remaining k steps. This is an *at most one* result. It does not manufacture a first successor for a halted, trapped, or invalid state.

That distinction is the difference between a partial function and a total function. Ao stepping can be presented as a partial function on states if terminal states have no result. It can instead be a total function returning either a successor or a declared no-step reason. Both interfaces can describe the same machine, but their theorem statements differ. The book uses an explicit outcome so that a trace checker cannot mistake “no successor” for missing software behavior.

Prefix and suffix reasoning follow from the same algebra. If $S \rightarrow^k T$, every $j \leq k$ selects a unique prefix endpoint in a deterministic run. The remaining segment has length $k - j$. Conversely, a claimed midpoint U is useful only after proving both $S \rightarrow^j U$ and $U \rightarrow^{k-j} T$. A matching program counter alone is insufficient because the registers, memory, conditions, and outcome at the join may differ.

5.3 Branches and control flow

Ao separates comparison from conditional control. The instruction `cmp r1, r2` computes the width- w subtraction $R[r_1] - R[r_2]$ to set Z, N, C, V ; it does not write the subtraction result to a register. A later conditional branch reads selected condition bits.

Suppose r_1 and r_2 both contain 5. After `cmp r1, r2`, $Z = 1$. The instruction `branch.eq 2` then takes its relative target:

$$pc + 4 + 4 \cdot 2.$$

If the branch itself begins at address 4, the target is 16. If the two registers differ, $Z = 0$, and the same branch advances to address 8.

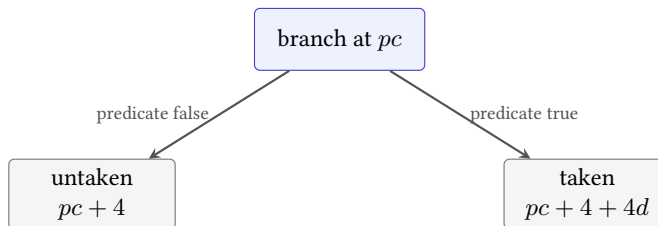


Figure 5.3: An Ao conditional branch chooses between the sequential address and a target measured from that sequential address.

The offset is relative to the sequential address, not to the current address. A faulty rule that uses $pc + 4 \cdot 2$ lands at 12 in this example. That near miss is useful: the wrong trace still reaches an aligned instruction, so a test must check the target rule rather than merely alignment.

The displacement d is a signed eight-bit value before extension to the word width. Multiplication by four scales instruction units to byte addresses. Adding it to $pc + 4$ makes displacement zero name the next instruction, not the branch itself. A backward branch uses a negative displacement. All three steps—sign extension, scaling, and choice of base—belong to the target rule.

Taken, not taken, and backward

Place a branch at address 20. Compute both successors for displacements 0, 2, and -3. For each displacement, separate the sequential address from the taken target before deciding which predicate value chooses it.

The paired architectures place the predicate differently. A later RV64 conditional branch will compare register values as part of the branch instruction. A common selected x86-64 pattern will have one instruction set condition flags and a later conditional jump read them. Ao deliberately uses the second broad shape. The comparison dimension is where the branch predicate lives, not which family receives the simpler label.

5.3.1 Control-flow graphs

A **basic block** is a sequence entered at its first instruction and left only at its end under the chosen graph construction. Ordinary the sequential successor keeps execution inside the block. A branch, jump, halt, trap, or declared entry point creates a boundary. The exact partition depends on which targets and failures the analysis includes.

A control-flow graph places these blocks or instruction addresses at nodes and adds possible successor edges. For a direct conditional branch, the local graph usually includes the sequential edge and taken target even if one predicate is impossible under the program's input premise. A trace selects one of those edges in one state. A state invariant can later remove an infeasible edge.

The distinctions prevent three common overclaims:

1. absence from sampled traces does not make a graph edge unreachable;
2. presence in a syntactic graph does not prove that an allowed state can take the edge; and
3. local reachability of an address does not prove that fetching and decoding there succeeds.

Reachability begins at the declared entry state or entry set. Inductively, the entry is reachable, and a successor of a reachable running state is reachable. This definition is semantic because it mentions states and steps. Graph reachability over a conservative edge set can include extra paths unless every edge has also been shown feasible.

Control point

A control point is an address or phase at which a proof states a relation or invariant. It need not be every instruction boundary. Loop heads, procedure entries, call returns, and final exits are common choices. The proof must still justify the intervening steps between consecutive control points.

Control points let two machines take different numbers of instructions while supporting the same higher-level argument. They also keep loop invariants readable: the invariant can describe the state at the head rather than every temporary state inside the body.

5.3.2 Reachability as a fixed point

Let E be a set of allowed entry states. For any state set X , define

$$F(X) = E \cup \{T \mid \exists S \in X, S \rightarrow T\}.$$

The first application contains entries and their immediate successors. Applying F again adds states two steps away, and continued application adds longer finite paths. The reachable set is the least set R satisfying $F(R) = R$. It contains the entries, is closed under a valid successor, and contains nothing not forced by those two requirements.

The word *least* prevents an easy but useless answer. The set of every mathematically possible Ao state is also closed under successors, but it says nothing about which states the program can reach from its entry. Inductive invariants often work in the opposite direction: choose a convenient set I that contains E and is closed under successors. Leastness then gives $R \subseteq I$. If no state in I is bad, no reachable state is bad.

This explains why an invariant can safely be an approximation. It need not equal the reachable set. It may include states that no concrete input can produce, provided the desired property holds throughout the larger set. A weaker approximation is easier to establish but may include a bad-looking state and fail to prove the goal. A more precise invariant can exclude that state at the cost of carrying more relationships.

Graph reachability applies the same closure to nodes and syntactic edges. It is usually cheaper because it forgets register and memory values. That forgetting is also why it admits paths whose branch predicates cannot all be satisfied together. Semantic reachability restores complete states. Symbolic execution keeps formulas describing families of such states. Abstract interpretation keeps a chosen summary. Each method spends a different amount of work to retain or discard information.

Control-flow graphs also reveal joins. A node d **dominates** a node n relative to an entry when every graph path from that entry to n passes through d . The entry dominates every graph-reachable node, and a node dominates itself. A loop

header commonly dominates the loop body when every iteration enters through that header. This makes the header a natural place for an invariant.

Dominance is a structural claim about the selected graph. It is not by itself a semantic theorem that the node is reachable. An unreachable node can have dominators under some graph conventions. Nor does dominance prove that two incoming machine states agree. At a join, values from different predecessor paths may differ; a compiler can use a merge operation, while a proof can use a disjunction or a relation that covers both cases.

Graph path versus feasible path

Draw a diamond with one test on $x = 0$ and, inside its true arm, a second test on $x \neq 0$. The syntactic graph contains the second test's true edge. Show that no state entering through the first true edge can take it. Then give an invariant at the inner test that removes the infeasible edge.

5.4 Four outcomes of bounded execution

A halted trace has executed `halt`. A trapped trace has encountered a declared failure, such as an invalid fetch or memory range. Both have a final state with no successor, but one records normal completion and the other records failure.

A third run may consume its requested step bound while its last state is still running. The runner has stopped looking; the program has not necessarily stopped. This is **bound exhaustion**. A fourth result can return a running state before a larger calling procedure decides whether to take another step. Neither result proves termination or looping.

The difference matters even for a one-instruction loop. Run an unconditional branch back to itself for one hundred steps. The bounded trace contains one hundred valid transitions and a running final state. It is evidence about that prefix. To show that execution never halts requires an argument about every later step, such as an invariant that the program counter always returns to the same instruction and no state component can cause another path.

A finite state description can thus give rise to an unbounded question. The machine is small; time need not be.

A runner contract can make the cases mechanical. Given a nonnegative bound b , begin with a trace containing only S_0 . Before each requested step, inspect the outcome. If it is halted or trapped, return that terminal result without adding a transition. If it is running and fewer than b steps have been taken, fetch and execute exactly one checked successor and append it. If exactly b transitions have been taken and the last state is still running, return bound exhaustion.

At bound zero, this algorithm performs no instruction. A halted input state is reported halted, a trapped input state is reported trapped, and a running input state exhausts the zero-step request. This boundary case prevents an implementation

Table 5.1: A bounded runner classifies evidence without changing machine state.

Result	Required final fact	What it establishes
Halted	outcome is halted	a halt transition or halted input was reached
Trapped	outcome is trapped	a declared failure transition or trapped input was reached
Bound exhausted	outcome is running and exactly b transitions were checked	one valid prefix of length b
Prefix returned	outcome may still be running and fewer than b steps were requested by the caller	the explicitly returned prefix only

from fetching once before checking its budget. It also keeps the trace law exact: a result reporting k transitions contains $k + 1$ states.

The runner's stopping decision is metadata about the observation procedure. It must not be written into the architectural outcome unless the modeled machine itself has a timer instruction or interrupt that performs such a transition. Keeping those layers separate lets the same Ao trace be checked by tools with different user-interface limits.

A bound is a resource limit, not a semantic verdict

Reaching 1,000 valid steps establishes a 1,000-step prefix. It does not show that the program halts at step 1,001, runs forever, or eventually traps. Each of those conclusions needs evidence beyond the exhausted run.

A loop proof supplies that further evidence by identifying an invariant. For the unconditional self-branch, the program counter returns to the same address, the branch preserves every other state component, and the outcome remains running. Applying the same rule again recreates the same state. An induction over the number of steps then covers every finite prefix. The proof uses the transition rule, not the patience of the runner.

5.4.1 Invariants and variants

An invariant is a predicate intended to hold whenever execution reaches a chosen control point. The proof has two obligations. Initialization shows that the entry path establishes the predicate. Preservation assumes it at one visit, executes the loop segment, and shows it again at the next visit. By induction, the invariant holds at every reached visit.

The invariant need not describe all state. It should carry exactly the facts needed for the next iteration and final theorem: counter bounds, address validity, preserved memory, relationships among registers, or a condition connecting machine words to mathematical values. A weak invariant may be true but unable to prove safety. An unnecessarily strong one may fail because it mentions scratch state that legitimately changes.

Loop-invariant induction

Let $I(S)$ be the invariant at loop head h . Prove that the entry execution reaches a state S_0 with $pc = h$ and $I(S_0)$. Next, for an arbitrary reachable S with $pc = h$ and $I(S)$, prove that one loop iteration either exits under the intended final condition or reaches a new head state S' satisfying $I(S')$. Induction covers every finite number of completed iterations.

This establishes preservation properties but not necessarily termination. A **variant** adds a value in a well-founded order, commonly a natural number, that strictly decreases on every returning iteration. If the body cannot get stuck and the variant cannot decrease forever, execution must eventually leave the loop.

Safety and termination can therefore split. An infinite self-branch has the invariant that the complete state repeats, which proves it never reaches bad state under the model. It has no decreasing natural variant and does not terminate. A countdown loop can have both an invariant keeping its counter in range and a variant equal to the remaining count.

The exit theorem uses the invariant plus the negated continuation condition. For example, if an invariant states $0 \leq r \leq n$ and the loop continues exactly while $r > 0$, reaching the ordinary exit yields $r = 0$. The control predicate is part of the mathematical conclusion, not merely a jump detail.

A complete countdown argument

Consider a loop segment entered with a word r that represents a natural number. At the head, it exits when $r = 0$. Otherwise the body replaces r with $r - 1$ and returns to the head. Assume the body cannot trap and preserves the code map and all state required to fetch the head again.

Let the input satisfy $0 \leq r = n < 2^w$. Use

$$I(r) \iff 0 \leq r \leq n$$

as the invariant and $V(r) = r$ as the variant. Initialization follows from $r = n$. On a continuing iteration, the guard gives $r > 0$. Therefore $r - 1$ is still a natural number, satisfies $0 \leq r - 1 < n$, and cannot wrap modulo 2^w . The invariant is preserved and the variant strictly decreases.

The natural numbers under strict decrease are well founded: no infinite sequence $v_0 > v_1 > v_2 > \dots$ exists. Hence the loop cannot return to its head forever. The no-trap premise rules out a failed body, so it eventually takes the ordinary exit. At that exit, the negated guard gives $r = 0$. Together, these facts prove termination and the final counter value.

Every premise in that proof earns its place. Removing the test $r > 0$ before subtraction permits a zero word to wrap to $2^w - 1$, so mathematical subtraction no longer matches the word operation used in the variant argument. Removing the no-trap premise permits execution to end before the claimed exit. Removing preservation of the program and head address permits the next visit to fetch another instruction. A decreasing register value alone is not a complete machine-level termination proof.

The choice of the natural numbers is also substantive. An integer-valued quantity can decrease forever:

$$0, -1, -2, -3, \dots$$

so the statement that the counter decreases is insufficient unless the proof supplies a lower bound or maps the state into another well-founded set. Some loops need a tuple of measures. Under an ordered-pair rule, (a, b) decreases when a falls, or when a stays fixed and b falls. The theorem must define that order and show that every returning path decreases it.

Variants establish more than eventual exit when the decrease is quantitative. If V begins at n , remains a natural number, and falls by at least one per iteration, there can be at most n returning iterations. Add the fixed instruction count for entry, each body path, and exit to obtain a step bound. That conversion needs care when different paths through the body have different lengths. The largest permitted path cost supplies a safe upper bound; a measured average does not.

A word is not automatically a natural variant

A machine register always contains a width- w bit pattern. A termination proof must state the mathematical interpretation of that pattern and prove that each update remains in the well-founded range. Modular decrease at zero wraps and does not support the ordinary countdown proof.

5.4.2 Partial and total correctness

Suppose the four-instruction example has the final condition $r_0 = 8$. We can ask two different questions. First: whenever the program reaches its normal exit, does the final state satisfy that condition? Second: from every allowed initial state, does the program actually reach that exit? The first question concerns **partial correctness**. The second question, joined to the first, gives **total correctness**.

For the straight-line example, the distinction is easy to miss. Fetch and decode succeed at four consecutive addresses, each instruction has one successor, and the fourth instruction halts. The same trace that establishes the result also establishes arrival at the result. Loops separate the two obligations. An invariant may say exactly what would be true at the exit while providing no reason that the exit will ever be reached.

Write $\{P\} p \{Q\}$ for a partial-correctness judgment. In this chapter it means: for every initial state satisfying P , every normal terminal state reached by program p satisfies Q . This definition does not promise that a normal terminal state is reached. It also needs an explicit policy for traps; otherwise a program that traps on every input can satisfy any condition stated only for normal exit.

A total-correctness judgment adds existence of a normal terminal trace:

$$\begin{aligned} P(S) \implies \exists k, T, S \rightarrow_p^k T, \\ \text{outcome}(T) = \text{halted} \quad \wedge \quad Q(T). \end{aligned}$$

For deterministic Ao, the terminal trace is unique when it exists. For a machine that permits multiple successors, total correctness commonly requires every admitted execution to terminate in an acceptable state. The quantifier choice must be printed because existential and universal termination are different claims.

Sequential composition illustrates the value of an intermediate assertion. If p establishes M from P , and q establishes Q from M , then the concatenated program can establish Q from P . Machine-level use of this rule also requires the exit of p to transfer to the entry of q , with every component expected by q represented in M . A source-level semicolon does not prove the program-counter join.

For a conditional with predicate B , prove one arm under $P \wedge B$ and the other under $P \wedge \neg B$. Both arms must establish the same final condition, or establish separate facts whose disjunction implies it. At the ISA level, B must match the actual branch predicate. Replacing an unsigned condition with a signed one can make both arm proofs locally correct for the wrong partition of inputs.

For a loop, partial correctness follows from initialization, preservation, and exit:

$$\begin{aligned} P \implies I, \quad \{I \wedge B\} \text{ body } \{I\}, \\ I \wedge \neg B \implies Q. \end{aligned}$$

Total correctness adds a well-founded variant and a no-failure obligation for each continuing body path. These are not decorative rules imposed on code. They are compressed forms of induction over the traces defined earlier.

A vacuous final condition and its repair

Let p be an unconditional self-branch that never traps and never halts. The partial claim that every halted result of p has $r_0 = 8$ is true because there are no halted results. The total claim is false. A useful specification can instead state that every reachable state remains running at the loop address

and preserves r_0 . That is a genuine safety property of the infinite behavior, not a disguised completion claim.

A true implication may say nothing about arrival

The implication *if the program halts, then $r_0 = 8$* is true for a program that never halts and never writes 8. Partial correctness is useful, but it must not be read as a termination claim. To obtain total correctness, add a termination argument under the same initial-state premise.

Traps require another separation. A contract may permit them, rule them out, or assign them their own final condition. If normal exit and trap are both terminal outcomes, a claim about halted states alone says nothing about trapped states. A robust program theorem therefore partitions the possible outcomes before stating what must hold in each part.

A bounded runner does not collapse this distinction. If the bound is reached, the runner has established a finite prefix. A sufficiently large bound may cover every run only when another argument supplies a uniform upper bound on execution length. That bound is a theorem premise or conclusion external to the step relation; it is not hidden inside the number entered at the command line.

This vocabulary will matter when later chapters compare two programs. One may produce the right value whenever it exits but diverge on an input where the other program halts. Agreement on final register values then fails to capture the difference in behavior.

5.5 Replacement in context

Suppose programs p and q leave the same value in r_0 for every allowed initial state. Can one replace the other before any suffix r ?

No. Let p also leave $r_3 = 0$, while q leaves $r_3 = 1$. They agree under an observation of r_0 . Choose a suffix that copies r_3 into the final output. The suffix exposes the difference that the earlier observation discarded.

The same counterexample works with condition state. Two prefixes may agree on registers while setting Z differently. A suffix beginning with `branch.eq` can choose different paths. Agreement on an output before the branch is not enough.

The repair is exact. Suppose p and q , from every allowed initial state, finish with the same outcome and agree on every state component that the deterministic suffix can read. The first suffix instruction then receives indistinguishable inputs. It produces matching values for the components needed by the second instruction, and the same argument repeats through the suffix. The final states agree under the declared observation.

The argument is a reader proof of a contextual rule, not yet a general theorem about all programs. Its premises need a precise read set, termination account, and observation. Later chapters will formalize those conditions as equivalence and refinement. For now, the counterexample has done its job: local replacement depends on context unless the state relation covers what the context can see.

Replacement through a deterministic suffix

Let two prefixes finish in running states that agree on every component the first suffix instruction can read. Determinism gives matching relevant outputs after that instruction. Repeat the argument for each later suffix instruction. If both executions reach the end under the same outcomes, their final states agree under the declared observation. The proof fails when a suffix reads a component on which the prefix states disagree.

The premise must follow the path actually taken. A suffix containing a branch may read one set of components on one path and another set on a second path. Agreement sufficient to choose the same first branch can simplify the rest of the proof. Agreement only at the final output of the prefixes cannot.

We can describe the required agreement with a projection π . The projection retains the components a context may observe: perhaps selected registers and the outcome, or perhaps registers, conditions, reachable memory, and program bytes. If $\pi(S_p) = \pi(S_q)$ after the two prefixes, replacement is sound only when every permitted context treats states with the same projection alike. Choosing π is therefore part of the claim, not a formatting choice made after the executions have been compared.

Call a suffix C **compatible** with π when equal projections at its entry lead either to matching declared failures or to final states equal under the chosen observation O . Then the replacement rule has a short form:

$$\pi(T_p) = \pi(T_q) \quad \wedge \quad C \text{ is compatible with } \pi \quad \implies \quad O(C(T_p)) = O(C(T_q)).$$

This notation abbreviates several proof obligations. Both prefixes must reach their stated entry states under the same input relation. The suffix must be legal from both states. If it can branch or loop, compatibility covers every admitted path rather than only one example. Terminal outcomes and traps must be compared according to the declared observation.

One sufficient way to prove compatibility is an instruction-by-instruction simulation. Establish that equal π -projections choose matching first steps and produce states related by a new projection. Repeat until the suffix terminates. A loop needs an invariant relating the two executions. A suffix with different step counts may need selected control points rather than one-to-one instruction pairing. Chapter 11 will generalize these cases.

A context can observe more than its final arithmetic result. It can branch on a condition, load from an address changed by the prefix, encounter a fault on one path, or use an indirect target obtained from a register. In a machine with code that stores can change, it might even fetch different instruction bytes. A proof that forgets any such input licenses a suffix designed to reveal the forgotten difference.

Closed-program agreement can be weaker

Two complete programs may agree under a final-output observation even though their intermediate states differ. That can be enough when nothing will be appended and no external observer sees the intermediate states. Replacement inside an arbitrary context is stronger: the context becomes an observer and may turn a hidden difference into a visible one.

The useful projection is rarely the whole physical machine. It is the state named by the model and exposed to the admitted contexts. Ao excludes caches, timing, interrupts, and self-modifying code, so an Ao contextual result cannot settle whether two real implementations leak different timing information. Conversely, retaining every Ao register when the suffix reads only one may be safe but needlessly strict. Later equivalence proofs will balance these two errors: forgetting an observable component and preserving irrelevant detail.

A context exposes one forgotten bit

Prefix p and prefix q both leave $r_0 = 7$, but they leave different values of Z . Append a suffix that branches to write 0 or 1 into r_0 according to Z . The composed programs now disagree on their visible output. The suffix did not create the earlier difference; it converted hidden state into an observed register.

5.6 Control contracts and costs

Ao gives every ordinary instruction one fixed four-byte length and gives its conditional branch a compare-then-jump shape. Neither companion architecture has exactly that contract. The comparison must name the source of the predicate, the base used for the target, the immediate range and scale, the state written on a jump, and the rules applied at the destination.

In the selected RV64I slice, a conditional branch such as `beq x5, x6, L` compares two registers inside the branch instruction. Its B-type immediate represents a signed displacement in multiples of two bytes and reaches a target within about four kibibytes of the branch address. The target base is the address of the branch, not the following instruction. A taken target that violates the active instruction-

Table 5.2: Selected control-transfer dimensions in the three machines.

Question	A0	RV64I slice	x86-64 slice
Predicate	stored condition bits	registers in branch	status flags for Jcc
Relative base	next address	branch address	next instruction
Length	four bytes	four-byte base forms	variable
Indirect source	future extension	register plus immediate	register or memory
Other write	none	optional link register	form-dependent

alignment rule can raise an exception; an branch not taken does not fetch that target merely because it was encoded. (RISC-V International 2026e)

The RV64I `jal` instruction is an unconditional direct jump that also writes the address of the following instruction to its destination register. Writing `x0` discards that link and makes the form an ordinary jump. Its signed displacement has a wider range than a conditional branch. `jalr` is indirect: it adds a signed immediate to a register value, clears the least significant bit of the result, writes a link when requested, and transfers to the resulting target. These rules make two common shortcuts wrong. A jump target is not always obtained from the program counter, and a control transfer can write architectural data as well as the program counter.

For a selected x86-64 near conditional jump, the predicate comes from status flags established by an earlier instruction. A relative displacement is added to the address of the instruction following the jump. Because x86-64 instructions have variable length, that base cannot be replaced by a fixed $pc + 4$ rule. A selected indirect `JMP r/m64` obtains its target from a register or memory operand rather than a displacement embedded as a complete direct target. The exact form, mode, operand size, canonical-address rules, and fault conditions must accompany any theorem. (Intel Corporation 2026a; Intel Corporation 2026b)

These differences do not make one family foundational and the other exceptional. They expose the parameters hidden by the broad word *branch*. A reusable control theorem should receive those parameters through a machine definition or a proved refinement relation.

5.6.1 Graphs, probabilities, and profiles

Compiler intermediate representations group instructions into basic blocks and require a terminator to describe how control leaves each block. LLVM, for example, defines a basic block as a single-entry, single-exit instruction sequence whose final instruction is a terminator. This is an intermediate representation contract, not a

proof that every listed successor is feasible for every source-language input.(LLVM Project 2026e)

An optimizer may attach an estimated or measured probability to outgoing edges. That probability answers a workload question: how often was or is an edge expected to be selected under a named distribution? Reachability answers an existence question: can some allowed state select it? Safety answers a universal question: do all reachable selections preserve a property? None of the three numbers or predicates substitutes for the others.

Profiles also carry economic consequences. Placing frequent paths together can improve instruction-fetch locality. Converting a hard-to-predict branch to another form can reduce lost work on a particular processor. Duplicating a block may simplify a hot path while increasing code size and instruction-cache pressure. A claim that a control rewrite is faster must therefore name the workload, processor, measurement unit, and code-size effect. Ao proves the architectural path, not those timing claims.

Testing adds another view. Statement or edge coverage records which selected items appeared in a finite test set. It can expose an entirely untested path, but full syntactic edge coverage does not prove all data properties and cannot show that no failing state exists on a covered edge. Conversely, an infeasible edge can prevent a naive coverage target from reaching 100 percent. Coverage is evidence about tests relative to an instrumentation and graph, not a replacement for reachability or proof.

Path counts grow quickly. If b independent binary decisions are all feasible, there are 2^b decision sequences before loops are considered. Ten decisions admit 1,024 sequences; thirty admit more than one billion. Dependencies can remove paths, and loops can add unbounded families. Symbolic execution, abstract interpretation, bounded checking, and proof invariants manage that structure in different ways. They do not make the combinatorial question disappear.

5.6.2 Timeouts and watchdogs

Services, embedded controllers, and batch systems often impose time or step limits. A caller may cancel work after a deadline. A watchdog may reset a component that fails to report progress. A transaction system may retry an operation. These mechanisms can protect availability, but each adds state and outcomes beyond the Ao program.

A timeout result means that the observer's resource policy expired. It does not prove that the program would never have completed. A reset changes the machine state and may discard evidence. A retry can repeat external effects unless the operation is idempotent or carries a transaction identifier. Thus an operational progress claim must name the clock, scheduler, interrupt or reset action, persistence rules, and externally visible effects.

Worst-case execution bounds are stronger than measurements of typical runs. To turn the countdown proof into a bound, one needs an upper bound on the

number of iterations and on the cost of every admitted iteration path under a named cost model. Caches, prediction, interrupts, and shared resources can invalidate a bound derived only from architectural instruction count. Chapter 16 will enlarge the state for those questions.

5.6.3 Indirect control and security

A direct branch carries its destination family in the instruction bytes. An indirect jump or return obtains a target from mutable state. If an attacker can corrupt that state, the instruction may redirect execution to code that was never an intended successor even though every fetched byte is executable. Control-flow integrity mechanisms constrain that target relation.

The ratified RISC-V control-flow-integrity extensions separate forward-edge and backward-edge protection. Zicfilp can require an indirect call or jump to land on a declared landing-pad instruction. Zicfiss supplies protected shadow-stack support for return addresses. Intel Control-flow Enforcement Technology similarly combines indirect-branch tracking with a shadow stack. These features require supported hardware, enabled policy, and cooperating software; their instruction bytes alone do not prove that protection is active. (RISC-V International 2026a; Intel Corporation 2020)

The foundational lesson is not a catalog of mitigations. It is that the set of targets computable from the instruction syntax can be larger than the set allowed by a security policy. A complete transition system either includes the policy state and its failure outcome or states that it models a mode in which the mechanism is absent.

5.7 Implementation boundary

An Axyum runner for Ao must fetch, decode, and step through the program while keeping halt, trap, bound exhaustion, and continued execution distinct. Object `OP.a0.run` records that obligation.

The live Axyum checkout already contains related machinery, but not this runner. Its solver defines a symbolic `TransitionSystem` interface with initial-state, transition, and bad-state predicates. Its bounded model checker returns a replay-checked reachable model, a statement limited to the searched bound, or an explicit unknown result. The verification crate adapts scalar loops to that interface, reflects selected MIR and LLVM control-flow graphs, checks bounded loops, and records branch decisions for one form of control-flow constant-time analysis.

Those capabilities do not define Ao program bytes, fetch, decode, complete Ao state, halt and trap transitions, or the four-way runner result in this chapter. The general scalar-loop adapter also stutters after a guard becomes false so that bad-state queries remain meaningful at later depths. That is a useful model-checking convention, but it is not Ao's rule that halted or trapped states have no successor.

Reusing the substrate requires an explicit adapter and proof of the correspondence, not a type-name match.

The first implementation milestone should therefore remain concrete. Define a canonical Ao program object and a complete width-eight initial state. Implement one checked step in Rust. Build the bounded runner by repeatedly calling that step, never by asking a solver to guess the next concrete state. Only after the executable trace route is stable should a symbolic transition system encode the same rules for bounded reachability questions.

The introductory Axeyum run should print a trace a person can compare with the book. Each row needs the step number, program counter, decoded instruction, changed components, and outcome. A replay command should consume the same initial state and program bytes, not a second handwritten formula that merely resembles them.

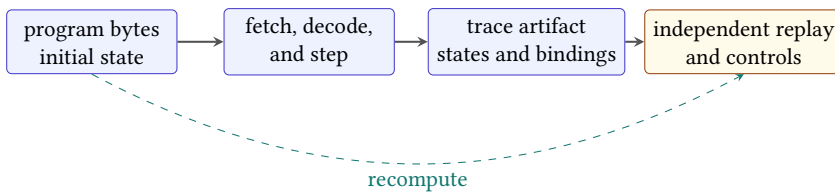


Figure 5.4: A future Axeyum trace route binds program bytes and initial state to an artifact, then independently replays every derived transition and control.

The artifact should not trust its own change summary. It stores complete states or a lossless initial state plus checked deltas; the replay checker reconstructs each successor and compares every component in scope. It also decodes from the bound program bytes at the recorded program counter. A printed mnemonic is a display field, not semantic input.

Termination labels need independent derivation. The replay checker inspects the last machine outcome and the requested bound. Halt and trap must arise from actual steps. Bound exhaustion requires a running final state and exactly the allowed number of steps. A continued-execution result may expose a shorter prefix without claiming that the machine stopped.

A solver-backed question comes later. It may ask whether two short Ao programs can finish with different observations under an input premise. If the answer is yes, the model must decode into an initial state and replay into two traces. If the answer is no, the route needs the generated formula, scope, and independent checking boundary. The trace printer remains useful in both cases because it connects the verdict to executable steps.

The controls attack control itself. Mutate a relative target by four bytes. Try to execute after a trapped state. Try to execute after halt. Report a running state at a finite bound as if the program had terminated. Each error can preserve the

arithmetic destination for a while, which is why trace checking must inspect more than the final register.

Add structural controls as the implementation grows. Remove one intermediate state while leaving the endpoints. Duplicate a state without a matching step. Change the program digest after producing the trace. Alter an unobserved register in one successor. Each mutation attacks a different binding: adjacency, step count, program identity, or full-state preservation.

For a first Python lesson, the public surface can stay small: construct an Ao program from bytes, construct a complete initial state, request a bounded run, print the returned trace, and invoke verification. The Rust layer should own fetch, decode, step, serialization, and checking. Python organizes the lesson and renders the explanation; it must not supply an alternate transition rule.

This runner does not accompany the draft. The traces in this chapter are hand executions of the stated Ao rules. They establish how the future runner must behave on these examples; they do not claim that all Ao traces have already been checked.

5.8 A complete Part I trace

We can now name every layer in the four-instruction example. The immediate bytes denote width-eight words. Registers and conditions belong to complete state. Each decoded instruction has explicit operands and effects. The program counter selects immutable code bytes and advances by four. The `add` uses modular arithmetic and records facts about the mathematical sum. The `halt` outcome prevents another step. An observation may expose r_0 alone, but the trace itself retains the rest.

Nothing in that account depends on calling Ao a RISC or CISC machine. Those labels would add less information than the rules we have printed. When the book turns to real architectures, the same inventory will let us compare one choice at a time.

One boundary remains even in our tiny example. We have written decoded instructions beside the trace. A processor receives bytes. Before an instruction can transform state, those bytes must select one legal operation, operand form, and length—or cause a decoding failure. Part II begins at that boundary.

5.9 Exercises

1. **Execute.** Run the four-instruction program in this chapter from $pc = 0$, with every register initially zero. List every complete state component that changes at each step.

2. **Break.** Replace the taken branch target rule $pc + 4 + 4d$ with $pc + 4d$. For a branch at address 4 with displacement 2, show the two traces diverging at their next program counters.
3. **Prove.** Give an invariant showing that an unconditional zero-displacement branch returns to the same instruction under the Ao target rule. Explain why any finite bounded run alone would be weaker.
4. **Transfer.** Write one register-predicate branch pattern for the later RV64 slice and one compare-then-jump pattern for the later x86-64 slice. List the state read by each branch instruction itself.
5. Construct two one-instruction prefixes that agree on r_0 but differ on a component read by a suffix. Give the suffix and trace both composed programs.
6. Act as the runner for a program that traps on its second instruction. Show why adding a valid third instruction creates no third transition.
7. State the exact result of running a still-running program at bounds 0, 1, and 2. Do not use the words halt or loop unless the trace warrants them.
8. **Execute.** For the program in this chapter, list the five states as rows with pc , changed registers, condition bits, and outcome. Confirm that each adjacent pair satisfies exactly one step.
9. **Break.** Make a runner label bound exhaustion as halt. Give a one-instruction self-loop on which the faulty label changes the observable outcome.
10. **Prove.** Formalize the self-loop argument by induction on the number of transitions. State the invariant and the base and induction cases.
11. **Transfer.** For one planned RV64 branch and one planned x86-64 conditional jump, list the state needed to select the path and the rule needed to calculate both target addresses.
12. **History.** Explain what Turing's finite machine table contributes to this chapter and why it should not be called a stored-program instruction set.
13. **History.** Compare a flow diagram with a concrete machine trace. Name one question each can answer that the other cannot answer by itself.
14. **History.** State the role of assertions at control points in Floyd's method. Relate initialization and preservation to ordinary induction.
15. **Prove.** Show directly that R^0 is an identity for trace composition on both the left and right.
16. **Prove.** Prove associativity of three trace concatenations, including the state count after shared endpoints are removed.

17. **Break.** Define multi-step execution using one or more steps where a later theorem expects zero or more. Give the failed base case.
18. **Prove.** From one-step determinism, prove endpoint uniqueness for every fixed step count. Identify where the proof uses existence.
19. **Break.** Join two traces whose final and initial program counters match but whose registers differ. Show why the combined sequence is invalid.
20. **Design.** Give a three-node syntactic control-flow graph with an edge infeasible under the entry condition. Supply the state invariant that removes it.
21. **Prove.** For the reachability transformer F , show that every closed state set containing the entries contains every finitely reachable state.
22. **Explain.** Distinguish an exact reachable set, an inductive set that may include extra states, and the set of all Ao states. Which can prove safety?
23. **Graph.** Draw a loop with a header that dominates its body and a separate unreachable block. State why dominance alone does not prove semantic reachability.
24. **Execute.** Apply the runner algorithm to a running input at bound zero, a halted input at bound zero, and a two-step program at bounds one and two.
25. **Audit.** Design four result constructors for a bounded runner. For each constructor, list the invariant a checker must establish.
26. **Break.** Check the execution budget only after stepping. Give the smallest bound and program that expose the extra transition.
27. **Prove.** Complete the countdown proof for initial value 4. List the invariant, guard, variant values, no-wrap fact, and final condition.
28. **Break.** Use an unbounded integer that decreases by one as a variant. Give an infinite decreasing sequence and repair the argument.
29. **Design.** Give a loop that needs an ordered pair of measures rather than one obvious counter. Define exactly when the pair decreases.
30. **Analysis.** A loop begins with variant n , takes at most six instructions on each returning iteration, and at most four instructions to exit. Derive a safe instruction-count bound and state every premise.
31. **Logic.** Give a program that satisfies a partial-correctness final condition vacuously. State a nonvacuous safety theorem about the same program.

32. **Prove.** Use an intermediate assertion to compose two program segments. Include the program-counter and outcome facts needed at their join.
33. **Break.** Prove two conditional arms under a signed predicate, then execute a state for which the real instruction uses an unsigned predicate. Show where the proof partitions inputs incorrectly.
34. **Transfer.** For RV64I beq, list predicate inputs, target base, displacement scale, range obligation, alignment consequence, and state writes.
35. **Transfer.** For one x86-64 near Jcc form, identify the source of the predicate and explain why a fixed $pc + 4$ target base is wrong.
36. **Security.** Compare a direct branch target with an indirect jump target obtained from mutable state. State the extra policy enforced by a landing-pad mechanism.
37. **Security.** Explain why landing-pad bytes do not prove that control-flow integrity is active. Name the hardware, mode, loader, and software premises that the claim needs.
38. **Economics.** A function contains 20 independent binary branches. Compute the number of decision sequences. Explain why this is neither a runtime prediction nor proof that all sequences are feasible.
39. **Measurement.** Separate edge reachability, branch probability, test coverage, and worst-case path cost for one diamond-shaped graph.
40. **Operations.** A timed-out request is retried after performing an external write. Give the additional state or transaction rule needed to avoid duplicating the effect.
41. **Context.** Choose a projection π , two prefix states equal under it, and a compatible suffix. Then change the suffix so it reads one discarded component and breaks compatibility.
42. **Axeyum.** Map the book's initial-state, step, bad-state, and bounded-result concepts to Axeyum's symbolic transition-system substrate. List every Ao component still missing.
43. **Axeyum.** Design negative controls for a trace checker that remove a state, alter a program byte, change a branch target, and report bound exhaustion as halt.
44. **Synthesis.** State a complete theorem for one bounded Ao run. It must bind program bytes, initial state, step count, final classification, complete states, and the limits of the evidence.

PART II

Reading Two Real Instruction Sets

Encoding and Decoding

Four bytes lie in memory at the program counter. They are values until a decoder treats them as an instruction. The decoder may produce an operation and operands, reject the pattern, or consume a different number of bytes than a neighboring architecture would. None of those choices yet says what the decoded instruction does.

This boundary is where software becomes unusually literal. A compiler may intend an addition. An assembler may print an addition. A disassembler may report an addition. The processor receives bytes. If the route from those bytes to the intended instruction is wrong, every later proof begins with the wrong object.

Bytes, instructions, and effects are different objects

An encoding is a byte string. Decoding maps that string, in a declared mode, to an instruction instance or a rejection. Execution maps the instruction instance and a state to a successor state. Assembly is notation for humans. Evidence about one map does not automatically establish the other.

6.1 Why instructions need encodings

An early stored-program machine needed a way to place operations and their arguments in the same finite storage used for numbers. That choice made a program inspectable, movable, and mechanically fetchable. It also created a permanent design problem: which parts of a finite word name the operation, which parts name operands, and what happens to patterns that have no assigned meaning?

A regular format gives the decoder fixed places to look. It can make hardware, teaching, and formalization easier. It may spend bits on fields an instruction does not need. A variable format can give common instructions short spellings and retain compatibility with older ones. It makes the first question—how many bytes belong to this instruction?—part of decoding itself.

Neither choice is simply “modern” or “old.” RISC-V’s base integer format uses regular 32-bit instructions, while its optional compressed extension adds 16-bit

forms. x86-64 retains a variable-length family shaped by decades of compatible extension. The useful comparison is not a slogan about RISC and CISC. It is a list of exact decoder obligations.

Compatibility leaves marks in the byte stream

Instruction formats are historical records. A field may exist because an earlier machine needed it. A prefix may acquire a new role in a later mode. An opcode region may be reserved so that a future extension has somewhere to grow. Reading an encoding is therefore partly mathematics and partly careful historical interpretation of a versioned specification.

The history began before a byte stream existed. The Electronic Numerical Integrator and Computer (ENIAC) had first users who described a calculation by setting switches and joining functional units with cables. The machine could perform electronic arithmetic quickly, yet changing the calculation required human work on the machine around it. A stored-program design moved at least part of that control description into addressable memory. Operation selections, addresses, and constants then had to share a finite word. Encoding was no longer only a wiring decision. It became a published boundary between programs and a machine.

That change has no defensible one-person creation story. Designs circulated, several groups developed related ideas, and proposed machines must be kept separate from working ones. The Manchester Small-Scale Experimental Machine, often called the Baby, ran a stored program in June 1948. Its 32-bit words held both data and instructions in a Williams–Kilburn tube. The surviving highest-factor routine used only seventeen instruction words, yet those words directed a run lasting fifty-two minutes. A small encoded object had acquired an indefinitely extensible consequence (Computer History Museum 2026b).

The Electronic Delay Storage Automatic Calculator (EDSAC) performed practical calculations in 1949. Its initial orders loaded later orders from punched tape, and its users developed a library practice in which prepared routines could be reused. An instruction encoding therefore served two audiences almost immediately: the physical control circuits that selected an action and the people and tools that prepared a program. Human notation, tape symbols, memory words, and circuit selection were related, but they were not the same representation.

The relationship became an economic promise as machines became product families. IBM presented System/360 as a compatible architecture spanning implementations with different price and performance points. Its published formats made program-visible fields part of that durable interface (Amdahl et al. 1964; International Business Machines Corporation 1964). Compatibility let software and training survive hardware replacement. It also constrained how future instructions could reuse or extend existing bit patterns. A reserved field was therefore both unused capacity and an option on later design.

The contrast between regular and irregular formats also grew from concrete constraints, not from two timeless philosophical camps. A fixed word can simplify boundary discovery and parallel field selection. Shorter forms can reduce program bytes, instruction-cache demand, and transfer bandwidth. Several formats within one fixed length can give different instructions the operand fields they need. Variable length can preserve old forms and spend more bytes only on instructions that need them. Each choice moves work among the program image, fetch path, decoder, compiler, and future extension space.

RISC projects made regular field layouts and compiler-visible operations an important design cluster. RISC-V continues that lineage in its base formats, but its compressed extension also gives common operations 16-bit forms. x86-64 preserves much older byte patterns while adding prefixes and opcode maps. Neither history supports the slogan that RISC always means one length or CISC means arbitrary chaos. The useful question remains exact: given a mode, feature set, address, and finite sequence of bytes, which structured result is permitted, and how many bytes does it consume?

The rest of this chapter uses one addition on three machines. Ao makes every field visible. RV64I shows a regular commercial-scale design. x86-64 shows a short variable-length instruction whose meaning depends on a prefix and an operand byte. The arithmetic result is familiar. The path used to select that arithmetic is not.

6.2 Four distinct objects

We will keep four representations separate.

Assembly spelling. Text such as `add r3, r5, r2`. It depends on a syntax and may contain aliases.

Instruction bytes. A finite sequence such as `10 2B 02 00`. Byte order and length are explicit.

Decoded instance. A structured value such as `Add(rd = 3, rs1 = 5, rs2 = 2)`.

Transition rule. The rule that reads two registers, writes their fixed-width sum, updates declared conditions, advances the program counter, and preserves everything else.

An assembler normally maps spelling to bytes. A decoder maps bytes to an instance. A disassembler maps an instance or bytes back to a chosen spelling. The reverse maps need not recover the original text because several spellings can denote the same instruction. A proof should therefore use the structured instance as its semantic hinge rather than requiring printed text to round trip character for character.

A successful disassembly is not an execution proof

A disassembler can correctly identify an instruction while an executor gives that instruction the wrong effect. An executor can implement addition correctly while its decoder assigns addition to the wrong byte pattern. Test and prove the two maps separately before testing their composition.

The mode belongs to the decoder input. The same x86 byte sequence can be interpreted differently in different execution modes. The enabled RISC-V extensions determine whether some patterns are ordinary instructions, compressed instructions, hints, reserved encodings, or illegal encodings. A decoder whose type is merely *bytes* $\rightarrow$ *instruction* hides information needed to state its contract.

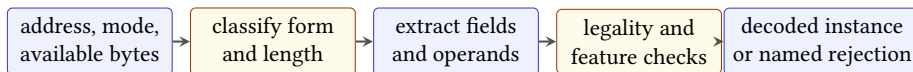
A more honest decoder accepts a source pin, mode, feature set, address, and byte sequence. Its result has the shape

$$\text{Decoded}(i, n) \quad \text{or} \quad \text{Reject}(r),$$

where n is the number of bytes consumed and r is a declared rejection reason. Ao fixes most of these parameters, but keeping the full shape in view prevents the small model from teaching an accidental universal rule.

6.2.1 Rejection makes decoding total

A useful decoder function returns an answer for every finite input supplied to its declared interface. That answer need not be an instruction. It may say that the bytes are incomplete, the address is misaligned, the pattern is reserved, the requested feature is disabled, or the form lies outside the implemented slice. Making these cases explicit avoids host-language exceptions and silent fallbacks becoming accidental architecture semantics.



The execution rule begins only after this pipeline returns a legal instance.

Figure 6.1: A staged decoder returns either a structured instruction or a named rejection. Execution begins only after decoding succeeds.

Incomplete input differs from an illegal complete encoding. A fixed-four-byte decoder given only three available bytes cannot yet inspect all reserved fields. A streaming decoder may request another byte. A fetch model may trap because the fourth address is unreadable. By contrast, four available Ao bytes with a set reserved bit form a complete pattern that the legality rule rejects. The surrounding step semantics decides how each decoder result becomes a machine outcome.

Address belongs in the input even for fixed-length formats. Ao and the selected RV64I subset require aligned instruction addresses. For variable-length x86, the address also determines which byte begins the parse. Starting one byte later can produce a different legal instruction sequence from the same memory. A decoder over an isolated byte array should therefore state that element zero is already the chosen instruction boundary.

Decoder result and fetch result

A decoder result classifies supplied bytes under a mode and feature set. A fetch result determines which bytes are available from machine memory at a program counter and whether reading them is permitted. Keeping the two relations separate allows the same pure decoder to be tested without claiming that every supplied byte string is fetchable in a running machine.

This separation also improves controls. Truncating the buffer should exercise an incomplete-input path. Setting a reserved bit should exercise legality. Changing the start address should exercise alignment or boundary selection. One generic rejection loses the evidence that all three checks actually fire.

Named rejection results should record the first architectural decision that prevents a legal instance, not an accident of the implementation. A useful introductory set contains:

Need more bytes. The supplied prefix can still begin an in-scope legal instruction, but the decoder lacks a required byte. This result says nothing about whether a machine fetch could obtain that byte.

Bad boundary. The declared start address violates an alignment rule or does not satisfy the selected decoder interface.

Reserved pattern. The complete bit pattern occupies space that the pinned specification does not assign to an enabled instruction.

Feature disabled. The pattern has a meaning under another declared extension or mode, but not under this input's feature set.

Outside selected scope. The source specification may define the form, but the book-owned decoder deliberately does not. This is an implementation limit, not an illegal-encoding claim about the architecture.

The categories prevent a small decoder from lying about completeness. Suppose it implements only integer addition. Returning “reserved pattern” for a legal load would attribute its own missing case to the ISA. Returning “outside selected scope” keeps the architectural fact and implementation boundary separate. A caller that needs a complete RV64I decoder must reject the smaller component as unsuitable even though every ADD it accepts is sound.

The classification order is part of the public contract when more than one condition applies. Three supplied bytes at a misaligned Ao address are both short and badly aligned. The decoder may report the boundary failure first or the missing byte first, but its specification and tests must choose. A richer result may retain several causes. Either design is clearer than depending on which internal check happened to run first.

A rejection result can itself be wrong

Give a selected-scope Ao decoder four bytes with a reserved bit set. It must not return “outside selected scope,” because the opcode form is in scope and its legality check failed. Now give it a well-formed opcode assigned only to a future book extension. That input should exercise the scope boundary rather than the reserved-bit control.

6.3 Capacity of an instruction word

An encoding begins with a counting constraint. A field of k bits has 2^k patterns. It can distinguish at most 2^k alternatives without using information from another field or from the decoding context. A three-bit Ao register field names eight registers. A five-bit RV64 register field names thirty-two. If an architecture needs forty registers in one otherwise fixed format, five bits cannot name them all. The designer must spend another bit, use an escape form, infer an operand, divide the register set by context, or change the format.

The same limit governs operation codes. An m -bit operation field has 2^m leaves if every pattern is assigned directly. A design can instead reserve one pattern as an escape into a second field. This does not create information for free. It gives the common operations short selectors while some uncommon operations spend more bits or require another decoding stage. Opcode maps, function fields, and prefixes are variations on this allocation of a finite naming space.

Suppose a simple instruction needs one of sixteen operations, three registers chosen independently from a set of eight, and no other information. Direct fields need

$$\log_2 16 + 3 \log_2 8 = 4 + 9 = 13$$

bits. A 12-bit format cannot contain all $16 \cdot 8^3 = 8192$ combinations, because it has only 4096 patterns. Some combinations must be removed, made implicit, or represented with another word. The pigeonhole principle means that more objects cannot occupy fewer distinct places. This argument is small, but it explains why an instruction table contains exclusions rather than every mnemonic accepting every operand shape.

Formats are tagged products

A fixed R-type format can be viewed as a product of finite sets: operation selector, destination, first source, and second source. A whole ISA is closer to a tagged sum of such products. The tag chooses a format; the remaining bits carry that format's fields. Overlapping tags are safe only when later rules, mode, or feature state resolve them uniquely.

Unused and reserved patterns matter to the counting. Assigning every pattern today maximizes the present vocabulary but leaves no direct name for a future extension. Reserving patterns reduces current capacity while preserving design room and helping a strict decoder catch corruption or version mismatch. Whether a reserved pattern traps, remains unspecified, or becomes a hint is an architectural rule. The arithmetic alone cannot decide it.

6.3.1 Packing and extraction

Let a field have width k , start at bit j , and contain a value a with $0 \leq a < 2^k$. Packing that field into an otherwise zero word gives

$$P_{j,k}(a) = a 2^j.$$

Extraction shifts the word down and reduces it modulo 2^k :

$$X_{j,k}(w) = \left\lfloor \frac{w}{2^j} \right\rfloor \bmod 2^k.$$

In bit notation, the same operation is

$$X_{j,k}(w) = (w \gg j) \& (2^k - 1).$$

For a bounded input, extraction inverts packing:

$$X_{j,k}(P_{j,k}(a)) = a.$$

The range premise does the real work. After the shift, the mask returns $a \bmod 2^k$. This equals a only because $a < 2^k$. If a caller tries to pack eight into a three-bit field, extraction returns zero. Silently masking the input would make an invalid register number look legal.

Several fields can be packed with bitwise OR when their occupied bit ranges do not overlap. Let fields a_i have ranges $[j_i, j_i + k_i)$. If every pair of these ranges is disjoint and every value is in range, then

$$W = \bigvee_i P_{j_i, k_i}(a_i) \implies X_{j_i, k_i}(W) = a_i.$$

The reader proof isolates one field. Its own packed bits survive the shift and mask. Every lower field is removed by the shift or lies below the mask's window; every higher field lies above that window and is removed by the mask. The machine proof can express the negation as a bit-vector formula and ask for a counterexample under the same range and non-overlap premises.

Bitwise OR is not a substitute for non-overlap

Two fields that set the same bit do not retain two independent values. OR merges them. Addition can be equally misleading because a carry can cross a field boundary. Prove the ranges disjoint before treating either operation as field concatenation.

6.3.2 Bit, byte, and display order

Instruction examples involve at least three orders.

Address order. Which byte occupies the lower memory address.

Numeric significance. Which byte or bit contributes the larger power of two to a word value.

Page order. Which field a manual draws on the left.

Little-endian memory places the least-significant byte at the lowest address. It does not reverse the bits within each byte, change the field numbers in the instruction word, or require a manual to draw bit zero on the left. Begin with the addressed bytes, construct the word according to the declared byte order, and only then apply word-bit field definitions. That sequence prevents a visual convention from becoming a semantic rule.

Sign extension has a similar order requirement. A (k)-bit pattern (u) represents the signed integer

$$\text{signed}_k(u) = \begin{cases} u, & u < 2^{k-1}, \\ u - 2^k, & u \geq 2^{k-1}. \end{cases}$$

Extending it to (n>k) bits preserves that integer while choosing its modulo- 2^n word representative. For a split immediate, every fragment must first be placed into its declared logical bit positions. Only the completed (k)-bit pattern has the sign bit named by the specification. Extending each fragment separately assigns several false sign bits and cannot reconstruct the intended displacement.

6.4 Codes, boundaries, and ambiguity

A fixed-length code makes every aligned (n)-byte boundary a candidate instruction boundary. A variable-length code must also determine where one codeword ends.

Information theory calls a code *prefix-free* when no complete codeword is the prefix of another. Such a stream can be decoded from left to right without waiting to learn whether an already complete word will grow into a longer one.

Machine instruction encodings need not be prefix-free when viewed as raw byte strings. A byte that is a complete instruction in one mode can act as a prefix or part of another instruction when the decoder begins in another mode or state. An opcode can announce that ModR/M, displacement, or immediate bytes follow. The decoder's state and the pinned starting address supply information not present in a bare set of byte strings.

Unique decodability is weaker than prefix freedom. It asks whether a complete stream has only one sequence of codewords, even if a decoder may need lookahead. For machine code, the useful claim is narrower and more explicit: from a declared boundary, in a declared mode and feature set, the specification selects at most one result and one consumed length. Starting at another byte may legally produce another instruction sequence. The architecture does not promise that arbitrary byte strings contain self-identifying global boundaries.

One wrong length changes a suffix

Suppose the correct first decode consumes three bytes at address p , but a faulty decoder consumes two. Even if both report the same broad operation for the first instruction, their next addresses are $p + 3$ and $p + 2$. The byte at $p + 2$ is an operand component in the first interpretation and the next opcode in the second. Every later line of disassembly can diverge. Length is therefore part of the semantic bridge from decoding to program execution.

This dependence explains why linear disassembly and recursive path following answer different questions. Linear disassembly repeatedly decodes the next byte after the reported length. Recursive path following uses decoded control-flow successors from known entries. The first can walk through embedded data; the second can miss code reached through unresolved indirect control. Neither method discovers a unique intended program from arbitrary bytes without additional assumptions.

6.5 The Ao encoding

Every Ao instruction occupies four bytes and begins at an address divisible by four. The program map is read in increasing address order.

Byte 0 is the opcode. In byte 1, bits 0–2 name rd , bits 3–5 name $rs1$, and bits 6–7 are reserved zero. In byte 2, bits 0–2 name $rs2$, bits 3–5 hold a branch condition, and bits 6–7 are reserved zero. Byte 3 holds a signed eight-bit immediate when the instruction uses one. An unused field must be zero. This last rule makes Ao stricter than a decoder that merely ignores irrelevant bits.

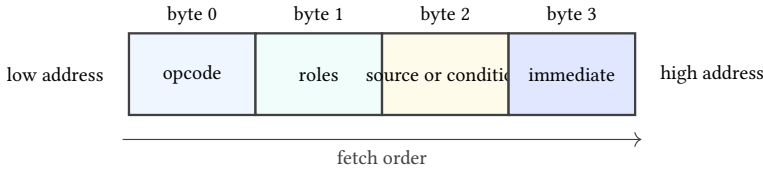


Figure 6.2: The four-byte Ao envelope. The detailed meaning of bytes 1–3 depends on the opcode, but their locations and the total length never change.

Consider

```
add r3, r5, r2
```

Listing 6.1: One Ao register addition

The opcode is 10. Packing $rd = 3$ and $rs1 = 5$ into byte 1 gives

$$3 + (5 \ll 3) = 3 + 40 = 43 = 2B.$$

Byte 2 contains $rs2 = 2$, so it is 02. Addition has no immediate or branch condition, so byte 3 and every unused field are zero. The complete encoding is

10 2B 02 00.

The decoder performs the argument in reverse. It checks the fetch address and length, reads opcode 10, rejects any set reserved bit, extracts the three register numbers, checks that the condition and immediate fields are zero, and returns $\text{Add}(3, 5, 2)$.

A0 addition round trip

Let E be the encoder for a legal Ao addition and D the decoder. To show $D(E(rd, rs1, rs2)) = \text{Add}(rd, rs1, rs2)$, use the fact that each register number is below eight. Masking the packed byte by 07 recovers rd . Shifting it right by three and masking by 07 recovers $rs1$. The same mask recovers $rs2$ from byte 2. The encoder writes zero to every reserved and unused field, so all legality checks pass.

That proof has a converse worth stating carefully. If the decoder accepts an Ao addition byte string b , encoding the returned instance reproduces b . Acceptance matters. Without it, a byte string with a reserved high bit could decode to the same three registers after masking but fail to reproduce the original byte. The reserved-bit check is what makes accepted encodings canonical.

Break the A0 encoding one field at a time

Begin with 10 2B 02 00. Change byte 1 to 6B, which sets a reserved bit. Change byte 2 to 0A, which leaves $rs2 = 2$ but sets an unused condition bit. Change byte 3 to 01. A strict decoder rejects all three. A decoder that only extracts operands accepts them, revealing three different omissions in one small test family.

Illegal encodings are not gaps outside the semantics. If a running Ao state fetches a complete but illegal instruction, its next state is trapped with an illegal-encoding reason. Thus the decoder's rejection becomes an architectural event through the step relation.

6.5.1 Canonical encoding

A decoder could ignore every unused Ao field and still recover the intended operation and operands from encodings produced by the official encoder. Such a decoder satisfies the forward round trip on encoder output. It fails the canonical form contract because many byte strings with nonzero unused fields would be accepted as the same instruction.

Canonical encodings provide one byte representation for each legal Ao instance. This makes byte equality and instruction equality coincide on accepted inputs:

$$D(b_1) = D(b_2) \quad \text{implies} \quad b_1 = b_2.$$

The implication relies on reserved and unused field checks. It would be false in an architecture that intentionally permits several encodings of one instruction.

Accepted A0 encodings are injective

Suppose two accepted Ao byte strings decode to the same instruction instance. The opcode and every used operand field must match that instance. Acceptance forces all reserved and unused fields to their canonical zero values. Each byte is therefore determined by the common instance, so the byte strings are equal. The same argument would fail if the decoder merely masked away an unused set bit.

Canonicity helps evidence handling. A manifest can store the legal bytes and the decoded object without choosing among aliases. It does not eliminate the need to store both: the bytes bind the claim to the program image, while the instance binds execution to structured operands. A digest of only one layer cannot detect every substitution in the other.

Real ISAs may admit aliases at the assembly or encoding level. A disassembler can also choose a preferred spelling that differs from the source text. Their correct

round-trip property may therefore use normalization: decode, re-encode to a designated canonical form, and compare meanings rather than demand reproduction of the original spelling.

6.6 RV64I fixed-width fields

The RV64I base integer ISA uses 32-bit base instructions. The optional compressed extension can add 16-bit instructions, but we exclude that extension in this example. The source pin is the ratified architecture for instructions available without supervisor privilege, version 20260120; RV64I itself is version 2.1 (RISC-V International 2026e).

An R-type instruction divides the 32-bit word into six fields.

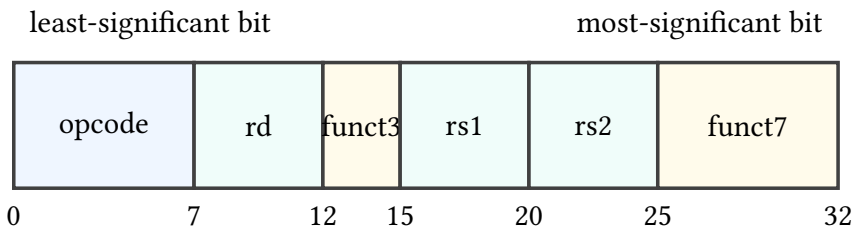


Figure 6.3: The RV64I R-type format, drawn from least-significant bit at left to most-significant bit at right. Printed manuals often reverse that direction.

For `add x3, x5, x2`, the fields are

field	value	role
<i>opcode</i>	0110011	integer register operation
<i>rd</i>	3	destination <i>x3</i>
<i>funct3</i>	000	operation selector
<i>rs1</i>	5	first source <i>x5</i>
<i>rs2</i>	2	second source <i>x2</i>
<i>funct7</i>	0000000	operation selector

Packing these fields gives the word 002281B3. In little-endian memory, the four bytes at increasing addresses are

B3 81 22 00.

The field diagram describes bit positions in the 32-bit instruction word. The byte sequence describes memory addresses. Confusing those two views is a common source of reversed examples.

The *opcode* alone does not identify `add`. It identifies the broad register-operation class. The *funct3* and *funct7* fields complete the selection. Changing

funct7 from 0000000 to 0100000 while leaving the other fields fixed changes this instance to sub *x3*, *x5*, *x2*. Changing only *funct3* can select another operation in the same broad class.

Decode from bytes without reading the mnemonic

Read B3 81 22 00 as a little-endian 32-bit word. Bits 0–6 are 0110011. Bits 7–11 give 3; bits 12–14 give 0; bits 15–19 give 5; bits 20–24 give 2; and bits 25–31 give 0. The table for the pinned RV64I version assigns that complete field combination to add *x3*, *x5*, *x2*. Only after this selection should the execution rule read *x5*, read *x2*, write their modulo-2⁶⁴ sum to *x3*, and advance the program counter.

RV64I adds another semantic condition absent from Ao: register *x0* reads as zero, and writes to it have no architectural effect. The encoding still has a five-bit destination field when its value is zero. Decoding add *x0*, *x5*, *x2* must retain that destination in the instruction instance; the execution rule supplies the discard behavior. Erasing the operand during decode would mix a state rule into a format rule.

Reserved and hint encodings require the same care. A pattern’s status depends on the exact ISA and extension set. “My current processor does nothing visible” does not establish that a pattern is universally a no-operation. The book will name the enabled subset before assigning meaning.

6.6.1 Several field layouts

RV64I base instructions in this window share a 32-bit length, but their fields do not all play the same roles. R-type arithmetic has two source registers and one destination. I-type arithmetic and loads replace the second register field with immediate bits. Stores need two registers but no destination, so their immediate is split around the other fields. Conditional branches also split and rearrange a signed displacement.

The decoder must reconstruct a logical immediate before sign extension. For a split field, concatenating pieces in memory-byte order is wrong; the source specification assigns each instruction bit to a particular result bit. Once the pieces are placed, the high immediate bit determines sign extension to the architectural calculation width. Scaling or an implicit zero low bit is a separate step for forms that require it.

Field extraction precedes interpretation

Suppose a 12-bit I-type immediate field is FFF. Extraction first returns the 12-bit pattern with value 4095 under an unsigned reading. The instruction rule then sign-extends that pattern, producing -1 under its signed reading and an all-ones 64-bit word. Calling the field itself negative before naming its width and extension rule compresses two operations into one.

Instruction selection and operand construction are likewise distinct. The opcode and function fields may select a load class; another field selects the load width and extension policy; register and immediate fields construct its operands. A decoder result should preserve those decisions explicitly so the executor does not inspect raw instruction bits again and create a second, possibly inconsistent decoder.

Do not sign-extend before reassembling a split field

Sign-extending each fragment separately invents high bits that were never part of the immediate. Place every fragment into the declared logical field, then apply the one extension rule to the complete field.

The feature set remains load-bearing even at fixed length. A 32-bit pattern reserved in RV64I may acquire meaning under an enabled extension. Soundness is therefore relative to the declared ISA string, not merely to XLEN and the word value. A decoder that accepts extension forms while claiming the base subset is too permissive; one that rejects enabled forms may be sound but incomplete for the larger scope.

6.6.2 Reconstructing a split immediate

The branch form makes the field rule concrete. Consider the RV64I instruction

```
beq x5, x2, +16
```

Listing 6.2: A branch sixteen bytes forward

The logical branch displacement is a signed thirteen-bit value whose low bit is always zero. The instruction does not store thirteen adjacent bits. It places logical bit 12 in instruction bit 31, logical bits 10 through 5 in instruction bits 30 through 25, logical bits 4 through 1 in instruction bits 11 through 8, and logical bit 11 in instruction bit 7. The low logical bit is supplied as zero rather than stored.

Sixteen has the thirteen-bit pattern

0 000000 1000 0 0,

when its bits are grouped in the order

$$imm[12] \quad imm[10:5] \quad imm[4:1] \quad imm[11] \quad imm[0].$$

Thus instruction bit 31 is zero, bits 30 through 25 are zero, bits 11 through 8 contain binary 1000, and bit 7 is zero. The remaining fields contain ($rs2=2$), ($rs1=5$), $funct3$ 000, and branch opcode 1100011. Packing them gives

00228863.

In little-endian memory, increasing addresses hold

63 88 22 00.

Decoding follows the reverse route. First construct the 32-bit word from the four addressed bytes. Next extract every instruction field by its word-bit position. Place the four immediate fragments into their logical positions and supply bit zero. The result is the thirteen-bit pattern for sixteen. Finally, sign-extend that complete pattern before adding it to the branch instruction's address. This order keeps byte assembly, fragment placement, signed reading, and target calculation as four visible operations.

The branch fragments recover sixteen

Bits 11 through 8 of 00228863 are 1000. Placing them in logical immediate bits 4 through 1 contributes ($8 \cdot 2 = 16$). Every other immediate fragment is zero, including the sign bit. The reassembled thirteen-bit value is therefore sixteen, and sign extension preserves it. A decoder that treats instruction bits 11 through 8 as logical bits 3 through 0 instead recovers eight and sends control to the wrong address.

The implicit low zero serves two purposes. It spends no encoding bit on a fact shared by every branch displacement, and it makes every target this form can encode two-byte aligned. The base RV64I subset used here has four-byte instructions, but the wider architecture reserves room for 16-bit instruction parcels. Alignment and displacement scale must therefore come from the pinned feature set rather than from the visual width of this one instruction.

A split immediate is not stored in reading order

Writing the four fragments beside one another in the order they appear from instruction bit 31 down to bit 7 does not produce the logical signed value. Each fragment has named destination positions. Draw those positions or write the shifts before performing sign extension.

6.7 Encoding and linking

The earlier examples use operands whose numeric fields are already known. A source program often names a symbol instead. An assembler may know the opcode and register fields but not the final distance to a label in another section or object file. It emits provisional bytes together with a relocation record. The linker—the tool that combines object files into a final program—chooses addresses, computes the required value, checks its range, and patches the designated fields.

These are different correctness claims. The assembler must select the right instruction form and describe the unresolved field accurately. The linker must calculate the value under the relocation rule, reject an out-of-range result or select an authorized relaxation, and write only the assigned bits. The final decoder must classify the linked bytes as the intended structured instruction. A source-level mnemonic alone does not bind the proof to the bytes that will execute.

Relocation is a typed patch obligation

A relocation record identifies a place in an object, a relocation kind, a symbol and optional addend, and the rule for producing the final field. Its kind determines the value calculation, field layout, scale, signed range, and overflow behavior. It is not a general instruction decoder, although applying it correctly requires knowledge of the target encoding.

For a split branch displacement, the linker cannot write one adjacent integer field. It calculates the signed displacement, verifies the scale and range, then distributes logical bits among the specified instruction positions. A control should move the target just outside the positive and negative limits. Another should change the relocation kind while keeping the symbol fixed. The link must reject or produce different bytes; silently truncating the value would redirect control.

Link-time relaxation adds another layer. A tool may replace one instruction sequence with a shorter or otherwise preferred legal sequence when final distances and features permit it. Correct relaxation is a program transformation, not a text substitution. It must preserve the declared observable behavior, update addresses and relocation effects, and leave a byte stream accepted by the target decoder. Chapter 10 returns to linking and calling conventions; Chapters 11 and 12 provide the relations needed to state preservation precisely.

Dynamic code generation repeats part of this pipeline at runtime. A just-in-time compiler chooses forms, writes bytes, resolves nearby targets, and transfers control to the generated region. Its trust boundary includes the encoder, writable-to-executable memory policy, instruction-cache coherence required by the platform, and the exact bytes admitted for execution. A correct pure decoder proves only one link in that chain.

Bind evidence to final bytes

Assembly text explains intent. Object bytes plus relocations describe work still to be done. Final linked or generated bytes are the object fetched by the machine. A complete execution artifact identifies which stage it records and binds its claim to the executable image.

6.8 x86-64 variable-length encoding

In 64-bit mode, an x86 instruction can contain legacy prefixes, a REX prefix, an opcode, a ModR/M byte, an optional scale-index-base (SIB) byte, an optional displacement, and an optional immediate. Not every instruction contains every part. The decoder must decide which parts are present as it moves through the byte stream. Our source is Intel’s version 092 instruction reference (Intel Corporation 2026b).

Consider Intel-syntax `add rax, rbx`. One encoding is

48 01 D8.

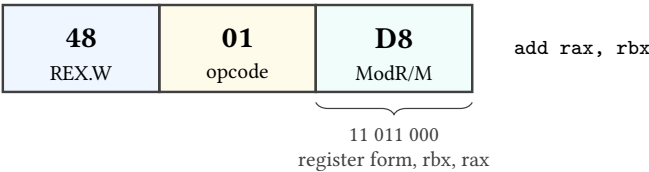


Figure 6.4: One three-byte x86-64 encoding of `add rax, rbx`. The byte boundaries are regular here; the instruction family as a whole is not fixed at three bytes.

Byte 48 is a REX prefix with the *W* bit set, selecting a 64-bit operand size for this form. Opcode 01 selects ADD with a register source and a register-or-memory destination. ModR/M byte D8 is 11 011 000 in binary. The 11 *mod* field selects a register destination rather than memory. The 011 *reg* field names *rbx*. The 000 *r/m* field names *rax*.

Remove byte 48 and 01 D8 is still a valid instruction, but the ordinary operand size is 32 bits: `add eax, ebx`. Changing or removing a prefix need not make the input illegal. It can produce a nearby legal instruction with a different state effect. That makes prefix mutations strong negative controls.

Change ModR/M from D8 to 18. Its high two bits are now 00, so the destination is memory addressed through *rax*, not the register *rax*. The decoder may then need more bytes for an addressing form. A one-bit change has crossed from register arithmetic into a memory operation and altered the possible failure behavior.

Do not infer an x86 instruction boundary from a line break

The next byte in a hex dump is not necessarily the next instruction. It may be a SIB byte, displacement, or immediate belonging to the current instruction. Conversely, a prefix-like byte may begin a separate instruction when parsing starts at another address. Instruction boundaries come from the decoder and the chosen starting address.

This property makes a damaged byte stream especially instructive. If one decoder consumes three bytes while another consumes two, their disagreement does not end at the first instruction. They begin the next decode at different addresses. A length bug can change the interpretation of the entire suffix.

6.8.1 A constrained walk through bytes

A practical x86-64 decoder cannot split every instruction into the same fixed slots. It walks through a bounded sequence of possible components. Prefix bytes alter mode-dependent attributes or extend fields. The opcode determines which later components are required. A ModR/M byte can select registers or a memory-addressing form. That form may require a SIB byte and displacement. The selected operation may finally require an immediate.

Later choices depend on earlier ones. The same byte value can be an opcode at one position and data for a displacement at another. A byte that resembles a prefix has that role only while the parser is still in a prefix-accepting position and the current mode gives it that meaning. Consequently, scanning for familiar byte values without decoder state cannot establish boundaries.

The walk should return both a structured instance and its consumed length. Execution uses the structured operands. The fetch loop uses the length to calculate the following instruction address. A disassembler uses both to print one line and advance through a buffer. If these consumers recalculate length independently, the system contains several decoders whose disagreements can desynchronize the program.

A memory form expands the parse

In the chapter's 48 01 D8 example, ModR/M mode bits 11 finish operand selection with registers and no displacement. Changing those bits to a memory mode can make the remaining ModR/M fields request a SIB byte, a displacement, or both. The opcode still belongs to ADD, but the instruction length, operand kind, address calculation, and possible faults can all change.

Incomplete x86 input is therefore position-sensitive. Ending after an opcode that requires ModR/M differs from ending inside a four-byte displacement. Both

need more bytes, but a diagnostic can name the missing component. Such diagnostics make truncation controls much stronger than one undifferentiated end-of-buffer error.

Length is part of decoder soundness

A decoder that selects the right operation and operands but reports the wrong length is unsound for program execution. The current step may appear correct, yet the next sequential instruction begins at the wrong byte and changes the decoded suffix.

6.9 Three decoder shapes

The examples perform the same broad arithmetic but do not have the same architectural contract.

Question	A0	RV64I	x86-64 example
Length	always four bytes	four bytes in the selected base subset	three bytes for this form
Operation selector	one opcode byte	opcode plus funct3 and funct7	prefix, opcode, and ModR/M
Operand names	three-bit fields	five-bit fields	ModR/M fields extended by REX bits
Arithmetic width	parameter w	64 bits for add	64 bits because REX.W is set
Condition writes	Z, N, C, V	none	selected status flags
Ordinary next PC	$pc + 4$	$pc + 4$	$pc + 3$ for this instance

The table blocks a tempting but false abstraction: “ADD means the same thing everywhere.” The destination sum may correspond under a state relation, yet the condition state, zero register, program-counter increment, legal inputs, and failure modes differ. Cross-ISA reasoning begins by naming those differences, not by hiding them behind a shared mnemonic.

A useful common decoded form

A comparison tool can translate all three instances into a book-owned form such as `AddReg(width, dst, left, right, condition policy, length)`. That form is not a new universal ISA. It is an explicit bridge carrying the facts later relations need. If the translation drops instruction length or condition behavior, it cannot support proofs that observe those facts.

6.10 Decoder correctness

There is no single useful sentence called “the decoder is correct” until we choose a reference and a scope. Four properties matter.

1. *Soundness*: every accepted byte string is assigned the instruction instance specified by the pinned source in the declared mode.
2. *Completeness*: every in-scope legal encoding is accepted.
3. *Length correctness*: the decoder consumes exactly the bytes belonging to the instruction.
4. *Rejection correctness*: reserved, illegal, incomplete, and out-of-scope patterns receive the declared result rather than an invented instruction.

Round-trip tests help but cover narrower statements. If $D(E(i)) = i$ for every supported instance i , the encoder does not produce an input that this decoder misreads. The property says nothing about byte strings the encoder never emits. Conversely, if $E(D(b)) = b$ for every accepted canonical byte string b , accepted inputs reproduce their bytes. Aliases and multiple legal encodings can require a normalization relation instead of literal equality.

An independent decoder gives another view of the same bytes. Agreement over a large corpus is valuable defect evidence. It is not proof against a shared misreading of the specification, nor does it establish execution semantics. The strongest practical route combines source-derived cases, independent implementations, generated boundary inputs, and small proofs for regular field operations.

6.10.1 Testing decoder partitions

Random byte strings often spend most of their time in common rejection paths. A more informative generator partitions the input space by form, operand boundary, reserved field, feature gate, length, and rejection class. It then draws legal and nearby illegal cases from every partition. Coverage means that each declared decision fired, not merely that many bytes were processed.

Mutation tests start from a source-derived legal encoding and alter one load-bearing feature. Set each reserved bit. Move every register field across its minimum and maximum. Change an opcode selector while holding operands fixed. Truncate at every byte boundary. For x86, change prefix presence, ModR/M mode, SIB demand, displacement size, and immediate length separately. For RV64, mutate each fragment of a split immediate before checking the reassembled value.

A decoder control matrix

A control matrix maps each decoder obligation to at least one accepted case and one mutation expected to change its result. Rows name obligations such as length, field extraction, reserved-bit rejection, feature selection, and canonical form. Columns record the witness bytes, expected class, actual class, and replay result.

Differential decoding belongs in the matrix but does not replace it. When two decoders disagree, retain the bytes, modes, versions, structured results, and consumed lengths. Resolve the witness against the pinned primary source. Silently choosing the majority result would turn implementation agreement into an authority it does not possess.

The corpus should also retain boring boundary successes. A decoder that rejects every input can pass an illegal-only suite. Legal minimum and maximum field values, shortest and longest in-scope forms, and enabled feature cases prove that the acceptance paths remain live.

Prove the regular pieces; test the tables

Field extraction is algebra. For a field of width k beginning at bit j ,

$$\text{field}_{j,k}(x) = (x \gg j) \& (2^k - 1).$$

One can prove that packing non-overlapping bounded fields and extracting them returns the inputs. Opcode assignments and mode-specific legality come from a versioned table. Keep the algebraic lemma separate from the source-dependent table check so a later manual revision does not masquerade as a change in bit arithmetic.

6.10.2 Composing decoding with execution

Let (D) be a decoder, (S) a structured instruction semantics, and (F) a fetch operation. A machine step can be factored as

$$\sigma \xrightarrow{F} (p, b) \xrightarrow{D_{m,e,p}} (i, n) \xrightarrow{S} \sigma'.$$

Here (m) is the mode, (e) the enabled feature set, (p) the starting address, (b) the available bytes, (i) the decoded instance, and (n) the consumed length. Each arrow carries a different obligation. Fetch must return the addressed, permitted bytes or a fetch failure. Decode must classify those bytes according to the pinned architecture. Execution must implement the effect of the structured instance. The ordinary next program counter must use the decoded length where the ISA requires it.

A composition theorem needs all three premises. Correct execution applied to the wrong decoded instance proves the wrong step. Correct decoding over bytes that could not have been fetched proves only a pure parser result. Correct operation selection with a wrong length can corrupt the next step. The factorization is valuable because it localizes a failure; it does not permit one local theorem to stand for the whole transition.

For deterministic in-scope forms, decoder soundness can be written

$$D_{m,e,p}(b) = \text{Decoded}(i, n) \implies \text{SpecDecode}_{v,m,e,p}(b, i, n),$$

where (v) pins the specification version. Completeness reverses the useful direction for all forms in the selected scope. Rejection correctness relates each named failure to a specification condition. A decoder may be sound but incomplete if it accepts only one legal ADD form. It may be complete but unsound if it accepts every required form plus a reserved pattern.

Reader proof for the A0 composed step

For bytes 10 2B 02 00 at an aligned readable address, fetch returns the four bytes in address order. The decoder's opcode, reserved-field, and operand checks return $\text{Add}(3, 5, 2)$ with length four. The ADD transition reads the old values of registers 5 and 2, writes their fixed-width sum to register 3, updates the declared Ao condition bits, preserves every other component, and advances (pc) by four. Changing byte 1 to 6B breaks the decode premise before execution can claim an ADD step.

Aliases require a relation rather than literal identity. Two assembly spellings may encode to the same bytes. Two byte strings may also decode to instances with the same architectural effect. Define a normalization function N or a semantic relation $\simeq$, then state the intended round trip:

$$D(E(i)) \simeq i \quad \text{or} \quad E(D(b)) = N(b).$$

Using literal equality where the ISA permits aliases falsely rejects a correct encoder. Using semantic equivalence where Ao promises one canonical byte string weakens a property that can and should detect nonzero reserved fields.

6.11 The cost of decoding

The architectural decoder is a mathematical classifier. A physical instruction frontend—the processor region that fetches and prepares instructions for execution—must perform related work under limits on time, area, energy, and bandwidth. These levels must be connected without identifying them.

Instruction density affects every actor that stores or moves program bytes. A shorter program can occupy fewer instruction-cache lines, require fewer page and memory transfers, reduce distribution size, and leave more shared cache capacity for neighboring code. The benefit depends on workload locality and the rest of the memory system. Saving one byte in cold code does not guarantee a measurable speedup; saving enough bytes in frequently executed code can change its cache footprint.

Regular length makes it easier to propose several instruction boundaries in parallel. Variable-length x86 instruction frontends first determine boundaries, then send instructions through available decode paths. Intel documents instructions up to fifteen bytes and notes that prefixes can change the length a frontend must recognize (Intel Corporation 2026d). Actual throughput, energy, and internal operation count are properties of a named processor, not consequences of the x86-64 ISA alone.

The trade is not simply density against speed. A fixed 32-bit base instruction can offer regular extraction while an optional 16-bit subset reduces common programs. A variable encoding can give frequent forms short representations while a decoded-operation cache avoids repeating some frontend work. Alignment, fusion, predictor structure, cache-line crossings, and implementation width can dominate a particular case. The book therefore records code bytes as an architectural fact and treats frontend cost as separately measured state.

The unit of cost must be named

Code size is measured in stored or transferred bytes. Fetch demand can be measured in cache lines or bytes per interval. Decode capacity may be measured in instructions or internal operations per cycle. Energy is joules, and silicon is area. None of these units can be replaced by the label “RISC” or “CISC.”

Compatibility has a balance sheet as well. Stable encodings protect deployed binaries, operating systems, compiler investment, documentation, and user training. They also force new designs to recognize old forms, preserve mode rules, and adjudicate scarce encoding space. Vendors can spend transistor and verification budgets behind the interface so old bytes run on new hardware. Customers receive continuity; processor designers inherit a larger test obligation.

Decoders also sit inside assemblers, disassemblers, debuggers, emulators, hypervisors, binary translators, profilers, security scanners, and just-in-time compilers. These consumers do not all need the same output. An assembler needs relocation-aware encoding. A debugger needs instruction length and printable operands. An emulator needs a complete structured effect. A scanner may need conservative handling of undecodable regions. Sharing one well-tested decoding core can reduce disagreement, but a common defect then reaches more tools.

6.11.1 Malformed bytes and security

An attacker who controls bytes can choose rare prefixes, truncated inputs, reserved combinations, and sequences that change meaning under another mode. A decoder written in an unsafe language may suffer an ordinary memory-safety bug while reading a promised displacement or immediate. A memory-safe decoder can still be semantically wrong: consume too many bytes, accept a forbidden form, omit a feature check, or disagree with the processor about a boundary.

Boundary disagreement matters to control-flow analysis and gadget discovery. A security tool may approve one instruction stream while execution begins at another reachable address and sees another. This does not mean that every variable-length ISA is insecure. It means the analysis theorem must bind entry points, mode, reachable control flow, and the same decoder contract used by the execution model.

Differential testing is especially useful here. Run the same bytes, start address, mode, and feature set through two independently developed decoders. Compare result class, structured operands, and length. Preserve disagreements as regression cases and resolve them against the primary specification. If both tools inherited the same table or code generator, their agreement is less independent than their package names suggest.

Fuzzing supplies volume; partitions supply purpose. Generate accepted forms, then mutate every length decision, reserved bit, register boundary, immediate boundary, mode switch, and feature gate. Truncate at every byte. Include legal successes so a decoder that rejects everything cannot appear robust. Check resource bounds too: a finite maximum instruction length should prevent one input from causing unbounded parser work.

Compatibility turns old bytes into present obligations

An old encoding is not merely a museum object when current processors and tools still accept it. It remains part of a commercial and security contract. The historical layer explains both the value of compatibility and the size of the modern decoder's obligation.

6.12 Axeyum route

Axeyum does not yet contain the architectural packages used in this chapter. At the inspected sibling revision `a9991fdad6c1e4b2bda596b46d2c8c715556ceae`, it provides reusable bit-vector expressions with extraction, concatenation, and extension, bit-level lowering, solver routes, SAT and DRAT machinery, and kernel reconstruction infrastructure. Its tests include LSB-first concatenation and narrow extraction behavior. It does not provide a reusable Ao decoder, RV64 decoder and semantics, or x86-64 decoder and semantics. This boundary is recorded in the

book's Axiyem guide and must remain visible until code and evidence replace the implementation obligations.

The first Axiyem exercise is therefore constructive. Define a small Ao decoded instruction type. Write a total decoder returning either a legal instance or a named rejection. Add an encoder for legal instances. Prove or exhaustively check the round trip at a small word width, then run controls that set every reserved or unused field.

The next layer can reuse Axiyem's word machinery to express field extraction and packing. A solver can search for a byte string that passes the decoder but fails re-encoding, or two legal instances that encode to the same bytes. A SAT or SMT result is a search result. Promotion to book evidence still requires a manifest, exact source revision, replay route, and negative control.

For RV64I and x86-64, do not begin by implementing the whole architecture. Pin the manual, select the chapter's finite instruction forms, record the mode and feature set, and build a decoder whose public result includes length. Compare against an independent mature decoder, but keep the book-owned rule as the object under test. Execution semantics come afterward.

Four questions to ask of every decoder run

Which exact manual revision supplies the rule? Which mode and extensions are enabled? How many bytes did the decoder consume? What changed when a reserved bit, prefix, field, or final byte was mutated? A transcript without those answers is difficult to interpret and too weak for a load-bearing claim.

6.13 Where the model stops

This chapter does not define all RV64I or x86-64 encodings. It does not cover RISC-V compressed instructions, vectors, privilege, or every extension. It does not cover the full x86 extended-prefix history or every addressing form. It names the architectural maximum length only to bound the decoder and frontend discussion. Later chapters add selected forms only when a proof needs them.

It also does not claim that equal decoded arithmetic establishes equal program behavior. The next chapters add data movement, address formation, arithmetic conditions, control transfer, and calling conventions. Only after those effects are explicit can Parts III and IV define equivalence and refinement between machines.

The working discipline is now fixed: preserve the original bytes, state the mode and enabled features, return length explicitly, distinguish every rejection class, and connect execution only to a legal structured instance. That discipline scales from Ao's four transparent bytes to a variable-length x86 stream without pretending their decoder problems are identical.

6.14 Exercises

1. **Execute.** Decode 10 2B 02 00 under Ao. List every legality check, the structured result, the number of bytes consumed, and the next program counter after execution.
2. **Break.** Construct four mutations of that Ao encoding: wrong opcode, reserved bit, unused condition, and nonzero immediate. State the required rejection for each.
3. **Prove.** Prove that packing and extracting Ao's *rd* and *rs1* fields round trips for register numbers below eight. Identify exactly where the range premise is used.
4. Pack RV64I sub x3, x5, x2. Give the 32-bit word and its four little-endian memory bytes. Explain which one field differs from the addition example.
5. Starting from RV64I word 002281B3, change only *rd* to zero. Separate what changes in the decoded instance from what changes in execution.
6. Decode x86-64 bytes 48 01 D8 by naming the contribution of every byte. Then remove 48; explain why the remaining sequence is not merely an incomplete version of the same instruction.
7. Change ModR/M D8 to 18. Which field changed, and why can the new form involve memory and additional addressing bytes?
8. Give an example showing why $D(E(i)) = i$ does not establish that the decoder rejects all illegal byte strings.
9. **Transfer.** Design a common decoded addition object rich enough to represent the three chapter examples. Mark which fields belong to decoding and which belong only to execution.
10. Design a differential test manifest for one RV64I form or one x86-64 form. Include the source pin, mode, generator domain, independent decoder, expected agreement, and at least two negative controls.
11. **History.** Separate the stored-program idea, a circulated design, and a working stored-program machine. Explain why one date cannot stand for all three.
12. **History.** Explain how initial orders, paper-tape input, memory words, and symbolic notation occupy different representation layers in an early stored-program workflow.
13. **History.** State one benefit and one continuing implementation obligation created by a compatible instruction architecture such as System/360.

14. **Count.** A format has a six-bit opcode and two independent five-bit register fields. How many raw combinations do those fields describe? How many bits remain in a 24-bit instruction?
15. **Prove.** Use the pigeonhole principle to show that a twelve-bit format cannot represent twenty operations with three independently chosen eight-register operands.
16. **Design.** Reserve one four-bit opcode as an escape into a second four-bit selector. Count the direct and escaped operation names. State which operations use the shorter selector.
17. **Break.** Pack the value eight into a three-bit field by silently masking it. Extract the result and explain why the apparent round trip fails.
18. **Prove.** Prove $X_{j,k}(P_{j,k}(a)) = a$ from the arithmetic definitions. Mark where $0 \leq a < 2^k$ is used.
19. **Prove.** Pack two bounded fields into disjoint ranges and prove that extracting either one ignores the other. Then give overlapping ranges that falsify the claim.
20. **Break.** Replace field concatenation by addition without a non-overlap premise. Construct the smallest carry that changes a neighboring field.
21. **Transfer.** A manual draws bit 31 on the left and bit 0 on the right. Memory holds a 32-bit little-endian word. Give a procedure that moves from addressed bytes to fields without treating page order as address order.
22. **Execute.** Reconstruct the signed integer represented by the 12-bit patterns 000, 7FF, 800, and FFF. Then give their 64-bit sign extensions.
23. **Break.** Divide a twelve-bit immediate into high seven and low five bits. Sign-extend the fragments separately and exhibit a value for which the reconstructed result is wrong.
24. **Codes.** Give a prefix-free binary code for three operations. Then give a uniquely decodable fixed-length code for the same operations. Explain why every fixed-length code is prefix-free.
25. **Break.** Use codewords 0 and 01. Show why the set is not prefix-free. State what extra decoder rule would be needed after reading 0.
26. **Explain.** Why can the same x86 byte begin one instruction at one address and occur inside another instruction from an earlier address without giving the architecture's decoder more than one result for the same declared input?

27. **Trace.** Let a correct decoder consume lengths 3, 2, and 4 from address 100. Let a faulty decoder report 2 for the first instruction. List the two sequences of proposed boundaries and the first point of disagreement.
28. **Compare.** Apply linear disassembly and recursive path following to a buffer containing reachable code, an embedded four-byte constant, and an unresolved indirect jump. Name one possible mistake for each method.
29. **Logic.** Write decoder soundness and completeness as two quantified implications. Construct a decoder that is sound but incomplete and one that is complete but unsound for a two-instruction scope.
30. **Logic.** Define a decoder that returns only Reject. Which properties can it satisfy vacuously? Which legal positive control exposes it?
31. **Prove.** State a composed Ao step theorem with separate fetch, decode, execution, preservation, and next-program-counter premises.
32. **Break.** Keep operation and operands correct but report one byte too short. Give a two-instruction byte stream whose second decoded operation changes.
33. **Normalize.** Give two assembly spellings or encodings that a chosen real assembler treats as aliases. State whether your round-trip test uses text equality, byte equality, structured equality, or semantic equivalence, and justify the choice.
34. **Economics.** A frequently executed loop shrinks from 68 bytes to 60 bytes on a machine with 64-byte instruction-cache lines. Give an alignment where the change reduces the number of lines and one where it does not. Explain why byte count alone is not a speed claim.
35. **Measurement.** Design an experiment separating stored code size, fetched bytes, decoded instructions, internal operations, cycles, and energy. Name the actor and unit for every measurement.
36. **Audit.** For a decoder used by an emulator, list the additional outputs needed beyond a disassembler's preferred printed mnemonic.
37. **Security.** Construct a test matrix for attacker-controlled x86 bytes covering truncation, prefix repetition, mode change, ModR/M memory selection, displacement length, and the architectural maximum length.
38. **Security.** Two decoders agree on the mnemonic but disagree on length. Explain why a control-flow scanner must treat this as a semantic disagreement rather than a formatting difference.
39. **Differential.** Two tools share the same generated opcode table. Explain why their agreement is weaker than agreement between independently derived implementations. State how the primary manual adjudicates a mismatch.

40. **Axeyum.** Express a three-bit extract-after-pack property as a bit-vector counterexample query. Give the range premise and the expected solver result.
41. **Axeyum.** Design a search for two distinct accepted Ao byte strings that decode to the same structured instance. Explain what SAT and unsatisfiable (UNSAT) would mean and what replay must check.
42. **Synthesis.** State a complete decoder claim for one selected ISA form. Bind source version, bytes, start address, mode, features, structured result, length, rejection behavior, execution handoff, controls, and excluded forms.

Data Movement and Address Formation

A move that appears to copy one value may also extend it, truncate it, discard it, or obtain it through an address calculation. To describe data movement we must name the source value, destination location, width, and every special rule attached to either operand.

Many machine-code mistakes hide here. The assembly line looks quiet: load, store, move, address. Yet a narrow load can decide whether the high bits become zeros or copies of a sign bit. A register name can decide whether an old upper half survives. An address expression can wrap before memory is checked. The instruction moves data only after these choices have determined what the data and location are.

A movement has a value rule and a location rule

The value rule says which bits reach the destination and how its full width is formed. The location rule says which register or memory byte range changes. Both rules belong to the instruction's meaning. Neither can be recovered from the mnemonic alone.

7.1 Why machines move data

Arithmetic circuits act on values already presented in the right places. Programs therefore spend much of their time arranging operands: bringing a constant into a register, loading a field from memory, saving a result, or forming the address of the next object. Early stored-program designs already had to distinguish transfers from arithmetic because storage locations and working registers played different roles.

The distinction remains, although modern machines blur it in different ways. RISC-V gives arithmetic instructions register operands and uses separate load and store instructions for memory. A selected x86-64 arithmetic form can name a memory operand directly. This difference affects encoding and intermediate

state. It does not by itself determine which program can be related to which. A two-instruction load-then-add and a one-instruction memory add may implement the same observed transformation under suitable fault and alias conditions.

The phrase “data movement” also covers operations that compute no memory access at all. An immediate instruction constructs a value from instruction bits. An address-generation instruction computes a number that may later be used as an address. A move to a special destination can discard the value. What unites the family is not literal copying. It is the controlled production or placement of a word.

The load/store boundary is a design choice

Instruction-set families have repeatedly chosen different places to draw the line between computation and memory access. The useful historical lesson is not that one side is pure and the other compromised. It is that an instruction set exposes one contract while implementations remain free to realize that contract with very different internal work.

The earliest stored-program machines made the separation physical and visible. An instruction word could name a storage address, while an accumulator—a working register used by the arithmetic unit—held a value for arithmetic. Moving a value between store and accumulator was not clerical work around the computation. It was how the machine presented an operand to its arithmetic circuits and returned a result to durable storage.

Repeated access exposed another problem. A program that processed successive array elements could alter the address field of its own next instruction, or repeat a longer sequence of arithmetic and rewriting. The Manchester Mark I added address-modification registers known as B-lines during its development in 1949. A stored modifier could change the effective address without changing the instruction word itself. The mechanism made a loop over successive locations easier to express and helped establish the modern index-register idea (University of Manchester 2026).

The historical vocabulary needs care. A B-line modified parts of an order in the design conventions of its machine. A present x86 index register participates in a base-plus-scaled-index expression. An RV64I load adds a signed immediate to one base register. These are related answers to repeated address formation, not one unchanged mechanism passed intact through time.

By 1964, System/360 published base, index, and displacement fields as part of a compatible architecture. Programs could calculate useful addresses from short instruction fields while operating systems and loaders placed code and data in a larger address space (Amdahl et al. 1964; International Business Machines Corporation 1964). That interface also became an economic commitment. New implementations could change internal circuits, yet existing programs depended on the same visible address equation and transfer widths.

Later instruction-set families divided the work differently. Load/store designs made memory access explicit and kept ordinary arithmetic on registers. x86 families retained forms in which an arithmetic instruction can name a memory operand and inherited overlapping register names from earlier widths. Compilers learned to select among these forms; processors learned to perform their own internal scheduling and address generation. The surface differences matter, but the durable questions remain the source value, effective address, access interval, destination view, and failure behavior.

This chapter holds the semantic questions steady across three machines. What value is produced? Where is it written? How is an effective address formed? Which failure prevents a write? What state is preserved? Those questions are more durable than a classification label.

7.2 Values, locations, and transfers

A **location** is a place named by the architecture that can hold or yield a value. In our models, ordinary register names and valid memory byte ranges are locations. An immediate is not a location. It is a value encoded in the instruction. An effective address is also a value until a memory operation uses it to select bytes.

We can describe a transfer with five items:

1. the source location or value-producing rule;
2. the source width;
3. the rule that forms the destination-width value;
4. the destination location; and
5. the effect on every other state component, including failure.

Several operations fit that contract while performing different kinds of movement.

Copy. Read one location and write the same-width value to another. The source normally remains unchanged.

Construct. Form a value from instruction bits or arithmetic, then write it without reading a data-memory source.

Load. Calculate an address, read a memory interval, assemble a source word, and form the destination-width value.

Store. Calculate an address, select source bits, split them into bytes, and update a memory interval.

Exchange. Read two locations and write each old value to the other location under one declared atomicity rule.

Address production. Calculate an offset and write the number without accessing the memory it could name.

Discard. Calculate or read a value whose destination rule leaves no ordinary register or memory change, as with a write to RV64 register `x0`.

The categories overlap at the level of implementation but differ in semantic obligations. An immediate construction and a load may produce the same word; only the load depends on data memory and can fail through its access. An LEA address production and a memory load may print similar brackets; only the load reads the selected bytes. A self-copy can leave the register map unchanged; it remains an executed step with a changed program counter.

Exchange shows why read-before-write order matters. If locations (A) and (B) are independent, the intended result is

$$A' = \text{read}_B(S), \quad B' = \text{read}_A(S).$$

Both right-hand sides use the entry state. A sequential host implementation that writes (A) first and then reads (A) for the second assignment loses the old value. It implements two copies, not one exchange. If the locations overlap, even this pair of equations needs an architecture-specific rule for the containing storage.

Unchanged data does not mean no effect

A move from a register to itself, a load that reproduces the old destination, or a store that writes bytes already present can leave the selected data unchanged. The instruction can still advance control, fault, trigger a device, or have cost outside the scalar architectural state. State equality must be stated over the chosen observation, not inferred from one equal value.

This taxonomy helps later proofs choose the right frame. A pure register copy frames memory. A load frames memory but reads it. A store changes memory but frames registers. An exchange writes two locations. Address production frames memory and cannot claim that the calculated address was readable. The mnemonic is evidence for none of these facts until the decoded form and its rule are fixed.

A complete movement claim

A complete movement claim binds the decoded instruction and entry state, identifies every old value read, derives any effective address, names the byte or register view accessed, constructs every destination bit, classifies

the outcome, and states the frame for all remaining architectural state. If the claim includes cost, safety, atomicity, or source-language objects, it also names the additional model and premises that make those words meaningful.

This checklist is deliberately longer than an assignment equation. The equation ($d' = s$) can describe the central value while hiding a changed upper register half, an invalid source address, a partial memory write, or a trap. Conversely, a full state theorem can show that a self-copy has no data change while still advancing the program counter. Completeness belongs to the state contract, not to the visual complexity of the instruction.

Consider an eight-bit source 80. Read as unsigned, it is 128. Read as a signed two's-complement byte, it is -128. Zero extension to 16 bits produces 0080. Sign extension produces FF80. The low eight bits agree; the destination words do not.

One byte, two full-width results

Suppose memory byte 12 is 80. A zero-extending byte load returns the 16-bit word 0080. A sign-extending byte load returns FF80. An observation restricted to the low byte cannot distinguish them. A later comparison, shift, store of the full destination, or wide addition can.

Truncation goes the other way. Writing an eight-bit location from a 16-bit source keeps the low eight bits and drops the rest. This is not arithmetic remainder calculated after a signed reading; it is a bit selection. The discarded bits may still survive somewhere else if the destination is part of a larger architectural register. That last possibility is architecture-specific.

The word “move” is therefore too weak for a proof statement. A useful claim says, for example, “copy the low eight bits of register 2 to memory address a , preserving all other bytes,” or “load four bytes, assemble them in little-endian order, and sign-extend bit 31 through a 64-bit destination.”

7.2.1 Overlapping locations

Two names can denote overlapping storage. The clearest example is the x86-64 RAX family, but memory ranges overlap too. A four-byte store at address 100 and an eight-byte store at address 96 both include bytes 100 through 103. Treating named locations as unrelated map keys would miss the interaction.

It is often safer to model one underlying object and define views into it. A subregister is a named view of part of a wider register. Its read extracts selected bits from the containing register. Its write replaces selected bits and then applies any architectural rule for the rest, such as filling the high half with zeros after a 32-bit general-purpose write. Likewise, a memory load extracts a byte interval

from one byte-addressed map, and a store updates that interval while preserving every address outside it.

Read footprint and write footprint

The read footprint of a step is the set of state components whose old values can affect the successor. The write footprint is the set of components the step may change. For memory, a footprint includes byte intervals rather than a single abstract memory name. For overlapping registers, it is expressed in the containing architectural storage.

Footprints support composition. Two steps with disjoint write footprints and no write-to-read conflict are candidates to commute. If either writes a byte or register portion the other reads, swapping them can change the result. The footprint test is a necessary structural check; faults and outcome ordering still need semantic reasoning.

A narrow write changes a later wide read

Start with RAX equal to 1122334455667788. Writing AA to AL changes the containing register to 11223344556677AA. A later read of RAX observes the narrow write even though its destination name was only AL. A model with independent AL and RAX cells would wrongly preserve the old 64-bit value.

7.2.2 Read and write rules

A location name can be modeled as a pair of functions over machine state. Its read function produces a value. Its write function accepts a new value and an old state, then returns a complete new state. For a well-behaved independent cell (L), two laws are expected:

$$\text{read}_L(\text{write}_L(v, S)) = v$$

and

$$\text{write}_L(\text{read}_L(S), S) = S.$$

The first says that reading immediately after a write returns the written value. The second says that writing back the current value changes nothing. These laws resemble a mathematical lens: a view into a larger object together with a disciplined update of that object.

Overlapping machine locations require stronger wording. Writing EAX and then reading RAX does not return the 32-bit input as a 64-bit word without a rule;

it returns the input in the low half and zeros in the high half. Writing AL and then reading AX combines the new low byte with an old neighboring byte. The correct laws therefore mention the containing register and the architecture's completion function.

Let a view V select bit set B of a containing word. A simple preserving write has the form

$$\text{put}_V(v, x) = (x \& \neg M_B) \mid \text{place}_B(v),$$

where M_B marks the selected bits. The old word supplies every bit outside the view's selection. A bit. A 32-bit x86 destination that fills the high half with zeros instead discards the old high half and constructs all 64 result bits from the 32-bit input plus zeros. Its write footprint is the entire containing register even though its printed name is EAX.

A preserving view changes only selected bits

For a selected bit, the mask removes the old bit and the placement term provides the corresponding new bit. For a bit outside the selection, the mask preserves the old bit and the placement term contributes zero. Splitting on membership in B proves both the read-after-write law for the selected view and the frame law outside it.

This formulation also handles memory. A memory location of width n at address a is a view of the byte map over interval $[a, a + n)$, with byte order determining the read and write conversions. Unlike a register view, it also has a domain condition: every byte must permit reading or writing under the selected policy before the operation commits.

7.2.3 Value equality and location identity

Two registers can hold equal words while remaining independent locations. Two address expressions can differ in their written form while naming the same memory interval. A useful semantics therefore separates value equality from a location relation.

For byte intervals $I = [a, a + n)$ and $J = [b, b + m)$:

Equal location. The intervals have the same start and width.

Disjoint. Their intersection is empty.

Partial overlap. Their intersection contains at least one byte, but neither interval is identical to the other.

Containment. Every byte of one interval lies within the other.

The half-open form makes the no-overlap test exact when arithmetic does not wrap:

$$[a, a + n) \cap [b, b + m) = \emptyset \quad \text{iff} \quad a + n \leq b \text{ or } b + m \leq a.$$

An endpoint that equals the next start is not an overlap. A four-byte access at 100 and another at 104 are adjacent and disjoint. Starts 100 and 103 overlap at byte 103. Under fixed-width addresses, these integer inequalities require a no-wrap premise or a representation as a set of one or two modular intervals.

Different expressions do not prove different bytes

Base 100 plus index 3 and base 102 plus index 1 both name address 103. Alias reasoning compares evaluated locations over allowed states, not the spelling of the address expressions.

An alias claim is quantified over states. Two expressions e_1 and e_2 must alias on a precondition (P) when every (P)-state gives overlapping intervals. They may alias when at least one allowed state overlaps. They do not alias when every allowed state is disjoint. A compiler that cannot prove the last condition must preserve dependences that a possible overlap could create.

7.3 Explicit movement in Ao

Ao separates register moves, immediate construction, loads, and stores. Every ordinary register has width w , so `mov rd, rs1` copies one complete word. It preserves memory and conditions, writes only the destination register, and advances the program counter by four. If source and destination are the same register, the architectural register map is unchanged, but the step still advances the program counter.

`movi rd, imm` sign-extends byte 3 to width w . At width 16, immediate 7F produces 007F; immediate 80 produces FF80; immediate FF produces FFFF. The instruction does not load a byte from data memory. The byte belongs to the immutable program encoding.

For `load rd, [rs1+imm]`, Ao first forms

$$a = R[rs1] + \text{sext}_w(\text{imm}8) \pmod{2^w}.$$

It then requires the entire $w/8$ -byte range beginning at a to be valid. If so, it assembles those bytes in little-endian order and writes the result to rd . If not, it traps without a partial register write.

The store form uses the same address rule. It writes all $w/8$ low-to-high bytes of $R[rs2]$ atomically in the architectural model. A failed range check changes the outcome to trapped and preserves memory.

A0 move preservation

For a legal `mov rd, rs1` step, prove that every register other than `rd` is unchanged, memory is unchanged, all four condition bits are unchanged, the outcome remains running, and the program counter advances by four. The proof is mostly projection of the transition definition. Writing the preservation clauses is the point: a destination equation alone does not state a complete instruction theorem.

The first Ao model has no byte load, halfword load, or partial register write. This omission is useful. Extension and truncation can be introduced as explicit bridge operations when we relate a real instruction to Ao rather than being hidden in the small machine's basic move.

7.3.1 An Ao store and load

Take width $w = 16$. Let $r_1 = 0010$, $r_2 = \text{BEEF}$, and let the signed immediate byte be 02. The store's effective address is

$$0010 + 0002 = 0012.$$

The access interval is $[0012, 0014)$. If both byte addresses permit writes, the store splits BEEF into low byte EF and high byte BE. Little-endian order writes EF at address 0012 and BE at address 0013. Every other memory byte, every register, and every condition bit is preserved. The program counter advances by four and the outcome remains running.

Now execute a load from the same base and immediate into r_3 . It reads the two old memory bytes before changing any register. Little-endian assembly gives

$$\text{EF} + 2^8 \text{BE} = \text{BEEF}.$$

The load writes that complete word to r_3 , preserves memory and the other registers, and advances control. This two-step round trip recovers r_2 's entry word in r_3 under four premises: both steps use the same effective address, both ranges are valid, no intervening step changes either byte, and both instructions use the same width and byte order.

Store then load reconstructs one word

For byte index $i \in \{0, 1\}$, the store writes

$$M'(a + i) = \left\lfloor \frac{\langle r_2 \rangle_u}{2^{8i}} \right\rfloor \bmod 2^8.$$

The later load forms

$$\sum_{i=0}^1 M'(a+i)2^{8i}.$$

Substitution returns the two base-256 digits of the original 16-bit word. Their weighted sum is its unique value below 2^{16} . Range validity and fault-free execution are needed before either state update exists.

Several nearby programs do not satisfy this theorem. A byte store followed by a 16-bit load leaves one byte from the old memory. A store at the final valid byte traps because its interval needs one more address. A big-endian load over the same two bytes returns EFBE. A load before the store reads the entry memory. These are boundary cases, not exceptions to a properly scoped claim.

Attack every premise of the round trip

Change the load immediate, remove the high byte from the valid range, insert a store to address 0013, and reverse the load’s byte assembly. Each mutation should produce a different witness or failure. A checker that still reports the original round trip is ignoring one of its inputs.

7.4 Effective-address calculation

An **effective address** is the offset produced by an instruction’s address calculation before the memory system determines whether the requested access is valid. In the flat portions of our models, that offset names the memory byte. More complete machines can add translation, segment bases, privilege checks, and other layers after it.

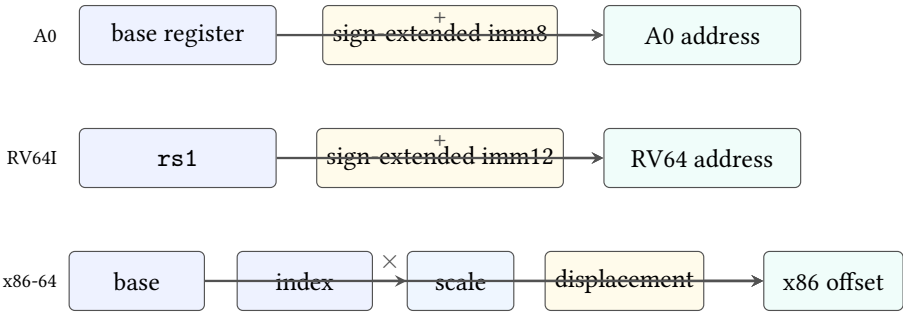


Figure 7.1: Selected effective-address shapes. Ao and RV64I use one base and a sign-extended immediate. A selected x86-64 form can combine base, scaled index, and displacement. Every result still needs a validity check before access.

Ao and the selected RV64I loads share the broad equation

$$a = base + signExtendedOffset \pmod{2^{64}}$$

when Ao is set to width 64. The immediate widths differ: eight bits for Ao and twelve bits for the selected RV64I form. Their encodings and failure environments also differ.

A selected x86-64 memory operand can compute

$$a = base + s \cdot index + displacement \pmod{2^{64}},$$

where the scale s is 1, 2, 4, or 8. A form need not use every component. The encoding can omit a base, omit an index, or use instruction-pointer-relative addressing. This chapter initially uses the ordinary 64-bit address size and a flat address space without nonzero FS or GS bases. Intel's current basic architecture manual supplies the complete mode-dependent rules (Intel Corporation 2026a).

Suppose $rbx=0x1000$, $rcx=3$, the scale is 8, and the displacement is 16. The selected x86-64 address is

$$1000 + 3 \cdot 8 + 16 = 1028.$$

The scale naturally matches an eight-byte array element, while the displacement can locate a field within or beyond the indexed object. The hardware does not know that story. It performs the declared arithmetic and access checks.

The equation reads all contributing registers from the entry state. If the destination register of a load is also the base or index, its new value cannot participate in its own address. For example, a selected load into RBX from $[RBX + 8]$ uses old RBX to choose the bytes and only then replaces RBX with the loaded word. A symbolic executor that performs the register write first creates a circular or incorrect address rule.

Scale multiplication and addition occur at the declared address width. Let the mathematical integer sum be

$$A = \langle base \rangle_u + s \langle index \rangle_u + d.$$

The ordinary fixed-width offset is $A \bmod 2^{64}$. A theorem may use that architectural word directly. If it intends the stronger claim that no wrap occurred, it must prove $0 \leq A < 2^{64}$. The signed displacement d can make the lower inequality fail even when the final modular word is a mapped address.

A valid final address after mathematical underflow

Let the base and index contributions be zero and the displacement be (-1). The mathematical sum is negative, while the 64-bit word result is FFFFFFFFFFFFFFFF. A model whose finite memory includes that address

could permit the final byte access. A no-underflow theorem cannot infer its premise from the successful access.

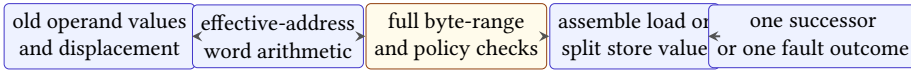
Address arithmetic is not an object-bounds proof

An effective address can be numerically valid while pointing outside the programmer’s intended array or object. The ISA supplies address arithmetic and architectural faults. Language-level provenance and object bounds require additional models and premises.

Address wrapping and range validity are distinct. If the word-sized addition wraps, the resulting address may still fall inside the finite modeled memory. A theorem that intends to exclude wrap must say so as a precondition. Merely checking the final byte range does not establish that the mathematical sum fit.

7.4.1 Address formation before access

A load or store proof is clearer when decomposed into stages. First read every old operand needed for the source and address. Next calculate the effective address as a word. Then check the complete byte range and any selected alignment, permission, or address-validity policy. Only after those checks does a load assemble bytes or a store split its source into bytes. Finally, commit one successor state or one declared failure outcome.



Every stage has a distinct mutation that a checker should reject.

Figure 7.2: A memory transfer crosses several semantic gates. Separating them makes failure ordering, preservation, and negative controls explicit.

This order prevents self-dependence errors. In a load whose destination register also supplies the base, the address uses the old register value. Likewise, a store reads its source word before updating any overlapping memory byte in the atomic architectural step. A functional state update naturally expresses this order when all right-hand sides are evaluated from the entry state.

Range checking must include width. For an n -byte access at word address a , the intended mathematical interval is $[a, a + n)$. Under a finite word address model, proving that interval valid requires more than checking $a + n - 1$ after modular arithmetic. The addition might wrap to a small final address. One safe premise is

$$\langle a \rangle_u + n \leq 2^w$$

together with membership of every address in the modeled region that permits the required read or write region.

Full-range validity from endpoint inequalities

Assume ordinary address arithmetic does not wrap, and a permitted memory region is the half-open interval $[b, e)$. An n -byte access beginning at a lies inside it exactly when $b \leq a$ and $a + n \leq e$. The first inequality protects the low endpoint. The second protects every byte through $a + n - 1$. Half-open intervals make an access ending exactly at e valid without admitting byte e .

Alignment is another policy, not a consequence of range validity. An eight-byte access at address 3 can lie wholly within mapped memory while failing a model that requires divisibility by eight. Other selected environments may permit the access, split it internally, or report a different failure. A machine-code theorem must pin the policy actually used rather than infer it from the operand width.

The distinction is visible in RV64I. A naturally aligned load or store must not raise an address-misaligned exception. For an access that is not naturally aligned, the base ISA leaves important behavior to the execution environment. That environment may support the access invisibly, complete it slowly through an internal path, or permit a contained or fatal trap. The instruction's byte width does not choose one universal result (RISC-V International 2026e).

Atomicity is another separate premise. The chapter's Ao step either commits all bytes or traps with none changed. A real environment may impose additional rules on misaligned, device, or concurrent accesses. Even when one thread sees the intended final bytes, another observer might distinguish internal pieces unless the architecture promises an atomic access of that class. Chapter 16 owns that concurrent extension.

Do not merge all memory failures

An out-of-range address, an alignment violation, a permission failure, and a later translation fault can preserve the same destination while arising at different stages. If the model distinguishes them or their priority, the trace and refinement relation must distinguish them too.

7.4.2 Success and named failures

A mathematical load need not throw a host-language exception or leave a half-built state. Define it as a total classifier over declared inputs. For width (n), destination (d), effective address (a), and entry state (S), its result can have the form

$$\text{LoadResult} \in \{\text{Success}(S'), \text{AddressFault}, \text{AlignmentFault}, \text{PermissionFault}, \text{PolicyOutsideScope}\}.$$

The selected model may omit or combine some categories, but it should say so. Success requires one complete successor. A failure either produces the architecture's declared trapped state or reports that the small semantic package declines the case. Declining a device access absent from the model is not the same claim as proving that the real machine raises an address fault.

Let $\text{bytes}(S, a, n)$ return the ordered byte sequence only when the full interval is readable. Let assemble_{LE} construct the little-endian source word, and let extend be the selected full destination rule. The successful value has the explicit composition

$$v' = \text{extend}(\text{assemble}_{LE}(\text{bytes}(S, a, n))).$$

The successor then updates destination view d with v' , updates the program counter and outcome as specified, and preserves every framed component. This factorization prevents a decoder, memory reader, and extension rule from becoming one opaque host function whose intermediate mistakes cannot be attacked.

A store has a dual composition. It selects the low $8n$ bits of the old source, splits them into n bytes, validates the whole write interval, and updates all bytes in one architectural successor. The source selection occurs before memory changes. A source value loaded from an overlapping memory range by an earlier instruction is ordinary entry state; a single store does not read its own partially written bytes.

Failure preserves the destination in A0

Assume the requested interval fails the Ao range check. The load rule selects the trapped branch before it constructs a register update. That branch changes the outcome and declared fault reason but retains the entry register map and memory map. Projection therefore proves that the destination register and all memory bytes are unchanged. A negative control that writes the first readable byte or clears the destination before checking must fail this frame theorem.

Fault priority matters when several checks fail. A misaligned address may also cross a permission boundary. One semantic package can choose an ordered check sequence if the architecture exposes a priority. Another can return a set of permitted failures if the source specification allows more than one. Choosing one convenient result without authority makes later trace comparison too strong.

The total-result design also clarifies testing. Every success partition needs a witness that reaches the write. Every named failure needs a witness and a frame check. Every declined case needs a caller test that refuses to treat the decline as an architectural trap. A decoder or executor that rejects every input can no longer masquerade as safe.

7.5 Data movement in RV64I

Our RV64I window uses the base integer ISA at XLEN 64. Register x0 always reads as zero, and writes to it have no architectural effect. The other 31 integer registers hold 64-bit words. Loads and stores form addresses by adding a sign-extended 12-bit immediate to rs1 (RISC-V International 2026e).

`ld x5, 16(x6)` reads eight bytes at `x6+16`, assembles a 64-bit little-endian word, and writes it to x5. `sd x5, 16(x6)` writes the low 64 bits of x5 to the same byte range. These are the closest selected counterparts to width-64 Ao load and store.

Narrow loads make the value rule visible. `lw` loads four bytes and sign-extends bit 31 through the 64-bit destination. `lwu` loads the same four bytes and fills bits 63–32 with zeros. Likewise, `lb` and `lbu` differ only in how they form the high 56 destination bits; the memory range and low byte can be identical.

Stores do not extend. `sb`, `sh`, and `sw` write the low 8, 16, or 32 bits of rs2. The high source bits are ignored for that access. Neighboring bytes outside the selected range remain unchanged.

One RV64 word load, two destination values

Let bytes at addresses a through $a+3$ be 00 00 00 80. Little-endian assembly gives the 32-bit word 80000000. An `lw` produces FFFFFFFF80000000; an `lwu` produces 0000000080000000. Both read the same bytes. The instruction's extension rule creates the difference.

7.5.1 Extension and destination width

Let u be the unsigned reading of an n -bit source and let the destination width be $m > n$. Zero extension preserves the same nonnegative reading:

$$\langle \text{zext}_m(x) \rangle_u = u.$$

Sign extension instead preserves the source's signed two's-complement reading. If the source high bit is zero, both extensions agree. If it is one, sign extension fills the new high bits with ones while zero extension fills them with zeros.

This gives an exact agreement condition useful in optimization proofs:

$$\text{zext}_m(x) = \text{sext}_m(x) \quad \text{iff} \quad \text{bit}_{n-1}(x) = 0.$$

The forward direction follows because the new high bits differ when the source high bit is one. The reverse direction follows because both operations then copy the low n bits and fill every new bit with zero.

A narrow load has two independent obligations

First prove that the instruction reads exactly the selected $n/8$ bytes and assembles the intended source word in the declared byte order. Then prove that its extension rule forms the full m -bit destination. Matching the low source bits does not discharge the high-bit obligation, and matching the final word does not by itself prove that the correct memory interval was read.

Narrow stores have the dual shape but no extension. They select the low source bits, split them according to byte order, and update exactly the destination interval. A round trip through a narrow store and wider sign-extending load does not generally recover the original wide register; it recovers the signed extension of the stored low portion.

An immediate arithmetic instruction can also serve movement. `addi x5, x6, 0` copies `x6` to `x5`. An assembler may print the pseudoinstruction `mv x5, x6`; a pseudoinstruction is a convenient notation that expands to a base instruction. The hardware-visible instruction is still `ADDI`. If the destination is `x0`, the calculated result is discarded. Decoding must retain `rd=0`; execution enforces the special destination behavior.

Pseudoinstructions belong to the notation layer

A convenient assembler alias can make source code easier to read without adding an ISA operation. Proof artifacts should record the decoded base instruction and its operands. The chapter may print the convenient spelling beside it, but should not count the alias as another machine instruction.

7.6 Data movement in x86-64

x86-64 names overlapping portions of a general-purpose register. In the RAX family, `rax` names 64 bits, `eax` the low 32, `ax` the low 16, and `al` the low 8. These are not four independent storage cells. The destination name and instruction form determine what happens to the whole architectural register.

In 64-bit mode, a 32-bit write to a general-purpose register zeros its upper 32 bits. Thus `mov eax, ebx` writes the low 32 bits from `EBX` and leaves `RAX` with a zero high half. A 16-bit write to `AX` or an 8-bit write to `AL` does not receive that automatic full-register completion rule; the higher bits outside the named view bits of `RAX` remain as specified by the instruction form.

The legacy high-byte name `AH` selects bits 15 through 8. It overlaps `AX` but not `AL`. In 64-bit mode, instruction forms using a REX prefix cannot encode `AH`, `BH`, `CH`, or `DH` as an 8-bit register operand. This is a decoder legality rule in addition to the view semantics. A proof that accepts every combination of a REX prefix and

an old high-byte name invents instructions the architecture does not provide (Intel Corporation 2026b).

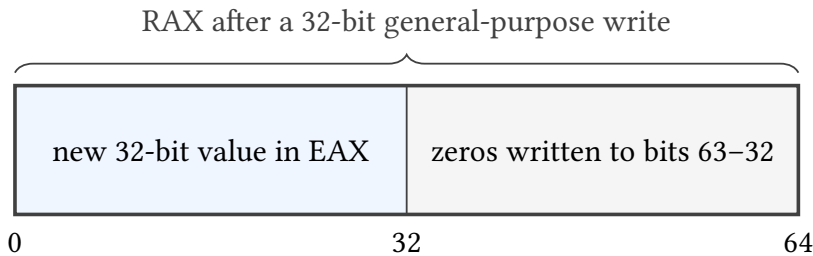


Figure 7.3: A selected 32-bit general-purpose destination write in 64-bit mode defines all 64 bits of the containing register: the high half becomes zero.

Let RAX initially be 1122334455667788. After writing AABBCDD to EAX, RAX is 00000000AABBCDD. After instead writing EEFF to AX, RAX is 112233445566EEFF. A proof that observes only the named destination width misses the difference in the containing register.

Writing AA to AL from the same initial state produces 11223344556677AA. Writing AA to AH produces 112233445566AA88. Writing AABB to AX produces 112233445566AABB. These cases cover distinct views of one containing word. They should be tested from the same nonzero background so an accidental high-half clearing or preservation rule cannot hide.

x86-64 also distinguishes moves that extend a smaller source. A selected zero-extending move fills the new high bits with zero; a selected sign-extending move copies the narrow sign bit. Ordinary MOV between equal-width register or memory operands does neither. The exact opcode and operand sizes must be named before the word “move” has a complete value rule. The pinned Intel instruction reference provides those form-specific contracts (Intel Corporation 2026b).

The LEA instruction is a useful boundary case. For a selected 64-bit form, it computes the offset described by a memory-style addressing expression and writes that number to a register. It does not read the memory bytes at that address. Therefore a bad pointer can be harmless under LEA and fault under a MOV load that uses the same printed brackets.

Address calculation is not memory movement

Choose base, index, scale, and displacement so the calculated offset lies outside the chapter’s valid memory. Compare a selected LEA-like address computation with a load from that address. The first can still produce the numeric word; the second must pass an access check before writing its destination.

Instruction-pointer-relative addressing adds a timing detail to the address equation. In a selected x86-64 form, the base is the address following the instruction, not its first byte. Because instruction length is determined by decode, address formation depends on faithful length decoding. Relocating the instruction and its referenced object together can preserve the displacement, but changing the encoding length at the same address changes the base and may require displacement repair.

The scale field also has limits. It multiplies the index by 1, 2, 4, or 8; it does not encode an arbitrary element size. An array of 12-byte records may need an earlier arithmetic instruction or a combination such as three times an index followed by scale four. The effective-address theorem should describe the actual arithmetic sequence rather than attribute source-level array semantics to one addressing form.

An address expression has no inherent object type

The same base-plus-index expression can locate an array element, a structure field, a table entry, or bytes outside every intended object. Machine semantics provides the numeric word and access behavior. Object identity and provenance enter only through a stronger language or compiler relation.

7.7 Relating movement operations

A real instruction refines an Ao movement—meaning that every selected real step is matched by the declared Ao behavior—when related starting states lead to related results under printed preconditions. The relation can translate register names and instruction addresses. It cannot discard a destination-width effect that a later observation can reveal.

For width-64 Ao load and RV64I 1d, a simple relation can map one Ao register to `rs1`, another to `rd`, and equate their finite memory windows byte for byte. The immediate relation must equate Ao's sign-extended eight-bit offset with RV64I's sign-extended twelve-bit offset for the selected values. Both accesses must be valid. The post-relation equates destination words, preserved memory, running outcomes, and logical successor positions.

The same direct relation does not cover 1w. Ao's width-64 load reads eight bytes, while 1w reads four and sign-extends. We can instead use a width-32 Ao instance and add an explicit sign-extension bridge, or define a more abstract operation that reads four bytes and produces a 64-bit extended result. The model should change visibly rather than pretending the widths match.

For x86-64 `mov eax, ebx`, an Ao width-32 move can match the low result. To relate full 64-bit states, the bridge must also require the x86 destination's upper half to be zero after the step. If the Ao state has only 32-bit registers, that fact may live in a representation invariant rather than a destination equality.

Fault agreement is part of movement refinement

Two instructions do not refine the same operation merely because their successful destination values agree. If one can trap on an address where the other succeeds, either exclude that input, relate the fault outcomes, or weaken the claim. Silently comparing only successful runs changes the theorem.

Aliasing adds another premise. Suppose the destination register contributes to the address. An architectural instruction reads the old operand values needed for its address and source before committing the destination write. A symbolic implementation that updates the register first can compute the address from the new value and still look plausible on non-aliasing tests. Read sets and write order should be explicit even when the mathematical transition is presented as one atomic step.

7.7.1 When memory operations commute

Reordering independent loads and stores is central to compilers and processors, but independence is a semantic claim. Two stores to disjoint byte intervals commute in this chapter's atomic memory model when both addresses and source values are already fixed by state neither store changes. Applying either store first yields the same final byte map because each update preserves every byte written by the other.

Overlap breaks the argument. If one store writes $[a, a + 4)$ and another writes $[a + 2, a + 6)$, their final values at bytes $a + 2$ and $a + 3$ depend on order. A load and store also fail to commute when the load reads a byte the store changes. Even disjoint memory intervals are insufficient if a destination register of the first load supplies the second instruction's address.

Disjoint stores commute

Let store A update interval I with byte function v_A , and store B update disjoint interval J with v_B . Consider an arbitrary address q . If $q \in I$, both orders leave $v_A(q)$ because $q \notin J$. If $q \in J$, both leave $v_B(q)$. Outside $I \cup J$, both preserve the entry byte. The final memories are therefore equal pointwise. The argument also requires both steps to have the same successful outcomes in either order.

Faults make the last sentence load-bearing. Suppose the first store succeeds and the second faults. In an architecture with one committed instruction per step, swapping them changes whether the first store becomes visible before the fault. A refinement that observes only final successful runs hides this difference.

Reordering needs a premise that both accesses succeed, or a richer argument about permitted fault behavior.

The same footprint reasoning explains why a single x86-64 memory-source arithmetic instruction cannot always be replaced by a load followed by register arithmetic. The expanded sequence introduces an intermediate state and perhaps a different fault boundary. Under a no-fault premise and an observation only at the sequence endpoints, the replacement may still be valid; without those conditions, instruction count is not the only change.

7.8 The cost of data movement

An ISA instruction counts an architectural transfer. A physical system may move the corresponding bits through register files, bypass networks, cache levels, translation structures, memory controllers, package links, and main memory. The same architectural load can be served by a nearby cache on one run and by distant memory on another. Its value rule remains stable while its latency, bandwidth demand, and energy change.

At the circuit level, movement changes electrical state. A wire and the gates connected to it have capacitance. Driving a different logic value requires charge to move, and the rough dynamic-energy term grows with capacitance and the square of supply voltage. Longer wires, larger fan-out, and wider buses can require more switching work. A register beside an arithmetic unit and a memory chip across a package boundary are therefore not interchangeable physical locations even when both hold the same 64-bit pattern.

Distance is not measured by the ISA address. Two consecutive virtual addresses may occupy the same cache line, different pages, or storage reached through different hierarchy levels. Conversely, a distant numeric address may already be present in a nearby cache. The physical path depends on mapping, history, sharing, and implementation state that the scalar architecture hides. The separation lets the chapter prove loaded-value equality without proving equal energy or time.

Bandwidth is a rate, not merely a byte count. Moving 64 bytes once and moving 64 bytes on every cycle have the same transfer size but different demand over time. A shared channel also creates opportunity cost: bytes used by one core, device, tenant, or process can delay another. An economic claim should name the owner of the resource, the sustained or peak interval, and whether the measurement includes useful payload, protocol overhead, or fetched bytes that the program never consumes.

Data placement can trade capacity for movement. Replicating a small read-only table near several workers spends storage to reduce repeated distant reads. Compressing data spends computation to reduce bytes moved. Recomputing a value can be cheaper than loading it on one system and more expensive on another. These choices are program and platform decisions above the ISA semantics, but the exact transfer widths and access patterns in this chapter are their input.

This gap is why an instruction count is not a movement-cost model. A model that prices every load equally can still be useful for a scoped search, but it does not establish equal time or energy. Horowitz’s technology survey made the scale difference between arithmetic and data access explicit: energy depends strongly on operation width and on how far storage is from computation (Horowitz 2014). The numerical table is tied to its 2014 process assumptions; the lasting lesson is to measure the relevant hierarchy rather than promote one old number to a constant of nature.

Count bytes and distance, not only move instructions

For an industrial cost claim, name how many bytes cross which boundary: a register-file port, cache line, on-chip link, memory channel, device link, or network. Then name the workload and time interval. “One load” leaves all of those quantities open.

Address generation consumes resources too. A processor may have dedicated address-generation paths, load and store queues, caches of recent address translations, and a limited number of memory ports. Their counts and capabilities belong to a named implementation. An x86 base-plus-scaled-index expression may fit one architectural operand yet compete for a different physical path than a simple base address on a particular processor. The manual gives the meaning; measurement and implementation documentation give the cost.

Code generation balances several accounts. Folding a load into an x86 arithmetic instruction can reduce code bytes and remove a named temporary. Using a separate load can expose scheduling freedom or permit reuse of the loaded value. On RV64, a compiler may need extra address arithmetic when an offset exceeds the signed immediate range. A shorter sequence is not automatically faster, and a faster sequence on one core is not automatically better under a code-size or energy budget.

7.8.1 Alias information and optimization

A compiler can reorder, combine, remove, or move memory operations only when the transformation preserves every behavior in scope. Alias analysis supplies facts about whether two pointer-based memory locations must, may, partially, or cannot overlap. LLVM exposes such results to memory-dependence and optimization clients; a may-aliasing store can block motion of a load because it might change the loaded bytes (LLVM Project 2026c).

The value of a no-alias proof is economic as well as logical. It can allow a value to remain in a register, eliminate a repeated load, remove an overwritten store, or enable vector work over independent elements. A false no-alias claim miscompiles the program. A conservative may-alias result preserves correctness but can forgo

those savings. Analysis precision therefore trades compiler time and complexity against runtime opportunities.

The bridge to ISA bytes must remain visible. A source-language object has lifetime, type, and provenance rules. An LLVM memory location has its own intermediate-representation contract. A machine access has an address and byte interval. A proof that a source-level no-alias fact justifies swapping two machine stores must relate all three layers and account for the address calculation actually emitted.

Copy avoidance changes the amount of movement rather than merely its spelling. Passing a reference, reusing a buffer, moving ownership, or streaming through one representation can avoid copying a large object. The saved work depends on object size and later access. An extra level of indirection may add loads or reduce locality. “Zero copy” should therefore name the boundary across which no copy occurs; data may still move through caches, devices, or networks.

7.8.2 Safety checks and leakage

A valid machine address is weaker than a safe source-language access. Bounds checking can prevent an array index from selecting bytes outside its object. Memory permissions can prevent writes to a page. Neither fact alone proves that the pointer has the intended lifetime or provenance. The model must name which safety property is being enforced and at which layer.

Checks also have ordering obligations. If an implementation reads secret or out-of-range data transiently before a later architectural fault, the final ISA state can still match the specification while timing or cache state leaks information. This chapter’s scalar transition does not model speculation or cache observations. Chapter 16 adds that boundary. The omission must prevent a broad “safe because it traps” claim here.

Memory-mapped devices provide another boundary. Reading the same device address twice may consume two inputs; writing can trigger an external action. Ordinary load elimination or commuting-store arguments assume memory behaves like the byte map defined in Chapter 4. They do not apply to a device region without an explicit effect model. Volatile language or compiler markers help preserve required accesses, but their exact guarantees must be connected to the platform contract.

Movement is where abstractions meet physics

At the ISA level, a load is a relation on state. In a processor, it recruits circuits and storage structures. In a compiler, it constrains reordering. In a cloud or device system, its bytes consume shared bandwidth and energy. The same word “load” is useful only when the current level and evidence are named.

7.9 Axeyum route

The live Axeyum checkout at revision `a9991fda` still lacks Ao, RV64I, and x86-64 architectural packages. It does contain array reasoning, symbolic memory, bounded checking with memory, and a toy bit-vector virtual machine with word-addressed load and store instructions. Those components prove that reusable substrate exists. They do not implement the chapter's byte-addressed Ao ranges, widths, byte order, traps, or complete state. This chapter therefore defines data-movement obligations rather than reporting completed machine proofs.

Start with an executable Ao transfer package. Represent the register map, finite byte memory, and running/trapped outcome. Implement `mov`, `movi`, `load`, and `store`. Reuse Axeyum's bit-vector machinery for extension, truncation, word addition, and byte extraction. Every operation should return a complete successor state rather than only its main result.

The smallest useful property family includes immediate sign extension, store-then-load reconstruction, preservation of untouched registers and bytes, range failure without partial writes, and wrapped-address controls. Run small widths exhaustively where possible. Use solver search at larger widths, but replay any model through the executable transition.

Then add thin adapters for the selected real forms. The RV64 adapter needs `x0`, 12-bit offsets, narrow load extension, and narrow stores. The x86-64 adapter needs selected subregister writes and the chosen effective-address form. Each adapter should name its manual revision and refuse every form outside its declared slice.

Build the checker around the staged pipeline in Figure 7.2. Record the old operands separately from the effective address so a mutation that substitutes the new destination value can be detected. Record the requested width and full interval so a first-byte-only range check cannot pass. Reconstruct loaded values and store bytes inside the checker rather than trusting either derived field in the artifact.

The initial control set should change one fact at a time: immediate sign extension, address wrap, access width, byte order, load extension, store truncation, x86 subregister preservation, and fault atomicity. Each control needs a small witness whose failure class is known in advance. That structure turns an Axeyum example into a lesson about the contract rather than a single opaque pass result.

A good first solver question

Ask whether zero-extending and sign-extending an n -bit value to a wider word ever agree when the source high bit is one. The solver should report no such value. Flip the premise to high bit zero and prove they always agree. The pair tests extension definitions, premise handling, and the negative-control route before any full instruction semantics depends on them.

An artifact for this chapter will eventually record inputs, initial memory, decoded instruction, calculated effective address, bytes read or written, full successor state, source pins, Axeyum revision, and control results. Until such an artifact is active, the examples remain reader-checkable specification work.

7.10 Where the model stops

This chapter uses finite byte memory and single-threaded atomic instruction steps. It does not model caches of address translations, page tables, weak memory, concurrency, memory-mapped devices, or speculative execution. It does not claim the full RV64I or x86-64 addressing repertoire.

The x86-64 discussion assumes ordinary 64-bit address size and a flat segment base for the selected examples. The RV64I discussion leaves access-fault and misalignment policy to a later execution-environment pin where the base ISA does. These limits are not footnotes to the theorem. They say which machine the theorem is about.

Within that machine, something broad has become exact. Ao, RV64I, and x86-64 can spell movement differently, yet the comparison reduces to finite words, named locations, address equations, byte ranges, and observations. Once those objects are written down, resemblance can give way to a relation that either holds or produces a counterexample.

The durable lesson is to resist treating movement as the easy part between interesting calculations. A value has a width and interpretation; a location has a footprint; an address has an arithmetic derivation; an access has a range and failure policy; and a destination write has a rule for the rest of its containing storage. Those facts determine whether later arithmetic and control flow receive the state the programmer intended.

This decomposition also gives a practical proof order. Establish operand reads, calculate the address, justify the complete access, construct the destination value, and finally prove preservation. When a witness disagrees, the failed layer tells the reader what kind of movement error occurred.

7.11 Exercises

1. **Execute.** At Ao width 16, trace `movi r2, 0x80`. Give the complete destination word, next program counter, preserved state, and outcome.
2. Trace a width-16 Ao store followed by a load at the same address. List the two bytes after the store and reconstruct the loaded word.
3. **Break.** Change the Ao load implementation so it checks only the first byte of the range. Construct a final-address example that exposes a partial or out-of-range read.

4. Give an eight-bit source for which zero extension and sign extension to 64 bits differ. Give a second source for which they agree, and explain why.
5. Compute the selected x86-64 effective address for base $0x2000$, index 5, scale 4, and displacement -8. Show the word arithmetic before any memory validity decision.
6. Starting from RAX FFEEDDCCBBAA9988, compute full RAX after writing 12345678 to EAX, 1234 to AX, and 12 to AL in three separate initial states.
7. Compare RV64I `lw` and `lwu` on bytes `FF FF FF FF`. Construct one observation that distinguishes the results and one that does not.
8. Explain why `addi x5, x6, 0` may be printed as a move while its decoded instruction remains `ADDI`. What should an evidence manifest record?
9. **Prove.** State and prove the preservation clauses for an Ao register move. Include the case where source and destination are equal.
10. Prove that two disjoint Ao stores commute under the chapter's atomic memory model. Identify the premise that fails when the byte ranges overlap.
11. **Transfer.** Write a state relation for one width-64 Ao load and RV64I `ld`. Name the register map, memory agreement, offset condition, validity premise, and post-state observation.
12. Extend that relation to a selected x86-64 load using base plus scaled index plus displacement. State which address-size and segment assumptions are required.
13. Design an Axeyum negative control that changes sign extension to zero extension. Give one concrete witness it must find and the failure class the checker should report.
14. **History.** Explain which programming problem an address-modification register helped solve on the Manchester Mark I. Distinguish its B-line rule from a modern scaled index.
15. **History.** State how a published base, index, and displacement contract supported the System/360 compatibility goal. Separate the stable architecture from an implementation's internal circuits.
16. **Explain.** Give two independent locations holding equal values and two different address expressions naming the same location. State which equality statements concern values and which concern locations.
17. **Prove.** For a preserving eight-bit view into a 16-bit word, prove read-after-write and preservation of the high byte outside the view.

18. **Break.** Model AL, AX, EAX, and RAX as independent cells. Give a two-instruction sequence whose result differs from the real overlapping-view model.
19. **Execute.** Starting from RAX 1122334455667788, separately write AA to AL and AH. Give both complete results and the selected bit ranges.
20. **Legality.** Explain why an instruction form with a REX prefix and AH as an encoded byte operand must be rejected even if an AH view function is defined.
21. **Intervals.** Classify $([100,104))$ against $([104,108))$, $([103,107))$, $([100,102))$, and $([100,104))$ as disjoint, partial overlap, containment, or equality.
22. **Prove.** Under no-wrap premises, prove the half-open interval no-overlap test $a + n \leq b$ or $b + m \leq a$.
23. **Break.** Apply the ordinary endpoint test to an eight-bit address interval beginning at 254 with width four. Show why modular wrap invalidates the unqualified argument.
24. **Alias.** Give two affine address expressions that must alias under one precondition, may alias under a weaker one, and cannot alias under a third. State the quantifier in each claim.
25. **Execute.** Reproduce the chapter's width-16 Ao store of BEEF. List every changed and preserved state component, not only the two memory bytes.
26. **Prove.** Derive the width-16 store-then-load theorem from the two base-256 digits. State all range, interference, width, and byte-order premises.
27. **Break.** Replace only the load's byte order in that round trip. Give the result and explain why successful access does not imply successful reconstruction.
28. **Address.** Calculate an unsigned 64-bit effective address for base zero and displacement (-1). Separate the mathematical sum, modular word, and memory-validity question.
29. **Address.** Let a load destination also be its base register. Give entry values and bytes that distinguish old-base address formation from an incorrect new-destination rule.
30. **Range.** For region $([4096,8192))$, give the exact validity inequalities for an eight-byte access. Test starts 4096, 8184, and 8185.
31. **Policy.** A four-byte RV64I load begins at address 3. List the outcomes an execution environment may permit and the one universal conclusion that natural alignment would have supplied.

32. **Transfer.** Compare RV64I sb, sh, and sw from the same source word. Give their write intervals, byte values, and frame conditions.
33. **Prove.** Show that zero and sign extension from (n) to (m) agree exactly when the source high bit is zero. Prove both directions.
34. **Break.** Prove only that an 1w and 1wu destination share their low 32 bits, then use the result as full-register equality. Give a one-instruction suffix that exposes the error.
35. **x86-64.** Derive a RIP-relative address from the address following an instruction and a signed displacement. Change the instruction length while holding its start and displacement fixed; calculate the changed target.
36. **x86-64.** An array element is twelve bytes. Give an arithmetic sequence or address construction for element i , and explain why the SIB scale field alone cannot encode scale twelve.
37. **Commute.** Two stores have disjoint memory intervals, but the first changes a register used to calculate the second address. Construct a witness showing that disjoint final intervals alone do not justify changing the order.
38. **Faults.** Give two stores where one succeeds and the other traps. Compare both orders and state which visible effects prevent a commutation claim.
39. **Compiler.** Distinguish must alias, may alias, partial alias, and no alias for four concrete byte-interval pairs. Name one optimization blocked by an unresolved may-alias result.
40. **Economics.** Compare moving a 64-byte object through a register, one cache line, and a main-memory channel. Name the bytes, boundary, actor, time interval, and missing measurements before making a cost claim.
41. **Design.** Give a precise meaning for “zero copy” at one system boundary. Name two other boundaries across which the same data may still move.
42. **Security.** Explain why an architecturally trapping out-of-bounds load does not by itself prove absence of speculative leakage. Name the extra observations and model needed.
43. **Devices.** Give a memory-mapped input register for which two loads cannot be replaced by one load and reuse. Identify the byte-map premise that fails.
44. **Axeyum.** Map Ao byte memory and complete-range validation onto Axeyum’s current array and symbolic-memory substrate. List every semantic component the adapter must add.

45. **Axeyum.** Design four controls for a store-then-load checker: wrong byte order, truncated range check, intervening overlap, and partial write on fault. Give the expected witness class for each.
46. **Synthesis.** State a complete movement theorem for one real load. Bind bytes, decoded form, old operands, address arithmetic, access interval, policy, assembled value, extension, destination view, frame, outcome, source version, and excluded processor-internal effects.

Arithmetic, Logic, and Condition State

At fixed width, addition produces a stored word even when the mathematical sum lies outside the word's unsigned or signed range. Carry and signed overflow describe different facts about that event. Confusing them is easy; making both rules explicit is easier.

Arithmetic also changes more than numbers. One machine writes condition bits that a later branch can read. Another writes only a destination register and asks a branch to compare registers directly. A third can materialize a Boolean answer as an ordinary word. The calculation and the place where its facts are recorded must both enter the proof.

A result word does not contain every arithmetic fact

The low w bits determine the stored result. They do not by themselves say whether the unsigned sum crossed 2^w , whether the signed sum left its range, or which condition state an instruction promised to update. Those are separate parts of the transition.

8.1 Why machines record arithmetic conditions

A program rarely computes only to keep the result. It also asks whether the result was zero, whether one value was less than another, whether an unsigned addition carried, or whether a signed calculation left the range its word can represent. Early arithmetic units exposed some of these facts as condition state because the next instruction could branch on them without storing another ordinary word.

That design remains visible in x86-64. Selected arithmetic and compare forms write named status flags, and later conditional instructions read Boolean formulas over those flags. RISC-V's base integer design usually keeps the condition in the instruction that needs it or writes an explicit zero-or-one comparison result. Ao uses four condition bits so that the hidden-state style can be printed in full before we compare it with the alternatives.

Neither state shape is intrinsically more semantic. A flag is architectural state if later instructions can observe it. A Boolean register is ordinary state with a convention about its value. A direct branch comparison can avoid materializing either. What matters is whether the surrounding program receives the information it needs.

Condition state is a compact memory of a calculation

A condition code preserves selected facts while discarding the operands and, for a compare instruction, often the arithmetic result itself. It is a small summary of a larger event. The summary is useful precisely because it is incomplete; proofs must therefore say which later questions it can answer.

8.1.1 Arithmetic before electronic circuits

The ideas in this chapter have several histories. Positional notation made a digit's place part of its value. Hand calculation and mechanical calculators made carrying a step that had to be performed reliably. Leibniz's published 1703 account showed addition, subtraction, multiplication, and division in binary notation. He was explaining arithmetic, not specifying a modern processor, but the account makes one lasting point clear: a small digit set can support a complete system of calculation once the place rules are fixed (Leibniz 1703).

Congruence supplied another language. Gauss's systematic use of congruence in 1801 lets us say that two integers have the same residue modulo 2^w without saying that they are equal as ordinary integers (Gauss 1801). A machine word later gave that distinction a physical boundary. It retains one residue and discards, redirects, or records information outside the selected width.

Boolean algebra and switching theory supplied a circuit language. Boole studied logical relations in algebraic form. Shannon showed in 1938 how relay and switching networks could be analyzed and synthesized with such algebra (Boole 1854; Shannon 1938). This lineage does not make the integer reading of a wire bundle automatic. Gates combine physical signals; the representation contract says when those signals mean a sum, sign, carry, or comparison.

Electronic machines then had to decide which arithmetic facts software could observe. Some exposed accumulator signs or overflow indicators. Some made a small condition field that later branches tested. IBM System/360 published a two-bit condition code as part of a compatible architecture: different instruction classes assigned documented meanings to the same architectural field (Amdahl et al. 1964; International Business Machines Corporation 1964). That history is important because it separates a temporary circuit signal from a durable software interface. Once programs branch on a condition code, compatibility gives its definition economic weight.

Multiplication shows the same meeting of mathematics and equipment. Booth's 1951 paper sought one process for signed binary operands represented in complementary form. Its motivation explicitly included reducing the equipment and special correction needed for mechanized multiplication (Booth 1951). Later arithmetic designs developed many ways to form and combine partial products. The lesson is not that one method won forever. A representation can make one operation regular while moving cost into another operation, a correction step, or more circuitry.

Three dates can describe one arithmetic idea

A mathematical identity may precede a practical circuit. A circuit may precede a published instruction rule. An instruction rule may later become a compiler assumption. Date the event being claimed: publication, working hardware, architecture, or software contract.

8.2 Four questions about addition

Let x and y be width- w words. Their ordinary nonnegative readings lie from zero through $2^w - 1$. Write

$$s = \langle x \rangle_u + \langle y \rangle_u$$

for the mathematical unsigned sum and

$$r = s \bmod 2^w$$

for the stored word. Four questions now have distinct answers:

1. What is the stored result r ?
2. Is r zero?
3. Is the high bit of r one?
4. Did the unsigned or signed mathematical reading leave its range?

Ao names the corresponding bits Z, N, C, V . Zero and negative depend only on the stored result:

$$Z \quad \text{iff} \quad r = 0, \quad N = \text{bit}_{w-1}(r).$$

Carry records an unsigned boundary:

$$C \quad \text{iff} \quad s \geq 2^w.$$

Signed overflow records a signed boundary. It holds exactly when the operands have the same high bit and the result has the other high bit.

8.2.1 Several views of one addition

It is useful to model one successful arithmetic event before placing it inside a machine state. For a selected operation and width, the event can contain

$$E = (r, z, n, c, v),$$

where r is the result word and the other fields are mathematical facts. Not every instruction exposes every field. A machine policy maps the event to architectural writes. For each condition location, the policy can:

Define. Write a specified event field or Boolean formula.

Clear or set. Write a fixed bit independent of the operands.

Preserve. Copy the entry-state bit into the successor.

Leave undefined. Make no promise that clients may rely on for the selected form.

Undefined is not a synonym for preserve. It is also not automatically a fresh random bit. A faithful model may represent an allowed set of successors, a symbolic value constrained only by the manual, or a declared refusal to model programs that observe the field. The choice affects equivalence. Two implementations that choose different undefined bits can both satisfy the instruction rule, while a later program that relies on one choice lies outside the portable contract.

The full state transition adds destination and frame rules to this event. If the operation succeeds, it writes the result where promised, applies the condition policy, advances control, and preserves the remaining in-scope state. If the operation can fail, its outcome class and commit rule are another field. Division by zero cannot be modeled merely by inventing an arbitrary quotient and calling the arithmetic event complete.

Separate fact, record, and observation

Carry may be a true mathematical fact about a widened sum. An instruction may or may not record it. A later observation may or may not include the location where it was recorded. Proofs become clearer when these three questions are answered separately.

The upper-left example in the figure adds 1111 and 0001. The unsigned readings are 15 and 1, so the mathematical sum reaches 16 and sets carry. The signed readings are -1 and 1, whose sum fits the signed range, so signed overflow is clear. The lower-right example adds 7 and 1. It has no unsigned carry, but the signed result 8 is outside the width-four signed range and sets overflow.

carry no carry	1111+0001=0000 $C = 1, V = 0$	1000+1000=0000 $C = 1, V = 1$
	0010+0011=0101 $C = 0, V = 0$	0111+0001=1000 $C = 0, V = 1$
	no signed overflow	signed overflow

Figure 8.1: Width-four addition supplies examples in all four carry and signed-overflow combinations. Neither condition implies the other.

8.2.2 Widening separates storage from range

Because two unsigned width- w readings sum to at most $2^{w+1} - 2$, one extra bit is enough to hold their exact sum. There are unique values $C \in \{0, 1\}$ and $0 \leq r < 2^w$ such that

$$\langle x \rangle_u + \langle y \rangle_u = C2^w + r.$$

The low part r is the stored word. The coefficient C is the carry. No approximation or particular circuit is involved. The equation is Euclidean division by 2^w .

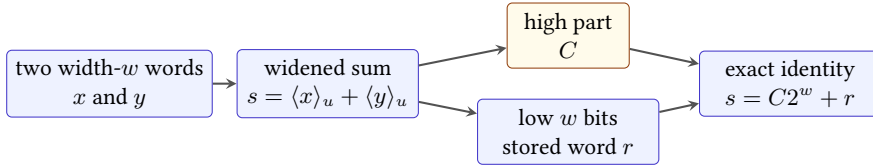


Figure 8.2: Widening makes the unsigned arithmetic exact. Truncation stores the low word, while the high bit records whether the unsigned range was crossed.

The identity gives two interchangeable definitions:

$$r = s \bmod 2^w, \quad C = 1 \quad \text{iff} \quad s \geq 2^w.$$

It also yields a useful test that avoids a wider host type. For width- w unsigned addition, carry occurs exactly when the stored result is below either operand. If no carry occurs, $r = x + y$ is at least each operand. If carry occurs, subtracting 2^w makes r smaller than each positive contributor. A proof assistant or solver can check this equivalence directly over bit vectors.

The stored word has another algebraic meaning. Width- w words under addition form arithmetic modulo 2^w . Addition is associative and has zero as an identity

even when intermediate mathematical sums cross the unsigned boundary. Carry is not part of that modular result; it is extra information about the chosen unsigned representatives. This distinction lets one theorem establish a modular calculation while another establishes that no range boundary was crossed.

One result, two claims

At width eight, $FA + 0A$ stores 04. The modular equation $250 + 10 \equiv 4 \pmod{256}$ holds without a precondition. The ordinary integer equation $250 + 10 = 4$ is false, while the widened identity $250 + 10 = 1(256) + 4$ is exact. A result theorem must say which of these meanings it promises.

Say which overflow you mean

The word *overflow* is incomplete in a fixed-width claim. Unsigned carry and signed overflow answer different range questions and can disagree. Name the reading, range, and condition bit.

8.2.3 Algebraic laws of word addition

Width- w words with modular addition and multiplication form a finite commutative ring. Addition and multiplication are associative and commutative. Zero and one are identities. Multiplication distributes over addition. Every word has an additive inverse. These laws hold without no-overflow premises because equality is equality of residues modulo 2^w .

The same structure is not an ordered copy of the integers. Following a modular addition around the greatest unsigned word returns to zero, so unsigned order cannot satisfy the ordinary implication

$$x < y \implies x + z < y + z$$

for every word z . At width four, $0 < 1$, but adding 15 gives 15 is not less than 0 under unsigned readings. Signed order has its own boundary and fails the same unrestricted compatibility. Order-preserving rewrites require a range premise that excludes wraparound.

Multiplicative cancellation also needs care. Over ordinary integers, a nonzero factor can be cancelled from an equality. Modulo 2^w , an even nonzero word can be a zero divisor. At width four,

$$2 \cdot 0 \equiv 2 \cdot 8 \pmod{16},$$

although $0 \neq 8$ as words. A word has a multiplicative inverse modulo 2^w exactly when its unsigned reading is odd. Thus cancellation by an odd word is valid; cancellation by an arbitrary nonzero word is not.

This distinction matters in low-level code. Multiplication by an odd constant is a permutation of width- w words and can be reversed by multiplication with its modular inverse. Multiplication by an even constant loses at least one low-bit distinction. A hash mixer, random-number transition, or bit scrambler that needs an invertible step must choose constants according to the finite ring, not ordinary nonzero arithmetic.

The signed reading is a useful bijection from word patterns to a centered integer interval, but it is not a ring homomorphism into ordinary integers. It maps a modular sum to the ordinary signed sum only when the latter remains inside its range. The unsigned reading has the same limitation at its upper boundary. A proof can always use the word-ring identity, then add a separate range lemma to recover the desired integer equation.

Odd words are exactly the invertible words modulo a power of two

If x is even, every product xy is even, so it cannot be congruent to one modulo 2^w . If x is odd, its greatest common divisor with 2^w is one. Bézout's identity gives integers a, b with $ax + b2^w = 1$; reducing modulo 2^w makes a a multiplicative inverse of x . The argument classifies every word.

A reversible word update that is not reversible over an interval

At width eight, the update $x \mapsto 5x + 1$ is a permutation because 5 is odd. The inverse uses the modular inverse of 5. If the same formula is required to stay within an ordinary application interval such as 0 through 99, modular invertibility alone says nothing: outputs may leave that interval and later range handling may lose information.

8.3 From a one-bit adder to a circuit

The widened identity describes what addition must mean. A circuit still has to produce the bits. Begin with input bits a, b . A half adder has sum bit $a \text{ xor } b$ and carry bit $a \text{ and } b$. Its four input rows are the ordinary one-column binary addition table.

A full adder also accepts carry-in c . Define

$$\begin{aligned} p &= a \text{ xor } b, & s &= p \text{ xor } c, \\ g &= a \text{ and } b, & c' &= g \text{ or } (p \text{ and } c). \end{aligned}$$

Here p says that the input pair will pass an incoming carry, while g says that the pair generates a carry on its own. For every one of the eight input triples,

$$a + b + c = 2c' + s.$$

This tiny equation is the circuit version of the widened identity. It is also a complete proof target: evaluate all eight Boolean rows, or derive the result from the definitions of exclusive-or, conjunction, and disjunction.

Full-adder correctness composes by place value

For bit position i , let c_i enter and c_{i+1} leave. Full-adder correctness gives $x_i + y_i + c_i = s_i + 2c_{i+1}$. Multiply by 2^i and sum from zero through $w - 1$. Every internal carry occurs once on each side and cancels. With $c_0 = 0$, the remaining equation is the widened word identity with low result bits s_i and final carry c_w .

One direct construction places w full adders in a chain. The outgoing carry of position i becomes the incoming carry of position $i + 1$. This ripple-carry design has a simple local proof and regular layout. Its longest dependence, however, can cross the whole word. The high result bit cannot settle until a carry generated near the low end has propagated through every intervening position.

Parallel carry methods reorganize that recurrence. For each position, a generate/propagate pair summarizes how an interval of bits responds to an incoming carry. Adjacent summaries combine with an associative operation. A tree can therefore compute distant carries in logarithmic logical depth rather than a linear chain. Kogge and Stone's 1973 recurrence method became one important basis for such parallel-prefix networks (Kogge and Stone 1973).

The smaller depth is not free. A prefix network may require more combination nodes, longer wires, larger fan-out control, more area, and more switching than a ripple chain. Other prefix shapes choose other points among depth, wiring, fan-out, and regularity. A real implementation also faces placement, process, voltage, clock, and workload constraints. Thus "fast adder" is not a complete engineering claim. It needs a width, technology, timing target, physical layout, and cost measure.

The proof circuit need not be the processor circuit

A ripple network is a clear constructive proof of word addition. A processor may use a different adder, split the operation across stages, or reuse arithmetic resources. Architectural correctness requires the same visible relation, not the same internal graph.

The physical boundary matters even when the architectural rule is timeless. Charging and discharging capacitance consumes energy. Wider structures place more nodes and wires in play; longer or more heavily loaded paths take more time to settle. Designers can trade parallel hardware for repeated use of a smaller unit. Verification effort changes too: regular local cells are easy to state, while optimized networks demand careful connection and equivalence checks. Mathematics sup-

plies identities, but a chip must pay for their realization in matter, time, energy, and engineering labor.

8.3.1 Overflow from the top carries

The carry chain provides a second way to calculate signed addition overflow. Let c_{w-1} be the carry entering the high bit and c_w the carry leaving it. Then

$$V = c_{w-1} \text{ xor } c_w.$$

To see why, inspect the high-bit full adder. If both operand high bits are zero, signed overflow occurs only when an incoming carry makes the result high bit one. Then $c_{w-1} = 1$ and $c_w = 0$. If both input high bits are one, overflow occurs when their negative sum wraps to a nonnegative high bit. Then $c_{w-1} = 0$ and $c_w = 1$. When the input high bits differ, the top adder either receives and emits a carry or receives and emits none, so the two carry bits agree and $V = 0$.

This formula is equivalent to the same-input-sign, different-result-sign rule. The two expressions use different intermediate facts. A proof system can establish their equivalence for all input bits, while a circuit may choose the form that fits its available signals and timing. An architecture that exposes OF or V promises the final Boolean fact, not either internal construction.

Compare two overflow implementations

At width four, calculate the carry entering and leaving the high bit for one example in each (C, V) class. Check both the top-carry XOR rule and the sign rule. Then invert only c_{w-1} in a faulty implementation and find a row that distinguishes the formulas.

8.3.2 Zero and negative conditions

The zero condition has no ambiguity between signed and unsigned readings. A width- w word has unsigned reading zero exactly when all bits are zero, and its signed reading is then also zero. Thus Z can support equality-to-zero tests for either reading. What produced the word is irrelevant once the instruction policy says that Z is defined from its result.

The negative condition is different. N copies the high bit, so it says that the stored word has a negative two's-complement reading. It does not say that an unsigned result is below zero, and it does not by itself say that a signed mathematical calculation produced a negative value. If signed overflow occurred, the stored signed reading has wrapped away from the mathematical result.

These facts explain several branch formulas. After a subtraction that does not overflow, N alone agrees with signed less-than. Without that premise, $N \neq V$ repairs the wrapped cases. Equality uses Z without a range premise. Unsigned

order uses the borrow convention rather than N , because the high bit is simply part of the unsigned magnitude.

Zero after subtraction means equal words

Modular subtraction stores zero exactly when $x - y \equiv 0 \pmod{2^w}$. Both unsigned readings lie in an interval of length 2^w , so their difference lies strictly between -2^w and 2^w . The only multiple of 2^w in that range is zero. Therefore the readings, and hence the words, are equal. The reverse direction is immediate.

Condition names still need an instruction policy. A move might leave Z unchanged even when it writes a zero word; a logic instruction might define Z from its result; a compare might define Z from a subtraction it does not store. The mathematical predicate and the architectural decision to record it are separate claims.

8.4 The signed-overflow rule

The sign rule is not a hardware incantation. It follows from the signed range. If one operand is nonnegative and the other negative, their sum lies between them. It cannot exceed both ends of the interval that the word represents. Therefore opposite-sign addition cannot overflow.

If both operands are nonnegative, signed overflow occurs exactly when the mathematical sum is at least 2^{w-1} . The stored high bit is then one, so the result appears negative. If both operands are negative, their sum can fall below -2^{w-1} . Modular reduction then yields a stored word whose high bit is zero. These are precisely the same-input-sign, different-result-sign cases.

Writing $h(z)$ for the high bit of word z , the rule is

$$V = \neg(h(x) \text{ xor } h(y)) \text{ and } (h(x) \text{ xor } h(r)).$$

A reader proof of addition overflow

Split on whether the two input high bits agree. When they differ, one signed operand is nonnegative and the other negative, so the sum stays within range. When both are zero, overflow means the nonnegative sum reaches the signed upper boundary, which makes the stored high bit one. When both are one, overflow means the negative sum crosses the signed lower boundary, which makes the stored high bit zero. These cases match the Boolean formula.

This proof handles every width $w > 0$. Exhausting all 256 pairs at width four is still useful, but it checks an instance. The case argument explains why the same rule holds at width 8, 32, 64, or another positive width.

8.4.1 Boundary cases for testing

A census can say more than whether every row passed. Let $M = 2^w$. There are M^2 ordered pairs of width- w words. For a fixed unsigned x , exactly x choices of y satisfy $x + y \geq M$. Therefore the number of carry-producing pairs is

$$\sum_{x=0}^{M-1} x = \frac{M(M-1)}{2}.$$

The remaining $M(M+1)/2$ pairs do not carry. These counts are useful negative evidence. A checker that reports every row as no-carry can still claim to have visited M^2 rows; it cannot match the partition count.

Signed overflow has another exact count. Put $H = 2^{w-1}$. The nonnegative signed readings are $0, \dots, H-1$. For fixed nonnegative x , the number of nonnegative y with $x + y \geq H$ is x , giving $H(H-1)/2$ positive-overflow pairs. The negative readings are $-H, \dots, -1$. Counting pairs whose sum is below $-H$ gives $H(H+1)/2$. Thus the total signed-overflow count is

$$\frac{H(H-1)}{2} + \frac{H(H+1)}{2} = H^2 = 2^{2w-2}.$$

Exactly one quarter of all ordered pairs cause signed overflow at every positive width. The positive and negative sides are not individually equal: the least signed value creates more lower-bound cases than the greatest positive value creates upper-bound cases. Their sum is the clean power of two.

The joint (C, V) partition carries still more information. For widths at least two, all four classes contain at least one pair. Four compact witnesses are:

class	x, y	reason
$(0, 0)$	$0, 0$	neither boundary is crossed
$(1, 0)$	$M-1, 1$	unsigned carry, signed $(-1) + 1$
$(0, 1)$	$H-1, 1$	positive signed overflow without carry
$(1, 1)$	H, H	two least signed values and an unsigned carry

At width one, the interpretation interval is too small for this same four-witness pattern, so the width premise matters. A test generator should either exclude that width from a four-class obligation or give it its own expected partition.

Counts, witnesses, and formulas test different failures

The universal Boolean proof can reject a wrong overflow formula. A complete small-width enumeration can reject a missed input. Expected partition counts can reject a checker that visits rows but classifies them incorrectly. Retained witnesses can make each class replayable. No one layer makes the others redundant because each exposes a different failure mode.

The boundary cases are worth naming. Adding one to the greatest signed positive word sets V and produces the least signed word. Adding negative one to the least signed word may set carry while clearing V , because the signed sum remains inside the signed range. Adding the two least signed words sets both conditions at widths above one. These examples defeat the vague rule that overflow means the high bit changed: many legitimate sums change the high bit, and some overflowing sums wrap to a word with the same high bit as one operand.

8.5 Subtraction, borrow, and comparison

Subtraction stores

$$r = x - y \bmod 2^w.$$

Ao defines C as *no unsigned borrow*: it is one exactly when the unsigned reading of x is at least that of y . This polarity must be printed because other conventions may name or invert the condition differently.

At width four, 0011-0101 stores 1110. Since unsigned 3 is below 5, a borrow was needed and Ao clears C . Conversely, 0101-0011 stores 0010 and sets C . In both cases, Z and N come from the stored result.

Signed subtraction overflows exactly when the input signs differ and the result sign differs from the minuend's sign. Subtracting a negative number from a nonnegative one can become too positive. Subtracting a nonnegative number from a negative one can become too negative. Same-sign subtraction cannot cross a signed boundary in that way.

Ao's `cmp rs1, rs2` performs the subtraction condition calculation without writing the result to an ordinary register. Its later signed less-than predicate is $N \neq V$. Its unsigned lower-than predicate is $\neg C$. Equality is Z . These formulas recover comparisons from the retained summary.

Why the sign bit alone is not signed less-than

At width four, compare 0111 with 1111. The signed readings are 7 and -1, so 7 is not less. The stored subtraction is 0111-1111=1000, whose high bit sets N . Signed overflow also sets V , so $N \neq V$ is false. Reading N alone would give the wrong answer.

The correction by V is the compact trace of wraparound. When signed subtraction stays in range, the result sign tells the ordering. When it overflows, that sign is reversed from the mathematical ordering, and the overflow bit reverses the decision back.

8.5.1 Three readings of subtraction

The same stored subtraction supports three claims. As a word equation,

$$r \equiv x - y \pmod{2^w}$$

always holds. As an unsigned natural subtraction, $r = x - y$ holds without wrap exactly when $\langle x \rangle_u \geq \langle y \rangle_u$. As a signed integer subtraction, the stored signed reading equals the mathematical difference exactly when $V = 0$. A theorem that promises correct subtraction must choose among these meanings.

Subtraction may also be implemented algebraically as addition of a two's-complement inverse:

$$x - y \equiv x + (\sim y) + 1 \pmod{2^w}.$$

This identity explains why an arithmetic unit can reuse addition machinery, but it does not settle flag polarity. The final outgoing carry of this addition is one exactly when no unsigned borrow is needed under Ao's convention. An ISA that defines a borrow flag with the opposite polarity must invert that fact at its architectural boundary.

Unsigned comparison from subtraction

Let $a = \langle x \rangle_u$ and $b = \langle y \rangle_u$. If $a \geq b$, the ordinary difference lies in $[0, 2^w - 1]$, so modular reduction changes nothing and Ao sets no-borrow C . If $a < b$, the negative difference is represented by adding 2^w , which is precisely the wrap associated with a borrow, and Ao clears C . Thus $\neg C$ is equivalent to unsigned less-than and C to unsigned greater-than-or-equal.

Equality needs less retained information. Modular subtraction stores zero exactly when the two width- w words are identical, so Z recovers word equality without choosing a signed or unsigned reading. Signed and unsigned order disagree on some pairs, but equality does not: both readings are one-to-one over the same finite set of words.

Negation reveals a special signed boundary. The least signed word represents -2^{w-1} . Its mathematical negation 2^{w-1} is outside the range, so modular negation returns the same word and sets signed overflow under a policy that reports it. Every other signed word has a negation inside the range. This is why an absolute-value routine needs an explicit policy for the least signed input.

One subtraction, two orders

At width eight, compare FF with 01. Unsigned 255 is greater than 1, so no borrow is needed and $C = 1$. Signed -1 is less than 1, and the signed predicate $N \neq V$ is true. The same subtraction summary answers both questions because the branch chooses a different formula.

8.6 Bitwise logic

For width- w words, AND, OR, and XOR apply the corresponding Boolean operation independently at each bit position. NOT complements every bit. This pointwise definition gives algebraic laws without choosing a signed or unsigned reading. For words x, y, z :

$$x \text{ xor } x = 0, \quad x \text{ and } x = x, \quad x \text{ or } 0 = x,$$

$$x \text{ and } (y \text{ or } z) = (x \text{ and } y) \text{ or } (x \text{ and } z).$$

Each word law follows by fixing a bit position and applying the one-bit Boolean law there. This proof pattern is useful: pointwise equality of all w bits implies equality of the words. It avoids interpreting the whole word as an integer when the operation itself is defined on bits.

Masks give these laws practical roles. AND with a mask selects positions where the mask has ones and clears the rest. OR forces selected positions to one. XOR toggles them. If masks m_1 and m_2 have no common set bit, then XOR updates by those masks commute. If they overlap, they still commute because XOR is associative and commutative, but replacing XOR with assignment to selected fields may not. The exact operator, not the visual idea of “changing some bits,” determines the theorem.

Logic also connects representation domains. XOR is addition in each bit over the two-element field, but ordinary word addition is not bitwise XOR because it propagates carries. The expressions agree exactly when no bit position generates a carry that affects a higher position. For nonnegative word readings, one useful condition is

$$x \text{ and } y = 0.$$

Under that condition, $x + y = x \text{ xor } y$ as width- w words. Without it, $0001 + 0001$ gives 0010, while XOR gives 0000. This near miss is a compact example of why an algebraic rewrite needs a premise.

No common one-bit means XOR equals addition

At the low bit, no input pair contains two ones, so it generates no carry. By induction, assume no carry enters position i . The mask premise says that the two input bits there are not both one, so the full adder emits their XOR and no carry. Every result bit therefore equals XOR, and no final carry is lost.

NOT illustrates a different bridge. Complementing every bit of x gives the width- w word whose unsigned reading is $2^w - 1 - \langle x \rangle_u$. Adding one produces the modular additive inverse. Hence

$$-x \equiv (\sim x) + 1 \pmod{2^w}.$$

That identity supports subtraction hardware and bit-level proofs. It does not say that bitwise NOT alone is arithmetic negation.

8.6.1 Condition policies for logic and shifts

Ao's AND, OR, XOR, and NOT instructions set Z and N from the result and clear C and V . The clearing is a policy, not a theorem about Boolean algebra. Another ISA may preserve, clear, define, or leave selected condition bits undefined for a corresponding operation.

The distinction matters after a sequence. Suppose an addition sets carry, a logical operation produces the same destination in two candidate programs, and a later branch reads carry. If one logical semantics clears carry while another preserves it, destination-only tests miss the difference.

Shifts add a further question: which bit was last shifted out? Ao uses the low unsigned value of $rs2$ modulo w as the count. A zero-count shift preserves the source result and clears C . A nonzero shift writes the last bit removed into C , sets Z , N , and clears V .

For width eight, shifting 10110001 left by three produces 10001000. The last bit removed is the original bit 5, which is one, so $C = 1$. Logical right shift fills from the high side with zeros. Arithmetic right shift repeats the original high bit. The two right shifts agree for a source high bit of zero and can differ when it is one.

Follow the bit that becomes carry

Write the source bits above positions 7 through 0. For left shifts by 1, 2, and 7, circle the last bit that leaves the word. Repeat for logical right shifts. Then test count zero separately; do not infer its condition rule from a diagram showing movement.

Large and zero shift counts are frequent semantic fault lines. Ao's modulo-width rule is part of its model. RV64I and x86-64 selected forms have their own count rules. A cross-ISA theorem must either relate those rules on a restricted domain or expose the difference.

8.6.2 The domain of a shift count

For a logical left shift by a mathematical count k with $0 \leq k < w$, the stored result has unsigned reading

$$(\langle x \rangle_u 2^k) \bmod 2^w.$$

For a logical right shift in the same count range, its reading is

$$\left\lfloor \frac{\langle x \rangle_u}{2^k} \right\rfloor.$$

These clean formulas do not define what happens for $k = w$, larger counts, or a negative mathematical count. The instruction semantics must first map its encoded or register-supplied count into an effective count domain.

Ao chooses the low unsigned count modulo w . At width eight, raw counts 0, 8, and 16 therefore all have effective count zero. A theorem restricted to raw counts below eight can use the simple formulas directly. A theorem over every input word must include the modulo operation. Confusing raw and effective counts is especially dangerous in cross-ISA work because two machines can agree on small counts and diverge exactly at the boundary.

Arithmetic right shift needs a signed reading. For an effective count below the width, it behaves like division by 2^k rounded toward negative infinity under the usual sign-extension rule. That rounding matters: arithmetic shift of -3 by one produces -2, whereas integer division specified to truncate toward zero would produce -1. Replacing a division with a shift therefore needs a premise or correction term, not just a claim that both operations divide by a power of two.

The carry bit from a nonzero shift is yet another output. For left shift by k , Ao records original bit $w - k$; for right shift it records original bit $k - 1$. The formulas apply only after proving $1 \leq k < w$. Count zero follows its own rule and cannot index either expression safely.

Normalize the count before indexing a bit

An implementation that calculates the shifted-out bit from the raw count may index outside the word even when the result shift masks that count correctly. Derive the effective count once, branch on zero, and use the same value for both the result and condition calculation.

8.6.3 Rotation preserves bits

A shift discards bits at one edge and introduces fill bits at the other. A rotation returns the departing bits at the opposite edge. For an effective left-rotation count $0 \leq k < w$, one mathematical form is

$$\text{rol}_w(x, k) = ((x \ll k) \text{ or } (x \gg (w - k))) \bmod 2^w,$$

with count zero handled separately so that $w - k$ is not used as an invalid word-width shift. The right rotation is symmetric.

Rotation is a permutation of bit positions. It therefore preserves the number of one-bits and is invertible: rotating left by k and then right by the same effective count recovers the original word. A logical shift is generally not invertible because discarded bits are absent from its result. This difference matters in hashing, cryptographic mixing, bit-field layout, and proofs that need to recover an input.

An ISA may also rotate through a carry flag, making the flag an additional bit in the permutation. That operation has a different state space from ordinary word rotation. The count normalization, flag input, and flag output must be included before the two can be compared.

8.7 Multiplication and division

The product of two width- w unsigned readings can require $2w$ bits. Let

$$P = \langle x \rangle_u \langle y \rangle_u = P_{hi} 2^w + P_{lo},$$

where $0 \leq P_{hi}, P_{lo} < 2^w$. A low-product instruction stores P_{lo} . An unsigned multiplication overflow predicate says $P_{hi} \neq 0$. A high-product instruction returns P_{hi} , which is valuable for arithmetic over integers stored in several words. These are different projections of one exact doubled-width product.

For signed operands, sign-extend both readings to the doubled width before multiplying. A low width- w product is the same bit pattern for signed and unsigned multiplication because both are multiplication modulo 2^w . The high half is not generally the same. Signed overflow is absent exactly when the full signed product is the sign extension of its low half.

The same low product can hide different high facts

At width eight, $\text{FF} * 02$ has low byte FE . Under unsigned readings, $255 \cdot 2 = 510$, whose width-sixteen code is 01FE , so the high byte is 01 . Under signed readings, $(-1) \cdot 2 = -2$, whose width-sixteen code is FFFE , so the high byte is FF . The low byte alone cannot recover which doubled-width reading was intended.

Shift-and-add multiplication follows the positional expansion of the multiplier. If bit y_i is one, add $x2^i$ to the partial product. Booth's signed technique instead uses transitions in runs of multiplier bits to select additions and subtractions in a complementary representation (Booth 1951). Modern implementations may use other recodings, partial-product trees, iterative units, or specialized arrays. The architectural product relation does not select one internal method.

Division produces a quotient and remainder. For unsigned divisor $d > 0$, Euclidean division gives unique q, r such that

$$n = qd + r, \quad 0 \leq r < d.$$

A machine instruction must also define its finite operand widths, which result pieces it stores, and what occurs when $d = 0$. Signed division adds a rounding rule and a special boundary. With quotient rounded toward zero, the remainder has the dividend's sign or is zero. Dividing the least signed value by -1 asks for a positive result outside the signed range.

Those two inputs—zero divisor and least-signed divided by negative one—show why division cannot be specified as an unrestricted ordinary function into one same-width quotient. An architecture might trap, return fixed values, define the result through a larger destination pair, or use another declared policy. RV64's M extension and x86-64 do not use identical exceptional behavior, so a cross-ISA theorem must either exclude these inputs or relate their distinct outcomes (RISC-V International 2026e; Intel Corporation 2026b).

A bit-vector division operator has its own total rule

A solver's fixed-bit-vector division is total according to its logic. A source language or ISA may trap, return a specified word, or leave a case outside a selected model. Reusing the solver operator without an adapter can silently prove the wrong exceptional behavior.

Multiplication and division also widen the economic range. A processor may build a fast pipelined multiplier, reuse a smaller iterative unit, omit an operation from a small core, or place wide arithmetic in a vector or accelerator block. The right choice depends on area, energy, throughput, latency, workload, and software compatibility. Instruction availability does not imply one cycle or one circuit, and instruction absence does not imply that programs cannot compute the operation.

8.8 Condition state in three machines

The state paths differ across our machines.

RV64I base ADD and SUB write the destination register but no x86-style arithmetic flags. `slt rd, rs1, rs2` writes one when the signed reading of `rs1`

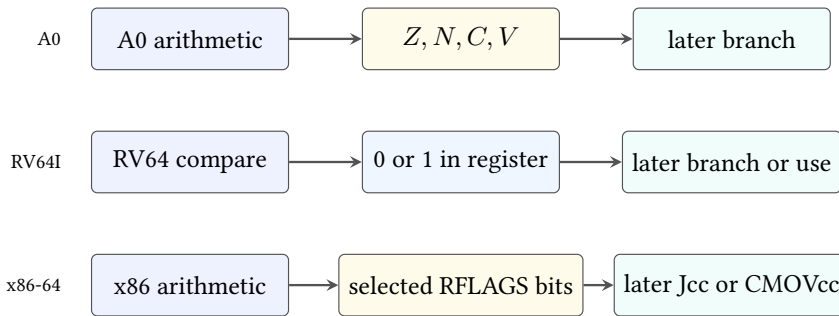


Figure 8.3: Selected ways to preserve a decision for later use. Ao and x86-64 write condition state; RV64I can write an explicit Boolean register.

is less than that of `rs2`, otherwise zero. `sltu` uses unsigned readings. Branch instructions can compare registers directly. These rules come from the pinned base ISA specification (RISC-V International 2026e).

x86-64 selected `ADD`, `SUB`, and `CMP` forms update arithmetic status flags including `CF`, `ZF`, `SF`, and `OF`. `CF` records the unsigned carry or borrow convention of the selected operation; `OF` records signed overflow; `ZF` records zero; `SF` records the result high bit. Later conditional forms interpret combinations of those bits. The exact affected and undefined flags remain form-specific in Intel’s pinned instruction reference (Intel Corporation 2026b).

Ao’s four names resemble that x86 subset, but resemblance is not identity. Ao deliberately defines one no-borrow polarity for subtraction and explicit policies for logic and zero-count shifts. A relation must map predicates after checking these definitions, not map letters because they look familiar.

Compare information, not storage shapes

An Ao condition bit, an RV64 zero-or-one register, an RV64 direct branch predicate, and an x86-64 flag formula can represent the same decision. A cross-machine relation should equate that decision on the declared states. It need not invent identical condition registers on all three machines.

8.8.1 Ao condition policy

For Ao addition and subtraction, `Z` and `N` come from the stored result. Addition sets `C` from the widened unsigned sum and `V` from signed range crossing. Subtraction sets `C` for no unsigned borrow and `V` from signed subtraction overflow. Compare performs the subtraction calculation and writes only the four conditions, leaving ordinary registers unchanged.

Ao logic operations write their result, set `Z`, `N`, and clear `C`, `V`. Its shifts set `Z`, `N`, clear `V`, and define `C` from the last removed bit for a nonzero effective count.

Count zero clears C under the Ao policy. These choices are deliberately complete. A checker never has to guess whether a condition is preserved, cleared, derived, or outside the model.

The frame is as important as the formulas. An arithmetic register instruction reads its named sources and entry condition state only when the operation requires it, writes the named destination and declared condition bits, advances the program counter, and preserves every other register and all memory. A compare does not write a destination register. A faulting arithmetic extension, if later added, must state whether it commits any condition state before the fault. A correct result word with an incorrect frame is still an incorrect transition.

8.8.2 RV64I conditions as data

Base RV64I ADD and SUB store low 64-bit results and do not write an arithmetic flag register. Signed and unsigned comparisons are explicit operations. For unsigned words x, y , let $r = x + y \bmod 2^{64}$. The carry identity gives

$$C \quad \text{iff} \quad r < x$$

when the comparison is unsigned. Thus an ADD followed by SLTU can place carry in a general register. The comparison could use y instead; both forms need a proof that relates the selected operand to the wrapped result. If the destination replaces an operand, register allocation must preserve whichever value the comparison still needs.

Adding an incoming carry needs care because $x + y + c$ can cross the word boundary in either of two stages. One sequence computes $t = x + y$, records $c_1 = [t < x]$, computes $r = t + c$, records $c_2 = [r < t]$, and returns c_1 or c_2 . Since c is zero or one, the two stage overflows cannot both require a value larger than one final carry. A proof can instead compare the exact widened sum directly. The staged version explains the ordinary-register dependencies an implementation must preserve.

RV64I branches compare two registers directly for equality or signed/unsigned order. SLT and SLTU are useful when the Boolean result must survive for later arithmetic or selection. A direct branch can avoid that stored Boolean when the decision is needed immediately. These are instruction-count and register- pressure choices, not differences in the mathematical predicate (RISC-V International 2026e).

8.8.3 x86-64 condition flags

For selected x86-64 ADD forms, CF records unsigned carry and OF records signed overflow; ZF and SF describe the stored result. ADC also reads entry CF as an incoming addend. SUB and CMP define the subtraction conditions, while SBB reads CF according to x86's borrow convention. The exact set of affected, preserved, or undefined flags depends on the named instruction and form (Intel Corporation 2026b).

Conditional instructions interpret formulas, not isolated letters. Equality uses ZF. Unsigned below uses CF; unsigned below-or-equal uses CF or ZF. Signed less-than uses SF $\neq$ OF; signed less-than-or-equal also admits ZF. The same subtraction therefore supports two orders only because later instructions select different formulas.

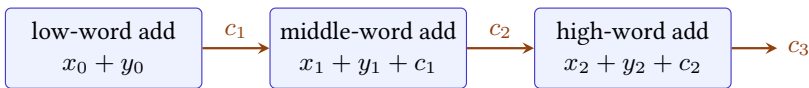
Implicit state can shorten an encoding and carry information between adjacent instructions. It can also create a dependence that is easy to miss in a destination-only data-flow graph. An intervening flag-writing instruction can break ADC, SBB, a conditional move, or a branch even when no named general register changes. Conversely, an instruction that avoids changing selected flags may preserve a live carry chain. A proof and an optimizer must use the manual's exact flag footprint, not the broad label "arithmetic."

One predicate, three storage paths

To retain unsigned $x < y$, Ao may execute compare and preserve $\neg C$ in condition state. RV64I may place the result of SLTU in a general register. x86-64 may execute CMP and retain CF. A relation can equate the Boolean decision while allowing different registers, flags, program lengths, and preserved state.

8.9 Carry chains

A word is often only one limb of a larger integer. Let a three-word unsigned number use little-indexed limbs x_0, x_1, x_2 , each of width w . Add the low limbs, store result limb r_0 , and pass carry c_1 into the next addition. Continue until the high limb produces final carry c_3 .



Each carry is data for the next word, whether stored in a flag or an ordinary register.

Figure 8.4: Multiprecision addition makes each carry a live input to the next word. The storage location differs by ISA, but the mathematical recurrence is the same.

For limb i , define the exact widened sum

$$s_i = \langle x_i \rangle_u + \langle y_i \rangle_u + c_i = c_{i+1}2^w + \langle r_i \rangle_u,$$

with $c_0 = 0$. Multiplying the equation for limb i by 2^{wi} and summing makes every internal carry cancel. The result is

$$X + Y = c_3 2^{3w} + R,$$

where X, Y, R are the three-word unsigned readings. The local widened identity therefore composes into a whole-number theorem.

Condition liveness, meaning whether a later step reads a condition before another step replaces it, is concrete here. On a flag-based machine, an instruction inserted between two limbs must not destroy the carry before the next add-with-carry consumes it. On a machine without an architectural carry flag, code may calculate the carry into an ordinary register and add it to the next limb. The state shapes differ, but both proofs need the same recurrence and must protect c_{i+1} across the intervening steps.

Signed multiple-word addition usually uses the same limb operations. Signed overflow is judged at the high end from the full-width operand and result signs, not by combining the per-limb overflow bits. Internal limbs are pieces of one large representation; interpreting each as a separate signed integer answers the wrong question.

Two-limb carry composition

Write the low identity as $x_0 + y_0 = c_1 2^w + r_0$ and the high identity as $x_1 + y_1 + c_1 = c_2 2^w + r_1$. Multiply the high equation by 2^w and add the low equation. The terms $c_1 2^w$ cancel from opposite sides, leaving the exact two-limb sum with low limbs r_0, r_1 and final carry c_2 .

8.10 Condition liveness

Consider two instruction sequences that always leave the same destination word. One writes the correct conditions; the other leaves old conditions in place. Under an observation of the destination alone, they can be equivalent. Before a condition-reading branch, they need not be.

Take width-eight addition of FF and 01. Both sequences store 00. The correct Ao conditions include $Z = 1, C = 1, V = 0$. If the faulty sequence leaves $C = 0$, a later branch.hs chooses another path. The destination-only claim and the flag-sensitive counterexample are both valid because they use different observations and contexts.

The distinction determines legal optimization as well as good testing. A compiler may replace an instruction with a flag-different sequence only when the surrounding program cannot observe those flags before they are overwritten. That fact can come from liveness analysis, a typed context, or a theorem whose observation omits the condition state.

Dead conditions require evidence

Conditions are not outside the observation merely because the current basic block does not print them as outputs. A successor branch, conditional move, carry chain, or surrounding context may read them. Prove that the relevant condition state is dead or preserve it.

The same reasoning applies to explicit Boolean registers. If an RV64 comparison result is stored in a scratch register that the observation omits, changing it may be harmless in one closed routine. If later code reads the register, the difference is visible. Hidden versus explicit state changes the analysis tools, not the need for a scope.

8.10.1 Backward reasoning about conditions

To decide whether a condition write matters, begin at the later use and reason backward. A branch that reads Z makes the most recent reaching definition of Z live. Earlier writes are dead if every path to the branch replaces Z first. If one path preserves an earlier value, that definition remains live on that path. The same analysis applies separately to C , V , and N ; treating the whole condition register as one indivisible value can be safe but unnecessarily conservative.

Partial condition writes complicate the picture. If an instruction defines Z , N while preserving C , a later unsigned branch may depend on a carry created several instructions earlier. If the architecture leaves a condition undefined, a theorem cannot retain its old value unless the selected semantics explicitly chooses that behavior. Undefined, preserved, and cleared are three different policies.

An optimization proof can encode liveness as an observation boundary. Show that both sequences agree on every location read before its next common overwrite. Conditions absent from that set may differ. This local relation is strong enough to justify replacement in the chosen context without making the false global claim that the instruction sequences have identical complete states.

A carry survives an unrelated-looking operation

Suppose an addition creates a carry for the next limb. A candidate replacement inserts a logical instruction before the add-with-carry. If that logical form clears C , the arithmetic destinations produced so far can still match, yet the next limb differs. Backward liveness from the add-with-carry identifies C as a required observation across the inserted instruction.

8.11 Policies for out-of-range results

The ISA defines an instruction transition. A programming language, library, or intermediate representation may place another contract above it. Several common policies begin from the same exact mathematical result but expose different outcomes.

Wrapping arithmetic. Store the residue modulo 2^w . This is useful for counters, hashes, low-level encodings, and algorithms stated in a finite ring.

Checked arithmetic. Return both a success/failure indication and, on success, a result inside the selected range. The caller decides how to handle failure.

Trapping or strict arithmetic. Transfer control or otherwise fail when the selected range is crossed. The failure behavior is part of the contract.

Saturating arithmetic. Clamp a result below the range to its minimum and a result above the range to its maximum. This avoids wraparound but changes the algebra.

Overflow-reporting arithmetic. Return a wrapped word together with a Boolean that reports the range crossing.

Current Rust application programming interfaces (APIs) make these families explicit in names such as `wrapping_add`, `checked_add`, `overflowing_add`, `strict_add`, and `saturating_add` (Rust Project Developers 2026). An interface name is not a theorem, but a good name forces the caller to choose a policy that an unqualified “add” would conceal.

Saturation is common where a wrapped result would be physically or perceptually absurd. An unsigned brightness value above its maximum may remain at maximum rather than return near zero. A controller may clamp an actuator request. Packed audio, image, and signal operations can apply saturation to each lane. Yet saturation loses the modular group laws: there is generally no additive inverse, and regrouping can change a result. An optimization valid for wrapping words may be false for saturated values.

Reassociation fails under saturation

Suppose signed values range from -8 through 7 and every addition saturates. Then $(7+7)+(-7)$ becomes $7+(-7) = 0$. But $7+(7+(-7))$ becomes $7+0 = 7$. Ordinary integer addition and modular word addition are associative; stepwise saturated addition is not.

8.11.1 Wraparound and security boundaries

Modular arithmetic is not itself a defect. The defect appears when one layer uses a modular result while another assumes an ordinary integer fact. A length calculation gives a common pattern. Suppose a message contains n records of s bytes plus a header of h bytes. The mathematical allocation size is

$$L = h + ns.$$

If L is computed in a width- w word without a range check, the stored length may be much smaller than the mathematical length. An allocator can then reserve the wrapped size while a later copy follows the original record count. The arithmetic mismatch becomes a memory-safety problem.

The proof obligation is not merely “check for overflow.” It must connect the check to every later interpretation:

1. establish that n , s , and h are in their declared domains;
2. prove that ns fits the selected width before adding the header;
3. prove that the final sum fits the allocation-size type;
4. allocate exactly that checked size; and
5. prove that every later write uses the same accepted bounds.

Checking only the final stored word against one operand can miss a wrapped multiplication. Widening after a narrow multiplication is also too late: the discarded high product bits cannot be recovered. Widen the operands before the operation, use a checked primitive, or prove a bound that makes the narrow calculation exact.

MITRE Corporation’s current CWE-190 entry records consequences that include data corruption, resource errors, altered control, protection bypass, and chains into memory corruption when wraparound affects sizes or bounds (MITRE Corporation 2026). The classification does not mean every modular counter is vulnerable. It identifies a recurring interface error: the program assumes a result that always increases or fits the integer range where the selected calculation does not guarantee one.

Economic calculations have the same semantic risk even when memory safety is not involved. Quantities, prices, rates, and totals may use different scales and widths. Saturation can hide lost magnitude; wraparound can reverse an ordering; silent truncation can erase fractional or high-order information. A sound design states units, scale, range, rounding, and failure policy before choosing an instruction sequence. A larger type reduces some risks but does not replace a bound on adversarial or long-running inputs.

A later wide cast cannot restore an earlier lost high part

If a narrow product wraps and is then widened, the wide value is only an extension of the low word. To preserve the mathematical product, widen before multiplication or use an operation that returns the full high and low parts.

8.11.2 Compiler range facts

LLVM integer addition without a no-wrap marker returns the modular result. Its `nuw` and `nsw` markers add claims that unsigned or signed wrapping does not occur. If the corresponding claim is violated, the result is poison under the current IR semantics (LLVM Project 2026d). Poison is not an x86 overflow flag and not an immediate hardware exception. It is an IR value whose allowed propagation gives optimizers freedom, subject to the IR's exact rules.

That freedom can justify transformations unavailable for arbitrary modular words. Over mathematical integers, $x + 1 > x$ is always true. Over width- w unsigned wrapping words, it is false at the greatest word. If an IR operation has a valid unsigned-no-wrap promise, the boundary case is excluded and the comparison may simplify. If the promise was inferred incorrectly, the optimization can expose the error far from the original addition.

No-wrap facts also support range analysis, loop reasoning, address calculation, and strength reduction. They are economically valuable because better proofs can remove tests or expose better machine instructions. They are also a verification burden: front ends, optimizers, and back ends must preserve the contract across every rewrite. A condition flag observed in assembly and a no-wrap fact in IR can describe related arithmetic, but a compiler needs a proved lowering rule between their levels.

Instruction selection must account for condition state as well. On x86-64, a candidate instruction may produce a desired word while defining flags differently from the source pattern. That replacement is legal only when the different flags are dead or separately repaired. On RV64I, comparisons and carry idioms often use general registers and direct branches instead. The dependence is more visible in the register graph, but it still consumes an instruction result and perhaps a register name.

8.11.3 Dimensions of arithmetic cost

Latency asks how long one dependent result takes. Throughput asks how often an implementation can begin or finish operations. Area asks how much hardware is committed. Energy asks how much physical activity a workload causes. Code size and register pressure ask what software must spend. Verification and compatibility add engineering and long-term costs.

Addition, multiplication, and division can occupy very different points in that space. Horowitz’s 2014 technology-specific comparison illustrates that arithmetic and data movement have materially different energy costs, and that the cost depends on width and location (Horowitz 2014). The dated quantities should not be copied as universal constants. The durable lesson is to state which operation, precision, storage level, and technology a cost claim concerns.

Multiprecision arithmetic makes the dependencies explicit. A carry chain across limbs can limit parallel work. Wider instructions can reduce the number of limbs but may use rarer or more expensive resources. Cryptographic code may prefer regular instruction sequences and avoid data-dependent branches or memory addresses. These choices balance arithmetic latency, instruction availability, portability, side-channel goals, and the effort required to verify the implementation.

Branch-free is not a complete constant-time claim

Removing a condition-reading branch eliminates one possible source-level variation. It does not prove equal execution time, equal cache behavior, or equal power on a physical processor. A constant-time theorem needs an observation model and implementation assumptions that include the effects it promises to hide.

Packed and vector arithmetic introduce one more policy axis. Does overflow wrap or saturate per lane? Does a condition summarize each lane, all lanes, or no lane? Are inactive lanes preserved, zeroed, or ignored? This chapter’s scalar laws can be lifted lane by lane only after those questions are answered. Chapter 16 treats that extension as a larger state and observation problem rather than pretending that one scalar flag covers it.

8.12 Axeyum route

The live Axeyum checkout at revision a9991fda already contains more than a generic solver. Its Rust term layer implements fixed-width addition, subtraction, multiplication, comparisons, shifts, rotations, extension, and unsigned and signed overflow predicates for addition, subtraction, and multiplication. The bit-vector lowering constructs Boolean networks, including a ripple-carry addition path. Python bindings expose the operations. Property tests compare the overflow builders with independent unbounded Python integer definitions across generated widths and boundary values.

That is strong reusable substrate, but it is not yet this chapter’s artifact. The checkout has no Ao arithmetic record containing result and Z, N, C, V , no instruction-specific logic and shift policies, no four-quadrant teaching census, no lesson about live conditions, and no thin RV64I or x86-64 adapter. A property test

over a term builder also has a different assurance claim from a solver proof export, an architectural transition check, or a width-parametric kernel theorem.

The first book-facing implementation goal is therefore to assemble, rather than duplicate, the pure substrate. Define a width-parametric Ao arithmetic result that contains the result word and every declared condition. At small widths, enumerate every input pair and compare that record with independent mathematical definitions. At selected larger widths, ask a solver for a counterexample to each equivalence: carry versus extended unsigned sum, signed overflow versus the sign formula, subtraction no-borrow versus unsigned order, and compare predicates versus signed or unsigned readings.

Every solver model should replay through the executable function. Negative controls should invert carry, confuse carry with overflow, use N instead of $N \neq V$ for signed comparison, preserve a logic flag that Ao clears, and handle count zero incorrectly. Each mutation has a small concrete witness, so a green route that cannot reject it is not exercising the promised rule.

The first solver obligations can be small and independent of instruction decoding. For arbitrary width- w bit vectors x, y , build an extended $(w + 1)$ -bit sum by zero-extending both operands. Compare its low w bits with the executable result and its high bit with executable carry. For signed overflow, compare the executable bit with the high-bit Boolean formula. Ask the solver for a state where either equality fails.

Subtraction needs its own obligation rather than reuse by name. Compare executable no-borrow with unsigned $x \geq y$, and executable signed less-than with the signed bit-vector predicate. Shift checks quantify over a raw count, derive Ao's effective count, and split zero from nonzero before stating the shifted-out-bit formula. These separate formulas make a failed control diagnostic: a carry mutation should not be reported as an unrelated signed-overflow failure.

At a teaching width, a census should retain counts as well as a pass bit. For addition, partition every ordered pair by (C, V) , then retain one witness from each class that contains a pair. Requiring all four classes prevents a checker from passing only easy rows. A second census for subtraction should cover equality, unsigned less-than, signed less-than, and the cases where the two orders disagree.

The Python layer should assemble these as explicit objects rather than hide them behind a single Boolean helper: an input domain, mathematical reference, executable function, observation fields, enumerated census, solver query, replayed witness, and semantic digest. The Rust layer already owns the width-safe term operations; the book artifact must add the architectural record and serialized evidence checks. Python can assemble lessons and display tables without becoming an unrecorded semantic authority.

From exhaustive rows to a universal formula

Width four has 256 ordered addition pairs. Exhaustive checking can classify all of them by C and V . The Boolean overflow proof covers every positive width. Axeyum can connect the two: enumerate the teaching instance, solve the general bit-vector negation at selected widths, and later reconstruct a width-parametric theorem in a stronger kernel route. Keep those assurance levels separate in the evidence record.

Thin RV64I and x86-64 adapters come after the pure laws. The RV64 adapter maps selected comparison results and arithmetic destinations. The x86 adapter maps selected flags for exact operand widths and forms. An evidence manifest must name the observation: destination only, selected conditions, or complete in-scope state.

8.13 Arithmetic beyond integer words

The same stored bits can participate in other numeric contracts. A fixed-point format assigns an implicit scale. If a word represents integer q with scale 2^{-f} , its numeric value is $q2^{-f}$. Addition requires aligned scales; multiplication doubles the fractional-bit count before a declared rescaling and rounding step. The underlying word operations remain finite, but a useful theorem must include scale and rounding.

Decimal arithmetic serves domains in which powers of ten and decimal rounding are part of the contract. Its digits, encodings, carries, and exceptional rules are not obtained merely by reading a binary word as unsigned. Financial software may also require exact decimal quantities, bounded integer minor units, or arbitrary precision. Which representation is right depends on the unit, range, rounding law, legal or business rules, and required audit trail.

Floating-point arithmetic represents a sign, a **significand**, meaning the field that holds the precision bits, and an exponent, and it has rounding modes, infinities, not-a-number values, signed zeros, and named exceptions. Its addition is not modular word addition, even though a floating-point value is encoded in bits. IEEE 754 standardizes a broad floating-point contract (IEEE Computer Society 2019); Chapter 16 explains why adding that contract expands both state and evidence.

Arbitrary-precision integers remove a fixed mathematical range only by making representation length and allocation part of the computation. They can still fail through unavailable memory, resource limits, or conversion back to a fixed width. Their running time and memory access can depend on operand size and value. “No overflow” therefore does not mean “no failure, cost, or side-channel question.”

Packed and vector formats divide storage into lanes. A scalar 64-bit addition propagates carry across every bit. Four 16-bit lane additions block carries at lane boundaries and may apply four separate saturation decisions. Both use 64 physical

input bits, but they implement different algebras. A proof must know the lane partition before reusing a scalar identity.

Representation is part of the arithmetic operation

The verb “add” does not identify a complete function. Width, integer or other format, signed reading, scale, lane shape, rounding, overflow policy, and failure outcome determine what inputs and outputs mean.

Conversion between these domains is an operation, not a change of notation. Converting a wide integer to a narrow word may reject, wrap, or saturate. Converting a scaled fixed-point value may round. Converting an integer to floating point may lose low-order precision even when the magnitude remains finite. Reinterpreting the same bits performs none of those numeric conversions; it changes only the reading. A complete conversion theorem names the source domain, destination domain, rounding or range policy, exceptional outcome, and information that can no longer be recovered. Tests should include both exact conversions and the first values on each side of every rounding, range, lane, or exceptional boundary named by that policy.

8.14 Where the model stops

This chapter proves a scalar fixed-width integer core. It compares saturation, fixed point, decimal, floating point, arbitrary precision, and packed lanes only well enough to mark their different contracts. It does not provide their complete transition systems or proof libraries. It also does not claim that one flag policy covers every x86-64 instruction form or optional RISC-V extension.

It also treats an instruction as an atomic state transition. Hardware may compute condition information through internal circuits unlike these formulas. The architectural theorem concerns the visible result, not the internal processor route that produced it.

The reward for this restraint is a reusable core. A few finite-word identities explain wraparound, comparisons, branches, and the legality of later program transformations. The machine keeps only bits. A proof can still recover the mathematical boundary those bits crossed.

The chapter’s recurring method is deliberately uniform: widen when the exact unsigned result matters, interpret high bits when signed range matters, and name every condition-state policy that later code can observe. That method scales from a four-bit teaching table to a multiple-word routine and from a reader proof to a solver obligation without changing the meaning of the claim.

8.15 Exercises

1. **Execute.** For every ordered pair of width-two words, compute the stored sum and Ao bits Z, N, C, V . Group the rows by the four (C, V) combinations.
2. Give width-eight additions with carry but no signed overflow, signed overflow but no carry, both conditions, and neither condition. Explain each using mathematical ranges.
3. **Prove.** Prove that opposite-sign signed addition cannot overflow. State the signed interval used by the proof.
4. Derive the Boolean high-bit formula for signed addition overflow and check it on all four examples in Figure 8.1.
5. For width four, compute Ao subtraction conditions for 0011-0101, 0101-0011, 0111-1111, and 1000-0001.
6. Show with a concrete pair why N alone does not implement less-than after subtraction. Show how $N \neq V$ repairs the case.
7. Compare logical and arithmetic right shift of 10010000 by counts 0, 1, 3, and 7 under Ao. Include the last shifted-out bit.
8. **Break.** Replace signed overflow with carry in an Ao addition implementation. Give the smallest-width counterexamples you can find in both directions.
9. Construct two sequences with the same destination result but different carry bits. Supply a suffix that observes the difference.
10. Explain how an RV64I slt result and an Ao $N \neq V$ predicate can represent the same signed comparison without identical machine states.
11. **Transfer.** State a relation between one selected x86-64 CMP form and Ao cmp. Include operand width, subtraction polarity, mapped conditions, program-counter relation, and excluded flags.
12. Design an Axyum manifest for the width-four carry/overflow census. Include the independent definition, row count, digest, replay check, and a negative control.
13. Extend a destination-only arithmetic equivalence claim to a condition-sensitive one. List the additional output conditions and tests required.
14. **History.** Read the cited passage from Leibniz's binary account. Separate what it demonstrates about arithmetic from any claim about electronic storage, gates, or instruction sets.

15. **History.** Explain why Shannon's switching analysis and the System/360 condition code belong to different stages of the story. Name the claim that each source can support.
16. **Circuit.** Write all eight rows of the full-adder truth table. Check $a + b + c = 2c' + s$ on every row.
17. **Prove.** Starting from the per-bit full-adder equation, derive the width- w widened addition identity. Show where every internal carry cancels.
18. **Design.** Compare a width-eight ripple-carry adder with a balanced carry tree. Give symbolic dependence depth, then list area, wiring, fan-out, energy, and verification questions that depth alone does not answer.
19. **Break.** Wire one full-adder carry output to the next position after its intended neighbor. Find the smallest input that distinguishes the broken circuit from word addition.
20. Prove $x \text{ xor } y = x + y$ when x and $y = 0$. Give a counterexample when the premise is removed.
21. Derive the unsigned-reducing formula for bitwise NOT, then derive $-x \equiv (\sim x) + 1 \pmod{2^w}$.
22. **Mask.** For a width-sixteen word, design masks that select bits 4 through 9, force bits 0 and 15 to one, and toggle every even position. State which operations commute and why.
23. **Prove.** Show that left rotation by k is inverted by right rotation by the same effective count. State the count domain and handle zero.
24. **Break.** Implement rotation with a raw expression containing a shift by $w - k$. Give the boundary input that makes the expression's shift domain invalid even though the intended rotation is defined.
25. Compute the full unsigned and signed width-sixteen products of FF and 02 when the operands begin as width-eight words. Explain why the low byte agrees and the high byte differs.
26. **Prove.** Show that signed and unsigned multiplication always have the same low width- w product word.
27. State and prove a test for absence of signed multiplication overflow using the high half and sign extension of the low half.
28. **History.** From Booth's paper, identify the representation problem and equipment concern that motivated the method. Do not describe the paper as a complete history of multiplication hardware.

29. For unsigned $n = 173$ and $d = 13$, compute quotient and remainder and check the Euclidean identity and remainder bound.
30. **Boundary.** Compare zero-divisor and least-signed divided by negative-one behavior for the selected RV64 and x86-64 forms. State the manual versions and every excluded mode or width.
31. **Solver.** Explain why an SMT fixed-bit-vector division term cannot be substituted directly for a trapping ISA instruction on every input. Describe the adapter or precondition needed.
32. Derive the two-stage RV64I carry idiom using ADD and SLTU. Prove that the final Boolean represents the carry from $x + y + c$, where c is zero or one.
33. **Register pressure.** Give an RV64I carry-producing sequence that replaces one source too early. Repair it and name the extra live value.
34. **x86-64.** For one pinned ADC form, list every explicit input, implicit input, destination, defined flag, and excluded flag. Then state its widened unsigned equation.
35. **Transfer.** Relate an x86-64 unsigned-below condition after CMP to an RV64I SLTU result. Include source-order polarity and all state that may differ.
36. **Liveness.** Draw a short control-flow graph with two paths to a carry-reading instruction. Make the carry definition dead on one path and live on the other. Perform the backward analysis.
37. **Undefined state.** Compare three models of a manual-declared undefined flag: preserve entry value, choose an unconstrained successor bit, and reject later observations. Which machine claims does each model permit?
38. For width-four signed saturation, find counterexamples to additive inverse, regrouping, and distributivity over a second saturated operation. State which algebraic law each example breaks.
39. **Policy.** Design one interface for a sensor accumulator under wrapping, checked, trapping, saturating, and overflow-reporting arithmetic. Explain which failure or boundary behavior a user observes in each version.
40. **Compiler.** Prove that $x + 1 > x$ for an unsigned no-wrap addition. Give the greatest-word counterexample when the no-wrap promise is absent.
41. **Break.** Attach an incorrect LLVM nsw marker to an addition that can overflow. Give an input and a later use through which poison can make the source-level expectation invalid.

42. **Economics.** Compare two implementations of a 256-bit addition: four 64-bit limbs with a serial carry and a dedicated 256-bit unit. Define latency, throughput, area, energy, code-size, and verification measures before arguing for either design.
43. **Security.** Rewrite a conditional arithmetic selection without a source-level branch. Then state why the rewrite alone does not prove constant time on a named physical processor.
44. **Packed arithmetic.** Give a two-lane example where per-lane saturation differs from treating the same bits as one scalar word. Name the extra lane structure required by the semantics.
45. **Axeyum audit.** Locate the live unsigned- and signed-overflow builders, their Python bindings, their independent property-test references, and the Boolean lowering for addition. Record the inspected revision.
46. **Axeyum design.** Specify the missing Ao arithmetic record and a four-quadrant census artifact. Include observations, source definitions, negative controls, solver route, replay, and assurance class.
47. **Synthesis.** Choose one arithmetic replacement across Ao, RV64I, and x86-64. State the input relation, mathematical fact, state projections, failure domain, cost model, and evidence needed for a defensible theorem.

Control Transfer

Most instructions continue with the next instruction in memory. Control-transfer instructions make another future possible. A conditional branch asks a question about the current state and chooses one of two addresses. A jump selects a destination without asking. A call also leaves behind an address from which execution can later continue.

These operations are small enough to fit in one line of assembly, yet they connect nearly every part of the machine model. Their predicates read registers or condition state. Their targets depend on instruction addresses, lengths, and signed displacements. Their repeated execution makes loops. A single error in the target convention can preserve every data result in a one-step test and still send the program into the wrong block.

Control flow is arithmetic over addresses plus a predicate

A direct conditional branch needs three facts: the address used as its base, the signed displacement or absolute destination, and the Boolean predicate that selects the target instead of the next sequential address. State all three before proving where execution goes.

9.1 Why programs choose

The earliest stored-program machines already needed more than a straight list of arithmetic operations. Repetition made a short program perform a long calculation. Conditional transfer let that repetition stop when a counter, remainder, or comparison reached the desired state. Subroutines then made one piece of code serve many callers. The program counter turned stored instructions from a static list into a path selected during execution.

Modern machines retain that foundation even when processors predict several paths, fetch ahead, or execute instructions out of order. Architectural semantics describe the committed path. A proof about a scalar instruction set therefore needs

no branch predictor, but it cannot omit the program counter, the branch predicate, or the possibility of a bad target.

Control transfer serves several present-day purposes:

1. a conditional statement chooses between two computations;
2. a loop returns to a test or body;
3. a jump enters another block without creating a return link;
4. a call records a continuation address and enters a procedure;
5. an indirect transfer obtains its destination from a register or memory;
6. a trap or return crosses a boundary whose rules exceed ordinary scalar control flow.

This chapter handles direct conditional branches, direct jumps, and the link values needed to prepare for procedures. Chapter 10 adds stacks and calling agreements. Indirect calls, privilege changes, exceptions, and speculation remain outside the present model unless a later section names them.

A loop compresses time into an address

A backward edge says that the same finite instruction bytes may act on a new state. The code need not grow with the number of iterations. What grows is the trace: one state after another, each justified by the same small transition rules.

9.1.1 From machine action to reasoning structure

Conditional succession predates the stored-program instruction sets in this book. Turing's 1936 machine changes its configuration according to the current configuration and scanned symbol. The formal machine is not an electronic ISA, but it makes conditional next-state choice part of a mathematical account of computation (Turing 1937a). Later electronic machines had to turn such choice into concrete orders, addresses, and state.

Stored instructions made backward transfer especially powerful. A finite region of memory could direct execution back to an earlier instruction and reuse the same arithmetic and test on changed data. The stored bytes remained finite while the possible trace length depended on input and state. This is the architectural root of the chapter's recurring contrast between program size and execution length.

By 1947, Goldstine and von Neumann used flow diagrams to plan calculations and paired iteration structure with induction assertions (Goldstine and Neumann 1947). The diagram was not merely a picture of instruction addresses. It gave

programmers a level at which paths, tests, and repeated regions could be discussed without copying every machine order.

Subroutines added a different control problem: enter shared code, then recover the caller's continuation. The 1951 EDSAC programming book by Wilkes, Wheeler, and Gill documented closed subroutines and a library built for a working stored-program machine (Wilkes, Wheeler, et al. 1951). An early linkage could construct or store a return transfer in ways unlike a modern link register or stack. The durable abstraction is the continuation address. The storage mechanism changed as architectures and software conventions developed.

The 1960s debate about structured control concerned human reasoning as well as machine power. Böhm and Jacopini established an expressiveness result for selected structured forms (Böhm and Jacopini 1966). Dijkstra's 1968 letter argued that unrestricted jumps made the relation between textual position and execution progress harder for people to follow (Dijkstra 1968). Neither result removed jump instructions from machines. Languages and compilers can present structured source constructs while lowering them to conditional and unconditional transfers.

Compiler analysis then made control flow a directed graph. Allen's 1970 paper expressed flow relationships in graph form so global questions about loops, definitions, and profitable code motion could be answered (Allen 1970). Dominance became one of the central relations; Lengauer and Tarjan later published a fast dominator algorithm (Lengauer and Tarjan 1979). The graph did not replace execution semantics. It organized possible edges so analyses could reason about many executions at once.

Prediction belongs to a still later layer. Once processors overlapped and pipelined work, waiting for every branch decision could leave resources idle. Published prediction studies, including Smith's 1981 comparison of strategies, treated likely path choice as an implementation-performance problem (Smith 1981). The architectural successor remains the one selected by the resolved predicate. Prediction changes which work begins early, how much wrong-path work may be discarded, and what physical effects need a larger observation model.

One arrow can belong to four histories

An arrow may denote a machine's next instruction, a programmer's flow diagram, a compiler's possible graph edge, or a predictor's guess. Those objects can refer to the same source construct, but they carry different evidence and failure meanings.

9.2 Fallthrough, target, and link

Suppose an instruction begins at address p and occupies L bytes. Its **fallthrough address** is the next sequential address, computed as

$$f = p + L.$$

Execution continues there when an ordinary instruction completes or a conditional branch is not taken. For a direct relative transfer, a signed displacement d is combined with a specified base b :

$$t = b + d.$$

The architecture decides whether b is the current instruction address, the fallthrough address, or another quantity, and whether the encoded displacement is scaled before addition.

For a predicate $P(S)$ over state S , the next program counter is

$$pc' = \begin{cases} t & \text{when } P(S), \\ f & \text{when } \neg P(S). \end{cases}$$

A direct jump uses the first line without a predicate. A linking jump also writes a **link value**, normally an address at which a later return can resume. The link is an architectural data result, not merely an annotation on the control-flow graph.

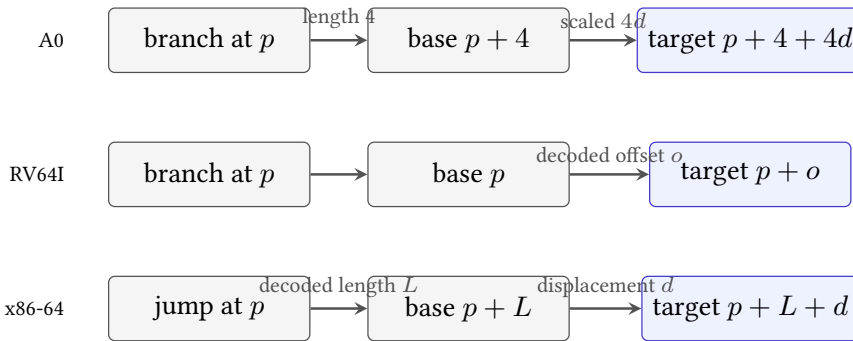


Figure 9.1: Three selected relative-target conventions. Similar assembly displacements do not imply the same base, scale, or instruction length.

Figure 9.1 exposes a common source of off-by-one-instruction errors. Ao deliberately uses $p + 4 + 4d$. Selected RV64I conditional branches add their sign-extended, encoded offset to the address of the branch instruction. Selected x86-64 relative jumps add a sign-extended displacement to the instruction pointer for the following instruction. RV64I instructions in this chapter are four bytes, while the x86-64 instruction length depends on the actual encoding. The source-pinned rules

appear in the RISC-V unprivileged specification and Intel instruction reference (RISC-V International 2026e; Intel Corporation 2026b).

Never infer the target base from the printed operand

An assembler may print a destination label, a byte displacement, or a symbolic expression. The architectural rule operates on encoded fields and a defined program-counter base. Decode the instruction and name that base before doing target arithmetic.

The same address, three calculations

Place a transfer at address 100. An Ao branch with encoded displacement 3 targets $100 + 4 + 4(3) = 116$. An RV64I branch whose decoded offset is 16 bytes targets $100 + 16 = 116$. An x86-64 near conditional jump of length 6 with displacement 10 targets $(100 + 6) + 10 = 116$. All three reach the same address, but no pair uses identical encoded numbers and conventions.

A target calculation that survives relocation

Let an instruction and its target both move by the same address delta k . If the relative displacement is target minus the architecture's base, that base also moves by k . The new displacement is $(t + k) - (b + k) = t - b$. Thus the encoded relative distance is unchanged, provided the moved target still fits the encoded field and alignment and encoding constraints still hold. The cancellation explains why relative branches help relocatable code; it does not prove that every relocation stays in range.

9.3 Choice in the transition relation

Let S be the set of complete architectural states. A deterministic scalar machine step is a partial function

$$\delta : S \rightarrow S.$$

It is partial when some states do not contain a legal next instruction or have already reached a terminal outcome. A conditional branch does not make this function **nondeterministic**, which means that one complete input state can have more than one allowed successor. Once the complete state and decoded instruction are fixed, the predicate has one Boolean value and the rule picks one successor. The programmer may not know that value, but incomplete human knowledge does not give the machine several allowed choices.

A transition relation $R \subseteq S \times S$ is more general. It can model a rule that deliberately leaves a choice open, external input, concurrency, or an abstraction that has forgotten predicate-relevant state. If an abstract state records only the program counter, a conditional block may have two related successors because the abstraction does not retain the register or flags that decide between them. The concrete machine can remain deterministic while its control-flow graph is branching.

This distinction prevents a common proof error. A CFG edge means that some represented state can select the successor. It need not mean that the machine may freely choose either edge from one complete state. When a symbolic executor forks at a predicate, it creates path conditions. Each child represents the states that make its edge concrete. Dropping those conditions produces a safe but coarse description that can contain extra paths.

Trace under a partial deterministic step

A finite trace $S_0, S_1, \dots, S_n$ satisfies the machine when $\delta(S_i) = S_{i+1}$ for every $i < n$. It is maximal when S_n has a terminal outcome or $\delta(S_n)$ is undefined under the selected semantics. A bounded prefix need not be maximal.

If the state space is finite and execution is deterministic, every infinite trace eventually repeats a complete state. From that point, the same future repeats because the step function has not changed. This pigeonhole argument can decide termination by exhaustive exploration in a sufficiently small, closed finite machine: either a terminal state is reached or a state repeats. The bound may still be enormous. A machine with m bits of complete state can have up to 2^m states before one accounts for invalid combinations.

Real program models often make the practical or mathematical space effectively unbounded. Memory may have no fixed teaching limit, input may arrive during execution, or a theorem may quantify over every word width and memory size. Turing's result places a deeper limit here: there is no algorithm that decides for every program and input whether the modeled computation halts (Turing 1937a). This does not make termination proofs futile. It says no universal automatic method succeeds on every case. Invariants, well-founded variants, restricted languages, finite abstractions, and interactive proofs remain powerful because they exploit structure supplied by the particular program and claim.

Finite hardware and an unbounded theorem are compatible

A manufactured machine has finite physical storage at an instant. A theorem may still range over an arbitrary memory bound, arbitrary input length, or a family of machines. The theorem's mathematical domain, not the amount of RAM on one workstation, determines whether exhaustive state enumeration is a complete method.

9.3.1 From predicate to selection signal

At the architectural level, the branch rule is an equation for pc' . A simple hardware implementation can evaluate candidate addresses and use a multiplexer controlled by the predicate to select one for the next fetch. That picture connects Boolean algebra to a physical circuit: voltage ranges encode bits, gates compute the predicate, adders compute addresses, and storage captures a selected next value on a clock boundary.

The equation does not dictate one circuit. A processor may begin fetching a predicted successor before the predicate resolves, compute alternatives in parallel, or represent the program counter across several pipeline stages. Energy is spent charging physical capacitances, moving instruction bytes, and discarding wrong-path work. Delay depends on circuit paths and implementation technology. The ISA hides those mechanisms so software has a stable committed meaning across different processors.

The economic value of that separation is substantial. One binary can run on several compliant implementations; a processor designer can change predictors or pipelines without changing the architectural branch result; and compiler writers can make portable correctness claims while adding processor-specific cost models. The same separation creates limits: ISA equivalence does not prove equal power, latency, cache behavior, or resistance to speculative leakage.

Do not put a cycle count into an architectural theorem

The scalar transition rule justifies which state commits next. A claim about how many clock cycles or joules that transition costs needs a named processor, measurement method, workload, and uncertainty account. Treat those facts as a different evidence layer.

9.4 Control transfer in Ao

Ao instructions are four bytes and begin at four-byte-aligned addresses. An ordinary instruction advances from p to $p + 4$. A conditional branch with signed eight-bit immediate d selects $p + 4 + 4d$ when its condition is true and $p + 4$

otherwise. The scale of four ensures that a target obtained from an aligned branch is aligned. It does not ensure that the target contains a complete legal instruction; fetch and decode still have to succeed.

Ao supplies eight branch predicates over Z, N, C, V . Equality and inequality read Z . Signed less-than and greater-than-or-equal test whether N and V differ or agree. Unsigned lower and higher conditions use the no-borrow bit C , with Z distinguishing strict from non-strict comparisons. The branch preserves registers, memory, and conditions. Its visible write is the program counter, unless the selected target causes a later fetch trap.

Consider this width-eight loop, with each line occupying four bytes:

```
00: movi r1, 1
04: cmp  r0, r1
08: branch.lo exit
0c: sub  r0, r0, r1
10: branch.ne loop
14: halt
```

Listing 9.1: Ao counts r_0 down to zero

The first comparison protects the subtraction when the initial unsigned value is zero. For a positive initial r_0 , subtraction decreases the unsigned reading by one without wrapping. The subtraction also updates Z , so `branch.ne` returns to the loop body exactly while the result is nonzero. The listing uses labels for reading; an encoder must replace them with signed displacements that satisfy Ao's target formula.

Encode both A0 branch displacements

Using the addresses in the listing, let `exit` denote address 20 and `loop` denote address 12. Solve $p + 4 + 4d = t$ for each branch. Check that both signed values fit in byte 3, then calculate the destinations again from the encoded values.

The branch at address 8 needs $d = 2$, because $8 + 4 + 4(2) = 20$. The backward branch at address 16 needs $d = -2$, because $16 + 4 + 4(-2) = 12$. Treating the current address rather than the fallthrough address as the base would send both branches four bytes early.

The byte encodings complete the claim. Ao branch opcode 30 uses zero in its unused byte 1, places the condition code in bits 3 through 5 of byte 2, and stores the signed displacement in byte 3. Thus `branch.lo exit` is 30 00 20 02: condition code 4 becomes 20, and displacement 2 becomes 02. The backward `branch.ne loop` is 30 00 08 FE: condition code 1 becomes 08, and -2 becomes the eight-bit word FE. Reserved-bit, predicate, and target checks can now begin from the four bytes rather than from the printed listing.

9.5 Control transfer in RV64I

The selected RV64I conditional branches compare two integer registers as part of the branch instruction. BEQ and BNE test equality. BLT and BGE use signed readings; BLTU and BGEU use unsigned readings. A taken branch adds its decoded, sign-extended B-type offset to the address of the branch instruction. A branch whose predicate is false proceeds to the following instruction (RISC-V International 2026e).

```

    beq  a0, x0, exit
loop:
    addi a0, a0, -1
    bne  a0, x0, loop
exit:
```

Listing 9.2: RV64I counts a0 down to zero

Here x0 supplies the constant zero. The branch itself reads both named registers. The addi instruction changes a0 but creates no x86-style zero flag; bne performs the next comparison directly. Assembler register names such as a0 carry ABI meaning, while the base ISA semantics identify an integer register. Chapter 10 separates those two layers.

RV64I also provides direct jumps with links. JAL rd,offset writes the address of the following instruction to rd and transfers to a PC-relative target. Writing the link to x0 discards it and yields a plain direct jump. JALR obtains a target from a register plus immediate, writes a link, and clears the target's low bit under its architectural rule. The direct loop above needs neither instruction, but procedures will.

A pseudoinstruction is not another semantic primitive

An assembler may print `j label, ret`, or a branch-against-zero mnemonic that expands to a base instruction. Proofs over machine code must use the decoded base instruction and its operands. Pseudoinstructions remain valuable explanations of programmer intent, but they do not add encodings.

The conditional-branch immediate is assembled from noncontiguous instruction bits and has alignment constraints. A chapter proof may begin with the decoded offset when encoding is not under study, but the evidence manifest must say so. A decoder theorem and a step theorem are different claims: the first recovers the offset; the second uses it to update the program counter.

9.5.1 Reconstructing the B-type immediate

An RV64I conditional branch uses the B-type instruction format. Its immediate bits are split across the instruction word. If I_j denotes instruction bit j , the decoded signed offset has the bit pattern

$$\begin{aligned} d[12] &= I_{31}, & d[11] &= I_7, & d[10:5] &= I_{30:25}, \\ d[4:1] &= I_{11:8}, & d[0] &= 0. \end{aligned}$$

The assembled 13-bit pattern is sign-extended to the architectural address width. The low zero means the encoded target difference is a multiple of two bytes. Under the chapter's RV64I-only scope, instructions are four bytes and valid instruction addresses have the stronger four-byte alignment. An ISA profile that includes compressed instructions changes that surrounding alignment fact without changing the B-type reconstruction.

The arrangement is easier to understand from the datapath than from the printed order. Register fields stay in stable positions across common RISC-V formats, and the sign bit remains at instruction bit 31. Immediate pieces use the remaining positions. Human readers must therefore reconstruct their place-value order before interpreting the signed number.

Decode a backward RV64I branch offset

Suppose the reconstructed bits are $d[12:0] = 1\ 111111\ 1111\ 0$, with the groups standing for bits 12, 10 through 5, 4 through 1, and 0, while bit 11 is also 1. The complete 13-bit word is all ones except its low zero: 111111111110_2 . Its signed two's-complement reading is -2 . A branch at address 100 therefore targets 98 under the base ISA formula. That decoded offset does not meet the four-byte target alignment assumed by this chapter's RV64I profile, so a taken branch would not justify an ordinary next instruction there. Decode, target arithmetic, and target validity are three separate judgments.

For a less compressed calculation, let the desired destination be 16 bytes behind a branch at address 100. The mathematical offset is -16 . In 13-bit two's complement it is $1\ 111111\ 1000\ 0$. The encoder scatters those bits into the B-type positions; the decoder gathers them, adds the low zero, and sign-extends. Recomputing $100 + (-16) = 84$ checks the semantic target. Re-encoding the gathered fields checks a different property: that no piece was swapped or omitted.

For a byte-level control, `bne a0,x0,-4` encodes as instruction word FE051EE3. In little-endian memory order its bytes are E3 1E 05 FE. Decoding recovers opcode 63, the bne function code, $rs1=10$, $rs2=0$, and offset -4 . At address 100, a true predicate therefore selects 96. This example binds byte order, scattered fields, signed reconstruction, predicate operands, and target arithmetic in one check.

Decoder-to-target composition

Let $\text{decodeB}(w) = d$ mean that instruction word w reconstructs offset d . Let the step rule for a true predicate set $pc' = pc + d$. A machine-code target theorem needs both facts. Proving the step rule for an arbitrary decoded d does not prove that the bytes encode that value; proving the decoder result does not prove that execution takes the edge. Compose the decoder theorem, predicate theorem, and step theorem explicitly.

9.6 Control transfer in x86-64

Selected x86-64 compare instructions perform a subtraction for flags without storing the arithmetic result. A following `Jcc` reads the condition formula associated with its suffix. For example, `JE` reads `ZF`; `JNE` reads its negation; signed less-than uses `SF ≠ OF`; and unsigned below uses `CF`. The exact flag effects and accepted encodings belong to the selected instruction forms in Intel's reference, not to the mnemonic name in isolation (Intel Corporation 2026b).

```

    test rdi, rdi
    je   exit
loop:
    sub  rdi, 1
    jne  loop
exit:
```

Listing 9.3: x86-64 counts RDI down to zero

`test rdi, rdi` computes a bitwise conjunction for flags and leaves `rdi` unchanged. It sets `ZF` exactly when the selected value is zero. `sub` then **decrements** the register, which means that it reduces the register value by one, and supplies the `ZF` consumed by `jne`. This dependency is real even though it is absent from the register operands printed on the jump.

x86-64 relative jump targets use the instruction pointer for the following instruction as their base. Because instruction lengths vary, the decoder must first determine that next address. Short and near conditional-jump encodings can express different displacement widths while implementing the same predicate. Replacing one encoding with another changes instruction length and therefore changes both the base and the displacement required for a fixed destination.

9.6.1 Instruction length and target arithmetic

A selected short conditional jump uses a one-byte opcode followed by a signed eight-bit displacement. Its following-instruction address is therefore two bytes after the opcode when no disallowed or separately modeled prefix changes the decoded form. A selected near conditional jump uses a two-byte opcode and a

signed 32-bit displacement, giving a six-byte base form. These are claims about specific encodings in 64-bit mode, not a rule that every printed `Jcc` has length two or six.

Place a short `JNE` at address 200 with displacement -10 . The next instruction begins at 202, so the target is 192. If a tool replaces it with a six-byte near form at the same address, retaining the byte value -10 as the new displacement would target 196. To preserve target 192, the repaired displacement must be -14 , because $200 + 6 - 14 = 192$. Widening a field without repairing its value can therefore change control even when both signed numbers fit.

In the selected forms, the short bytes are `75 F6`: opcode 75 followed by the eight-bit form of -10 . The repaired near bytes are `0F 85 F2 FF FF FF`: opcode bytes `0F 85` followed by the little-endian 32-bit form of -14 . A decoder that reports only the `jne` predicate has not yet established either length or target.

Prefix and mode handling make real decoding broader than this example. A trusted decoder must establish the complete instruction boundary before a relative target checker uses the following-instruction address. Searching the byte stream for an opcode-like byte is insufficient: the same byte can occur inside an immediate, displacement, or preceding instruction. This is why instruction-boundary recovery and target recovery belong in one source-pinned decoder route.

A disassembler listing is evidence only through its decoder

Addresses and lengths printed by a disassembler are derived results. They are useful to inspect, but a proof package must name the decoder version and input bytes or independently check the boundaries. Copying the printed destination back into the expected result merely makes the listing check itself.

A flag-reading jump has a hidden-looking input

The condition suffix identifies which flags the jump reads. Register equality at the jump is irrelevant unless preceding instructions established the required flags. Moving, replacing, or inserting a flag-writing instruction between compare and jump can change the path without changing any explicit jump operand.

The `x86-64` loop, the `RV64I` loop, and the `Ao` loop implement the same small mathematical recurrence only under declared input and observation relations. They do not have identical registers, instruction counts, condition state, or program-counter values. A proof must relate those differences instead of erasing them.

9.7 The cost of control layout

The architectural graph says which edges are possible. A code layout assigns addresses to its blocks. That assignment determines which successor is the fallthrough, how far every direct edge travels, which displacement encodings fit, where alignment padding appears, and which instruction bytes share nearby memory regions. Two layouts can implement the same graph while differing in size and physical behavior.

Fallthrough is especially valuable because it needs no taken transfer at the end of a block. A compiler can place a likely successor immediately after its predecessor and invert a conditional predicate if the ISA offers the matching condition. This may remove an unconditional jump from the common path. It also changes which edge is taken, so the benefit depends on code size, prediction, processor behavior, and the actual path distribution.

Current LLVM machine block placement uses branch probability, block frequency, loop, post-dominator, profile, and alignment information among its inputs (LLVM Project 2026f). That source is evidence about one current implementation, not a timeless definition of optimal layout. The placement problem has several competing measures: hot-path fallthrough, total branch distance, padding, instruction-cache locality, code duplication, and the risk that a profile does not represent future inputs.

Profile counts are observations of selected runs. They can estimate that an edge is common without proving that another edge is impossible. A compiler may use profile-guided layout for expected performance while retaining code for rare legal paths. A proof of semantic equivalence must cover every allowed path, not only the profile's high-frequency edges.

9.7.1 Displacement range

A relative encoding has a finite signed range and sometimes a scale or alignment constraint. Moving blocks can push an edge outside that range. An assembler or linker may then choose a longer encoding, introduce an intermediate transfer stub, or materialize a destination through additional instructions. This process is often called branch relaxation when the tool selects a form after final distances are known.

The repair changes more than one immediate field. A longer instruction moves later addresses. Those shifts can change other relative distances, alignment, debug locations, and relocation entries. A fixed-point layout process may need to repeat until every chosen form fits. A proof-preserving tool must show that the repaired sequence reaches the same symbolic destination under its preconditions and preserves every additional observed state component.

Ao avoids this engineering breadth by declaring one fixed branch form. That clarity creates a strict range limit. A real program that needs a farther Ao destina-

tion would require a book-defined longer sequence or a changed ISA; a tool must not silently pretend that the signed byte grew wider.

A semantically equal edge can cost more after layout

Suppose a common block and its successor first fit a short relative transfer. Adding unrelated code between them may force a longer encoding. The source control graph and destination label remain the same, yet code size, following addresses, decode work, and cache placement can change. Semantic equality of the edge does not imply equal physical cost.

9.7.2 Branch prediction

A pipelined processor may need a likely next fetch address before an older branch has produced its resolved predicate and target. A predictor uses stored history, address structure, static rules, or combinations of these signals to guess. Correct guesses keep useful work available. A wrong guess requires the implementation to discard or repair wrong-path work before architectural commitment.

Prediction quality is a distributional claim. Accuracy on one workload does not imply accuracy on another, and accuracy alone omits the cost of each miss, the value of correct-path work, predictor storage and energy, and interference among branches. Smith's 1981 study belongs to the historical development of these strategies (Smith 1981); current processors use implementation-specific designs whose precise behavior cannot be inferred from the ISA.

If-conversion replaces selected control dependence with data selection or conditionally executed operations. It can avoid a hard-to-predict branch, but it may execute work from both paths, extend data dependencies, increase register pressure, or perform an operation that was unsafe on the path not selected path. The transformation therefore needs a semantic safety proof and a processor/workload cost argument.

Fewer branches does not automatically mean faster

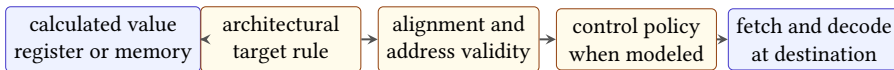
A branch removal can add instructions, lengthen dependencies, or execute work that the branch would skip. Compare complete sequences under a stated processor and workload model; do not optimize by mnemonic count alone.

9.8 Direct and indirect transfers

A direct transfer carries a destination relative to the instruction or names one through relocation. Once its bytes and address are fixed, target arithmetic usually determines one destination. An **indirect transfer** obtains at least part of its

destination from current machine state. The same instruction bytes may therefore reach many addresses on different executions.

This difference changes the proof. For a direct branch, faithful decode and correct target arithmetic can identify the destination locally. For an indirect jump, local instruction semantics only explain how a register or memory value becomes a candidate program counter. A surrounding invariant must restrict that value to an allowed target set. A compiler might derive the set from a switch table, a procedure proof from a saved continuation, or a control-flow integrity policy from approved entry points.



Computing an address does not by itself justify execution from that address.

Figure 9.2: An indirect destination passes through several proof gates before the next instruction may be justified. Some machines omit or add policy gates, but the target value alone is never the whole argument.

RV64I JALR, for example, adds a sign-extended immediate to a register value and clears the least significant bit of the result. A theorem must use that transformed target rather than equating the new program counter with the raw register. Selected x86-64 indirect branches obtain their destinations from register or memory operands under their own mode and operand rules. Core Ao has no indirect jump, so a proof must not manufacture one from a register that happens to contain an address.

Target validity is a second question. Depending on the model, the destination may need suitable alignment, a canonical address, mapped executable memory, an instruction boundary, and bytes that decode legally. A hardware or operating-system policy may add another restriction. These checks need not all occur in the transfer instruction itself. A model may update the program counter first and report a fetch failure on the next step. The trace should preserve that ordering instead of replacing it with one generic failure.

A finite target set needs provenance

Listing the destinations seen in tests does not establish the allowed target set. Derive the set from a table bound, type invariant, saved-link contract, or other stated premise. Then show that every calculated destination belongs to it and that every member is a valid entry under the selected semantics.

Return instructions are important indirect transfers. Their intended target comes from a prior call, but that history is not automatic evidence. The proof must

connect the current link register or stacked word to the continuation created at the matching call and show that intervening execution preserved it. Chapter 10 turns that historical connection into a procedure contract.

9.8.1 Calls and continuations

Suppose a call begins at p , occupies L bytes, and selects q , the entry of the **callee**—the procedure being called. Its ordinary continuation is $k = p + L$. The call transition has two conceptual results: control moves to q , and some architecture-defined location receives enough information to recover k . A link register writes the value directly. A stack-oriented sequence may place it in memory. An early machine may modify or construct a later transfer. These mechanisms differ, but each creates a value whose future use must be justified.

A return proof is relational across time. At the call, establish that the saved value denotes k . Across the callee, show that the chosen storage follows the calling agreement. At return, show that the loaded or named value is still the matching continuation and satisfies the target rules. Finally, resume in a state related to the caller's pre-call state, except for declared results and permitted changes. Merely proving that the return target is executable misses the matching-call property.

Nested calls show why one unqualified return variable is not enough. If procedure A calls B , and B calls C , the active continuation for C is in B , while the older continuation for B remains in A . A stack, a set of link registers with a convention, or another activation structure must preserve this last-created, first-used order. Chapter 10 makes the storage discipline explicit; this chapter supplies the control fact that gives the stored word its meaning.

A continuation is an address with provenance

The numeric value k is not by itself a return proof. The proof needs the event that created k , the activation to which it belongs, the preservation argument between call and return, and the target-validity rule at its use.

Target soundness and target completeness

For an indirect transfer under a precondition, target soundness says that every destination the machine can select belongs to the declared target set. Target completeness says that every destination claimed as possible can actually be selected by some allowed entry state. Soundness protects execution from an omitted bad destination. Completeness prevents a coarse analysis from adding spurious edges that may obscure invariants or make a later proof needlessly hard.

The two directions need different witnesses. A soundness failure is one entry state whose selected destination lies outside the set. A completeness claim needs, for each listed destination, an allowed state that reaches it. Many safety arguments require soundness but not completeness. An exact control-flow graph requires both, which is why a graph inferred from a few runs is generally neither a safe graph with every possible edge nor an exact account.

9.8.2 Jump tables and target safety

A multiway source choice can lower to a chain of conditional branches, a tree of comparisons, arithmetic selection, or an indirect jump through a table. The best sequence depends on case density, value distribution, code size, target ISA, relocation model, and processor costs. A jump table is attractive for a dense range because one bounds check and one indexed lookup can select among many cases. It also turns ordinary data and address facts into direct premises of a control-flow proof.

Suppose source selector x has cases 10 through 13. After proving $10 \leq x < 14$, set $i = x - 10$, so $0 \leq i < 4$. Let table base b be aligned and let each entry occupy eight bytes. The load address is

$$a = b + 8i.$$

The index bound implies $b \leq a < b + 32$, subject to a no-wrap premise in the machine address width. A memory contract must make all eight bytes beginning at each such a readable. The loaded word may be an absolute address, a relative offset from the table or instruction, or another representation chosen by the compiler and relocation format. The proof must apply that representation rule before the ISA's indirect-target transformation.

Let the four resolved destinations be q_0, q_1, q_2, q_3 . Target soundness follows from a chain of facts: the bounds check restricts i ; address arithmetic selects table entry i without wrap; the memory lemma returns the declared entry contents; relocation or offset arithmetic resolves that content to q_i ; the ISA target rule produces the same candidate; and target validity permits execution at q_i . Omitting any link leaves a plausible attack or compiler bug: an unchecked index, wrapped address, table bytes that later instructions can change, incorrect relocation, low-bit transformation, or invalid destination.

Completeness asks another question. For every q_j claimed in the graph, there must be an allowed selector value—here $x = 10 + j$ —whose execution reaches that destination. Duplicate table entries make the target set smaller than the case set. A default case lies outside the table route because it is selected by the failed bounds check. The exact CFG should record both structures rather than infer five distinct indirect targets from five source labels.

Read-only is a temporal premise, not a table adjective

Placing a jump table in a read-only section can support its integrity under a named loader and protection model. The bytes are not mathematically immutable merely because a file format labels the section. State who maps the table, when relocation writes occur, and which later agents may modify its memory.

The industrial tradeoff is not one-dimensional. A table consumes data space and can add a load and an indirect branch. A comparison tree consumes code and direct branches. Profiles, cache placement, target prediction, relocation cost, hardening rules, and the number of cases all affect the choice. The semantic proof first establishes that each candidate implements the source choice; performance evidence then compares them under a named environment.

9.8.3 Control-flow integrity

Control-flow integrity (CFI) requires dynamic transfers to follow edges allowed by a declared control-flow policy. The influential 2005 work by Abadi, Budiu, Erlingsson, and Ligatti formulated the property together with an attack model and enforcement approach (Abadi et al. 2005). The graph or equivalence classes used by a concrete policy determine its precision. “Has CFI” is not a complete security claim without those allowed sets and assumptions.

For a direct branch with a fixed valid target, policy checking may add little. Indirect calls, jumps, and returns are the main challenge because their raw targets come from state that instructions can change. An enforcement mechanism may require a valid landing instruction, attach a tag or identifier to approved destinations, or check that the target belongs to a class. The architectural target calculation still occurs; policy decides whether that candidate may be used.

Return protection has a temporal shape that a forward target class may not capture. A return is intended to reach the continuation created by the matching active call. A protected shadow stack can maintain a second copy of return addresses and compare or use it at return. The proof must relate calls, nested activations, returns, exceptional unwinding, and the protected storage. Merely showing that the returned-to address begins some valid function is weaker.

Current Intel CET and ratified RISC-V control-flow-integrity extensions expose selected landing-pad and return-protection mechanisms (Intel Corporation 2020; RISC-V International 2026a). Their exact coverage depends on enabled features, instruction forms, operating-system support, and software instrumentation. The book can use them as concrete architecture examples without claiming that either mechanism proves the source program’s intended control graph.

Four properties should remain separate:

1. target arithmetic produces the architecture’s candidate address;

2. target validity permits fetch and decode at that address;
3. the CFI policy admits the corresponding edge or target class; and
4. the application proof says that this allowed edge is semantically correct for the current state.

An enforcement mechanism can satisfy the third item while a policy is too coarse and admits an unintended but class-compatible target. A perfect policy check does not repair a wrong arithmetic predicate or a corrupted application invariant. Conversely, a semantically intended target can fault if it is unmapped or violates an enabled landing rule.

Allowed control is not necessarily intended control

A coarse policy may admit several destinations for one indirect transfer. CFI can prevent leaving that set while still allowing the wrong member. State the target partition and the application invariant separately.

Speculation creates another boundary. An architecturally rejected path or a path whose predicate is false may begin transient execution before the branch or policy check resolves. Architectural CFI and correct committed control do not by themselves prove that no information about the processor's hidden physical state leaked from such work. That claim needs predictor, cache, timing, and transient-state observations routed to Chapter 16.

9.9 Control-flow graphs

A **basic block** is a maximal instruction sequence entered at its first instruction and left only after its last, under the selected analysis. Interior instructions have one ordinary successor inside the block. A terminating transfer, return, or trap may end the block. The exact boundary depends on what the analysis treats as an instruction and which exceptional edges it includes.

A control-flow graph is a directed graph $G = (V, E)$. Its vertices are blocks or, in a finer graph, individual instructions. An edge $(u, v) \in E$ says that the selected semantics admits a transfer from the end of u to the start of v under some state in scope. This is a possibility claim. It does not say that every execution taking u reaches v , or that both outgoing edges of a conditional branch are reachable from the program entry.

For direct branches, decoding and target arithmetic can often enumerate local successors. State reasoning then removes infeasible edges. For indirect transfers, even the local successor set may require a value analysis, relocation table, type invariant, or policy. A graph is therefore always relative to a semantic package

and abstraction. A graph that omits traps, exceptions, or indirect targets should not be called complete for a model that observes them.

9.9.1 Reachability from an entry

A vertex v is reachable from entry e when there is a finite path

$$e = v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_k = v.$$

Graph reachability is necessary but may not be sufficient for semantic reachability. Consecutive edges in a coarse graph may require incompatible machine states. A path-sensitive analysis retains enough state information to rule out such combinations. An actual trace gives one semantically feasible path but does not enumerate all reachable vertices.

Unreachable code can arise from an always-false predicate under the input contract, an unused block, an impossible indirect target, or bytes that no legal predecessor enters. Removing it may reduce code size and analysis work. The economic benefit does not lower the proof burden: deleting a block needs evidence that no allowed execution observes it.

9.9.2 Dominance over paths

A vertex d **dominates** vertex v when every graph path from the entry to v passes through d . The entry dominates every reachable vertex, and each reachable vertex dominates itself. If $d \neq v$, the dominance is strict. A vertex's immediate dominator is the closest strict dominator in the resulting dominator tree.

Dominance supports proofs about definitions and checks. If a bounds-check block dominates a memory access, every graph path to that access passes through the check. That graph fact still needs a state fact: the check must establish the right predicate, and no intervening operation may invalidate it. A dominance tree records unavoidable control points, not the meaning of their instructions.

To disprove that d dominates v , supply one entry-to- v path that avoids d . To prove dominance by direct enumeration, quantify over every path—not only the paths observed in tests. Practical compilers use algorithms rather than enumerate exponentially many paths. The relation and its use are the teaching target here; algorithmic efficiency becomes important on large program graphs (Allen 1970; Lengauer and Tarjan 1979).

A simple dominance proof by predecessor closure

Let d be the proposed dominator of v . Mark d , then repeatedly mark any vertex all of whose reachable predecessors are marked. If this justified closure marks v , every path to v must cross d : the last edge into each newly marked vertex comes from an already marked predecessor. This is a proof method

for a finite graph, not the fast dominator algorithm used by a production compiler.

9.9.3 Cycles and repetition

A directed cycle is a path with at least one edge from a vertex back to itself. A **strongly connected component** is a maximal set of vertices in which each vertex can reach every other. A component with more than one vertex, or a single vertex with a self-edge, contains a cycle. Collapsing each component to one vertex yields a component graph with no directed cycle.

These are graph facts. A reachable cycle shows that the graph admits repeated control, but not that any allowed execution repeats it forever. A state predicate may force an exit after finitely many traversals. Conversely, a nonterminating execution in a finite control graph must revisit some control location, although its data state may keep changing.

A **back edge** is often defined relative to dominance: edge (u, h) is a back edge when h dominates u . The destination h acts as a loop header for the natural-loop construction. Not every cycle in an arbitrary graph has one entry that dominates all its vertices. Compiler loop representations and transformations must state how they handle such graphs rather than assume every cycle came from a structured source loop.

Post-dominance reverses the path question. A vertex p post-dominates u when every path from u to a selected normal exit passes through p . The choice of exits and treatment of an execution that never reaches an exit, or of traps, matter. A join block may post-dominate both arms of a terminating conditional while failing to post-dominate an arm that can loop forever or trap.

9.9.4 Four questions about one graph

Consider vertices E, T, B, J, X with edges

$$E \rightarrow T, \quad T \rightarrow B, \quad T \rightarrow J, \quad B \rightarrow T, \quad J \rightarrow X.$$

The graph resembles a pre-tested loop. From entry E , all five vertices are graph-reachable. Vertex T dominates B, J , and X , because every path from E to any of them passes through the test. Vertex B does not dominate J : the path E, T, J avoids it. Edge $B \rightarrow T$ is a back edge because T dominates B . The set $\{T, B\}$ is strongly connected, while each of E, J, X forms its own component.

Now attach predicates. Let the edge $T \rightarrow B$ require counter $c > 0$, let $T \rightarrow J$ require $c = 0$, and let B replace c by $c - 1$. Under the entry condition $c = 0$, graph reachability still finds a path to B , but semantic reachability does not. Under $0 \leq c \leq n$, both graph edges are locally possible across the allowed set, while any one execution chooses only one on each visit. Under an invariant that c is natural

and strictly decreases in B , the cycle is reachable for positive inputs but cannot be traversed forever.

This one example yields four different answers:

1. the local CFG records both successors of T ;
2. reachability from E removes no vertex without state information;
3. dominance locates T as an unavoidable control point for the body;
4. the invariant and natural-number variant prove semantic safety and termination for the selected entry contract.

No graph algorithm supplies the last item merely by finding the back edge. Conversely, the state proof benefits from the graph because T gives one recurring point at which the invariant and variant can be stated.

Adding one edge changes dominance but not instruction meanings

Add edge $E \rightarrow B$. The instructions inside T and B retain their local semantics, and B remains reachable. But T no longer dominates B , so $B \rightarrow T$ no longer meets the stated back-edge definition. A CFG repair can therefore invalidate a loop proof even when no arithmetic instruction changed.

9.9.5 Reducible control flow

Structured source constructs commonly lower to graphs in which loop entries are well behaved: a header dominates the loop body, and back edges return to that header. Such reducible graphs support natural-loop analyses and many transformations. Arbitrary jumps or some low-level transformations can form regions with several entries. The graph remains executable, but a compiler may need node splitting, a more general region representation, or an analysis that does not assume one dominating header.

This distinction has an economic effect. Optimizers, binary rewriters, coverage tools, decompilers, and security instrumentation all pay for the precision of their control representation. A simpler graph class can make an analysis faster and its transformations easier to validate. Converting a general graph to that class may duplicate code, change layout, and alter the physical costs described earlier. Structure is therefore both a reasoning resource and an engineering tradeoff.

A graph shape does not supply an invariant

A loop header and back edge locate repeated control. They do not state which machine values remain related, why accesses are safe, or why a variant decreases. Those are semantic obligations over states at the chosen control point.

9.10 One loop in three machines

Let the input be a nonnegative mathematical integer n that fits in the selected word without wrapping during the loop. Each program maintains a machine word whose unsigned reading is the number of decrements remaining. The common control-flow graph has an entry test, a decrementing body, a back edge, and an exit.

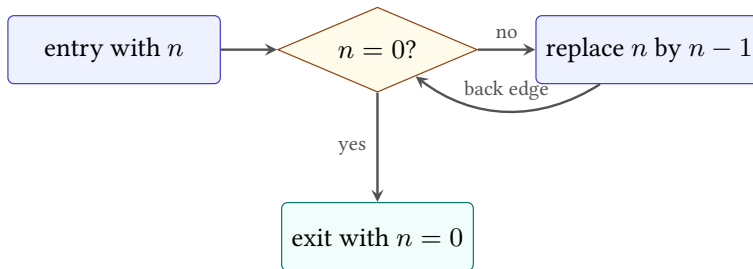


Figure 9.3: The common mathematical control shape beneath the three listings. The machine-specific predicates and target conventions implement these edges.

At the head of the body, use the invariant

$$0 < \langle r \rangle_u \leq n,$$

where r denotes the machine's related counter register. The body changes the reading from $k > 0$ to $k - 1$. The value therefore remains in range and strictly decreases as a natural number. When it becomes zero, the back-edge predicate is false and execution reaches the exit.

Termination and final value of the counted loop

If the input is zero, the entry test reaches the exit without executing the body. Otherwise let the counter reading at the first body entry be $n > 0$. Each body execution subtracts one from a positive reading, so modular subtraction agrees with natural subtraction and cannot wrap. The natural measure decreases from n through $n - 1, \dots, 1, 0$. A strictly decreasing natural

measure cannot decrease forever. At zero, each machine's related not-equal predicate is false, so the program exits with counter value zero.

The premise that subtraction occurs only at a positive value is load-bearing. Delete the entry test and start at zero: fixed-width subtraction produces the maximum unsigned word, after which the loop may take $2^w - 1$ further iterations before returning to zero. The original termination conclusion may still hold for a finite width, but the proof, trace length, and intended cost claim have changed.

The invariant is also not a complete cross-ISA relation. Such a relation must identify Ao r_0 , RV64I a_0 , and x86-64 RDI; relate the values at chosen control points; state what memory must agree on; map Ao's Z and x86-64's ZF when their branches depend on them; and permit the RV64I state to lack that flag. Chapter 12 develops this style of relation.

Stuttering between related control points

One machine may need an extra compare or test instruction before a branch. The other may combine comparison and transfer. A simulation can relate states only at loop heads and exits, allowing several steps on one side for one or several on the other. This is stuttering, not an error, provided the relation is restored at the declared observation points.

9.10.1 Safety, progress, and termination

The counted-loop argument combines three claims that should remain visible. *Safety* says every visited instruction and memory access is defined and the subtraction does not wrap under the intended natural-number reading. *Progress* says a nonterminal state can take another justified step. *Termination* says the run eventually reaches the exit. A decreasing natural measure helps with termination only after safety and progress connect each iteration to the next loop head.

The separation becomes useful when a proof fails. A counter may decrease on every completed iteration while an invalid branch target traps before the next head. The measure argument is then true but irrelevant to normal termination. A loop may remain safe forever while its measure fails to decrease. Or it may reach the exit with the expected counter while corrupting an observed memory location. No one of these facts substitutes for the others.

Iteration count is another conclusion, not a side effect of termination. Add a ghost natural number i counting completed decrements and strengthen the invariant at the loop head to

$$i + \langle r \rangle_u = n.$$

Initially $i = 0$ and the counter reads n . One body step increases i by one and decreases the counter reading by one, preserving the sum. At exit the counter is

zero, so $i = n$. The ghost value is proof bookkeeping; it need not exist in machine state.

Why the variant must be checked at one control point

A variant can rise inside an iteration and still decrease from one loop head to the next. Choose a recurring control point, prove that every return to it has a smaller natural variant, and prove that execution cannot avoid either that return or an exit forever. Comparing values at arbitrary adjacent instructions can reject correct loops or accept a cycle that bypasses the measurement point.

9.11 From traces to control-flow claims

A trace records the actual successor at each step. For a chosen input, a checker can decode the instruction at the recorded program counter, calculate again its predicate and target, and compare the complete successor with the next state in the trace. This catches a wrong edge, a wrong decrement, a stale condition bit, or execution after a terminal outcome.

One trace does not prove a general loop theorem. Running the loop from inputs zero through 255 proves facts about those chosen executions under a trusted enumeration and checker. The invariant proof covers every input in its stated domain because it reasons about an arbitrary positive counter and a decreasing measure. Machine evidence can support that argument by checking the step supporting theorems or refuting their negations, but a bounded replay cannot silently become an unbounded theorem.

A bounded run has three distinct endings

A bounded interpreter may halt, trap, or exhaust its step bound while the machine is still running. Bound exhaustion is not a machine trap and does not prove that execution will continue forever. It says only that this run prefix did not reach a terminal outcome within the supplied budget.

A control-flow graph also differs from a trace. The graph contains possible edges admitted by local branch analysis. A trace contains the one edge selected in each visited state. Proving that an edge is unreachable needs a state argument, not merely its absence from a few traces.

The graph can be checked at several strengths. A syntactic graph connects each decoded direct transfer to its calculated destination and adds fallthrough where appropriate. An indirect edge may conservatively point to many possible targets. A state-restricted graph removes edges whose predicates or target values are

impossible under a supplied invariant. Reachability from the entry then removes disconnected code. Each reduction needs evidence: a visually tidy graph is not a proof that the omitted edge cannot execute.

Loops arise from cycles in this reachable graph, but a cycle alone does not identify an induction invariant or termination measure. Conversely, an unrolled trace can revisit an address without revealing every possible cycle. Graph structure organizes the proof obligations; semantic reasoning over states discharges them.

9.12 Axeyum route

The audited Axeyum checkout at revision `a9991fda` supplies more control-flow substrate than a bit-vector library alone. It has a generic transition-system interface, bounded model checking, memory-aware bounded checking, k-induction, certified QF_BV safety routes, and replay-checked counterexample models. Its toy bit-vector VM already has conditional branches, explicit true and false targets, validated programs, static CFG edges and basic blocks, symbolic target and edge reachability, source-aware traces, and independent trace replay.

That VM is a useful precedent, not the book machine. Its program counter names instruction indices, its branches use its own equality tests and targets, and its fuel and outcome rules belong to that package. It lacks Ao instruction bytes, the $pc + 4 + 4d$ target rule, the Z, N, C, V predicate family, and Ao fetch and trap behavior. The checkout also lacks source-pinned RV64I and x86-64 decoder adapters, the three-machine loop relation, and proof objects for the chapter's dominance claims. Renaming the toy VM would hide these differences rather than discharge them.

Start with Ao because its complete state and fixed-width encoding are owned by this book. Reuse Axeyum's transition, symbolic exploration, reporting, and replay patterns through a separate Ao adapter. Define a decoded branch containing a condition code and signed displacement. Define the predicate as a pure function of Z, N, C, V , then define the two successor-PC formulas. Only after unit tests cover every condition and both outcomes should the branch enter the general Ao step function.

The first evidence object should contain an initial state, instruction bytes, decoded branch, selected predicate value, expected successor state, and the semantic-package digest. Its checker should decode again and calculate the step rather than trusting recorded derived fields. A small command-line wrapper can then print a teaching trace while its exit status depends on full agreement.

Controls for the first branch checker

Use a positive case whose taken target differs from fallthrough. Pair it with mutations that invert the predicate, omit the displacement scale, use *pc* instead of $pc + 4$ as the Ao base, or change one reserved encoding bit. Each mutation must make the checker fail for the intended reason. Also retain a valid case with a false predicate so a checker that always chooses the target cannot pass.

The RV64I and x86-64 routes come later. Each needs a source-pinned decoder slice before its transition function can carry machine-code claims. The RV64I control should mutate a split immediate field or confuse signed and unsigned predicates. The x86-64 control should alter a prefix, opcode, displacement, or decoded length. A handwritten branch formula can test arithmetic; it cannot stand in for those decoders.

For the counted-loop theorem, separate four evidence levels:

1. replay one complete trace for a small input;
2. enumerate every input at a teaching width;
3. use bounded model checking for a named depth and replay any model;
4. prove invariant preservation or safety at a named width through k-induction or a certified QF_BV route; and
5. reconstruct a width-parametric decreasing-measure proof through a kernel route when that route exists.

The manifest must label which level was achieved. Solver UNSAT at width 64 is a strong finite-width result, but it is not automatically a theorem quantified over every positive width. Bounded safety, k-inductive safety, and termination are also distinct. A proof certificate can reduce trust in the solver verdict; it still proves only the encoded formula. The publication manifest should record the adapter revision, semantic digest, solver route, certificate checker when present, replay result, and negative controls.

What the existing toy VM can teach immediately

Its static graph can test edge extraction, its symbolic search can produce a target-reachability witness, and its replay route can reject a corrupted trace. Those are genuine Axeyum lessons. They become Ao evidence only after an Ao-specific decoder and step relation connect the same evidence pattern to the book's bytes and state.

9.13 Obligations of a branch proof

Branch correctness is often compressed into the phrase “the target is right.” That phrase hides several separate obligations. Keeping them separate makes both reader proofs and negative controls more useful.

First comes *fetch and decode*. The current program counter must identify a complete legal instruction. The decoder must recover the intended condition, register fields, and signed displacement. A proof that starts from an already decoded branch may assume this layer, but it must say so.

Second comes *predicate agreement*. The implementation must ask the question named by the instruction. Signed and unsigned comparisons can choose different edges for the same word pair. Ao and selected x86-64 forms must read the condition state produced by the preceding instruction; an RV64I branch must read the specified register values. Preserving the right target while inverting the predicate is still a wrong branch.

Third comes *target arithmetic*. The proof must identify the base, extension rule, scale, and arithmetic width. It must also distinguish the taken target from fallthrough. Alignment, canonical-address, and mapped-code requirements belong here when the selected machine model includes them.

Fourth comes *state preservation*. A conditional branch usually changes the program counter and preserves most other architectural state. “Reached the right block” does not justify an unnoticed register, memory, flag, or outcome change. A linking jump is different because its link destination is an intended additional write.

Finally comes *context*. The successor address must lead to code whose execution is covered by the surrounding claim. A locally correct edge into an illegal byte sequence traps on the next step. A locally correct loop edge does not prove termination. Those are composition properties beyond the branch’s single transition.

Five obligations for a direct conditional branch

Check, in order: legal fetch and faithful decode; the exact predicate over the declared input state; taken and not-taken program-counter formulas; preservation of every other observed component; and the premise needed to continue from the chosen successor. A proof may discharge the layers separately, but the whole-program claim needs all five.

This decomposition locates counterexamples. A wrong immediate field is a decode failure. A stale ZF is a predicate failure. A missing Ao scale factor is a target failure. A jump that clears an unrelated register is a preservation failure. A correct backward edge in a loop without a decreasing measure is a context failure. Merely reporting that “the branch test failed” throws away the lesson carried by the witness.

9.14 Where the model stops

This chapter treats architectural steps as atomic and sequential. It excludes branch prediction, speculative execution, pipeline recovery, timing, caches, and side channels. Those mechanisms can make two architecturally equivalent paths observably different to a timing adversary, but they require another state and observation model.

We also exclude interrupts, exceptions beyond Ao's declared traps, privilege transfers, self-modifying code, concurrent modification, and weak memory. Indirect control flow receives only the link and target facts needed for the next chapter. A full account must also specify target validity, canonical addresses, control-flow enforcement, and the memory from which an indirect destination may be loaded.

Within the boundary, the central fact is exact and surprisingly broad. A finite program becomes an indefinitely extensible process because the program counter may revisit an address. The mystery does not lie in an unspecified machine. It lies in the consequence of a rule small enough to write on one line: choose a successor, then apply the same semantics again.

9.15 Exercises

1. **Execute.** Trace both outcomes of an Ao branch.eq at address 40 with displacement -3 . Give the successor program counter and all preserved state components.
2. Compute the Ao displacement needed to branch from addresses 0, 12, and 100 to address 40. State which values fit the signed byte and which targets meet the alignment rule.
3. **Break.** Replace Ao's taken-target rule $pc + 4 + 4d$ with $pc + 4d$. Give a branch for which the mutation accidentally agrees and one for which it reaches a different legal instruction.
4. A four-byte RV64I branch lies at address 100 and has decoded offset -20 . A six-byte x86-64 near jump lies at address 100 and has displacement -20 . Compute both targets and explain the difference.
5. Explain why changing an x86-64 conditional jump from a short encoding to a longer encoding requires displacement repair even when its address and destination label appear unchanged.
6. For each Ao predicate eq, lt, lo, and hi, give values of Z, N, C, V that take and do not take the branch. Identify condition combinations that cannot arise from one selected comparison, if your claim assumes such a comparison.

7. Rewrite the RV64I counted loop to count upward from zero to an input bound. State whether its comparison is signed or unsigned and give an input premise that prevents wraparound.
8. **Prove.** Strengthen the counted-loop invariant to say exactly how many body executions have occurred. Use it to prove both final value and iteration count.
9. Delete the entry test from each loop and start the counter at zero at width four. Trace enough steps to identify the new iteration count. Explain why the original natural-number proof no longer applies at the first step.
10. **Transfer.** Define relation checkpoints for the three loop listings. Map counter values, control locations, outcomes, and the condition information required for the next branch; identify scratch or flag state that the final observation may omit.
11. Give two instruction sequences that leave the same counter register but different x86-64 flags immediately before `jne`. Supply initial values for which the chosen edges differ.
12. Explain the difference among a trace, a control-flow graph, and a proof that a graph edge is unreachable. Give one fact each can establish.
13. Design a bounded-run result type that distinguishes halt, trap, and bound exhaustion. Give a negative test for an implementation that reports bound exhaustion as proof that execution continues forever.
14. Design the first Ao branch artifact for Axeyum. Include bytes, decoded instruction, initial state, expected successor, semantic digests, checker command, and four target or predicate mutations.
15. **Transfer.** State the obligations for replacing an Ao condition-reading branch with an RV64I register-comparing branch. Include the state relation before the branch and the relation required after both taken and not-taken outcomes.
16. **History.** Compare conditional succession in Turing's machine with a conditional branch in a stored-program ISA. Name one common mathematical idea and two machine details that Turing's model does not supply for Ao.
17. Goldstine and von Neumann used flow diagrams and induction assertions. Draw a flow diagram for the counted loop and place an assertion at its entry, body, and exit. Explain which assertion acts as the loop invariant.
18. Explain why Böhm and Jacopini's expressiveness result does not imply that real instruction sets should omit jump instructions. Then state the separate human-reasoning concern in Dijkstra's argument.

19. **Decode.** Write a language-neutral list of steps that reconstructs the signed RV64I B-type offset from instruction bits 31, 30 through 25, 11 through 8, and 7. Include the implicit low zero and sign extension.
20. Encode RV64I branch offsets 16, -4 , and -4096 into their B-type immediate pieces. Reconstruct each result before accepting it. State which boundary assumption changes if the compressed extension is in scope.
21. **Break.** Swap reconstructed RV64I immediate bits 11 and 12. Find one offset for which the faulty decoder agrees with the correct decoder and one for which it disagrees. Explain why a round trip through the same faulty encoder would be a weak test.
22. A short x86-64 conditional jump begins at address 250 and targets 224. Compute its signed displacement. Replace it with the selected six-byte near form at the same address and compute the repaired displacement.
23. Give bytes in which an opcode-like value occurs inside a displacement. Explain why scanning for that value cannot recover x86-64 instruction boundaries. State what evidence a proof-oriented decoder must return instead.
24. **Relocate.** Prove algebraically that moving a relative branch and its target by the same delta preserves the needed displacement. Then give three conditions under which the relocated instruction can nevertheless fail.
25. For the graph with edges $E \rightarrow A$, $E \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$, list the reachable vertices and the dominators of each vertex. Give a path witness for every proposed non-dominance result.
26. **Break.** Add edge $B \rightarrow D$ to the preceding graph. Recompute only the dominance facts that change and explain why reachability alone does not reveal the change.
27. Compute the strongly connected components of a graph with edges $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow B$, $C \rightarrow D$, $D \rightarrow D$. Draw its component graph and prove that the component graph has no directed cycle.
28. Give a directed cycle that has no natural-loop header under the stated dominance definition because it has two entries. Explain one action a compiler or analysis might take and one cost of that action.
29. In the chapter's worked loop graph, add a trap edge from the body. Recompute post-dominance under two conventions: normal exit only, and normal or trapped termination. State why the exit convention is part of the claim.

30. **Prove.** Use the predecessor-closure method to prove a dominance fact in a finite graph of your design. State why marking a vertex when only one predecessor is marked would be unsound, and give a counterexample.
31. Construct a CFG path whose consecutive edges are individually feasible but whose complete path is infeasible because the required state predicates conflict. Name the additional state fact a path-sensitive analysis must keep.
32. A profile observes a branch as taken 990 times and not taken 10 times. State one supported frequency estimate, two semantic conclusions that do not follow, and two reasons the distribution may change in production.
33. **Layout.** Arrange a diamond-shaped graph in two linear orders. For each order, identify fallthrough edges and required jumps. Under a supplied edge-frequency table, compare expected taken transfers without claiming a processor-independent running time.
34. A linker changes one branch to a longer form and moves three later blocks. Describe a fixed-point relaxation algorithm. State the termination argument if every branch form can only grow through a finite ordered set of encodings.
35. Give a case in which if-conversion is semantically invalid because an operation on the path not selected can trap. Give another in which it is semantically valid but plausibly slower because it makes a dependency longer.
36. A predictor is 95 percent accurate on one workload. List at least four quantities needed to turn that fact into a performance or economic claim. Separate facts defined by the ISA from facts about one implementation.
37. **Policy.** Define a coarse CFI target class containing three function entries. Construct a state in which a transfer reaches an allowed but application-wrong member. State the stronger application invariant that would exclude it.
38. Compare forward-edge CFI with a shadow-stack return property. Give a two-level nested-call trace for which “return to any function entry” is too weak, and identify the temporal relation a correct return must preserve.
39. Separate target arithmetic, target validity, CFI policy, and application correctness for one RV64I JALR. Give a distinct counterexample to each property while holding the earlier properties true when possible.
40. **Continuation.** A call at address 100 has length four and enters a callee at 400. State the saved continuation, then give a nested call and return sequence. Write an invariant over the abstract continuation stack.

41. Compare a link-register convention with a memory stack for nested calls. Identify which facts belong to the ISA and which require an ABI or procedure contract. Do not assume that either mechanism alone proves matching returns.
42. Design a target-soundness proof for a four-entry jump table. Include the index bound, entry-width calculation, load validity, architectural target transformation, and membership conclusion. Then state what target completeness would additionally require.
43. **Axeyum.** Map the existing toy VM's branch, CFG, symbolic reachability, and replay facilities to four Chapter 9 lessons. For each, state the additional adapter fact needed before the evidence can make an Ao claim.
44. Specify a bounded-model-checking property for the counted loop at width eight and depth 260. Explain what a satisfying model means, what an unsatisfiable bounded query means, and why neither result alone proves a width-parametric termination theorem.
45. Strengthen the preceding route with k-induction. State a base obligation, an inductive-step obligation, and a safety property. Explain why k-inductive safety still differs from termination.
46. **Negative controls.** Design mutations for an Ao branch checker that corrupt, one at a time, decode, predicate, target base, displacement scale, preserved memory, and outcome. Name the expected failure class for each mutation.
47. Create an evidence manifest for one RV64I backward branch. Include source specification revision, bytes, instruction address, decoded fields, target, target-alignment profile, semantic digest, checker, and a corrupted immediate control.
48. **Synthesis.** Prove the counted loop for one of the three machines from instruction bytes to normal exit. Organize the proof under the chapter's five branch obligations, the loop invariant, the decreasing variant, and the final observation. Mark every assumed decoder or arithmetic lemma.

Procedures, Stacks, and ABIs

A machine can transfer control and preserve a continuation without knowing what a function is. A compiler can place an argument in a register without changing that register's instruction-set meaning. A procedure emerges when separately written pieces of code agree about entry, return, values, scratch space, and preservation.

That agreement is an **application binary interface**, or ABI. It sits between source-language intention and instruction semantics. Confusing the layers produces brittle explanations: an ISA register gets called an argument register in every context, a stack-growth convention is mistaken for a hardware law, or a successful return is credited to RET without a proof of the saved address and stack discipline.

A call mechanism is not a calling convention

The ISA explains how control reaches a target and how a continuation is recorded. The ABI explains where arguments and results live, which locations survive the call, how the stack is aligned, and who restores each resource. Procedure correctness needs both layers and must name each separately.

10.1 Why procedures require agreements

Early programmers quickly learned that repeating instruction sequences by copying them was costly and error-prone. A reusable subroutine stored one body and let many sites transfer to it. Reuse created a new problem: the body had to know where to return, where to find its inputs, where to place its result, and which working locations belonged to its caller.

Assemblers and languages supplied conventions before those conventions became large platform documents. Linkers then had to connect separately assembled objects. Debuggers needed to recover call chains. Operating systems and shared libraries needed independently produced code to agree for years. The modern ABI records that durable binary agreement. It covers much more than calls, but this chapter uses its procedure-calling slice.

The idea developed through working constraints, not through one timeless calling convention. Burks, Goldstine, and von Neumann's 1946 logical-design report placed stored orders, memory, arithmetic, and control inside one automatic computing instrument (Burks et al. 1946). Reusing a stored sequence could save scarce program memory. Yet entry into that sequence did not by itself say how to return to any one of several possible callers.

The EDSAC work made that problem concrete. Wilkes, Wheeler, and Gill described closed subroutines, link orders, parameters, and a library for an operational stored-program computer (Wilkes, Wheeler, et al. 1951). Wheeler's 1952 paper then presented subroutine use to the young ACM community (Wheeler 1952). These sources document a real programming practice, but their link mechanisms should not be retold as if EDSAC already had a modern stack ABI. A return transfer could be constructed or modified, and storage conventions were shaped by that machine's orders and memory.

A library turned control transfer into cooperation

A reusable routine is more than a backward or forward edge. Many callers must agree on its entry, parameters, working storage, result, and return. The early subroutine library therefore joined a technical mechanism to a social one: programmers could build on code whose internal steps they did not rewrite.

Recursive procedures raised the storage requirement. Two active calls of the same procedure share instruction bytes but cannot generally share one set of parameters, local values, saved links, and intermediate results. The ALGOL 60 report made recursive procedure activation part of an influential language setting (Backus et al. 1963). Implementations needed a distinct activation record for every live call. A last-created, first-used stack is a natural representation when calls return in properly nested order.

The stack was not forced by the mathematical idea of a procedure. A compiler can keep a leaf activation entirely in registers, assign storage statically when recursion is excluded, split or eliminate a frame, or move an activation to another region when its lifetime outlasts ordinary stack order. The lasting foundation is one live record per required activation and a rule that finds the right record. Stack memory is the dominant implementation for ordinary nested calls because its allocation and release match their lifetime order.

As systems grew, local calling practice became a platform contract. Object files named external symbols and relocations. Linkers joined separately built units. Loaders mapped shared objects. Debuggers and exception handlers needed to recover older machine states. Foreign-language calls needed compatible data representations as well as compatible registers. The resulting ABI is a versioned interface among processors, operating systems, compilers, linkers, runtimes, libraries, and

tools. Its stability has economic value: a library can serve software built by other organizations and, within promised compatibility limits, by older toolchains.

The same need exists today in compilers, foreign-function interfaces, dynamic instrumentation, binary translation, and formal verification. A compiler may optimize freely inside a procedure while still owing a precise boundary to every caller. A proof that two function bodies compute the same number is insufficient if one corrupts a preserved register, leaves the stack incorrectly aligned before a nested call, or returns through the wrong address.

A procedure boundary makes separate construction possible

The caller need not know how the callee computes. The callee need not know why the caller wants the result. Both must know the narrow contract at the entry and exit. That controlled ignorance is what permits separate compilation.

10.2 The procedure contract

For one scalar procedure, write the contract as six declarations:

1. *entry control*: the address and machine state from which the body begins;
2. *inputs*: registers or memory locations containing arguments;
3. *outputs*: locations containing results on normal return;
4. *volatile state*: scratch locations the caller must assume may change;
5. *preserved state*: locations the callee must restore before return;
6. *return control*: the saved continuation and rule that resumes it.

Stack bounds, alignment, allowed traps, and memory ownership refine these parts. The contract also needs an observation: perhaps the result, preserved registers, stack pointer, reachable memory, and normal-return outcome. It need not require temporary registers to recover their entry values.

Caller-saved and callee-saved name obligations

A caller-saved location may be overwritten by a conforming callee, so a caller with a live value there must save it before the call. A callee-saved location must have its entry value again when the callee returns normally, so a callee that uses it must save and restore it. Neither term says who physically owns a register; each assigns a proof obligation at the boundary.

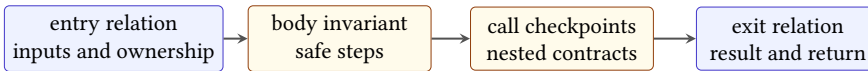
The two disciplines can implement the same preservation fact. If a value must survive a call, the caller may move it out of volatile state, or the callee may promise to preserve its chosen location. Performance, register pressure, and code size influence the division. Semantics requires only that the printed contract be met.

A value that is dead needs no ceremony

Suppose the caller places a temporary in a volatile register and never reads it after the call. The callee may overwrite it without a save. If a later edit makes the value live across the call, the old sequence becomes wrong unless the caller saves it or moves it to a preserved location. Liveness turns an ABI permission into a concrete save obligation.

10.2.1 Four layers of a procedure theorem

A useful theorem about a procedure is not merely a relation between two snapshots. It joins four layers of reasoning. The *entry relation* connects the caller's abstract arguments and resources to concrete machine locations. A *body invariant* explains why every reachable internal state remains safe and still has enough information to finish. Each *call checkpoint* establishes the entry contract of a nested callee and records what the current procedure must keep across that call. Finally, the *exit relation* connects the machine state back to the promised result, preserved resources, and continuation.



An endpoint comparison is sound only when the execution between the endpoints is justified.

Figure 10.1: A procedure proof crosses four layers. Skipping the middle two turns an endpoint comparison into an assumption about the body.

This division prevents a common circular argument. Suppose a test begins in a valid entry state and happens to finish in a valid exit state. That observation does not show that all valid entries finish, that intermediate memory accesses are permitted, or that the body cannot jump somewhere else. The trace or inductive proof must connect the endpoints. Conversely, a safe trace that halts at the wrong continuation does not establish the procedure contract.

For a straight-line leaf, the body invariant may be no more than a symbolic expression for each changed register and a claim that the next instruction is defined. A loop needs a genuine loop invariant. A non-leaf body needs a checkpoint before every nested call. The theorem should become larger only when the control structure becomes larger; the boundary vocabulary remains the same.

Normal-return procedure theorem schema

Assume an entry state satisfies the procedure precondition and entry relation. Show that each instruction step is defined and preserves the body invariant. At a nested call, derive the callee's precondition, apply its procedure theorem, and reestablish the caller's invariant from the callee's exit relation. At a normal return, derive the result relation, restored stack pointer, preserved locations, allowed memory effect, and saved continuation. This establishes only the normal-return behavior named by the theorem; a trap, divergence, or exceptional exit needs a separate clause.

10.3 Stacks and frame invariants

An architectural stack pointer is an ordinary register with special uses or conventions. Ao has none. RV64I base instructions do not force x2 to be a stack pointer; the RISC-V ABI names it sp. In x86-64, RSP has instruction-level roles in CALL, RET, PUSH, and POP, while the ABI adds alignment, frame, and preservation rules.

A downward-growing stack allocates a frame by subtracting from the stack pointer and releases it by adding the same size. “Downward” refers to numerical addresses, not to the way a diagram is drawn. The allocated frame is a byte range, and every load or store still needs an address, width, byte order, alignment policy, and range premise.

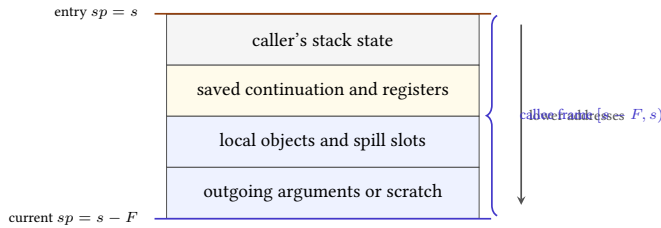


Figure 10.2: A downward-growing frame after allocation. The current stack pointer marks its low address; the caller's stack state lies at higher addresses.

Let s be the entry stack pointer and let a callee allocate F bytes. After $sp' = s - F$, its frame occupies the half-open interval $[sp', s)$. A store of k bytes at address a stays within the frame exactly when

$$sp' \leq a \quad \text{and} \quad a + k \leq s.$$

The half-open form handles adjacent objects without overlap and makes the upper-bound proof explicit.

Balanced allocation restores the stack pointer

Assume address arithmetic does not wrap and the entry pointer is s . Allocating F bytes produces $s - F$. Releasing the same frame produces $(s - F) + F = s$. This proves pointer restoration, not frame correctness. A separate argument must show that every access stayed within allocated and permitted memory and that every saved value was restored before release.

Alignment is a modular invariant. If the ABI requires alignment A , then a required checkpoint satisfies $sp \bmod A = 0$. Subtracting a frame size that is a multiple of A preserves alignment. A call mechanism that pushes a return address may deliberately change the congruence observed at callee entry, so the checkpoint and mechanism must be stated together.

Balanced arithmetic does not prove a safe frame

A prologue and epilogue can restore the same stack-pointer value while writing outside the frame, overwriting the saved return address, restoring the wrong register value, or violating alignment at an intervening call. Check the whole interval and every required checkpoint.

10.3.1 Nested stack frames

Suppose a procedure enters with stack pointer (s), allocates (F) bytes, and calls a second procedure that allocates (G) bytes. Immediately before the nested allocation, the first frame is $([s-F, s))$. After it, the nested frame is $([s-F-G, s-F))$. The intervals touch at $(s-F)$ but do not overlap. This simple calculation is the spatial reason one procedure may keep saved values in its frame while another procedure uses the stack below it.

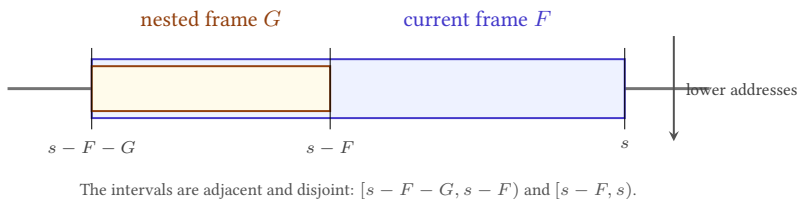


Figure 10.3: Nested downward-growing frames as half-open byte intervals. The orientation of the drawing is secondary; the address inequalities are the claim.

The argument depends on more than subtraction. Neither computation may wrap. Both intervals must lie in mapped memory that permits writes and is owned by the stack. The nested callee must respect its own frame bound rather than

store upward into the caller's frame. If it receives a pointer into the caller's frame, any write through that pointer must be separately allowed by the contract. Stack discipline supplies disjoint default regions; explicit pointer sharing can weaken that separation.

A frame proof is therefore a small memory-safety proof. For each slot, record an offset, width, alignment, and lifetime. Prove that its interval is inside the frame and disjoint from every simultaneously live slot unless intentional aliasing is part of the design. Then prove that the frame as a whole is inside the permitted stack region. Checking only the largest printed offset misses the width of the access: an eight-byte store beginning four bytes below the upper boundary crosses it.

10.3.2 Call trees and active stacks

Across an entire execution, procedure activations form a rooted tree. The initial activation is the root. Each call creates a child of the activation that issued it. Sibling calls may occur at different times, and the same procedure body may label many distinct tree vertices. The tree records history and nesting, not separate copies of the instruction bytes.

At one instant in ordinary single-threaded execution, only one root-to-current path is active. If calls return to their immediate parents, activation removal is last-in, first-out: the most recently created live child must finish before its parent continues. A linear stack can represent exactly that active path. Pushing creates a record at one end; popping removes the current record and reveals its parent.

Let $A_0, A_1, \dots, A_n$ be the active activations from the root to the currently executing one. Let F_i be the frame size of A_i , and let the entry stack pointer be s_0 . When downward allocation does not wrap, define

$$s_{i+1} = s_i - F_i.$$

Frame i occupies $[s_{i+1}, s_i)$. For $i < j$, the recurrence gives $s_{j+1} \leq s_j \leq s_{i+1}$, so the two half-open frame intervals are disjoint. They meet only at an endpoint when every intervening frame has size zero. This arithmetic is the spatial image of nested lifetime order.

Induction over call depth

At depth zero, the active-frame sequence is empty or contains only a declared root region, so its intervals are disjoint. Assume depths through n have valid, disjoint frames and current pointer s_n . A nested call that proves no wrap, sufficient mapped stack space, required alignment, and private range $[s_n - F_n, s_n)$ adds one interval below every live older interval. It cannot overlap them. On normal return, restoring s_n removes exactly that newest interval and recovers the induction hypothesis. The proof does not cover a call that escapes ordinary nesting or an activation retained after return.

The last sentence matters for modern language implementations. A **coroutine** is a routine that can suspend one activation and resume another. A continuation can preserve more than the immediate return address. A closure can keep referenced data alive after the declaring call returns. A compiler may place such state in a heap object, a segmented stack, or a runtime-managed frame. These mechanisms do not invalidate the stack proof. They fall outside its premise that live procedure activations form one properly nested path represented by adjacent intervals.

A stack diagram is a theorem about lifetimes

Boxes drawn above one another are justified only when allocation, nesting, range, and release premises hold. An escaped pointer, suspended activation, or transfer outside the current call nesting can keep data live after the pictured pop. State the lifetime model before using the diagram as a proof.

10.3.3 Space exhaustion and address wrap

The recurrence $s' = s - F$ can be arithmetically defined while the intended frame is not backed by usable memory. A finite thread stack has a resource limit. Guard pages can turn growth beyond a boundary into a fault. Some systems require a large allocation to touch intervening pages so one subtraction cannot jump over a guard region. That page-touching sequence is often called a **stack probe**.

A frame theorem therefore separates at least four questions. Does subtraction wrap at the machine width? Does the resulting interval lie within the thread's permitted stack region? Does every needed page become mapped or committed under the platform rule? Does each load or store stay within the allocated frame and use permitted alignment? Equality after adding F back answers only the first question under its arithmetic premises.

Stack size also has an industrial cost. A larger reserved or committed stack can reduce the number of threads or tasks a process can support. A smaller one can make deep recursion or large local objects fail earlier. Split stacks, heap-allocated activations, and stackless asynchronous state machines choose different allocation and scheduling costs. These are runtime and workload decisions, not properties of CALL, RET, or an ISA stack-pointer register.

10.4 Procedures in Ao

Core Ao has direct relative jump, conditional branches, and eight ordinary registers. It has no call instruction, register-indirect jump, implicit stack pointer, or ABI. That is a useful boundary rather than an omission to conceal.

A fixed-call-site teaching program can jump directly to a body and let that body jump directly back to one known continuation. It can designate r_6 as a stack pointer and r_7 as a saved link by convention. Yet core Ao cannot use an arbitrary

value in r_7 as the next program counter. A body shared by multiple callers therefore needs duplicated return blocks, a dispatch scheme, self-modifying code outside the model, or an explicit instruction-set extension.

Do not smuggle a return instruction into Ao

Writing a return address into r_7 does not make `jump r7` legal when the Ao specification defines only a relative-immediate jump. Examples must either use a fixed direct return or declare an Ao extension with its own encoding, semantics, traps, and evidence controls.

This limitation clarifies the construction order. First define a procedure contract independently of any particular call opcode. Then show a fixed Ao instance satisfying it. If the book later adds an indirect-jump extension, prove that its general call/return sequence refines the same contract. The contract survives even as the mechanism becomes richer.

For a fixed-return Ao leaf, choose conventions only for the example: r_0 holds one input and the result, r_6 is an aligned stack pointer, r_4 and r_5 are preserved, and r_1, r_2, r_3, r_7 are volatile. The body may compute without a frame. Its proof must show the result relation, preservation of r_4, r_5, r_6 , permitted scratch changes, unchanged memory, and a direct jump to the unique continuation.

10.5 Procedures in RV64I

RV64I JAL writes $pc + 4$ to a destination register and transfers to a PC-relative target. The common call convention writes the link to x1, named ra. JALR can transfer to an address computed from a register and immediate while writing another link. The assembler spelling `ret` denotes the appropriate base-instruction form that jumps through ra; it is not a new architectural instruction (RISC-V International 2026e).

The RV64 integer ABI gives selected registers procedure roles. It names x2 as sp, uses a0–a7 for integer arguments, uses a0 and a1 for integer return values, classifies t0–t6 as temporaries, and requires s0–s11 to be preserved. The standard stack grows toward lower addresses and is aligned to a 128-bit boundary at procedure entry; the standard ABI requires that alignment through procedure execution (RISC-V International 2026c).

Consider a leaf function that adds one to its first integer argument:

```
add_one:
    addi a0, a0, 1
    jalr x0, 0(ra)
```

Listing 10.1: An RV64I leaf under the selected integer ABI

The ISA explains both instructions. The ABI explains why entry `a0` is the argument, exit `a0` is the result, `ra` contains the caller's continuation, and no preserved register needs saving. The mathematical precondition decides whether the desired result is modular addition or ordinary integer addition without wrap.

A non-leaf function cannot assume `ra` will survive its own nested call: the nested linking jump writes a new continuation there. It must preserve its incoming return address, often in its stack frame, before making the call and restore it before returning. That need follows from liveness plus the call mechanism, not from a rule that every function must have the same prologue.

Find the first overwritten continuation

Trace a function entered with `ra=100` that immediately calls another function using `jal ra, target`. Record `ra` after the nested call. Explain why a later `jalr x0, 0(ra)` cannot return to 100 unless the original value was kept elsewhere and restored.

10.5.1 A complete RV64 frame

Consider a non-leaf procedure that must preserve its incoming continuation and the preserved register `s0`. It also keeps the input in `s0` across a call to `helper`. One selected uncompressed RV64I sequence is:

```
addi sp, sp, -16
sd   ra, 8(sp)
sd   s0, 0(sp)
addi s0, a0, 0
jal  ra, helper
add  a0, a0, s0
ld   s0, 0(sp)
ld   ra, 8(sp)
addi sp, sp, 16
jalr x0, 0(ra)
```

Listing 10.2: A selected RV64I non-leaf frame

If entry $sp = s$, the frame is $[s - 16, s)$. The saved `s0` slot is $[s - 16, s - 8)$, and the saved `ra` slot is $[s - 8, s)$. Both eight-byte stores lie inside the frame, the slots are disjoint, and subtracting 16 preserves the standard 16-byte stack alignment. The nested call may replace `ra` and volatile argument registers, but its contract must preserve `s0`, `sp`, and both caller-owned save slots. After return, the two loads recover the entry values before the frame is released.

For this exact register assignment, the little-endian bytes outside the relocated call are:

Instruction	Bytes in address order
<code>addi sp, sp, -16</code>	13 01 01 FF
<code>sd ra, 8(sp)</code>	23 34 11 00
<code>sd s0, 0(sp)</code>	23 30 81 00
<code>addi s0, a0, 0</code>	13 04 05 00
<code>add a0, a0, s0</code>	33 05 85 00
<code>ld s0, 0(sp)</code>	03 34 01 00
<code>ld ra, 8(sp)</code>	83 30 81 00
<code>addi sp, sp, 16</code>	13 01 01 01
<code>jalr x0, 0(ra)</code>	67 80 00 00

The `jal` bytes depend on the final distance to helper; a relocation or final layout calculation must bind them. At a call address whose helper is 48 bytes ahead, `jal ra, 48` is `EF 00 00 03`. Publishing fixed call bytes without publishing the call address and target would leave the central field unjustified.

RV64 non-leaf preservation

Assume valid aligned entry, no address wrap, frame bytes that permit writes, a valid call target, and a helper contract that returns normally while preserving `s0`, `sp`, and the two slots. The prologue stores the entry `ra` and `s0` in disjoint owned intervals. The body keeps the original input in `s0`; the helper cannot change it. The epilogue reloads the stored entry values, releases exactly 16 bytes, and jumps through the restored `ra`. Thus the procedure returns to its caller with entry `s0` and `sp`, subject to its declared result in `a0`. Every premise is needed by a specific instruction or nested contract.

10.5.2 Leaf procedures

A **leaf procedure** makes no procedure call on the path under discussion. A **non-leaf procedure** does. The distinction does not say that a leaf is short, uses no stack, or cannot loop. A leaf may allocate a large frame, spill registers, and run for millions of steps. A non-leaf may contain only a few instructions around one call.

The distinction matters because a leaf can often keep its incoming continuation in the location established by its caller. On RV64, a nested `jal ra, target` replaces `ra`, so a non-leaf procedure with a live incoming continuation must keep it elsewhere. On x86-64, a nested `CALL` pushes another return address below the current one. The original continuation remains in memory, but its address relative to the moving stack pointer changes with frame allocation and additional pushes. These are different mechanisms producing the same proof obligation: the current procedure must still be able to reach its caller after the nested procedure returns.

Leaf status can be path-sensitive. A fast path may return without calling, while a slow path invokes a helper. The whole procedure is ordinarily treated as non-leaf

for frame design unless the paths have separately justified entry and exit sequences. An optimizer that removes the last call may remove a now-unneeded continuation save, but it must prove that no remaining path or indirect transfer invokes a callee.

10.6 Procedures in x86-64

In 64-bit mode, a selected near `CALL` records the address of the following instruction on the stack and transfers to its target. A matching near `RET` obtains the return instruction pointer from the stack and advances the stack pointer. The exact operand, width, canonical-address, and fault rules belong to the pinned instruction forms in Intel’s reference (Intel Corporation 2026b).

The System V AMD64 ABI assigns the first six integer or pointer arguments to `RDI`, `RSI`, `RDX`, `RCX`, `R8`, and `R9` when their classifications permit. It uses `RAX` for the first ordinary integer return value. Among the general-purpose registers, `RBX`, `RBP`, and `R12–R15` are preserved across ordinary calls; many others are volatile. Immediately before an ordinary call, the selected stack alignment rule makes the end of the input argument area a multiple of 16 bytes. After the call pushes an eight-byte return address, $RSP + 8$ has that alignment at function entry (x86 psABIs Project 2023).

```
add_one:
    lea rax, [rdi + 1]
    ret
```

Listing 10.3: An x86-64 System V leaf function

The selected `LEA` form computes the result without changing arithmetic flags. That detail is not required by the basic integer return contract, but it can matter to a stronger context. The function uses no preserved register and allocates no frame. `RET` nevertheless reads stack memory and changes `RSP`; calling it a register-only leaf would hide those architectural effects.

The System V red zone is a further ABI rule: qualifying leaf code may use a specified region below the current stack pointer without first adjusting `RSP`, subject to the ABI’s interruption rule. It is not a general x86-64 hardware property and is not shared by every x86-64 calling convention. This chapter’s worked proof does not rely on it.

10.6.1 A System V stack frame

Suppose a System V AMD64 procedure needs to preserve `RBX`, keep its first argument across a helper call, and reserve 32 bytes of private scratch. At entry, let $RSP = s$, with the caller’s continuation stored in $[s, s + 8)$ and $s + 8$ divisible by 16. One selected sequence is:

```
push rbx
```

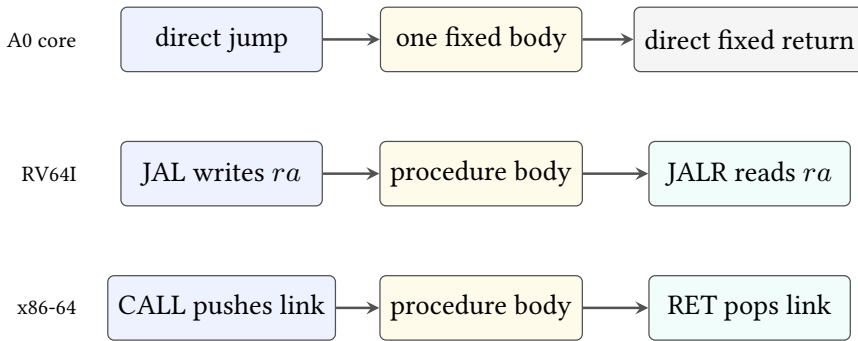


Figure 10.4: Selected call and return mechanisms. The ABI assigns meanings to the link location, stack pointer, argument, result, and preserved state.

```

sub    rsp, 32
mov    rbx, rdi
call   helper
add    rax, rbx
add    rsp, 32
pop    rbx
ret
  
```

Listing 10.4: A selected System V AMD64 non-leaf frame

The push first changes RSP to $s - 8$ and stores entry RBX in $[s - 8, s)$. Because $s + 8$ is divisible by 16, $s - 8$ is also divisible by 16. Subtracting 32 produces $s - 40$, still divisible by 16 at the nested-call checkpoint. The private area is $[s - 40, s - 8)$. The save slot and caller continuation are outside that private interval; no two of the three intervals share a byte.

The fixed bytes are 53 for push rbx, 48 83 EC 20 for sub rsp, 32, 48 89 FB for mov rbx, rdi, 48 01 D8 for add rax, rbx, 48 83 C4 20 for add rsp, 32, 5B for pop rbx, and C3 for ret. A direct near call begins with E8 and carries a four-byte relative displacement fixed by layout and relocation. As in the RV64 example, opcode evidence and relocation evidence are separate.

On return from helper, RSP again equals $s - 40$. Adding 32 reveals the saved RBX; pop restores it and changes RSP to s ; ret reads the original continuation and changes RSP to $s + 8$, the caller's value before its call. The proof must account for both stack changes made implicitly by call and ret. Merely showing that the explicit additions cancel the subtractions is incomplete.

The return address is caller-owned live data

The current procedure may read the stacked continuation as part of return, but it may not treat the word as private scratch. A frame layout that includes $[s, s + 8)$ in the callee's local region that permits writes has erased the very value needed to justify control after `ret`.

10.6.2 The Microsoft x64 agreement

Microsoft x64 assigns the first four ordinary integer or pointer parameter positions to RCX, RDX, R8, and R9, subject to its type rules. The caller also reserves a four-slot parameter area on the stack, often called **shadow space**, even when the callee receives those four values in registers. A callee may use the area to save the incoming register parameters. System V AMD64 instead begins its selected integer sequence with six registers and defines a red zone for qualifying code. The two agreements must not share one frame diagram (Microsoft Corporation 2025).

The preserved-register sets also differ. Microsoft x64 treats RDI and RSI as nonvolatile, while System V treats them as volatile argument registers. Microsoft preserves the low 128 bits of selected vector registers; System V's vector-register rules differ. A bridge compiled for one convention must save, move, and restore values according to both sides. Using the same `CALL` opcode does not make the boundaries compatible.

Microsoft unwindability constrains non-leaf prologues and epilogues so unwind data can describe their effects. This creates a direct connection among code generation, exception recovery, debugging, and binary compatibility. A custom assembly routine can execute legally while failing the platform contract because its frame changes cannot be recovered through the required metadata.

Table 10.1: Selected x86-64 procedure differences. The rows are ABI rules, not instruction-set properties.

Question	System V AMD64	Microsoft x64
First integer positions	RDI, RSI, RDX, RCX, R8, R9	RCX, RDX, R8, R9
Caller stack area	overflow arguments and selected save areas	four-slot parameter area plus overflow arguments
Area below current RSP	selected 128-byte red zone	no equivalent general red-zone promise
Selected preserved difference	RDI, RSI volatile	RDI, RSI nonvolatile
Unwind route	DWARF call-frame information on common ELF systems	platform function tables and unwind codes

The contrast has an economic dimension. One platform ABI lets independently built compilers, libraries, debuggers, profilers, and operating-system tools share

binaries. Supporting another ABI requires new calling sequences, object-format integration, unwind generation, tests, and maintenance. A cross-platform compiler backend earns compatibility by implementing those contracts; it does not obtain compatibility merely by emitting x86-64 instructions.

10.6.3 ISA effects and ABI composition

It is tempting to divide the layers by vocabulary: instructions belong to the ISA, while register names belong to the ABI. The real boundary is functional. The ISA defines possible state transitions. The ABI selects some of those transitions and adds conditions under which separately built components may compose.

For example, RV64 JAL can place a link in many destination registers; the procedure convention commonly selects x1. An x86-64 near CALL has an architectural stack effect, but the ABI determines the alignment and ownership facts that must hold around it. A direct jump is an ordinary ISA transfer; it becomes a tail call only when the outgoing argument state satisfies the next procedure's entry contract and the current procedure's obligations have already been discharged.

This gives a useful proof order. First apply the instruction rule to establish the exact link, program-counter, register, and memory effects. Next apply the ABI rules that interpret the affected locations as a continuation, argument, result, stack area, or preserved resource. Finally apply the procedure theorem that connects the abstract caller and callee. Reversing that order risks using the word "call" as if it already guaranteed a valid target, valid stack, and eventual return.

The distinction also explains why an ABI violation need not be an illegal instruction execution. A body can follow every ISA rule while entering with a misaligned stack, returning a value in the wrong register, or damaging a callee-saved register. Architectural validity is necessary for ABI conformance, but it is not sufficient.

Name the x86-64 ABI, not merely the ISA

System V AMD64 and Microsoft x64 use the same broad instruction set but differ in argument registers, stack-area rules, and preservation details. A theorem about one calling convention must not be labeled only "x86-64." Name the ABI and the selected platform assumptions.

10.7 Argument classification

An argument list is a typed mathematical sequence. An ABI must map that sequence into a finite set of register positions and a memory layout. The map depends on type size, alignment, representation class, earlier arguments, and the selected calling convention. It is not correctly summarized by saying "arguments go in registers."

For a simple integer-only model, let $R = (r_0, \dots, r_{m-1})$ be the ordered argument-register positions. A scalar word consumes the next free position. If no position remains, the caller stores the argument in a stack argument area with the required alignment. A two-word value may need two positions and can force an unused gap when the ABI requires an aligned pair. A value too large for direct passage may instead consume one pointer position while the caller places the object in memory.

This is a small allocation algorithm. Its state is the next register position and next stack offset. Its invariant says that each processed argument has one declared location, stack intervals are aligned and disjoint, and the next free locations do not overlap earlier assignments. The final map becomes part of both caller and callee preconditions.

Agreement by one deterministic classifier

Let C be a deterministic classification function of the ABI revision, type sequence, and calling-convention variant. If caller and callee use the same inputs to C , induction over the argument sequence gives the same location for every argument: the initial allocation states agree, and each classification step consumes the same register positions and stack bytes. If a type layout, variant, or ABI revision differs, the induction premise fails. Matching source-level parameter names cannot repair the binary map.

The standard RISC-V integer convention provides eight positions, a0 through a7. A scalar no wider than XLEN uses one available register or is passed on the stack when none remains. Selected values up to twice XLEN can use a register pair; larger aggregates are passed by reference. Stack arguments follow type and machine-word alignment rules up to the stack alignment. The specification gives additional rules for floating-point, vectors, C++ objects, and **variadic procedures**. A variadic procedure has a fixed named prefix but does not fix the type and position of every later argument (RISC-V International 2026c).

System V AMD64 first classifies an argument into pieces and classes before it assigns registers or memory. A small structure is not automatically a pointer, and a floating-point field need not consume an integer register. When a value cannot be passed under the register rules, it is placed in the caller's stack argument area with its required layout. The full algorithm has cleanup and class-merging rules; a proof should import the official classifier or implement and test that algorithm rather than invent a mnemonic shortcut (x86 psABIs Project 2023).

The ninth RV64 integer argument crosses the boundary

Take nine independent 64-bit integer arguments under the selected standard RV64 integer convention. Arguments zero through seven use a0 through a7. At procedure entry, argument eight begins at stack offset zero. The caller must make that memory valid and aligned; the callee must load the same eight bytes. If an earlier argument expands to two register positions or is passed by reference, run the classifier again. Counting source parameters is not enough.

10.7.1 Large return values

When a result does not fit the direct return locations, an ABI can require the caller to allocate result storage and pass its address as an implicit argument. The procedure then writes the result through that pointer. The apparent output has become an input capability plus a memory effect.

A proof needs the hidden pointer's location, range, alignment, ownership, and alias relation to ordinary arguments. It must show that every result store lies inside the caller-provided object and that required padding or untouched bytes follow the ABI and language contract. The returned register value, if any, may have a separate rule. Guessing that it contains the hidden pointer can create a false portability claim.

This mechanism explains why source signatures alone do not determine machine signatures. Two compilers that disagree about structure layout, triviality, or calling-convention variant can place the same apparent result differently. Foreign-function declarations must match the binary classifier as well as the source type spelling.

10.7.2 Variadic calls

A variadic procedure accepts a fixed named prefix followed by a number of additional arguments chosen at each call. The callee does not have a complete declared type sequence for those additional values. The ABI therefore defines placement, promotion, save-area, and traversal rules so runtime code can find them.

RISC-V's selected convention can require an aligned register pair for a two-word-aligned variadic argument; once such an argument goes to the stack, later variadic arguments also go there. Its `va\_list` route can require the callee to save unused incoming argument registers into an ordered area before traversing them (RISC-V International 2026c). System V AMD64 has its own register-save-area and metadata rules. Microsoft x64 duplicates selected floating-point values into general registers for unprototyped or variadic calls under its documented cases (Microsoft Corporation 2025).

The security and correctness lesson is direct. The runtime traversal trusts the caller and format-specific code to agree about count and types. Reading an extra value or reading one under the wrong type can use the wrong width, alignment, or register class. A format-string vulnerability is therefore not merely a text problem; it can become an incorrect walk through an ABI-defined argument representation.

Do not teach the exceptional case as the ordinary rule

This chapter’s proofs use fixed scalar signatures. Aggregate and variadic examples explain why the full ABI is richer; they do not silently extend the narrow add-one theorem. Every implemented classifier must name its admitted types and reject the rest.

10.8 One leaf contract in three machines

Define a mathematical procedure I that accepts a width- w word x and returns $x + 1 \bmod 2^w$. Its boundary contract observes the result, normal return to the caller’s continuation, restored stack pointer, preserved registers, and all memory outside declared scratch. It permits named volatile registers and condition state to differ.

The three instances relate their input and result locations:

Table 10.2: A narrow procedure relation for the add-one leaf examples.

Machine	Input/result	Continuation	Preserved slice
A0 fixed case	r_0	known direct target	r_4, r_5, r_6
RV64 ABI	a0	ra	sp, s0–s11
SysV AMD64	RDI/RAX	top stack word	RSP, RBX, RBP, R12–R15

The table is deliberately a slice. The full ABIs classify floating-point, vector, aggregate, variadic, and memory arguments; define more registers and process state; and interact with linking and unwind information. A proof of these leaf examples imports only the rows it uses.

Reader proof for the shared add-one contract

Relate the three entry argument words to one arbitrary x . Each body stores $x + 1 \bmod 2^w$ in its declared result location. None writes memory or a declared preserved data register. Ao reaches its one fixed continuation by a direct jump. RV64I reads the unchanged incoming ra in JALR and leaves sp unchanged. x86-64 RET reads the continuation written by CALL and restores RSP to its pre-call value after consuming that word. Therefore the related

observations agree on result, normal return, preserved slice, stack restoration, and nonscratch memory.

This proof contains machine-specific premises. The x86-64 case assumes the return-address load is valid and contains the caller's continuation. The RV64 case assumes `ra` names a valid aligned return target under the selected semantics. The Ao case has only one statically fixed caller. If any premise is removed, the common mathematical result does not rescue the control claim.

10.9 Nested calls and frame preservation

A nested call makes preservation visible. Suppose a function receives x , computes an intermediate u , calls another function, and then combines its result with u . The value u is live across the call. If it occupies a volatile register, the caller must save it. If it occupies a callee-saved register, the current function must restore that register for its own caller before returning.

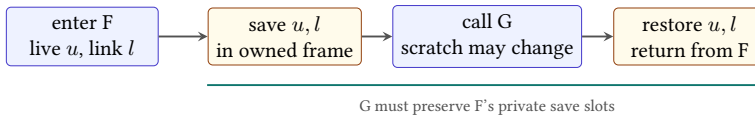


Figure 10.5: Preservation obligations cross call boundaries. A live intermediate and the caller's continuation must survive the nested call by declared means.

A conventional frame may contain a saved continuation, saved preserved registers, local objects, spill slots, and outgoing stack arguments. Their layout is not established merely by drawing boxes. Each slot needs a byte interval; distinct live objects need disjoint intervals unless sharing is intentional; each access needs the right width; and the epilogue must load the same saved values before deallocation.

Preservation by save, call, and restore

Let location q contain entry value v , and suppose the callee may change q . Before the nested call, store v in a private frame slot m . The call may change q but must not change m under the nested contract. After return, load m into q . The final value of q is then v . The proof depends on slot validity, the absence of nested writes to the same bytes, correct width and byte order, and normal return to the restore instruction.

10.9.1 Normal and exceptional exits

Unwinding after exceptions is harder because the ordinary epilogue may not run. A procedure may also trap on an invalid access, fail a checked arithmetic operation, receive an asynchronous signal, diverge, or terminate the process. Those outcomes cannot be silently folded into a theorem whose conclusion says that control returns to the saved continuation.

One clean specification partitions behavior. On a normal exit, the result and ABI preservation clauses hold. On a named exceptional exit, a different relation describes the transferred control, visible state, and resources that must remain recoverable. Divergence is a trace property rather than an exit state. A partial-correctness theorem may say what follows *if* normal return occurs; a total-correctness theorem must additionally rule out every other behavior permitted by its model.

Platform unwind metadata and language exception rules provide a route to recover saved registers and older frames without executing the ordinary epilogue. The metadata is then part of the binary contract and deserves its own validation: its description of frame changes must agree with the actual instructions at each covered program point. Debug information that merely prints a plausible reconstructed call history is weaker evidence than a checked correspondence between unwind rows and instruction semantics. Full unwinding remains outside this first scalar model, but the boundary is now explicit (DWARF Debugging Information Format Committee 2017).

10.9.2 Unwind information

DWARF call-frame information uses a **canonical frame address**, or CFA, as a stable reference from which an older frame’s values can be recovered. A row applies over a range of instruction addresses and gives rules such as “the caller’s stack pointer is CFA” or “the caller’s saved register is eight bytes below CFA.” The rules can change after each prologue or epilogue instruction because the machine state changes there.

Return to the System V sequence. At function entry, one possible descriptive row uses $CFA = RSP + 8$: the caller’s pre-call stack pointer. After `push rbp`, current RSP equals $s - 8$, so the same address is $CFA = RSP + 16$, and saved RBX lies at $CFA - 16$. After `sub rsp, 32`, current RSP equals $s - 40$, so $CFA = RSP + 48$, while the saved register remains at $CFA - 16$. One fixed offset from current RSP could not describe all three ranges.

The metadata checker must walk instruction effects and metadata rows together. At each covered program point, it can construct a symbolic caller state, apply the row’s recovery expressions to current state, and compare the result with the actual entry values. A mutation that omits the stack subtraction from the CFA update should fail only after that instruction. A mutation that moves the saved-register offset should fail as soon as the save is claimed.

A successful backtrace is one witness

A debugger that walks one stopped execution shows that its selected rows and memory happened to produce plausible older addresses. It does not prove every covered program point, every prologue path, or every register rule. Validate the metadata against instruction semantics and retain failing address ranges.

Fault ordering also matters. If a prologue adjusts the stack pointer and then faults while saving a register, the machine is neither at the entry boundary nor at the normal exit boundary. A proof system that models faults needs the intermediate state and the architectural rule selecting the reported fault. If the theorem excludes faults, its precondition must be strong enough to make every prologue, body, and epilogue access valid.

10.10 Protecting return control

The continuation is ordinary machine state with extraordinary authority. On x86-64 it occupies stack memory that permits writes. In a conventional RV64 non-leaf frame, a saved copy of `ra` does too. If an erroneous or hostile store replaces that word, the ordinary epilogue may restore every other value and then transfer control to an address chosen by the replacement. Balanced stack arithmetic does not establish continuation integrity.

Suppose a frame contains a local byte array below a saved continuation. An unchecked copy that crosses the array's upper bound can reach the saved word. The exact direction depends on the frame layout, but the proof issue does not: the local object's write-permitted interval must be disjoint from every protected slot, and each store must remain inside the interval it owns. Chapter 4's range and alias rules therefore become security premises at a procedure boundary.

A **stack canary** is a value placed between selected local objects and protected frame state. The prologue writes the expected value. Before an ordinary return, the epilogue compares the current value with that expectation and takes a failure path if they differ. The comparison can detect a contiguous overwrite that crosses the canary on its way to the continuation. It does not make the overwrite impossible.

The canary argument has narrow premises. The attacker must not learn and reproduce the expected value; the bad write must affect the checked bytes; and every exit that can return must perform the check before using the continuation. A noncontiguous write may skip the canary. An arbitrary-address write may target the continuation directly. A canary also says nothing about an overwritten function pointer, object field, or saved data register. Its property is detection for a selected frame layout and corruption class, not general control-flow integrity.

A **shadow stack** keeps a protected second copy of return addresses. On a call, the mechanism records the expected continuation in the ordinary call state and

in the protected shadow state. On return, it compares the continuation about to be used with the protected copy. Equality permits the return; disagreement raises a control-protection failure rather than following the corrupted ordinary copy. Intel Control-flow Enforcement Technology supplies such a return-protection mechanism for selected x86-64 environments (Intel Corporation 2020).

Let ordinary continuation stack K and protected stack H initially agree. A well-nested call with continuation (k) appends (k) to both. A well-nested return requires their last elements to agree and removes both. Induction over the call history then preserves $K = H$ for executions that use only these rules. If an ordinary-memory store changes the last element of K without changing H , the next protected return rejects it. This is the reader proof of the central comparison. A machine proof must additionally model the architectural shadow-stack state, access restrictions, exception delivery, and the exact call and return forms in scope.

The induction also exposes the boundary. A shadow stack protects saved return continuations against selected corruption. It does not prove that the callee target of an indirect call was valid, that a tail jump reached an allowed entry, or that the called procedure obeyed its data contract. Those are forward-edge or functional properties. RISC-V control-flow-integrity extensions likewise distinguish landing-pad checks for selected indirect transfers from return-address-stack protection; the mechanism and enabled extension must be named rather than compressed into the phrase “CFI” (RISC-V International 2026a).

Table 10.3: Three defenses answer different questions. None replaces memory safety or the procedure’s functional contract.

Mechanism	Checked event	Important limit
Stack canary	selected frame value still matches before return	corruption may bypass, preserve, or avoid the checked value
Shadow stack	return continuation matches a protected copy	does not validate arbitrary indirect-call or jump targets
Landing-pad or target check	selected indirect transfer reaches an allowed entry form	does not prove the target procedure’s result or preservation behavior

These defenses also change the industrial cost model. A compiler may add prologue and epilogue instructions, reserve a register or architectural state, emit new metadata, and constrain tail-call or hand-written assembly patterns. An operating system must save, restore, and authorize the added state across threads and exceptions. A loader may need to reject or adapt binaries whose declared protection properties do not match their code. Compatibility is not free: old libraries, just-in-time compilers, signal trampolines, debuggers, and foreign runtimes all meet the procedure boundary.

The correct theorem therefore names the defense and its observation. For a canary, prove that a selected corruption class cannot reach a normal return without a mismatch. For a shadow stack, prove that a return using a different continuation cannot complete normally under the enabled mechanism. For an indirect-target check, prove that selected transfers reach only accepted landing sites. None of those conclusions proves memory safety, semantic equivalence, or resistance to every code-reuse attack.

10.11 Observing ABI conformance

A function need not restore volatile registers, temporary flags, or bytes in its private frame after release when the contract makes them unobservable. It must restore the stack pointer and every callee-saved register on normal return. It must not damage caller-owned memory. The return value must appear in the declared location, and control must reach the saved continuation.

The rule is observational equivalence with a structured boundary. Two bodies may use different frame sizes, instruction counts, scratch registers, and internal control flow while conforming to the same contract. Conversely, identical return values do not establish conformance if a preserved component differs.

The smallest useful ABI counterexample

Run two add-one bodies from related entries. Both return the expected result. One also replaces a preserved register value 7 with 8. The result-only observation reports agreement; the ABI observation reports the concrete register mismatch. Appending a caller that reads that register turns the boundary failure into a visible program difference.

Tail calls alter the return path. A proper tail transfer may reuse the current caller's continuation instead of creating another one, but only after the current frame and preservation obligations have been discharged. Calling a jump a tail call does not prove that the ABI state at the target is valid.

10.11.1 Tail calls

Let caller A call procedure F with continuation k . An ordinary call from F to G creates another continuation k_F ; G returns to F , and F later returns to k . A tail call removes the need for k_F . Procedure F establishes G 's entry contract and transfers control while arranging for G 's normal return to use k directly.

The transformation needs four groups of facts:

1. F 's result obligation is exactly the result that G will deliver, or a declared relation connects them without further work in F ;

2. outgoing arguments occupy the locations required by G , including any stack arguments whose moves might overlap incoming ones;
3. F has restored its preserved registers, released or safely reused its frame, and retained every caller-owned byte; and
4. the transfer mechanism gives G the original continuation and a valid ABI entry state.

Stack arguments make the second item subtle. Moving several values within one overlapping area can overwrite a source before it has been copied. The compiler needs a safe move order, a temporary, or a parallel-copy algorithm. Register arguments can have the same issue when the source and destination assignments form a cycle. Tail position in source syntax does not solve this machine-level packing problem.

Tail-call composition

Assume F 's entry contract from A , a verified prefix from that entry to the tail checkpoint, and the four obligations above. The checkpoint state satisfies G 's entry contract with continuation k . Apply G 's procedure theorem. Its normal exit returns the promised result and preserved state to k . Because F owed no computation after the call and had already discharged its frame and preservation duties, this state also satisfies F 's normal-exit contract to A . No intermediate return to F is needed.

Tail calls can reduce maximum stack depth and remove call/return work. They can also make debugging or profiling histories less direct, require argument shuffles, inhibit platform unwind expectations, or be unavailable across calling conventions. A performance claim therefore names the generated sequence, ABI, toolchain, and workload; the semantic theorem names the continuation and boundary relation.

10.12 Binary boundaries

Separate compilation works because a producer can publish a symbol and binary contract without publishing its internal instruction choices. A consumer can compile a call from the declaration and ABI. The linker then resolves symbol references and relocations; a loader may bind shared-library references later. The procedure theorem depends on that binding reaching a body that satisfies the declared contract.

Dynamic linking adds identity and time. A procedure linkage sequence may first enter a resolver, update a table, and only later jump directly to the selected body. Symbol interposition can permit another definition to replace the one a compiler expected. Versioning can restrict compatible definitions. These mechanisms affect

which code implements a symbol and which extra state changes occur around the call. They do not change integer addition, but they can change the whole-program target and observation.

Stable ABI boundaries have substantial economic value. Operating systems and language distributions can update shared libraries without rebuilding every consumer. Hardware vendors can ship new implementations that run existing binaries. Tool vendors can support debugging and profiling through common unwind and symbol formats. The same promise creates costs: old layouts and calling rules may constrain future design, and compatibility testing must cover code produced by many toolchains and years.

10.12.1 Foreign-function interfaces

A **foreign-function interface**, or FFI, connects components whose source languages or runtimes differ. Selecting the same platform calling convention is necessary, but usually not sufficient. At least five agreements must be explicit:

1. the binary calling convention and symbol name identify the same entry;
2. every shared value has one compatible size, alignment, field layout, and valid-value set;
3. pointers carry agreed ownership, permission to write, lifetime, and alias rights;
4. allocation and release occur through compatible owners; and
5. exceptions, panics, cancellation, and callbacks cross the boundary only through declared routes.

Consider a pair (p, n) passed as pointer and length. Register placement does not prove that $[p, p + n)$ is readable, that multiplication for element width does not wrap, that the data lives for the call, or that the callee may retain the pointer. Those facts belong to the FFI contract above the ABI. A safe language wrapper may check them before entering a smaller unsafe core.

Layout is similarly load-bearing. A source record with the same field names can use different padding, enumeration tags, Boolean representations, or niche values in two languages. A stable FFI type needs a representation promise and tests or proofs tied to the compiler and platform profile. Serialization is an alternative boundary: it pays conversion cost to replace in-memory layout agreement with a byte-format agreement.

Do not unwind through an undeclared language boundary

One runtime's exception or panic object may not satisfy another runtime's unwind, cleanup, or ownership rules. Catch it before the boundary or use a specified cross-language exception ABI. A matching stack pointer at entry does not make arbitrary nonlocal control safe.

The industrial choice among a direct ABI call, generated binding, remote procedure call (RPC), serialization, or process boundary trades latency, copying, isolation, deployment independence, and compatibility. The narrowest correct boundary is not always the cheapest to maintain. A proof should state the chosen boundary; a cost study should measure it under the expected message sizes and call rates.

10.13 Axeyum route

The audited Axeyum checkout at revision `a9991fda` already has a real modular-call proof route at the checked scalar LLVM level. It can parse selected direct calls, verify a scalar leaf body against a `ScalarCallContract`, replace that body with a verified contract, and compare the modular transition system with the inlined one. Its tests bind source and body digests, propagate requirements and conditions under which each operation has a defined result, replay failing call sites in source, exercise mutation controls and large finite differentials, and feed bounded checking or k-induction.

That route is narrower than this chapter. It deliberately rejects callee memory, nested calls, indirect calls, non-scalar signatures, and broader control flow. It models LLVM scalar values, not an architectural stack, continuation word, prologue, epilogue, or platform ABI. A parsed LLVM `tail` marker does not prove a machine-level tail-call contract. The checkout still lacks executable book `Ao` and source-pinned RV64I/x86-64 procedure semantics, System V or Microsoft ABI packages, unwind rows, and cross-language boundaries.

The first book implementation should reuse this verified-contract pattern but add memory and continuation state. It should not begin by encoding an entire platform ABI or by building an endpoint-only checker that ignores the trace.

Represent an entry observation, a normal-return observation, input and result maps, volatile and preserved location sets, stack-pointer checkpoints, owned memory intervals, and an expected continuation. A trace adapter supplies the first and last related states. The contract checker compares every promised component and reports the first mismatch by class.

Then connect one `Ao` fixed-return trace. Its positive case computes add-one and preserves the declared slice. Controls change the result, one preserved register, the final stack pointer, a caller-owned byte, and the continuation. Every mutation must fail through the corresponding contract clause.

RV64I and x86-64 adapters require source-pinned decoder and step packages before their traces count as machine-code evidence. RV64 controls should overwrite `ra` across a nested call, omit a saved `s` register, or misalign `sp`. x86-64 controls should damage the stacked return address, restore the wrong RSP, omit an RBX restore, or enter a nested call at the wrong alignment.

Contract checking and semantic checking are different

A contract checker can prove that two supplied endpoint states satisfy an ABI relation. It does not prove that instruction bytes generated those endpoints. A trace checker connects every adjacent state to executable semantics. A complete evidence route needs both, plus decoder provenance and controls that fail in each layer.

For a solver-backed preservation theorem, symbolically execute the selected body from arbitrary contract-satisfying inputs and ask whether any promised exit component can differ. If the solver finds a counterexample, replay it through the executable semantics. For an unsatisfiable result, retain the exact formula, semantic digests, and the strongest available independent evidence. Keep the theorem limited to the body, ABI slice, memory region, and normal-return path actually encoded.

The existing LLVM contract route supplies a useful control before any ISA adapter exists. Verify a memory-free scalar leaf both by replacing its call with its body and by applying its modular contract. Then mutate its requirement, result, or defined-operation clause. The modular route must either reject the false declaration or produce a source-attributed counterexample. This proves that contract composition machinery is real. It does not prove a saved-register, frame, or return-address claim until those state components enter the semantics and observation.

10.14 Where the model stops

The executable proof core in this chapter covers ordinary scalar calls and normal returns. Selected sections explain aggregate and variadic classification, hidden result pointers, dynamic linkage, unwinding, FFI, stack exhaustion, guard pages, and suspended activations so their obligations are visible. The shared add-one theorem and proposed first ISA adapters do not claim executable semantics for those extensions.

Full models for floating-point and vector classes, thread-local state, position-independent linkage, system calls, signals, interrupts, language exceptions, coroutines, privilege changes, and control-flow enforcement remain outside the chapter. It also excludes asynchronous stack modification, concurrency, timing, and speculative leakage. A valid architectural frame can still expose secrets through access

patterns under a stronger observation. Those claims require another state and observation model.

The narrow result remains powerful. A procedure boundary lets code written at different times, in different languages, and by different people compose through a finite agreement. The machine sees words, addresses, and control transfers. The ABI turns those objects into a promise one component can keep for another.

10.15 Exercises

1. Separate ISA facts, assembler spellings, and ABI rules in the RV64I `add_one` listing.
2. Do the same for the x86-64 listing. Include RDI, RAX, RSP, the stacked continuation, and RET.
3. **Execute.** Let an x86-64 call begin with `RSP=0x1000` and return address `0x402006`. Trace the stack-pointer and return-address memory effects of the selected near CALL and matching RET.
4. Explain why core Ao cannot implement a reusable return through r_7 . State the minimum new semantic capability an extension would need.
5. **Prove.** For nonwrapping address arithmetic, prove that allocating and releasing an F -byte frame restores the entry stack pointer. Then give three frame errors that this equality does not exclude.
6. For a 32-byte downward-growing frame at entry pointer 256, list its byte interval. Decide whether 8-byte stores at 224, 248, and 252 stay inside it.
7. Give frame sizes that preserve 16-byte alignment and sizes that break it. State the checkpoint at which your claim applies.
8. Mark each selected RISC-V ABI register as argument/result, temporary, preserved, link, or stack pointer: `a0`, `a7`, `t2`, `s3`, `ra`, and `sp`.
9. A live value occupies `t0` across a nested RV64 call. Give two correct preservation strategies and identify who owes each save and restore.
10. **Break.** Remove the save of incoming `ra` from a non-leaf RV64 function. Construct the shortest trace that returns to the wrong address.
11. Give a System V AMD64 body that returns the right value but violates one callee-saved-register obligation. Append a caller that observes the damage.
12. Explain why a leaf x86-64 function containing only `lea` and `ret` still has a memory-reading return step.

13. **Transfer.** Write the complete narrow state relation used by the three add-one examples. Include input, result, continuation, preserved state, stack state, memory, outcome, and permitted scratch differences.
14. Extend that relation to a nested call with one live intermediate. State the premise that no nested write shares an address with the frame slot.
15. Design the first procedure-contract artifact for Axeyum. Include endpoint states, trace reference, observation sets, owned memory, ABI source pin, and five independently firing negative controls.
16. Distinguish a decoder proof, trace proof, endpoint contract check, and cross-ISA procedure refinement. Explain what evidence each still lacks.
17. Explain why a tail jump is ABI-correct only after the current function's frame and preservation obligations have been discharged.
18. **History.** Compare an EDSAC closed subroutine with a modern RV64 procedure. Identify the common continuation problem and three mechanism or convention details that must not be projected backward.
19. Explain why recursive activation requires distinct live records even though every activation executes the same procedure bytes. Give one implementation that does not place the complete record on a conventional contiguous stack.
20. Draw the activation tree for a procedure f that calls g , calls g again after it returns, and whose first g calls h . At the deepest point, list the one root-to-current path represented by the active stack.
21. **Prove.** Formalize the induction over call depth for three frame sizes F_0, F_1, F_2 . Include no-wrap, stack-region, alignment, and disjointness premises. Give one counterexample when each premise is removed.
22. A 64-bit stack pointer is 32 and a procedure subtracts 64. Compute the wrapped word and the intended mathematical interval. Explain why adding 64 back can recover 32 without proving a valid frame.
23. Distinguish reserved stack address space, committed or mapped pages, allocated frame bytes, and live object slots. State which layer a guard page and a stack probe protect.
24. **Execute.** Trace every stack pointer and save-slot interval in the worked RV64 non-leaf sequence. Begin with $sp = 256$, $ra = 1000$, and $s0 = 7$. Allow the helper to replace ra and $a0$, then complete return.
25. Decode 13 01 01 FF and 23 34 11 00 under the selected RV64I rules. Show how the register and immediate fields establish the first two prologue steps rather than trusting the assembly listing.

26. **Break.** Swap the two RV64 load offsets in the epilogue. State the final `ra` and `s0` values from unequal saved inputs and identify which preservation and continuation clauses fail.
27. Change the RV64 frame size from 16 to 24 without changing the slots. Show the new intervals and alignment at the helper-call checkpoint. Separate memory safety from ABI alignment.
28. A direct RV64 call is at address 100 and its helper is at 148. Compute the displacement and explain which evidence binds the final `jal` bytes. Give a relocation mutation that keeps the opcode legal but reaches another aligned function.
29. **Execute.** Trace the worked System V AMD64 frame from an entry RSP of `0x1008`. List the current RSP, continuation slot, saved RBX slot, and private interval after each stack-changing instruction.
30. Decode the fixed bytes `53, 48 83 EC 20, 5B, and C3` with a source-pinned decoder. State which architectural and ABI facts remain unproved after successful decode.
31. **Break.** Replace `add rsp, 32` with `add rsp, 24` in the worked System V sequence. Trace `pop` and `ret`; identify the first wrong memory read and the final control consequence.
32. Compare the same four-integer source signature under System V AMD64 and Microsoft x64. Give every incoming register location, then state the caller stack area required by Microsoft x64.
33. Write a bridge that receives two values in System V's first two integer positions and calls a Microsoft x64 procedure. State the required register moves, stack area, alignment, preserved-set obligations, and return-value map.
34. **Break.** Construct an x86-64 function that is legal under one of the two selected ABIs but violates the other solely by changing RDI. Append a caller that exposes the difference.
35. Draw a frame with a 24-byte local array, an eight-byte canary, and a saved continuation. State one overwrite the canary detects and two corruption patterns it does not detect. Name every premise needed for the detection claim.
36. **Prove.** Model an ordinary continuation stack K and protected shadow stack H . Prove by induction over well-nested call and protected return events that $K = H$ is invariant. Then add an ordinary-memory corruption event and show why the next mismatching protected return cannot complete normally.

37. Compare return-edge protection with a landing-pad check on an indirect call. Give one attack excluded by each mechanism and one functional or memory-safety failure excluded by neither.
38. Run the simple allocation classifier on nine RV64 integer arguments. Then replace argument two with a selected two-word aligned value and compute again every later location under the pinned rule.
39. **Prove.** Define the state and transition of the simple register-and-stack argument classifier. Prove by induction that all assigned stack intervals are disjoint and that the next free offset remains aligned.
40. Give two source structures with the same total size but different field classes under a selected ABI. Explain why size alone cannot choose argument registers. Route the exact answer to the official classifier.
41. A large result uses a hidden result pointer. Write the caller and callee memory contracts for a 24-byte result. Include alignment, valid range, ownership, aliasing with inputs, and the final observation.
42. **Break.** Let the hidden result object overlap a read-only input object. Construct an execution in which a result store changes a later input load. State the nonoverlap or sequencing premise needed to recover the theorem.
43. Explain why a variadic callee may need a register-save area even when no named source parameter refers to it. Give one wrong-type traversal that reads the wrong width or class.
44. For the worked System V prologue, write the three described CFA rows: entry, after push `rbx`, and after sub `rsp, 32`. Recover the caller stack pointer and entry `RBX` from a symbolic state at each point.
45. **Negative control.** Keep the unwind row's CFA offset unchanged after stack allocation. Identify the first instruction range in which recovery fails. Then mutate only the saved-`RBX` offset and compare the failure.
46. Give a trace whose one stopped state produces a plausible backtrace under wrong unwind metadata. Explain why that witness does not prove the other instruction ranges.
47. **Tail call.** Transform an ordinary call-and-return sequence into a tail transfer. State the original continuation, outgoing argument map, frame-release proof, preserved-state proof, and the target procedure's entry contract.
48. Construct a cyclic parallel move among three argument registers at a tail-call checkpoint. Show why a sequential copy loses a value and repair it with one temporary or a cycle-aware move algorithm.

49. Compare a direct ABI call, serialized message, and process-local RPC for one high-rate small operation. Name semantic boundaries and at least five cost or deployment measures. Do not infer performance from call count alone.
50. **FFI.** Specify a pointer-and-length boundary between two languages. Include representation, byte-range arithmetic, ownership, mutability, lifetime, error, callback, and unwind behavior.
51. Give a record type whose source field names agree but whose padding, Boolean, enumeration, or invalid-value rules can differ. State the representation promise and negative test needed before sharing it by address.
52. **Axeyum.** Separate what the current verified scalar LLVM direct-call route proves from what an RV64 ABI frame theorem needs. Name the existing contract, inlining, replay, BMC, and k-induction assets, then list the missing memory, continuation, decode, and ABI adapters.
53. Design an Axeyum mutation matrix for the worked RV64 frame. Corrupt the frame size, save offset, restored register, restored continuation, helper contract, final stack pointer, and return target independently. Give the expected failing obligation for each mutation.
54. **Synthesis.** Prove one non-leaf procedure from instruction bytes through normal return. Organize the proof by decode, instruction effects, frame intervals, nested-call checkpoint, unwind agreement, ABI exit relation, and continuation provenance. Mark every excluded exit class.

PART III

Proving Claims About Programs

Equivalence, Observation, and Refinement

Two programs can produce the same answer and still disagree about memory, conditions, traps, running time, or whether they return at all. They can agree for one caller and fail for another. They can implement the same mathematical operation while inhabiting machine states with no matching register names. The word *equivalent* becomes useful only after these choices are fixed.

This chapter turns agreement into a statement with visible parts. A precondition selects inputs. Program semantics produces behavior. An observation selects what the claim exposes. Equivalence compares programs in one state space. Refinement relates programs whose states or permitted behaviors differ. A counterexample supplies an allowed input and an inspectable execution that breaks one required clause.

Equivalence is always relative to a contract

Name the initial states, termination policy, trap policy, observation, and program semantics. If a surrounding context will use the replacement, name what that context may read. Without those parts, “same program” is an intuition rather than a theorem statement.

11.1 Why program agreement matters

Programmers have replaced instruction sequences for as long as they have optimized code. A hand optimizer shortened an address calculation. A compiler reordered operations or allocated another register. A **linker**, the tool that combines object files, relaxed a long transfer into a shorter one. Each transformation carried an implicit promise: the change would not alter what mattered to the rest of the program.

Modern systems make that promise at larger scale. Optimizing compilers perform thousands of rewrites. Binary translators move code between instruction sets. Cryptographic implementers replace clear scalar formulas with sequences

chosen for speed or side-channel discipline. Superoptimizers search for a least-cost program within a declared language. Formal tools compare implementations with specifications. All depend on a defensible meaning of agreement.

The strongest meaning is often unnecessarily strict. A scratch register may differ after a closed function. Temporary condition bits may be dead. A source machine may have no counterpart for a target machine's flags. The weakest meaning is unsafe: comparing only the desired numeric result can hide a trap, wrong return address, or caller-owned memory corruption. The task is to choose the smallest observation that still protects every permitted use.

The earliest optimization arguments were often local and operational. Replace one instruction sequence with another, run examples, and inspect the result. That practice could find errors and save scarce time or memory, but it left the word "same" informal. The difficulty was not only the number of inputs. A program could fail to terminate, damage a location outside the intended result, or reach the same final value by a path whose intermediate actions mattered to another observer.

Floyd's 1967 account of assigning meanings to programs attached propositions to points in a flow graph. Each command had to carry a true assertion at its entry to a true assertion at its exit. The framework was meant to support rigorous arguments about correctness, equivalence, and termination (Floyd 1967). Hoare's 1969 axiomatic treatment made the familiar precondition–command–postcondition shape a compositional proof interface (Hoare 1969). Neither contribution reduced program meaning to one output number. Both made the assumptions and promised result part of the mathematical object.

Later operational semantics represented a program as transitions among states. Relations between states then supported simulations: if one program moves, another program can make a corresponding move or sequence of moves and restore the relation. Refinement calculi used an order rather than equality: an implementation may choose one behavior from several permitted by an abstract specification. These developments explain why modern compiler correctness theorems are commonly directional preservation claims rather than claims that source and target executions are literally identical.

Two industrial strategies grew from this foundation. A verified compiler proves semantic preservation for the compiler's admitted source programs and passes. CompCert demonstrated that approach for a realistic compiler, with a machine-checked proof connecting its source and target semantics (Leroy 2009). **Translation validation** instead checks the source and target produced by one compilation. Pnueli, Siegel, and Singerman gave that approach its name and an explicit verification architecture (Pnueli et al. 1998). The two strategies place proof effort in different locations; neither removes the need to define the behaviors being preserved.

Alive2 applies bounded translation validation to LLVM's intermediate representation. Its 2021 report described solver-backed checking integrated with LLVM

practice, stated its bounded and unsupported cases, and reported 47 new defects found during the work (Lopes et al. 2021). That number is a dated result from one study, not a timeless defect rate. Its deeper lesson is that an equivalence checker can affect both implementation and specification: counterexamples exposed compiler errors, while other investigations led to clarifications of the intermediate-language semantics.

The economics follows the proof boundary. Proving a compiler can amortize a large argument over many compilations, but maintaining the proof constrains changes to passes and semantics. Validating each translation follows an evolving compiler without proving its whole implementation, but it adds work to compilation and can time out or reject unsupported cases. Engineering teams pay in proof maintenance, solver time, build latency, triage, and trusted-base review. A sound comparison names which of those costs moves; it does not call formal assurance free.

Optimization is a promise about the future

A replacement is not justified merely because two isolated executions look alike. It is justified when every allowed future use receives the behavior the contract promised. Context gives local equality its program-wide meaning.

11.2 Programs produce behaviors

Let P be a deterministic program and S an allowed initial state. Its execution may reach a normally halted state, reach a trapped state with a reason, or continue without a terminal state. Write

$$Beh(P, S)$$

for that behavior. A finite bounded runner may also stop because its external step budget is exhausted, but bound exhaustion is evidence about a prefix, not a fourth architectural behavior.

For a terminating behavior, let $final(P, S)$ denote the terminal state. A normal-result observation A is applied only where its domain is defined. A total behavior observation can instead map normal return, trap, and divergence to distinct forms while applying A to a normal final state.

Termination-sensitive observed behavior

Fix an observation A . The observed behavior $B_A(P, S)$ reports one of: normal return together with A of the final state; trap together with the declared trap observation; or divergence. Two executions agree only when these outer behavior classes agree and their exposed contents agree.

Divergence is a mathematical property of an unbounded execution. A thousand-step prefix does not establish it. Conversely, a termination argument may use an invariant and decreasing measure without enumerating the entire trace. Chapter 9's counted loop supplied exactly that form of reasoning.

Some claims intentionally use **partial correctness**: if both programs return normally, their results agree, but termination is handled elsewhere. That statement can be valuable, yet it does not authorize replacing a terminating program with a diverging one. The theorem and its use must retain the weaker termination policy in the name or scope.

A result observation must not erase behavior class

If one execution returns $r_0 = 7$ and another traps after previously writing $r_0 = 7$, projecting both states to r_0 hides the difference. Compare normal, trapped, and divergent behavior before comparing a result value.

11.2.1 Several behaviors from one input

The selected Ao, RV64I, and x86-64 slices are deterministic: a legal running state has at most one next architectural state. Larger models may admit several behaviors from one input. A source language may leave evaluation order unspecified. A concurrent machine may allow several event orders. An abstract specification may intentionally permit several implementation choices.

Then $Beh(P, S)$ is a set rather than one item. Agreement can mean set equality, inclusion, or agreement on selected possibilities. A refinement often requires every implementation behavior to be allowed by the specification:

$$Beh(Q, T) \subseteq Beh(P, S)$$

for related inputs. The implementation may resolve choices, but it may not add an outcome forbidden by the specification.

Two common questions differ. A **may** property asks whether at least one allowed execution has an observation. A **must** property asks whether every allowed execution has it. Showing that two programs can both return 7 does not show that they must return 7; one may also trap. A checker must retain the quantifier over behaviors.

This chapter keeps the executable examples deterministic so that one trace can explain a witness. The set view is still useful because it clarifies the direction of refinement and prepares the boundary discussion in Chapter 16. It also prevents an endpoint equation from being generalized beyond the model that made it single-valued.

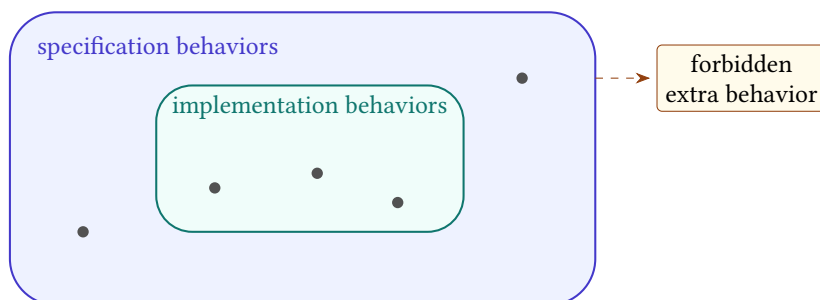


Figure 11.1: A deterministic behavior is one path. A nondeterministic specification admits a set; an implementation refines it when its behaviors remain inside that set.

11.3 Preconditions and the promised domain

A **precondition** $\Phi(S)$ is a predicate on initial state. It can require valid memory ranges, aligned code, buffers that do not overlap, a bounded counter, related ABI state, or a specific operand width. Equivalence under Φ quantifies over every initial state satisfying it, not merely the examples used to discover the claim.

For Ao, compare these one-instruction fragments at a running aligned program counter:

```
P: movi r0, 0
Q: xor r0, r0, r0
```

Listing 11.1: Two candidate ways to clear `r0`

Both leave $r_0 = 0$ and advance by four on a legal fetch. They differ in condition state: `movi` preserves Z, N, C, V , whereas Ao `xor` sets Z, N from its zero result and clears C, V . Under an observation of r_0 , next pc , and running outcome, the fragments agree. Under full state they generally do not.

Add the precondition that entry conditions already equal $(Z, N, C, V) = (1, 0, 0, 0)$. Now the two successors agree on those condition bits as well. With unchanged remaining registers and memory, they reach the same complete successor state. The precondition did not change either instruction; it selected the inputs on which their complete effects coincide.

Removing one premise produces a witness

Remove the entry-condition premise and choose $C = 1$. Program P preserves $C = 1$; program Q clears it. The initial state is a concrete witness against full-state equivalence. Appending `branch.hs` turns the hidden condition difference into different control flow.

Preconditions must be usable by the intended caller. A theorem requiring the two programs' outputs to be equal as an input premise is circular. A theorem requiring a buffer range to be valid may be useful when the caller already establishes that range. Proof systems can validate implications among preconditions, but the book must still explain why the chosen premise matches the application.

Strengthen and weaken a claim

For P and Q , write three contracts: result-only agreement with no condition premise; full-state agreement with the exact condition premise; and a false full-state claim with one premise removed. Give the witness for the false claim before asking a solver to find it.

11.3.1 Sources of preconditions

A precondition may be inherited from a caller, derived from earlier instructions, imposed by an ABI, or chosen to make a local optimization true. Those sources have different proof obligations. An inherited premise must be part of the public interface. A derived premise needs a proof from the prior state. An ABI premise needs a pinned profile. An optimization restriction must not exclude inputs the original program promised to handle.

For a candidate rewrite, one can ask for a **weakest precondition**: the largest set of initial states on which the desired agreement holds. In the clear-register example under full-state observation, the condition tuple must already equal the tuple written by XOR. That exact equality is the weakest condition-state premise when all other legal-fetch premises are fixed.

Weakest does not automatically mean best. A complicated disjunction may be hard for callers to establish or readers to understand. A slightly stronger aligned-range premise may match a public interface better than an exact bit-level formula. The book should state both the sufficient premise used and, when important, whether it is known to be necessary.

A caller must discharge the premise

Suppose a rewrite is valid only when carry is zero. If the preceding instruction clears carry by its declared semantics, sequential composition can establish the premise. If carry is merely dead after the rewrite, that does not show it was zero before the rewrite. Entry and exit facts must not be exchanged.

Solver-assisted precondition discovery can suggest a condition by analyzing counterexamples. The resulting formula remains a hypothesis until checked against the executable semantics and simplified without changing its domain. Discovery and theorem admission are separate routes.

11.4 A hierarchy of agreement

Let Φ be a precondition and A an observation. For programs over the same deterministic state space, define termination-sensitive observational equivalence by

$$P \equiv_{\Phi, A} Q \quad \text{when} \quad \Phi(S) \implies B_A(P, S) = B_A(Q, S)$$

for every initial state S .

The symbol $\equiv$ earns its name only after three elementary laws are checked. A relation E is an **equivalence relation** when it is reflexive, symmetric, and transitive. Reflexive means $E(x, x)$: every program agrees with itself. Symmetric means $E(x, y)$ implies $E(y, x)$: exchanging the two sides changes no claim. Transitive means $E(x, y)$ and $E(y, z)$ imply $E(x, z)$: validated replacements can form a chain.

Observed-behavior equality has these properties when the precondition, semantics, termination policy, and observation remain fixed. Change one of those parts and transitivity no longer follows automatically. A result-only claim from P to Q and a full-state claim from Q to R do not share one relation. Nor may a termination-insensitive first link silently feed a termination-sensitive second link.

Every observation $A : X \rightarrow Y$ induces a relation on states:

$$x \ker(A) x' \quad \text{exactly when} \quad A(x) = A(x').$$

This **kernel relation** groups states that the observation cannot tell apart. Equality of complete state has singleton groups. A result-register observation groups all states with the same selected result, regardless of scratch registers or forgotten conditions. Choosing an observation therefore chooses a partition of the state space into visible classes.

Suppose observation A can be computed from B : there is a function f with $A = f \circ B$. If $B(x) = B(x')$, applying f gives $A(x) = A(x')$. Thus

$$\ker(B) \subseteq \ker(A).$$

The finer observation has the smaller kernel because it identifies fewer states. This reversal is easy to forget. More visible information means fewer pairs counted as equivalent.

Refinement uses weaker algebra. A **preorder** is reflexive and transitive but need not be symmetric. Let $I \preceq S$ mean every behavior of implementation I is permitted by specification S . The program refines itself, and two refinement steps compose, but the specification need not refine the implementation: it may deliberately permit choices that the implementation has resolved. If both $I \preceq S$ and $S \preceq I$, the two are equivalent under that behavior model even when their representations differ.

Behavior inclusion is a refinement preorder

For every behavior set X , $X \subseteq X$, so refinement is reflexive. If $X \subseteq Y$ and $Y \subseteq Z$, then every member of X is a member of Z , so refinement is transitive. Symmetry fails: the singleton set $\{\text{return } 0\}$ is a subset of $\{\text{return } 0, \text{return } 1\}$, while the reverse inclusion is false. The implementation has removed a permitted choice rather than changed an answer into an equal set.

Full-state equivalence is the special case in which the observation retains the complete terminal state and behavior class. Trace equivalence is stronger again if it requires agreement after each corresponding step. Equal final states do not imply equal traces: one program may use a temporary register, take more steps, or visit another control location and later restore agreement.

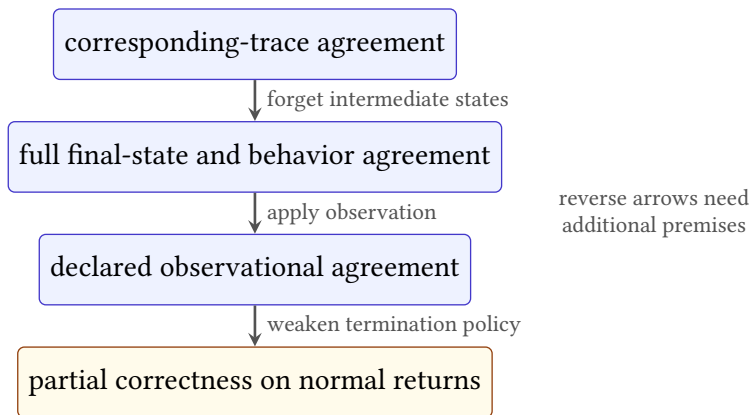


Figure 11.2: Common agreement notions retain different information. Downward implications hold for fixed compatible definitions; upward implications need additional premises and usually fail.

If observation A can be calculated from observation B , then agreement under B implies agreement under A . The converse can fail because A forgets information. This order lets a proof library reuse a strong result for weaker clients without pretending that every client needs full state.

Full-state equivalence implies observational equivalence

Assume both programs have the same behavior class and, on normal return, the same complete final state for every S satisfying Φ . An observation is a function. Applying the same function A to equal final states yields equal

results. Trap and divergence agreement is retained by the outer behavior comparison. Therefore $P \equiv_{\Phi, A} Q$ for every declared A .

The converse fails whenever A is not injective over reachable final states. The clear-register example supplies two unequal condition states that map to the same result observation. No philosophical argument is needed; the two states and projection are enough.

11.4.1 Choosing an observation

An observation is not limited to selecting fields. It can calculate a mathematical reading, normalize several machine outcomes, record an address trace, or combine smaller observations. If A_1 observes the result and A_2 observes memory, their product

$$(A_1 \times A_2)(S) = (A_1(S), A_2(S))$$

retains both. Agreement under the product implies agreement under either projection.

One observation is **finer** than another when the coarser observation can be calculated from it. Full state is finer than a register projection. A complete trace is finer than its final state. Trap class plus reason is finer than trap class alone. This ordering is about retained distinctions, not about which theorem is more valuable.

The coarsest useful observation depends on the client. For a closed arithmetic expression, one result word may suffice. For a procedure, continuation and frame state matter. For constant-time code, an address or branch trace may matter even when results agree. Choosing too fine an observation rejects safe optimizations; choosing too coarse an observation accepts unsafe ones.

Table 11.1: Sample observations and distinctions they retain.

Observation	Retains	Forgets
Result word	one declared output	scratch, memory, conditions
Procedure result	result, outcome, continuation, frame	caller-saved scratch
Complete final state	every terminal component	intermediate movement
Address trace	selected memory locations and order	data values unless added
Complete trace	every selected architectural state	omitted microarchitecture

Observation conversion should be explicit in a proof library. If a theorem uses complete final state and a client needs only a result, apply the result projection as a lemma. Do not copy the theorem and silently give its scope a different name.

11.5 Observing real instructions

The Ao clear-register pair was chosen to expose forgotten condition state. A real-machine comparison should do more than replace the register names. It must account for encodings, instruction lengths, architectural side effects, and the control point at which results are compared.

On RV64I, consider these two one-instruction fragments:

```
P: addi a0, a0, 0
Q: or   a0, a0, x0
```

Listing 11.2: Two RV64I ways to copy a0 to itself

The first adds the sign-extended immediate zero. The second computes bitwise OR with architectural zero. For every input word x , both write x to a0. Neither instruction writes condition flags because base RV64I has no such architectural flag register. Each occupies four bytes and advances to the next aligned instruction under the selected sequential semantics. Their little-endian bytes are 13 05 05 00 and 33 65 05 00.

RV64I full-state agreement for the selected pair

Assume legal aligned fetches of the stated bytes and the same entry state except for program memory containing the selected fragment. Both instructions read entry $a0=x$. Immediate addition produces $x + 0 \bmod 2^{64} = x$; OR with x0 produces x or $0 = x$. Both write only a0, advance by four, preserve memory and all other registers, and remain running. After relating the two code regions and their corresponding next program counters, the complete architectural data states agree.

The phrase “except for program memory” matters. If code bytes are part of the state observation, the two initial states are not equal. A relational claim maps the first code region to the second while requiring equal data state. If the machine keeps immutable program bytes outside data state, the same fact appears in the semantic-package premise instead.

On x86-64, a familiar write-zero comparison has a different answer:

```
P: xor eax, eax      ; 31 C0
Q: mov eax, 0        ; B8 00 00 00 00
```

Listing 11.3: Two x86-64 ways to write zero to RAX

Both selected forms write zero to the 32-bit destination and thereby clear the upper half of RAX. They do not have the same length. The larger difference is that XOR writes selected arithmetic flags, while MOV leaves those flags unchanged. The instruction reference also leaves the auxiliary-carry flag undefined for the selected

logical operation. A claim of equal RAX at corresponding join points can be true. A claim of equal complete state is generally false.

Choose an entry with carry set. After MOV, carry remains set. After XOR, carry is clear. A following carry-conditional branch distinguishes the fragments. Choose instead a context that writes every flag before using it and maps the two different-length exits to one logical join. The result difference disappears under that restricted context. The example joins encoding, semantics, observation, and contextual use in one small case.

Do not compare unequal-length fragments at one numeric exit

Starting both x86-64 fragments at address p gives exits $p + 2$ and $p + 5$. Their next instruction pointers should not be declared equal. Place each in its own code region and relate corresponding join points, or include explicit control transfers and compare after the common join. A result-only proof must not erase control by accident.

The three cases now separate three lessons. Ao makes hidden condition state visible in the book's small machine. RV64I gives a base-ISA pair with no flag side effect and equal instruction lengths. x86-64 shows how partial-register write rules, flag effects, undefined flag state, and variable length alter the contract. "Copy" and "clear" are mathematical intentions; equivalence still belongs to the selected machine semantics.

11.6 Refinement across state spaces

Equivalence starts both programs from one state. **refinement** uses an input relation $R_{in}(S, T)$ between source state S and implementation state T , plus an output relation R_{out} between their behaviors or terminal states. One useful deterministic form is

$$R_{in}(S, T) \implies R_{out}(Beh(P, S), Beh(Q, T)).$$

The relation may map Ao r_0 to RV64I a_0 , ignore an x86-64 scratch register, relate little-endian memory regions at different base addresses, and connect different program-counter locations by control points. It does not make the state spaces identical.

Refinement is directional. An implementation may allow fewer behaviors than an abstract specification while satisfying every abstract requirement. For our deterministic scalar slices, direction often appears only in the chosen input and output relations. It becomes essential when one input can permit several behaviors, a property called **nondeterminism**, underspecification, concurrency, or implementation-defined choices.

A relation carries meaning across machines

An observation projects one state into visible information. A state relation states when two differently shaped states represent the same abstract situation. Cross-ISA refinement needs the relation before execution and after execution; observation alone cannot align the machines.

Chapter 12 develops stepwise simulations. For now, notice that endpoint refinement can be true even when instruction counts differ. Ao may require a comparison followed by a branch, RV64I may compare registers in the branch, and x86-64 may read flags set earlier. The relation needs to hold at selected entry and exit points, not after every unmatched instruction.

11.6.1 Simulation as a step argument

An endpoint test can compare two completed executions without explaining why every admitted execution reaches related endpoints. A **forward simulation** supplies that bridge. Let source states use transition $s \rightarrow_S s'$, target states use $t \rightarrow_T t'$, and let $R(s, t)$ say that the two states represent the same abstract situation.

The simplest rule works in **lockstep**, meaning that one source step matches one target step. It requires

$$R(s, t) \text{ and } s \rightarrow_S s' \implies \exists t'. t \rightarrow_T t' \text{ and } R(s', t').$$

It also requires related initial states and a terminal rule connecting related terminal states to the promised output observation. The step rule alone says nothing about where execution begins or what a final relation means.

Finite lockstep simulation

Assume related initial states $R(s_0, t_0)$. Suppose the source makes n steps. Prove by induction on n that the target can make n matching steps to a related state. At zero steps, choose t_0 . For the induction step, apply the hypothesis to obtain $R(s_n, t_n)$, then apply the simulation rule to the next source step and obtain t_{n+1} with $R(s_{n+1}, t_{n+1})$. If s_n is terminal and the terminal rule makes t_n terminal with related observation, the endpoint refinement follows.

Real translations rarely move in lockstep. One Ao operation may correspond to several RV64 instructions. A target may use a setup instruction that changes a scratch register without advancing the source. Replace one target step by a finite sequence $t \rightarrow_T^* t'$. This is often called **stuttering**: one side takes internal steps while the other remains at the same logical point.

Allowing zero target steps creates a termination danger. A target could match every source step by doing nothing forever while the source progresses. A sound

proof adds a progress measure, well-founded order, or rule that prevents an infinite sequence of zero-progress matches. For finite acyclic fragments, one can instead give an explicit positive bound on the target segment for each source edge.

One source move, two target moves

Suppose a source instruction computes $r_0 \leftarrow r_1 + r_2$ and sets a zero condition. A target first adds into a result register and then compares that register with zero. The intermediate target state need not relate to a completed source state under the boundary relation: the numeric result exists, but the target condition representation is not ready. Relate the source state before the instruction to the target state before both instructions, and the source successor to the target state after the compare. The two-instruction segment restores the relation.

Simulation direction matters when behavior is not deterministic. A forward simulation commonly shows that each source move can be matched by a target move. Depending on which system is source and which is implementation, that may prove the wrong inclusion. If a source specification admits several choices and an implementation selects one, the desired conclusion is that every implementation behavior is allowed by the source. State the behavior inclusion first; then choose the simulation orientation that establishes it.

A relation is a **bisimulation** when it supplies matching moves in both directions. It can support a symmetric behavioral equivalence, but it is stronger than many compiler refinements need. Requiring it can reject a correct implementation that resolves an abstract choice or removes an internal step. Stronger proof rules are not automatically better contracts.

A diagram of paired arrows is not yet a simulation proof

The relation must cover every admitted related state, not only one trace. The step rule must cover traps and terminal states or exclude them through proved premises. A stuttering rule needs progress. A nondeterministic claim needs the right quantifier and direction. Missing any one of these can leave an attractive diagram that proves no endpoint theorem.

The relation itself deserves negative controls. Remove the source program counter from R , and two unrelated source blocks may map to one target state. Forget a live memory region, and a store mismatch may disappear. Relate signed and unsigned readings without a range premise, and the next comparison can diverge. A simulation checker should reject at least one mutation aimed at each load-bearing relation component.

11.7 Contexts expose forgotten state

A **context** is surrounding program structure with a hole into which a fragment is placed. Write $C[P]$ for context C containing fragment P . A local result observation does not automatically justify $C[P] \equiv C[Q]$, because the suffix may read state that the local observation forgot.

Return to `movi r0, 0` and `xor r0, r0, r0`. A suffix that immediately writes all conditions before reading them cannot expose their difference. A suffix beginning with `branch.hs` can. Contextual replacement therefore needs either stronger local agreement or a restriction on what the context may observe before forgotten state is overwritten.

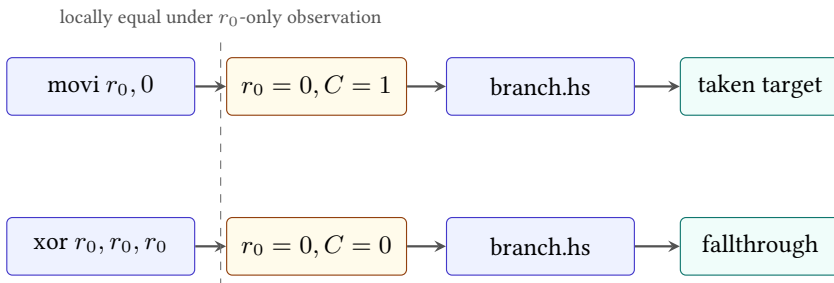


Figure 11.3: A narrow local observation can hide a condition difference. A suffix that reads carry converts it into a distinct control path.

A sufficient contextual-replacement rule

Suppose fragments P and Q terminate from every allowed entry and leave states related by R . Suppose the context's next step preserves R , and every later pair of related steps preserves it until the final observation. By induction over the context steps, the executions remain related. If R -related terminal states have equal final observations and behavior classes, then $C[P]$ and $C[Q]$ agree under that observation.

This rule reveals the work hidden by “the context cannot tell.” We need a relation after the fragment, a preservation argument for the suffix, and an observation compatible with that relation. A read/write or liveness analysis can establish a simpler special case when all differing locations are dead until overwritten.

Dead at one point does not mean absent from the state

A condition bit or scratch register may be irrelevant to the current output and live again in the next instruction. Deadness is a property of a location relative to a program point and future uses. It is not a permanent semantic property of the location.

11.7.1 A footprint rule for replacement

Let D be the set of locations on which fragments P and Q may differ at their common exit. If every path in the suffix writes each location in D before reading it, and the write does not itself depend on another difference, then those exit differences cannot affect the later observation. This is a semantic form of dead-state elimination.

The order matters. A suffix that reads carry and then writes all flags can still distinguish the clear-register fragments. A suffix that first executes an instruction with fully defined condition outputs and only then branches may erase the difference. The proof follows paths, not just unordered read and write sets.

Memory makes the rule relational. A store to $[p, p + 8)$ replaces a differing byte only when the suffix's effective address is known to alias that byte. A load through another symbolic pointer may expose it. Nonaliasing or must-alias facts therefore become premises of the contextual proof.

Overwrite-before-read replacement rule

Assume P and Q terminate with states equal outside D . Along every allowed suffix path, each location in D is overwritten with equal values before any instruction or final observation reads it. All other instructions receive equal inputs and have deterministic equal effects. Induct over suffix steps. After the last location in D is overwritten, the complete states agree; the remaining execution and observation therefore agree.

This rule is sufficient, not necessary. Two contexts may read unequal values and nevertheless calculate equal outputs. A relational invariant can prove that more general case. The overwrite rule is valuable because a compiler's live-variable and effect analyses can often establish it automatically.

11.8 Counterexamples as traces

The negation of a finite-width observational-equivalence claim asks for an allowed initial state whose observed behaviors differ:

$$\Phi(S) \text{ and } B_A(P, S) \neq B_A(Q, S).$$

A solver may return assignments for thousands of internal formula variables. Those assignments are not yet a reader-facing counterexample. Decode the initial architectural state, run both programs through the same executable semantics used by the claim, and print the first meaningful divergence.

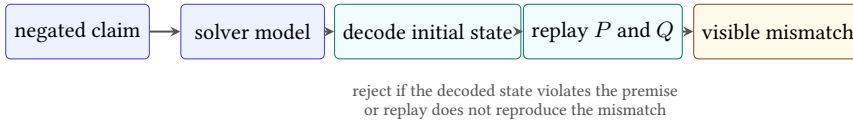


Figure 11.4: A useful solver counterexample returns to the semantic level. Formula assignments become an initial state, two checked traces, and a visible failed observation.

For the clear-register pair with unconstrained carry, a compact report can say: entry $C = 1$; P executes `movi` and preserves $C = 1$; Q executes `xor` and writes $C = 0$; full-state comparison first differs at carry. If a contextual claim is under test, append the branch and show the two successor program counters as well.

The **first divergence** depends on the comparison. Traces of unequal length may have no step-for-step alignment. For identical one-step fragments, the first unequal successor is clear. For optimized routines, compare related control points or report the first violated relation rather than matching step numbers mechanically.

Three counterexample classes

An unequal final register with matching normal returns is a value witness. A normal return on one side and a trap on the other is an outcome witness. A pair with equal final observations but a differing intermediate secret-dependent address is a trace witness under an address-trace observation. The claim decides which class matters.

Counterexample minimization can improve explanation. Prefer small word values, short traces, few changed memory bytes, and the earliest failing component, provided minimization reruns the semantic checker. A smaller printed model that no longer satisfies the formula is not evidence.

Minimization should preserve a named witness predicate. Start from a replayed failure. Try setting irrelevant registers to zero, shrinking the memory region supplied with starting bytes, memory region, simplifying word values, or lowering a loop count. After every change, rerun the precondition, both executions, and the same observation. Keep the change only if the same failure class remains.

The smallest numeric value is not always the clearest witness. For signed overflow, boundary values explain the rule better than a solver's arbitrary large assignment. For aliasing, two short visibly overlapping ranges are better than unrelated high addresses. The minimizer can use a lexicographic cost: behavior-

class mismatch first, earliest divergence second, number of nonzero state fields third, then magnitude.

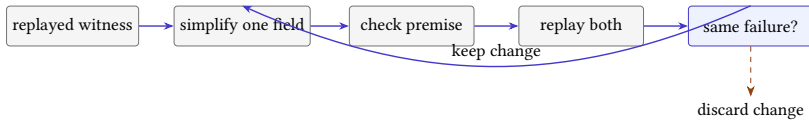


Figure 11.5: Counterexample minimization is a checked loop. A proposed simplification is kept only when semantic replay preserves the named failure.

A minimized trace should retain provenance to the original solver model. The report records which fields changed and which checker reconfirmed the witness. This keeps the explanatory example connected to the machine-derived result without pretending that the minimized state was the solver’s original output.

11.9 Traps and undefined domains

A precondition can exclude a trap only when the theorem says so. For example, a load/store equivalence may require both byte ranges to be valid. Under that premise, no range trap occurs. Without it, the programs may trap differently because they compute different addresses or access different widths.

Do not confuse an undefined mathematical helper with an architectural trap. A partial specification function may be undefined outside its domain. An Ao running step instead produces a declared trapped outcome for an invalid data range or illegal instruction. A semantics can also become accidentally *stuck* if no rule applies and no trap was defined. Total executable semantics should distinguish legal successor, declared trap, terminal input, and malformed external data.

Trap equivalence itself has choices. A narrow claim may require only that both sides trap. A stronger claim may require the same reason and preserved state. An even stronger platform claim may expose fault addresses or exception codes. Print which trap observation is used; “both fail” is not a complete rule.

Choose a trap observation

Construct two Ao loads that both access outside finite memory but begin from different destination-register values. Compare them under three policies: trap class only; trap class and reason; complete trapped state. Identify which differences each policy can expose.

11.9.1 Undefined behavior and asymmetry

Source and intermediate languages can assign no ordinary requirement to some executions. In C, selected operations outside the language’s defined domain have

undefined behavior. LLVM distinguishes several mechanisms, including poison values whose later use can trigger immediate undefined behavior and freeze, which selects a usable value from an otherwise unstable one under the pinned language rules (LLVM Project 2026d). These are language semantics, not architectural trap instructions.

Let D_P be the initial states on which source program P has defined behavior. A compiler-preservation statement can require target agreement only for states in D_P . Outside that set, the source contract supplies no ordinary result to preserve. This direction gives optimizers freedom, but it also makes a symmetric equation misleading: the target may be defined on more inputs than the source without violating source-to-target refinement.

A premise is part of the optimization

Suppose a source operation adds one to a signed value under a rule that excludes signed overflow. Replacing it with a mathematical simplification is justified only when the source's no-overflow premise holds or when the intermediate operation explicitly carries that promise. At the maximum signed input, a target that wraps to the minimum word has produced a hardware value, but the source-to-target theorem has no defined source value to compare there. Removing the premise and reporting equality on all words changes the claim.

Poison also defeats naive testing. A test runner may choose one concrete value and make a transformation appear stable, while the semantics permits other uses or makes a later control operation undefined. A validator must model the language's value domain and propagation rules rather than substitute an arbitrary machine word at the first occurrence. Conversely, an architectural trap must not be labeled undefined merely because the validator lacks a rule for it.

Refinement can be stated through defined-behavior inclusion. For each admitted source input, every behavior of the target must correspond to a behavior the source permits. If the source has one defined result, the target may not add a different normal result or trap. If the source permits a set of results, the target may select a subset. If the source behavior is undefined, this particular preservation obligation is silent; separate security or platform contracts may still restrict the target.

More undefined behavior makes a theorem easier and a language weaker

Declaring a troublesome input undefined removes a proof obligation. It does not improve the program. A semantic design should justify its undefined cases through the language contract, not add them until an optimization

validates. The validator checks a declared language; it must not enlarge the undefined domain to obtain a successful verdict.

A negative control can target this boundary. Remove a no-overflow premise from a valid rewrite and require the checker to return the signed boundary input. Replace `freeze` with an unconstrained fresh value at every use and require an example with two uses to fail. Treat an architectural memory fault as undefined and require a trap-sensitive comparison to reject the weakened model. These mutations test three different semantic layers.

11.10 Composing claims

Equivalence is reflexive, symmetric, and transitive when its precondition, behavior policy, semantics, and observation are fixed appropriately. These properties permit chains of validated rewrites, but changing a scope in the middle can break the chain.

Suppose $P \equiv_{\Phi, A} Q$ and $Q \equiv_{\Phi, A} R$. For every allowed state, equality of observed behavior gives $B_A(P, S) = B_A(Q, S)$ and $B_A(Q, S) = B_A(R, S)$. Transitivity of equality yields $B_A(P, S) = B_A(R, S)$. If the first claim instead uses precondition Φ_1 and the second Φ_2 , the composed claim needs at least their conjunction or another premise implying both.

Sequential composition also needs an intermediate-domain argument. If fragments P_1 and Q_1 leave related states, and P_2 and Q_2 agree only for states satisfying Ψ , then the first pair must establish Ψ for the second. Matching endpoint observations from the first pair may be too weak to establish that premise.

Congruence is equivalence that survives construction

An equivalence relation is a congruence for a program constructor when replacing related parts inside that constructor preserves the relation. Contextual equivalence asks for agreement in every allowed context and is therefore designed for replacement. Directly proving all contexts is often hard; simulations, logical relations, type systems, and effect analyses supply tractable sufficient conditions.

11.10.1 Composition through an intermediate state

Suppose P refines to Q through relations R_{in}^{PQ} and R_{out}^{PQ} , and Q refines to R through corresponding relations for QR . Composition needs an intermediate witness: for every related P - and R -entry pair, there must be a Q entry satisfying both input relations. The two output relations must likewise imply the desired direct PR relation.

Writing this witness explicitly prevents a common gap. One proof may interpret a buffer at base a ; the next may expect it at base b . One may expose normal-versus-trap outcome; the next may forget trap reason. One may require a preserved register that the next treats as scratch. The shared program name Q does not guarantee that its two contracts meet.

Relational composition pattern

Choose intermediate entry T such that $R_{in}^{PQ}(S, T)$ and $R_{in}^{QR}(T, U)$. Apply the first refinement to relate the behavior of P from S to Q from T . Apply the second to relate that same Q behavior to R from U . Finally use a relation- composition lemma showing that the two output relations imply R_{out}^{PR} . Quantify over every direct input pair for which such a compatible T is guaranteed.

In a compiler pipeline, the intermediate witness is often a translated program plus related state. In a cross-ISA proof, it may be an Ao state that both real machines represent. In an optimization chain, it may simply be the common machine state. The pattern is the same; only the relations change.

Optimization pipelines should record the exact contract of each rewrite. Combining destination-only, flag-sensitive, memory-sensitive, and termination-insensitive facts without conversion rules can produce an unsupported final claim even when every individual checker returns success.

11.11 Checking finite equivalence claims

Direct testing runs selected examples. It is fast and excellent for debugging, but its conclusion covers only those examples. Exhaustive enumeration covers every state in a genuinely finite declared domain; it becomes expensive as width, registers, memory, or program length grows.

Symbolic execution represents many inputs with expressions and accumulates path conditions. It can expose how a disagreement depends on an input and can feed a solver. A direct solver encoding instead asks whether the precondition and observation mismatch can both be true. Both routes rely on the sound connection between machine semantics and generated formulas.

A reader proof can establish a parametric law, such as XOR clearing a register at every width, when the definitions admit algebraic reasoning. A proof assistant can check that argument against formal definitions. These methods are complementary: tests diagnose, enumeration calibrates tiny domains, solvers search wide finite spaces, and proofs explain universal structure.

Agreement among routes is valuable but does not merge their scopes. If a direct-enumeration tool and solver agree at width eight, that cross-checks two

Table 11.2: Checking routes support different scopes.

Route	Strongest direct scope	Central risk
Examples	named initial states	untested inputs
Enumeration	complete finite declared domain	omitted states or transitions
Solver query	generated finite formula	encoding and verdict trust
Formal proof	stated theorem over formal definitions	definition and assumption boundary

implementations on one finite domain. A width-parametric proof needs its own argument. Conversely, a proof over an abstract step rule does not validate a decoder until bytes are connected to that rule.

The practical sequence mirrors the book's pedagogy. Start with a hand-chosen witness and its complete trace. Enumerate a miniature width to test the state inventory. Generalize the disagreement formula and ask a solver for a model. Replay every returned model. Only then invest in a certificate or parametric proof whose exact statement has survived those cheaper attacks. Each stage sharpens the claim carried by the next.

11.12 Translation validation

A compiler can validate one local rewrite, one pass, or one complete source–target pair. The larger the boundary, the more interactions it can catch and the harder its formulas become. Local checking obtains small queries and precise blame but depends on every transformation entering the checked route. End-to-end checking sees the actual produced program but must align larger control-flow graphs, memory effects, and language boundaries.

Validation time competes with build time. A developer may accept a solver query that takes seconds in continuous integration but not for every edit. A safety-critical release may pay more to retain checked evidence. A just-in-time compiler has a much smaller latency budget because the application waits for code. These are actor-specific costs: developer feedback, release assurance, or application startup. One timing number cannot represent all three.

A timeout is not a counterexample and not a proof. A production validator needs at least four outcomes: validated, refuted with replayed witness, unsupported, and resource limit. Treating the last two as success creates a silent hole; treating every unsupported construct as a compiler defect creates unusable noise. The policy decides whether code generation stops, falls back to a known route, emits an unvalidated binary with a visible status, or escalates for review.

Counterexample quality changes labor cost. A formula model naming hundreds of temporary variables sends a compiler engineer back through the encoding. A report that identifies the source input, first differing value, relevant instruction

pair, and semantic premise can become a regression test. Replay and minimization are therefore not cosmetic reporting features. They shorten the path from solver result to repaired compiler or corrected specification.

Validation also has a supply-chain boundary. A library producer may validate its object files, but later linking, relocation, binary rewriting, loading, or just-in-time specialization can change the executed bytes. An end-to-end claim must either include those stages or bind the validated semantics to the final artifact through hashes and checked transformations. “The compiler output was validated” is not automatically a theorem about the loaded process image.

Cryptographic code illustrates observation cost. Functional equality may be enough for an ordinary arithmetic optimization. A constant-time contract can also compare branch decisions or memory-address traces. The richer observation creates more solver state and may reject a faster sequence whose addresses depend on secret data. That rejection is valuable only when the security contract actually includes the selected leakage model; Chapter 16 states the larger timing and processor-implementation boundary.

Table 11.3: Validation placement moves both assurance and engineering cost.

Placement	Main benefit	Main boundary or cost
Rewrite rule	small query and precise blame	unchecked ways to bypass the rule
Compiler pass	sees interactions within one pass	needs an input/output semantics for the pass
Whole compilation	checks the actual produced pair	graph alignment, memory, and solver scale
Post-link binary	closer to executed bytes	source meaning and relocation provenance are harder to recover
Runtime or JIT	checks specialized code and assumptions	validation latency is paid during execution

The economic conclusion is not that one placement wins. It is that assurance and cost should meet at a named boundary. Teams can validate high-risk passes, sample broader pipelines, require complete validation for release profiles, or use a verified compiler for a constrained source language. The evidence must say which strategy was used and what happened when checking did not finish.

11.13 Axeyum route

The audited Axeyum checkout at revision a9991fda already contains a real scalar translation-validation route. Its tests reflect selected Rust MIR and LLVM IR into one term arena over shared symbolic inputs. They prove equality for admitted examples, check selected LLVM defined-operation conditions, and evaluate concrete

samples through the reflected terms. Cases include signed shifts, branch-to-select conversion, strength reduction, switch lowering, and hypothesis-conditioned agreement.

The negative route is equally important. A committed wrong-transform corpus contains an off-by-one shift, logical shift substituted for arithmetic shift, flipped select arms, a dropped mask, and a signed comparison translated as unsigned. The solver must return a countermodel. The test then evaluates both reflected programs at the reported input and requires different results. A model that does not replay is a test failure, not a refutation.

That route is stronger than the inherited chapter claimed, but narrower than the machine-program model taught here. It handles selected scalar MIR and LLVM IR, not Ao, RV64, or x86-64 instruction bytes. Its current comparison does not offer the chapter's general observation object over architectural registers, conditions, memory, traps, continuations, and traces. It is translation validation within one shared term vocabulary, not yet a cross-ISA simulation.

The first book adapter still needs `a0-state`, `a0-step`, and `a0-run`, with code-byte provenance and negative controls. Represent an Ao query with semantic-package digests, two decoded programs, word width, precondition, behavior policy, observation, and a bound or termination argument. Generate the negation of the exact reader statement. When a solver finds a mismatch, decode a complete Ao initial state and replay both programs. Reject the evidence if replay does not reproduce the mismatch.

For an unsatisfiable result, preserve the formula and solver configuration. The strongest available route should export evidence that an independent checker can verify or reconstruct the theorem in a kernel. The assurance label must still say whether the result covers one width, a finite width set, or a width-parametric statement.

The first A0 equivalence query

Compare `movi r0, 0` with `xor r0, r0, r0`. Query result-only equivalence first; a deliberate destination mutation must produce a replayed witness. Then query full-state equivalence without the condition premise; the tool should find a condition-state witness. Finally add the exact premise and check the stronger claim. The sequence tests both proof and counterexample paths without requiring a long program.

Negative controls must target every layer. Mutate a destination or condition effect in Ao semantics. Omit one observed component. Invert the precondition. Corrupt one decoded model field before replay. Change the formula digest after the solver runs. A route that rejects only false programs but accepts a stale formula or nonreplaying model has not established the advertised claim.

Why replay is load-bearing

The solver establishes a result about the generated formula. Replay checks that a satisfying assignment decodes to an allowed architectural state and that executable semantics produces the reported difference. Without replay, a bug in formula generation or model decoding can manufacture a persuasive but irrelevant assignment. Replay does not validate an unsatisfiable verdict; that needs formula provenance and a certificate or stronger checking route.

The Python or PyO3 surface should make the teaching path small: construct two programs, choose a named observation, state a precondition, request a check, and receive either bounded evidence or a typed counterexample trace. The lower Rust layer owns semantic definitions, digests, solver translation, and replay. The convenient interface must not invent a broader result than the evidence returned by that layer.

11.14 Where the model stops

This chapter uses deterministic sequential architectural behavior. It does not cover concurrency, weak memory, probabilistic behavior, external input during execution, interactive systems, fairness, timing, energy, cache traces, speculation, or quantitative leakage. Each can require sets or distributions of behaviors and richer observations.

It gives the general forward-simulation rule and a small segmented example but does not yet prove a complete cross-ISA program. Chapter 12 supplies sustained control-point relations and machine-to-machine simulations. Chapter 13 adds program languages and costs; agreement alone says nothing about minimality or speed.

Within the boundary, a precise equivalence statement turns optimization from hope into a claim that a concrete witness can refute. The counterexample is not an embarrassment. It is the smallest state in which our promised sameness breaks, and therefore one of the clearest explanations the machine can give us. Preserving that explanation requires the report to retain the starting state, both outcomes, the selected observation, and the first component that violates the relation. A bare Boolean loses the lesson at the moment it finds the bug.

11.15 Exercises

1. Give two Ao final states related by an r_0 -only observation but separated by a full-register observation, a condition observation, and an outcome observation.

2. **Execute.** Trace `movi r0, 0` and `xor r0, r0, r0` from entry conditions $(0, 1, 1, 1)$. List the complete successors and their first differing components.
3. State the weakest observation in which the two clear-register fragments agree without a condition precondition. Explain one context that makes that observation insufficient for replacement.
4. **Prove.** Prove their full-state equivalence under the exact entry- condition premise printed in this chapter. Include unchanged registers, memory, program counter, and outcome.
5. **Break.** Remove each condition conjunct from that premise in turn. Give a concrete entry state when removal makes the full-state claim false, or explain why a conjunct was redundant.
6. Give a pair of programs with equal final states but different trace lengths. State a final-state observation that equates them and a trace observation that separates them.
7. Distinguish normal return, trap, divergence, and bound exhaustion. Which are architectural behaviors in the model, and which is a runner result?
8. Write a partial-correctness statement that permits termination mismatch. Explain why it cannot alone authorize replacement in a terminating context.
9. Let observation A be computable from B . Prove that agreement under B implies agreement under A . Give a noninjective counterexample to the converse.
10. Define input and output relations between the Ao, RV64I, and x86-64 add-one procedures from Chapter 10. Do not require identical register names or program-counter values.
11. **Transfer.** State endpoint refinement for those three procedures, including behavior class, result, continuation, stack restoration, preserved state, and memory.
12. Build a suffix that observes the carry difference between the two Ao clear-register fragments. Trace the first control-flow divergence.
13. Prove the sufficient contextual-replacement rule for a suffix of n steps by induction on n .
14. Give two trapping Ao executions that agree under trap-class observation but differ under a complete trapped-state observation.
15. Show how preconditions compose when $P \equiv_{\Phi_1, A} Q$ and $Q \equiv_{\Phi_2, A} R$. State a sufficient premise for relating P and R .

16. Design a counterexample report containing initial state, semantic digests, both traces, first violated relation, and final observations. Explain which fields a reader needs and which solver-internal fields may remain archived.
17. Design the first Axiom Ao equivalence manifest. Include the exact query, width, precondition, observation, behavior policy, expected result, replay command, formula digest, and five negative controls.
18. A solver returns a model that fails executable replay. Give at least three possible fault classes and explain why the original equivalence verdict cannot use that model as evidence.
19. **History.** Explain what changes when an informal instruction replacement is restated with Floyd-style assertions at control points. Which part becomes a precondition, which part becomes a preservation obligation, and which behaviors still need an explicit policy?
20. Compare whole-compiler verification with translation validation. Name the object proved or checked, how often the work is performed, one maintenance cost, and one failure mode for each route.
21. Alive2's published study reported a dated number of discovered defects. Explain why that number supports the usefulness of the route but does not measure a timeless LLVM defect rate or prove that every LLVM pass is covered.
22. **Prove.** Show directly that equality of observed behaviors is an equivalence relation when semantics, precondition, termination policy, and observation are fixed. Give a failed transitivity chain when the middle claims use incompatible observations.
23. Let $A(r, m, c) = r$ and $B(r, m, c) = (r, c)$. List the equivalence class of one selected three-component state under each kernel. Prove $\ker(B) \subseteq \ker(A)$ and refute the reverse inclusion.
24. Construct the product observation of result register, outcome class, and selected memory range. State the three projections and prove that product agreement implies agreement under each one.
25. Give two incomparable observations: neither can be calculated from the other. Use concrete states to show both failures, then give their product as a common finer observation.
26. **Prove.** Prove that subset inclusion on behavior sets is a preorder. Give two distinct behavior sets that refine each other only if they are equal, and one strict refinement that is not symmetric.

27. A specification permits results 0 or 1; an implementation always returns 0. State the desired refinement direction. Then reverse the relation and show the concrete behavior that breaks it.
28. **Execute.** Decode and execute RV64I bytes 13 05 05 00 and 33 65 05 00 from three different values of a0. List all architectural locations each form reads and writes.
29. **Prove.** Prove the RV64I self-copy equivalence for every 64-bit input. Include related code regions, corresponding program counters, program memory policy, unchanged registers, memory, and outcome.
30. **Break.** Change the second RV64 instruction's rs2 from x0 to a1. Ask for the smallest replayed input that separates the result observations, and explain why a1=0 is not a universal repair.
31. **Execute.** Begin the x86-64 write-zero pair with RAX set to 0xffffffffffffffff and carry set. Trace RAX, the selected arithmetic flags, and the next instruction pointer after each fragment.
32. Write the narrowest useful relation between the unequal-length x86-64 fragments' exit points. Then construct one context that respects the relation and one carry-reading context that distinguishes the fragments.
33. **Break.** Claim complete-state equivalence for the x86-64 write-zero pair. Give independent witnesses based on carry, instruction pointer, and the selected undefined auxiliary-carry behavior. State which are deterministic witnesses under the pinned semantics.
34. Define a lockstep forward simulation for a two-instruction straight-line program. List the initial, step, terminal, and observation obligations rather than drawing only paired arrows.
35. **Prove.** Prove the finite lockstep-simulation theorem by induction on source trace length. State exactly where determinism is used, if at all.
36. Give a segmented simulation in which one source instruction corresponds to two target instructions. Show why the intermediate target state need not satisfy the boundary relation.
37. **Break.** Permit every source step to match zero target steps with no progress measure. Construct an infinite source execution that the stationary target appears to simulate. Add a well-founded condition that rejects it.
38. Compare forward simulation, backward simulation, and bisimulation for a specification that permits two results and an implementation that chooses one. Which direction establishes the desired behavior inclusion?

39. Mutate a state relation by removing the program counter. Construct two unrelated basic blocks that now appear related and show the first invalid step match admitted by the weakened relation.
40. Mutate a relation by forgetting one writable memory byte. Construct a store pair that passes the weakened endpoint relation but is exposed by a following load.
41. A source addition is defined only when signed overflow does not occur. State a source-to-target refinement over its defined domain. Give the maximum signed input as a witness against the same claim after its premise is removed.
42. Explain why replacing poison with one arbitrary test value is not a sound model of every use. Design a two-use example that distinguishes a stable frozen value from independently chosen values.
43. Distinguish an architectural trap, a partial mathematical function outside its domain, LLVM poison, and an accidentally stuck semantic state. For each, state what an equivalence checker should report or compare.
44. **Break.** Modify a validator so that every unsupported instruction is treated as undefined source behavior. Explain why more claims now pass and why none of those new successes is trustworthy.
45. Design a four-outcome production policy for validated, refuted, unsupported, and resource-limited translations. State what a developer build, release build, and just-in-time compiler do for each outcome.
46. Compare rewrite-level, pass-level, whole-compilation, post-link, and JIT validation. Choose one boundary for a safety-critical release and justify the extra semantics, latency, and artifact provenance it requires.
47. A validated object is changed by relocation and a binary rewriter. State the hashes, semantic relations, and replay steps needed before making a claim about the loaded bytes rather than the compiler's earlier output.
48. Extend functional equality with an address-trace observation for a table lookup. Construct two functions with equal results but secret-dependent addresses on only one side. Route timing and cache conclusions to the correct model boundary.
49. Audit Axeyum's current MIR/LLVM equivalence tests. Classify the checked defined-operation result, solver equality result, concrete sample evaluation, and replayed wrong-transform model by evidence role and scope.
50. Design a negative control for each missing Ao adapter layer: decoder, step semantics, observation, formula binding, model decoding, and replay. Each mutation must make the route fail for its own reason.

Relating Different Instruction Sets

An Ao state cannot equal an RV64 state or an x86-64 state. Their register files have different names and sizes. Their program counters visit different addresses. Their instructions occupy different byte sequences. Ao and x86-64 expose condition state that base RV64I does not possess. Literal state equality would reject every useful comparison before execution began.

Cross-ISA reasoning replaces equality with a relation. The relation says which machine objects represent the same input, where corresponding results must appear, which memory regions match, how control locations line up, and which state may differ. A simulation then shows that execution preserves this shared meaning even when the machines take different numbers and shapes of steps.

Relate meaning, not register names

A cross-machine theorem needs an input relation, control-point relation, memory relation, outcome relation, and output relation. It may deliberately ignore scratch registers or condition state, but only after showing that the rest of the allowed computation cannot expose those differences.

12.1 Why translation needs a relation

Programs have crossed machine boundaries since assemblers, compilers, and emulators first separated a programmer's operation from one machine's code. Retargeting a compiler asks whether different instructions implement the same source operation. Emulation asks whether one machine can reproduce another's steps. Binary translation asks whether already encoded code can move between architectures. Hardware verification often compares an implementation with a simpler reference machine.

Today the same question appears in verified compilers, portable cryptography, instruction lifting, migration tools, and **superoptimization**, the search for an improved instruction sequence within a declared language and cost model. A translator can produce the expected result on test inputs while mishandling a fault,

high address, condition bit, calling convention, or loop exit. A destination-only comparison is a useful test; it is not a cross-machine correctness theorem.

The abstraction level also matters. A source-language integer may correspond to a word in each machine. A source function may correspond to entire machine routines. One abstract step may require several target instructions, while one target instruction may combine several abstract operations. The proof should choose a level at which the relation is both true and useful.

The problem began before instruction sets acquired their modern name. A program prepared for one early computer could not simply be copied to another whose orders, word size, memory, and input devices differed. Porting meant recovering the intended operation and expressing it again through another machine's resources. Assemblers made symbolic spelling less dependent on numeric opcodes. Compilers went further by placing a source language and one or more intermediate forms above the target machine.

Retargeting separated two questions that handwritten ports had mixed. First, what does the source program mean? Second, does the selected back end preserve that meaning in the target's states and behaviors? An intermediate language can reduce duplicated compiler engineering, but it does not answer the second question automatically. Every lowering pass and machine description still needs a semantic connection to its neighbors.

Emulation approaches the boundary from the other side. Instead of translating a source language, it begins with guest-machine state and instructions. An interpreter can decode one guest instruction, apply its guest effect, and repeat. A dynamic translator groups guest instructions, emits host code, and keeps that code while the relevant assumptions remain valid. The grouping can improve execution speed, but it makes control maps, cached translations, faults, and cache resets part of the correctness problem. A cache reset discards a translated block when an assumption behind it no longer holds.

Bellard's 2005 QEMU paper documented a portable dynamic translator supporting several guest and host architectures, together with full-system and Linux user-mode use (Bellard 2005). The paper is historical evidence for an implemented translation architecture, not a universal proof that every guest behavior matched every host behavior. Its importance here is structural: a practical translator needs decoded guest meaning, a host realization, temporary state, control transfer, and services at the boundary between translated code and the surrounding system.

The QEMU development manual inspected on 2026-08-30 still exposes those boundaries. Its Tiny Code Generator groups guest instructions into translation blocks under recorded virtual-CPU assumptions. Block lookup uses the simulated program counter and state. Writes to translated code can force blocks to be discarded, and exception support maps a host program counter back to the guest instruction (QEMU Project 2026). These mechanisms do not prove that QEMU is correct. They identify the state that a faithful block relation must include: entry assumptions, guest state updates, exits, code identity, and fault locations.

Apple's Rosetta 2 gives the same problem a current platform purpose: retain access to x86-64 applications while hardware and native software move to Apple silicon. Apple's developer documentation, inspected on 2026-08-30, presents the translation environment as the route for executing x86-64 instructions on Apple silicon (Apple Inc. 2026). Apple's security documentation distinguishes just-in-time and ahead-of-time translation. It also connects translation to loading, code-page hashes, code signatures, and trust caches (Apple Inc. 2021). These details show that binary translation is not only an instruction substitution algorithm. It participates in a chain that identifies bytes, authorizes code, constructs translated artifacts, and enters a process.

Compatibility has economic value because installed software does not migrate at the speed of a processor release. Application developers need time to ship native builds. Users may depend on an old application or plug-in. A platform vendor can use translation to reduce the immediate cost of changing architectures. The vendor then assumes another cost: translator development, testing across a wide binary corpus, security integration, storage or cache management, performance diagnosis, and a support-lifetime promise.

The costs occur in different units. Ahead-of-time work can move latency away from application startup but creates translated artifacts that need identity and storage. Just-in-time work sees actual execution and dynamic code but adds runtime compilation and cache management. Native multi-architecture binaries avoid some translation work but increase build, distribution, and testing surface. A sound comparison names the application, workload, cache state, translation mode, and actor paying the cost.

Formal ISA descriptions reduce another source of duplication. The adopted RISC-V Sail model records instruction formats, encoding and decoding, and architectural semantics in a language with executable and proof-assistant routes (RISC-V International 2026b). One maintained semantic source can support testing, emulation, documentation, and proof generation. Generated routes do not eliminate provenance: the theorem still depends on the model revision, enabled extensions, generation path, and connection to the actual bytes.

These systems also show why translation performance cannot be reduced to one percentage. A translation can spend time before first execution, during a first hot path, during cache lookup, or on every translated instruction. It can consume memory for translated blocks and metadata, storage for an ahead-of-time artifact, and engineering labor for compatibility tests. The appropriate denominator may be application startup time, elapsed workload time, translated bytes, resident memory, energy, or support years.

No row gives a timeless verdict that translation is fast or cheap. A current measurement must name the machine, software revision, workload, cache state, baseline, and date. A compatibility benefit should likewise name what remains usable and for whom. The installed application that buys a user another year may

Table 12.1: Practical translation decisions have different payers and units.

Decision		Actor paying	Useful measurement	Semantic boundary
Interpret steps	guest	emulator operator	time per workload; dispatch work	guest state and external devices
Translate blocks	hot	runtime and user	cold and warm elapsed time; cache memory	block entry, exits, faults, cache reset
Cache translated code		platform vendor	stored bytes; lookup and validation time	code identity, permissions, freshness
Ship native variants		developer and distributor	build/test time; download bytes	each native binary and supported platform
Retain translation support		vendor and user	engineering effort; supported applications and years	documented guest subset and operating services

be more important than steady-state throughput; a cloud operator running millions of translated process-hours may reach the opposite conclusion.

A relation is a small bilingual dictionary

The relation does not translate every fact in either machine. It records the facts needed for one shared computation: this register carries the input, these bytes represent the same array, these locations are corresponding loop heads, and these outcomes mean the same kind of completion.

12.1.1 Three translation questions

Compiler correctness, instruction lifting, and binary translation all cross semantic boundaries, but they begin with different objects. A compiler starts from a source-language program and usually relates whole behaviors through several intermediate languages. A lifter translates decoded machine instructions into an intermediate form for analysis. A binary translator turns one encoded machine program into another executable program.

In compiler verification, the source may leave implementation choices unspecified or declare some executions outside its guarantees. The target has concrete words, registers, and faults. A semantic-preservation theorem must state how source behaviors correspond to target behaviors; CompCert is a major demonstration of this style at realistic scale (Leroy 2009).

A lifter has a more local initial question: does executing the lifted intermediate statement reproduce the decoded instruction’s architectural effect under a state relation? If the lifter drops flags, faults, or address width, its intermediate language may be too weak for later analyses. A useful lifting theorem therefore names both the translated effect and the observation under which omitted state is safe.

A binary translator owns control layout as well as local operations. Expanding one source instruction into several target instructions moves branch targets and may require scratch storage. Calls and returns must respect a target ABI. Code addresses embedded in data or calculated at run time can make relocation observable. Local instruction results intended for reuse in a larger argument, are necessary, but a whole-program control relation is what composes them.

Table 12.2: Related translation tasks begin and end at different semantic levels.

Task	Source and target	Central proof question
Compiler	source language to machine code	are allowed source behaviors preserved?
Lifter	decoded instruction to intermediate statement	does one lifted movement match one machine movement?
Binary translator	encoded machine program to encoded machine program	do local matches compose through control and procedure context?
Emulator	guest states to host execution	does each guest movement have a faithful finite host realization?

The Ao-to-real-ISA direction in this book resembles a small reference-machine refinement. It does not claim that Ao is a source language or that either real machine implements every Ao behavior. The candidate slices and relations are chosen per theorem. Keeping that scope narrow is what makes the proof inspectable.

12.1.2 RISC and CISC as design shapes

RV64I often exposes operations through regular fixed-width instructions and explicit registers. Selected x86-64 instructions may combine a memory access with arithmetic, use a destination as an implicit source, or update a rich status register. These broad RISC and CISC tendencies change useful cut points. They do not determine which proof is easier.

Consider adding a memory word to an **accumulator**, a register that holds the running result. An RV64 implementation may use a load followed by register addition. One selected x86 form may add a memory operand directly. The relation can compare the RV64 intermediate loaded word with the internal value consumed by the x86 instruction, or it can place cut points before and after the whole movement. The latter avoids inventing an architectural x86 state that does not exist between load and add.

Conversely, a single architectural instruction can have many cases. An x86 memory operand may require variable-length decode, address calculation, segment and mode assumptions, range checks, a memory read, arithmetic, status updates, and instruction-length advancement. Two RV64 instructions may give the proof more visible intermediate states even though the program has more steps. Step count and semantic complexity are different measures.

Choose a relation boundary that both machines possess

Relate the states immediately before an RV64 load-plus-add pair and an x86 memory-source add. Execute each complete segment. At the endpoints, require equal accumulator words, corresponding next control points, preserved related memory, and normal outcomes. Do not require a fictional x86 state halfway through its atomic architectural instruction.

The same principle applies in reverse when one architecture exposes an effect through a register and another through flags. Relate the Boolean or arithmetic meaning at a point where both are available. Architecture-family labels help predict where mismatches may appear; only decoded semantics and a stated relation settle the theorem.

This view also explains why the book studies both families from one scalar foundation. Ao supplies a compact semantic vocabulary, not a claim that every real instruction decomposes into one canonical RISC-like sequence. Sometimes one Ao movement matches several real instructions. Sometimes one real instruction matches several Ao movements at once. The relation may choose either grouping when the endpoints, intermediate fault policy, and progress argument are explicit. Universality comes from the relational method, not from making one architecture imitate the surface form of another.

That flexibility is disciplined rather than arbitrary: every chosen grouping must still preserve the declared observation, expose its freedom clauses, and survive a control aimed at the semantic difference it intentionally crosses.

12.2 Cross-machine state relations

Let S_A , S_R , and S_X range over Ao, RV64, and x86-64 states. A relation for one routine can be written as a conjunction of smaller clauses.

1. *Value relation*: selected registers or memory objects carry equal width- w words or related mathematical readings.
2. *Memory relation*: corresponding byte intervals contain equal or translated bytes, with declared base addresses and ownership.
3. *Control relation*: each machine's program counter belongs to a named corresponding control point.
4. *Condition relation*: required predicates agree, even if one machine stores them in flags and another recomputes them.
5. *Outcome relation*: running, normal return, and selected traps correspond under the theorem's policy.

6. *Frame relation*: stack pointers, saved locations where execution continues, arguments, results, and preserved state satisfy the chosen ABI slice when procedures are in scope.
7. *Freedom clause*: scratch registers, dead flags, code addresses, and private bytes not mentioned above may differ.

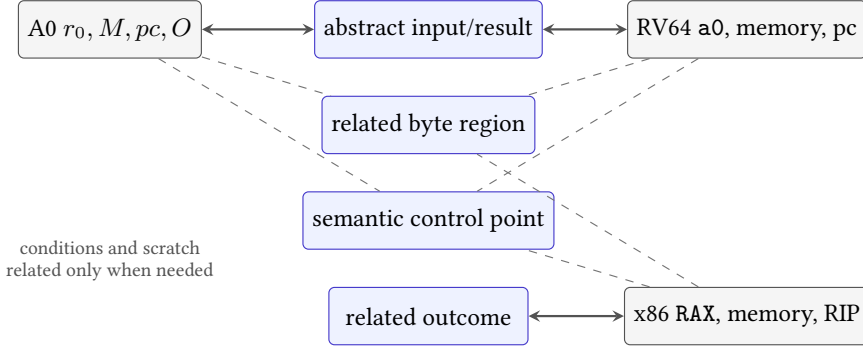


Figure 12.1: A relation aligns selected meanings across three unlike states. Gray components may differ only when the theorem and its contexts leave them free.

The freedom clause is as important as the required clauses. Without it, readers may assume either that every omitted component must match or that every omission is harmless. A typed relation should list ignored and scratch state explicitly and prevent the final observation from reading it.

Mathematically, a cross-machine relation is a subset $R \subseteq \Sigma_A \times \Sigma_T$, where the two state sets need not have the same shape or size. It need not be a function. One abstract state can relate to several target states that differ only in scratch registers or code layout. Nor need every target state represent an admitted abstract state. The entry precondition selects the pairs for which the theorem makes a promise.

A functional relation is sometimes convenient. A representation function $\alpha_T : \Sigma_T \rightarrow \Sigma_M$ can map target states into a common mathematical state, while α_A does the same for Ao. Then

$$R(s_A, s_T) \quad \text{when} \quad \alpha_A(s_A) = \alpha_T(s_T).$$

This construction explains the relation through a shared meaning. It can also be too rigid. A partially initialized target state, a source with unspecified choices, or a memory injection that grows during allocation may require a relation that is not captured by one fixed abstraction function.

The familiar simulation square is a **commuting diagram**. Begin with related states. Move across the top by one source segment and down the side by the relation. The proof must find a target segment along the bottom whose endpoint is related to the source successor. The square “commutes” because either path—source

movement then relation, or relation then target movement—reaches a related pair. This is a statement about paths and relations, not literal equality of endpoints.

Relations can be indexed by control points. Write R_q at entry, decision, update, or exit. A segment from q to q' proves

$$R_q(s, t) \text{ and } s \rightarrow_A^+ s' \implies \exists t'. t \rightarrow_T^* t' \text{ and } R_{q'}(s', t').$$

The plus on the source side requires progress through at least one source step. The star on the target side permits a finite sequence, possibly empty. A rank or structural argument must control empty matches if the theorem covers unbounded execution. For the fixed straight-line routine below, every segment lists its concrete finite instructions instead.

Composition introduces an intermediate relation. If R_{AR} relates Ao to RV64 and R_{RX} relates RV64 to x86-64, their relational composition is

$$(R_{AR}; R_{RX})(a, x) \quad \text{when} \quad \exists r. R_{AR}(a, r) \text{ and } R_{RX}(r, x).$$

The witness r must satisfy both contracts at once. One relation cannot use an RV64 register as protected data while the next declares it arbitrary scratch. This is why pass-by-pass compiler proofs record state mappings and why a shared intermediate language alone does not make a translation chain correct.

Control points name related moments

A control point is a semantic location such as routine entry, branch decision, loop head, memory update, or normal exit. It maps to one or more concrete program-counter values in each implementation. States at matching control points may be related even when states between them have no direct partner.

Numeric program-counter equality is usually irrelevant across ISAs. What matters is that Ao at its *decision* label corresponds to RV64 at its branch and x86-64 after the flags used by its conditional jump have been established. The relation can pair those distinct addresses through one control-point tag.

12.2.1 Observation before relation

A relation can become needlessly strong when its author starts by pairing registers. Begin instead with the client-visible contract. Does the caller observe one result word, a returned memory region, normal versus trapped termination, preserved registers, or the next continuation? The relation then needs enough state to carry those observations through the computation.

Suppose a routine receives two words and returns their sum. A result-only test might compare Ao r_0 , RV64 $a0$, and x86 rax at exit. A procedure theorem also relates entry arguments, stack discipline, continuations, and preserved registers. A

contextual-replacement theorem may need condition state if a surrounding private convention reads it. These are three different claims about the same instruction sequences.

Table 12.3: Observation strength determines relation strength.

Claim	Required relation	May remain free
Result computation	inputs, result, normal out-come	scratch and continuation
Callable routine	arguments, result, frame, continuation	caller-saved scratch
Contextual replace-ment	every state later read by al-lowed contexts	state unreachable to those con-texts

After choosing the observation, write a control-point table. Each row names a logical moment. Each machine column gives the concrete address or decoded instruction associated with that moment. The last column states the relation that must hold there. Gaps in this table often reveal that one machine performs an operation earlier or later than the others.

logical	A0	RV64I	x86-64	relation
entry	control point	control point	control point	clause family
decision	control point	control point	control point	clause family
update	control point	control point	control point	clause family
exit	control point	control point	control point	clause family

Figure 12.2: A control-point table turns three address spaces into one sequence of logical moments. Local proofs connect adjacent rows.

The table is not itself a proof. It is a proof plan. For every adjacent pair, ask which instructions execute, which relation clauses they read, which clauses they restore, and which intermediate states remain intentionally unrelated. This turns a broad cross-ISA claim into finite local obligations.

Make freedom executable

Choose one scratch register omitted at exit. Change its initial value while holding every related component fixed. Then append a context that reads it. Either the context must be excluded, or the relation must be strengthened.

12.3 Memory correspondence

Equal bytes at numerically equal addresses are only one possible memory relation. A translated routine may place the source array at base a and the target array at base b . For a length n , a simple relation is

$$M_A[a + i] = M_T[b + i] \quad \text{for every } 0 \leq i < n.$$

Outside those intervals, the theorem may require equality, preservation, or nothing, depending on ownership and observation.

Loads also need address correspondence. If Ao's effective address is $R_A[r_1] + d_A$ and RV64's is $R_R[x_5] + d_R$, the input relation must imply that both select corresponding offsets within the related regions. Equal loaded words then follow from byte agreement, width, and matching little-endian join rules.

Related bytes give related little-endian words

Assume width $w = 8k$, corresponding bases a, b , and $M_A[a + i] = M_T[b + i]$ for $0 \leq i < k$. Each load joins byte i with weight 2^{8i} . Term-by-term equality gives equal sums modulo 2^w , so the loaded words are equal. The proof fails if widths, byte order, or accessed offsets differ.

Stores require a frame condition. Corresponding stores should write related bytes inside their owned intervals, and every caller-owned observed byte outside those intervals should remain unchanged. Merely comparing the written result misses collateral memory damage.

A memory relation must survive aliasing

Two symbolic base registers may identify overlapping ranges. If a proof treats them as disjoint, **nonaliasing** belongs in the input relation: nonaliasing says the admitted ranges share no byte. If overlap is allowed, the simulation must account for the shared bytes and write order.

12.3.1 Memory injections

Equal-length intervals are enough for one fixed array. Compilers and binary translators also move stack frames, split global objects, allocate helper storage, and choose target addresses that did not exist in the source. A **memory injection** records how each live source block enters target memory. For a source block b , let

$$j(b) = (b', \delta)$$

mean that source offset i corresponds to target offset $i + \delta$ in block b' . The injection is partial: a source block outside the observation may have no target partner. It is

injective over live, observable blocks unless the proof deliberately permits their representations to overlap.

The formula is only the address clause. A usable injection also preserves the facts that make an access legal:

1. if source offset i lies within block b , then $i + \delta$ lies within b' ;
2. readable source bytes correspond to readable target bytes with the same declared value relation;
3. a writable source range maps to a target range the translated store may write;
4. distinct source locations that the source can distinguish do not collapse onto one target byte;
5. target-private blocks, such as a spill area, cannot overlap the image of caller-owned source storage.

These clauses connect set theory to machine safety. The injection maps locations; preservation of bounds, permissions, and separation makes the map useful for execution. A relation that maps values but not access rights can turn a legal source load into a target fault. A relation that maps two independent source blocks onto overlapping target ranges can change the result of a later load after either block is stored.

Two source blocks move without becoming aliases

Let source blocks u and v each contain eight bytes. Map u to target block t_0 at offset 16 and v to t_1 at offset 0. A load from $(u, 3)$ corresponds to $(t_0, 19)$; a load from $(v, 3)$ corresponds to $(t_1, 3)$. The equal numeric offset 3 does not create aliasing because the target blocks differ. If both source blocks instead map into t_0 , their translated intervals must be disjoint.

Allocation makes the relation evolve. Suppose the source allocates a fresh block b and the target allocates b' . Extend j with $b \mapsto (b', \delta)$ while leaving every old mapping unchanged. Freshness and separation show that old pointers keep their meaning. Deallocation can remove a mapping only when no later source observation may use the block. Thus a simulation relation may grow along a trace rather than remain one fixed function from entry to exit.

A translated load preserves its word

Assume $j(b) = (b', \delta)$, source range $[i, i + k)$ is readable, the mapped target range $[i + \delta, i + \delta + k)$ is readable, and corresponding bytes are equal. Bounds preservation makes both loads defined. Offset preservation selects matching byte positions. Equal byte order and width make their joins equal. Remove any one premise and the conclusion can fail through a fault, wrong address, reversed reconstruction, or different value.

Concrete machines present flat architectural addresses, while compiler proofs often use blocks to retain object identity and separation. A cross-ISA route must say where that structure comes from. It may be an admitted region partition supplied by the harness, a loader map, an ABI frame description, or a richer intermediate memory model. Recovering unique source objects from arbitrary machine addresses is not automatic.

12.4 Forward simulation

A **forward simulation** begins with related states. Whenever the source makes a matched step or segment, the target makes zero or more steps to a state related to the source successor. Repeating this argument follows the source execution forward.

Allowing zero target steps is **stuttering**. It handles an abstract state change that the target already represents, bookkeeping steps ignored by the relation, or different placement of control points. To rule out a target that stutters forever while the source progresses, a simulation may require a well-founded rank that decreases on zero-progress matches.

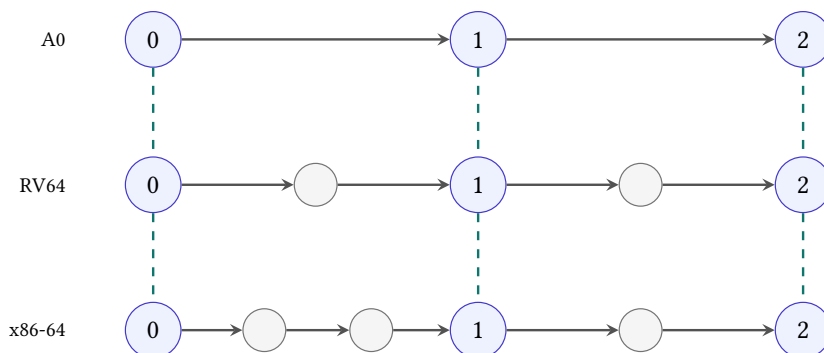


Figure 12.3: Forward simulation relates synchronization points, not every row. One A0 segment can match several RV64 or x86-64 instructions.

For finite straight-line routines, the proof can divide code into segments between control points. For loops, the relation must be invariant at every revisit, and progress or termination needs a separate argument. For branches, predicate agreement ensures that related states select corresponding edges.

From local simulation to an endpoint relation

Assume entry states are related. For each source segment between control points, the target has a finite matching segment whose endpoint restores the relation. Induct over the source segments. The base case is entry. The local simulation supplies the next related pair in the inductive step. At a terminal source point, the outcome clause and output relation therefore hold for the target endpoint. Looping executions additionally require a coinductive or invariant argument rather than finite induction over a fixed segment count.

Forward simulation is directional. It shows that target behavior can be matched by the specification under the selected orientation only if the step clause is stated that way. With nondeterminism, reverse simulation may be needed to exclude extra target behaviors. Our selected scalar examples are deterministic, but the direction should still appear in the theorem.

12.4.1 Transferring safety and termination

A local simulation usually establishes a safety shape: if related execution reaches the next cut point, the new states are related. That conditional does not yet say that the target reaches the point. A target might trap, diverge in an internal loop, or stop because a runner exhausted its bound. Progress adds the fact that a legal matched segment can take a next movement. Termination adds a global argument that repeated movements reach an exit.

For a deterministic finite straight-line segment, progress often follows by executing each decoded instruction and showing that no premise for a trap is violated. For a loop, a rank or source termination theorem is also needed. If one source iteration matches a finite target segment of at least one step, source termination can transfer. If zero-step matches are allowed, the proof must show that they cannot postpone progress forever.

Outcome refinement needs the same separation. One policy may require normal return to match normal return and every selected trap to match its counterpart. Another may prove only that normal source executions have normal target matches. The second theorem is weaker and can still be useful, but it cannot be quoted as universal behavioral equivalence.

Segment simulation obligation

For related source and target states at control point q , a segment obligation names a source path, a finite target path, their required entry premises, the next control point q' , and the relation restored at their endpoints. It also names every failure outcome permitted before q' .

The word “finite” needs evidence. In a hand proof, the target path may be a fixed list of instructions. In a symbolic proof, a rank or bounded internal semantics may establish that the path is finite. A trace bound chosen by the checker is only a search resource until the theorem connects it to program structure.

12.4.2 When reverse simulation is required

Imagine a target program that returns the correct result for every related source run but also has an extra path that leaks into an unrelated successful result. A source-to-target existence statement can ignore the extra path. Behavioral equivalence cannot. A reverse simulation, determinism argument, or set-inclusion proof must rule it out.

Our three scalar machines are deterministic once bytes, state, and fault rules are fixed. This makes a forward simulation stronger: the target does not get to choose another successor. The theorem should still cite determinism rather than inherit it from intuition. Later models with concurrency, interrupts, or underspecified behavior lose that shortcut.

12.5 Absolute value on three machines

Let x be a width- w word and $\langle x \rangle_s$ its signed reading. The mathematical absolute value lies outside the signed width- w range when x is the minimum signed value -2^{w-1} . We therefore choose one of two honest specifications:

1. modular absolute value, which returns x when nonnegative and $-x \bmod 2^w$ otherwise; or
2. mathematical absolute value under precondition $\langle x \rangle_s \neq -2^{w-1}$.

The worked proof uses the second. The precondition is not a decoder fact or an ABI rule. It is the domain on which a positive signed result represents the mathematical absolute value.

Fix the code bases at zero and use the following three implementations. The Ao comparison is written as a subtraction into scratch register r_2 . The subtraction establishes the signed condition while leaving the input in r_0 available for the update path.

```

00: sub      r2, r0, r1
04: branch.ge done
08: sub      r0, r1, r0
done:

```

Listing 12.1: Ao absolute value, with r1 initialized to zero

```

00: bge  a0, x0, done
04: sub  a0, x0, a0
done:

```

Listing 12.2: RV64I absolute value

```

00: mov  rax, rdi
03: test rax, rax
06: jns  done
08: neg  rax
done:

```

Listing 12.3: x86-64 absolute value in the selected local contract

These addresses determine the relative branches. Table 12.4 gives the complete byte strings in increasing address order. Ao uses the Chapter 6 four-byte envelope: selected opcode 11 means register subtraction, selected opcode 30 means conditional branch, and condition code 3 means signed greater-than-or-equal. The branch at address 4 stores displacement 1 because $4 + 4 + 4(1) = 12$. The RV64 words and x86-64 instruction forms follow the pinned architecture references (RISC-V International 2026e; Intel Corporation 2026b).

Table 12.4: Exact code bytes for the three absolute-value routines.

Machine	Address	Bytes in memory order	Decoded instruction
A0	0	11 02 01 00	sub r2,r0,r1
A0	4	30 00 18 01	branch.ge 12
A0	8	11 08 00 00	sub r0,r1,r0
RV64I	0	63 54 05 00	bge a0,x0,8
RV64I	4	33 05 A0 40	sub a0,x0,a0
x86-64	0	48 89 F8	mov rax,rdi
x86-64	3	48 85 C0	test rax,rax
x86-64	6	79 03	jns 11
x86-64	8	48 F7 D8	neg rax

For RV64I, the branch word is 00055463; little-endian memory reverses its byte groups to 63 54 05 00. Its offset is 8, so a taken branch at address 0 reaches address 8. The subtraction word is 40A00533. An assembler may print it as the pseudoinstruction `neg a0, a0`, but the decoder returns `sub a0, x0, a0`. The distinction matters because the proof concerns the decoded base instruction.

For x86-64, the short jump displacement is measured from address 8, the address following the two-byte jns. Adding 3 reaches address 11. The three machines therefore place *done* at addresses 12, 8, and 11. A control relation makes those unequal numbers denote one logical exit.

The bytes above were assembled and disassembled locally with LLVM's machine code tools. That independent calculation is a useful transcription check. It does not turn the worked simulation into machine evidence: Ao lacks an executable decoder here, and the chapter does not yet bind any of the three decoders and step relations into an Axeyum object.

At entry, relate Ao r_0 , RV64 a0, and x86-64 RDI to the same input word x . Require Ao $r_1 = 0$. The x86 destination RAX is initially free because its first instruction copies the input. Relate running outcomes, valid code, and routine-entry control points. Memory is unchanged by all three routines and may be required equal or separately preserved under the local contract.

At the decision point, Ao's scratch subtraction establishes $N = V$ exactly for signed $x \geq 0$ in this comparison. x86 test establishes the sign predicate read by jns. RV64 bge compares a0 directly with x0. Thus all three choose their done edge exactly when the common signed reading is nonnegative.

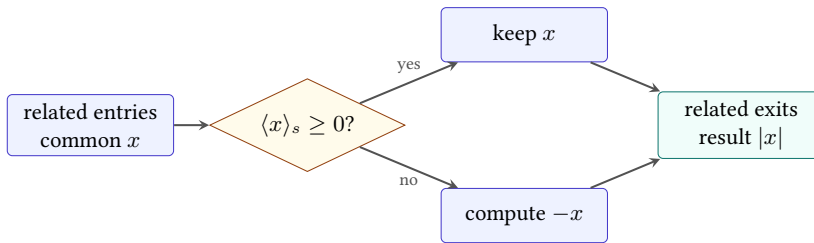


Figure 12.4: The absolute-value simulation splits on one shared signed predicate. Each path restores a common result relation at *done*.

Reader simulation for absolute value

Take arbitrary related entries satisfying the minimum-value exclusion. Split on the common signed reading. If $x \geq 0$, predicate agreement sends all three routines directly to *done*; their result locations contain x , which equals $|x|$. If $x < 0$, all take the negation path. Ao and RV64 subtract x from zero; x86-64 negates its copied word. Fixed-width arithmetic gives the same word $-x \bmod 2^w$. The precondition makes its signed reading positive and equal to $|x|$. At *done*, the result words, outcomes, and preserved memory are related. Scratch condition state may differ.

12.5.1 A synchronization table

The three traces need not have a related state after every instruction. Their relations meet at entry, decision, optional update, and exit. Table 12.5 makes those meetings explicit. A machine may perform no instruction between two rows when its current state already carries the meaning required by the later relation.

Table 12.5: Synchronization points for the absolute-value simulation.

Point	A0 <i>pc</i>	RV64 <i>pc</i>	x86 <i>rip</i>	Required relation
Entry	0	0	0	inputs carry x ; A0 $r_1 = 0$; outcomes run
Decision	4	0	6	result paths carry x ; predicates mean $\langle x \rangle_s \geq 0$
Update	8	4	8	all selected the negative edge; inputs still carry x
Exit	12	8	11	results carry the same modular word; memory and outcomes agree

The entry-to-decision segment takes one Ao step, zero RV64 steps, and two x86 steps. Zero RV64 steps are safe here because its branch reads *a0* directly; the decision relation derives its predicate from the state rather than from a stored flag. A rank for this isolated stutter is the number of setup movements still required before the next shared branch. It begins at two for x86, falls after each setup instruction, and is already zero for RV64. No machine can remain in setup while that natural number decreases forever.

From decision, the nonnegative path executes the Ao branch, the RV64 branch, and the x86 jump. Their endpoints are the three exit addresses. The negative path first reaches the update row, then executes one subtraction or negation per machine. Thus the local segment lengths are unequal, but every segment is finite and restores the next named relation.

Take $x = 3$. Ao computes $3 - 0$ into scratch, all predicates say nonnegative, and the branch reaches 12. RV64 branches from 0 to 8. x86 copies and tests 3, then jumps from 6 to 11. The observed results remain 3. Take $x = -3$, represented by FFFFFFFFFFFFFFFFD at width 64. All three select the update row and produce 3 before reaching their unequal exit addresses. These two traces cover the branch shapes; the case split in the reader proof covers every admitted word.

For the excluded minimum word, each modular negation returns the same bit pattern as the input. The modular-refinement claim still holds. The mathematical-positive-result claim fails because 2^{w-1} has no positive signed width- w representation. This counterexample shows why the input domain belongs beside the relation.

12.5.2 Local branch movements

The reader proof above compresses setup, decision, and update. Expanding those movements shows exactly where architecture-specific facts enter.

At Ao entry, $r_1 = 0$ is a premise supplied by the harness. The scratch subtraction writes r_2 and condition state without changing r_0 . Its signed predicate follows from the Ao subtraction rule. At RV64 entry, x_0 supplies zero and the branch compares register readings directly; there is no setup condition state. At x86 entry, `mov rax, rdi` copies the input, then `test rax, rax` sets the sign and zero information read by `jns`.

The decision relation can therefore be written as

$$D(x, S_A, S_R, S_X) \quad \text{iff} \quad \begin{array}{l} R_A[r_0] = R_R[a_0] = R_X[rax] = x, \\ (N_A = V_A) \iff \langle x \rangle_s \geq 0, \\ SF_X = 0 \iff \langle x \rangle_s \geq 0, \end{array}$$

with the RV64 branch evaluating the last mathematical predicate from its two source registers. The formula does not equate Ao N with x86 SF : Ao's signed comparison uses $N = V$, while x86 `test` clears overflow and records the input high bit in SF . It equates the question they answer.

On the negative path, the update lemma assumes $\langle x \rangle_s < 0$. Ao and RV64 compute $0 - x$ by subtraction. x86 `neg` computes the same modular word. The minimum-value exclusion is not needed for word equality; it is needed only when converting that word to the positive mathematical value $-\langle x \rangle_s$. Keeping those proof steps separate shows that the modular theorem is strictly broader.

Table 12.6: Local obligations in the absolute-value simulation.

Movement	Main fact
Entry setup	all three result paths begin from the common word x
Predicate	each machine selects the nonnegative edge exactly for $\langle x \rangle_s \geq 0$
Keep path	result locations retain x and frames remain intact
Negate path	all result locations receive $-x \bmod 2^w$
Exit	outcome, result observation, continuation policy, and memory agree

Each row admits a focused negative control. Remove the x86 move to break entry setup. Change one signed predicate to unsigned to break the decision. Add an unexpected write to break a frame. Negate the wrong register to break update. Redirect the return to break exit. These controls diagnose more precisely than one end-to-end expected result.

Trace both paths at width four

Use inputs 0011, 1101, and 1000. Record the shared signed predicate, each path, and final word. Identify which input satisfies modular absolute value but violates the mathematical-range specification.

12.6 Relating and forgetting condition state

Ao and x86-64 need condition information at their branches. RV64I does not retain an arithmetic-flags register. The simulation relation should therefore be control-point specific. At the decision point, relate the Boolean question “is the common signed input nonnegative?” to Ao’s $N = V$, RV64’s register comparison, and x86-64’s selected sign condition. At the routine exit, omit conditions when the output contract does not expose them.

Forgetting flags at exit is sound only for contexts that establish their own required condition state before reading it. If the x86 caller branches on flags immediately after the routine under a private convention, those flags belong in the output relation. Ordinary ABI rules and a custom local contract can expose different state.

Abstraction can change across control points

The entry relation may ignore destination scratch. The decision relation may require predicate agreement. The exit relation may forget conditions but require a result and continuation. One monolithic equality is less useful than a family of relations indexed by semantic control point.

Undefined or form-specific flags require special care. A relation must not read an architectural value the selected instruction leaves undefined. It may instead relate only the predicate whose source rules define it, choose another instruction form, or restrict the context so that the undefined component is unobserved.

12.7 Fault and outcome correspondence

The absolute-value routine uses registers only, but instruction fetch, illegal encoding, invalid targets, and terminal input states can still affect execution in a complete model. A cross-ISA theorem normally restricts entry to valid code and running outcomes, then proves corresponding normal exits.

Memory routines need stronger fault agreement. One ISA may permit a misaligned access, whose starting address is not a multiple of the required alignment, access that another traps or decomposes. A theorem can require aligned valid ranges, relate both trap classes, or refine a source operation into a checked target

sequence. Silently discarding inputs on which only one side traps changes the precondition and must be visible.

Shared normal results do not repair unequal faults

If one implementation traps before producing its result and another returns, their normal-result formulas may still be identical. The behavior relation fails unless the theorem explicitly uses a weaker policy that is appropriate for its context.

Likewise, related termination is not established by checking a fixed trace bound. A simulation can transfer termination from a source to a target when each source progress segment has a finite target match and stuttering cannot continue forever. If the target may take an unbounded internal loop between control points, endpoint tests do not exclude divergence.

12.7.1 Translation fault policies

Fault names rarely line up across machines. One source instruction may report an alignment fault, while a target sequence first performs an address calculation and then reports a page or access fault. An operating environment may convert either event into a signal, exception object, process exit, or guest-machine trap. The relation should therefore compare the meaning exposed at the selected boundary rather than equate architecture-specific numeric codes by accident.

There are three common policies. A *fault-preserving* translation relates each admitted source fault to a target event with the same client-visible meaning. A *safe-input* theorem assumes enough alignment, presence, and permission to prove that neither side faults. A *normal-behavior* refinement speaks only about source executions that finish normally. The last policy is weakest. It can establish a useful compiler lemma, but it cannot support a claim that a binary translator faithfully presents all guest exceptions.

Consider an eight-byte load at address $a + 4$. In a model that requires eight-byte alignment, the source traps before reading a byte. A target ISA may permit the misaligned access and return a word. Equal results on aligned tests do not decide this input. The translator can restrict entry to $a \bmod 8 = 0$, synthesize an explicit alignment check and translated trap, or adopt a theorem that excludes the source fault. Each choice changes the contract.

12.7.2 Returns and continuations

A routine that computes the right word can still fail as translated code. The source caller may expect a saved register to survive, a stack pointer to be restored, and execution to resume at one continuation. The target ABI may put arguments in different registers, reserve stack space differently, or use a different return

mechanism. Chapter 10 supplies these contracts; a cross-ISA relation applies them on both sides.

At call entry, relate logical arguments rather than equal register names. Also relate the source and target stack regions through a frame injection, record their saved continuations, and protect each ABI's preserved registers. At return, require the logical result, restored stack state, preserved-register clauses, and related continuation points. Caller-saved scratch may differ because both callers agreed not to rely on it.

A correct value with a wrong continuation

Suppose an x86-64 translation places the expected result in RAX but increments the saved return address by one byte. A result-only observation accepts the endpoint. A procedure relation rejects it because the next fetch is not the target continuation corresponding to the source return. Likewise, a target that leaves its stack pointer eight bytes low has not implemented the source call even if the current result register is correct.

Wrappers often mediate the mismatch. An entry wrapper moves arguments from the host ABI into registers used by translated code. An exit wrapper moves the result back, restores host-preserved state, and enters the host continuation. Those instructions belong to the proof and cost account. Calling them runtime support does not place their effects outside semantics.

12.8 Axeyum route

The current Axeyum checkout already contains a nearby but different form of translation validation. Its test suite can reflect selected Rust mid-level intermediate representation (MIR) and LLVM intermediate representation (LLVM IR) into one term arena, then compare scalar results. Cases include wrapping negation represented as a MIR unary operation on one side and LLVM subtraction from zero on the other. A wrong-transform corpus changes shifts, selections, masks, or signedness and requires returned countermodels to replay against the reflected terms.

That substrate contributes reusable ideas: typed bit-vector terms, solver queries, countermodel replay, negative controls, and the discipline of comparing two independently reflected forms. It does not establish this chapter's claim. MIR and LLVM terms are not decoded Ao, RV64I, or x86-64 states. Scalar result equality does not cover machine-code bytes, program counters, architectural flags, memory injections, faults, or ABI continuations.

The checkout inspected on 2026-08-30 does not contain reusable Ao, RV64I, or x86-64 decoder-and-step packages or typed cross-machine relations. This chapter therefore specifies the required route without presenting the absolute-value simu-

lation as machine-established evidence. The distinction is productive: the existing route shows how formulas and replay should behave, while the missing adapters identify which semantics must be added rather than assumed.

Implementation should proceed in dependency order:

1. complete and test Ao state, decoder, step, and bounded-run packages;
2. implement a source-pinned RV64I slice for the exact branch, subtraction, and jump forms used by the selected routine;
3. implement a source-pinned x86-64 slice for the exact move, test, conditional jump, negation, and return forms used;
4. define typed control-point relations with register, memory, condition, program-counter, outcome, and freedom clauses;
5. check each local simulation segment and compose the endpoint theorem;
6. when a query is **satisfiable**, meaning that some assignment makes its formula true, decode and replay its counterexample through all affected executable semantics.

The relation should be data, not an informal callback hidden in a test. A manifest names semantic-package digests, source revisions, program bytes or decoded programs, control-point map, precondition, relation schema, observation, and expected evidence class. The checker reports which clause first failed.

Negative controls for the absolute-value simulation

Swap signed bge for unsigned bgeu; invert one branch; negate the wrong register; omit the x86 copy; relate RDI rather than RAX at exit; admit the minimum signed word while claiming a positive mathematical result; or let one implementation write an observed memory byte. Each mutation has a small witness and should fail a distinct relation or semantic clause.

At small widths, exhaustive enumeration can check every related entry and both paths. At width 64, a solver can search for a violation of each segment or the composed endpoint relation. A model must become three concrete traces and a failed relation clause. An unsatisfiable formula needs its exact digest and the strongest available certificate or kernel route; a solver message alone does not create a cross-ISA theorem.

Counterexample triage should move from the outside inward. First validate that the model satisfies the serialized precondition. Then construct concrete machine states and check the entry relation. Decode the bound program bytes. Replay each segment. Finally compare the reported failed clause with the concrete endpoint values. A failure at an earlier stage invalidates the later interpretation.

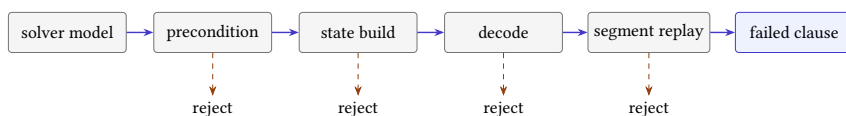


Figure 12.5: A solver assignment becomes a cross-ISA counterexample only after it survives construction, decoding, replay, and relation checking.

This order distinguishes a genuine semantic disagreement from a generator bug. For example, a symbolic model may assign an out-of-range register index that a real decoder rejects. It may describe code bytes different from those in the manifest. It may satisfy a formula whose branch target uses the wrong base. None is evidence that the two actual machine programs disagree; each is evidence that a binding layer needs repair.

Why two matching formulas are not enough

Encoding $y = x$ for nonnegative inputs and $y = -x$ otherwise on both sides checks a shared mathematical formula. It does not establish decoding, instruction effects, branch targets, scratch writes, memory preservation, traps, or control-point progress. The formula becomes useful evidence only after executable source-pinned semantics connect machine bytes to it.

The introductory Python or PyO3 API can expose named routines, widths, relations, observations, and returned counterexample traces. Rust should own the typed semantic objects, digests, solver encoding, and replay. Convenience at the boundary should reduce setup work, not erase which route and scope supported the result.

12.8.1 Expose the relation first

An introductory interface should not begin with a Boolean function named `equivalent`. It should let the reader construct the three entry states, inspect the control-point map, execute one segment, and ask which relation clauses hold. Only then should it offer a composed comparison.

```

case = abs_case(width=8, value=0xfd)
states = case.entry_states()

print(case.relation("entry").explain(states))
segments = case.step_to("decision", states)
print(case.relation("decision").explain(segments.ends))

report = case.compare_to_exit()
```

```
print(report.first_failed_clause())
```

Listing 12.4: Illustrative future relation-first interface

The sketch is not a current Axeyum import. Its design requirement is important: `explain` reports satisfied and failed clauses with concrete values. A negative control that changes RV64 `bge` to `bgeu` should point to predicate or control disagreement, not return an undifferentiated false value.

At the Rust layer, a control point can be an enumeration and each relation a typed structure containing value, memory, control, condition, outcome, frame, and freedom clauses. Segment checkers consume decoded traces and return their endpoint states. Solver formulas are derived from the same structures, and any model is replayed by those executable segment checkers.

Chapter 15 provides the next scale test. Its XOR loop reuses this machinery but adds memory, a loop invariant, and repeated cut points. Building the small absolute-value route first would therefore create reusable semantics rather than a disposable demonstration.

The relation-first interface also changes how a failed comparison is taught. Suppose the terminal values disagree. A flat comparator can report only the two numbers. A structured route asks where the relation last held, which segment broke it, and which clause failed first. If value correspondence fails after the decision point while control correspondence still holds, inspect the selected arithmetic step. If control fails while values remain related, inspect the predicate and target. If the frame clause fails, look for an unexpected scratch or memory write even when the selected result is correct.

That diagnosis is more than a convenience. It mirrors the composition proof. Each segment has premises, a local movement, and an endpoint relation. The first broken endpoint identifies the first local lemma that cannot be reused. A minimized counterexample can then retain only the state components named by that lemma, making the difference small enough to replay by hand.

A counterexample should name its control point

For cross-machine work, an input alone is rarely an adequate counterexample. Retain the three entry states, the last related control point, the first failed relation clause, and the decoded steps that crossed the gap. Those facts turn disagreement into a repairable semantic question.

12.9 Where the model stops

This chapter covers deterministic scalar architectural routines over selected integer instructions. It excludes floating point, vector state, atomics, concurrency, weak memory, privilege, interrupts, exceptions across frames, self-modifying code,

dynamic linking, timing, architecture-hidden state such as cache and predictor contents, speculation, and side-channel observations.

The worked routine is a semantic teaching instance, not evidence that current Axeyum can decode or execute its real machine code. It also does not claim that the three instruction sequences have equal cost. Chapter 13 defines candidate languages and costs; Chapter 15 develops a richer scalar routine through the whole evidence chain.

Within this boundary, a state relation lets unlike machines share one exact story. Their bytes, registers, flags, and step counts remain different. The simulation shows why those differences still converge on the same declared meaning.

12.10 Exercises

1. List the seven clauses of a cross-machine state relation and give one absolute-value fact belonging to each.
2. Relate Ao r_0 , RV64 a_0 , and x86-64 RDI at routine entry. State the width and signed-reading convention.
3. Explain why x86-64 RAX may be unconstrained at entry but must be related at exit in the worked routine.
4. **Execute.** Trace all three absolute-value routines at width four for 0011, 1101, and 1000. Include predicate and result relations at every control point.
5. **Prove.** Prove the nonnegative branch of the reader simulation, including program-counter, result, memory, and outcome clauses.
6. Prove the negative branch under the minimum-value exclusion. Identify the exact step that uses the precondition.
7. **Break.** Replace RV64 bge with $bgeu$. Find the smallest-width witness and replay the path difference.
8. State a modular absolute-value specification that includes the minimum signed word. Explain why its final bit-vector relation holds while a positive signed-result claim fails.
9. Draw control-point matches for one Ao segment corresponding to two or more x86-64 steps. Identify the intermediate x86 state intentionally omitted from the relation.
10. Give a stuttering simulation that would incorrectly permit infinite target stuttering. Add a natural-valued rank that rules it out.

11. **Transfer.** Define a memory relation for equal 16-byte arrays at different base addresses. Include bounds, byte correspondence, ownership, and outside-memory preservation.
12. Prove that corresponding little-endian bytes produce equal width-32 loads. Then break the proof by reversing one join order.
13. Give an aliasing counterexample to a relation that treats two overlapping output ranges as independent. State a sufficient nonaliasing premise.
14. Relate Ao and x86 condition state only at the absolute-value decision point. Explain why the exit relation may omit it under one context and must retain it under another.
15. Construct one pair of implementations with equal normal results but different fault behavior. State three possible outcome relations and which ones accept the pair.
16. **Prove.** Formalize the local-to-endpoint simulation argument by induction over a finite list of control-point segments.
17. Design an Axyum manifest for the three-machine absolute-value routine. Include source pins, semantic digests, decoded programs, control points, precondition, relation clauses, observation, and seven negative controls.
18. A returned solver model violates the relation but fails RV64 executable replay. List possible generator, decoder, and semantic fault classes. Explain why the model cannot support the claimed counterexample.
19. **Decode.** Recover every Ao operand and reserved field from 11 02 01 00. Explain why the destination is scratch rather than the routine result.
20. **Encode.** Derive the Ao branch bytes 30 00 18 01 from its address, destination, condition code, and target formula. Give one mutation that remains a legal branch but reaches the wrong control point.
21. **Decode.** Treat 63 54 05 00 as one little-endian RV64 instruction word. Recover its two register fields, signed condition, and offset. Recompute its taken target from address zero.
22. Explain why an RV64 disassembler may print `neg a0, a0` for 33 05 A0 40 even though the decoder theorem should name `sub a0, x0, a0`.
23. Derive the x86 short-jump destination from bytes 79 03 at address 6. Give the incorrect destination obtained by using the jump's own address as the displacement base.

24. **Audit.** Reassemble the RV64 and x86 listings with an independent tool. Record tool version, target triple, bytes, and disassembly. State which parts of the chapter this checks and which semantic claims remain unchecked.
25. Write all local segment lengths in Table 12.5 for both branch outcomes. Identify the zero-step match and explain why it does not permit infinite stuttering.
26. Replace the setup rank with an ordered pair that counts remaining x86 setup instructions and remaining source control points. Order the pair lexicographically and show where it decreases.
27. **Prove.** Compose an Ao-to-RV64 relation with an RV64-to-x86 relation. Give a concrete scratch-register conflict that prevents one RV64 state from serving as the existential witness.
28. Give a deterministic transition system in which forward simulation is enough to exclude an extra target successor. Then add one nondeterministic target edge and explain which conclusion is lost.
29. **Memory.** Define an injection from two source blocks into one target block. Choose offsets that preserve separation for two eight-byte objects, then choose offsets that violate it.
30. Extend that injection after one source and target allocation. State the freshness fact needed to prove that every old mapping remains valid.
31. **Break.** Keep byte values equal under a memory injection but drop permission preservation. Construct a readable source load whose target match faults.
32. Construct two source pointers that are unequal but become equal under a bad target mapping. Give a later store and load that expose the collapse.
33. **Faults.** For a misaligned eight-byte access, write a fault-preserving contract, a safe-input contract, and a normal-behavior contract. Compare their admitted inputs and conclusions.
34. **ABI.** Relate one two-argument procedure under the RISC-V integer calling convention and one selected x86-64 calling convention. Name argument, result, preserved-register, stack, and continuation clauses.
35. Give a translated routine with the correct result and restored stack but one damaged preserved register. Design a caller that makes the damage observable.
36. **History.** Compare an interpreter with a dynamic binary translator. For each, identify the unit decoded, the stored translation state, the point where a cached block must be discarded, and one proof obligation.

37. **Industry.** Build a measurement plan that separates cold startup, warm translated-code cache, and steady-state execution. Name the workload, hardware, translation mode, baseline, actor, and unit for every reported cost.
38. Compare ahead-of-time translation, just-in-time translation, and a multi-architecture native binary from the perspective of a platform vendor, application developer, and user. Do not combine their costs into one number.
39. **Security.** Explain why code-page hashes, signatures, and a translation cache belong to the provenance relation even when instruction semantics are correct. Give one stale-cache failure.
40. **Transfer.** Choose one QEMU translation block or another documented emulator unit. Sketch its guest entry relation, host-private state, exit map, fault boundary, and cache-reset premise. Mark every fact that still needs source or executable evidence.
41. **Axeyum.** Compare the current MIR-to-LLVM scalar route with the required Ao-to-RV64 route. List the reusable term, solver, replay, and control components, then list the decoder, state, control, memory, fault, and ABI components that cannot be inherited.
42. Design a negative-control matrix with one mutation for code bytes, signed predicate, arithmetic destination, memory frame, fault policy, saved register, continuation, and minimum-value precondition. Name the first relation clause each mutation should break.

Program Languages, Costs, and Minimality

Two programmers replace a six-instruction sequence with four instructions. The replacement passes every test they run. One programmer says that the new sequence is shorter. The other says that it is the shortest possible. The first statement needs a count. The second needs a universe.

That universe is easy to leave implicit. Perhaps an instruction may use any register, or only registers that are dead at the replacement site. Perhaps a constant may appear as an immediate, arrive in a register, or be loaded from memory. Perhaps two instructions with different byte encodings count as the same operation. On x86-64, a sequence with fewer instructions can occupy more bytes. On either real machine, a sequence that looks cheap in the ISA manual can be slow on one processor and fast on another. “No cheaper program exists” has no stable meaning until these choices have been printed beside the claim.

This chapter constructs that missing universe. We will first make a small candidate language, then choose a cost, and finally rule out every cheaper program in it. The worked case is deliberately small enough to check with a pencil. Its value is not the difficulty of adding two. Its value is that no premise can hide in the machinery.

A minimality claim has three named parts

A program is minimal only relative to a candidate language, an equivalence contract, and a cost order. Change any one of the three and the claim changes. The ISA name by itself supplies none of them.

13.1 From sequences to complete searches

Programmers have always looked for shorter or faster ways to express a fixed calculation. A compiler performs the same work mechanically when it replaces an expensive instruction pattern with a cheaper one. The old name *peephole optimization* describes the scale: look through a small window at a few adjacent

instructions, recognize a pattern, and substitute another. The window is small, but the question inside it is already mathematical. Do the old and new programs agree for every allowed input?

In 1987 Henry Massalin described a **superoptimizer**: a system that searches for the smallest program implementing a function rather than relying only on a catalog of known rewrites.(Massalin 1987) Later work made peephole superoptimization more systematic and made complete searches scale through better decomposition and parallelism.(Bansal and Aiken 2006; Lu and Bodik 2025) These systems differ in language, search method, and cost. They share a pressure that matters here: the word *smallest* forces the search space into the statement.

Massalin searched Motorola 68020 instruction sequences in increasing order of length and estimated cost. Candidate programs first ran on selected test values. Only survivors received a more expensive check. The reported programs were commonly four to thirteen instructions long. This arrangement made a large search useful on the hardware of 1987, but the probabilistic filter did not become a proof merely because the search began at length zero.

Bansal and Aiken changed the engineering shape of the task. Their system harvested short target sequences, searched for replacements offline, and stored useful rewrites in a table for later compiler use. Its language made instruction instances from an operation and valid operands. It restricted immediate and symbolic constants so that the set remained finite, and it identified candidates that differed only by permitted register or constant renaming. Those decisions bought search capacity. They also became premises of every smallest-sequence claim produced by that search.

Between those two systems, Denali used goal-directed theorem proving rather than direct sequence enumeration. Its intended users were willing to spend an overnight run on an important inner loop instead of demanding the throughput of an ordinary compiler. Denali represented alternative computations in an equivalence graph. **Satisfiability** asks whether a formula has at least one assignment that makes it true. Denali used a satisfiability solver to ask whether any represented computation fit within a cycle budget. A failed search at $K - 1$ and a witness at K had the upper-bound/lower-bound shape we need. Yet the authors called the result near-optimal: completeness depended on the background axioms, matching time, and heuristics that could stop expansion. They printed the caveat next to the search principle rather than letting the solver inherit coverage by reputation.(Joshi et al. 2002)

Other optimization traditions establish narrow lower bounds in different spaces. Franchetti and Püschel generated selected SIMD permutations through rewrite rules and dynamic programming, with minimum shuffle counts for the covered classes.(Franchetti and Püschel 2008) Equality-saturation systems search an equivalence graph built from declared rewrites. Instruction schedulers search assignments of operations to times and resources. These are neighbors, not synonyms. A minimum in a rewrite graph does not by itself cover every byte string, and a minimum

schedule does not choose every instruction. The history therefore gives us no useful priority slogan. It gives us several ways a finite space can be made exact.

13.1.1 Rewrites, schedules, and programs

Three optimization questions can begin from the same computation and still range over different mathematical sets. A rewrite system begins with an expression and closes it under declared equations. A scheduler begins with operations and dependences, then assigns them to times and processor resources. An instruction search begins with a grammar of candidate programs and asks which of those programs has the required behavior. The sets can overlap without being equal.

Let E_0 be an initial expression and let R be a set of bidirectional rewrite rules. The rewrite closure contains every expression reachable from E_0 by finitely many applications of rules in R . If R includes commutativity and associativity of addition, it can represent many spellings of one sum. If it omits a multiplication-by-constant identity, expressions using that identity remain outside the closure even when they compute the same function. Minimum cost in the closure means minimum among reachable expressions, not minimum among all semantically equivalent machine programs.

An equality graph, often called an e-graph, stores many such related expressions compactly. An equivalence class groups nodes that the rewrite engine has established equal. Saturation repeatedly applies rules until no new represented equality appears or a resource policy stops the process. Extraction then selects a low-cost represented expression. The extraction can be exact while saturation remains incomplete: it can find the cheapest member of the graph without proving that the graph contains every equivalent program.

Denali makes this distinction historically concrete. Its equivalence graph and axioms created candidate computations, while a solver searched for one that met a schedule bound. Matching limits and heuristic stopping could leave useful computations unrepresented. The schedule decision could still be correct for the represented graph. The limitation entered through coverage, not through the arithmetic meaning of a found witness.

Instruction scheduling fixes more structure. Suppose a dependence graph has operations as vertices and required orderings as edges. A schedule assigns each operation a start time and a compatible execution resource. It must respect every dependence and resource-capacity rule. Minimizing the final completion time can prove that no shorter schedule exists for those operations under that processor model. It does not ask whether a different instruction sequence computes the same function with fewer operations.

Register allocation changes the schedule universe again. Two operations that could overlap with unlimited temporary values may need a different order when only a few registers are available. Spilling a value introduces loads and stores, so allocation can change the operation graph that scheduling receives. Unison combines these choices rather than optimizing them in isolation. Its optimality

gap belongs to the combined constraint model and objective, not to all machine programs with the same input-output function.

The distinctions can be summarized by three lower-bound statements:

1. No represented rewrite of E_0 has extraction cost below c .
2. No legal schedule of the fixed operation graph finishes before time c .
3. No program in candidate language L meets the semantic contract with cost below c .

Each statement can be valuable. None should borrow the quantifier of another. An equality proof supports the relation between represented expressions. A schedule proof supports timing inside a declared resource model. A program-minimality proof supports exclusion inside a declared grammar and semantics. Combining them requires explicit bridges: instruction selection from expressions, dependence preservation, register and memory effects, encoding, and the final observation.

The noun after minimum matters

Minimum expression, minimum schedule, minimum encoded sequence, and minimum measured time are different claims. Each names a universe, an equivalence relation, and a cost. A tool may solve one of them exactly without representing the others.

13.1.2 Tests do not prove lower bounds

Massalin's selected test values illustrate a distinction that remains useful. Executing a candidate on a few inputs can reject it quickly. One wrong result is a conclusive counterexample. Agreement on the selected inputs is only a reason to perform the next, more expensive check.

Suppose a finite input domain has N values and a wrong candidate fails on b of them. If each test is sampled independently and uniformly, one test misses the failure with probability $1 - b/N$. A suite of q independent tests misses every bad input with probability

$$(1 - b/N)^q.$$

The calculation explains why random filtering can save work when many inputs separate most bad candidates. It does not establish correctness because b is unknown before verification, the sampling may not be independent, and a candidate may fail on one rare boundary value.

Machine arithmetic supplies exactly such boundaries. Zero, one, the largest unsigned value, the largest positive signed value, and the most negative signed value often expose wraparound or flag errors. Memory candidates may fail only at the last valid starting address. A branch replacement may differ only when a

displacement crosses its signed range. Deliberately chosen edge tests can be more effective than uniform samples, but their success still does not change the universal quantifier.

A safe search pipeline may therefore use tests asymmetrically. Any observed failure permanently rejects the candidate. A survivor advances to exhaustive evaluation, symbolic verification, or another complete route. The lower-bound count includes both classes: candidates rejected by concrete counterexamples and survivors rejected by complete evidence. If a timeout leaves survivors unresolved, the stratum remains open.

Tests also help audit the semantic encoder. Replay a lifted solver model in an independent interpreter on both ordinary and boundary inputs. A mismatch shows that model lifting, bit ordering, decoding, or transition clauses disagree. Agreement raises confidence in the bridge but does not prove its completeness. The proof obligation and the test obligation complement one another; neither should be renamed as the other.

A search can serve at least three different goals.

1. It can find any program that meets a specification.
2. It can improve on a known program under a chosen cost.
3. It can show that no cheaper candidate meets the specification.

The first two goals need a witness. The third needs coverage. If a search finds a three-instruction program, then a three-instruction program exists. It does not follow that two instructions are impossible. To establish that lower bound, every candidate of cost zero, one, and two must be represented and rejected, or a separate argument must rule out the same space.

That asymmetry explains why optimization practice often stops after finding a good sequence. A witness is compact and cheap to replay. A failed search may mean that the tool timed out, its grammar omitted the needed operation, its encoding was wrong, or there truly is no candidate. Only the last reading is a lower bound, and it is the reading that requires the most care.

13.2 Declare the candidate language

A **candidate language** is the exact set of programs allowed to compete with the reference program. For a bounded search, we want this set to be finite and mechanically enumerable. An ISA manual is an input to its design, not the language itself. The manual may describe privileged instructions, optional extensions, hundreds of operand forms, implementation-dependent features, and behaviors outside the replacement site. A theorem about a selected scalar fragment must say selected scalar fragment.

The most useful language declaration answers eight questions.

Candidate-language contract

A bounded candidate-language contract declares:

1. the decoded instruction forms that may occur;
2. the operand forms and widths for each instruction;
3. the registers, constants, and temporary storage available at entry;
4. the memory regions candidates may read or write;
5. the legal starting addresses and decoding policy;
6. the precondition on initial states;
7. the maximum program length or cost bound; and
8. the observation used to compare final behavior.

Together these clauses determine which byte strings or decoded sequences are candidates and what it means for one to work.

The order matters less than the completeness. A register restriction without an operand restriction may still admit an implicit accumulator. A ban on stores does not ban loads. A list of decoded operations does not say whether all their encodings are admitted. A maximum of four instructions does not bound four x86-64 instructions to one fixed byte length. Each clause closes a different door.

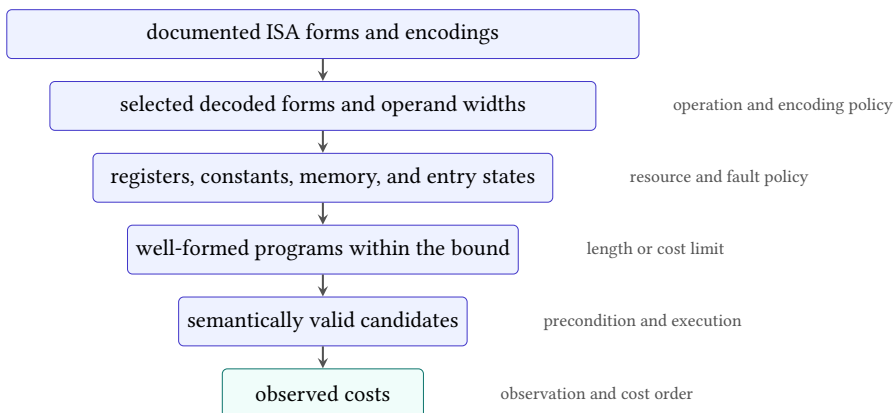


Figure 13.1: A bounded search does not range over an ISA name. Each declared choice narrows the source universe until a finite set of observed candidate behaviors remains.

13.2.1 Decoded forms and byte encodings

There are two sound ways to start. An *extensional* language prints every allowed decoded instruction instance. This is pleasant for a dozen candidates and impossible for a million. An *intensional* language gives a finite grammar: choose an operation from this list, a destination from this set, and each source from its permitted set. The grammar must expand to the same concrete instances the encoder or symbolic translator uses.

Decoded instructions and byte strings are not interchangeable. Ao assigns one four-byte encoding to each legal decoded instance, so the bridge is simple. RV64I uses fixed 32-bit base instructions, but the compressed extension adds 16-bit encodings and its own operand restrictions. x86-64 may encode closely related operations with different prefixes, immediate widths, or addressing forms. A search over meanings can quotient duplicate encodings when cost ignores bytes. A byte-minimal search cannot.

Canonicalization means choosing one standard representative from several candidates treated as equivalent. It can reduce work without weakening a claim, but only when the discarded candidates are genuinely redundant. Suppose addition is commutative and both sources are pure registers. Keeping only `add r0, r0, r1` and dropping `add r0, r1, r0` preserves the possible result functions. If the two forms have different encodings, fault behavior, or costs, that quotient is no longer harmless. The proof obligation for a search-space reduction is the same as for any other rewrite: every removed candidate must have a retained representative with the same observed behavior and no greater cost.

13.2.2 Count the grammar

A finite grammar lets us calculate how many instruction instances it names. Suppose form f has m_f operand positions. If position j admits $n_{f,j}$ choices independently of the others, the product rule gives

$$I_f = \prod_{j=1}^{m_f} n_{f,j}$$

instances of that form. Distinct forms combine by the sum rule, so the whole instruction alphabet has

$$I = \sum_{f \in F} I_f$$

instances. If every position in a program may choose any alphabet member, there are I^k sequences of exact length k . The number of sequences up to length K , including the empty program, is a finite geometric sum. When $I = 1$, the sum is $K + 1$. When $I \neq 1$, the geometric-series identity gives

$$1 + I + I^2 + \cdots + I^K = \frac{I^{K+1} - 1}{I - 1}.$$

Take a small register language with four writable registers. It has a two-operand move, a three-operand add, and an add-immediate whose immediate is chosen from $\{-1, 0, 1, 2\}$. With independent operand choices, move contributes $4 \cdot 4 = 16$ instances, add contributes $4^3 = 64$, and add-immediate contributes $4 \cdot 4 \cdot 4 = 64$. The alphabet therefore contains 144 instances. Lengths zero through three contain $1 + 144 + 144^2 + 144^3 = 3,006,865$ syntactic programs before any semantic quotient. One extra instruction level multiplies the largest stratum by 144. The exponent, not a mysterious solver mood, explains much of the cost.

The product formula applies only when choices are independent. A compressed RV64 form may restrict which registers its fields can name. An x86-64 memory operand may allow an index except for one register code, and the available scratch values at step t may depend on earlier definitions. Then the language is a finite tree rather than a fixed alphabet power. At each node we sum the legal outgoing choices. An **enumerator** is the program that visits every legal candidate in the declared order. It should report both the simple grammar counts and the counts that remain after typing, resource, and encoding checks. A disagreement between those counts is evidence of work to do, not an opportunity to search less.

13.2.3 Types and resource state

A realistic candidate generator does not choose each instruction from one unchanging alphabet. Earlier choices determine which later operands make sense. A register may become available only after it is defined. A condition may be read only after an instruction produces it. A memory load may require a live base address and a proof that the selected width is permitted. The grammar therefore carries a small **resource state** as it descends the candidate tree.

Consider a straight-line word language with two live inputs, r_0 and r_1 , and two initially unavailable scratch registers, t_0 and t_1 . Each scratch register may be defined once. A move chooses one unavailable scratch destination and one live source. An add chooses one unavailable scratch destination and two live sources. After either instruction, its destination becomes live. At the root, the live set has size two and the unavailable set has size two.

The first instruction has

$$2 \cdot 2 + 2 \cdot 2^2 = 12$$

legal instances: four moves and eight adds. After any first instruction, three registers are live and one scratch register remains unavailable. The second instruction again has

$$1 \cdot 3 + 1 \cdot 3^2 = 12$$

legal instances. There are therefore $12 \cdot 12 = 144$ legal two-instruction programs. This total did not come from squaring a fixed alphabet. It came from summing the outgoing choices at one resource state and repeating the count at its successor states.

Table 13.1: The resource state changes the legal choices at each program position. Source repetition is allowed, so ordered source pairs are counted.

Position	Live sources	Fresh destinations	Moves	Adds
1	r_0, r_1	t_0, t_1	$2 \cdot 2 = 4$	$2 \cdot 2^2 = 8$
2	three registers	one register	$1 \cdot 3 = 3$	$1 \cdot 3^2 = 9$

Now change one rule: an add may not use the same source twice. At the first position it has $2 \cdot 2 \cdot 1 = 4$ instances, not eight. At the second it has $1 \cdot 3 \cdot 2 = 6$, not nine. The two-instruction total becomes $(4 + 4)(3 + 6) = 72$. A single distinctness condition halves the language. A checker that enforces the condition while the published count omits it has not proved a bound over the printed language.

Types add another component to the resource state. Suppose a comparison defines a Boolean condition while addition requires word sources. It is not enough to mark a register live. The generator must record whether its current value is a word, a condition, an address, or another admitted type. For each instruction form f , define a transition

$$q \longrightarrow_f q',$$

where q records live typed values, fresh destinations, condition availability, and any other bounded resource. The candidate count at depth k obeys the recurrence

$$N(q, 0) = 1, \quad N(q, k + 1) = \sum_{f \text{ legal in } q} \sum_{i \text{ instances of } f \text{ in } q} N(q_{f,i}, k).$$

The recurrence is a finite dynamic program when the resource-state set, instruction forms, operand sets, and depth are finite. It is also an executable specification for the expected count. An independently written enumerator can be checked against it state by state instead of only comparing one final number.

Memory effects need the same care. One common symbolic technique threads a memory token through every load and store. A load consumes the current token and produces a value while preserving the token. A store consumes the token and produces a successor token. This discipline prevents a candidate from reading a value before the store that defines it or silently reordering two observable stores. The token is not hardware. It is a bookkeeping device for the order already required by the semantics.

The count must distinguish syntactic rejection from semantic failure. A move from an unavailable scratch register is not a candidate in this grammar. A well-typed add that computes the wrong result is a candidate that fails the specification.

Mixing those categories makes diagnostics misleading and can hide an accidental restriction. A useful search report therefore prints, for each depth and resource state, the number of grammar instances, type rejections, resource rejections, decoded candidates, and semantic failures.

Reader proof: the resource recurrence is complete

Use induction on the remaining depth k . At depth zero, the empty suffix is the only suffix, so $N(q, 0) = 1$. For the induction step, every legal suffix that contains an instruction has one unique first form f and one unique concrete instance i legal in q . Executing that grammar transition produces $q_{f,i}$. By the induction hypothesis, $N(q_{f,i}, k)$ counts every legal continuation of length k exactly once. The two sums partition suffixes by their unique first instance. They omit none and duplicate none.

13.3 Symmetry and resources

13.3.1 Symmetry quotients

Two scratch registers may have different names and the same role. Suppose t_0 and t_1 are initially dead, have the same width, may appear in the same operand positions, cost the same to encode, and are absent from the final observation. Swapping their names throughout a candidate produces another legal candidate with the same cost. The pair forms a symmetry orbit: a set of programs related by a permitted renaming.

We can keep one representative from each orbit. Let π swap t_0 and t_1 , both in programs and in machine states. Write $\text{run}(P, s)$ for the result of running program P from state s . The required semantic fact is

$$\text{run}(\pi(P), \pi(s)) = \pi(\text{run}(P, s)).$$

Equivariance means that renaming before execution agrees with renaming after execution. The precondition and observation must also ignore the difference, and cost must satisfy $C(\pi(P)) = C(P)$.

Reader proof: first-use scratch naming is complete

Take any legal candidate that uses the two symmetric scratch registers. If its first scratch use names t_0 , retain it. If the first use names t_1 , swap the two names everywhere. The renamed program is legal, costs the same, and has the same observed behavior by equivariance. Its first scratch use now names t_0 . Thus every removed candidate has a retained representative. Programs using neither scratch register are unchanged.

The proof fails as soon as the names acquire different meanings. An ABI may make one register callee-saved. One name may need a longer x86-64 encoding. The initial state may contain a constant in only one register. A later instruction may have an implicit operand that aliases one name. In any of these cases the swap may change legality, behavior, or cost. Sorting register names is then a bug wearing the clothes of a symmetry reduction.

13.3.2 Counting with group actions

The scratch swap is the smallest example of a **group action**: a set of reversible renamings acts on a set of candidates. Applying the identity renaming changes nothing. Applying two renamings in sequence agrees with their composite renaming. Each candidate belongs to an orbit containing all candidates reachable by the allowed renamings.

If m scratch registers are fully interchangeable, all $m!$ permutations may act on their names. It is tempting to divide the raw candidate count by $m!$. That division is correct only when every orbit has size $m!$. Some candidates may use no scratch register, or may possess another structure that is fixed by a nonidentity renaming. Their orbits are smaller. Blind division can therefore produce a noninteger answer or, worse, an integer that is still wrong.

For two scratch registers, the acting group contains the identity and the swap π . Let X be the finite candidate set. The identity fixes all $|X|$ candidates. Let $\text{Fix}(\pi)$ be the candidates unchanged by the swap. The number of orbits is

$$\frac{|X| + |\text{Fix}(\pi)|}{2}.$$

To see why, separate X into one-element and two-element orbits. A one-element orbit contributes one candidate to both terms in the numerator. A two-element orbit contributes two candidates to the first term and none to the second. In either case, division by two contributes exactly one orbit.

This argument is the two-element case of Burnside's counting lemma. For a finite group G , it counts orbits by averaging the number of candidates fixed by each renaming:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

The notation X/G means the set of orbits, not ordinary numeric division. The formula is useful as an independent count. An enumerator that chooses a standard representative that emits one representative per orbit should produce exactly this many representatives.

Counting orbits does not prove that replacing each orbit by one representative preserves the optimization problem. The group must also preserve language membership, observed behavior, and cost. Burnside's lemma answers how many syntactic orbits exist under a declared action. Equivariance and cost preservation answer whether one member can safely stand for the others.

The first-use naming rule gives a constructive representative. With three fully symmetric fresh registers, name the first one used t_0 , the next new one t_1 , and the third t_2 . This rule resembles the standard naming of bound variables in logic: the spelling changes while the binding structure does not. The representative is useful because it can be generated directly, without first constructing every renamed candidate.

The constructive rule and the orbit count check different failures. If the rule emits too many programs, the representative rule is incomplete. If it emits too few, it has collapsed distinct orbits. If its count is right but legality or cost differs inside an orbit, the reduction is still unsound. A durable search records the group action, representative rule, expected orbit count, and preservation argument separately.

13.3.3 Constants as resources

The expression $x + 1$ looks as though it contains a free constant. Machines do not receive constants from typography. A one may be encoded in an immediate field, held in an entry register, constructed from zero, read from a literal pool, or obtained through a special instruction. Those routes consume different instructions, bytes, registers, and memory permissions.

Consider a language with only register-register addition. If $r_1 = 1$ is an entry premise, then add r_0, r_0, r_1 computes $x + 1$ in one instruction. Remove that premise and the same text reads an arbitrary second input. Add an immediate form and the register is unnecessary. The function has not changed; the candidate language has.

Do not smuggle a constant through the harness

A test harness that initializes a register or memory cell has granted every candidate access to a resource. State that resource in the precondition and language contract. Otherwise the search and the printed claim concern different machines.

Temporaries work the same way. A dead register at a compiler replacement site may be available as scratch. A live register may be used only if the candidate restores it and the observation checks that restoration. Stack space, flags, and vector registers are not free merely because a calling convention makes some of them caller-saved. The local context decides which changes are observable.

13.3.4 Memory policy

A candidate that may use memory needs a region, permissions, alignment rule, aliasing policy, and fault policy. “One temporary stack slot” should name its address relation to the stack pointer, its width, and whether it can overlap any input or

output. If a reference program never faults but a candidate can fault on an allowed state, normal-result agreement does not rescue the candidate.

Memory also threatens finiteness. If an address immediate may be any word, the grammar has 2^w choices at width w . That is finite in mathematics and useless as a naive enumeration. A language may restrict addresses to a declared set of symbolic regions, or it may encode address choices in a solver. Either choice belongs in the scope. “Memory instructions permitted” is not enough information to reproduce the set.

13.4 The equivalence contract

Chapter 11 defined program agreement relative to a precondition and an observation. A minimality search inherits that entire contract. Let S be the specification program or function, P a candidate, $A(s)$ the allowed-input predicate, and O the observation. The candidate works when

$$\forall s, \quad A(s) \implies O(P(s)) = O(S(s)).$$

If execution can trap, halt, or diverge, O must retain whichever behavior classes the replacement context can detect. A result-only projection may be appropriate for a pure straight-line exercise. It is too weak for a compiler rewrite when the continuation can read flags or observe a fault.

This universal condition is where testing stops. Tests sample states. The candidate must work on every state allowed by A . For a tiny width, direct enumeration can check all inputs. At width 64, symbolic bit-vector reasoning can represent all 2^{64} inputs without listing them one by one. The symbolic formula is still accountable to the instruction semantics and the observation that produced it.

The language and equivalence contract interact. If candidates may use a temporary register, then the observation must say whether its final value matters. If candidates may access memory, the precondition must supply valid memory and the observation must compare the permitted effects. If a candidate may be shorter than the reference, the harness must define where execution stops rather than letting it run into whatever bytes happen to follow.

One omitted flag changes the winner

Suppose an Ao sequence clears r_0 with arithmetic and leaves $Z = 1$, while a shorter move also writes zero but leaves different condition bits. Under an observation containing only r_0 , the move may win. Under an observation that also contains (Z, N, C, V) , it does not. Neither answer corrects the other. They answer different replacement questions.

13.5 Cost models

A **cost model** maps a candidate program and its declared context to an ordered value. The simplest model counts instructions:

$$C_{\text{inst}}(P) = |P|.$$

For Ao’s fixed-width encoding, byte length is four times instruction count, so the two costs induce the same order. This convenience is a feature of the teaching machine, not a universal property of instruction sets.

For a variable-length encoding, define

$$C_{\text{bytes}}(P) = \sum_{i=1}^{|P|} b(P_i),$$

where $b(P_i)$ is the encoded length of instruction P_i . A two-instruction x86-64 sequence may occupy fewer bytes than one instruction with a long immediate and addressing fields. An RV64 search that admits the compressed extension can similarly prefer two 16-bit instructions over one 32-bit instruction only if the cost order permits a tie or includes another component; it cannot pretend that instruction count and byte count are the same measurement.

An abstract weighted model assigns a declared weight to each instruction instance or class:

$$C_{\text{weight}}(P) = \sum_i W(P_i).$$

Weights can represent an engineering preference, but their interpretation must be named. A multiply might receive weight four and an add weight one. That does not say the multiply takes four cycles on a real processor. It says the theorem uses the printed table.

Table 13.2: Three cost models can order the same candidates differently. The numbers are illustrative, not measurements of a processor.

Candidate	Instructions	Bytes	Declared weight
P_A : one rich instruction	1	7	4
P_B : two compact instructions	2	4	2
P_C : three simple instructions	3	6	3

The table has three different winners: P_A by instruction count, P_B by bytes and declared weight, and no universal winner independent of the question. Calling one sequence “better” would discard exactly the information a minimality statement needs.

Cost can also depend on the replacement site. A sequence that needs one dead temporary may be admissible where that register is free and inadmissible where it

carries a live value. Saving and restoring the register changes both the candidate and its cost. Alignment can change encoded padding or the placement of an x86-64 instruction across a fetch boundary. A literal already present in nearby memory is a different resource from a literal that the replacement must introduce. For this reason, the most general cost has the shape $C(P, K)$, where K records the declared context. A context-independent model is still useful; it simply fixes or ignores those terms and says so.

13.5.1 Latency and throughput

The ISA defines architectural behavior. It does not fully define how many cycles a concrete processor spends on a sequence. Latency can depend on operand values, cache state, branch prediction, instruction alignment, and the **microarchitecture**, the processor's internal execution design. Processor generation and the instructions around the replacement can also change the cycle count. Throughput asks a different question from dependency-chain latency. A model for either must name the processor data, scheduling assumptions, and context used to combine individual costs.

Measured performance introduces noise and environmental state. A benchmark can compare two sequences on a named machine under a named protocol. It cannot by itself establish a universal minimum over all programs, and a minimum in an abstract schedule model need not be the fastest sequence on silicon. The honest result is often a pair: a theorem about the model and a measurement about the machine.

Unison supplies a measured case. It integrates register allocation with instruction scheduling through constraint programming. Its speed objective assumes constant instruction latencies, an explicit simplification of dynamic processor behavior. On the Hexagon study, Unison reported a zero optimality gap for 81 percent of the selected functions within the time limit. When the authors ran hot Media-Bench functions on the processor, 60.8 percent became faster than LLVM output, while 17.7 percent became slower, by as much as 11.1 percent. (Castañeda Lozano et al. 2019)

The paper then removed pipeline stalls in an idealized Hexagon experiment. The share of slowed functions fell from 17.7 percent to 3.8 percent, and the rank correlation between estimated and measured speedup rose from 0.70 to 0.89. The solver had not failed to optimize its objective. The objective had hidden processor behavior that affected the measurement.

That distinction has an economic form. A compiler team pays for model design, solver time, benchmark machines, regression analysis, and support for each processor family. A user pays at run time through latency, energy, cloud capacity, or binary size. Spending another minute in a solver can be sensible for code deployed on millions of devices and absurd for code compiled once to run once. The unit of account belongs beside the cost: build seconds per function,

bytes per binary, cycles per invocation, joules per task, or dollars per fleet-hour. None is a free translation of another.

Measurement also changes the mathematical object. Repeated timings form a distribution, not one exact scalar. The experiment needs a warm-up policy, input set, frequency and power controls, isolation rule, sample count, and a summary statistic. A confidence interval can describe uncertainty in a measured difference. It cannot prove that no unmeasured candidate is faster. Search and measurement may work together, but they retain different quantifiers.

13.5.2 Search budgets

Optimization consumes resources before optimized code consumes fewer of them. The complete cost comparison therefore has at least two time scales. Let B be the one-time cost of building the model and search route, S the cost of running the search for one program region, V the cost of validating and deploying its result, and g the saving from one execution. If the optimized region runs n times, the simplest condition for recovering the investment is

$$ng > B + S + V.$$

The quantities may be measured in processor time, energy, engineering hours, or money, but they must use a common unit before addition. The inequality is not a performance theorem. It is a decision model that makes the assumed reuse visible.

Different deployment settings change every term. Ahead-of-time optimization for a widely distributed library can amortize a long search across many machines and years. A just-in-time compiler operates while a user waits, so its search budget may be milliseconds. Firmware for a device with strict capacity or energy limits may justify hours of offline work for a handful of hot routines. A research artifact may value a certified lower bound even when the witness saves no production cost, because the evidence answers a different question.

Expected execution count is itself a forecast. Workloads change, products retire, and a faster successor processor may reduce the value of today's saving. Benefits received years later may also be valued less than costs paid now. A serious deployment model can attach probabilities and time values to the terms instead of treating n as certain. The simple inequality remains useful because it exposes where those assumptions enter; adding uncertain numbers does not turn the decision into a theorem. It makes disagreement about the forecast visible before the search budget is spent unnecessarily.

The build pipeline also has an opportunity cost. If one difficult function occupies a solver worker for an hour, that worker cannot optimize other functions during that hour. A production system needs a policy for allocating a finite portfolio of search time. It may spend more on hot code, on code shared by many products, or on regions where a cheap preliminary search shows a large gap. The policy should separate three outcomes: proved optimal, improved but not proved

optimal, and no accepted improvement. Treating all three as “solver finished” destroys the information needed to revise the budget.

Parallel search changes elapsed time and total work differently. Eight workers may reduce wall-clock delay without reducing processor-hours. Portfolio solving may run several heuristics or encodings because their failure patterns differ. That can improve the chance of an early witness or proof, but it also increases compute, storage, and operational complexity. A report should state both elapsed time and aggregate work. Cloud billing, energy use, and capacity planning follow the second number even when a developer experiences only the first.

Proof production has its own price. Logging clauses consumes I/O and storage. Checking a retained proof consumes more time. Keeping formulas, manifests, and tool versions requires artifact management. These costs buy a different property from a fast unlogged solver verdict: another implementation can recheck the lower bound after the search process has ended. Whether that property is worth its cost depends on the claim. A transient compiler heuristic may need regression tests. A published statement that no cheaper program exists needs stronger evidence.

Model maintenance is often larger than one search run. Processor descriptions change, toolchains acquire new instruction forms, and deployments move to new microarchitectures. A sequence optimized under an old model may remain functionally correct while its performance rationale expires. The durable artifact should therefore separate semantic validity from cost-model validity. A decoder or semantics change can invalidate correctness. A processor-model change can invalidate the ranking without changing the computed result.

There is also a risk term. An incorrect optimization may impose debugging, rollback, security, and reputation costs far larger than its expected speed saving. Validation reduces that risk but does not make it zero. A useful deployment calculation considers not only expected gain but the probability and consequence of error. This is one reason tiny, replayable witnesses and negative controls matter economically: they shorten diagnosis when something goes wrong.

Optimization has a balance sheet

Search time, proof storage, model maintenance, benchmark operation, and validation are inputs. Code size, latency, energy, capacity, and confidence are outputs. The chosen search policy is rational only relative to a workload, deployment horizon, failure cost, and evidence requirement.

13.5.3 Orders, ties, and multiple objectives

Costs need not be single numbers. The adjective **lexicographic** means that we compare the first component and consult the next one only to break a tie. A cost

with this order might minimize bytes first and then instructions among equal-byte candidates:

$$C(P) = (C_{\text{bytes}}(P), C_{\text{inst}}(P)).$$

Lexicographic order makes the first component decisive. A Pareto order instead keeps every candidate for which no other candidate is at least as good in all components and strictly better in one. The result is a frontier, not one winner.

Ties matter. Two different programs can have the same minimum cost. A search for *a* minimum witness may return either. A search claiming uniqueness must rule out every other equivalent candidate at the same cost after stating which syntactic differences count. Register renaming, commutative operands, and duplicate encodings can make uniqueness a much stronger claim than minimality.

Well-founded cost orders

A lower-bound search works by exhausting costs below a witness. This argument needs a cost order with no infinite descent in the searched region. Natural numbers and finite lexicographic tuples of natural numbers have that property. Arbitrary real-valued estimates or partially ordered profiles need a more careful statement of what “all cheaper candidates” means.

A **well-founded order** is an order with no infinite sequence $c_0 > c_1 > c_2 > \dots$. Natural-number instruction counts are well founded: each strict decrease consumes at least one unit, so descent must stop. A finite lexicographic tuple of natural numbers is also well founded. Its first changing component decreases, while all earlier components stay fixed.

Well founded does not mean totally ordered. Suppose each cost pair lists bytes first and cycles second. Under a Pareto order, costs $(4, 3)$ and $(5, 2)$ are incomparable. Neither dominates the other. A complete search can still establish the whole Pareto frontier by showing that every excluded candidate is dominated by a retained candidate. It cannot call one frontier member the minimum without another policy.

Real-valued cost predictions require special care. If a model admits every positive real cost, then $1, 1/2, 1/4, \dots$ descends forever. A finite candidate language still has a finite image under that cost, so the actual instance has minima. The proof should rely on that finite image rather than claiming that the real numbers themselves are well founded. This small distinction keeps an order-theory slogan from doing work it cannot do.

13.6 Matching upper and lower bounds

Let L be the candidate language, $E(P, S)$ the equivalence condition, and $C(P)$ the cost. A witness $P^* \in L$ with $E(P^*, S)$ supplies an upper bound $C(P^*)$. Minimality also needs

$$\neg \exists P \in L, \quad C(P) < C(P^*) \wedge E(P, S).$$

That second formula is the lower bound. It says that the cheaper region is empty of working programs. The two statements do different jobs and deserve different checks.

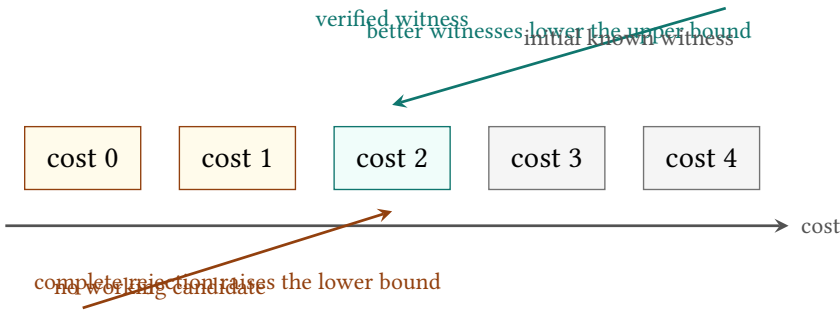


Figure 13.2: A witness lowers the best known upper bound. Exhausting or refuting cheaper cost strata raises the lower bound. Minimality follows only when the two meet under one language and equivalence contract.

A common search alternates the two sides. It begins with a reference program as an upper bound, searches lower costs, and replaces the witness whenever it finds an improvement. When every smaller stratum has been rejected, the last witness is minimal. Another search asks for cost zero, then one, then two, and stops at the first satisfiable cost. That method is complete only if each earlier query covered its entire cost stratum.

Timeouts do not raise a lower bound. Nor does a solver's failure to find a candidate unless the algorithm's completeness and termination for that query have been established. "No result in one hour" is a measurement of the search, not a property of the language.

13.7 An Ao lower bound

We now make every choice explicit. Fix an Ao word width $w \geq 2$. At entry, $r_0 = x$ is the input and output register, and $r_1 = 1$ is a read-only resource. Other registers and memory are unavailable. Candidates are straight-line sequences over these six decoded instances:

```

mov r0, r0      mov r0, r1
add r0, r0, r0   add r0, r0, r1
add r0, r1, r0   add r0, r1, r1

```

Listing 13.1: The complete tiny candidate alphabet

Every instruction executes normally in a valid code region. The harness stops after the declared number of instructions. The observation contains only the final value of r_0 and normal completion; it ignores pc and the Ao condition bits. Cost is instruction count. The specification is

$$S(x) = x + 2 \pmod{2^w}.$$

The task is to find the least cost among candidates of length at most two.

There is a two-instruction witness:

```

add r0, r0, r1
add r0, r0, r1

```

Listing 13.2: A two-instruction witness for adding two

After the first step, $r_0 = x + 1$. The instruction does not change r_1 , so the second step produces $x + 2$, with all arithmetic taken modulo 2^w . This replay establishes the upper bound two.

The lower bound is small enough to see as a table. A zero-instruction program returns x . The six one-instruction instances return only the following five functions; the two mixed-source additions coincide because word addition is commutative.

Table 13.3: Every program below cost two in the tiny Ao language. A single input separates each observed function from $x + 2$.

Candidate form	Final r_0	Separating input
empty or <code>mov r0, r0</code>	x	any x
<code>mov r0, r1</code>	1	$x = 0$
<code>add r0, r0, r0</code>	$2x$	$x = 0$
<code>add r0, r0, r1</code> or reversed	$x + 1$	any x
<code>add r0, r1, r1</code>	2	$x = 1$

Reader proof: no cheaper candidate adds two

At cost zero the output is x , which differs from $x + 2$ because $2 \not\equiv 0 \pmod{2^w}$ when $w \geq 2$. At cost one, the table is a complete split over the six allowed instruction instances. The constant one differs from the specification at $x = 0$. The function $2x$ also differs there. The function $x + 1$ differs for every input because one and two are distinct words. The constant two differs at $x = 1$. Thus no candidate of cost less than two works for every input. The

two-step witness works for every input, so the minimum instruction count in this printed language is two.

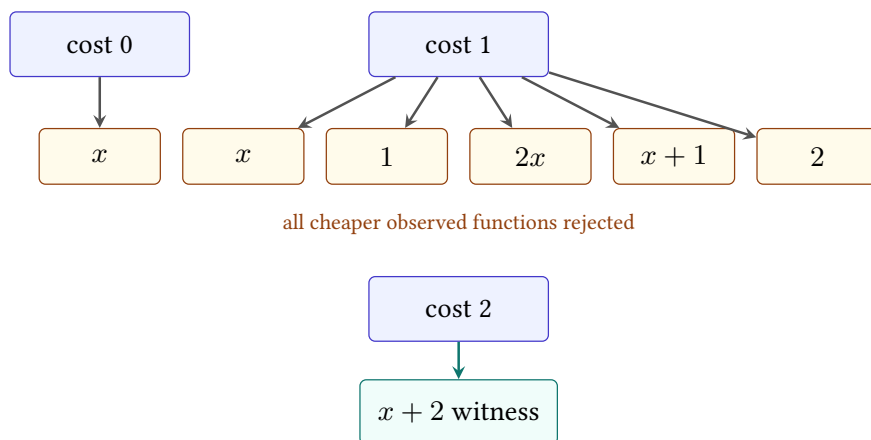


Figure 13.3: The lower-bound argument classifies every observed function below cost two. The witness occupies the first cost stratum that contains the specification.

The case now has a complete reader argument for its teaching language. It is not yet a machine-backed book result: no current object binds this exact grammar, enumeration, checker, and negative control. That distinction matters even when the mathematics fits on half a page. The later evidence route must check the bytes and semantics it claims, not borrow confidence from this table.

13.7.1 Why each clause matters

The width premise excludes $w = 1$, where $2 \equiv 0 \pmod{2}$ and the empty program already implements $x + 2$. Making r_1 read-only prevents the first instruction from destroying the constant used by the second. Ignoring condition bits permits the two-step witness to satisfy a result-only specification; a stronger observation would have to specify the desired final conditions. Banning immediates excludes a one-step `addi` form. Banning memory excludes a table lookup or loaded constant. Limiting the register set keeps the one-step alphabet exactly as printed.

Add just one instruction, `addi r0, r0, 2`, and the old lower bound no longer describes the new language. This is not a counterexample to the argument. It is a reminder that minimality does not float above scope.

Break the claim cleanly

Extend the alphabet with `movi r0, 2`. Does the minimum change? Now extend it with `addi r0, r0, 2`. Explain why the first extension leaves the lower bound intact while the second supplies a cheaper witness. State the modified language before giving either answer.

13.8 Enumeration as evidence

The pencil proof used a useful compression: six one-step programs produced only five observed functions. A program search can exploit the same idea. Breadth-first enumeration begins at the empty program, extends each retained prefix by every instruction instance, executes or symbolizes the result, and groups candidates with equal observed behavior. For a deterministic finite machine at a tiny width, a complete truth table can serve as the behavior key.

Suppose two prefixes have the same observed state transformer and the same available resources, while one costs no more than the other. Any common suffix gives the same observed result, so the more expensive prefix can be discarded. This is semantic quotienting. It can shrink a search dramatically, but the equality used for grouping must retain every state component that a future suffix can read. Grouping only by current r_0 would be unsound if a later instruction can read flags, r_1 , or memory.

For larger widths, a solver can represent instruction choices and symbolic states. A bounded query asks whether there exist decoded instructions and an allowed input relation such that the candidate meets the universal specification. The quantifier structure needs care. To refute equivalence of a fixed candidate, a solver searches for an input where the outputs differ. To synthesize a candidate that works for all inputs, the conceptual form is $\exists P \forall s$. Many practical systems alternate candidate generation with counterexample discovery rather than handing that formula to one solver unchanged.

13.8.1 Covering cheaper candidates

Direct enumeration, symbolic encoding, and dynamic programming can answer the same bounded question, but their evidence looks different.

Direct enumeration generates every candidate, checks its well-formedness, and compares its behavior with the specification. At width four, a truth table with 16 rows gives a complete behavior key for a unary function. The route is easy to inspect. Its cost grows with the number of syntax trees and input states. A durable run records the grammar count, the enumeration order, and the final count in each rejection class. If the grammar predicts 144 one-step instances and the run visits 143, the lower bound has a hole even if every visited candidate fails.

A symbolic encoding replaces choices with variables. For each step, selector variables choose an instruction family and its operands. Transition constraints connect the symbolic input state to the next symbolic state. A cost constraint restricts the selected sequence to one cheaper stratum. The resulting formula is satisfiable when some represented candidate meets the encoded specification and unsatisfiable when none does. A refutation can make the last statement portable, but a separate argument must still connect selector assignments to legal candidates in both directions. Every satisfying selector assignment must lift to one legal program. Every legal program in the stratum must also have a selector assignment.

The first line protects soundness of witnesses. The second protects completeness of lower bounds. Testing only the model-lifting direction leaves the dangerous half unchecked.

Dynamic programming stores one least-cost representative for each behavior and resource state. In the tiny Ao case, the five functions below cost two are the table entries. Extending an entry by one instruction produces entries for the next layer. If two prefixes have the same complete continuation-relevant behavior, the higher-cost one can be discarded. The method can prove a lower bound when the behavior key and extension rule form a complete quotient. It can also fail quietly when the key drops a flag or resource that a later suffix reads.

Searches often combine the three. Concrete tests reject easy candidates. Symbolic verification separates survivors. A semantic table prevents the same behavior from returning under another spelling. A satisfiability solver refutes the final symbolic stratum. The combination is sound only when the handoffs preserve coverage. Fast rejection is an engineering technique; it does not excuse an unrepresented candidate.

13.8.2 Soundness and completeness of the encoding

It helps to follow one stratum from machine language to a Boolean formula that a separate program can check. Fix a cost c and a finite separating input set T . For every program position, Boolean selector variables choose an instruction form and its operands. Exact-choice clauses require one legal choice at each active position. Additional clauses enforce typing, resource transitions, the cost equation, and the stop policy.

For each input $s \in T$, make a separate symbolic copy of the machine state. The selector variables are shared across copies because the same program must run on every input. State variables are not shared because each run begins with a different input. Transition clauses connect the selected instruction at position j to the state before and after that position. Final clauses require the declared observation to match the specification.

Call the resulting formula $F_c(T)$. It is satisfiable exactly when some program of cost c meets the contract on every input in T . The right-to-left direction is **encoding completeness**: every legal program has selector values and state values that satisfy the clauses. The left-to-right direction is **encoding soundness**: every

satisfying assignment decodes to a legal program whose replay meets the sample contract. Model lifting tests only the second direction. A lower bound also needs the first.

For the one-instruction $x + 2$ example, take $T = \{0, 1\}$. One selector chooses among the six instruction instances. Two state copies execute that same choice. The final constraints demand results two and three, respectively. The table already showed that no row produces both results. Therefore $F_1(\{0, 1\})$ is unsatisfiable. Since any universally correct program must be correct on zero and one, this finite-sample refutation is enough to exclude the complete cost-one stratum.

Conjunctive normal form, or CNF, permits only clauses made from disjunctions of **literals**, meaning Boolean variables or their negations. Bit-vector operations and exact-choice constraints must therefore be reduced to Boolean gates and then to clauses. A fresh variable may name the output of an internal gate. Clauses constrain that variable to equal the gate's function. This introduction of names is often called a Tseitin encoding. It can keep the formula proportional to the circuit size instead of expanding a nested expression exponentially.

The clause generator becomes part of the trust boundary. A missing clause may admit a false witness. An extra clause may remove a real candidate and create a false lower bound. A SAT proof checker establishes only that the emitted CNF is unsatisfiable. It does not establish that the CNF faithfully represents the candidate language or transition semantics.

The four links from refutation to lower bound

First, prove or test that legal programs of cost c map into satisfying selector assignments whenever they meet the sample contract. Second, retain the exact CNF bytes and check that their digest matches the manifest. Third, check the solver's refutation against those bytes with an independent proof checker. Fourth, use the subset argument: a program correct on every allowed input would be correct on every input in T . Unsatisfiability of $F_c(T)$ then excludes every universally correct program in the stratum.

Proof logging changes the operational design of the search. The generator must use a solver route that can emit an accepted proof format. Preprocessing must either log its transformations or produce a proof for the transformed formula that can be connected to the original. Splitting a large formula into cases must retain a cover proof. Formula, proof, and checker versions must be stored before temporary files disappear.

There are useful independent controls for each link. Remove one legal selector and confirm that the grammar-count comparison fails. Flip one transition bit and confirm that concrete replay rejects a lifted model. Delete proof bytes and confirm that the proof checker fails. Change one language digest while keeping the old proof and confirm that the manifest refuses to compose the claim. These mutations

separate an actual evidence route from a script that prints success after several unrelated commands.

13.8.3 Branch-and-bound search

A **branch-and-bound search** divides the candidate choices into branches and carries a lower estimate of the best cost each branch can still achieve. If the estimate is no better than the current witness, the search cuts that branch. The witness supplies the bound. The branch estimate supplies the reason that the cut cannot hide an improvement.

For additive nonnegative instruction weights, a prefix of cost c cannot lead to a program below c . That lower estimate is weak but sound. A stronger estimate may add the least possible cost of the unresolved outputs or resources. Its proof obligation is monotonic: every completion of the prefix must cost at least the estimate. An optimistic guess that sometimes exceeds a real completion can prune the optimum.

Memoization and meet-in-the-middle search offer different savings. **Memoization** stores the result for a behavior already reached so the search does not solve it twice. A meet-in-the-middle method enumerates prefix and suffix summaries separately and joins compatible pairs. Both depend on a summary sufficient for composition. A suffix that reads flags cannot be joined using a prefix summary that retained only r_0 . The same state boundary governs enumeration, dynamic programming, and decomposition.

13.8.4 Counterexample-guided search

Counterexample-guided inductive synthesis (CEGIS) keeps a finite set of test inputs. It finds a candidate that works on those inputs, then asks a **verifier**, the checker for universal agreement, for an allowed input where the candidate fails. A counterexample joins the test set, and the loop repeats. If the verifier finds none under a sound complete route, the candidate meets the universal contract. If no candidate can satisfy the accumulated examples, the current cost stratum may be empty, but the lower bound still depends on how candidate choices and universal correctness were encoded.

This loop turns failures into useful information. The values $x = 0$ and $x = 1$ in Table 13.3 are exactly such separating inputs. They eliminate whole classes of candidate functions. The search is not merely trying programs; it is learning which distinctions the specification requires.

The tiny language gives a complete trace. Begin the cost-one stratum with the test set $T = \{0\}$. The one-step candidate `add r0, r1, r1` returns two, so it passes that test. Universal verification finds $x = 1$: the candidate still returns two, while the specification returns three. Add one to T . No one-step candidate in Table 13.3 works on both zero and one, so a complete candidate generator reports the cost-one sample problem unsatisfiable. Any program correct for every input would be

correct on those two inputs. The unsatisfiable sample problem therefore rules out the whole cost-one stratum, provided the generator encoded every candidate in it.

At cost two, the generator can return the repeated-add witness. A verifier now asks whether there exists an allowed x such that

$$((x + 1) + 1) \bmod 2^w \neq (x + 2) \bmod 2^w.$$

The formula is unsatisfiable. The reader reason is associativity of modular addition; a machine route may instead retain a checked refutation. The loop terminates with a cost-two upper bound and a cost-one lower bound.

The quantifiers explain each result. Synthesis asks

$$\exists P \in L_c \forall s, \quad A(s) \implies O(P(s)) = O(S(s)),$$

where L_c is one cost stratum. Candidate generation over a finite test set replaces the universal range by a conjunction over T . Verification fixes P and searches for the opposite statement,

$$\exists s, \quad A(s) \wedge O(P(s)) \neq O(S(s)).$$

A satisfying verifier query supplies the next test. An unsatisfiable verifier query establishes that fixed candidate's universal agreement under the encoded semantics. An unsatisfiable sample query establishes an empty stratum only when its program variables cover all of L_c . These are two different unsatisfiable formulas. A report that does not say which one failed is unfinished.

Termination also has a scope. If L_c and the input domain are finite, each failed candidate can yield a counterexample and only finitely many candidate behaviors remain. A deterministic loop that never repeats an eliminated behavior must stop. With an infinite grammar, an incomplete verifier, or a resource cap, the same loop may return unknown or run forever. The letters CEGIS name the alternation; they do not grant completeness.

13.8.5 Retaining independently checked evidence

A lower bound is **independently checked** when it carries evidence that a separate program can accept or reject without repeating the original search. Such a result needs more than a final line saying that a solver returned an unsatisfiable verdict. The evidence package should retain enough information to reconstruct the claim:

A **refutation** is a proof that the formula has no satisfying assignment. It supplies the logical evidence for one cheaper stratum; the package must still bind that formula to the language.

1. a serialized candidate-language contract;
2. the specification, precondition, observation, and cost order;

3. the exact decoder and semantic version or digest;
4. the formula for every cheaper cost stratum, or a reproducible generator;
5. a witness at the claimed minimum cost;
6. a refutation for each cheaper satisfiability query when the solver route can export one; and
7. a negative control that corrupts a witness, formula, or proof and is rejected.

One concrete package can make those dependencies hard to confuse. Its root record contains digests of five child records: language, semantics, specification, cost, and observation. A witness record names the root digest, the chosen instructions, their bytes where byte cost matters, and a replay result. Each cheaper stratum record names the same root digest, its cost predicate, the generated formula digest, the refutation digest, and the proof checker version. A coverage record gives the grammar cardinality and records every sound quotient used before encoding.

Table 13.4: A lower-bound package binds the claim in both directions. Changing a parent record invalidates every child digest that depended on it.

Record	Binds	Independent check
language	forms, operands, resources, bounds	recount and enumerate
semantics	decode and step relation	replay test corpus
claim	precondition, specification, observation	type and scope check
witness	minimum-cost candidate	concrete or symbolic replay
stratum	cheaper formula and refutation	proof checker
coverage	quotients and complete strata	representative proof and counts
control	deliberate mutation	required rejection

Suppose a later edit adds `addi r0, r0, 2`. The language digest changes. An old cost-one refutation still proves its old formula unsatisfiable, but its parent digest no longer matches. The package must refuse the lower-bound claim before reading the proof. A perfect proof of yesterday's smaller language cannot certify today's larger one.

A long search may split a stratum into cells. Then the package needs a coverage argument as well as a refutation for each cell. If choices $g_1, \dots, g_m$ form finite branch groups, the cells must be exactly the Cartesian product $g_1 \times \dots \times g_m$. Every tuple must occur once, and each tuple's added assumptions must match its proof. Missing one cell leaves a candidate region open. Duplicating a cell does not close the hole. The checker should reconstruct the product rather than trust a line count.

The witness checker and lower-bound checker should fail for different reasons. Change the second `add` in the Ao witness to `add r0, r0, r0`; replay should find an input where the result differs. Add a legal one-step `addi` candidate without

regenerating the cheaper-space formula; a language-closure check should catch the mismatch before a solver result is trusted. Truncate a refutation; the proof checker should reject it. These attacks test three different links.

A solver verdict is not a portable lower bound

An unsatisfiable verdict reports what one solver said about one formula. A checked refutation can support the claim that the formula has no satisfying assignment. Neither establishes that the formula represents the printed candidate language unless the encoding bridge is also checked.

13.9 Negative controls for search

The most revealing negative control gives the checker something it ought to accept but the claimed lower bound says cannot exist. Add a known cheaper witness to the language, rebuild the search space, and require the lower-bound check to fail. This tests candidate coverage and not merely proof parsing.

Other controls target nearby faults:

- omit one operand form from enumeration while leaving it in the printed grammar;
- reverse one subtraction operand in the symbolic semantics;
- observe r_0 but accidentally drop a required flag;
- allow a temporary in synthesis but freeze it in verification;
- compute byte cost from decoded mnemonics instead of actual encodings;
- accept a solver log after deleting the load-bearing proof bytes; or
- change the precondition between the witness and lower-bound queries.

A positive run says the components agree on the intended instance. A negative control shows that at least one nearby wrong instance crosses a failure boundary. It does not prove the checker has no other bugs. It makes the checking claim falsifiable and gives future maintainers a precise alarm.

13.10 RV64 and x86-64 search spaces

Ao makes instruction-count minimality unusually clean. Its instructions have one width, its first scalar model has a small explicit state, and the book owns its definition. The real-ISA slices add obligations before they add glory.

The $x + 2$ example shows why a lower bound does not transfer by analogy. In a selected RV64I language that admits `addi a0, a0, 2`, one instruction suffices. Its encoding is `13 05 25 00`. In 64-bit x86 mode, `addq %$2, %rdi` has bytes `48 83 c7 02` and also supplies a one-instruction in-place witness. The Ao proof remains correct because neither immediate form belongs to its six-instance alphabet.

x86-64 adds another choice before we can call the witness equivalent. The `addq` form writes arithmetic flags. The four-byte `leaq 2(%rdi), %rdi`, encoded as `48 8d 7f 02`, computes the same register result without those flag effects. Under a result-only observation, both may compete. If a following branch reads flags, the reference behavior decides which, if either, is legal. Equal instruction count and equal byte length do not settle the semantic contract.

Table 13.5: The same source-level expression enters three different candidate languages. The minimum shown is only the first visible upper bound.

Machine slice	Candidate	Instructions	Bytes
A0 six-form language	two register adds	2	8
RV64I with <code>addi</code>	<code>addi a0, a0, 2</code>	1	4
x86-64 selected forms	<code>addq %\$2, %rdi</code>	1	4

The table does not prove either real instruction minimal. That conclusion would still require a complete zero-cost stratum and the exact instruction language. It demonstrates something narrower and more useful: changing the constant route changes the candidate universe before any search begins.

For an RV64I scalar search, name the base ISA version, permitted extensions, architectural integer width, privilege assumptions, and exact operand forms. Decide whether compressed instructions compete. State how illegal encodings, misaligned control flow, loads, stores, and traps enter the observation. If the language includes only register-register integer operations, say so. “RISC-V program” is much too large a noun.

For x86-64, decode mode and encoding are central. Name 64-bit mode, selected instruction forms, operand and address sizes, allowed prefixes, register classes, memory forms, and the portions of the flags register that matter. Decide how partial-register writes and implicit operands enter the state. If cost is bytes, retain the actual encoding. If cost is instruction count, explain how multiple encodings of one decoded instruction are handled.

Neither slice authorizes a claim about RISC against CISC. One may expose an interesting contrast: a regular fixed-width language makes byte cost easy to relate to instruction count, while a variable-length language makes encoding part of the optimization problem. That is a statement about declared features, not two architectural essences.

Prior systems have claimed instruction-sequence optimality under their own languages and models.(Bansal and Aiken 2006) The contribution sought here is

therefore not a priority slogan. It is a per-instance route in which the reader can inspect the language, replay the witness, check the cheaper-space refutations, and break the checker with a negative control.

13.11 Axeyum route

The live Axeyum checkout supplies more than generic solver layers and less than the three-machine route this book needs. Its search crate has complete bounded AVX2 shuffle encodings. A satisfying assignment is lifted to a typed program and replayed through separate execution code. Unsatisfiable bounds can carry DRAT refutations checked against the generated formula. One example constructs a two-instruction byte reversal over a declared unary language containing `vpshufb` and `vperm2i128`. It checks that no zero- or one-instruction candidate in that language works. Another encoding assigns family weights and checks one bound at a time.

The same crate has an exhaustive-cover design for large combinatorial instances. It checks that every branch group belongs to the formula, that the recorded cells equal the full Cartesian product, that no cell is duplicated, and that every cell carries an accepted refutation. It can also compose cell proofs into one refutation of the original formula. Those mechanisms are directly relevant to a durable lower-bound package.

The missing boundary remains substantial. Axeyum does not yet supply the book's reusable Ao decoder and transition system, nor complete RV64 or x86-64 semantic and candidate-language packages. Its AVX2 shuffle semantics track byte provenance for a selected operation family; they are not an x86-64 machine semantics. We should not place a Python convenience function over that gap and call the result an ISA proof.

The first implementation route should follow the dependency order already visible in the reader argument.

1. Generalize the existing Rust search contracts beyond the selected AVX2 families: decoded forms, resources, precondition, observation, bound, and cost.
2. Connect an Ao instance of that contract to an executable decoder and step semantics owned by the book's machine definition.
3. Enumerate the tiny language and replay every candidate concretely at a small width, including the zero- and one-instruction layers above.
4. Translate the same semantics to Axeyum bit-vector expressions and check candidate equivalence by searching for a counterexample input.

5. Reuse the existing satisfying-model lift, DRAT checking, and exhaustive cover obligations, while binding them to canonical language and semantic digests.
6. Add mutations for language closure, semantic replay, witness failure, missing cover cells, and truncated proof bytes.
7. Expose a small Python interface only after Rust owns these meanings and the evidence record.

Python is useful for teaching and orchestration. A reader should be able to construct the six-form language, request a bounded search, inspect the witness, and print a counterexample without writing Rust. The Python object must serialize to the same canonical contract the Rust checker reads. It must not reimplement instruction semantics or decide what a successful proof means.

The teaching display should expose the search strata as well as the winner. For each cost below the upper bound, it can show the number of syntactic candidates, the number rejected by typing or resource rules, the number eliminated by concrete counterexamples, and the obligations sent to the solver. Those counts make the lower-bound argument visible. They also catch a damaged enumerator: a suspiciously empty stratum is a question, not a gift.

One small witness from every rejection class is useful. A malformed operand explains the grammar. A clobbered scratch register explains the observation. A boundary input explains semantic failure. The retained examples teach why the candidate language and harness are part of the theorem rather than setup around it.

```
# Illustrative API: this interface is not yet implemented.
language = CandidateLanguage.a0(
    width=4,
    forms=["mov.rr", "add.rrr"],
    registers=["r0", "r1"],
    constants={"r1": 1},
    writes=["r0"],
    observation=["normal", "r0"],
    max_instructions=2,
)
result = search_minimal(
    language,
    spec="r0_out == r0_in + 2",
)
result.replay_witness()
result.check_lower_bound()
```

Listing 13.3: The intended shape of a future teaching interface

The comment is part of the lesson: the interface is a design target, not a reported capability. Once it exists, the book should replace illustrative code with

an executable example and bind any machine-backed result to the exact Axeyum revision and checker route.

The real-ISA layers come later. First establish that decoded bytes and Ao steps agree, then lift selected RV64 forms, then selected x86-64 forms. The existing AVX2 result can inform the search and proof design, but it cannot reverse this dependency. Only after the semantic bridges survive their own negative controls should the same minimality engine range over them. The search engine is not the hard foundation. Meaning is.

13.12 Where the model stops

No claim here ranges over all programs on any architecture. The model excludes self-modifying code, concurrency, interrupts, privileged state, caches, speculative execution, and whole-program link layout. It does not infer hardware speed from an instruction count or turn a timeout into a lower bound.

Even within straight-line scalar code, a model may omit a useful instruction, constant route, temporary, or encoding. Such an omission does not make the bounded result false. It makes the result narrower. The remedy is to publish the language so another person can extend it and see whether the minimum changes.

The construction has a modest and surprising reach. Its language is finite because we chose the pieces and the bound. Once those choices are fixed, checking every cheaper program can support a statement about all of them. A small list of instructions becomes a universal negative, but only inside the border we drew. Publishing that border lets the next search enlarge it without misquoting the result already obtained.

13.13 Exercises

1. **Execute.** At width four, replay the two-instruction Ao witness for inputs 0, 1, 7, 14, and 15. Record the intermediate and final r_0 .
2. Enumerate all six one-instruction candidates in the tiny language without quotienting commutative additions. Confirm that their observed functions match Table 13.3.
3. **Prove.** Generalize the reader argument from adding two to adding k by repeated addition of the entry constant one. State a language in which k instructions are minimal, or explain why the present language admits a shorter construction for some k .
4. Find every place the width premise $w \geq 2$ enters the lower-bound argument. Give the exact result at width one.
5. **Break.** Add `addi r0, r0, 2` to the language. Write the new one-step function table and identify the failed sentence in the old proof.

6. Add `sub r0, r0, r1`. Does it create a cheaper witness for $x + 2$? Does it create new observed functions? Answer for width two and width three.
7. Change the observation so that it includes Ao's (Z, N, C, V) . Define the specification's desired final conditions, then determine whether the old witness still works.
8. Allow r_1 to be overwritten and add `mov r1, r0` plus addition forms writing r_1 . State the preservation rule needed if the caller observes r_1 at exit.
9. Give two concrete x86-64 instruction sequences that would be ordered differently by instruction count and encoded byte length. Pin the mode and write the bytes before comparing them.
10. For RV64, compare a language containing only 32-bit RV64I instructions with one that admits a named compressed extension. Which parts of the candidate-language contract must change?
11. Design a lexicographic cost that minimizes memory accesses, then bytes, then instruction count. Give two candidates tied on the first component and separated by the second.
12. Construct three abstract candidates whose costs form a Pareto frontier under bytes and latency. Explain why the frontier has no single minimum without another policy choice.
13. **Act as the checker.** Mutate the second instruction of the Ao witness in three ways. For each mutation, give the smallest separating input you can find and the observed wrong result.
14. Design a negative control that detects omission of one legal operand form from an enumerator. State which component must fail before the solver is called.
15. A search reports no candidate of cost at most three after a one-hour timeout. List the additional evidence needed before this observation can support a lower bound.
16. State the semantic condition under which two search prefixes may be grouped by their current state transformer. Give a counterexample to grouping only by r_0 when later instructions can read Z .
17. **Transfer.** Write the candidate-language contract for replacing a two-instruction RV64I integer sequence at one compiler site. Include live registers, extensions, memory, traps, observation, and cost.

18. Design the first Axeyum evidence record for the tiny Ao search. Include canonical language bytes, semantic revision, witness, formula digests, checker result, and one negative-control mutation. Mark which fields the Python layer may display and which meanings Rust must own.
19. **Count.** Reproduce the 144-instance grammar calculation. Then add a subtraction form with one destination and two register sources. Give the new alphabet size and the number of sequences of length at most three.
20. A form has destinations from four registers, its first source from four registers, and its second source from three registers unequal to the destination. Explain why the independent product formula is wrong, then count the legal instances.
21. Draw the first two levels of a candidate tree in which the first instruction may define one new temporary and the second may read any value defined so far. Label the outgoing count at each kind of node.
22. **Break.** Write an enumerator that accidentally treats a destination choice and a source choice as independent when they share a no-alias rule. Give one legal program it retains and one illegal program it must reject.
23. For an alphabet of size $I > 1$, derive the geometric-sum formula for all lengths through K by multiplying the sum by I and subtracting. State why the $I = 1$ case needs a separate expression.
24. The 144-instance grammar gains one additional immediate value. Compute the new alphabet size and the factor by which the exact length-four stratum grows.
25. **Prove.** Formalize the two-scratch-register swap as a permutation π . List the three separate facts needed about legality, semantics, and cost before the first-use standard naming rule is sound.
26. Give an x86-64 case where swapping two register names changes encoded byte length because one name requires an additional prefix bit. Explain which line of the symmetry proof fails under byte cost.
27. Give an ABI case where two registers have equal instruction cost but different preservation duties. State whether the failure is in the language, observation, or cost contract.
28. Extend first-use canonicalization from two to three fully symmetric scratch registers. Describe a canonical naming rule and prove that every orbit has a representative satisfying it.

29. Two candidates have costs $(4, 3)$, $(5, 2)$, and $(6, 4)$ under (bytes, cycles). Find the Pareto frontier. Then choose a lexicographic order and identify its winner.
30. **Break.** Give a scalar cost function with negative instruction weights. Construct an infinite descending sequence if the language admits unbounded repetitions. State two different repairs to the cost contract.
31. Prove that a finite candidate language has a minimum under any real-valued cost function, even though the ordinary order on the real numbers is not well founded.
32. A compiler minimizes expected cycles using profile probabilities. Change one branch probability enough to reverse the order of two candidates. State which context bytes or records must be pinned with the result.
33. Compare latency and reciprocal throughput for a chain of four dependent operations and four independent operations. Explain why adding per-instruction latencies models the first case better than the second.
34. Design a measurement protocol for two 20-byte straight-line x86-64 sequences. Name the processor, inputs, warm-up, repetitions, frequency policy, summary statistic, and uncertainty report. State the strongest conclusion the measurement can support.
35. Unison's idealized no-stall experiment reduced the slowed-function share from 17.7 percent to 3.8 percent. Calculate the change in percentage points. Explain why the experiment diagnoses a model boundary but does not prove that all remaining error comes from one source.
36. Give one deployment where spending 60 seconds of solver time per hot function is economically plausible and one where it is not. Name the actor who pays compile time and the actor who receives the run-time benefit.
37. **CEGIS.** Replay the cost-one trace with test set $T = \{0\}$. List every one-step candidate that passes the first test, not only the constant-two candidate. Find a separating input for each survivor.
38. Start the same CEGIS loop with $T = \{1\}$. Which one-step candidates pass? Add one counterexample that makes the sample problem unsatisfiable.
39. Prove the finite-sample lower-bound step: if no $P \in L_c$ satisfies the specification on T , then no $P \in L_c$ satisfies it on every allowed input. Identify the direction of implication that would be invalid.
40. **Break.** Let the candidate generator omit one program from L_c . Show how its sample query can be unsatisfiable while the full stratum contains a correct program.

41. Distinguish the two unsatisfiable formulas in a CEGIS run: candidate generation over T and counterexample search for a fixed P . For each, write what the absence of a satisfying assignment establishes.
42. Give a finite CEGIS loop that repeats the same candidate forever because it fails to retain a blocking clause. State the progress measure needed for a termination proof.
43. **Act as the checker.** A package contains proofs for 15 of 16 Cartesian-product cells and duplicates one of the 15. Explain why its row count still looks correct and which two independent checks reject it.
44. Change one byte in a serialized language contract without changing a stored stratum formula. Trace the digest mismatches that should prevent the old proof from being credited.
45. A proof checker accepts a refutation of formula F . List the remaining steps needed to establish that F represents every candidate cheaper than the witness.
46. Compare an equality-saturation extraction claim with an instruction-byte minimality claim. Name the candidate universe, equivalence source, and cost in each. Give one reason neither claim implies the other.
47. Read Massalin's candidate filtering order and Bansal–Aiken's offline rewrite-table design. Identify one engineering choice in each that increases capacity and one premise that choice adds to the resulting claim.
48. **Transfer.** Choose one minimum-shuffle result from Franchetti–Püschel. Write the narrowest candidate-language and lower-bound statement you can justify from its rewrite system. Do not turn it into a claim about every instruction sequence.
49. Inspect Axeyum's unary AVX2 byte-reversal example. Separate the witness replay, the zero/one-step formula, the DRAT proof, and the proof check. State what additional semantic bridge would be needed before calling it an x86-64 machine-code result.
50. Inspect Axeyum's weighted unary AVX2 encoding. Replace its declared family weights with instruction count. Predict which records and formula digests must change, even if the same witness remains optimal.
51. Design one negative control for each Axeyum cover obligation: branch membership, complete Cartesian product, unique cells, and accepted cell refutations. Each control should damage only the named obligation.

52. **Count and prove.** Reproduce Table 13.1. Then forbid repeated add sources and derive the 72-program total. Prove that your cases partition the legal two-instruction programs.
53. Extend the resource grammar with a comparison that defines a Boolean condition and a select that reads one condition and two word values. Give a resource-state representation that prevents either form from reading a value of the wrong type.
54. Add one store form to the resource grammar. Define the memory-token transition for a two-store sequence and explain why grouping prefixes only by register values is no longer a sound semantic quotient.
55. For the cost-one Ao stratum and $T = \{0, 1\}$, introduce selector variables for all six candidates. Write exact-choice clauses in prose or CNF and state the final-result constraints for both input copies. Explain why the selectors are shared and the state variables are not.
56. **Break the encoding.** Remove the clause that prevents two instruction selectors from being true together. Describe a satisfying assignment that no legal one-instruction program can realize. Which direction of the encoding theorem fails?
57. A proof checker accepts a refutation after the solver simplified the original formula. State two acceptable ways to connect the proof to the original formula and one unacceptable way that merely trusts the preprocessor.
58. Use the break-even inequality with $B = 200$ engineering hours, $S = 2$ machine-hours, $V = 3$ engineering hours, and a saving worth one micro-dollar per execution. Convert the terms to one declared monetary model and calculate the required execution count. List two assumptions that make the result uncertain.
59. Compare a search run using one worker for eight hours with a run using eight workers for ninety minutes. Calculate wall-clock time and processor-hours. Give one setting that values each measure more strongly.
60. Separate semantic validity from cost-model validity for an optimized x86-64 sequence after deployment moves to a new processor. Name the evidence that can be reused and the measurements or model checks that must be repeated.
61. A wrong candidate fails on 16 inputs in a domain of 256 values. Compute the probability that 20 independent uniform tests miss every failure. Explain why this probability is not a correctness certificate.

62. A two-register swap acts on 100 candidates, 20 of which use neither scratch name and are fixed by the swap. Use the two-element orbit formula to count the orbits. Explain why dividing 100 by two gives the wrong answer.
63. For three symmetric scratch registers, state the first-use canonical naming rule and identify one ABI change that destroys the group action. Say whether the failure affects legality, observation, cost, or more than one.

Evidence That Can Fail

A command prints the word we hoped to see. The terminal returns success. We save the output and call it evidence. Six months later, someone changes one byte of the claimed formula and the same command still succeeds.

The first run told us less than we thought. Perhaps the checker ignored its input. Perhaps the command checked a proof against a different formula. Perhaps the formula did not encode the program claim. Perhaps a shell pipeline discarded the checker's failure code. The desired word on a screen was a fact about one execution. It was not yet a reason to believe the theorem.

Evidence becomes useful when the route can lose. A witness checker must reject a wrong witness. A proof checker must reject a damaged proof. A manifest checker must notice a changed semantic package. The failure need not be dramatic. A nonzero exit code at the right boundary is enough. What matters is that the claimed result controls the outcome.

This chapter builds that route from first principles. We will separate tests, complete computations, solver verdicts, certificates, and kernel reconstruction. We will bind each result to its statement and semantics. Then we will examine an awkward case found in an earlier generation of this very book: a proof file almost a megabyte long that carried no load-bearing information.

Evidence must discriminate

Evidence supports a claim only when a relevant change to the claim, input, or supporting object can make its check fail. A saved success message has no such power by itself.

14.1 Why solvers produce proofs

Boolean satisfiability solvers answer whether a propositional formula has an assignment that makes it true. A satisfying answer can be accompanied by the assignment. Another program can substitute the values and check every clause. An unsatisfiable answer is harder: there is no assignment to hand back. The claim

ranges over all assignments, and a bare verdict asks the reader to trust the solver's search, implementation, memory, and output path.

Proof logging changed that bargain. Instead of returning only an unsatisfiable verdict, a solver can emit a sequence of additions and deletions that leads to contradiction. The DRAT format became practical because it could express the reasoning used by modern SAT solving and preprocessing, while a separate checker could validate the log. The accompanying `drat-trim` system also used backward checking and proof trimming to reduce the portion that must be retained.(Wetzler et al. 2014)

The next question was how much checker code the result should trust. The linear resolution asymmetric tautology (LRAT) format adds explicit hints to clausal steps so a simpler checker can validate them. Its authors demonstrated certified checkers in Coq and ACL2, moving the final decision into existing theorem-prover environments.(Cruz-Filipe et al. 2017) For satisfiability modulo theories, proof formats must also describe reasoning about equality, arithmetic, arrays, bit vectors, and other theories. Alethe grew from the proof language used by veriT toward a common interchange format for such SMT reasoning.(Schurr et al. 2021)

The historical movement is not from untrustworthy software to perfect software. It is from one opaque decision to a chain of smaller, inspectable decisions. A proof-producing solver may be enormous and aggressively optimized. If a much smaller checker rejects invalid proof steps, the solver can leave the trusted base. Formula generation and semantic translation do not disappear, however. A flawless proof of the wrong formula remains a flawless proof of the wrong formula.

14.1.1 Resolution proofs

The logical foundation is older than modern SAT solving. In 1965 J. Alan Robinson presented resolution as a machine-oriented inference principle. His setting was first-order theorem proving, where substitution and logical inference had to work together. The propositional form used in SAT proof checking is smaller: it combines two clauses that disagree on one Boolean variable.(Robinson 1965)

A **Boolean variable** has one of two values, false or true. A **literal** is a variable such as p , or its negation $\neg p$. A **clause** is a disjunction of literals. It is true when at least one of its literals is true. A formula in conjunctive normal form is a conjunction of clauses, so every clause must be true.

Resolution rule

Let A and B be disjunctions of literals that do not mention the displayed occurrence of variable p . From parent clauses

$$A \vee p \quad \text{and} \quad B \vee \neg p,$$

resolution derives the **resolvent**

$$A \vee B.$$

The variable p is the pivot. Duplicate literals may be removed. A clause containing both x and $\neg x$ is always true and need not strengthen a refutation.

The rule is sound for a case reason. Suppose one assignment satisfies both parents. If it makes p false, the first parent can be true only because A is true. If it makes p true, the second parent can be true only because B is true. In either case, $A \vee B$ is true. Every model of the parents is therefore a model of the resolvent.

The smallest useful end point is the **empty clause**. A clause is true when one of its literals is true. The empty clause has no literal that could make it true, so every assignment makes it false. Deriving it from the input clauses proves that no assignment satisfies all inputs.

Consider this four-clause formula:

$$\begin{aligned} C_1 &= p \vee q, & C_2 &= \neg p \vee q, \\ C_3 &= p \vee \neg q, & C_4 &= \neg p \vee \neg q. \end{aligned}$$

Every possible pair of truth values is forbidden. If q is false, the first two clauses demand both p and $\neg p$. If q is true, the last two clauses demand both p and $\neg p$. Resolution records the same argument without listing all four assignments.

Table 14.1: A complete resolution refutation. Each derived row names its two parents and pivot. The last row is the empty clause.

Row	Clause	Parents	Pivot
1	$p \vee q$	input	–
2	$\neg p \vee q$	input	–
3	$p \vee \neg q$	input	–
4	$\neg p \vee \neg q$	input	–
5	q	1, 2	p
6	$\neg q$	3, 4	p
7	empty clause	5, 6	q

Row 5 follows because resolving $p \vee q$ with $\neg p \vee q$ removes the two pivot literals and leaves q . Row 6 leaves $\neg q$. Row 7 resolves those unit clauses and leaves nothing. The proof is small enough to check with a pencil, but it already has the shape of a proof file: input clauses, numbered derived clauses, parent references, pivots, and a required final contradiction.

Reader proof: an accepted resolution trace is sound

Maintain this invariant after every accepted row: each stored clause is a logical consequence of the original input clauses. The invariant holds for an input row because that clause is one of the assumptions. For a derived row, the checker first confirms that both parents already satisfy the invariant. Resolution soundness then makes the resolvent a consequence of those parents and therefore of the inputs. Induction gives the invariant for every row. If the final accepted row is the empty clause, the inputs imply false. Hence the input formula has no satisfying assignment.

This proof identifies what a small checker must do. It must parse literals and clauses without changing their meaning. It must reject an unknown parent, a future parent, or a parent that has been deleted. It must verify that the claimed pivot occurs positively in one parent and negatively in the other. It must calculate the resolvent rather than trust a printed child. Finally, it must require the empty clause. Checking several valid intermediate rows and then accepting end-of-file would not establish contradiction.

The invariant proves checker soundness only under assumptions about the implementation. An integer overflow in a variable identifier, a parser that drops a minus sign, or a set representation that loses a literal can violate the mathematical model. A machine-checked implementation can reduce that gap. Differential tests, malformed inputs, and mutation controls can expose ordinary defects. None of those measures changes which exact code and runtime remain trusted.

14.1.2 Producer and checker responsibilities

The seven-row trace carries explicit parents and pivots. Modern conflict-driven SAT solvers perform many more transformations. They learn clauses, delete clauses, eliminate variables, simplify formulas, and use inference rules that would make a fully explicit elementary-resolution log expensive to produce. Proof formats divide the resulting work differently.

DRAT permits clause additions and deletions. Its redundancy condition is expressive enough for the solving and preprocessing techniques considered by its designers. The producer can log changes close to the form in which a solver already performs them. The checker does more discovery work: it must establish that an added clause has the required redundancy property. DRAT-trim can check backward from the contradiction, retain the proof steps that matter, and add useful deletion information.(Wetzler et al. 2014)

LRAT adds ordered hints. These hints name clauses used during propagation and make the redundancy check more direct. A noncertified tool may translate a DRAT proof into LRAT; a certified checker then validates the LRAT result. The translator need not be trusted, because a bad translation should fail the certified

check. The 2017 work demonstrated such checkers in both Coq and ACL2.(Cruz-Filipe et al. 2017)

SMT proofs have a broader vocabulary. A solver may reason about equality, linear arithmetic, arrays, quantified formulas, and other theories while also performing propositional search and preprocessing. Alethe extends the SMT-LIB term language and permits proof rules at different levels of detail. Coarse steps ease proof production but require a checker or elaborator to fill larger gaps. Fine steps enlarge the proof and producer burden while allowing a simpler reconstruction step. The format must also state which theories and rules a checker actually supports.(Schurr et al. 2021)

Table 14.2: Proof formats make different engineering trades. A supported format does not imply that every solver feature or theory has a checked route.

Format	Producer supplies	Checker supplies	Principal boundary
DRAT	additions and optional deletions	redundancy discovery and final contradiction	exact CNF and supported DRAT semantics
LRAT	additions, deletions, and ordered hints	direct hinted validation	hint production may be untrusted; LRAT checker remains trusted
Alethe	SMT terms and coarse or fine rule steps	theory checks and possible elaboration	supported rule and theory vocabulary

The table is not a ladder. A short DRAT proof checked by one implementation, a larger LRAT proof checked inside a theorem prover, and an Alethe proof reconstructed into a kernel theorem answer different trust and language questions. Proof size, production cost, checker size, theory coverage, and reconstruction footprint are separate coordinates.

Nor is conversion transitive by reputation. If formula F produces DRAT proof D , a converter emits LRAT proof L , and a certified checker accepts L , the checked conclusion concerns the formula bytes read with L . The manifest must still bind those bytes to F , and F to the semantic claim. The accepted LRAT proof can remove the DRAT solver and converter from the trusted base. It cannot repair an unbound or mistranslated formula.

14.2 Six forms of support

The book uses six **trust classes**: definition, trace, computation, verdict, certificate, and kernel reconstruction. Object `EVID.def.trust-class` records these names. They describe how a result is supported and where trust remains. They are not medals and do not form one simple ranking for every question.

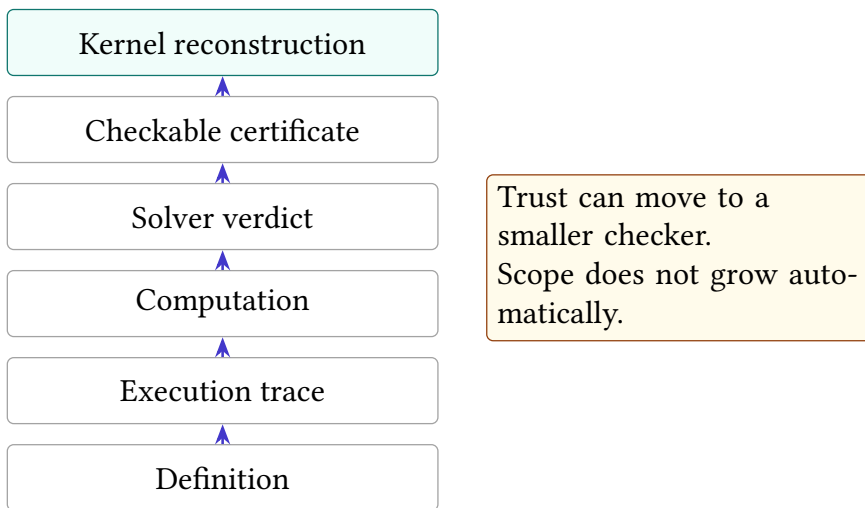


Figure 14.1: Six trust classes used by the book. Moving upward can reduce or relocate trust, but no rung repairs a bad statement, semantic model, or scope.

14.2.1 Definition

A definition fixes meaning. It may introduce a word, machine state, decoder, observation, cost, or relation. Definitions need internal consistency and clear dependencies, but they do not become empirical facts by being executed. The Ao word set and state tuple are examples. A checker can confirm that their records satisfy a schema; it cannot discover whether the author chose the most useful teaching machine.

Definitions still carry a trust boundary. If a later formula implements a different addition rule, the formula does not prove a statement about the defined Ao instruction. Versioning and semantic digests help expose that mismatch. They cannot decide whether the prose and formal definition express the same intended idea. The reader remains part of the route.

14.2.2 Trace

A trace replays a particular execution. Given initial state, program bytes, and steps, a checker can decode each instruction and confirm the next state. This can strongly support an example or a counterexample. It does not cover inputs absent from the trace.

Trace evidence is often the right support for an existence claim. One failing input refutes universal equivalence. One valid two-instruction sequence establishes an upper bound. In both cases, a small concrete object carries the claim. The checker should still compare the complete observed state, reject an illegal decode, and fail if any step changes.

14.2.3 Computation

A complete finite computation covers a declared domain by enumeration or a checked recurrence. At width four, evaluating a function on all sixteen words is complete for that width. A breadth-first search that visits every state in a finite graph can establish reachability or its absence, provided the transition generator and coverage test are correct.

Computation differs from testing by coverage, not by how many cases happened to run. Ten million random tests remain samples. Sixteen cases are complete when the domain contains exactly sixteen. A durable computation records the domain, enumeration order or invariant, program revision, result digest, and the condition that makes the runner fail.

14.2.4 Verdict

A solver verdict reports the result of a symbolic decision procedure. A satisfying verdict should carry a model that can be decoded and replayed. An unsatisfiable verdict may be reproducible by running the same or another solver, but without a portable proof it still asks us to trust the solver and translation route.

Agreement between two independent solvers is useful. It catches many ordinary implementation errors. It is not the same as checking a certificate, because the solvers may share an encoding bug, a library, or an unsupported-theory interpretation. Report solver agreement as corroboration, not as a changed kind of object.

14.2.5 Certificate

A **certificate** is a finite object whose checker can validate the claimed result without repeating the original search. A satisfying model is a positive certificate when replay checks it. A DRAT or LRAT refutation can certify that a propositional formula has no satisfying assignment. An Alethe proof can record supported SMT reasoning. A rational equality or inequality certificate can be checked with exact arithmetic.

Certificates shift trust from the producer to the checker, but only for the statement the certificate checker receives. The checker must parse the same formula bytes, apply sound rules, and require the claimed conclusion. A proof format is not magic dust sprinkled on a solver log.

14.2.6 Kernel reconstruction

Kernel reconstruction translates the result into a term checked by a small logical kernel. This can produce a theorem whose dependency footprint is inspectable. The route is strongest when the term contains the mathematical reasoning rather than an axiom that merely repeats the desired conclusion.

Reconstruction can also be structural. A module may import a result as an assumption, attach provenance, and show where it is used. That is useful attestation, but the kernel has not checked the external search. The trusted footprint distinguishes these cases. A kernel label without that measurement is too coarse.

Higher on the diagram does not mean broader

A kernel term for a width-four lemma may have a smaller trusted checker and a narrower mathematical scope than a complete width-64 computation. Trust class and claim scope answer different questions. Print both.

14.3 Binding claims to checkers

An evidence route is a chain. The claim names an ISA slice, precondition, observation, and cost. Semantic code turns instructions into state changes. A generator translates the negated claim into a formula. A solver produces a model or refutation. A checker validates that object. The report must bind the result back to the original claim.

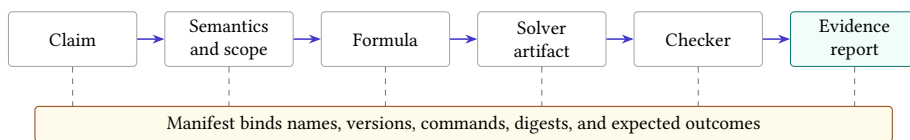


Figure 14.2: A proof checker validates one link near the end. Digests and a manifest bind the checked bytes to the semantics, scope, and claim that give those bytes meaning.

Every arrow admits a characteristic error.

- The prose may describe full architectural state while the observation compares only one register.
- The semantic package may reverse subtraction operands or mishandle a partial-register write.
- The formula generator may omit one instruction form or one cost layer.
- The solver may return a wrong model or verdict.
- The proof checker may accept a malformed step or never require the final contradiction.
- The report may point to stale bytes after a rebuild.

Testing only the final checker cannot catch all six errors. The route needs local controls and end-to-end controls. A semantic mutation should change a trace. A missing instruction should change the candidate-language digest. A damaged proof should fail proof checking. A stale report should fail its hash comparison.

14.4 Evidence manifests

An **evidence manifest** is a small record that binds one claim to the objects and commands used to support it. Object `EVID.def.manifest` records the book's contract. A complete manifest names:

1. the claim identifier and exact scope;
2. semantic package names, versions, and digests;
3. every input, witness, formula, proof, and result file with raw digests;
4. the producer and its configuration;
5. the checker command and expected success condition;
6. the negative control and expected failure class;
7. the trust class and any kernel footprint; and
8. limitations that remain outside the route.

The manifest is not the evidence itself. A manifest file asserting that a proof was checked can lie as easily as prose. Its value is binding: it tells an independent runner which bytes to hash, which command to run, and what outcome must change under the control.

14.4.1 Raw bytes and derived files

Cryptographic hashes identify byte sequences. They do not identify meanings. The book records SHA-256 digests of raw files before packaging because archive metadata, compression headers, or regenerated line endings can change a container without changing its mathematical content. When a compressed file is the distributed object, record both the distributed bytes and the raw payload when practical.

A matching hash establishes identity with the recorded bytes, not correctness. If the original formula encoded the wrong semantics, preserving it perfectly preserves the error. Hashes close the stale-file and substitution gaps. They do not close the semantic gap.

14.4.2 Hashes, signatures, and provenance

A digest is a short value computed from bytes. A **cryptographic hash** is designed so that finding two useful inputs with the same digest is computationally difficult. When the manifest records both the algorithm and digest, a later runner can check whether the bytes changed. The runner still needs a trusted copy of the manifest. An attacker who can replace both the artifact and its unsigned digest can make the pair agree.

A digital signature adds an identity claim. A signer uses a private key to sign selected bytes; a verifier uses the corresponding public key to check the signature. This establishes that a holder of the private key signed those bytes, assuming the signature system and key distribution are sound. It does not establish that the signer understood the bytes, ran the checker, or made a true statement.

Provenance records where an artifact came from and which steps produced it. A useful record can name the input materials, command, tool version, environment, output products, and outcome. Chaining such records connects the product of one step to the material of the next. The chain makes substitution and stale-input errors visible.

The in-toto framework gives an industrial comparison. A signed layout declares the expected supply-chain steps and the authorized parties. Signed link metadata records materials, products, commands, and byproducts for performed steps. Verification checks the chain against the layout. The target files remain opaque to the framework: process integrity does not establish an arbitrary semantic property of their contents.(Torres-Arias et al. 2019)

An evidence manifest has the same chain shape but a different final question. Its layout may say: generate formula from semantic package, solve formula, check proof, and assemble report. Link records bind the input and output bytes at every stage. The proof checker must still determine whether the proof is valid, and the semantic argument must still connect the formula to the book claim.

Table 14.3: Four mechanisms close different gaps. None subsumes all the others.

Mechanism	Establishes	Does not establish
digest	equality with recorded bytes	correctness, origin, or availability
signature	selected key signed selected bytes	truth or competent review
provenance chain	declared steps connect materials to products	semantic correctness of each step
certificate check	artifact is valid for the formula read	formula represents the intended theorem

Canonical serialization matters whenever structured data is hashed or signed. Two JSON files can represent the same key-value map while ordering keys or spacing characters differently. Hashing raw bytes treats them as different. A

canonical form fixes one byte representation for the structured value. The manifest must say whether its digest binds raw transport bytes, canonical structured bytes, or both.

Canonicalization creates another trusted step. If the canonicalizer drops an unknown field, two records with different meanings may acquire the same canonical bytes. A safer design parses into a versioned typed structure, rejects unknown required fields, serializes deterministically, and tests a round trip. The raw distributed digest remains useful because it exposes any change before parsing.

Availability is independent again. A digest can identify a proof that no repository retains. A signature can authenticate an inaccessible object. Long-lived evidence needs an archival location, stable identifiers, and enough license and build information for another person to obtain and check it. The manifest should treat a known digest with missing bytes as an unavailable outcome, not a successful identity check.

Integrity is not truth

Hashes protect identity. Signatures bind keys to bytes. Provenance binds declared steps. Certificate checkers validate formal relations. The theorem route needs all relevant links and must state the assumption introduced by each one.

14.4.3 Command outcome contracts

A checker command needs more than a transcript. Its process exit status must depend on the finding. A shell command such as “run checker, then print the previous exit code” is easy to misread when pipelines, timeouts, or later commands overwrite the status. The route should execute one focused checker and treat any unexpected result, timeout, resource exhaustion, or parse error as failure.

Text matching deserves the same care. Searching for the substring `unsat` can accept `not-unsat`, a diagnostic that quotes the input, or two contradictory verdict lines. A wrapper should parse the expected record or count exactly one anchored verdict, then test that count. The checker, not the surrounding prose, must decide success.

Four outcomes, only one success

A lower-bound check may return: the refutation is valid; the formula is satisfiable; the proof is malformed; or the checker exceeded its resource budget. Only the first supports the lower bound. Treating every non-satisfying outcome as equivalent would turn a crash into mathematics.

14.5 Negative controls

A **negative control** is a nearby false claim, damaged object, or altered route that the checker must reject for a named reason. Object `EVID.prin.negative-control` records the rule that every checked route needs one. The label *negative* describes the expected result, not the control's importance.

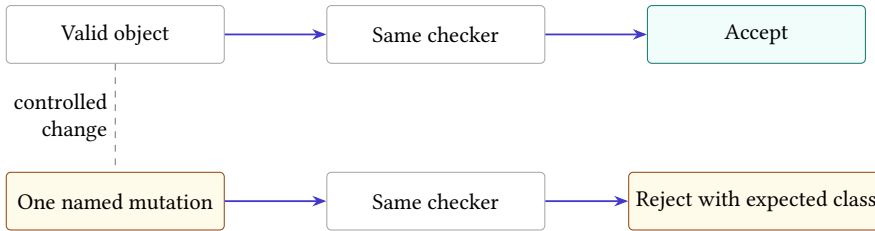


Figure 14.3: The production object and one deliberate mutation travel through the same checker. Acceptance on the left and the named rejection on the right show that the route distinguishes this fault.

A useful control targets a plausible failure at the same layer as the positive claim. For a trace, change one next-state value. For a model, flip one output bit and replay it. For a formula, add a known satisfying witness to a language claimed empty. For a proof, delete a load-bearing step. For a manifest, change one semantic digest. For kernel reconstruction, add one assumed law and require the dependency footprint to contain at least one entry.

The control must share the production path. A hand-written test of a toy parser says little about the command that checks the published artifact. Use the same decoder, semantics, formula reader, proof checker, and wrapper wherever the fault is meant to travel.

14.5.1 Expected failure classes

“It failed” is not precise enough. Suppose a damaged proof also has a wrong hash. If the manifest rejects the hash before the proof checker runs, the test has not shown that proof checking can reject the damage. Both checks may be valuable, but they answer different questions.

Name the expected failure class: digest mismatch, parse rejection, semantic counterexample, illegal decode, unsupported proof rule, missing contradiction, resource exhaustion, or nonempty kernel footprint. Then assert that the control reached that boundary. This prevents a broken setup from impersonating a successful control.

A negative control never supports the main claim

Correct rejection shows that the route detects one selected fault. It does not show that the positive object is correct, that every fault is detectable, or that the theorem's scope is right. Keep the control beside the evidence, but do not count it as a second proof.

14.5.2 Positive controls

Some checkers reject everything. A negative control alone would make such a checker look excellent. Run the production object or a known valid miniature through the same path and require acceptance. The pair establishes one small discrimination: this valid instance crosses the boundary and this invalid neighbor does not.

When testing a resource limit, include a case that completes inside the budget. When testing an unsupported proof rule, include a supported rule that passes. When testing a semantic mutation, replay an unmutated instruction as well. Controls work in pairs because a useful checker must distinguish, not merely approve or object.

14.5.3 Mutation and metamorphic controls

A hand-written negative control changes one object. **Mutation testing** systematizes that idea by defining many small changes, called mutations, and requiring the test route to kill them. A mutation is killed when it changes the expected outcome in the declared way. A surviving mutation may expose a test gap, a dead condition, or a change that is actually irrelevant to the claim.

Useful checker mutations include accepting an unknown parent, skipping a literal, reversing a pivot sign, treating end-of-file as contradiction, collapsing timeout into rejection, or ignoring a digest field. Each mutation targets one guard. The test harness should prove that it built and ran the mutated checker before reporting a kill. A compilation failure says the mutant did not build; it does not show that a test detected wrong behavior.

Mutation testing can also change artifacts rather than code. Delete one proof step, alter one clause identifier, change one semantic digest, truncate one formula, or add a legal candidate to a supposedly exhausted language. Artifact mutations are easier for readers to reproduce. Code mutations reach guards whose absence may still accept every small published artifact.

A **metamorphic test** starts from a transformation with a predicted relation between outcomes. It is useful when the exact output is difficult to know in advance. Permuting clauses in a CNF formula should not change satisfiability. Renaming variables bijectively should preserve it. Adding a duplicate clause should preserve

it. Adding a new contradictory unit pair should make the formula unsatisfiable, though it may also make a long proof vacuous.

The prediction must match the evidence layer. Clause permutation preserves the mathematical formula but can invalidate a proof format whose hints use line identifiers. The correct metamorphic expectation may be: verdict unchanged, old proof rejected, regenerated proof accepted. Treating every byte-level transformation as proof preserving would confuse formula semantics with certificate syntax.

Table 14.4: Controls should attack every evidence link and name the expected boundary.

Link	Mutation	Required observation
claim to semantics	include one additional observed flag	claim or semantic digest changes
semantics to formula	reverse subtraction operands	tiny model cross-check disagrees
formula to proof	change one input clause	proof check rejects or proof digest is stale
proof to checker	delete a required derivation	invalid-step or missing-contradiction result
checker to report	force checker exit 137	resource-exhausted report, never verified
manifest binding	change one artifact digest	identity check rejects before execution
kernel route	insert one assumption	dependency footprint becomes nonempty

The matrix prevents one successful mutation from standing in for an entire route. A proof-byte mutation says nothing about formula completeness. A semantic mutation that is caught by a hash mismatch says nothing about the interpreter. Each control needs an expected boundary and evidence that execution reached it.

Why a surviving mutation needs interpretation

Let route R accept valid object x , and let mutation m produce $m(x)$. If the route also accepts $m(x)$, two explanations are possible. The mutation may preserve every property that R is meant to check, or the route may fail to discriminate a relevant change. Decide which by restating the checked relation. If x and $m(x)$ are equal under that relation, survival is expected. If not, the survivor is evidence of a missing or dead guard.

Coverage counts matter. A report should distinguish mutants generated, mutants that built, mutants that ran, mutants killed at the intended boundary, mutants killed elsewhere, survivors, and ambiguous cases. Reporting only a percentage can

reward a harness that fails to compile most mutations. The denominator is part of the claim.

The economics are favorable for small trusted components. A solver may be too large for systematic mutation, but a parser, proof checker, manifest validator, and outcome wrapper have narrow contracts. Mutating them can reveal acceptance paths that ordinary positive examples never exercise. Retaining a small mutation suite also protects future refactoring: a removed guard should make a specific control survive.

A high mutation score is not a soundness theorem

Mutation operators sample plausible faults. They do not enumerate every wrong implementation. A perfect score supports the sensitivity of named guards; it does not prove that the checker implements the mathematical relation.

14.6 Checker design

A checker is easiest to trust when its job can be stated as a short transition system. Its input is a formula and a sequence of certificate steps. Its state contains only the facts needed to judge the next step. Each accepted step must preserve a named invariant. The final state must contain the particular fact that closes the claim. This description is more useful than saying that the checker is “simple.” A thousand lines can implement a small relation badly; ten thousand lines can implement it with unusually clear boundaries.

For a clause proof, a **literal** is a Boolean variable or its negation. One useful invariant is that every clause in the active database is either an original clause or has passed the format’s admissibility test. Deletion may remove a clause from the active database, but it must not invent a new one. Parsing must reject malformed literals and out-of-range identifiers. The accepting state must contain the empty clause, not merely an end-of-file marker or a line containing a familiar word. These requirements turn implementation review into questions with yes-or-no answers.

The same pattern applies beyond propositional logic. An arithmetic certificate checker may accumulate polynomial identities or inequalities. A program trace checker may update architectural state one instruction at a time. A kernel reconstruction route may translate each external step into a term whose type states the local obligation. In every case, ask four questions:

1. What is the checker’s state?
2. Which invariant must every accepted transition preserve?
3. Which malformed or unsupported inputs cause rejection?
4. What exact final state licenses the reported conclusion?

These questions also expose dangerous convenience behavior. A parser that silently skips an unknown instruction has changed the certificate language. A checker that replaces an unsupported theory step with a solver call has moved the trust boundary. A wrapper that catches every exception and returns success has erased the distinction between evidence and failure. Fail-closed behavior means that absence, ambiguity, and unsupported cases cannot cross the acceptance boundary.

The accepting state is part of the theorem

Do not define success as “the checker finished.” Define the exact state that the checker must reach, and make every other termination mode distinct.

There is a useful engineering consequence. Keep parsing, transition checking, and reporting separate. The parser produces a typed object or a precise parse error. The transition checker consumes only typed objects. The reporter serializes the outcome without deciding it. This division does not establish implementation correctness, but it makes accidental acceptance paths easier to find and makes negative controls easier to aim.

14.6.1 Hand audit before automation

Before trusting a new checker on a large artifact, construct the smallest example that exercises each rule. Write the expected state after every step. Then alter one premise, one identifier, and the final step in separate copies. The valid example should pass; each altered copy should fail for the predicted reason. This hand audit is the certificate analogue of executing a short instruction trace before running a whole program.

Large benchmarks test scale. They rarely isolate meaning. A million-step proof can pass even when a rarely used rule is wrong, and its size makes diagnosis hard. Tiny rule-directed examples supply semantic resolution; large examples supply operational stress. A serious evidence route needs both.

14.7 A vacuous proof file

An earlier evidence-led version of this repository contained a DRAT file of 957,982 bytes for a one-step bounded search. Axeyum’s checker accepted it. The same checker also accepted the file after its first half was deleted. The checker was not unsound. The underlying formula already contradicted itself under unit propagation, so the last empty-clause step was enough. Everything before it was decoration.

To see the mechanism, begin with a tiny conjunctive normal form formula, a conjunction of clauses in which each clause is a disjunction of literals:

$$F = (a) \wedge (\neg a).$$

The first unit clause forces a to true. The second forces it to false. **Unit propagation** repeatedly applies forced single-literal clauses; here it reaches conflict without any search or added lemma.

One common DRAT check asks whether an added clause has the reverse unit propagation property: temporarily negate that clause, add those negated literals as units, and see whether propagation reaches conflict. For the empty clause there are no literals to negate. If the original formula already conflicts under propagation, the empty clause passes immediately.

Reader argument: why the prefix is irrelevant

Assume a conjunctive normal form formula F reaches conflict by unit propagation alone. Consider any candidate DRAT log whose final addition is the empty clause. A backward checker tests whether that empty clause follows by reverse unit propagation. Negating the empty clause adds no assumptions, so the check runs unit propagation on F itself. By the premise, that run conflicts. The final step is therefore accepted without using any earlier proof step. Delete or replace the prefix and the same reason remains.

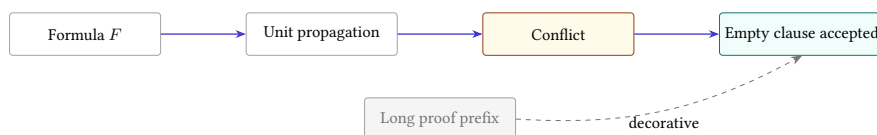


Figure 14.4: When the original formula already conflicts under unit propagation, the final empty clause is accepted without support from the preceding proof steps. The formula, not the long prefix, carries the refutation.

The archived formula contained 2,663 variables, not merely the two clauses above. The structural fact was the same. A small repository script, `scripts/up_refutes.py`, checks whether a standard CNF formula reaches a unit conflict before a proof file is credited as load-bearing. The archived control used another formula that reached a propagation fixpoint without conflict; its actual refutation was then needed and a truncated proof was rejected.

The lesson is subtler than “large proofs can be fake.” The accepted proof file was syntactically valid, and the formula really was unsatisfiable. The mistake was attribution. The repository said the megabyte proof supported the claim when the original clauses already contained a cheaply checkable contradiction. The better evidence route records the unit-propagation check and says that the proof prefix contributes nothing.

Perform the vacuity check by hand

Run unit propagation on $F_1 = (a) \wedge (\neg a \vee b) \wedge (\neg b)$. Then run it on $F_2 = (a \vee b) \wedge (\neg a \vee b) \wedge (a \vee \neg b)$. Which formula reaches conflict without a decision? For the other, give a satisfying assignment or identify the first decision needed.

14.8 Formula generation

A checked refutation establishes that one concrete formula is unsatisfiable. For a program claim, we also need to know what the variables and clauses mean. The generator must cover every allowed candidate, implement the instruction semantics, negate the desired equivalence correctly, and encode the chosen cost bound.

A useful translation argument has two directions. **Soundness of the encoding** says that any satisfying formula assignment decodes to a real counterexample or candidate in the declared language. **Completeness of the encoding** says that every real counterexample or candidate in that language can be represented by a satisfying assignment. For a lower bound, both directions matter. Soundness prevents spurious models; completeness prevents the formula from silently omitting the cheaper program that defeats the claim.

The reader proof can use a miniature. At Ao width two and program length one, enumerate the instruction selectors and show how each selected operation determines the next-state bits. Decode a satisfying assignment back to one instruction and one separating input. Then explain how the production generator repeats the same construction at the target width and bound.

Cross-checks help at this bridge. Generate tiny instances and compare the formula's models with direct enumeration. Mutate one instruction semantic rule and require disagreement. Count the candidate forms produced by the grammar and compare that count with an independent expansion. These checks do not replace a general translation theorem, but they expose the exact surface the theorem must cover.

14.9 Independent checking

Calling a checker independent requires a concrete account. A checker in the same binary may still use different algorithms and parsers. A separate binary may share the same library that contains the bug. Two tools may implement one format from separate specifications. Independence has dimensions: codebase, algorithm, parser, runtime, proof representation, semantic implementation, and institutional origin.

State which dimensions differ. Axeyum's proof-producing SAT core and its DRAT checker are separate implementations in the same repository. Its in-memory,

streaming, and file-backed checking paths can be compared, but shared clause types and rule definitions remain part of the trust story. LRAT can move the result to a simpler hint-following checker. Alethe output can be checked by an external implementation only for the rules that implementation supports. Unsupported rules must remain visible refusals, not silently trusted steps.

Agreement can still be diagnostically valuable

Suppose direct enumeration, a bit-vector solver, and a SAT encoding all return the same width-four counterexample. Their agreement does not create a formal certificate. It does sharply localize a later disagreement at width 64: the shared mathematical specification is less suspect than the widened encoding or solver route.

14.10 Kernel reconstruction

A kernel check is only as informative as the term and environment supplied to the kernel. If reconstruction creates an axiom named after the desired result and then returns that axiom, the kernel has confirmed type consistency, not the external reasoning. The dependency footprint should list every assumption reached by the final term.

This caution follows from the basic architecture of an interactive theorem prover. A large front end may parse notation, search for a proof, call automation, and elaborate omitted arguments. A much smaller kernel checks the resulting term against a type theory. The mathematical proposition appears as a type; a proof appears as a term of that type. The kernel need not trust the search procedure because a bad procedure should fail to produce a term that type-checks. This separation is often called the **de Bruijn criterion**: proofs generated by complex or untrusted machinery are checked by a small, independent proof checker.

The criterion reduces one kind of trust. It does not decide what the theorem means, choose sound primitive axioms, verify the compiler that built the kernel, or prove that the displayed source corresponds to the checked term. Those boundaries should be named rather than folded into the word “formalized.” A practical trust graph contains at least the source statement, elaborated declaration, imported definitions and axioms, kernel, runtime, and the report that extracts the result.

14.10.1 Definitions, theorems, and assumptions

A definition introduces a name with a body. The kernel can unfold that body when checking later terms. A theorem introduces a name with a proof term. The kernel can inspect the term’s type and dependencies. An axiom introduces a name and type without a term that derives it. All three may be convenient to use in later prose, but only the last adds an unproved premise.

This difference makes an assumption footprint useful. Start at the final declaration, follow its dependency graph, and collect primitive axioms and any opaque external facts treated as axioms. A footprint is more precise than counting the word “axiom” in source files. An unused experimental axiom does not weaken a theorem that cannot reach it. One deeply imported axiom does matter even if it appears in a distant module.

Not every listed assumption is a defect. Classical choice, quotient principles, or a trusted semantic postulate may be intentional. The report’s job is first to expose the dependency. Mathematical review can then ask whether the assumption is standard, avoidable, or exactly the proposition the artifact was supposed to establish. An axiom whose name restates the desired optimization theorem is circular. A foundational axiom explicitly accepted by the logic is a different kind of commitment.

Table 14.5: A kernel result must be read together with the route that produced and reported it.

Layer	Useful check	Residual question
source statement	review notation, scope, and quantifiers	is this the intended claim?
elaboration	inspect generated declaration and imports	did notation elaborate as intended?
proof term	kernel type-checking	which definitions and axioms are reachable?
kernel and logic	small implementation and metatheory	are primitives and implementation trusted?
extraction or report	compare names, hashes, and footprints	did the report describe the checked object?

14.10.2 Reflection

Proof by reflection replaces a long, manually elaborated derivation with a verified computation. First define a data representation of formulas, programs, or certificates inside the logic. Then define a checker over that representation. Prove once that whenever the checker returns acceptance, the interpreted proposition follows. For a concrete artifact, the kernel evaluates the checker and applies the soundness theorem.

The attraction is scale. A million elementary certificate steps need not become a million hand-written tactic commands. The proof term can refer to the checker, its soundness theorem, and the artifact. The checker remains ordinary defined code whose behavior is connected to logic by proof. Efficient native evaluation may add an execution boundary, so the route must say whether the kernel reduced the computation, trusted a compiled evaluator, or checked a compact result returned by one.

Reflection also makes representation part of the theorem. If an external DRAT file is parsed into an internal list of steps, soundness of the internal checker does not by itself prove soundness of the parser. One route verifies the parser. Another keeps the parser outside the trusted base but reserializes and compares canonical bytes. A third emits an explicit internal term and lets elaboration reject malformed structure. Each route pays a different cost and leaves a different boundary.

The formula bridge is equally important. Suppose a reflected checker proves that a CNF object is unsatisfiable. The book wants a statement about machine programs. A separate theorem must say that any cheaper program satisfying the declared observation would yield a satisfying assignment of that CNF. Without this theorem, reflection has proved a precise fact about a data structure, not the advertised lower bound.

Two established teaching routes clarify the choice made here. *Software Foundations* presents its logical development as Coq scripts: definitions, theorems, proofs, and many exercises inhabit the proof assistant from the start. *Concrete Semantics* pairs sustained textbook explanation with executable Isabelle theories and a large exercise program. (Pierce et al. 2026; Nipkow and Klein 2014) Both make small formal objects available for readers to change and recheck.

This chapter adds an external-artifact boundary because industrial solvers often emit files before a proof assistant sees them. The reader must therefore inspect parsers, raw-byte identity, resource outcomes, and converter trust in addition to the internal proof. The approaches are complementary. Proof-script pedagogy makes the logic directly editable; artifact pedagogy makes the handoffs and failure modes directly visible. A complete book route should let the same small example travel through both.

Theory reconstruction can be stronger. A linear-arithmetic certificate may be translated into exact rational equalities and order steps, then checked from the kernel's arithmetic laws. A propositional LRAT proof may be replayed by a verified checker whose soundness theorem lives in the prover. The resulting term can have a small, measured trusted surface.

Search-based program minimality poses a further composition problem. The kernel needs the final propositional contradiction and a link from decoded machine programs to the generated clauses. Importing the SAT result as an assumption does not create that link. A future Ao route may use proof by reflection: define a small checker in the logic, run it on the serialized language and certificate, and apply a kernel-checked soundness theorem. Until that route exists, the manifest must distinguish checked external evidence from kernel reconstruction with mathematical content.

There is no contradiction in publishing both. An external certificate route may scale first and permit independent checking today. A smaller kernel route may later reduce the trusted base for a subset of examples. The manifest should record them as two evidence paths that meet at an explicitly stated claim, not as upgrades that erase the history or limitations of the earlier path.

14.11 Production and retention costs

Evidence is not free exhaust from a solver run. A producer spends processor time to log proof steps, storage bandwidth to write them, disk capacity to retain them, and engineering effort to keep the route working. A checker spends time and memory again. An archive pays for durable storage, migration, and discoverability. These costs help determine which evidence route is practical.

Let S be solver time without proof logging, L the added logging time, C checker time, A archive and transfer cost, and M maintenance effort. The direct evidence cost is

$$E = S + L + C + A + M,$$

after every term has been converted to a common unit. This expression does not say whether the evidence is worth producing. It makes the inputs visible so they can be compared with the value of independent audit, faster incident diagnosis, regulatory assurance, or a publishable lower bound.

Proof size affects several terms at once. A larger log takes longer to write and transmit. It may exceed local disk or continuous-integration limits. It may require a streaming checker because the whole file cannot fit in memory. Trimming can reduce retained size while adding another transformation step. The archive should retain enough information to connect the trimmed proof to the original formula and checker result.

Hints move costs between parties. A producer or converter that supplies LRAT hints performs work that a DRAT checker would otherwise discover. The hint-following checker can then be smaller and easier to certify. This trade may be attractive when many independent users check one published result: conversion cost is paid once, while simpler checking is repeated.

The economic asymmetry resembles compilation. One expensive proof-production run can be amortized across many readers, release gates, or downstream products. A private one-off optimization may need only witness replay and regression tests. A public claim of minimality or safety has a longer audience and greater cost of error, so retaining a portable refutation can be rational even when rerunning the solver is cheap today.

Independent implementation is another investment. Writing a second checker from the same precise format specification costs more than invoking the first binary twice. It can remove shared parser and algorithm defects. Reconstruction inside a proof assistant costs still more when theory steps need elaboration, but it may reduce the final trusted code and expose an assumption footprint. The correct comparison names which trust is removed for the added effort.

Evidence also changes incident response. Suppose a later solver release returns a different verdict. A retained formula and proof let maintainers ask separate questions: does the old proof still check, did formula generation change, or did only search behavior change? Without the artifacts, the team may have to reconstruct

an old environment before it can localize the difference. Diagnostic value is part of the return on retention.

There are negative externalities. Automatically logging every proof can fill a shared filesystem and evict other work. Large uploads consume network and review capacity. A checker job that receives no resource budget can delay an entire build queue. Production policy should classify claims and allocate evidence accordingly rather than enabling the strongest route indiscriminately.

Table 14.6: Evidence policies can match claim value and expected reuse. These are engineering defaults, not logical rankings.

Setting		Plausible retained support	sup- port	Reason
local heuristic		witness and focused regression		short lifetime and low reuse
release optimization		formula, witness, checker result, controls		repeated deployment and rollback needs
published bound	lower	manifest, formula, portable proof, checker, controls		independent long-lived audit
safety-critical claim		independently checked proof plus measured assumptions		high failure consequence and review value

The policy must remain fail closed. A high evidence cost can justify reporting that no certificate was produced. It cannot justify relabeling a verdict as a certificate. Budget and trust class are separate fields.

Evidence is an engineered product

Proof production, checking, transfer, storage, maintenance, and future replay form one lifecycle. A durable route chooses these costs in proportion to the claim and records any weaker stopping point honestly.

14.12 Resource failures

Proof checking consumes time, memory, file descriptors, and disk space. A certificate can be much larger than the formula and more expensive to check than the original search. If the operating system kills the checker, no logical verdict has been produced. Treating absence of a counterexample as a lower bound would confuse resource exhaustion with contradiction.

The checker contract should distinguish verified, rejected, unsupported, timed out, and resource exhausted outcomes. Only the first supports the positive evidence row. Record budgets and peak use when they determine whether another

person can reproduce the check. For large proof streams, record whether the checker retained the whole proof, traversed a backward core, or used a file-backed representation.

Resource controls can fail too. A timeout wrapper must deliver termination to the actual checker process and propagate its outcome. A disk-space preflight must inspect the filesystem that will hold the proof. A memory estimate is not a measurement. The workbench is part of the epistemology when its limits can change which answers survive.

Evidence also has a maintenance cost. A certificate format may remain stable while its checker, build system, or runtime disappears. Keep the smallest portable artifacts that preserve the route: plain formula and certificate files, a machine-readable manifest, checker source or a durable source reference, and tiny controls. Containers can make reproduction convenient, but they should package this material rather than become its only description.

When a route is rerun years later, preserve the old report beside the new one. Do not silently replace digests or normalize away a changed outcome. The difference may reveal a repaired checker, a changed resource limit, or an unstable dependency. Evidence is a record of a particular discrimination at a particular boundary; its history can itself become diagnostic evidence.

14.12.1 Reproduction as a second experiment

The vocabulary needs care because different communities have used the same words differently. The ACM artifact policy distinguishes three operational relations. **Repeatability** means that the same team obtains a consistent measurement with the same setup. **Reproducibility** means that a different team obtains the result using the same experimental setup and author artifacts. **Replicability** means that a different team obtains the result with an independently developed setup and artifacts. (Association for Computing Machinery 2020)

These relations change different dependencies. Repeatability catches unstable runs and forgotten local state. Reproducibility tests whether another team can use the supplied package. Replicability tests whether the written specification contains enough information for an independent implementation and whether the result survives that implementation. None automatically proves that the original research question was well chosen.

Artifact properties are separate again. The ACM policy distinguishes an artifact that was evaluated, one that is available in a durable repository, and a result that another team validated. An available artifact may never have been run by a reviewer. A functional evaluated artifact may be unavailable to the public. A reusable artifact exceeds the functional documentation and structure threshold. Badges should therefore be read as specific review outcomes, not as one general seal.

A certificate can be reproducible without an expensive experiment. Another team hashes the formula and proof, builds the checker, and obtains the same

Table 14.7: Repetition, artifact review, and logical checking answer different questions.

Activity	What changes	Strongest direct conclusion
repeat run	time, perhaps nondeterministic choices	same team and setup can obtain a compatible result
reproduce artifact	team changes; author package remains	another team can operate the supplied route
independent replication	team and implementation change	result survives a separately built route
artifact evaluation	reviewer exercises declared package	package meets stated review criteria
certificate check	proof artifact is checked against formula	formal relation holds for exact checked inputs

accepted relation. Independent replication is stronger when that team writes a separate checker from the format specification. Yet both teams may still consume the same faulty formula generator. The evidence report should show which components were reused.

Nondeterminism needs a declared relation rather than byte equality. A solver may choose different learned clauses or emit a different valid proof on each run. Requiring identical proof bytes would reject legitimate variation. Instead require that every produced proof checks against the same formula and that the resulting trust and scope records agree. For a deterministic canonicalizer, byte equality may be the right contract.

Performance reproduction needs tolerances. Checker wall time varies with processor, operating system, cache state, and concurrent load. A claim that a proof checks in exactly 3.000 seconds is brittle. A claim that it checks within a declared resource budget on a named reference setup can be tested. Logical acceptance should remain exact even when performance measurements admit variation.

Reproduction should begin from the manifest, not from the author's shell history. The person repeating the experiment obtains the named inputs and checks their raw digests. That person then builds or identifies the checker version, runs the declared command, and compares the structured outcome with the contract. Only then should the person run the negative control. This order separates an identity failure from a checking failure and both from a control failure.

Environment capture needs judgment. Recording every installed package can bury the dependencies that matter. Recording none can make the result unrepeatable. Prefer a narrow executable description: operating-system and processor assumptions when relevant, compiler or interpreter version, locked dependencies, build command, checker digest, limits, and the exact input digests. If nondeterminism remains, record the seed and say which outputs may vary.

Agreement after reproduction strengthens the operational claim: another run with the bound inputs reached the same outcome. It does not repair a wrong

formula or broaden a finite theorem. Reproduction repeats the route; independent validation changes some part of it. The evidence report should say which one occurred.

14.13 Axeyum route

The live Axeyum checkout contains substantial evidence machinery. This claim is tied to the inspected revision recorded in this chapter’s research log. Its propositional layer can produce and check DRAT, stream proofs, and perform backward, file-backed, and budgeted checking. It also contains an independent LRAT checker and an elaborator from supported DRAT steps. That LRAT route is deliberately narrower than the name alone might suggest: it follows positive reverse-unit-propagation hints, and it rejects additions that require the full resolution-asymmetric-*tautology* rule. Rejection here is useful information about route support, not evidence that the source refutation is false.

Axeyum’s solver layer makes another important distinction explicit. An evidence check has three broad results: *emphverified*, *emphnothing to check*, or *emphfailed*. A bare unsatisfiable verdict with no transferable certificate is “nothing to check,” not success. So is an undecided result. A present certificate that does not replay is a failure. This three-way type prevents an empty or absent artifact from becoming a green Boolean by accident.

An evidence report joins the evidence to provenance and to a per-result trust ledger. The ledger lists reductions on which that particular result depended and whether the run certified each one. This is stronger than a page that says, in general, “the solver is verified.” Different queries may take different routes through bit blasting, arithmetic, finite expansion, or an external adapter. Their assurance therefore differs even when the printed verdict is the same word.

Selected SMT routes carry Alethe proofs. Some recheck bit-blast, Tseitin, and resolution steps; some invoke arithmetic-aware checking; some validate a guarded finite-instantiation rule against the original assertions. Other routes reconstruct arithmetic reasoning or report a kernel dependency footprint. The exact evidence variant and its trust ledger, rather than the Axeyum name, determine what was checked. These capabilities are route-specific; no one command turns an arbitrary ISA statement into a kernel theorem.

This architecture illustrates a general rule. Capability belongs to a route:

$$\begin{array}{l} \text{input fragment} \longrightarrow \text{reduction} \longrightarrow \text{certificate kind} \\ \longrightarrow \text{checker} \longrightarrow \text{reported dependencies} \end{array}$$

Removing any term changes the claim. “LRAT checked” without the supported step class is incomplete. “Alethe checked” without the accepted theory rules is incomplete. “Kernel reconstructed” without the imported assumptions is incomplete. A useful interface makes these qualifiers ordinary output rather than footnotes found only after failure.

For the book, Rust should own evidence meaning. It should define the canonical manifest, hash raw inputs, classify checker outcomes, run controls, and reject unsupported trust claims. A Python interface may make the route pleasant to teach:

```
# Illustrative API.
# The book manifest interface is not yet implemented.
record = EvidenceManifest.load("a0-add-two.json")
record.verify_digests()
record.replay_witness()
record.check_certificate()
record.run_negative_control()
print(record.trust_boundary())
```

Listing 14.1: The intended shape of a future evidence inspection interface

The interface should return typed outcomes rather than booleans that collapse rejection, timeout, and unsupported proof rules. It should display the exact claim, semantic digests, checker version, and failure class. Python may orchestrate and explain; Rust remains the authority for parsing, checking, and canonical serialization.

That division is pedagogical as well as technical. A reader should be able to open a notebook, inspect a manifest, corrupt one clause, and read the typed rejection without learning Rust first. The Python object must not silently reinterpret the Rust result. It may turn an enumeration into a clear sentence, but it must preserve whether the result was verified, absent, unsupported, exhausted, or failed. Convenience belongs above the boundary; evidence meaning belongs below it.

The present book repository is not at that endpoint. Object `OP.evid.manifest-checker` records the missing implementation obligation. The active artifact tree has the schema design but no completed Ao or real-ISA evidence package to run through it. Existing Axeyum solver machinery is a foundation. It does not fill the semantic and manifest gaps automatically.

14.13.1 Reports for the next decision

An evidence report is not merely a receipt. It is an instrument for deciding what to trust, what to rerun, and what to investigate. Its first screen should answer five questions in ordinary language: what exact claim was checked, which subject bytes were used, what outcome occurred, which controls ran, and what remains trusted. Detailed logs and proof statistics can follow. A reader should not need to infer success from the last line of a solver transcript.

Failure reports deserve the same care. A rejected proof should identify the first invalid step when possible. An unsupported rule should name the rule and checker capability. A digest mismatch should show the expected and observed identity without pretending that parsing began. A resource failure should name the exhausted budget. These distinctions guide different repairs: change the artifact,

select another checker route, retrieve the correct bytes, or allocate a justified budget.

Reports must also be stable enough to compare. Give outcome classes and trust identifiers versioned machine-readable names. Keep explanatory prose for people, but do not make downstream gates search that prose for words such as “success.” When a schema changes, preserve the old report and provide an explicit migration. Otherwise a reporting improvement can silently rewrite the historical meaning of an evidence archive.

The best report invites attack. It names a command for the positive case and commands for the controls. It identifies the small files a reader can inspect by hand. It points to the formula generator and semantic pins rather than hiding them behind a badge. This design turns skepticism into a reproducible operation. A reader can ask a sharper question, alter one boundary, and learn from the exact way the route refuses.

For teaching, the report can disclose its structure in stages. Early examples show a witness and replay outcome. Later examples add formula identity, certificate checking, controls, provenance, and dependency footprints. The data model remains cumulative, so the apparent simplicity of the first example does not require a different meaning of “verified.” The student’s view grows while the underlying contract stays exact.

This presentation also teaches restraint. The interface should never celebrate an unavailable certificate, hide an unsupported rule, or replace a measured assumption with a general claim about the tool. Clear reporting is part of the proof route because readers act on what the report says.

14.14 Where the model stops

This chapter does not claim that a negative control proves a checker sound. It does not claim that a certificate validates its formula generator. It does not treat matching hashes as semantic correctness, solver agreement as a proof, or kernel import as reconstruction.

The archived vacuous-proof case concerns one former formula and checker route. It does not impeach DRAT. In fact, the checker behaved according to the proof rule; the repository’s interpretation was too generous. The repair is better classification and a propagation precheck, not distrust of every short proof.

No finite collection of controls anticipates every fault. The aim is narrower and more useful: bind the claim to exact bytes, make each checker outcome depend on the finding, attack the likely failure boundaries, and state what remains trusted. A machine-checked result is not the end of human judgment. It is human judgment arranged so that more of its mistakes become visible.

14.15 Exercises

1. **Classify.** For each of the following, name the trust class and nearest remaining trust boundary: five random tests, all sixteen width-four inputs, one satisfying model, one solver verdict, one checked DRAT file, and a kernel term with a nonempty assumption footprint.
2. **Execute.** Replay a three-step Ao trace. Change only the final program counter and state the exact failure a complete trace checker should report.
3. Give a satisfying assignment for $(a \vee b) \wedge (\neg a \vee c) \wedge (\neg c)$. Explain why replaying the assignment is easier than supporting an unsatisfiable verdict.
4. Run unit propagation by hand on $(a) \wedge (\neg a \vee b) \wedge (\neg b \vee c) \wedge (\neg c)$. List the forced literals in order and identify the conflicting clause.
5. **Prove.** Generalize the reader argument about a propagation- refutable formula and a final empty clause. State every premise the conclusion needs about the backward checker.
6. Construct a formula that is unsatisfiable but does not conflict under unit propagation from the empty assignment. Explain why an actual proof or search is needed.
7. **Break.** Remove one load-bearing clause from a tiny refutation. Distinguish parse failure, invalid proof step, and failure to derive the empty clause.
8. Design a negative control for a model replayer that accidentally ignores the most significant output bit. Give a positive model and its one-bit mutation.
9. Design a control for an Ao trace checker that updates registers correctly but ignores instruction decoding. Ensure the control reaches the decoder boundary rather than failing a hash check first.
10. Explain why a proof of a byte-identical formula can still fail to support the intended theorem when the semantic package digest changes.
11. Write the minimal fields of an evidence manifest for the two-instruction Ao witness from Chapter 13. Include scope, semantics, observation, cost, witness, checker, and one control.
12. Two solvers return the same verdict, but both consume a formula from one generator. Draw the shared trust boundary and name one check that adds genuine independence.
13. A checker returns code 137 after using all available memory. Explain why the outcome supports neither satisfiability nor unsatisfiability. Specify how the manifest should record it.

14. **Act as the checker.** Review a command that pipes solver output through a text search. List three ways the pipeline can return success without establishing the intended verdict, then design a stricter wrapper contract.
15. Give separate negative controls for formula generation and proof checking. Explain why corrupting a file hash tests neither semantic layer.
16. State soundness and completeness obligations for translating the tiny Ao one-instruction language into a Boolean formula. Give one bug violating each direction.
17. **Transfer.** For one selected RV64 instruction equivalence, list the source pin, decoder, semantics, formula, model or proof, checker, and negative controls that a complete route would bind.
18. Design the Rust and Python boundary for the future book manifest API. Identify which layer owns canonical bytes, typed outcomes, hash checking, display, and user convenience.
19. **Resolve.** For the four-clause formula in Table 14.1, verify each resolvent by considering both truth values of its pivot. Then explain why the empty clause cannot be satisfied.
20. Replace one parent of the first resolution step with a clause that does not contain the complementary pivot. Should a checker report a parse error, a missing-pivot error, or a false final theorem? Defend the layer at which the error belongs.
21. Add the tautological clause $r \vee \neg r$ to the four-clause formula. Does the old refutation remain valid? Use entailment, not an appeal to solver behavior, to justify the answer.
22. Permute the input clauses and consistently rename their identifiers. State the predicted relation between the old and new outcomes. Is this a mutation control or a metamorphic control?
23. Rename every occurrence of p to r and every occurrence of q to s . Give the corresponding renamed proof. Identify the invariant that makes the transformation safe.
24. A DRAT checker discovers antecedents, while an LRAT checker follows listed hints. Compare producer work, artifact size, checker search, and the effect of a wrong hint. Do not describe either format as universally better.
25. Explain why an untrusted DRAT-to-LRAT elaborator can be acceptable when the final LRAT checker is independently implemented. What must happen when the elaborator emits a bad hint chain?

26. Axiom's inspected LRAT route accepts positive reverse-unit hints but not full RAT additions. Write the correct outcome for a valid DRAT proof whose essential step needs RAT. Why is "false theorem" the wrong label?
27. Alethe may record a theory inference as one coarse step or many detailed steps. Compare portability, checker complexity, proof size, and diagnostic quality for the two choices.
28. Draw a trust graph for a proof produced by one solver, translated by an elaborator, and accepted by a checker extracted from a proof assistant. Mark which components may be untrusted without making acceptance unsound and which inputs still bind the theorem's meaning.
29. **Hash carefully.** Two manifests contain semantically identical JSON objects with different key order and whitespace. Their byte hashes differ. Give two sound policies for handling this fact and one unsound policy.
30. Explain why a valid digital signature on a manifest establishes neither that its theorem is true nor that the signer understood it. What useful claim can the signature establish?
31. A manifest names an artifact by a strong digest, but every public copy has disappeared. Classify what remains established about identity, availability, and reproducibility.
32. Design a canonical encoding for a list of signed 64-bit integers. Specify byte order, length framing, versioning, and rejection of noncanonical representations.
33. Compare the chapter's evidence manifest with an in-toto layout and link record. Name two useful structural similarities and one semantic guarantee that neither provenance system supplies by itself.
34. Construct a control matrix for a formula generator, proof producer, proof parser, checker, and report renderer. Give at least one targeted fault for each component and the expected rejection point.
35. A mutation campaign creates 100 malformed proofs. Ninety-eight are rejected, one times out, and one is accepted. Report the result without turning it into a misleading "98 percent sound" score. What should be investigated first?
36. Give a proof mutation that is syntactically valid but semantically invalid. Then give a byte mutation that should fail before parsing. Explain why both controls are useful.

37. Design a metamorphic test for a satisfying model under a permutation of variable names. State the mapping required between the original and transformed models.
38. Adding a redundant clause can shorten, lengthen, or leave unchanged a solver's proof. What logical outcome must remain invariant? Why should a test avoid requiring byte-identical proofs?
39. A checker and its negative-control generator share the same parser bug. Explain how the apparent independence collapses. Propose a control produced through a different representation.
40. Estimate the evidence cost $E = S + L + C + A + M$ for two routes: a small replayable model and a large unsatisfiability certificate. Define your units and explain which comparison is meaningful even if the totals are approximate.
41. A proof is expensive to produce but cheap to check many times. Find the number of checks after which producing the proof is cheaper than rerunning a solver, using symbolic costs first and then one numerical example.
42. Decide which artifacts to retain for ten years when storage is limited: source, generated formula, witness or proof, checker binary, checker source, build environment, and logs. Rank them and state what capability is lost at each omission.
43. Compare the cost of keeping a large certificate with the expected cost of reconstructing it after an incident. Include the risk that the original solver or build environment is no longer available.
44. Classify three reruns using the ACM terminology: the original team on the original setup, a different team using the author's artifacts, and a different team independently implementing the method. Explain why these relations do not form a simple ladder of logical certainty.
45. A parallel solver is nondeterministic: it always returns unsatisfiable, but its proof order and runtime vary. Write a reproduction criterion that is strict about the theorem and tolerant about irrelevant variation.
46. Explain why reproducing the same wrong result from the same flawed generator is possible. Design an independent validation that changes the shared boundary.
47. Axeyum reports `NothingToCheck` for an uncertified unsatisfiable verdict. Write the user-facing sentence a Python layer should display. It must be clear without weakening the typed result.

48. Compare a global statement, “Axeyum supports Alethe,” with a route statement naming the input fragment, proof rules, checker, and trust ledger. Give an example in which the global statement leads a reader astray.
49. Inspect the future manifest interface in the chapter. Add typed outcomes for unsupported proof rules, resource exhaustion, digest mismatch, and a failed negative control. Which outcomes may a command-line program map to the same nonzero exit code, and which distinctions must the report preserve?
50. **Capstone.** Specify an evidence package for one small theorem from this book. Include canonical subject bytes, semantic pins, claim, artifact, checker, outcome vocabulary, one positive control, two negative controls, a reproduction command, and a candid trust-boundary statement. Exchange packages with another reader and record every ambiguity they find.

PART IV

Synthesis and Boundaries

One Algorithm, Three Machines

A memory region contains words. We want one word that summarizes them: load each word, combine it with an accumulator by exclusive OR, and return the accumulator. The routine is small enough to trace on paper. It is large enough to bring together almost every distinction in this book: words and their readings, byte-addressed memory, effective addresses, loops, observations, calling conventions, cross-machine relations, and evidence that can fail.

The three programs will run on Ao, RV64I, and x86-64. “The same algorithm” will not mean that their registers have the same names or that their traces have the same number of steps. It will mean that each program satisfies one mathematical contract and that explicit relations connect their states to that contract.

One specification, three different movements

We do not prove that the instruction sequences look alike. We prove that each sequence consumes the same logical input region and returns the same XOR reduction, while respecting its own machine and calling contract.

15.1 The reduction problem

A **reduction** combines a sequence into one result. Addition produces a sum. Minimum keeps the least element. Conjunction asks whether every Boolean value is true. Exclusive OR records whether each bit position appeared an odd or even number of times. Reductions appear in checksums, statistics, database aggregation, parallel programs, cryptographic constructions, and the inner loops of scientific codes.

Long before a compiler could translate such a loop automatically, programmers had to decide where the accumulator lived, how a pointer advanced, and how an end condition was represented. Modern processors make the same movement at a larger scale. Vector and parallel reductions combine many elements at once, then merge partial accumulators. Those faster forms raise questions about ordering,

associativity, floating-point rounding, and optional extensions. We begin with the scalar integer case because its complete meaning fits on the page.

Exclusive OR is a particularly clean teaching operation. For words of one fixed width it is associative and commutative:

$$(x \text{ xor } y) \text{ xor } z = x \text{ xor } (y \text{ xor } z), \quad x \text{ xor } y = y \text{ xor } x.$$

Zero is its identity, and every word cancels itself:

$$x \text{ xor } 0 = x, \quad x \text{ xor } x = 0.$$

These laws explain the loop invariant without hiding any overflow convention. XOR acts independently at every bit position and always returns another word of the same width.

This routine is not a cryptographic hash. Reordering the input words leaves the result unchanged, and equal words cancel in pairs. The example is useful because its weakness is visible: it teaches a machine proof, not a security claim.

15.1.1 XOR as two-element addition

The four facts above are not unrelated conveniences. A bit can be read as an element of the two-element field

$$\mathbb{F}_2 = \{0, 1\}.$$

Addition in this field is addition modulo two. Its table is

+	0	1
0	0	1
1	1	0

and this is exactly the truth table for exclusive OR. In particular, $1 + 1 = 0$. A repeated one cancels because two is zero modulo two.

A width- w word is a vector of w such field elements. Word XOR adds these vectors coordinate by coordinate:

$$(x \text{ xor } y)_j = x_j + y_j \pmod{2} \quad (0 \leq j < w).$$

Associativity, commutativity, the zero identity, and self-cancellation follow one bit at a time from field addition. This viewpoint also explains why no carry moves between bit positions. Ordinary word addition couples neighboring positions through carry; XOR leaves every coordinate independent.

The vector-space language gives useful derived laws. XORing a fixed mask twice restores the original word. XORing a collection records the parity of the number of one bits at each coordinate. Any linear map over $\mathbb{F}_2$ commutes with the fold:

$$T(x_0 \text{ xor } \cdots \text{ xor } x_{n-1}) = T(x_0) \text{ xor } \cdots \text{ xor } T(x_{n-1}).$$

The premise that T is linear matters. An arbitrary nonlinear transform need not pass through XOR.

The reduction is a column of parity checks

Place the input words as rows of a bit matrix. The output bit in column j is one exactly when that column contains an odd number of ones. Swapping rows cannot change a column count. Duplicating a row twice changes every affected count by two and therefore changes no output bit.

This matrix picture separates the proved function from possible uses. It is a perfect implementation of column parity. It is a poor fingerprint of row order and multiplicity because those facts are intentionally discarded.

15.1.2 The algebra of a fold

A set A , a binary operation $\star$, and an element e form a **monoid** when the operation is associative and e is a two-sided identity:

$$(x \star y) \star z = x \star (y \star z), \quad e \star x = x = x \star e.$$

Commutativity is not required. Fixed-width words with XOR and zero form a commutative monoid. Strings with concatenation and the empty string form a noncommutative monoid. Both can be folded, but only the first permits arbitrary reordering.

For a list $[x_0, \dots, x_{n-1}]$, define the left fold

$$\begin{aligned} \text{foldl}(\star, e, []) &= e, \\ \text{foldl}(\star, e, x :: xs) &= \text{foldl}(\star, e \star x, xs). \end{aligned}$$

An equivalent right-recursive definition combines the head with the fold of the tail. Associativity and the identity law prove that the two definitions agree for a monoid. Without associativity, parenthesization is observable.

The loop in this chapter is a left fold made explicit as state. Its accumulator is the first argument of the next fold step. Its pointer selects the next list element from memory. Its count says how much of the logical list remains. The loop invariant is therefore not a clever guess: it is the defining equation of the fold split at the current position.

The fold-splitting lemma

Let $u ++ v$ denote list concatenation. For every monoid,

$$\text{fold}(u ++ v) = \text{fold}(u) \star \text{fold}(v).$$

Prove the statement by induction on u . The empty case uses the identity law. In the step case, unfold the first element, apply the induction hypothesis, and

use associativity to move parentheses. For XOR, this lemma says that two independently reduced memory chunks may be combined by one final XOR.

The splitting lemma licenses chunking; commutativity licenses reordering the chunks. A parallel runtime needs to know which freedom it uses. If the operation is associative but not commutative, it may build a balanced tree that preserves left-to-right leaf order. If it is also commutative, workers may finish and combine in a different order without changing the mathematical result.

15.1.3 Work and span

Two cost measures clarify the difference between the printed scalar loop and a reduction tree. **Work** counts all primitive combines. **Span** counts the longest chain of dependent combines under an ideal supply of workers. Reducing n nonempty inputs needs $n - 1$ binary combines, so both the scalar fold and a balanced tree have $\Theta(n)$ work.

Their spans differ. The scalar accumulator creates a chain: combine 0 must finish before combine 1 begins. Its combine span is $n - 1$. A balanced binary tree halves the number of live partial results at each level. Its combine span is $\lceil \log_2 n \rceil$, with partially filled levels when n is not a power of two.

Table 15.1: The algebra permits several schedules with the same XOR result. Cost claims still depend on a machine and data-movement model.

Schedule	Combine work	Combine span	Added obligation
left scalar fold	$n - 1$	$n - 1$	one accumulator chain
balanced tree	$n - 1$	$\lceil \log_2 n \rceil$	tree construction and partial results
k chunks	$n - 1$	about $n/k + \log k$	partition and final combination
distributed tree	$n - 1$ logical combines	topology-dependent	communication, failure, and placement

Span is not elapsed time. A tree can lose to a scalar loop on a small input because worker startup, synchronization, and data movement cost more than the saved dependency depth. A memory-resident XOR loop can become limited by load bandwidth rather than XOR throughput. Work and span reveal available parallelism; they do not price the machine that realizes it.

Guy Blelloch's treatment of prefix sums made this algebra a central parallel building block. A reduction returns the final aggregate. A **scan** returns every prefix aggregate. The latter appears sequential at first, yet a tree can compute it with linear work and logarithmic span under the parallel model. (Blelloch 1990) Our routine asks only for the final XOR, but its invariant already names the prefix values that a scan would return.

15.1.4 Several meanings of reduction

At one core, a compiler may recognize an accumulator dependence and replace several scalar iterations with vector lanes and a horizontal combine. LLVM's loop vectorizer documents XOR, AND, OR, multiplication, and addition as recognized reduction operations and applies a target cost model before choosing a vectorization and unrolling factor. (LLVM Project 2026a) Recognition does not prove that every loop is safe to widen. Alias checks, alignment, trip count, target support, and language rules still matter.

Across processes, Message Passing Interface (MPI) reduction combines values contributed by a group and returns the aggregate to a chosen root. Its contract assumes a user operation is associative and can exploit associativity or commutativity to change the evaluation order. The standard calls out floating-point addition because mathematical associativity does not imply bitwise equality after finite rounding. (Message Passing Interface Forum 2015)

Across a data center, MapReduce adds key grouping, partitioning, scheduling, communication, and recovery around map and reduce functions. Dean and Ghemawat's system let a runtime distribute work across commodity machines and manage failures and data placement. (Dean and Ghemawat 2004) The word "reduce" survives, but the system contract is much larger than one binary operator over one in-memory array.

These scales share an algebraic center and differ in state. The scalar loop tracks a pointer and accumulator. A vector loop adds lane state, masks, and a tail. An MPI collective adds process ranks, messages, and a root. A distributed dataflow adds keys, partitions, retries, and durable intermediate data. The proof grows because the machine grows, not because XOR changes.

That distinction has a historical analogue. Simple parity and longitudinal redundancy checks were attractive because a human or a small circuit could update them while data moved. They detected selected transmission faults with little state. Their algebra also admitted undetected patterns. Stronger cyclic codes and later cryptographic hashes were developed for stronger fault and adversary models. Hamming's foundational treatment makes the central design question explicit: which error patterns a code can detect or correct (Hamming 1950). The lesson is not that XOR is obsolete. It is that an operation earns meaning from its contract: the same reduction can be a useful diagnostic checksum, a reversible mask component, or a dangerously weak authenticator.

15.1.5 From parity bit to reduction tree

Parity is a physical as well as algebraic idea. A circuit can update one bit as data bits pass: leave the state unchanged for zero and toggle it for one. After the stream, the state records whether the number of ones was even or odd. The accumulator loop is the word-wide version of that transducer. It keeps 64 parity states in parallel.

Hamming's 1950 paper placed parity checks inside a larger geometry of code words and error patterns. A single parity relation can detect every odd number of changed bits but misses every even number. Several carefully chosen check relations can locate and correct selected errors. The historical lesson for this chapter is structural: a compact summary is useful only relative to a declared fault model.

Early machine loops made physical order explicit. One accumulator receives one value at a time because memory and arithmetic units present a serial interface. The mathematical fold is broader than any one loop instruction pattern. Once machines gained several functional units, vector lanes, processors, and networked workers, the same associative operation could be scheduled as several partial folds followed by a merge.

That change did not remove the accumulator. It multiplied and arranged accumulators. A vector program keeps lane or vector partials. A shared-memory tree keeps one partial per subtree. A message-passing collective keeps one partial per process or communication stage. A distributed dataflow keeps one partial per key and partition. Every level adds identities for pieces of work and rules for when a partial may be combined.

Blelloch's scan treatment is important because it turns this family into a general algorithmic primitive rather than a collection of special sums. The operator and identity become parameters. Associativity licenses the tree. Applications then reuse the same work/span structure while changing the data and operator. The abstraction succeeds because its algebraic premise is explicit.

MPI makes that premise part of a collective interface among processes. Its reduction operation can exploit associativity, and commutativity when available, to change evaluation order. MapReduce broadens the operational setting again: keyed values are grouped, partitions are scheduled across machines, and failed tasks may be rerun. The 2004 system description assigns parallelization, placement, scheduling, and failure handling to the runtime, not to the user's binary operator.

Compiler vectorization brings the history back inside one program. A compiler must recognize that an apparent loop-carried dependence is a reduction rather than an arbitrary recurrence. It then asks whether language semantics and target costs permit reassociation or widening. Current LLVM documentation names XOR among supported reduction forms. That recognition depends on the same algebra this chapter proves by hand.

The sequence is not a march in which each stage replaces the last. Serial folds remain useful inside vector chunks and distributed workers. Parity remains useful inside stronger codes and protocols. The lasting development is the separation of operator law, schedule, machine state, and system contract.

Our contract asks only for the reduction value. It does not say that unequal regions have unequal reductions, that a changed word is always detected, or that an attacker cannot preserve the result. Keeping those tempting claims out of the theorem is part of formal precision. It also demonstrates why proof does not rescue

a poor specification: a flawless implementation proof can establish exactly the weak property that was requested.

15.2 The algorithmic contract

Fix width 64. Let M be byte-addressed memory, a the starting address, and n an unsigned count. Define the little-endian word load from Chapter 4 as $L(M, q)$. The desired result is

$$\text{foldxor}(M, a, n) = L(M, a) \text{ xor } L(M, a + 8) \text{ xor } \cdots \text{ xor } L(M, a + 8(n - 1)),$$

with value zero when $n = 0$.

Ellipsis is convenient prose but a poor base definition. The recursive form names both cases exactly:

$$\begin{aligned} \text{foldxor}(M, a, 0) &= 0, \\ \text{foldxor}(M, a, n + 1) &= L(M, a) \text{ xor } \text{foldxor}(M, a + 8, n). \end{aligned}$$

An indexed definition is useful for connecting the fold to loop states. Let $W_i = L(M, a + 8i)$. Then

$$\text{foldxor}(M, a, n) = \text{xor}_{0 \leq i < n} W_i.$$

The large-XOR notation means a fold with identity zero. It is not an unexplained infinite operation. The index set is finite, and the upper bound is excluded. When $n = 0$, the index set is empty and the declared identity gives zero.

The recursive and indexed definitions agree by induction on n . For zero, both are the empty fold. In the step from n to $n + 1$, split off the word at index zero in the recursive view, or split the indexed range into $\{0\}$ and $\{1, \dots, n\}$. Reindex the second range after advancing the address by eight. Associativity then gives the same parenthesization.

More generally, for every $0 \leq i \leq n$, the region splits at word i :

$$\text{foldxor}(M, a, n) = \text{foldxor}(M, a, i) \text{ xor } \text{foldxor}(M, a + 8i, n - i).$$

This is the memory form of the fold-splitting lemma. The loop accumulator stores the first term. The unread suffix described by the pointer and count stores the second term implicitly. At exit, the suffix length is zero, so its fold is the identity.

Prove the memory split without ellipsis

Fix M , a , and n . Induct on i . At $i = 0$, the prefix is empty and zero is the XOR identity. For the step, take the first word from the old suffix, append it to the old prefix, and apply the recursive equation to the suffix. Associativity

moves the new word across the split. The range and no-wrap premises ensure that both word loads name the same eight bytes before and after reindexing.

The program contract needs more than this equation. Each of the n loads must be valid. Address formation must not wrap around the 64-bit address space. The code region must be executable under the selected machine model. The calling convention must place the pointer and count where the listing expects them. On normal return, the result register must equal $\text{foldxor}(M, a, n)$, and memory must be unchanged.

It helps to factor the precondition. Define

$$\begin{aligned} \text{NoWrap}(a, n) &\iff n = 0 \text{ or } a + 8n - 1 < 2^{64}, \\ \text{Readable}(M, a, n) &\iff \text{every byte address in } [a, a + 8n) \text{ is readable.} \end{aligned}$$

These formulas use natural arithmetic on the right. The implementation uses 64-bit modular arithmetic. The no-wrap premise is the bridge that makes the two readings agree for every pointer value reached by the loop.

The interval formula also exposes a useful off-by-one distinction. The last byte read is $a + 8n - 1$, while the pointer after the last iteration is $a + 8n$. The program computes that one-past address but does not dereference it. A memory model may permit the arithmetic value even when no byte at that address is readable. The contract should not demand a readable byte merely because a pointer temporarily names the end of a region.

For $n = 0$, the expression $a + 8n - 1$ would underflow if evaluated as an unsigned word. That is why NoWrap treats the empty case separately. Mathematical notation can hide evaluation order; an executable precondition checker cannot. It should test zero before computing the last byte address, or use checked arithmetic whose failure is a typed rejection.

15.2.1 Frame conditions

The result equation constrains one register. A complete contract also says what the program may change. Here the memory frame is simple: the routine performs loads but no stores, so every memory byte after normal return equals its initial value. For a caller, callee-saved registers, the stack pointer, and the return mechanism add architecture-specific frame obligations.

Frame reasoning prevents a false implementation from winning by hidden work. A routine could compute the right XOR while using the input array as scratch and then fail to restore one byte. A result-only test would accept it. A frame clause rejects it even when the corrupted byte never affects the accumulator.

The frame can be stated globally or by a footprint. A global equality $M' = M_0$ is strongest and easy here. For a routine that writes an output buffer, the useful statement is that locations outside the declared write set are unchanged. The proof then combines local instruction write sets into a whole-program frame. This is the

same compositional idea used by separation logic: reason about the owned region while preserving a disjoint frame.

Readability is not ownership. The routine needs permission to read the input region but not to modify it. In a concurrent setting, read permission alone also would not guarantee that another agent leaves the bytes stable. This chapter assumes one fixed architectural memory during the call. Chapter 16 returns to interference and memory models.

XOR-reduction call contract

The input consists of an initial memory M_0 , address a , and count n . The precondition requires every range $[a + 8i, a + 8i + 8)$, for $0 \leq i < n$, to be readable and requires the address calculations not to wrap. Normal return must preserve M_0 and place $\text{foldxor}(M_0, a, n)$ in the declared result register. The contract observes normal return, the result word, and memory preservation.

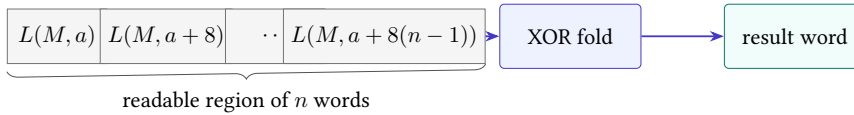


Figure 15.1: The contract connects a logical region of n words to one result. Pointer motion and accumulator updates are implementation details.

The contract deliberately permits $n = 0$. In that case the readable region is empty, no load occurs, and the result is zero. The address a need not be readable because no byte at that address is accessed. A precondition that always demanded eight readable bytes would be stronger than the routine needs.

Alignment is separate. Core Ao permits a word access to begin at any address provided the full byte range exists. The selected RV64I environment may trap or handle a misaligned load outside the base instruction semantics, depending on its execution environment. The selected x86-64 form permits an unaligned ordinary load in this model. For one clean three-way theorem, we require a to be divisible by eight. This is a theorem premise chosen for the relation, not a universal statement about all three machines.

A valid first address does not validate the region

Checking a alone is insufficient. The final access begins at $a + 8(n - 1)$, and all eight of its bytes must exist. The no-wrap and range premises quantify over every iteration.

15.3 The language-independent loop

The algorithm carries four logical values: the original memory M_0 , the number i of words already consumed, the current pointer $p = a + 8i$, and the accumulator

$$x = \text{foldxor}(M_0, a, i).$$

It also carries a remaining count $c = n - i$. The following **pseudocode** is a readable algorithm sketch rather than an executable language:

```
p := a
c := n
x := 0
while c != 0:
    x := x xor load64(M, p)
    p := p + 8
    c := c - 1
return x
```

Listing 15.1: The logical XOR-reduction loop

The order of the three body updates matters to a step-by-step proof even though the final mathematics is simple. The load uses the old pointer. The accumulator gains word i . The pointer then names word $i + 1$, and the remaining count becomes $n - (i + 1)$.

Loop invariant

At the test before iteration i , where $0 \leq i \leq n$, memory equals M_0 , the pointer equals $a + 8i$, the remaining count equals $n - i$, and the accumulator equals $\text{foldxor}(M_0, a, i)$.

Reader proof of the logical loop

Initialization. Before the first test, $i = 0$. The pointer is a , the count is n , memory is unchanged, and the accumulator is zero. These are exactly the four invariant clauses because the reduction of an empty prefix is zero.

Preservation. Assume the invariant at a test with $c \neq 0$. Then $i < n$, so the precondition makes $L(M_0, a + 8i)$ valid. The load reads that word without changing memory. XOR changes the accumulator to

$$\text{foldxor}(M_0, a, i) \text{ xor } L(M_0, a + 8i) = \text{foldxor}(M_0, a, i + 1).$$

The pointer addition produces $a + 8(i + 1)$, and decrement produces $n - (i + 1)$. Thus the invariant holds for the next test.

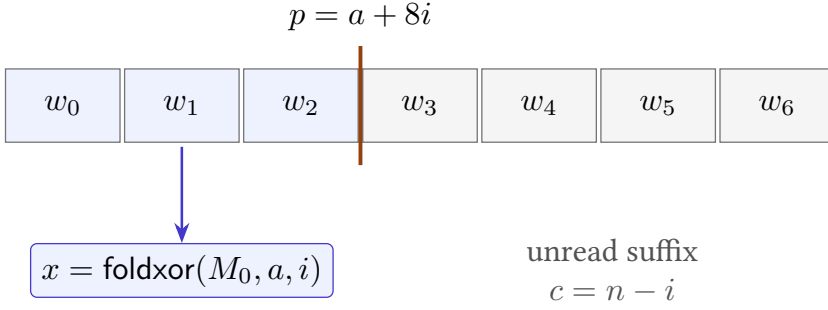


Figure 15.2: The invariant cuts memory into a consumed prefix and an unread suffix. The accumulator summarizes exactly the prefix.

Exit. If the test sees $c = 0$, the invariant gives $n - i = 0$, hence $i = n$. The accumulator is therefore $\text{foldxor}(M_0, a, n)$, memory still equals M_0 , and the return value satisfies the contract.

Termination. The remaining count is a natural number. Every body execution begins with a positive count and decreases it by one without wraparound. It therefore reaches zero after exactly n iterations.

This proof has two parts that bounded testing cannot replace. Preservation is quantified over every legal iteration, and termination covers every natural input count admitted by the contract. Testing can still find mistakes in an implementation or in the connection between its registers and these logical variables.

15.3.1 The loop invariant

The loop-head invariant is enough for a paper proof when the body is treated as one atomic command. A machine executes several instructions. To connect those instructions, place an assertion after each logical phase.

At the head before iteration i , write H_i :

$$p = a + 8i, \quad c = n - i, \quad x = \text{foldxor}(M_0, a, i), \quad M = M_0.$$

When $c > 0$, the load produces a temporary u . The post-load assertion is

$$D_i : \quad H_i \wedge u = L(M_0, a + 8i).$$

The pointer, count, and accumulator have not yet changed. This assertion is where effective-address formation, access validity, byte order, and load width meet.

After the combine, write

$$C_i : \quad p = a + 8i, \quad c = n - i, \quad x = \text{foldxor}(M_0, a, i + 1), \quad M = M_0.$$

The temporary may remain available, but the proof no longer needs it. After the pointer update,

$$P_i : \quad p = a + 8(i + 1), \quad c = n - i, \quad x = \text{foldxor}(M_0, a, i + 1).$$

After decrement, $c = n - (i + 1)$, so the next loop head satisfies H_{i+1} . Condition state produced by decrement must also justify the chosen backward edge on Ao and x86-64. RV64I instead compares the new count register directly.

Table 15.2: Intermediate assertions isolate the meaning of one logical body.

Assertion	Newly established fact	Main proof boundary
H_i	prefix, pointer, and remaining count agree	entry or prior branch
D_i	temporary is the next logical word	address, memory, endian load
C_i	accumulator includes word i	bitwise XOR and destination write
P_i	pointer names word $i + 1$	modular addition and no wrap
H_{i+1}	count and control advance together	decrement, flags or comparison, branch

This family prevents a common proof shortcut. If a trace disagrees after the load, the XOR lemma is irrelevant. If it agrees through C_i but branches wrongly, memory and XOR are already exonerated for that iteration. Assertions serve both proof composition and fault diagnosis.

15.3.2 The termination variant

The invariant says what remains true. The **variant** says why the loop cannot continue forever. Here the remaining natural count c is bounded below by zero and strictly decreases on every back edge. These are separate obligations. An implementation that decrements correctly but loads the wrong address terminates while violating correctness. One that computes each visited prefix correctly but never decrements can preserve a weakened invariant while failing termination.

Machine subtraction wraps, so “strictly decreases” is not a free property of 64-bit words. It follows from the guard $c > 0$: for a positive unsigned word, subtracting one yields the ordinary natural predecessor with no wrap. The branch must consume that new value or the condition state derived from it. The proof therefore connects data, arithmetic, and control in one progress argument.

Exact execution also has a resource interpretation. The loop takes exactly n body iterations, but an unbounded mathematical n does not promise completion

within a particular wall-clock limit. A timeout reports that an experiment exhausted its budget. It does not refute the termination theorem.

15.3.3 From algorithm to machine theorem

The pseudocode is not an instruction-set-neutral executable. Its word `load64` already assumes an address unit, byte order, access width, and failure policy. Its subtraction assumes a positive remaining count. Its return assumes an interface with a caller. Lowering the loop means discharging obligations, not replacing words with mnemonics.

First comes **representation**. Each logical variable needs a machine location at each cut point. The accumulator may remain in one register. The pointer and count may be overwritten because the contract observes their initial values and final result, not their final register contents.

Second comes **operation refinement**. A machine load must reconstruct $L(M, p)$. XOR must implement bitwise exclusive OR at width 64. The pointer update must represent addition by eight without wrapping on admitted inputs. The count update must represent natural-number subtraction by one. The branch must continue exactly when the new count is nonzero.

Third comes **control refinement**. Setup reaches the first loop cut point. One body path reaches the next cut point. The zero-count path reaches a successful terminal observation. Invalid fetches, loads, or returns must not be relabeled as success. Program-counter equations differ, but this shape is shared.

Fourth comes **frame preservation**: state outside the allowed write set remains unchanged. Memory is the main frame component here. ABI-preserved registers and stack state join it for the real-machine versions that a caller can invoke as procedures. A result equation alone would miss both forms of damage.

Table 15.3: The lowering obligations carried by every listing.

Obligation	Required connection
Representation	registers and memory correspond to p, c, x, M_0
Operation	load, XOR, add, and decrement implement the logical body
Control	setup, body, exit, and outcome follow the loop structure
Frame	memory and protected procedure state remain unchanged
Encoding	fetches bytes decode to the instruction instances in the proof

The last row is easy to omit when reasoning from assembly text. A proof of the mnemonic sequence is not yet a proof about executable bytes. The final artifact must bind assembled bytes, instruction boundaries, decoded operands, and the semantic instances used by the step relation. This chapter prints readable assembly because those decoders do not yet exist in Axeyum; that absence stays visible in the evidence boundary.

Build the cut-point map first

For each listing below, mark entry, the first state satisfying B_0 , the load state, the next loop-head state, and the successful terminal state. Write which logical variables are meaningful at each mark.

15.4 The Ao implementation

Use width-64 Ao with the following entry convention: $r_1 = a$, $r_2 = n$, and memory M_0 . Register r_0 is the result. Registers r_3, r_4, r_5 are scratch. The listing uses labels for reading; encoding replaces each label with the signed displacement defined in Chapter 9.

```

00: movi      r0, 0
04: movi      r4, 8
08: movi      r5, 1
0c: cmp       r2, r0
10: branch.eq done
loop:
14: load      r3, [r1+0]
18: xor       r0, r0, r3
1c: add       r1, r1, r4
20: sub       r2, r2, r5
24: branch.ne loop
done:
28: halt

```

Listing 15.2: Ao XOR reduction

The first comparison handles the empty input without loading. The subtraction sets Ao's zero condition from the new count, so the backward branch can reuse that result. The XOR also changes conditions, but the later subtraction overwrites them before `branch.ne`. This ordering is a small live-state argument: the branch depends on conditions from `sub`, not conditions from `xor`.

At the loop head, relate r_1 to p , r_2 to c , and r_0 to x . Registers $r_4 = 8$ and $r_5 = 1$ are fixed helpers. The Ao state also contains the program counter, condition bits, outcome, program bytes, and memory. The invariant does not erase these components. It constrains those needed for the proof and permits the current condition bits to have whatever values the preceding legal path produces.

The backward branch begins at byte address 36 and targets byte address 20. Ao's rule is $p + 4 + 4d$, so

$$36 + 4 + 4d = 20,$$

which gives $d = -5$. The forward branch at address 16 targets address 40 and therefore uses $d = 5$. Both fit the signed byte field.

A complete two-word A0 trace at loop granularity

Let $n = 2$, let the first word be u , and let the second be v . At the first loop head, $(r_1, r_2, r_0) = (a, 2, 0)$. After one body, $(r_1, r_2, r_0) = (a + 8, 1, u)$. After the second body, $(r_1, r_2, r_0) = (a + 16, 0, u \text{ xor } v)$. The second backward branch is not taken. Execution reaches `halt` with the required result.

This summary suppresses the instruction-level intermediate states. A machine trace must retain them: the state after the load, the state after XOR, both arithmetic updates, and the branch successor. The loop-granularity relation holds at selected cut points, not after every instruction.

15.4.1 Ao failure controls

Four small mutations test different obligations. Change the load displacement from zero to eight; the first iteration skips the first word. Change the pointer increment from eight to one; the next load overlaps the first word. Change the decrement helper from one to zero; termination fails. Change the backward displacement from -5 to -4 ; control returns to the XOR rather than the load, reusing stale r_3 . Each mutation should produce a distinct counterexample or control-flow failure.

For the empty case, poison the bytes at a or make the address unreadable. The valid program must still halt normally with zero because it never loads. That control distinguishes a true zero-iteration path from an implementation that reads once.

15.4.2 Five checked Ao movements

Assume the Ao loop-head relation at address 14 and $i < n$. The five instructions from 14 through 24 can be proved in sequence.

1. `load r3, [r1+0]` reads the eight bytes beginning at $a + 8i$. Range validity and little-endian reconstruction give $r'_3 = L(M_0, a + 8i)$. Other registers and memory are unchanged.
2. `xor r0, r0, r3` writes the next prefix fold to r_0 . It also writes condition state, but no subsequent branch may consume those particular condition values.
3. `add r1, r1, r4` uses the preserved helper $r_4 = 8$. The no-wrap premise changes modular addition into $a + 8(i + 1)$.
4. `sub r2, r2, r5` uses $r_5 = 1$ and positive $r_2 = n - i$. It writes $n - (i + 1)$ and sets the zero condition exactly when $i + 1 = n$.

5. `branch.ne` loop reads that condition. If another iteration remains, its decoded target is 14; otherwise its fall-through is 28. Both paths reach the cut point selected by the logical state.

This proof uses two helper invariants, $r_4 = 8$ and $r_5 = 1$. They are not part of the mathematical fold, but they are part of the implementation relation. A mutation to either helper can break pointer or count movement even when every opcode remains valid.

The Ao entry path also needs two cases. For $n = 0$, the comparison and forward branch reach 28 without entering the body. For $n > 0$, the same comparison falls through to 14 with H_0 . The terminal instruction is `halt`, so the Ao observation is a harness-defined successful halt whose r_0 value is interpreted as the result. It is not an ABI return.

A0-to-logical body theorem

Given decoded program bytes, normal Ao outcome, the loop-head relation, $i < n$, readable input bytes, and no wrap, exactly five Ao steps reach either the next loop head or the terminal fall-through. The resulting state has unchanged memory, the next prefix fold in r_0 , the next pointer in r_1 , and the next remaining count in r_2 . Its branch successor agrees with whether $i + 1 < n$.

15.5 The RV64I implementation

Use the standard integer calling names for exposition: `a0` initially holds a , `a1` holds n , and `a0` carries the return value. Because the pointer and result share `a0` at different times, copy the pointer to temporary `t0` before clearing the accumulator.

```

    addi t0, a0, 0
    addi a0, x0, 0
    beq  a1, x0, done
loop:
    ld   t1, 0(t0)
    xor  a0, a0, t1
    addi t0, t0, 8
    addi a1, a1, -1
    bne  a1, x0, loop
done:
    jalr x0, 0(ra)

```

Listing 15.3: RV64I XOR reduction

Every instruction belongs to the base integer slice used by this book (RISC-V International 2026e). The `addi` forms perform copying, zero construction, pointer advance, and count decrement. The conditional branches compare `a1` directly with

the architectural zero register. No arithmetic flags exist in the selected state. The final `jalr` returns through the incoming link register under the calling convention discussed in Chapter 10.

At the RV64 loop head, `t0` represents p , `a1` represents c , and `a0` represents x . Temporary `t1` is unconstrained before the load and equals the current word afterward. The state relation must also say that the address translation used by memory is identity on the input region, or state another explicit mapping.

The instruction `ld` forms an effective address from `t0` plus the sign-extended immediate zero and reconstructs a 64-bit little-endian value. The pointer increment is modulo 2^{64} , but the contract's no-wrap premise makes its mathematical reading equal to $a + 8(i + 1)$. Likewise, decrement is modular machine arithmetic; the loop guard and invariant ensure it is applied only to a positive count.

The proof uses the ABI without confusing it with the ISA

The ISA gives meanings to integer-register numbers, `ld`, branches, and `jalr`. The ABI gives the names `a0`, `a1`, `t0`, `t1`, and `ra`, and states which registers a caller may expect to survive. Both layers are needed for a callable routine. They are different sources of obligations.

The listing changes `a1`, `t0`, and `t1`. Under the selected calling convention these are caller-saved registers. It does not adjust the stack or call another routine. Its memory action is read-only. A correctness statement at ISA level can ignore ABI preservation; the callable-routine theorem cannot.

15.5.1 Explicit data control

RV64I has no implicit status register in this proof. The loop guard reads `a1` and `x0` as ordinary register operands. This simplifies the last-writer argument but does not remove control semantics: the branch decoder, comparison, signed immediate, target addition, and program-counter update must all agree.

At `ld`, the base register `t0` contains $a + 8i$ and the immediate is zero. The load lemma proves the same D_i assertion used by `Ao`. The next `xor` reads `a0` and `t1` and writes the next fold to `a0`. It has no condition side effect in RV64I.

The two `addi` instructions use sign-extended 12-bit immediates. `Eight` encodes the pointer step. All one bits in the immediate field encode -1 for the count step. Both execute modular 64-bit addition. Their logical readings come from different premises: pointer no-wrap for the first and positive remaining count for the second.

The backward `bne` compares the new `a1` with architectural `x0`. Register `x0` always reads as zero and discards writes. The proof needs the read rule even though this program never writes `x0`. The decoded target `0c` begins the next load. A non-taken branch reaches the return instruction at 20.

RV64I-to-logical body theorem

Assume the exact bytes in Table 15.4 decode at the named addresses, the loop-head relation holds at 0c, and $i < n$. Five RV64I steps establish the next prefix, pointer, count, and unchanged memory. The fifth step reaches 0c exactly when $i + 1 < n$, and otherwise reaches 20. No theorem about arithmetic flags is required.

The return theorem is separate. `jalr x0, 0(ra)` forms its target from the incoming return-address register, clears the low target bit according to the selected instruction rule, and discards the link because the destination is `x0`. The harness must supply an aligned allowed continuation and must interpret reaching it as normal return. The loop invariant alone says nothing about that continuation.

15.6 The x86-64 implementation

Under the selected System V integer convention, `rdi` holds a , `rsi` holds n , and `rax` carries the result. This Intel-syntax listing uses only scalar integer instructions.

```

xor    eax, eax
test   rsi, rsi
je     done
loop:
xor     rax, qword ptr [rdi]
add     rdi, 8
sub     rsi, 1
jne     loop
done:
ret

```

Listing 15.4: x86-64 XOR reduction

Writing `eax` clears all of `rax` in 64-bit mode (Intel Corporation 2026a). This is a selected x86-64 subregister rule, not a generic property of narrow writes. The `test` instruction computes a logical conjunction for flags without storing the result. Here it sets the zero flag exactly when the count is zero. The memory-source XOR loads a 64-bit word and combines it with the accumulator. The subtraction sets the zero flag from the new count, and `jne` consumes that flag.

At the loop head, `rdi` represents p , `rsi` represents c , and `rax` represents x . The relevant status-flag values are those created by the path into the head. The cross-machine relation need not equate x86 status flags with RV64 state because RV64 has no corresponding component. It must ensure that each machine chooses the same logical exit condition at the compared cut points.

The listing mutates `rdi` and `rsi`. They are caller-saved under the selected ABI, as is `rax`. The routine preserves the stack pointer and callee-saved registers

and returns through the address already on the stack. These facts are not visible in the mathematical fold equation, but they are part of the procedure contract.

15.6.1 Memory, arithmetic, and flags

The memory-source `xor` performs the load and combine within one architectural transition. Its proof premise still names both components. The effective address is the old `rdi`; the operand width is 64 bits because of the `REX.W` form; the eight bytes are reconstructed according to the selected little-endian rule; and the result overwrites all 64 bits of `rax`.

That instruction also sets logical status flags and clears selected others. The loop does not branch immediately, so those values are dead. `add` then overwrites arithmetic flags, and `sub` overwrites them again. The `jne` instruction must read the zero flag written by `sub`. A machine proof establishes this def-use chain rather than saying only that “the flags work out.”

The `sub rsi, 1` result is zero exactly when the positive pre-state count was one. Therefore $ZF=1$ exactly when $i + 1 = n$. `jne` continues when $ZF=0$, which is the same condition as $i + 1 < n$. The short branch target is the memory-source `XOR` at 07; fall-through is 14.

x86-64-to-logical body theorem

Assume the exact bytes in Table 15.5, the loop-head relation at 07, normal memory access, and $i < n$. Four x86-64 steps establish the next prefix, pointer, count, and unchanged memory. The final step reaches 07 exactly when another word remains and reaches 14 otherwise. The proof permits intermediate flags but requires the zero-flag def-use chain from subtraction to branch.

The `ret` instruction adds a memory obligation absent from the displayed fold. It reads the continuation from the stack, advances `rsp`, and transfers control. A valid System V call harness must provide that stack word and require callee-saved registers to survive. Saying that the loop is read-only refers to the input array; return still reads stack memory.

Equivalent results can hide unequal fault behavior

If the readable-region premise is removed, the three programs need not fail in the same way. Alignment policy, address translation, and fault delivery differ. Our theorem relates normal executions under the printed precondition. It does not assert universal agreement for invalid memory.

15.7 Bytes and machine boundaries

The real-ISA listings can be assembled now even though Axeyum cannot yet execute them. Assembly gives exact byte witnesses for code size, boundaries, and branch displacement. It does not prove the instruction semantics or the loop theorem. Those remain separate obligations.

For a reproducible check, the chapter’s research log records LLVM assembler version 21.1.8, the target triples, the scalar feature selection, the source syntax, and independent disassembly. The RV64I listing assembled to 36 bytes. Every instruction is four bytes. The x86-64 listing assembled to 21 bytes with lengths from one to four bytes.

15.7.1 RV64I field placement

Using entry address zero, the RV64I text is:

Table 15.4: One assembled RV64I encoding of the scalar reduction. Bytes appear in increasing memory-address order. Pseudoinstruction spellings in the disassembly do not change the encoded base instructions.

Address	Bytes	Source instruction	Decoded reading
00	93 02 05 00	<code>addi t0,a0,0</code>	copy pointer
04	13 05 00 00	<code>addi a0,x0,0</code>	zero accumulator
08	63 8c 05 00	<code>beq a1,x0,done</code>	branch to 20
0c	03 b3 02 00	<code>ld t1,0(t0)</code>	load one 64-bit word
10	33 45 65 00	<code>xor a0,a0,t1</code>	combine accumulator
14	93 82 82 00	<code>addi t0,t0,8</code>	advance pointer
18	93 85 f5 ff	<code>addi a1,a1,-1</code>	decrease count
1c	e3 98 05 fe	<code>bne a1,x0,loop</code>	branch to 0c
20	67 80 00 00	<code>jalr x0,0(ra)</code>	return

An assembler may print `mv`, `li`, `beqz`, `bnez`, and `ret` when disassembling these bytes. These are aliases for the base instructions shown in the source listing. The semantic proof should decode the instruction word into its base operation and operands. It should not treat a preferred pretty-printer spelling as another instruction.

The forward branch at address 08 reaches 20, a displacement of 24 bytes. The backward branch at 1c reaches 0c, a displacement of -16 bytes. RISC-V branch immediates encode signed multiples of two in split instruction fields. Reconstructing the target requires field extraction, sign extension, and addition to the branch address. The table’s target is a check on that complete path.

15.7.2 x86-64 instruction lengths

The x86-64 text assembled as follows:

Table 15.5: One assembled x86-64 encoding of the scalar reduction, using Intel syntax and entry address zero.

Address	Bytes	Instruction	Main effect
00	31 c0	xor eax, eax	clear rax; set flags
02	48 85 f6	test rsi, rsi	set flags from count
05	74 0d	je done	branch to 14
07	48 33 07	xor rax, [rdi]	load and combine
0a	48 83 c7 08	add rdi, 8	advance pointer
0e	48 83 ee 01	sub rsi, 1	decrease count; set flags
12	75 f3	jne loop	branch to 07
14	c3	ret	pop and transfer to continuation

The two short conditional branches use signed 8-bit displacements measured from the address after the branch instruction. At 05, the next address is 07; $7 + 13 = 20$, or 14. At 12, the next address is 14; $20 - 13 = 7$, or 07. The same numerical displacement would denote a different target if instruction lengths changed before the next address was formed.

The memory-source XOR is one architectural instruction and one table row. Its semantics still include effective-address calculation, an eight-byte memory read, reconstruction of a 64-bit operand, XOR, destination write, and flag updates. Fewer decoded instructions does not mean fewer semantic effects.

Exact bytes answer a narrow but necessary question

The tables establish one assembler's byte sequence, boundaries, and branch targets. They do not establish that a processor executes the bytes according to this chapter, that the ABI entry state is valid, or that the fold theorem follows. Those claims need decoder, semantic, harness, and proof bindings.

15.7.3 Three sets of footprints

At a loop head, each machine reads a pointer, remaining count, accumulator, program bytes, and eight input bytes during the body. Their visible writes differ. Ao writes a load temporary, accumulator, pointer, count, conditions, and program counter. RV64I writes the corresponding integer registers and program counter but has no arithmetic flags in the selected state. x86-64 writes the accumulator, pointer, count, status flags, and instruction pointer; its memory-source XOR has no separate architectural load temporary.

The cross-state relation keeps logical values and forgets representation-only state after showing that forgotten writes cannot disturb later obligations. For example, Ao and x86 condition state is not compared with RV64I. It is not ignored

Table 15.6: Architectural footprints for one positive-count body. Program fetch and the changing program counter are included conceptually; names below focus on data and condition state.

Machine	Main reads	Main writes
A0	r_1, r_2, r_0, r_4, r_5 , conditions at branch, eight memory bytes	r_3, r_0, r_1, r_2 , conditions, program counter
RV64I	t0, a1, a0, x0, eight memory bytes	t1, a0, t0, a1, program counter
x86-64	rdi, rsi, rax, flags at branch, eight memory bytes	rax, rdi, rsi, status flags, instruction pointer

locally: the control lemma proves that the last writer makes each branch choice agree with the new count.

15.7.4 Static and dynamic counts

The printed Ao program has 11 static instructions and 44 bytes. RV64I has nine instructions and 36 bytes. The x86-64 program has eight instructions and 21 bytes in the assembled layout above. These facts concern one source and one encoding selection.

Dynamic counts depend on n . For zero, all three execute four to six setup, test, exit, or return instructions and no body. For positive n , the Ao and RV64I bodies each execute five instructions per word. The x86-64 body executes four because load and XOR share one instruction.

Table 15.7: Exact instruction counts for the printed paths. A branch counts whether taken or not taken. These are not cycle counts.

Machine	Static instructions	Dynamic, $n = 0$	Dynamic, $n > 0$
A0	11	6	$6 + 5n$
RV64I	9	4	$4 + 5n$
x86-64	8	4	$4 + 4n$

Every positive path performs exactly n architectural data loads, or $8n$ input bytes. Once the working set exceeds fast storage, this common traffic can dominate differences in scalar instruction count. Conversely, a tiny hot array may expose front-end, dependency, or branch costs. A serious performance comparison must name the data size, layout, cache state, target processor, compiler or assembler choices, repetition method, and measured quantity.

15.7.5 The execution harness

Raw bytes do not choose their own entry state. A harness places code in executable memory, constructs the input region, initializes registers and stack state, chooses a

continuation, starts execution, and classifies the outcome. These choices are part of the theorem whenever the chapter says the routine can be called.

The logical input constructor takes (M_0, a, n) . The Ao harness writes a to r_1 , n to r_2 , sets the program counter to 00, loads the program image, and declares `halt` at 28 successful. The RV64 harness places a in $a0$, n in $a1$, and an allowed continuation in ra . The x86 System V harness uses `rdi` and `rsi`, places an allowed continuation at the top of the stack, and sets `rsp` to that stack word.

Table 15.8: Harness obligations make unlike entry and return mechanisms comparable through one call contract.

Machine	Logical inputs	Successful terminal observation	Additional frame
A0	$r_1 = a, r_2 = n$	halted at declared site with r_0 result	program and memory frame
RV64I	<code>a0=a, a1=n, valid ra</code>	control at allowed continuation with <code>a0</code> result	callee-saved registers and stack unchanged
x86-64	<code>rdi=a, rsi=n, valid stack return</code>	control at popped continuation with <code>rax</code> result	callee-saved registers; prescribed <code>rsp</code> advance

The terminal memory clauses require care. Ao and RV64I do not use stack memory in their printed bodies. x86 `ret` reads eight bytes from the stack but does not write them. All three preserve the input region. A global memory equality remains possible if instruction fetch and return are modeled as reads and no environment mutates memory during the call.

Calling conventions also constrain alignment and preserved registers. The printed routines use only caller-saved data registers, so they need no save/restore prologue. The x86 return changes `rsp` by consuming the return address; preservation means it reaches the ABI-prescribed post-return value, not that the callee's final pre-return `rsp` equals the caller's post-return `rsp`. The observation must compare the right cut points.

A permissive harness can prove the wrong routine

If every address is accepted as a continuation, a corrupted return target may look successful. If any `halt` counts as Ao return, a branch into an error `halt` may pass. If the harness initializes scratch registers to convenient values, it can hide a missing setup instruction. Entry and terminal predicates must be as exact as the program theorem.

Harness controls should poison unconstrained conveniences. Initialize scratch registers with nonzero, unequal words. Put guard bytes around the input and stack

regions. Choose a continuation different from nearby code addresses. Run the empty case with an unreadable input pointer. These choices test that the routine depends only on declared inputs and returns through the intended mechanism.

15.7.6 Byte provenance

The assembled tables use entry address zero because every control transfer in the listings is relative or register-based. Relocating the whole text preserves internal relative displacements. It changes absolute program-counter values and therefore changes the machine states recorded in a trace. The manifest must bind the load address even when raw text bytes remain identical.

Source labels are not stored in the final text. They help the assembler compute displacements. A proof artifact should carry raw bytes and decoded targets, with source listings as explanatory material. If a new assembler chooses the same bytes, that is useful reproduction. If it chooses different but semantically equivalent bytes, the result is a new program artifact that needs its own digest and decode report.

Independent disassembly checks one direction: bytes map back to expected instruction instances and boundaries. It can share ISA tables or libraries with the assembler, so its independence should be described. Execution against a separate semantic decoder adds another dimension. Hand reconstruction of the two branch targets supplies a tiny reference that does not depend on a pretty-printer's label recovery.

For x86-64, boundaries are part of decoding because instruction lengths vary. A one-byte insertion can shift every later address and change relative branch targets even when the branch bytes stay fixed. RV64I base instructions retain four-byte boundaries, but a changed word can still decode to a valid different operation. Hashes detect byte changes; decoder controls show whether changed bytes are rejected or interpreted as a different program.

15.8 Local simulation lemmas

The three-machine proof kit below compresses many instruction facts into one body paragraph. A formal development should expose them as small lemmas. The load lemma starts from related pointers and memory and ends with equal loaded words. It depends on effective-address formation, range validity, access width, and little-endian reconstruction. It says nothing yet about the accumulator.

The combine lemma starts from equal logical accumulator values and equal loaded words. It concludes that the three destination words agree after XOR. Ao and x86 also update condition state; RV64I does not. Those side effects are allowed by the relation but must be described because later control may observe them.

The pointer lemma connects three modular additions to the ordinary equality $p' = p + 8$. Its proof uses the no-wrap premise. The decrement lemma connects machine subtraction to $c' = c - 1$ and uses $c > 0$. The control lemma then shows

that the Ao zero condition, RV64 register comparison, and x86 zero flag select continuation exactly when $c' \neq 0$.

These lemmas meet in a deliberate order. Proving the branch before showing which instruction last wrote its condition state would leave a gap. Proving the load without the address relation would compare bytes from different locations. Proving arithmetic only as a word equality would not justify the natural-number decrease used for termination.

Small lemmas localize a counterexample

A load mismatch points toward address or memory semantics. An equal load followed by unequal accumulators points toward XOR or register writes. Equal body values followed by unequal paths points toward conditions or targets.

One may also prove each machine correct against the logical loop separately and derive pairwise agreement by transitivity. That structure often scales better than one ternary relation. The direct three-way presentation keeps the shared cut points visible. An Axeyum implementation may store one machine-to-logical simulation per architecture and construct its comparison report from their common logical observations.

15.8.1 Factoring through the logical machine

Let a logical state be the tuple

$$\ell = (i, p, c, x, M, o),$$

where o is running or returned. The logical body moves H_i to H_{i+1} when $i < n$, and the exit moves H_n to returned with result x . This small transition system is not a fourth instruction set. It is a proof interface containing only the state needed by the algorithm.

Define relations R_A , R_R , and R_X from Ao, RV64I, and x86-64 states to logical states. Each relation maps its architecture's pointer, count, accumulator, memory, control point, and normal outcome to the tuple. It may also constrain helper registers or flags required to take the next local path. State outside the algorithm and calling contract remains framed or abstracted.

For each architecture Q , the simulation obligation has the form

$$R_Q(s, \ell) \wedge \ell \longrightarrow_L \ell' \implies \exists s'. s \longrightarrow_Q^+ s' \wedge R_Q(s', \ell').$$

The superscript plus means one or more machine steps. An initialization clause connects the call state to H_0 . A terminal clause connects the logical return to the architecture's successful observation. A failure-reflection clause prevents a trap or decode refusal from being mistaken for that return.

Architecture-to-logical simulation package

For one listing, a package contains an entry relation, a loop-head relation, an exit relation, a body path theorem, an exit path theorem, a frame theorem, and a progress measure. It also binds the byte decoder and the exact program image used by every path theorem.

This factorization has three benefits. Each proof mentions one real machine instead of all three. Adding a fourth machine requires one new relation, not three pairwise proofs. When two programs disagree, their logical images show which side first left the common contract.

Pairwise agreement is then a corollary. If $R_A(s_A, \ell)$ and $R_R(s_R, \ell)$, both simulations reach terminal states related to the same logical result. The same argument connects either to x86-64. Transitivity is not magic: it depends on using the same logical input, memory interpretation, scope, and terminal observation in both packages.

15.8.2 Finite stuttering

Between cut points, one logical transition corresponds to several machine steps. These intermediate steps **stutter** in the logical state: they prepare or partly implement one logical movement without yet reaching the next related cut point. Stuttering is safe only when the machine cannot remain in the intermediate region forever or escape to an unreported outcome.

A local rank can prove this progress. Assign each instruction position within the selected body a decreasing distance to the next cut point. A legal step either decreases that distance or reaches the next logical state. The loop count proves progress across bodies; the local distance proves progress within one body. Together they rule out an implementation that cycles among setup instructions while preserving a coarse relation.

The body paths have different lengths: five instructions for Ao, five for RV64I, and four for x86-64. Their intermediate assertions also differ. Ao and RV64I expose the loaded word in a register. The x86 memory-source XOR moves directly from the head relation to a post-combine relation in one architectural step. The logical interface accommodates both because it demands the same next cut point, not the same internal staging.

A relation that holds only at the beginning and end is not enough

A faulty implementation may leave the relation, trap, loop forever, or corrupt framed state before returning the expected result on one test. The simulation package needs local path, progress, and frame theorems. Matching endpoint values from selected executions is weaker evidence.

15.9 Relating three traces

The listings have different setup lengths and different numbers of instructions per iteration. A lockstep relation would fail immediately. We instead compare **cut points**: entry, loop head, and normal return. One logical body corresponds to several machine steps on every architecture, but the number and placement of those steps need not agree.

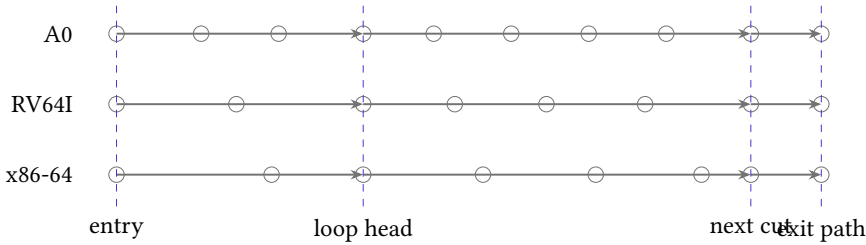


Figure 15.3: Different instruction traces meet at common logical cut points. The dashed bands represent initialization, one body, and exit.

Define relation B_i at the loop head. It requires all three memories to agree on the input region and to remain equal to M_0 . It maps Ao r_1 , RV64 $t0$, and x86 rdi to $a + 8i$. It maps Ao r_2 , RV64 $a1$, and x86 rsi to $n - i$. It maps Ao r_0 , RV64 $a0$, and x86 rax to the reduction of the first i words. It also places each program counter at its own loop test or body cut point and requires each machine to remain in normal execution.

Three-machine forward simulation

Entry to first cut point. Each setup sequence preserves memory, copies or retains the input pointer, constructs the zero accumulator, and retains the count. Therefore B_0 holds.

Body correspondence. Assume B_i and $i < n$. The three effective addresses all denote $a + 8i$. By memory agreement and common little-endian reconstruction, the loads return one word $u = L(M_0, a + 8i)$. Each accumulator becomes its old value XOR u . Each pointer advances by eight, and each count decreases by one. The no-wrap and positive-count premises connect the modular updates to ordinary arithmetic. Thus B_{i+1} holds at the next cut point.

Exit correspondence. Under B_i , all three loop tests see zero exactly when $n - i = 0$. They therefore exit together at the logical level, although through different instructions. Then $i = n$, and all three result registers equal $\text{foldxor}(M_0, a, n)$. Their return mechanisms establish the selected procedure observations.

The proof composes two ideas. The loop invariant connects each implementation to the mathematical fold. The cross-state relation connects corresponding implementation cut points. Either route can be used alone, but together they explain both correctness and agreement.

15.9.1 The quantified global result

The chapter's main theorem can now be written without saying that the programs are simply "the same." Let Q range over the three selected machine packages. Let $\text{Init}_Q(M_0, a, n, s)$ mean that machine state s contains the exact program bytes, the harness mapping in Table 15.8, and a normal entry outcome. Let $\text{Run}_Q(s, t)$ mean that zero or more legal machine steps take s to a successful terminal state t . Let $\text{Result}_Q(t)$ project the declared result register.

Under the common range, no-wrap, alignment, code, and harness preconditions, the desired result is

$$\begin{aligned} \forall M_0, a, n, Q, s. \quad & \text{Pre}(M_0, a, n) \wedge \text{Init}_Q(M_0, a, n, s) \\ \implies \exists t. \quad & \text{Run}_Q(s, t) \wedge \text{Result}_Q(t) = \text{foldxor}(M_0, a, n) \wedge \text{Frame}_Q(s, t). \end{aligned}$$

Existence states termination and successful return. The result equality states functional correctness. The frame states preservation. A deterministic step relation can strengthen existence to a unique execution, but uniqueness is not needed to show the fold result if every admitted execution satisfies the same observation.

The three-machine agreement corollary quantifies over three initial states constructed from the same logical input. If each reaches its successful terminal state, then

$$\text{Result}_A(t_A) = \text{Result}_R(t_R) = \text{Result}_X(t_X) = \text{foldxor}(M_0, a, n).$$

This equality is derived through the specification. It does not equate final program counters, scratch registers, condition state, stack representation, or trace length.

Several premises are deliberately universal. Memory contents are not fixed to the attractive three-word example. The count is not bounded by a test depth. Word bits range over all width-64 patterns. Several premises are deliberately restricted. Memory remains stable, the region is readable and aligned, address arithmetic does not wrap, code bytes are fixed, and returns satisfy the selected harness.

The theorem has four named layers

The fold theorem belongs to mathematics. Each machine simulation belongs to a pinned semantic package. The harness theorem belongs to an ABI or declared Ao observation. The agreement corollary composes those layers. Evidence must say which layers were reader-proved, executable, solver-checked, certificate-checked, or reconstructed in a kernel.

15.9.2 Failure behavior

The global theorem begins under a readable aligned region and valid return harness. Removing those premises does not produce one shared failure result. Ao may report its model's invalid access. An RV64 execution environment may handle or trap a misaligned load according to a boundary outside the selected base instruction rule. x86-64 permits many unaligned ordinary loads but can still fault for translation or permission reasons. Return failures also use different mechanisms.

A robust comparator therefore has a sum of outcomes: normal return, trap or fault, decode refusal, unsupported state, bound exhaustion, and infrastructure failure. The functional theorem covers the normal branch under Pre. A separate refinement theorem could relate selected fault classes, but it would need explicit exception and environment models. Treating every nonreturn as one equal outcome would erase information needed for both debugging and security analysis.

The negative controls exercise these excluded paths without broadening the theorem. An unreadable empty pointer should still take the normal zero case because no access occurs. An unreadable one-word region should fail at the load boundary. A corrupted return target should fail at the harness boundary. The report shows that the classifier distinguishes these cases; it does not claim that the architectures share one fault semantics.

15.9.3 What the relation forgets

Ao condition bits, x86 status flags, and RV64 temporary values need not agree. Instruction bytes and program counters differ. Ao ends through `halt` in the pedagogical listing, whereas the two callable real-ISA listings return to a continuation. A direct theorem between those terminal states must either map Ao halt to a harness-defined return observation or give Ao a fixed-return wrapper as described in Chapter 10.

The distinction is substantive. Without the harness, “all three return” is false for the printed Ao program. The clean statement is that all three reach their declared successful terminal observation with the same result. A later Axeyum package should encode that observation rather than force unlike control states to be equal.

15.10 A concrete execution

Let $a = 256$, $n = 3$, and let the three loaded words be

$$u = 0f0f, \quad v = 00ff, \quad z = 0ff0,$$

shown with leading zeroes omitted. The successive accumulator values are

$$0, \quad 0f0f, \quad 0ff0, \quad 0000.$$

The pointer values at loop heads are 256, 264, 272, and 280. The remaining counts are 3, 2, 1, and 0. The final zero is not evidence that no work occurred; these particular words cancel.

Table 15.9: The logical trace for the three-word example.

i	pointer	remaining	accumulator
0	256	3	0000
1	264	2	0f0f
2	272	1	0ff0
3	280	0	0000

A useful test suite includes more than this attractive cancellation. Test an empty input, one word, two unequal words, two equal words, a region near the highest permitted address, and a pattern with high bits set. For each case, compare the complete outcome class and memory, not only the result register.

15.10.1 Three instruction traces

Each row in the logical table corresponds to a loop-head cut point on all three machines. The program counters differ:

Table 15.10: Cut points and intermediate assertions for one positive iteration. Addresses are hexadecimal offsets from each listing’s entry.

Logical point	A0	RV64I	x86-64
loop head H_i	14	0c	07
post-load D_i	18	10	combined with next row
post-combine C_i	1c	14	0a
post-pointer P_i	20	18	0e
post-count	24	1c	12
next head or exit	14/28	0c/20	07/14

For the first word $u = 0f0f$, all three heads represent $(i, p, c, x) = (0, 256, 3, 0)$. Ao and RV64I first place u in their temporary registers. x86-64 obtains u as the memory operand of the XOR transition. At C_0 , all accumulators equal u . After pointer and count movement, the branches return to heads representing $(1, 264, 2, u)$.

The second iteration loads $v = 00ff$. Bitwise combination gives $0f0f \text{ xor } 00ff = 0ff0$. The next heads represent $(2, 272, 1, 0ff0)$. The third word is $z = 0ff0$, so the final combine cancels the accumulator to zero. The pointers become 280 and counts become zero.

At that point, each continuation branch chooses exit. Ao reaches 28 and executes `halt`. RV64I reaches 20 and executes `jalr` through the harness-provided `ra`. x86-64 reaches 14 and executes `ret` through the continuation stored at `rsp`.

Their terminal machine states are not equal. Their declared terminal observations all contain the result zero and unchanged input memory.

Replay discipline for the concrete case

At every machine step, record raw program bytes, decoded instruction, old state, memory access if any, new state, and outcome. At each cut point, project the machine state through its architecture-to-logical relation. Require the four logical values in Table 15.10 to agree. If a projection first fails, report the prior successful assertion and the current instruction rather than only the final unequal result.

The example's final zero is a useful diagnostic trap. A broken executor that returns zero without reading memory passes a result-only check for these three words. Trace length, load footprints, cut-point values, and a noncancelling positive control prevent that accidental pass.

15.11 Breaking the proof

Negative controls should target the binding chain from Chapter 14. At the specification layer, change the fold identity from zero to all ones. At the memory layer, reverse the byte order of one decoder. At the instruction layer, change one pointer increment from eight to four. At the control layer, branch on zero rather than nonzero after decrement. At the ABI layer, place the count in the wrong entry register. At the observation layer, forget memory preservation and introduce a store that the result-only comparison ignores.

Each control needs a small separating input. The empty input directly separates the wrong identity. A two-word region with distinct halves exposes a four-byte pointer increment. A count of one exposes an inverted continuation branch. A count of zero exposes an implementation that loads before checking. A memory snapshot before and after exposes the hidden store.

Find the first separating iteration

Change the RV64 pointer update to `addi t0, t0, 16`. For $n = 3$, write the addresses loaded by the valid and mutated programs. Choose three word values so that the first result disagreement occurs only after the second load.

Counterexamples should replay as traces. A solver assignment saying $n = 2$ and giving two memory words is not yet an explanation. Decode the program, execute both versions from the assigned state, and print the first cut point where pointer, count, accumulator, outcome, or memory agreement fails.

15.11.1 Boundary controls

A large end-to-end mismatch proves that something differs. A layered control explains where the route first loses the contract. The expected rejection point is therefore part of the control definition.

Table 15.11: A control matrix for the three-machine package. Each mutation should survive earlier identity and parsing checks, then fail at its named boundary.

Boundary	Mutation	Small separator	Expected first failure
specification	use all-ones identity	$n = 0$	fold contract
input map- ping decoder	swap pointer and count	$a \neq n$	entry relation
	flip an opcode field	any legal call	decode or byte binding
load seman- tics	reverse byte join	alternating bytes	D_i load assertion
combine	use OR instead of XOR	overlapping one bits	C_i combine asser- tion
pointer	advance by four	$n = 2$, distinct half- words	P_i pointer asser- tion
count	subtract zero	$n = 1$	variant or next- head relation
control	invert final branch	$n = 1$	successor relation
frame	inject one store	any nonempty region	memory frame
ABI	clobber a saved register	sentinel saved value	return-frame check
observation	accept trap as return	forced invalid access	outcome classifier
report	omit a failed control	one known mutation	manifest/report gate

A byte mutation that breaks parsing tests the identity and decoder boundary. It does not test whether a correctly decoded xor has the right meaning. For that purpose, keep the bytes valid and mutate the semantic rule or expected relation. Conversely, a semantic mutation that never reaches the mutated rule is a weak control. The separating input must activate the target.

Metamorphic controls test relations among valid cases. Prepending a zero word should leave the result unchanged while increasing the count and shifting the old words by one position. Permuting words should preserve XOR. Splitting a region into two chunks should make the whole result equal the XOR of the chunk results. Applying the same word mask to two added copies should cancel their joint contribution. Each transformation predicts a result without requiring a second implementation of the full fold.

Some transformations should *not* preserve the complete observation. Permuting input bytes changes memory identity even when the XOR result stays the same.

A relation that observes only the result may call the cases equivalent; the procedure contract also observes memory preservation within each run, not equality between two different initial memories. State which relation the metamorphic test asserts.

15.11.2 Proof mutations

The proof can be wrong while all three listings are right. Remove the no-wrap premise and the pointer lemma becomes invalid near 2^{64} . Replace $i < n$ with $i \leq n$ before the load and the final iteration may read past the region. Let D_i use the new pointer instead of the old pointer and the load assertion shifts by one word. Treat x86 flags from XOR as if they survived the later subtraction and the branch justification cites the wrong producer.

These are valuable review controls because theorem statements and proof scripts can pass type checking while proving an unintended claim. A weakened precondition, observation, or decoder link may make a proof easy for the wrong reason. The manifest should bind those definitions, and the review should include mutations that change them while leaving program bytes untouched.

The strongest small control set crosses all three directions:

1. keep semantics fixed and mutate bytes;
2. keep bytes fixed and mutate semantics or the relation;
3. keep both fixed and mutate the report or observation.

Passing this set does not prove the route sound. It demonstrates that the route can discriminate several errors it is specifically designed to reject.

15.12 Costs after correctness

The three listings invite counting, but one number cannot settle which is best. Ao uses fixed four-byte instructions and materializes constants in registers. RV64I also uses fixed-width base instructions but has immediate forms and a zero register. x86-64 uses variable-length encodings and a memory-source XOR. Instruction counts, encoded bytes, decoded operations, loads, branches, and estimated cycles are different costs.

For a positive count, every listing executes its setup and entry test, one load and combine per word, pointer and count updates, and a continuation test. The final not-taken branch is still an executed instruction. Symbolic dynamic counts can be derived from the listings, but they describe abstract paths, not processor timing. Caches, prediction, instruction fusion, and other microarchitectural facts are outside the scalar ISA model.

Code-byte comparison requires actual encodings. The x86 memory-source form may save a separately decoded load while using more bytes for other instructions.

RV64 compressed encodings would change byte cost, but the optional compressed extension is outside this chapter’s language. Admitting it for one candidate and not another would change the optimization question.

The loop is not a minimality theorem. Another implementation might count up toward an end pointer, unroll two loads, use a different branch layout, or return through a harness-specific convention. Chapter 13 requires a finite candidate grammar, cost, precondition, observation, and lower-bound argument. These listings are transparent correctness witnesses, not globally optimal programs.

There is nevertheless a useful comparison exercise. Count static instructions in each printed body, then count dynamic instructions for $n = 0$, $n = 1$, and general positive n . Next assemble only after fixing exact syntax, profiles, and entry addresses, because branch displacements and x86 instruction lengths depend on that layout. Finally, keep the resulting table descriptive. No column should be silently promoted into “the cost.”

On real hardware, even a fixed byte sequence has no single context-free time. The first load may miss in cache; later loads may stream. A branch predictor may learn the continuation pattern. Front-end and execution resources may overlap instructions. Those phenomena are important, but they demand a microarchitectural model and measurement method. The ISA proof establishes functional movement across architectural states, not a cycle count.

15.12.1 Arithmetic intensity and bandwidth

The scalar body consumes eight input bytes and performs one useful XOR combine per word. Ignoring instruction fetch and bookkeeping, its arithmetic intensity is about

$$I = \frac{1 \text{ combine}}{8 \text{ input bytes}}.$$

This is low. Once data must arrive from a slower level of the memory hierarchy, the attainable word rate can be bounded by sustainable byte bandwidth divided by eight. Making XOR twice as fast cannot cross a limit set by unchanged data delivery.

The Roofline model popularized this comparison between operation throughput, data bandwidth, and arithmetic intensity. (S. Williams et al. 2009) Its original examples concern floating-point kernels, but the discipline applies here if the units are stated honestly. We can count integer combines per byte instead of floating-point operations per byte. The result is a diagnostic upper bound, not a prediction of exact cycles.

Cache changes which bandwidth is relevant. A small array reused many times may remain near the core. A one-pass large array may stream from memory. An array that shares cache capacity with other work can behave differently again. Prefetching may overlap some latency, but it does not eliminate transferred bytes. Measurement should report the working-set size and whether setup warmed the data before timing.

The code reads but does not write the array, which reduces data traffic relative to an in-place transform. It still writes architectural registers, but those writes normally do not become eight-byte memory transfers. A proof about architectural state and a model of physical traffic count different objects. Mixing them would make both claims unclear.

15.12.2 Vectorization

A vector XOR loop loads several words, combines them into a vector accumulator, and performs a horizontal XOR at the end. If the vector holds q words, the main loop can consume about q scalar elements per vector iteration. Multiple vector accumulators may shorten dependency chains further.

The proof uses the chunking lemma. Each lane or vector accumulator reduces a subsequence. The horizontal combine folds the partial results. Because XOR is associative and commutative, lane grouping and combination order do not change the word result. The frame and memory arguments must still prove that every input word is loaded exactly once and no byte outside the region is observed.

Three new cases appear:

1. The length may be smaller than one vector.
2. The length may not be a multiple of the vector capacity.
3. The starting address may not meet a preferred vector alignment.

A scalar cleanup loop, a masked final vector, or a length-specific short path can handle the tail. These strategies have different instruction, fault, and proof behavior. A masked load must establish that inactive lanes do not access out-of-range bytes under the selected ISA semantics. A scalar tail reuses the scalar theorem but adds a handoff invariant between vector and scalar prefixes.

The ratified RISC-V vector extension contains integer reduction operations, including `vredxor`. vs. Its vector state adds vector registers, active length, element width, grouping, masks, and control registers. Inactive source elements are excluded while the scalar seed is included. (RISC-V International 2026d) Those rules illustrate why “replace the loop with one vector instruction” is not a proof. The new instruction has a wider state and its own scope.

x86-64 offers several generations of packed integer instructions, but the available width and exact instruction forms depend on the selected extension profile. Wider encodings can also change code size, register preservation, transition costs on some implementations, and compiler decisions. The Intel optimization manual treats vectorization and memory behavior as processor and code-shape questions rather than a universal ISA ranking. (Intel Corporation 2026d)

15.12.3 Floating-point reduction

If XOR is replaced by binary floating-point addition, the scalar loop still has an accumulator and pointer. Its mathematical freedom changes. Rounded addition generally fails associativity:

$$\text{round}(\text{round}(x + y) + z) \neq \text{round}(x + \text{round}(y + z))$$

for some representable inputs. A balanced tree, several lane accumulators, and the left scalar fold can therefore produce different final bit patterns.

One contract may require the exact left-to-right result. Another may permit a bounded numerical error. A third may specify a reproducible tree. Compiler flags and library interfaces choose among these freedoms. Calling every form “the sum” hides the main semantic decision.

The RISC-V vector specification makes the distinction explicit for floating reductions: it provides ordered and unordered forms with different evaluation requirements. MPI likewise permits order changes based on declared algebra and warns about floating-point results. The systems are solving a common problem at different scales: performance freedom must be granted by the operation’s contract, not inferred from a mathematical notation that ignores rounding.

15.12.4 Distributed reduction

Suppose k workers each hold one chunk. Local XOR uses the scalar or vector theorem. The partial words must then move through a communication tree. The logical combine work remains $k - 1$ for the partials, but elapsed time now includes message latency, bandwidth, synchronization, and slow or failed workers.

A simple communication model assigns each message a startup cost α and each transferred byte a cost β . Sending one 64-bit partial costs roughly $\alpha + 8\beta$ before topology and contention. A balanced tree has logarithmic message rounds under ideal placement. A centralized gather has a short description but can concentrate traffic and work at the root.

Retries are safe for this pure reduction only when a retried chunk does not get counted twice. XOR makes accidental duplication especially deceptive: two copies of the same partial cancel instead of doubling. The wrong answer may look plausible or even become zero. A distributed contract therefore needs chunk identity and exactly-once logical contribution, even if physical work is retried.

MapReduce addresses a broader environment by tracking partitions and re-running failed tasks. MPI collectives assume a different process and failure model. Neither system’s name proves the application operator, grouping, or recovery contract correct. At data-center scale, provenance and control records from Chapter 14 become part of the reduction evidence.

15.12.5 Reuse and economic cost

The transparent scalar listing is attractive for teaching, boot code, small inputs, and a trusted reference. A compiler-generated vector loop can improve throughput across many calls but adds target variants, tail paths, and testing. A hand-tuned assembly family adds further maintenance and dispatch costs. A distributed implementation makes sense only when data volume, location, or latency goals justify communication and operational complexity.

Optimization has a break-even point. Let D be development and validation cost, s the saved time per call, N the expected call count, and V the value of one unit of saved time. The direct benefit is NsV . The change is economically justified only after that benefit exceeds D plus ongoing maintenance, energy, deployment, and failure costs. The equation does not put a universal price on performance. It forces the assumptions into view.

Proof investment follows a similar curve. A machine-checked scalar reference can validate many optimized implementations and survive hardware changes. A full proof of every microarchitecture-specific path may cost more and expire sooner. The useful boundary often proves the stable algorithm and ISA-level contracts, then tests or translation-validates generated variants against that reference.

The scalar proof prepares a parallel proof

Because integer XOR is associative, a vector or tree reduction may partition the region, reduce each part, and combine partial results. Its proof adds a partition lemma and a relation for vector lanes or worker states. It also adds alignment, tail, and extension-specific obligations. Chapter 16 places those states beyond the present scalar core.

15.13 Axeyum route

Axeyum already supplies bit-vector expressions, solver routes, evidence reports, propositional proof checking, selected SMT proof support, and kernel reconstruction machinery. Those general capabilities are useful ingredients. The live checkout does not yet supply the reusable Ao, RV64I, and x86-64 semantic packages required to execute these three listings as book artifacts. This chapter therefore separates the reader proof above from a future machine route below.

The first implementation layer should be Rust. Instruction decoding, typed machine states, step relations, trace replay, memory-range checks, and cross-state relations are trust-bearing core logic. They belong in small Axeyum crates with exhaustive unit tests and negative controls. Python bindings should then expose those checked operations for lessons, notebooks, and artifact runners. Python is the workbench; it should not become a second, quietly different semantics.

An introductory Python session should eventually have this shape:

```
from axeyum.machines import a0, rv64, x86_64
from axeyum.examples import xor_reduce

case = xor_reduce.case(
    address=256,
    words=[0x0f0f, 0x00ff, 0x0ff0],
)
a0_run = a0.run(xor_reduce.a0_bytes(), case.a0_state())
rv_run = rv64.run(
    xor_reduce.rv64_bytes(), case.rv64_state()
)
x86_run = x86_64.run(
    xor_reduce.x86_bytes(), case.x86_state()
)

report = xor_reduce.compare(a0_run, rv_run, x86_run)
report.require_controls()
print(report.boundary())
```

Listing 15.5: Illustrative future Axeyum lesson interface

The listing sketches a future design; the import is not currently supported. The eventual API must return typed outcomes, decoded traces, first-divergence information, semantic-package digests, and control results. It must reject unsupported instructions and malformed bytes. It must not turn a timeout, missing checker, or decode error into successful evidence.

required executable evidence route					
	decoder	step	trace	relation	report
A0	to build	to build	to build	to build	to build
RV64I	to build	to build	to build	to build	to build
x86-64	to build	to build	to build	to build	to build

Figure 15.4: The case study needs separate support at the definition, execution, relation, and certificate layers. A higher layer cannot repair a missing lower binding.

15.13.1 A staged artifact route

The Ao package comes first because its semantics are defined by this book. A complete package needs state types, encoder and decoder, single-step behavior, bounded traces, range-checked little-endian memory, and controls for every in-

struction used here. The XOR-reduction artifact can then execute the Ao listing and compare its terminal state with a direct fold computation.

Each real-machine package needs a pinned source boundary, decoder coverage for the exact byte forms in its listing, and executable step semantics. The first cross-ISA milestone is not the whole loop. It is one load, one XOR, one pointer update, and one branch under explicit state relations. Only after those local steps pass positive and negative controls should the route compose them into the loop invariant.

A bounded solver query can search for a disagreement up to a chosen maximum count and finite memory shape. If it finds one, replay it through all three executors. If it finds none, the report supports only that bound and encoding. The universal reader proof still depends on the stated semantic lemmas. A future certificate route may reduce the bounded query to a checkable formula, but it must bind formula generation to the machine definitions as Chapter 14 requires.

The evidence manifest for this example should name more than three program files. It should bind the fold specification, precondition, observation, machine-package versions, ABI profiles, assembled bytes, decoded listings, initial-state constructors, trace bounds, comparison command, and controls. The report must distinguish successful normal comparison from a trap, bound exhaustion, decode refusal, unsupported instruction, digest mismatch, and control failure.

Two reproduction routes are useful. The lesson route takes a short word list and prints a human-sized trace like the table above. The artifact route consumes raw program and memory bytes and emits a structured report. Both must invoke the same Rust semantics. If the lesson reimplements the machines directly in Python, agreement with the artifact might compare two subtly different definitions.

Table 15.12: Minimum manifest bindings for the three-machine case study.

Binding	Question it answers
Claim and scope	Which width, inputs, outcomes, and exclusions are asserted?
Semantics	Which A0 definition and pinned real-ISA slices interpret bytes?
Programs	Which raw bytes, entry addresses, and decoded instructions run?
Harness	How do logical inputs and successful outputs map to machine states?
Checker	Which command and version decide the comparison?
Controls	Which named mutations must the same route reject?

The empty case is the best first end-to-end control because it crosses every setup and return layer without accessing data memory. The one-word case then fires the load and XOR paths. The two-word case fires the backward edge and pointer update. A test plan built in this order diagnoses more than a single large

example: when a stage first fails, the newly exercised movement is small enough to inspect.

After those concrete cases, symbolic execution can replace the word contents with bit-vector variables. The expected result is the symbolic XOR of the loaded variables. Symbolic addresses should wait until the range model and alias policy are explicit; otherwise one query silently combines instruction correctness with a much harder memory-model question.

The concrete ladder should also include values chosen for semantic edges, not only convenient arithmetic. An all-zero word checks that XOR itself does not manufacture bits. Repeating the same word twice checks cancellation. A word with only the high bit set catches accidental signed reasoning. Alternating bytes make an endian reversal visible. Placing the array at the highest valid starting address exercises the range premise without crossing it. None of these cases replaces the invariant, but each attacks a different connection between the logical fold and a machine trace.

From test ladder to proof obligations

For each concrete case, record the semantic feature it exercises. Then replace the particular values with variables while keeping the same precondition and observation. The resulting obligation should explain why the case generalizes: XOR identity, cancellation, byte reconstruction, range validity, pointer advance, or loop progress. A test with no named obligation is useful for fault finding but carries no clear part of the proof.

The same ladder gives controls a deliberate order. Reverse one load's byte join for the alternating-byte case. Change one branch predicate for the empty case. Advance the pointer by the wrong word size for the two-word case. Remove the memory-frame clause and show that an injected store escapes detection. Each mutation should fail first at the relation component designed to catch it. If a later, unrelated check fails instead, the report is too coarse or an earlier binding is missing.

15.13.2 Shared contracts, separate semantics

The three executors should share generic byte and evidence infrastructure where that sharing is semantically neutral. They should not share one "universal instruction" implementation that erases the architectural differences the comparison is meant to test. Ao, RV64I, and x86-64 need separate state types, decoders, and step functions with a common reporting interface.

A narrow Rust layout could separate five concerns:

1. a word-and-memory crate with checked byte ranges and little-endian loads;
2. one machine crate per architecture slice, owning decode and step;

3. a routine package containing exact bytes, entry constructors, and controls;
4. a relation package mapping each trace to logical cut points;
5. an evidence runner producing the Chapter 14 manifest and typed report.

Sharing the memory load can reduce duplicate code, but it also creates a shared trust boundary. An endian bug there affects every machine equally and can survive three-way agreement. At least one tiny reference should reconstruct words independently from raw bytes. Agreement among architecture executors is most informative when the common substrate is measured rather than forgotten.

The Python layer should expose immutable snapshots or owned copies of typed Rust states. If a notebook can mutate register dictionaries behind the executor's back, replay no longer means running the same transition relation. Python conveniences may select examples, format traces, and plot cut points. They should call Rust for decode, step, range checks, relation evaluation, and evidence outcomes.

15.13.3 The assurance ladder

The first runnable artifact need not wait for a universal theorem, but every stage needs a precise label:

Table 15.13: Incremental assurance for the flagship case. Later rows depend on earlier semantic bindings; they do not strengthen missing ones.

Stage	Direct evidence	Remaining boundary
decode examples	bytes map to expected instruction records	decoder coverage and semantics
concrete traces	selected calls reach expected cut points	untested inputs and reader-to-code link
exhaustive tiny cases	all cases in a finite small domain agree	production width and unbounded count
bounded symbolic check	no disagreement within named bounds	encoding, bounds, and solver trust
checked certificate	exact bounded formula is unsatisfiable	formula generation and semantic bridge
machine-to-logical proof	all admitted states simulate the fold	kernel assumptions and source pins
three-machine corollary	terminal observations agree by common logic	exclusions in the shared contract

Concrete execution should arrive early because it gives readers something to inspect and controls something to break. Exhaustive width-four or width-eight variants can then check decoder and relation shape. Production-width symbolic queries test wider bit behavior under bounded counts. The universal loop proof uses induction and architecture lemmas; it cannot be obtained merely by raising a finite trace bound.

An end-to-end report should show one row per architecture and one row for the derived comparison. A machine row records byte digest, decode coverage, initial-state digest, trace outcome, terminal observation, frame result, and controls. The comparison row records the common logical case, cut-point alignment, first divergence if any, and the exact relation version. A missing machine row makes the comparison unavailable, not partially verified.

15.13.4 The present stopping point

The live Axeyum audit found rich solver and evidence machinery but no reusable Ao, RV64I, or x86-64 executor. Therefore the assembled bytes in this chapter are code-size and decode test vectors, not Axeyum proof artifacts. The reader proof is a mathematical plan over the selected semantics, not a report from a completed kernel route.

Keeping that gap visible protects implementation order. The project should not begin with a three-way solver query over placeholder operations. It should first make each decoder reject unsupported bytes, make each step replay tiny positive and negative examples, and bind the harness. Only then does the cross-machine relation have real states to compare. Solver and certificate machinery amplify a semantics; they cannot supply a missing one.

Several established textbook forms meet in this chapter. A conventional algorithms treatment teaches the loop invariant by asking for initialization, maintenance, and termination. That structure appears here, but the machine listings force two additions: intermediate assertions inside one body and a frame describing state the algorithm must not damage.

Software Foundations makes definitions and proofs executable as Coq scripts. A reader changes an inductive definition or theorem and lets the proof assistant expose the consequences. *Concrete Semantics* pairs informal explanation with Isabelle theories for languages and program reasoning. (Pierce et al. 2026; Nipkow and Klein 2014) Those approaches show the value of keeping the formal object small enough to run and edit.

This case study adds a comparative architecture layer. The logical loop is not compiled away in the exposition. It remains the common proof interface. Each ISA receives its own decoder, state mapping, body theorem, control path, frame, and terminal harness. This arrangement keeps the formal-development virtue of small definitions while exposing the engineering boundaries hidden by source-level pseudocode.

The recommended reading order follows increasing commitment:

1. compute XOR and memory loads for one concrete input;
2. prove the word algebra and fold-splitting lemma;
3. prove the language-independent loop invariant;

4. trace one architecture through all intermediate assertions;
5. prove its machine-to-logical simulation;
6. repeat for the other machines and derive agreement;
7. attack bytes, semantics, relations, frames, and reports with controls;
8. only then compare costs or consider vector and distributed schedules.

This order prevents notation from outrunning understanding. It also prevents benchmark excitement from preceding correctness. A reader first knows what the bytes mean, then why the loop works, then why the implementations agree, and finally which alternative schedules the algebra permits.

15.13.5 Alternating proof and execution

Writing all abstract proofs before executing a byte can allow the semantics to drift away from the artifact. Writing all executors before stating the invariant can produce many tests with no theorem. Alternate the two. Each definition should enable a small execution; each execution should motivate a lemma or control; each lemma should state which bytes and states it will later govern.

For example, the two-word trace motivates H_i , D_i , and C_i . The general body proof explains why those rows extend to every admitted i . The pointer-by-four mutation tests whether the executable relation uses the same pointer lemma. The checked artifact then returns evidence about that bound definition, not a parallel universe of prose.

Exercises also climb this ladder. Computations establish fluency. Proofs make premises explicit. Breaking tasks test whether the route discriminates errors. Cost problems separate abstract and physical measures. Transfer problems ask which structure survives a new ABI, vector schedule, or distributed system. The capstone requires the reader to compose rather than repeat isolated facts.

15.13.6 Why the routine is foundational

The routine is intentionally small, but its proof shape is not small in importance. A production memory kernel still needs an input layout, range premise, loop or vector invariant, frame, calling contract, and cost model. A compiler validation still needs source and target observations, a state relation, and a way to align unequal steps. A published solver result still needs exact bytes, semantics, controls, and an evidence boundary.

XOR removes complications that would obscure this architecture of reasoning. There is no carry, signed overflow question, or floating rounding. The operator is associative and commutative. The array is read-only. The body has one load and one combine. Consequently, every remaining obligation is load-bearing rather

than decorative: if the proof still needs a harness, decoder, range lemma, stuttering relation, and frame here, larger routines will not make those needs disappear.

The example is also falsifiable at every layer. A reader can compute one word, mutate one displacement, invert one branch, corrupt one byte, weaken one observation, or remove one theorem premise. The resulting failure has a named place in the proof. This makes the case suitable for education: beauty comes not from pretending the machine is simple, but from seeing many distinct movements meet one compact invariant.

Future flagship cases can add one complication at a time. Modular sum adds carry reasoning. Minimum adds an order and signedness choice. Copy adds writes and possible overlap. Vector XOR adds lanes, masks, and tails. Concurrent reduction adds interference and a memory model. The present theorem becomes a reusable base and a control: each extension should say exactly which old lemmas survive and which new state enters.

That extension discipline is economically useful too. Reusing a proved load, range, harness, or simulation component lowers validation cost only when its premises still hold. Reuse by name is not enough. A changed vector width, address space, ABI, or fault policy may invalidate the connection while leaving the high-level loop familiar. The dependency ledger should make genuine reuse visible and force changed boundaries back through review. For a classroom proof, that ledger may be a table of lemmas and premises. For an industrial proof, it should also record versions, generated artifacts, tool settings, and the controls that failed when a dependency was deliberately changed.

15.14 Where the model stops

The mathematical argument establishes the XOR-fold loop for the printed logical algorithm. The instruction-level arguments explain how the selected listings instantiate its variables and control. The cross-machine simulation identifies the cut-point relation needed for agreement. These are reader proofs over the semantics stated in this book and the pinned real-ISA sources.

The chapter does not yet present machine-produced Axeyum evidence for the real listings. It does not cover optional compressed or vector instructions, different ABIs, arbitrary misaligned accesses, concurrent memory changes, memory-mapped input, faults outside the precondition, or timing. It does not claim the listings are minimal or fastest. Those exclusions are part of the result's meaning.

The example also marks a transition in scale. We have moved from one instruction to a loop and from one state relation to three executions. Yet the method did not change: define the words, inventory the state, state the observation, prove a local movement, compose movements at named cut points, and bind any machine-derived evidence to the exact claim.

15.15 Exercises

1. **Execute.** Trace all three logical iterations in the table from the byte contents of memory, showing each little-endian reconstruction.
2. Trace the Ao listing for $n = 0$. Name every executed address and explain why no data-memory read occurs.
3. Compute the two Ao branch displacements from the printed addresses and reconstruct both targets from the encoded values.
4. State the loop invariant after the load but before XOR. Which additional temporary value must it name?
5. Prove that XOR reduction is unchanged by swapping two adjacent input words. Explain why that is a weakness for a security checksum.
6. Prove that appending the current reduction value to a region makes the new reduction zero.
7. **Break.** Change the pointer increment from eight to four. Give the smallest count and memory bytes that separate the programs.
8. Change the Ao backward branch displacement from -5 to -4 . Trace the first two visits to the resulting target.
9. Delete the RV64 empty-input branch. Give an input allowed by the original contract that now fails, and classify the failure.
10. Replace `x86 xor eax, eax` with a write to `ax`. Explain which initial register values expose the subregister error.
11. Write the complete read and write sets for one loop body on each machine, including program counter and condition state.
12. Define a relation between the Ao state after `load` and the RV64 state after `ld`. Include the temporary loaded word.
13. **Prove.** Expand the body-correspondence paragraph into one lemma per instruction group, with precondition and postcondition for each lemma.
14. Strengthen the observation to require scratch registers to agree. Show why the printed three-way theorem then fails or becomes unnatural.
15. Weaken the observation by dropping memory preservation. Construct a program that returns the right result while corrupting one input byte.
16. Change the specification from XOR reduction to modular sum. Identify the parts of the invariant and listings that change, and the parts that remain.

17. Add a stride s to the contract. State sufficient no-wrap and readable- range premises for addresses $a + si$.
18. Permit n to use every 64-bit unsigned value. Explain why termination in natural-number arithmetic still needs an address and resource story.
19. Design one positive and four negative controls for the future Axeyum Ao artifact. Name the expected outcome class of each.
20. Design a bounded disagreement query for counts at most three. List its symbolic inputs, transition constraints, observation, and replay requirement.
21. **Transfer.** Rewrite the routine for a different calling convention. Separate instruction semantics from the new register and preservation rules.
22. Compare a scalar XOR reduction with a vector version conceptually. List the additional state, instruction, and reduction-order questions that move the vector proof beyond this chapter's model.
23. **Bit algebra.** Reproduce the addition table for F_2 and prove associativity by checking all eight triples of bits. Explain how the bit proof lifts to width-64 words.
24. View three width-four words as rows of a bit matrix. Compute their XOR by column parity, then verify the same result by hexadecimal XOR.
25. Prove that $x \text{ xor } m \text{ xor } m = x$. State which algebraic laws you used and which one would fail for ordinary addition modulo 2^{64} .
26. Give a nonlinear function on width-two words that does not commute with XOR folding. Supply the smallest counterexample to the claimed homomorphism.
27. Classify each structure as a monoid, commutative monoid, or neither: integer addition with zero, strings with concatenation, subtraction with zero, maximum with the least word, and function composition with the identity.
28. Prove by induction that the left and right XOR folds of a finite list agree. Mark the exact use of associativity and the identity law.
29. **Chunk.** Split a seven-word region after word three. Write the two memory folds and prove that XORing them yields the whole result.
30. Give an associative but noncommutative operation. Explain which balanced reduction trees preserve the sequential result and which leaf permutations do not.
31. Derive the combine work and span for balanced trees with $n = 1, 2, 3, 5, 8$. Draw the partially filled trees rather than quoting the asymptotic result.

32. A machine has four workers. Under an ideal combine model, compare the span of a scalar fold, four equal chunks followed by a balanced merge, and one balanced tree for $n = 16$. State what the model omits.
33. Distinguish reduction from scan. For words $[a, b, c, d]$, write the inclusive XOR scan, exclusive XOR scan, and final reduction.
34. **Range proof.** Show that $a + 8n - 1 < 2^{64}$ implies every byte of every selected word lies below 2^{64} . Treat $n = 0$ separately.
35. Explain why the one-past pointer $a + 8n$ need not be readable. Give a memory layout in which the last word is readable but the one-past address is not.
36. Design an executable no-wrap check that cannot underflow for $n = 0$. Specify its typed outcomes for valid, wrapped, and unavailable ranges.
37. Starting from H_i , derive D_i , C_i , P_i , and H_{i+1} for one concrete two-word memory. Include every unchanged logical component.
38. Give a loop that preserves a plausible accumulator invariant but never terminates. Then give a terminating loop that violates the fold. Explain why invariant and variant obligations are independent.
39. Expand the Ao-to-logical body theorem into five Hoare-style triples. Include helper-register, condition-state, and program-counter premises.
40. Mutate Ao r_4 from eight to sixteen. Choose the smallest region for which the first iteration succeeds and the second load reveals the error.
41. Decode the forward Ao branch displacement $d = 5$ and backward displacement $d = -5$ using the chapter's branch-target equation. Show the intermediate signed values.
42. In Table 15.4, reconstruct the immediate of `addi a1, a1, -1`. Explain why the mathematical decrement is valid only under the positive-count premise.
43. Reconstruct both RV64I branch targets from their split immediate fields. Compare the result with the addresses printed in the table.
44. Explain why disassembling `addi t0, a0, 0` as `mv t0, a0` does not change the semantic proof. What artifact should remain canonical?
45. Strengthen the RV64 return harness to require a particular continuation address. List the alignment, `ra`, target, and terminal-observation obligations.
46. In Table 15.5, compute both short-branch targets from the address following each branch and its signed byte displacement.

47. Change the x86 `add rdi, 8` encoding to an equally long `add rdi, 16`. Which checks still pass, and which intermediate assertion first fails?
48. Draw the x86 zero-flag def-use chain from entry test through one complete body. Mark every instruction that overwrites the flag and every branch that reads it.
49. The x86 loop is described as read-only, yet `ret` reads memory. Reconcile these statements by separating input-region, stack, and global memory footprints.
50. Verify the static byte and instruction totals for all three listings. Then derive the dynamic formulas in Table 15.7 from executed paths rather than pattern matching.
51. For $n = 0, 1, 2, 100$, compute all three dynamic instruction counts. What does the constant setup term contribute for small inputs?
52. Each positive path reads $8n$ data bytes. Add a hypothetical four-byte store per iteration and compare useful operations per transferred byte under a clearly stated traffic model.
53. A system sustains B input bytes per second for a large streaming array. Derive the bandwidth upper bound on 64-bit words reduced per second. Evaluate it for one named hypothetical B , without calling it a benchmark.
54. Design a measurement plan that distinguishes a cache-resident XOR loop from a memory-streaming loop. Name data sizes, warming, repetitions, reported statistics, and environmental controls.
55. **Vector tail.** For a vector capacity of four words and $n = 10$, partition the region into full vectors and a tail. Give separate proof obligations for a scalar cleanup and a masked final vector.
56. Prove that four lane-local XOR accumulators followed by a horizontal XOR equal the scalar fold. State where commutativity is used, if at all, for a fixed round-robin lane assignment.
57. Give three floating-point values for which two parenthesizations of addition can differ after rounding. Explain why the XOR proof cannot simply be copied to floating-point sum.
58. Compare an ordered and unordered floating-point reduction contract. Which permits a balanced tree? Which result properties could a numerical-error contract state instead of bit equality?
59. In the communication model $\alpha + m\beta$, compare a centralized gather and balanced tree for eight workers sending eight-byte partials. Count rounds and messages; state the topology assumptions.

60. A distributed runtime executes one chunk twice and includes both partials. Explain why XOR duplication cancels and design chunk-identity metadata that detects the error.
61. Compare MPI reduction and MapReduce for this example. Separate operator algebra, key grouping, process or worker state, communication, and failure handling.
62. Use the break-even variables D, s, N, V to compare a scalar reference with a vector implementation. Add one maintenance term and show how the threshold call count changes.
63. Draw three separate relations from Ao, RV64I, and x86-64 states to one logical state. Derive RV64I–x86 terminal agreement without defining a direct relation between every intermediate register.
64. Give a faulty stuttering implementation that remains between two cut points forever. Define a local rank that rejects it.
65. Ao takes five steps per body and x86-64 takes four. Explain formally why this prevents lockstep bisimulation but permits the forward simulation used in the chapter.
66. Build one metamorphic test from the chunking theorem and one from input permutation. State which manifest fields must differ between the source and transformed cases.
67. In Table 15.11, choose four mutations and write the exact typed report expected at the first failing boundary. Do not use one generic “false” result.
68. Mutate only the observation so that a trap counts as normal return. Give a case where all instruction semantics are correct but the evidence claim becomes false.
69. Design the Rust types for one machine state, decoded instruction, transition outcome, and cut-point relation result. Explain which types the Python layer may display but must not reinterpret.
70. Map one concrete example through every row of Table 15.13. State the strongest claim supported at each stopping point.
71. **Capstone proof.** Choose one architecture and write a complete machine-to-logical proof package: entry, body, exit, frame, progress, decoder binding, and failure reflection. Then derive agreement with either of the other architectures through the common logical fold.

The Edge of the Model

Ao and the two scalar slices make many program claims precise. They do not contain every state that a modern processor exposes or every behavior that software can observe. The boundary is not a miscellaneous list of hard problems. Each item in this chapter identifies structure absent from the model we built.

That way of organizing omissions matters. “The model does not support floating point” sounds like a missing feature checkbox. “The state has no rounding mode, floating-point exceptions, or representation for NaN” names what a definition must add. One description invites a larger instruction table. The other begins a semantic design.

Every boundary names missing state or missing observation

When a claim falls outside the scalar model, ask what new state can affect the movement and what new observation can distinguish outcomes. Extend those definitions before extending the proof machinery.

16.1 Models as instruments

A scientific model is built for questions. A map that shows streets may omit soil composition; a geological map may omit turn restrictions. Neither is false merely because it cannot answer the other’s question. It becomes misleading when its user forgets the purpose of the abstraction or treats an omitted distinction as an established equality.

Our scalar architectural model answers questions about decoded instructions, word and memory effects, control transfer, normal and trapped outcomes, selected calling conventions, and observations over those states. It can support a theorem that three loops return the same XOR reduction. It cannot, without extension, support a theorem that they consume the same energy, reveal the same information through a cache, behave alike under another thread’s writes, or survive a page-table change.

The recurring error is to ask a strong question of a weak observation. If timing is absent, every timing difference is invisible. If speculative state is absent, a proof cannot constrain speculative accesses. If the program map is immutable, a theorem cannot describe a store that changes future decoded instructions. Silence in the model is not evidence of absence in the machine.

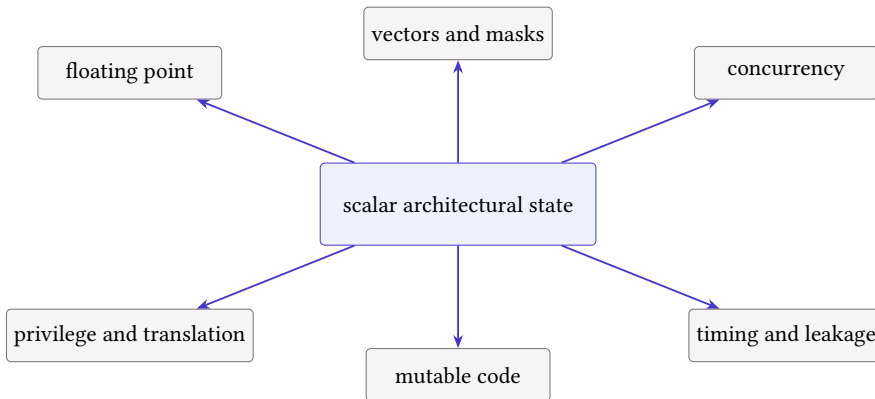


Figure 16.1: The scalar architectural core is surrounded by extensions, each of which adds state, transitions, observations, or all three.

Honest model extension

An extension states the new values and state components, defines how transitions read and write them, enlarges observations only as required, and revisits every preservation or refinement theorem whose premises mention the old state. It also supplies a conservative-embedding argument when old programs are expected to retain their old meaning.

The final clause protects earlier work. Adding vector registers should not quietly change scalar addition. Adding virtual memory should explain when the old flat-address model embeds into the new model, perhaps under an identity translation and stable permissions. The extension may reveal that an earlier claim needed a stronger premise. That is a correction, not a failure of the method.

16.1.1 Abstraction and questions

The idea has roots deeper than computer architecture. Mechanics replaces a body by a point mass when shape is irrelevant to the question. Circuit theory replaces a region of matter by voltage, current, resistance, and capacitance. A finite-state machine replaces a physical controller by states and labeled transitions. In every case, the abstraction is useful because selected observations are preserved, not because the omitted world has ceased to exist.

Computer science makes that choice unusually explicit. Let C be a set of concrete states, A a set of abstract states, and

$$\alpha : C \longrightarrow A$$

an abstraction function. Two concrete states may map to one abstract state. The map forgets distinctions. That forgetting is safe for a claim only if the claim's observation also forgets them. If c_1 and c_2 have the same architectural registers and memory but different cache state, an architectural observation may identify them. A cache-timing observation may not.

The same point can be stated without requiring a function. A relation $R \subseteq C \times A$ can connect several concrete representations to one abstract meaning. A soundness obligation then has the familiar simulation shape: when $c R a$ and the concrete system moves to c' , the abstract system must make zero or more allowed moves to some a' with $c' R a'$. The relation, allowed stuttering, and observation determine what has been hidden.

The commuting square at a model boundary

Begin with related states $c R a$. Take one concrete step $c \longrightarrow_C c'$. Find an abstract execution $a \longrightarrow_A^* a'$ and prove $c' R a'$. Finally show that the declared observations agree. If the concrete step changes a distinction that the abstract observation cannot express, either prove the distinction is irrelevant to this claim or enlarge the abstract state and observation.

This square explains both the power and the danger of a model. Once it commutes, infinitely many physical configurations may be handled through one small theorem. When it does not commute, a proof inside the abstract model may still be impeccable while saying nothing about the concrete case that broke the connection.

16.1.2 Conservative extension

Suppose M_0 is the scalar model and M_1 adds component q . An embedding $E : S_0 \rightarrow S_1$ chooses the M_1 representation of every old state. For old instruction I , the central obligation is

$$s \longrightarrow_{0,I} s' \iff E(s) \longrightarrow_{1,I} E(s').$$

The right side must include the new component's intended value. If q is a rounding mode, an integer instruction should preserve it. If q is a leakage trace, an old load may append an event, so exact state equality is the wrong claim. The embedding theorem may instead compare an old observation with a projection of the richer state. The form of conservativity therefore depends on what the extension makes visible.

Proposition: transport of an old invariant

Let P be an invariant of the old transition system. Assume that every old initial state embeds as a new initial state and that every new step on an old instruction from an embedded state reaches another embedded state exactly as the old step does. Then $P(E^{-1}(s))$ holds on every embedded state reached by an old program in the new system.

Reader proof

Use induction on the length of the new execution. The initial-state premise gives the base case. In the step case, the conservative-step premise produces the corresponding old successor. Apply the old invariant's preservation lemma, then embed that successor. The proof fails immediately if a new transition can enter an unembedded state during the old program.

The proposition is modest but practical. It tells an implementer exactly why regression tests alone are insufficient. Tests sample states. Conservative extension quantifies over every admitted old state and instruction. Tests can attack the theorem; they cannot silently replace its universal premise.

16.1.3 The cost of a model

Richer models can answer richer questions, but they create more states, transitions, cases, serialized fields, and proof obligations. That cost is not merely solver time. Engineers must review specifications, maintain decoders, update test generators, investigate counterexamples, and decide whether old evidence remains applicable. An extension that doubles a state record may multiply the reachable combinations by far more than two.

The economic choice is therefore claim-driven. A compiler team proving a scalar arithmetic rewrite may not need cache replacement state. A cryptographic library evaluating leakage may need address and branch traces but not an exact cycle model. An operating-system team changing page tables needs translation and invalidation behavior that an application proof can assume. The smallest adequate model reduces both false confidence and needless verification expense.

16.2 Floating-point state

An integer word represents one residue modulo a power of two, with optional signed and unsigned readings. A floating-point datum has a different structure: sign, significand, exponent, finite and infinite values, signed zero, and values that represent “not a number.” Operations depend on a destination format and a

rounding direction. They may also report exception conditions. IEEE 754 specifies formats, operations, exception conditions, and default handling for binary and decimal arithmetic (IEEE Computer Society 2019).

The familiar algebraic laws need review. Floating-point addition is generally not associative because intermediate rounding depends on grouping. Two mathematical expressions that are equal over real numbers may produce different floating-point values. NaN comparisons do not behave like an ordinary total order. Positive and negative zero compare equal in selected operations while retaining distinguishable encodings and behavior elsewhere.

A regrouping can change a result

Choose a format and finite values where adding a small y to a large x rounds back to x , but adding y to $-x$ first leaves a representable small result. Then $(x + y) + (-x)$ can differ from $x + (y + (-x))$. A proof that rewrites by real-number associativity would be invalid for that floating-point operation.

A concrete three-bit-significand example makes the lost law visible. Use binary arithmetic with round-to-nearest, ties-to-even, and a wide enough exponent range. Let $x = 4$, $y = 1/2$, and $z = -4$. The exact sum $(4+1/2)$ lies halfway between the adjacent representable values (4) and (5); ties-to-even selects (4). Thus

$$\text{fl}(\text{fl}(x + y) + z) = 0.$$

But $(y+z=-7/2)$ is representable, and adding (4) gives $(1/2)$. Hence

$$\text{fl}(x + \text{fl}(y + z)) = 1/2.$$

Nothing random occurred. Each operation followed one exact rounding rule. The difference arose because a finite representation was applied at two different points in the expression tree.

16.2.1 Representation, rounding, and error

A finite binary value has the conceptual form

$$(-1)^s m 2^e,$$

with restrictions on the significand m and exponent e . A concrete format encodes these pieces into fields and reserves patterns for zeros, infinities, and NaNs. A semantic account must separate the bit encoding, the mathematical value or class it denotes, the exact real operation, and the rounding function that returns to the format. Collapsing these levels makes it easy to prove a theorem about real addition and accidentally apply it to a rounded instruction.

Goldberg’s 1991 tutorial organized floating-point arithmetic around precisely these system consequences: representation error, guard digits, cancellation, exact rounding, special values, exception handling, languages, and compilers (Goldberg 1991). IEEE 754 did not merely choose field widths. It standardized operations and rounding behavior so that programs could rely on a shared arithmetic contract across conforming systems (IEEE Computer Society 2019).

If x is a nonzero exact real result and $\hat{x}$ its represented result, relative error measures

$$\frac{|\hat{x} - x|}{|x|}.$$

This measure is useful away from zero, but it does not replace a bit-level specification. Near underflow, for signed zero, and for NaNs, a numerical error bound and an encoding observation answer different questions. A proof package may need both: one theorem about exact instruction bits and another about the error of a larger numerical algorithm.

Exception flags add history. Two computations can finish with the same result bits while only one raises inexact, overflow, underflow, division-by-zero, or invalid-operation state. If client code reads the flags, result-only equivalence is too weak. If the environment traps on a selected exception, the flag becomes a control-flow boundary rather than passive metadata.

16.2.2 Reproducible floating-point results

Parallel hardware and optimizing compilers make reassociation economically tempting. A balanced sum has less dependency depth than a left-to-right sum. Vector lanes can accumulate partial values concurrently. Fused operations can round once where separate operations round twice. These changes may improve throughput or accuracy for a chosen workload, yet they need not reproduce the same bits as the original evaluation order.

Industrial users therefore choose among contracts. A strict contract can fix format, rounding mode, expression grouping, exception behavior, and perhaps the treatment of subnormal values. A numerical contract can permit different bits while bounding error. A statistical application may accept a tolerance on a final quantity. A debugging or scientific-reproduction workflow may require bitwise repeatability across runs and machines. None is universally best; each has a different optimization surface and validation cost.

The proof should mirror that choice. Bit equality calls for a semantics of every rounding point. An error theorem calls for assumptions about magnitudes, conditioning, and accumulated error. Reproducibility across thread counts calls for a fixed reduction tree or an order-independent method. Writing “approximately equal” without a metric, tolerance, and domain is not a weaker theorem. It is no theorem at all.

An Ao floating-point extension would need floating registers or a declared reuse of integer registers, supported formats, operation semantics, rounding state, exception flags, and conversion rules. The observation must say whether it compares exact encodings, numerical classes, exception state, or some combination. A solver route needs floating-point theories or a sound reduction to bit vectors; a certificate route must cover whichever semantics it uses.

Bit-pattern equality and numerical agreement differ

Two floating-point results may be numerically interchangeable for one client yet have different zero signs, NaN payloads, or exception histories. State the observation before calling an optimization correct.

16.3 Vector state

A packed or vector instruction does not merely apply a scalar operation to a wider word. It introduces lanes, element widths, lane ordering, masks, tail policy, and wider register state. Scalable vector designs may make the active length a state-dependent quantity rather than a constant in the program text. Loads and stores add stride, grouping, fault, and partial-completion questions.

Chapter 15's XOR reduction suggests the appeal. Several words can be loaded and combined in parallel, then the lanes can be reduced to one word. The correctness proof needs a partition of the input region, a mapping from memory indices to lanes, an invariant for partial lane accumulators, and a final horizontal reduction lemma. A tail whose length is not a multiple of the vector width needs masking or a scalar remainder path.

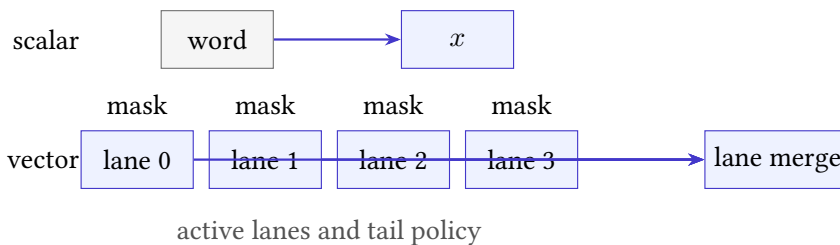


Figure 16.2: A scalar reduction carries one accumulator. A vector reduction adds lane state, an activity mask, a tail policy, and a final lane merge.

The existing scalar proof is still useful. It supplies the specification and the algebraic XOR laws. It does not supply lane semantics or the partition proof. Optional vector extensions belong here rather than at the book's center because the scalar core is the shared foundation for both RISC and CISC architectures; a particular vector extension is one elaboration of that core.

An honest Axeyum vector package should begin with a small lane model and controls for inactive lanes, tail elements, lane order, and masked faults. Old archived shuffle or bit-manipulation artifacts may inform tests, but they do not define the new package’s semantics or establish its claims.

16.3.1 Vector element classes

The ratified RISC-V vector extension gives a useful concrete inventory. It adds 32 vector registers and control state including vector length, vector type, a restart position, fixed-point rounding state, and saturation state (RISC-V International 2026d). For one instruction, element indices can be partitioned into prestart, active, inactive, and tail classes. The partition depends on the restart index, current vector length, mask, element width, and register grouping.

This taxonomy prevents an imprecise rule such as “apply the scalar operation to every lane.” An active element performs the operation and may raise an exception. An inactive masked element does not perform the operation. A tail element lies beyond the current vector length. Prestart elements precede the restart position after a resumable trap. Destination policies say whether inactive and tail elements remain undisturbed or may take an allowed agnostic value.

The agnostic policy is a direct meeting of semantics and implementation cost. If inactive elements had to remain unchanged, a processor using register renaming might need to read and copy values from an old physical destination register. Permitting those values to be overwritten can remove that work when software promises not to observe them. The looser semantic result buys implementation freedom, but it also forbids a proof from assuming convenient zeroes or stable old values.

Table 16.1: Element classes require different proof obligations.

Class	Defining condition	Required argument
Prestart	index below restart position	old destination is preserved; no element exception
Active	in body and mask true	scalar operation, address, result, and possible fault are correct
Inactive	in body and mask false	no operation or fault; destination follows mask policy
Tail	index at or beyond vector length	no operation or fault; destination follows tail policy

For a vector XOR reduction, let A_j be the sequence of input indices assigned to lane j . A suitable cut-point invariant is

$$v_j = \text{xor}_{i \in A_j} M[p + 8i]$$

for every active accumulator lane, together with a disjoint-union statement that the A_j cover exactly the processed prefix. The horizontal reduction then uses associativity and commutativity to merge lane folds. Memory safety still needs separate range facts for each active address. Algebra cannot prove that a masked-off load did not fault unless the vector semantics says it was not performed.

16.3.2 Scalable vector length

In a fixed-width vector ISA, the binary can name a width known during compilation. In a vector-length-agnostic design, one binary can operate across implementations with different physical vector lengths. Software repeatedly requests an element count, receives the length chosen for the next strip, and advances by that amount. The invariant quantifies over the chosen length rather than baking one lane count into the theorem.

This flexibility changes validation. Testing on one implementation exercises only its length choices. A universal proof must cover every permitted length, including zero where allowed, short final strips, masks, overlaps, and restart positions. A compiler cost model must also decide when setup, masking, and horizontal reduction repay their overhead. A vector instruction count is not a speedup theorem; data layout, memory bandwidth, dependency chains, and frequency effects remain outside that count.

The business value can be large because one maintained kernel may span a family of implementations. So can the failure cost: a tail bug occurs exactly where production inputs stop aligning with the most frequently benchmarked length. The proof boundary should make the rare path as explicit as the full vector path.

16.4 Concurrency and event structures

The book's executions are single-threaded sequences. At each step one instruction reads one state and produces one successor. With multiple agents, memory actions can overlap in time, observations can depend on ordering, and a global sequence may be too strong or too weak a description.

Lamport's formulation of sequential consistency asks for results equivalent to one sequential order that preserves each processor's program order (Lamport 1979). Modern weak-memory models permit selected reorderings while constraining them through relations such as program order, reads-from, coherence, and synchronization. The object of study becomes a set or graph of events plus consistency conditions.

Consider two threads that each write one location and read the other. A single-thread proof can establish each thread's local instruction effects. It cannot determine which paired read results the whole system permits. That question needs shared-memory events, an architectural memory model, and an observation containing both threads' outcomes.

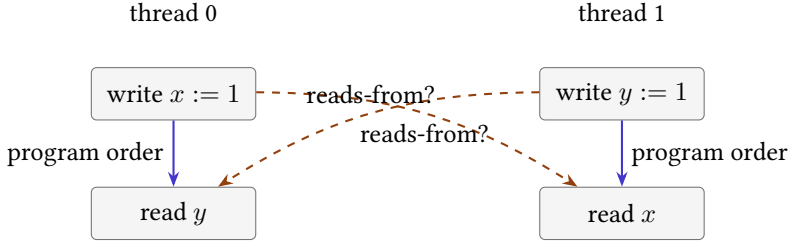


Figure 16.3: Concurrency adds relations across local traces. Program order alone does not determine reads-from or the allowed global observation.

Atomics add indivisibility and ordering contracts. Fences constrain event relations. Read-modify-write operations combine a read and conditional write under special rules. A state extension with merely two program counters is therefore insufficient; it would still lack the memory-event semantics that make the extra execution meaningful.

The proof style changes as well. Invariants may describe shared resources and interference. Refinement may compare sets of allowed executions rather than one successor at a time. A counterexample is an event graph with a forbidden or surprising outcome, and replay means validating every event and relation against the memory model.

16.4.1 A four-event litmus test

Let shared locations x and y begin at zero. Two threads execute

Thread 0: $x := 1$; $r_0 := y$,
 Thread 1: $y := 1$; $r_1 := x$.

Name the writes W_x and W_y , and the reads R_y and R_x . Program order gives $W_x \text{ po } R_y$ and $W_y \text{ po } R_x$. Initial writes I_x and I_y supply zero. The outcome $r_0 = r_1 = 0$ chooses reads-from edges $I_y \text{ rf } R_y$ and $I_x \text{ rf } R_x$.

Under sequential consistency, one total order must extend both program-order edges and explain both reads. For R_y to read zero, it must precede W_y in that order. For R_x to read zero, it must precede W_x . Combining those requirements gives

$$W_x < R_y < W_y < R_x < W_x,$$

an impossible cycle. The all-zero outcome is therefore forbidden by sequential consistency. A weaker architectural model may permit it because stores can become visible to the other thread after the local following load.

Cycle proof for the store-buffering outcome

Do not argue from wall-clock phrases such as “the writes happen first.” Name the events and relations. Derive each order edge from program order or the chosen read value. Compose the four strict edges. Irreflexivity of a strict total order rejects the cycle. For a weaker model, identify exactly which cross-thread order edge is absent rather than declaring the hardware “out of order.”

This tiny program is a **litmus test**: a small concurrent program with a selected final condition used to distinguish memory models or probe hardware. Litmus tests do not prove a complete model correct. They reveal specific allowed or forbidden outcomes and are especially effective as negative controls for a formal rule.

16.4.2 Axiomatic and operational models

An operational model describes machines that move: buffers receive writes, messages propagate, and instructions commit under transition rules. An axiomatic model first proposes an execution graph and then accepts it only if relations such as program order, reads-from, coherence order, and from-read meet global constraints. The first style supports stepwise intuition. The second can state forbidden cycles compactly.

The two presentations need an equivalence theorem if a tool uses one while an explanation relies on the other. Work on x86-TSO provided both an abstract machine with store buffers and an axiomatic account, together with a mechanized relationship and litmus-test validation (Owens et al. 2009). This history matters because vendor prose, observed processors, and formal models did not automatically coincide. A memory model is a negotiated architectural contract, not a transcript of one laboratory run.

Coherence is also narrower than consistency. A coherence relation orders writes to one location. It does not by itself determine the order between a write to x and a read of y . Reads-from says which write supplied a read’s value. From-read connects that read to later writes in coherence order. Architectural rules constrain combinations of these relations. Treating all of them as one vague “memory order” hides the very distinction a weak-memory proof must use.

16.4.3 Composing concurrency contracts

A programmer normally writes atomics in a source language, not raw memory events. The language assigns meaning to atomic orders and data races. A compiler maps those operations to target instructions and fences. The architecture constrains hardware executions. Correctness therefore requires a three-level argument:

1. the source execution satisfies the language memory model;

2. the compilation scheme maps every source constraint to adequate target operations;
3. every permitted target execution corresponds to a permitted source behavior under the stated relation.

A data-race-free theorem may let ordinary code recover a simpler reasoning model under specific synchronization premises. It does not make races harmless or erase the compiler layer. Likewise, adding a fence can reduce the allowed hardware behaviors, but its placement and kind must match the source ordering being implemented.

Industrial cost appears in both directions. Stronger ordering can simplify reasoning yet restrict compiler and hardware scheduling. Weaker ordering can enable performance but transfers complexity to language designers, compiler writers, library implementers, verification tools, and application programmers. The economic question is not merely how many cycles a fence costs. It includes the engineering cost of preventing, reproducing, and diagnosing outcomes that arise only under rare interleavings.

16.5 Privilege and virtual memory

Our scalar memory maps addresses directly to bytes and checks whether a range exists. An operating system and privileged architecture add translation, permissions, modes, exceptions, and control registers. A virtual address may name different physical storage in different address spaces. The same address may be readable, executable, or forbidden depending on privilege and page attributes.

Address translation is itself a movement through state. Page-table entries are read, permissions are checked, and translation caches may retain derived information. A load theorem that begins with “effective address a ” no longer reaches memory directly. It needs a translation result or fault. An instruction fetch and a data load may use related but differently constrained translation paths.

The smallest clean extension can treat translation as an abstract function of virtual address, access kind, and privileged state. A deeper model can expose page walks and translation caches. The right choice depends on the theorem. A user-level functional proof may assume a stable valid mapping. A page-table update proof cannot hide the walk. A side-channel proof may need translation- cache observations that both functional models omit.

Contextual address meaning

In a translated machine, an address does not identify storage by itself. Its meaning depends at least on the address space, access kind, privilege state, translation structures, and permission policy selected by the model.

Interrupts and exceptions add another control source. The next instruction may not be the architectural successor of the current user instruction; execution may enter a handler with saved state. Returning from that handler has its own privilege and restoration rules. A proof that assumes uninterrupted procedure execution must state that premise or model the interference.

16.5.1 Stateful address translation

A useful abstract translation relation has the form

$$\text{translate}(S, v, k, m) \in \text{Ok}(p, a, S') \cup \text{Fault}(f, S').$$

Here S contains privileged and translation state, v is a virtual address, k is the access kind such as read, write, or execute, and m is the current mode or address-space identity. Success returns a physical address p , attributes a , and possibly updated state S' . Failure returns a typed fault f . Allowing S' matters when translation updates accessed or dirty information or fills a translation cache.

Even an abstract function must define page offset and page identity. For a page size 2^q , write

$$v = 2^q \text{vpn}(v) + \text{off}(v), \quad 0 \leq \text{off}(v) < 2^q.$$

A successful mapping replaces the virtual page number with a physical frame number and preserves the offset. Permission checks depend on the whole range, not only its first byte: a multi-byte access can cross a page boundary and require two translations with different outcomes.

A translated address is not yet an authorized access

A page-table walk may produce a physical address while permissions reject the access kind. A permission may allow data reads but not instruction fetch, or allow a privileged mode but not a user mode. Keep translation, authorization, and physical-memory access as distinct proof steps.

16.5.2 Translation caches

A translation lookaside buffer retains derived mappings so that common accesses need not repeat a full page-table walk. That cache is valuable because translation is on the critical path of memory access. It also creates two views of mapping state: page-table memory and cached translations. After a page-table update, an invalidation or synchronization protocol must prevent an obsolete cached mapping from authorizing later use.

The proof pattern resembles self-modifying code. A theorem may not read a page-table entry, change the table, and then reuse the old translation without a stability premise. It must show that the relevant mapping remained unchanged,

or model the invalidation and obtain a fresh result. With several processors, the protocol must also identify which agents have observed the change.

Virtualization can add a second translation stage. A guest virtual address is translated to an intermediate address, which is translated again to host physical storage. Permission and fault information can arise at either stage. The composition looks simple as a mathematical function, but its event semantics can include multiple walks, caches, and invalidations. The model should expose only the depth needed by the theorem while refusing claims that depend on omitted interactions.

16.5.3 Isolation and resource economics

Historically, virtual memory did more than enlarge the apparent address space. It supported protection, relocation, sharing, and the allocation of scarce physical storage among active computations. Denning's working-set model described a process through the information it had recently referenced and connected locality to memory demand (Denning 1968). When too many needed pages are absent, servicing faults can dominate useful execution: the system thrashes.

That history explains why a functional address model and a performance model diverge. The same load can return the same byte whether its translation and data are cached or whether it triggers expensive work. A correctness theorem may abstract those costs. A capacity-planning or latency theorem may not. Page size, working-set size, fault rate, translation-cache reach, memory pressure, and storage latency become relevant observations.

Cloud and container systems add an economic layer. Address-space isolation helps consolidate workloads on shared machines. Overcommit and demand paging can increase useful machine occupancy, but tail latency and failure risk grow when working sets compete. Huge pages can reduce translation pressure while increasing allocation granularity and internal fragmentation. A proof should not choose that policy. It should make clear which functional guarantees survive every policy and which performance claims require measured deployment assumptions.

16.6 Self-modifying code

Ao deliberately keeps an immutable program map separate from mutable data memory. That design makes decoding stable: the instruction at a program counter cannot change because of an ordinary store. Real systems can generate, load, patch, or overwrite code. Once data writes can affect future fetches, the program is part of evolving state.

The naive extension is to use one byte map for both fetch and data access. That is necessary but not always sufficient. Instruction and data views may require architectural synchronization before new bytes are guaranteed to be fetched. Decoders with variable-length instructions may see changed boundaries. Another

thread may race with the modification. Permissions may prohibit simultaneous write and execute access to a page.

A store can invalidate the next proof step

Suppose a proof decodes the instruction at address p , then reasons about a store, then applies the saved decoded instruction at p . If the store can alter bytes in that instruction, the proof has reused stale syntax. It must show that the written range is disjoint from the fetched range or perform the architecture's required synchronization and decode again.

This boundary reaches ordinary tooling. Linkers relocate instructions and loaders place segments before execution begins. Just-in-time compilers create new code while a process runs. Dynamic patching changes code already in use. Each activity needs a phase or transition model that binds bytes to the moment at which their semantics is claimed.

An Axeyum extension should therefore make decode results depend on a code-memory version or digest. A negative control can modify one instruction byte after a cached decode and require the stale step to be rejected. Another can omit a required synchronization event and require the execution route to refuse the stronger claim.

16.6.1 Publishing executable bytes

It is useful to distinguish three moments. During **construction**, a tool writes bytes that are not yet promised as executable instructions. During **publication**, permissions and synchronization make a particular byte version eligible for fetch. During **execution**, a processor fetches and decodes a published version. The phase boundary prevents a proof from treating every partially written buffer as a program.

Let C_t denote code bytes at logical version t , and let $P(h, p) = t$ record the version published to hart or processor h at address p . A fetch is allowed to bind an instruction only from the version named by P . A store creates a newer code version but does not by itself update every publication record. The synchronization protocol changes those records under architecture-specific premises.

RISC-V makes this distinction explicit. The Zifencei extension specifies FENCE.I to synchronize earlier visible data stores with later instruction fetches on the same hart (RISC-V International 2026g). It does not by itself make another hart's instruction fetches observe the writes. A multiprocessor publication protocol must also order the stores and arrange remote instruction-fetch synchronization.

Variable-length x86-64 instructions make partial replacement especially important. Changing a prefix or length-determining byte can alter the boundary and meaning of following bytes. An atomic replacement strategy, stop-the-world

Table 16.2: A code-publication proof crosses several distinct boundaries.

Boundary	Claim	Example control
Generation	emitted bytes encode intended instructions	flip one immediate bit
Visibility	completed stores reach required observers	omit the data-ordering step
Fetch synchronization	later fetches use the new version	omit local or remote instruction synchronization
Permission	the region is executable under policy	retain write-only permission
Concurrency	no observer sees a forbidden partial patch	interrupt a multi-byte change

protocol, indirection through a stable entry, or architecture-supported patching scheme must say which intermediate byte sequences can be fetched. Calling the final bytes correct does not constrain the transition between the old and new programs.

Just-in-time compilation and dynamic patching make this an economic concern, not a curiosity. Publication too often increases synchronization and permission-changing overhead. Publication too rarely delays optimized code or security fixes. Keeping memory writable and executable at once enlarges an attack surface, while switching permissions and coordinating processors costs time. A model can clarify the choices by separating generation correctness, publication correctness, and steady-state execution performance.

16.7 Timing and leakage

Two programs can be functionally equivalent and still take different time, touch different cache sets, contend for different resources, or draw different power. The scalar model projects all of those distinctions away. It cannot establish constant-time behavior because it contains neither a time measure nor the events through which time varies.

The Spectre work demonstrated that speculative execution can affect microarchitectural state and reveal information even when architectural control later discards the speculative path (Kocher et al. 2019). A purely architectural trace can show that the discarded instructions wrote no committed register or memory value. That fact does not describe their cache effects or the time needed by a later probe.

A constant-time model might record an abstract leakage trace containing branch directions and memory addresses while omitting actual cycle counts. A cache model might record sets, tags, replacement state, and hits. A contention model might add execution ports or shared queues. A power model needs a different

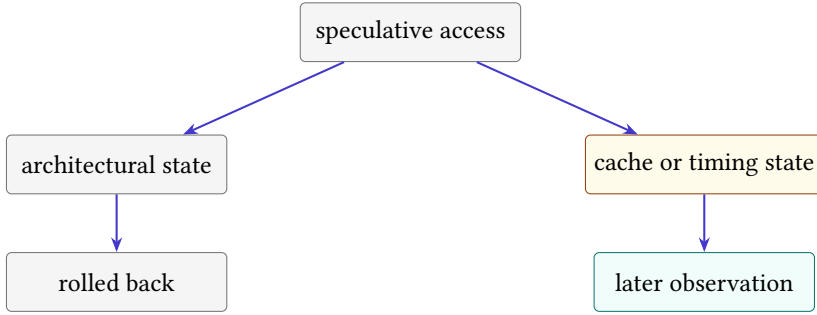


Figure 16.4: Architectural rollback can erase committed effects while a richer observation retains timing or cache consequences.

abstraction again. There is no single side-channel state that answers every leakage question.

Functional equivalence is not a security theorem

Equal architectural outputs under equal public inputs do not imply equal timing, equal memory-access patterns, or equal speculative effects. A security claim must name secrets, public observations, attacker capabilities, and the leakage model.

The proof relation also changes. A **noninterference** statement compares two runs that differ in secret data and requires their public observations to agree. Negative controls should change a secret-dependent address or branch and require the leakage checker to notice. A result-only checker would accept the mutation and thereby expose its own inadequate observation.

16.7.1 Noninterference

Partition an input into public part u and secret part k . Let $L(P, u, k)$ be the leakage observation produced by program P . A basic deterministic noninterference condition is

$$\forall u, k_1, k_2. \quad L(P, u, k_1) = L(P, u, k_2).$$

The quantifiers show why an ordinary functional test is insufficient. One run cannot compare two secrets. A pair of runs with selected secrets is still only a test of those values. A universal proof needs a relational invariant or a symbolic self-composition that tracks two states at once.

Real policies often permit selected secret-dependent output. An authentication routine may reveal success or failure; an encryption routine reveals the ciphertext. The theorem then conditions on an allowed declassification function D : if

$D(u, k_1) = D(u, k_2)$, the remaining public observations must agree. Naming D prevents the phrase “except intended output” from becoming an unlimited escape hatch.

A relational invariant for an address-trace policy

Run two copies of the program with equal public inputs and possibly different secrets. Relate their program counters, public registers, and trace lengths. At every load or store, prove that the two effective addresses are equal, then append them to both traces. At every branch, either exclude direction from the observation or prove equal directions. At return, trace equality follows from the invariant. A secret-indexed table lookup breaks the address step even when both runs return the same value.

16.7.2 Constant-time models

In cryptographic engineering, **constant time** commonly means that selected control flow and memory addresses do not depend on secrets. That discipline aims to remove important timing channels without claiming that every physical execution lasts an identical number of clock cycles. Interrupts, cache state, frequency changes, contention, and analog effects can still vary.

The abstract leakage model is valuable because it supports universal reasoning over program inputs. Measurement is valuable because it can expose effects missing from the abstraction. The two forms of evidence answer different questions. Statistical timing tests can find a distributional difference on a tested platform; failure to find one is not a proof for every platform and input. A source-level address proof can miss compiler-introduced branches or target instructions. Binary-level checking is closer to deployment but still inherits its machine and leakage model.

Current cryptographic guidance treats side-channel behavior as an implementation concern alongside the mathematical algorithm. Information can leak through cache timing, power, electromagnetic behavior, or faults even when the algorithm’s abstract security argument is sound. The economic consequence is a second validation program: teams must maintain protected implementations, inspect generated code, test across supported platforms, and respond when a new microarchitectural channel changes the attacker model.

16.7.3 Speculative effects

Speculative execution predicts a future control path and performs work before the prediction is known to be correct. On a wrong prediction, architectural results are discarded. Spectre showed that discarded operations can still change microarchitectural state in a way a later timing probe detects (Kocher et al. 2019).

A richer model can represent this with two projections. The architectural projection contains committed registers, memory, and control state. The leakage

projection contains selected speculative events. Rollback restores the first projection but need not erase the second. Functional equivalence proves agreement only after the architectural projection; a security theorem must also relate the leakage projections.

Mitigations occupy different layers. A source transformation can remove a secret-dependent access. A compiler barrier can restrict code motion. An ISA barrier can constrain speculation under an architectural contract. Hardware can change prediction or cache behavior. Process isolation can restrict the attacker. Each mitigation needs a model containing the mechanism it claims to control. Otherwise a successful check can certify syntax while the operative channel remains outside the theorem.

16.8 Verified compilation

Our cross-ISA proofs begin with machine programs on both sides. A compiler starts higher: source syntax, types, undefined or unspecified behavior, intermediate representations, optimization passes, assembly, object files, linking, loading, and target execution. Each level has states and observations, and each translation needs a preservation statement.

CompCert showed that a realistic compiler could be developed with a proof of semantic preservation from a substantial C-like source language to assembly (Leroy 2009). Its importance here is methodological. The theorem does not say “the compiler is correct” without qualification. It names source and target semantics and a relation between their behaviors.

Undefined source behavior is a sharp boundary. If a source execution has no promised meaning after an invalid operation, a target behavior need not refine a result the source never specified. Conversely, machine details such as finite memory or resource exhaustion may introduce target outcomes that a source model must accommodate. The exact direction and premises of refinement matter.

Separate compilation adds interfaces between units. Linking adds symbol resolution and relocation. Loading adds addresses, permissions, and initial machine state. A verified compiler theorem cannot by itself establish a claim about a final executable if tools outside the proof or unchecked assumptions break the chain. The evidence manifest must bind the whole route actually used.

Axeyum can contribute solvers, certificates, and kernel reconstruction to local obligations in such a tower. It does not currently provide this book with a verified source language, compiler, linker, loader, and complete real-machine semantics. The right introductory path is to verify one small translation pass or one instruction-selection rule, then compose only after the interfaces and controls are stable.

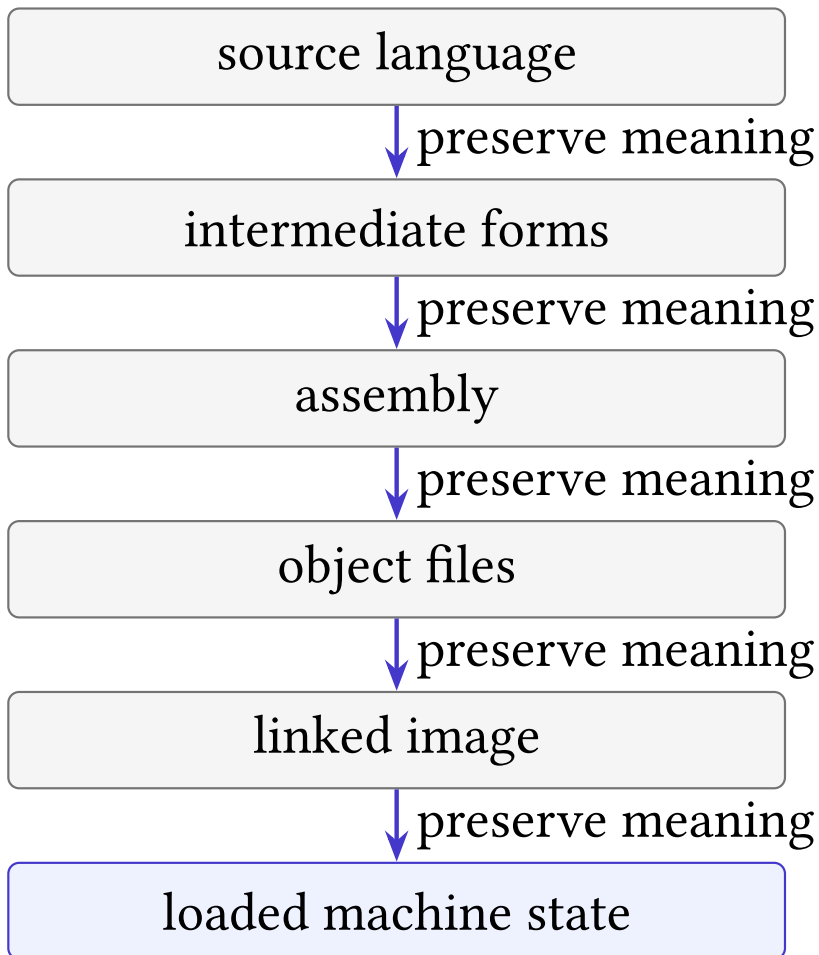


Figure 16.5: A source-to-machine claim crosses several semantic interfaces. Each arrow needs a preservation theorem or an explicitly trusted boundary.

16.8.1 Behavior preservation

Let $\mathcal{B}_S(P)$ be the set of source behaviors of program P , and let $\mathcal{B}_T(C(P))$ be the target behaviors after successful compilation. A simple deterministic slogan would demand equality. Real compiler theorems need more care because source languages can permit several behaviors, target executions can refine choices, and both sides can include termination, divergence, and external interactions.

The CompCert statement is instructive: when compilation succeeds, the observable behavior of generated assembly improves on an allowed source behavior. Its formal endpoints are not source text and an executable file. They are elaborated CompCert C abstract syntax and assembly abstract syntax, before assembling and linking (Leroy 2009). Preprocessing, parsing, object emission, linking, loading, libraries, and the physical target remain separate boundaries unless additional theorems bind them.

Optimization changes syntax aggressively. Dead code disappears. Expressions are reassociated where the source contract permits it. Registers replace variables. Control flow is reshaped. Semantic preservation connects behaviors through simulations between intermediate languages. Requiring identical states at each pass would reject useful compilation and confuse representation with meaning.

One-pass compiler proof

Define source states S , target states T , and a relation $R(S, T)$. Show that related initial states arise from the translation. For each source step, prove that the target takes a finite matching sequence and restores R , using a decreasing rank when the target may stutter. Relate external events and terminal outcomes. Then prove that translation failure is reported rather than treated as a successfully compiled program.

Pass composition is mathematically attractive. If P_1 preserves behavior from language L_0 to L_1 , and P_2 preserves behavior from L_1 to L_2 , their composition preserves behavior from L_0 to L_2 , provided the intermediate definitions and observations match. In practice, that proviso is load-bearing. Serialization, pass options, target data layout, and versioned intermediate forms must agree with the theorem being composed.

16.8.2 Undefined behavior

An undefined source operation is not a nondeterministic instruction that may return any convenient value while the rest of the program retains its usual contract. It removes the source guarantee described by the language semantics. Optimizations can rely on the premise that defined executions do not take that path. A machine trace observed after source undefined behavior therefore does not refute a compiler theorem whose source-side premise has failed.

This boundary creates an engineering obligation. Front ends, sanitizers, static analyses, proof tools, and programmers must establish the source preconditions that the compiler theorem assumes. A verified compiler reduces one class of translation risk. It does not verify application logic or prove that every source execution is defined.

Target-specific failures also need a place in the behavior model. Finite memory, stack exhaustion, external calls, traps, and resource limits can interrupt execution. A theorem may exclude them under premises, represent them as allowed target outcomes, or prove that compilation preserves a source counterpart. Silently dropping them would make the target model stronger than the deployed machine.

16.8.3 The trusted base

Every unverified arrow in the compilation tower becomes a trusted process and a regression surface. A compiler upgrade, assembler change, linker script, library replacement, loader policy, or target revision can invalidate an old end-to-end claim while leaving the source and final output tests unchanged. An evidence manifest should bind versions, options, intermediate hashes, and the theorem or checker used at each arrow.

This accounting has economic value. It shows where independent validation would remove the most consequential trust and where it would merely duplicate a low-risk step. It also shows why “formally verified” is not a property that automatically spreads from one component to a whole product. Assurance grows by composing explicit interfaces; marketing language cannot supply a missing arrow.

16.9 Extending the model

Every extension should pass four gates. The **state gate** inventories new components and preservation rules. The **step gate** defines legal transitions and failure classes. The **observation gate** states which new distinctions the theorem exposes. The **evidence gate** binds executors, formulas, certificates, and controls to those definitions.

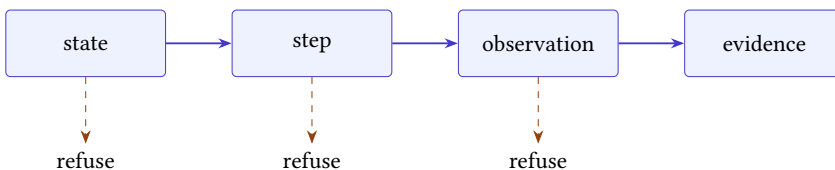


Figure 16.6: An extension reaches evidence only after state, transition, and observation contracts are explicit.

Start with a conservative miniature. A floating-point lesson might support one format, one rounding mode, and addition, while rejecting every other form. A concurrency lesson might support two threads and a tiny event language. A virtual-memory lesson might use one-level translation. Small scope is honest when unsupported cases are visible refusals rather than silent defaults.

The Rust layer should own the typed semantic core and evidence-critical checking. Python should construct examples, call that core, display traces, and help readers mutate controls. This repeats Chapter 15’s division. As the model grows, a single implementation boundary becomes more valuable, not less.

Machine evidence cannot replace the conservative-extension argument. Before reusing an old scalar theorem, show that executing an old instruction from an embedded old state produces the embedded old successor and does not touch new state in an observable way. A counterexample to that property identifies an interaction that the extension must expose.

Template for a conservative extension

Let E embed old states into new states, s be an old legal state, and I an old instruction. Show that the new decoder recognizes the same instruction and that one new step from $E(s)$ reaches $E(s')$ exactly when the old step from s reaches s' . State separately how new components are initialized and preserved. Then prove that the new observation of $E(s)$ agrees with the old observation of s .

16.9.1 The boundary ledger

Before implementing an extension, record one row for every new semantic component. The row should name its type, initial value, readers, writers, preservers, observation, serialization, source authority, and at least one control. A second table should list every old theorem that mentions affected state or observation. This is a dependency ledger for meaning.

Table 16.3: Minimum ledger fields for four representative extensions.

Extension	New component	Principal reader or writer	Control that must distinguish it
Floating point	rounding mode	arithmetic operation; control write	halfway input under two modes
Vector	mask and length	vector step; length-setting step	inactive lane containing a faulting address
Concurrency	reads-from edge	consistency checker	read linked to an incompatible write
Leakage	address trace	load/store step; security observation	secret-indexed access with equal result

The ledger is not a substitute for semantics. It is a completeness check that makes omissions visible early. A field with no reader may be decorative. A writer with no preservation theorem may invalidate old reasoning. An observed field with no serializer may disappear from evidence. A source pin with no version or digest may drift while reports retain an old claim identifier.

Implementation should then proceed in small vertical slices. Define one typed state component. Add one transition that uses it. Display one concrete trace. State one local theorem. Add a positive example and a mutation that the old model would miss. Only after that path works should the project add symbolic inputs or a solver encoding. This order keeps a solver from giving a precise answer about an incomplete semantic object.

Six obligations for accepting an extension

Prove that constructors establish well-formed new state. Prove each new step is defined or refuses with a typed reason. Prove preservation for untouched components. Prove the declared observation is computed from the richer state. Prove conservative embedding for the old scope. Finally, bind executable evidence to exact definitions and show that a semantic mutation fails. If any obligation remains open, report the narrower achieved claim.

16.10 Three boundary failures

The first failure is **silent completion**: an unsupported case is given a convenient default. A floating-point decoder treats an unknown format as binary64. A vector checker treats absent mask semantics as “all lanes active.” A memory model orders events that it does not know how to relate. The tool returns an answer, but the answer belongs to an invented model.

The repair is refusal. Unsupported instructions, formats, privilege modes, events, and observations must have typed failure outcomes. A report can then distinguish “the claim is false” from “this semantic route does not cover the claim.” That distinction is as important at the model edge as it was for solver evidence in Chapter 14.

The second failure is **decorative state**. New fields appear in a state record but no transition reads or writes them. Adding a rounding-mode field does not create floating-point semantics if every operation still uses one hard-coded rounding rule. Adding a cache map does not create a leakage model if speculative and committed accesses never update it. State earns its place by participating in defined movements and observations.

The third failure is **accidental theorem transport**. An earlier proof is reused because the old syntax still parses. Yet a new transition may change an old instruction’s effect, a new observation may reveal an old hidden write, or a new

context may distinguish programs that were interchangeable before. Conservative embedding must be established; textual inclusion is not enough.

Table 16.4: Typical boundary mistakes and controls that expose them.

Mistake	Hidden assumption	Negative control
Silent completion	unknown means default	supply one unsupported form
Decorative state	new field cannot matter	vary it while holding old state fixed
Accidental transport	old theorem survives automatically	expose a new observation or context
Layer collapse	one checker covers every layer	mutate the unbound translation step

A fourth failure, layer collapse, appears when one successful check is allowed to stand for several missing connections. A verified event-graph checker does not show that machine bytes generated that graph. A floating-point solver does not show that an instruction decoder selected the intended format. A compiler proof does not verify an unmodeled loader. The claim-to-evidence chain grows as the model grows; no link becomes optional because another link is strong.

16.11 Questions for every extension

Adding state should immediately produce proof questions. For each new component q , ask how it is initialized, which transitions can read it, which can write it, and which transitions must preserve it. Ask whether it affects legality or only results. Ask whether the observation exposes it directly or through later behavior. Finally ask how a malformed or adversarial value of q reaches a refusal.

For floating point, q might be the rounding mode. Initialization selects a mode; arithmetic reads it; a control-register write may change it; integer addition preserves it. The result observation may reveal it only indirectly through rounded values, while a full-state observation sees it directly. A control changes the mode on a halfway case and requires a changed result.

For concurrency, the new object may be a reads-from relation rather than a register field. Its endpoints must be compatible read and write events, values must agree, and the whole graph must meet the architectural consistency rules. A control redirects one read to an incompatible write and requires rejection. This shows why “state component” should be read broadly: event relations and histories can be semantic state even when no hardware register stores them.

For leakage, the component might be an address trace. Every selected load and store appends an address; non-memory instructions preserve the trace; the security observation projects it. A control changes a secret-dependent index while preserving the final architectural result. The functional checker still accepts, but the leakage comparison must reject.

Write the five questions before the semantics

Choose a boundary component and answer: how is it initialized, who reads it, who writes it, who preserves it, and which observation can distinguish it? Then design one unsupported input and one semantic mutation for the checker.

This worksheet prevents feature lists from outrunning definitions. It also creates the first test plan. Each answer suggests a positive path, a preservation check, and a negative control. When no observation can distinguish the new state, ask whether the theorem truly needs it or whether its effect is only latent in a larger context.

16.12 Axeyum beyond the scalar core

The next Axeyum lessons should grow one semantic dimension at a time. A useful sequence is: one binary floating-point addition with a fixed format; the same operation under two rounding modes; a four-lane integer operation with a mask; two single-event threads under sequential consistency; then one permitted weak-memory outcome. Each lesson adds one state idea and one checker attack.

The Rust package for a lesson should expose constructors that make unsupported cases impossible or explicit. The Python layer should show the state before and after a step, run a positive miniature, fire a named mutation, and print the trust boundary. Only after the miniature is stable should symbolic inputs and solver routes be added.

Kernel reconstruction belongs last in this sequence, not because it is unimportant, but because it can only reconstruct the proposition it receives. First bind bytes or events to semantics. Then validate the semantic relation. Then translate a bounded obligation and check its certificate. A kernel proof over an unbound formula would certify the wrong layer with great confidence.

16.13 Choosing the next boundary

The most attractive extension is not always the best next one. Choose a claim whose missing state is small enough to define completely, whose official sources can be pinned, whose examples fit in a reader trace, and whose controls can fail for recognizable reasons. A fashionable topic with an ambiguous semantic boundary will produce impressive artifacts and weak conclusions.

For this book's workbench, a narrow floating-point operation or a two-thread memory litmus test would be plausible next lessons after the scalar packages exist. A full speculative out-of-order processor or verified optimizing compiler would be a different project. The distinction is one of semantic surface, not ambition.

There is also a maintenance test. A good extension makes its new state visible in definitions, traces, observations, serialization, and controls. If adding one rounding mode or mask bit requires unexplained special cases throughout the old scalar executor, the boundary has not been isolated cleanly. The design should let old scalar claims retain their former meaning while new claims opt into the richer state.

The scalar core remains valuable at every edge. Words still encode fields. States still need inventories. Instructions still produce movements. Proofs still depend on observations. Counterexamples still need replay. Evidence still needs a claim, a checker, and a control. The universal method is not one instruction set; it is the discipline of naming meaning before trusting agreement.

The boundary is therefore an invitation, not an apology. Each omitted concept opens a new version of the book's central mystery: how a finite inscription can govern a movement, how different inscriptions can share meaning, and how a checker can know. The answers become harder as state grows, but they remain approachable when the new distinctions are named one at a time. Precision does not drain the wonder from computation. It lets us see exactly where the wonder has moved.

16.14 Exercises

1. **Execute.** Choose one floating-point format and classify positive zero, negative zero, infinity, and one NaN by their fields.
2. Give a concrete finite floating-point example in which reassociating three additions changes the result. Name the rounding mode.
3. Define an observation that treats all NaNs alike and another that compares their encodings. Give a pair the two observations classify differently.
4. **Break.** Add vector lanes but omit the activity mask from state. Construct two executions that the incomplete model confuses.
5. State a loop invariant for a four-lane XOR reduction with a masked tail. Name the partition represented by each lane.
6. Explain why a scalar proof of XOR associativity is useful but insufficient for a vector memory-access proof.
7. Draw the events for the two-thread store-and-load example. Add program-order and two possible reads-from relations.
8. State Lamport's sequential-consistency condition in your own words. Which part cannot be expressed by two unrelated local traces?

9. Design a negative control that removes one fence from a concurrent program and changes an allowed observation.
10. Add an abstract address-translation function to Ao. List its inputs and its success and fault outcomes.
11. Give premises under which flat Ao memory embeds into that translated model as an identity mapping.
12. **Break.** Allow a store to change program bytes while retaining a cached decode. Give the shortest stale-instruction trace you can.
13. Define a code-memory version check that would reject the stale trace.
14. Give two programs with equal return values and different address traces. Explain why result equivalence does not establish constant-time behavior.
15. Define public and secret inputs for a two-run noninterference claim. State the observation that must agree.
16. Add a speculative cache event to one path and roll back architectural state. Which observation detects the remaining difference?
17. List the semantic interfaces between a C-like expression and a loaded machine instruction. Mark one trusted boundary at each unverified tool.
18. Explain why compiler correctness cannot supply meaning to a source program after undefined behavior.
19. Draft a conservative-extension theorem for adding one floating-point register to Ao while running an old integer instruction.
20. Design positive and negative controls for the four extension gates.
21. **Transfer.** Choose one boundary from this chapter and write a one-page Axiom package plan: types, transitions, observation, source pins, checker route, and refusal cases.
22. Compare two possible next projects—a floating-point addition miniature and a complete weak-memory model—by semantic surface and evidence burden.
23. Define an abstraction function from a cache-bearing machine state to the scalar architectural state. Give two concrete states that it identifies.
24. Construct an observation for which the two cache-bearing states in the previous exercise are distinguishable. Which earlier theorem must change?
25. Prove the invariant-transport proposition by induction, explicitly showing where the conservative-step premise is used.

26. **Break.** Add a new state component whose constructor does not establish its well-formedness condition. Trace the first old theorem that can no longer be transported.
27. In the three-bit-significand system from the chapter, reproduce both parenthesizations of $4 + 1/2 - 4$, including the tie-to-even decision.
28. Give two floating-point computations with the same final result bits but different exception flags. State an observation that separates them.
29. Compare bitwise reproducibility, a relative-error bound, and an interval result as contracts for a parallel sum. Name one client for each.
30. Design a fixed binary reduction tree for n floating-point values when n is not a power of two. State what remains reproducible across thread counts.
31. Partition eight vector element indices into prestart, active, inactive, and tail sets for chosen values of restart position, vector length, and mask.
32. **Break.** Let an inactive vector load raise a fault. Explain which architectural rule and which proof obligation the model violates.
33. Prove that the lane sets in a four-lane strided XOR reduction form a disjoint cover of the processed prefix.
34. Write a vector-length-agnostic loop invariant whose preservation works for every selected strip length from one through the remaining element count.
35. Derive the sequential-consistency cycle for the all-zero store-buffering outcome using only named program-order and reads-from consequences.
36. Give an operational store-buffer execution that permits the same all-zero outcome. Identify the moment at which each buffered write propagates.
37. Explain the separate roles of coherence order, reads-from, and from-read for a three-write, two-read execution on one location.
38. Choose one source atomic order and sketch the compiler and hardware obligations needed to preserve it on RV64 and x86-64.
39. Define translation for a two-page read that begins four bytes before a page boundary. Enumerate all success and fault combinations.
40. Prove that translation preserves the page offset when it succeeds. Explain why this fact alone does not establish access permission.
41. **Break.** Update one page-table entry but retain a stale translation cache entry. Give a trace that accesses the old physical frame.

42. Compare small and huge pages using translation-cache reach, allocation granularity, internal fragmentation, and page-fault cost. Do not assume one is always superior.
43. Model two-stage translation as a composition of partial functions. Distinguish a first-stage permission fault from a second-stage missing mapping.
44. Give a code-publication trace with generation, visibility, fetch synchronization, permission, and execution events. Name the version at each fetch.
45. Explain why a local RISC-V FENCE . I is insufficient to publish new code to another hart. State the additional coordination obligation.
46. **Break.** Patch the first byte of a variable-length instruction while another processor fetches it. List the byte sequences that the model must permit or exclude.
47. State noninterference with an explicit declassification function for a password checker that may reveal only a Boolean result.
48. Self-compose a two-branch program and give a relational invariant that proves its branch-direction trace independent of a secret.
49. Compare a source-level constant-time proof, a binary leakage check, and a timing experiment. State one blind spot of each.
50. Add speculative and committed event lists to a machine state. Define rollback and an observation that preserves the speculative cache access.
51. Formalize semantic preservation for one compiler pass as behavior-set inclusion. State how compile-time refusal appears in the theorem.
52. Draw the complete trusted path from preprocessed source to a loaded instruction for one ordinary toolchain. Mark versions, hashes, and unchecked arrows.
53. Fill the boundary ledger for one new rounding mode, including initialization, reader, writer, preserver, observation, serializer, and negative control.
54. **Capstone.** Select one boundary and produce the full extension case: state delta, step rules, observation, conservative embedding, concrete trace, historical source, industrial tradeoff, positive check, mutation, evidence manifest, and explicit stopping point.

Acknowledgments

Every book is a relay. Thanks are owed to the maintainers of the open tools this template leans on— $\text{\LaTeX}$ and its remarkable package ecosystem, the font designers whose work gives these pages a voice, and the print-on-demand platforms that turned publishing into something one person can finish at a kitchen table.

Thanks also to the readers of the earlier books whose margins, literally and figuratively, taught us which of these conventions survive contact with real paper.

* * *

Any errors that remain are, of course, the author's alone—the template can enforce widow penalties, but not good judgment.

About the Author

Michael J. Bommarito II writes about the craft of making books with software—and makes the occasional book to prove the tooling works. This volume was produced end-to-end with the template it describes: one YAML file of metadata, three font profiles, and a build system that treats a manuscript the way good engineers treat code.

Publisher inquiries may be directed to Bommarito Press.

Sources

- Abadi, Martín, Mihai Budiu, Úlfar Erlingsson, and Jay Ligatti (2005). “Control-Flow Integrity.” In: *Proceedings of the 12th ACM Conference on Computer and Communications Security*, pp. 340–353. DOI: 10.1145/1102120.1102165. URL: <https://cse.usf.edu/~ligatti/papers/cficc.pdf> (visited on 08/30/2026).
- Allen, Frances E. (July 1970). “Control Flow Analysis.” In: *ACM SIGPLAN Notices* 5.7, pp. 1–19. DOI: 10.1145/390013.808479.
- Amdahl, Gene M., Gerrit A. Blaauw, and Jr. Brooks Frederick P. (Apr. 1964). “Architecture of the IBM System/360.” In: *IBM Journal of Research and Development* 8.2, pp. 87–101. DOI: 10.1147/rd.82.0087.
- Apple Inc. (Feb. 18, 2021). *Rosetta 2 on a Mac with Apple Silicon*. Apple Platform Security. URL: <https://support.apple.com/guide/security/rosetta-2-on-a-mac-with-apple-silicon-secebb113be1/web> (visited on 08/30/2026).
- (2026). *About the Rosetta Translation Environment*. Apple Developer Documentation. URL: <https://developer.apple.com/documentation/Apple-Silicon/about-the-rosetta-translation-environment> (visited on 08/30/2026).
- Association for Computing Machinery (Aug. 24, 2020). *Artifact Review and Badging*. Version 1.1. Association for Computing Machinery. URL: <https://www.acm.org/publications/policies/artifact-review-and-badging-current> (visited on 08/30/2026).
- Backus, John W., Friedrich L. Bauer, Julien Green, C. Katz, John McCarthy, Alan J. Perlis, Heinz Rutishauser, Klaus Samelson, Bernard Vauquois, Joseph H. Wegstein, Adriaan van Wijngaarden, and Michael Woodger (Jan. 1963). “Revised Report on the Algorithmic Language ALGOL 60.” In: *Communications of the ACM* 6.1. Ed. by Peter Naur, pp. 1–17. DOI: 10.1145/366193.366201. URL: https://www.algol60.org/reports/algol60_rr.pdf (visited on 08/30/2026).

- Bansal, Sorav and Alex Aiken (2006). “Automatic Generation of Peephole Superoptimizers.” In: *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 394–403. DOI: 10.1145/1168917.1168906.
- Bellard, Fabrice (Apr. 2005). “QEMU, a Fast and Portable Dynamic Translator.” In: *2005 USENIX Annual Technical Conference*. Anaheim, CA: USENIX Association. URL: <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator> (visited on 08/30/2026).
- Blelloch, Guy E. (Nov. 1990). *Prefix Sums and Their Applications*. Tech. rep. CMU-CS-90-190. School of Computer Science, Carnegie Mellon University. URL: <https://www.cs.cmu.edu/~scandal/papers/CMU-CS-90-190.html> (visited on 08/30/2026).
- Böhm, Corrado and Giuseppe Jacopini (May 1966). “Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules.” In: *Communications of the ACM* 9.5, pp. 366–371. DOI: 10.1145/355592.365646.
- Boole, George (1854). *An Investigation of the Laws of Thought*. London: Walton and Maberly.
- Booth, Andrew D. (1951). “A Signed Binary Multiplication Technique.” In: *The Quarterly Journal of Mechanics and Applied Mathematics* 4.2, pp. 236–240. DOI: 10.1093/qjmam/4.2.236. URL: <https://www.ece.ucdavis.edu/~bbaas/281/papers/Booth.1951.pdf> (visited on 08/30/2026).
- Burks, Arthur W., Herman H. Goldstine, and John von Neumann (June 28, 1946). *Preliminary Discussion of the Logical Design of an Electronic Computing Instrument*. Princeton, NJ: Institute for Advanced Study. URL: <https://hdl.handle.net/2027.42/3972> (visited on 08/30/2026).
- Castañeda Lozano, Roberto, Mats Carlsson, Gabriel Hjort Blindell, and Christian Schulte (July 2019). “Combinatorial Register Allocation and Instruction Scheduling.” In: *ACM Transactions on Programming Languages and Systems* 41.3, 17. DOI: 10.1145/3332373. arXiv: 1804.02452. URL: <https://arxiv.org/pdf/1804.02452v5>.
- Computer History Museum (2026a). *Magnetic Core Memory*. Documents the multiple inventors, patent disputes, and predominantly female manufacturing labor behind core memory. Computer History Museum. URL: <https://www.computerhistory.org/revolution/memory-storage/8/253> (visited on 08/30/2026).
- (2026b). *The Stored Program*. Manchester Baby chronology, highest-factor routine, and stored-program historical cautions. Computer History Museum. URL: <https://www.computerhistory.org/revolution/birth-of-the-computer/4/87> (visited on 08/30/2026).

- Cruz-Filipe, Luís, Marijn Heule, Jr. Hunt Warren, Matt Kaufmann, and Peter Schneider-Kamp (2017). “Efficient Certified RAT Verification.” In: *Automated Deduction – CADE 26*. Vol. 10395. Lecture Notes in Computer Science. Springer, pp. 220–236. DOI: 10.1007/978-3-319-63046-5_14. URL: <https://www.cs.utexas.edu/~marijn/publications/lrat.pdf> (visited on 08/29/2026).
- Cybersecurity and Infrastructure Security Agency, National Security Agency, Federal Bureau of Investigation, et al. (Dec. 6, 2023). *The Case for Memory Safe Roadmaps*. Cybersecurity and Infrastructure Security Agency. URL: <https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps> (visited on 08/30/2026).
- Dean, Jeffrey and Sanjay Ghemawat (Dec. 2004). “MapReduce: Simplified Data Processing on Large Clusters.” In: *6th Symposium on Operating Systems Design and Implementation*. USENIX Association, pp. 137–150. URL: <https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/> (visited on 08/30/2026).
- Dennard, Robert H. (June 4, 1968). “Field-Effect Transistor Memory.” U.S. pat. US3387286A. International Business Machines Corporation. URL: <https://patents.google.com/patent/US3387286A/en> (visited on 08/30/2026).
- Denning, Peter J. (1968). “The Working Set Model for Program Behavior.” In: *Communications of the ACM* 11.5, pp. 323–333. DOI: 10.1145/363095.363141.
- Dijkstra, Edsger W. (Mar. 1968). “Go To Statement Considered Harmful.” In: *Communications of the ACM* 11.3, pp. 147–148. DOI: 10.1145/362929.362947. URL: <https://ir.cwi.nl/pub/28228> (visited on 08/30/2026).
- DWARF Debugging Information Format Committee (Feb. 2017). *DWARF Debugging Information Format*. Version 5. DWARF Standards Committee. URL: <https://dwarfstd.org/doc/DWARF5.pdf> (visited on 08/30/2026).
- Floyd, Robert W. (1967). “Assigning Meanings to Programs.” In: *Mathematical Aspects of Computer Science*. Vol. 19. Proceedings of Symposia in Applied Mathematics. American Mathematical Society, pp. 19–32. DOI: 10.1090/psapm/019/0235771.
- Franchetti, Franz and Markus Püschel (2008). “Generating SIMD Vectorized Permutations.” In: *Compiler Construction*. Vol. 4959. Lecture Notes in Computer Science, pp. 116–131. DOI: 10.1007/978-3-540-78791-4_8. URL: <https://users.ece.cmu.edu/~franzf/papers/cc08.pdf>.
- Gauss, Carl Friedrich (1801). *Disquisitiones Arithmeticae*. Sections 1–3 introduce the language and notation of congruence. Leipzig: Gerhard Fleischer.

- GNU Compiler Collection Project (2026). *Machine Descriptions*. URL: <https://gcc.gnu.org/onlinedocs/gccint/Machine-Desc.html> (visited on 08/30/2026).
- Goldberg, David (1991). “What Every Computer Scientist Should Know About Floating-Point Arithmetic.” In: *ACM Computing Surveys* 23.1, pp. 5–48. DOI: 10.1145/103162.103163. URL: https://docs.oracle.com/cd/E19957-01/816-2464/ncg_goldberg.html (visited on 08/30/2026).
- Goldstine, Herman H. and John von Neumann (Apr. 1, 1947). *Planning and Coding of Problems for an Electronic Computing Instrument. Part II, Volume I*. Princeton, NJ: Institute for Advanced Study. URL: <https://www.ias.edu/sites/default/files/library/pdfs/ecp/planningcodingof0103inst.pdf> (visited on 08/30/2026).
- Hamming, Richard W. (1950). “Error Detecting and Error Correcting Codes.” In: *The Bell System Technical Journal* 29.2, pp. 147–160. DOI: 10.1002/j.1538-7305.1950.tb00463.x. URL: <https://www.nokia.com/bell-labs/publications-and-media/publications/error-detecting-and-error-correcting-codes/>.
- Hoare, C. A. R. (Oct. 1969). “An Axiomatic Basis for Computer Programming.” In: *Communications of the ACM* 12.10, pp. 576–580. DOI: 10.1145/363235.363259.
- Horowitz, Mark (Feb. 2014). “Computing’s Energy Problem (and What We Can Do About It).” In: *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323.
- IEEE Computer Society (2019). *IEEE Standard for Floating-Point Arithmetic*. IEEE. DOI: 10.1109/IEEESTD.2019.8766229. URL: <https://standards.ieee.org/ieee/754/6210/> (visited on 08/29/2026).
- Intel Corporation (June 16, 2020). *A Technical Look at Intel Control-Flow Enforcement Technology*. Intel Corporation. URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html> (visited on 08/30/2026).
- (Aug. 2026a). *Intel 64 and IA-32 Architectures Software Developer’s Manual. Volume 1: Basic Architecture*. 253665-092. Version 092. Intel Corporation.
 - (Aug. 2026b). *Intel 64 and IA-32 Architectures Software Developer’s Manual. Combined Volumes 2A, 2B, 2C, and 2D: Instruction Set Reference, A–Z*. 325383-092. Version 092. Intel Corporation.
 - (Aug. 2026c). *Intel 64 and IA-32 Architectures Software Developer’s Manual. Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide*. Version 092. Intel Corporation.

- (2026d). *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Instruction length determination, frontend decoding, and length-changing prefix guidance. Intel Corporation. URL: <https://www.intel.com/content/dam/doc/manual/64-ia-32-architectures-optimization-manual.pdf> (visited on 08/30/2026).

International Business Machines Corporation (1964). *IBM System/360 Principles of Operation*. A22-6821-0. The original edition specifies the five basic instruction formats and their register, address, immediate, and length operand fields. IBM Systems Reference Library. URL: https://bitsavers.trailing-edge.com/pdf/ibm/360/princOps/A22-6821-0_360PrincOps.pdf (visited on 08/30/2026).

Joshi, Rajeev, Greg Nelson, and Keith Randall (2002). “Denali: A Goal-Directed Superoptimizer.” In: *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation*, pp. 304–314. DOI: 10.1145/512529.512566. URL: https://www.bitsavers.org/pdf/dec/tech_reports/SRC-RR-171.pdf.

Kilburn, Tom (Dec. 1947). *A Storage System for Use with Binary Digital Computing Machines*. University of Manchester transcription of the contemporary report. Telecommunications Research Establishment. URL: <https://curation.cs.manchester.ac.uk/computer50/www.computer50.org/kgill/mark1/report1947.html> (visited on 08/30/2026).

Kilburn, Tom, David B. G. Edwards, Michael J. Lanigan, and Frank H. Sumner (Apr. 1962). “One-Level Storage System.” In: *IRE Transactions on Electronic Computers* EC-11.2, pp. 223–235. DOI: 10.1109/TEC.1962.5219356.

Kocher, Paul, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom (2019). “Spectre Attacks: Exploiting Speculative Execution.” In: *2019 IEEE Symposium on Security and Privacy*, pp. 1–19. DOI: 10.1109/SP.2019.00002. URL: <https://www.cs.cmu.edu/~18742/papers/spectre.pdf>.

Kogge, Peter M. and Harold S. Stone (Aug. 1973). “A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations.” In: *IEEE Transactions on Computers* C-22.8, pp. 786–793. DOI: 10.1109/TC.1973.5009159.

Lamport, Leslie (1979). “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs.” In: *IEEE Transactions on Computers* C-28.9, pp. 690–691. DOI: 10.1109/TC.1979.1675439. URL: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/How-to-Make-a-Multiprocessor-Computer-That-Correctly-Executes-Multiprocess-Programs.pdf>.

- Leibniz, Gottfried Wilhelm (1703). *Explanation of Binary Arithmetic*. English translation of “Explication de l’arithmétique binaire”. URL: <https://www.leibniz-translations.com/binary> (visited on 08/27/2026).
- Lengauer, Thomas and Robert E. Tarjan (July 1979). “A Fast Algorithm for Finding Dominators in a Flowgraph.” In: *ACM Transactions on Programming Languages and Systems* 1.1, pp. 121–141. DOI: 10.1145/357062.357071.
- Leroy, Xavier (2009). “Formal Verification of a Realistic Compiler.” In: *Communications of the ACM* 52.7, pp. 107–115. DOI: 10.1145/1538788.1538814. URL: <https://inria.hal.science/inria-00415861v1/file/compcert-CACM.pdf>.
- LLVM Project (2026a). *Auto-Vectorization in LLVM*. LLVM Project. URL: <https://llvm.org/docs/Vectorizers.html> (visited on 08/30/2026).
- (2026b). *TableGen Overview*. Official documentation for declarative target records, including the worked x86 instruction-definition example. URL: <https://llvm.org/docs/TableGen/> (visited on 08/30/2026).
 - (2026c). *Alias Analysis Infrastructure*. Current generic alias-analysis interface and memory-effect contracts. LLVM Project. URL: https://llvm.org/doxygen/AliasAnalysis_8h_source.html (visited on 08/30/2026).
 - (2026d). *LLVM Language Reference Manual*. Integer operations, no-wrap flags, poison values, comparisons, shifts, multiplication, and division. LLVM Project. URL: <https://llvm.org/docs/LangRef.html> (visited on 08/30/2026).
 - (2026e). *LLVM Programmer’s Manual*. BasicBlock class and terminator contract. LLVM Project. URL: <https://llvm.org/docs/ProgrammersManual.html> (visited on 08/30/2026).
 - (2026f). *Machine Block Placement*. Current block-placement implementation using branch probability, block frequency, loop, and profile information. LLVM Project. URL: https://llvm.org/doxygen/MachineBlockPlacement_8cpp_source.html (visited on 08/30/2026).
- Lopes, Nuno P., Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr (June 2021). “Alive2: Bounded Translation Validation for LLVM.” In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. Association for Computing Machinery, pp. 65–79. DOI: 10.1145/3453483.3454030. URL: <https://web.ist.utl.pt/nuno.lopes/pubs.php?id=alive2-pldi21> (visited on 08/30/2026).
- Lu, Sirui and Rastislav Bodík (Oct. 2025). “HieraSynth: A Parallel Framework for Complete Super-Optimization with Hierarchical Space Decomposition.” In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2, 384. DOI: 10.1145/3763162.

- Massalin, Henry (1987). “Superoptimizer: A Look at the Smallest Program.” In: *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 122–126. DOI: 10.1145/36206.36194.
- Message Passing Interface Forum (June 2015). *MPI: A Message-Passing Interface Standard, Version 3.1*. Message Passing Interface Forum. URL: <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report/node111.htm> (visited on 08/30/2026).
- Microsoft Corporation (Apr. 1, 2025). *Overview of x64 ABI Conventions*. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-170> (visited on 08/30/2026).
- MITRE Corporation (2026). *CWE-190: Integer Overflow or Wraparound*. Version 4.20. Common Weakness Enumeration. URL: <https://cwe.mitre.org/data/definitions/190.html> (visited on 08/30/2026).
- Moore, Edward F. (1956). “Gedanken-Experiments on Sequential Machines.” In: *Automata Studies*. Ed. by Claude E. Shannon and John McCarthy. Annals of Mathematics Studies 34. Defines finite sequential machines and studies experiments that distinguish their internal states. Princeton, NJ: Princeton University Press, pp. 129–153.
- Neumann, John von (June 30, 1945). *First Draft of a Report on the EDVAC*. Philadelphia: Moore School of Electrical Engineering, University of Pennsylvania.
- Nipkow, Tobias and Gerwin Klein (2014). *Concrete Semantics: With Isabelle/HOL*. Springer. DOI: 10.1007/978-3-319-10542-0. URL: <https://concrete-semantics.org/> (visited on 08/30/2026).
- Owens, Scott, Susmit Sarkar, and Peter Sewell (2009). *A Better x86 Memory Model: x86-TSO*. Technical Report UCAM-CL-TR-745. University of Cambridge Computer Laboratory. URL: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-745.pdf> (visited on 08/30/2026).
- Patterson, David A. and Carlo H. Séquin (1981). “RISC I: A Reduced Instruction Set VLSI Computer.” In: *Proceedings of the 8th Annual Symposium on Computer Architecture*, pp. 443–458.
- Pierce, Benjamin C., Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, and Brent Yorgey, eds. (2026). *Software Foundations*. University of Pennsylvania. URL: <https://deepspec.github.io/sf/> (visited on 08/30/2026).
- Pnueli, Amir, Michael Siegel, and Eli Singerman (1998). “Translation Validation.” In: *Tools and Algorithms for the Construction and Analysis of Systems*. Vol. 1384. Lecture Notes in Computer Science. Springer, pp. 151–166. DOI: 10.1007/BFb0054170.

- QEMU Project (2026). *Translator Internals*. QEMU Project. URL: <https://www.qemu.org/docs/master/devel/tcg.html> (visited on 08/30/2026).
- RISC-V International (2026a). *Control-flow Integrity*. Ratified Zicfilp landing-pad and Zicfiss shadow-stack specification. RISC-V International. URL: <https://docs.riscv.org/reference/isa/unpriv/unpriv-cfi.html> (visited on 08/30/2026).
- (2026b). *Formal Specification of the RISC-V ISA*. RISC-V International. URL: <https://github.com/riscv/sail-riscv> (visited on 08/30/2026).
 - (2026c). *RISC-V ABIs Specification*. Version 1.0. RISC-V International. URL: <https://docs.riscv.org/reference/abi/> (visited on 08/29/2026).
 - (2026d). *The RISC-V Instruction Set Manual. V Standard Extension for Vector Operations, Version 1.0*. RISC-V International. URL: <https://docs.riscv.org/reference/isa/unpriv/v-st-ext> (visited on 08/30/2026).
 - (Jan. 20, 2026e). *The RISC-V Instruction Set Manual, Volume I. Unprivileged Architecture*. Version 20260120. RV64I base integer instruction set, version 2.1. RISC-V International.
 - (2026f). *The RISC-V Instruction Set Manual, Volume II. Privileged Architecture*. Current ratified specifications library copy inspected on the access date. RISC-V International. URL: https://docs.riscv.org/reference/isa/_attachments/riscv-privileged.pdf (visited on 08/30/2026).
 - (2026g). *Zifencei Extension for Instruction-Fetch Fence, Version 2.0*. RISC-V International. URL: <https://docs.riscv.org/reference/isa/unpriv/zifencei.html> (visited on 08/30/2026).
- Robinson, J. Alan (Jan. 1965). “A Machine-Oriented Logic Based on the Resolution Principle.” In: *Journal of the ACM* 12.1, pp. 23–41. DOI: 10.1145/321250.321253. URL: <https://www.cs.tufts.edu/~nr/cs257/archive/john-alan-robinson/resolution.pdf> (visited on 08/30/2026).
- Rust Project Developers (July 14, 2026). *Primitive Type u8. Rust Standard Library Documentation*. Version 1.97.1. Documents checked, wrapping, overflowing, strict, and saturating arithmetic method families. Rust Project.
- Schurr, Hans-Jörg, Mathias Fleury, Haniel Barbosa, and Pascal Fontaine (2021). “Alethe: Towards a Generic SMT Proof Format.” In: *Electronic Proceedings in Theoretical Computer Science* 336, pp. 49–54. DOI: 10.4204/EPTCS.336.6. URL: <https://www.verit-solver.org/papers/pxtp2021.pdf> (visited on 08/29/2026).
- Shannon, Claude E. (Dec. 1938). “A Symbolic Analysis of Relay and Switching Circuits.” In: *Transactions of the American Institute of Electrical Engineers* 57.12, pp. 713–723. DOI: 10.1109/T-AIEE.1938.5057767.

- Smith, James E. (1981). "A Study of Branch Prediction Strategies." In: *Proceedings of the 8th Annual Symposium on Computer Architecture*. IEEE Computer Society, pp. 135–148.
- Sperry Rand Corporation (1970). *UNIVAC 1108 Multi-Processor System: Processor and Storage. Programmer's Reference*. Revision 1. UP-4053. Copyright dates 1966 and 1970; sections 4.2.1–4.2.2 specify ones'-complement signed numbers and two signed zero patterns. UNIVAC Division, Sperry Rand Corporation.
- Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos (Aug. 2019). "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." In: *28th USENIX Security Symposium*. USENIX Association, pp. 1393–1410. URL: <https://www.usenix.org/system/files/sec19-torres-arias.pdf> (visited on 08/30/2026).
- Turing, Alan M. (1937a). "On Computable Numbers, with an Application to the Entscheidungsproblem." In: *Proceedings of the London Mathematical Society*. 2nd ser. 42.1, pp. 230–265. DOI: 10.1112/plms/s2-42.1.230.
- (1937b). "On Computable Numbers, with an Application to the Entscheidungsproblem." In: *Proceedings of the London Mathematical Society*. 2nd ser. 42.1, pp. 230–265. DOI: 10.1112/plms/s2-42.1.230.
- University of Manchester (2026). *Manchester Mark 1*. Working-machine chronology and B-line address-modification registers. University of Manchester. URL: <https://curation.cs.manchester.ac.uk/computer50/www.computer50.org/mark1/MM1.html> (visited on 08/30/2026).
- Wetzler, Nathan, Marijn J. H. Heule, and Jr. Hunt Warren A. (2014). "DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs." In: *Theory and Applications of Satisfiability Testing – SAT 2014*. Vol. 8561. Lecture Notes in Computer Science. Springer, pp. 422–429. DOI: 10.1007/978-3-319-09284-3_31. URL: <https://www.cs.cmu.edu/~mheule/publications/drat-trim.pdf> (visited on 08/29/2026).
- Wheeler, David J. (1952). "The Use of Sub-routines in Programmes." In: *Proceedings of the 1952 ACM National Meeting (Pittsburgh)*. Pittsburgh, PA: Association for Computing Machinery, pp. 235–236. DOI: 10.1145/609784.609816.
- Wilkes, Maurice V. and John B. Stringer (Apr. 1953). "Micro-programming and the Design of the Control Circuits in an Electronic Digital Computer." In: *Mathematical Proceedings of the Cambridge Philosophical Society* 49.2, pp. 230–238. DOI: 10.1017/S0305004100028322.
- Wilkes, Maurice V., David J. Wheeler, and Stanley Gill (1951). *The Preparation of Programs for an Electronic Digital Computer. With Special Reference to the EDSAC and the Use of a Library of Subroutines*. Cambridge, MA: Addison-Wesley.

- Williams, Frederic C. and Tom Kilburn (Sept. 25, 1948). “Electronic Digital Computers.” In: *Nature* 162, p. 487. DOI: 10.1038/162487a0.
- Williams, Samuel, Andrew Waterman, and David Patterson (Apr. 2009). “Roofline: An Insightful Visual Performance Model for Multicore Architectures.” In: *Communications of the ACM* 52.4, pp. 65–76. DOI: 10.1145/1498765.1498785. URL: <https://escholarship.org/uc/item/78h8v7mr> (visited on 08/30/2026).
- x86 psABIs Project (Sept. 26, 2023). *System V Application Binary Interface. AMD64 Architecture Processor Supplement*. Version 1.0. x86 psABIs Project. URL: <https://gitlab.com/x86-psABIs/x86-64-ABI/-/wikis/uploads/221b09355dd540efcbe61b783b6c0ece/x86-64-psABI-2023-09-26.pdf> (visited on 08/29/2026).

Glossary

A

application binary interface. A binary-level agreement about calling, register use, stack layout, data representation, and other boundaries between separately built program components.

architectural state. The machine components that an instruction-set model exposes to software and uses to determine architectural behavior. See also machine state; microarchitecture.

B

basic block. A maximal instruction sequence entered at its first instruction and left only after its last instruction.

bisimulation. A relation in which each side can match every admitted step of the other while preserving the relation.

branch prediction. A processor technique that guesses a control-transfer outcome or target before the architectural predicate has been resolved.

C

candidate language. The exact finite or recursively described set of programs allowed to compete in a search or minimality claim.

canonical encoding. The one designated byte representation chosen for a decoded instruction when several byte strings could otherwise express the same operation.

carry out. The bit beyond a fixed word's highest position when an unsigned addition is evaluated at one extra bit of width. See also signed overflow; wraparound.

certificate. A finite evidence object that a separate checker can validate without repeating the producer's entire search.

condition code. A stored arithmetic fact, such as zero, carry, sign, or overflow, that a later instruction may inspect.

constant time. A program property defined relative to a leakage model in which the declared observations do not vary with secret data.

continuation. A permitted sequence of computation that runs after the program fragment currently being studied.

control-flow graph. A graph whose vertices represent program regions and whose edges represent possible transfers of control.

control-flow integrity. A policy that restricts indirect control transfers to a declared set of permitted targets.

cost model. A function that assigns a declared cost to a candidate program in a stated machine and execution context.

counterexample. A concrete admitted input or execution that violates a universal claim.

D

data memory. The finite byte-addressed map that holds the mutable data state in the Ao teaching machine.

decoder. A function that reads instruction bytes and returns either one structured instruction or a declared rejection.

E

effective address. The address value calculated from an instruction's address-forming operands before the memory access is attempted.

evidence manifest. A record that binds a scoped claim to semantic inputs, artifacts, checking commands, expected outcomes, and controls.

execution trace. A sequence of machine states in which every adjacent pair is connected by the declared single-step relation.

F

forward simulation. A state relation under which each source step can be matched by an allowed target execution while related states are preserved.

frame condition. The part of a transition rule that states which components of machine state remain unchanged.

I

implicit effect. A state change required by an instruction even though the changed component is not printed as an operand.

independent checking. Validation by a checker whose logic and inputs are sufficiently separate from the evidence producer to catch relevant producer errors.

initial-state premise. A condition that restricts the machine states from which a claim promises to hold.

instruction encoding. The rule that represents a structured instruction as bytes or fixed-width fields.

instruction semantics. The transition rule that assigns an instruction its effect on machine state.

invariant. A property that holds initially and is preserved by every relevant transition.

K

kernel reconstruction. Rebuilding a result as a term or derivation accepted by a small proof-assistant kernel.

L

leaf procedure. A procedure whose reachable control flow makes no nested procedure call.

link value. The continuation address recorded by a call-like control transfer for a later return.

little-endian. A byte order that stores the lowest-weight byte of a multibyte word at the lowest address.

M

machine state. The complete information that a machine model uses to determine or constrain its next behavior. See also architectural state.

mask. A fixed-width word used to select, clear, preserve, or test chosen bit positions.

memory aliasing. The condition in which two access expressions may designate overlapping bytes in the same memory.

memory injection. A relation mapping live source memory blocks and offsets into target blocks while preserving the required permissions and contents.

memory safety. A property that restricts memory operations to objects, regions, lifetimes, and permissions allowed by the governing language or system policy.

minimality. The conjunction of a valid candidate at some cost and a lower bound excluding every cheaper candidate in the declared language.

N

negative control. A deliberately false claim or damaged artifact that the same checking route must reject for a declared reason.

non-leaf procedure. A procedure with a reachable nested call and therefore an obligation to preserve its own continuation and required state.

noninterference. A two-execution property stating that changes to secret inputs cannot change the observations declared public.

O

observation. A declared function that selects the part of a machine result a claim requires two executions to agree on.

operand form. A rule that says where an instruction obtains an input or destination and how that value is read or written.

P

partial correctness. The claim that a program satisfies its postcondition whenever it terminates normally from an allowed initial state. See also total correctness.

precondition. A predicate selecting the initial states or inputs on which a program claim promises to hold.

program. A finite collection of encoded instructions together with the address and rules used to fetch and execute them.

program equivalence. Agreement of two programs under a declared precondition, observation, and treatment of outcomes.

proof certificate. A certificate whose successful checking establishes a formal proof of the encoded claim.

provenance. The recorded origin and transformation history of source material, semantic inputs, artifacts, and published evidence.

R

reachable state. A state obtained from an admitted initial state by zero or more declared machine steps.

read footprint. The architectural information an instruction must inspect to determine its successor. See also write footprint.

refinement. A directional relation requiring every admitted behavior of an implementation to correspond to behavior allowed by its specification.

refutation. A proof that a formula has no satisfying assignment.

rotation. A fixed-width bit operation that moves positions around a cycle without discarding any bit.

S

serialization. An encoding of structured state or data as a sequence suitable for storage or transfer.

signed overflow. The condition in which the mathematical signed result lies outside the range represented by the chosen word width. See also carry out; wraparound.

single step. One application of the machine's transition rule to a running state and the instruction selected for execution.

stack frame. The region of stack memory governed by one active procedure invocation's layout and preservation rules.

state relation. A predicate stating when states from the same or different machines represent corresponding situations.

stuttering. A simulation allowance in which one machine takes finite internal steps while the related abstract state does not advance.

superoptimization. Search for a lowest-cost program in an explicitly declared candidate language under a stated equivalence and cost model.

T

tail call. A call performed by transferring the current procedure's continuation obligation to another procedure instead of adding a new return point.

total correctness. Partial correctness together with a proof that execution terminates under the stated premises.

translation validation. Checking a particular translation result rather than proving the translator correct for every possible input program.

V

vacuous proof. A formally accepted argument that establishes a claim only because an unintended empty domain, false premise, or disconnected encoding removed the intended cases.

variant. A value in a well-founded order that decreases as a loop progresses and supports a termination argument.

verified compilation. Compilation accompanied by a proof that translated programs preserve the behaviors required by the source and target semantic contracts.

virtual address. An address interpreted through a translation and protection context before it identifies physical storage or a fault.

W

word. A fixed-width pattern of bits interpreted under explicitly declared operations and readings.

wraparound. Fixed-width arithmetic returning to the low end of the representative range after crossing its upper boundary.

write footprint. The architectural components an instruction may change during its transition. See also read footprint.

Index

- Ao teaching machine, xx
- ABI, *see* application binary interface
- aliasing, 93
- alignment, 92
- application binary interface, 263
- architectural state, 35
- assembly language, 55
- Axeyum, xxix

- basic block, 117
- binary notation, 3
- bisimulation, 307
- Boolean operation, 10
- borrow, 206
- bounded execution, 119
- branch instruction, 116
- branch prediction, 242
- byte order, 20

- calculation
 - universal rules, xvii
- candidate language, 357
- canonical encoding, 147
- carry out, 15
- certificate, 397
- CISC, xx
- concurrency, 485
- condition code, 195
- constant time, 492
- contextual equivalence, 124
- continuation, 39
- control transfer, 232
- control-flow graph, 117
- control-flow integrity, 246

- cost model, 366
- counterexample, 311
- cross-ISA translation, 325

- data memory, 81
- decoder
 - totality, 140

- effective address, 176
- equivalence
 - contextual, 310
 - full-state, 303
 - observational, 303
- evidence
 - trust classes, 395
- evidence manifest, 399
- execution trace, 113

- flags, *see* condition code
- floating-point arithmetic, 480
- forward simulation, 307, 336
- frame condition, 31, 434
- full adder, 201

- implicit effect, 59
- independent checking, 408
- initial-state premise, 39
- instruction
 - decoding, 156
 - encoding, 137
- instruction encoding, 137
- instruction semantics, 53
- invariant, 44
- ISA, *see* instruction set architecture

- kernel reconstruction, 409
- leaf procedure, 271
- link value, 232
- linker, 152
- little-endian, 20
- liveness
 - condition state, 216
- load instruction, 86
- location, 169
- lower bound, 371
- machine claim, xxii
- machine state, 31
- mask, 10
- memory
 - address translation, 95
 - aliasing, 93
 - alignment, 92
 - byte-addressed, 81
 - cost, 99
 - safety, 102
- memory aliasing, 93
- memory fault, 96
- memory injection, 334
- memory safety, 102
- minimality, xxiv, 371
- model boundary, 477
- modular arithmetic, 6
- negative control, 402
- non-leaf procedure, 271
- noninterference, 492
- object ledger, xxx
- observation, 39
- observational equivalence, 39
- operand, 59
 - explicit, 67
 - implicit, 67
- operand form, 59
- partial correctness, 122, 299
- precondition, 301
- procedure, 263
- program, xxvii, 109
- program equivalence, 297
- proof
 - certificate, 391
 - kernel reconstruction, 409
 - machine proof, xxvi
 - negative controls, 402
 - reader proof, xxvi
- proof artifact, *see* certificate
- proof certificate, 391
- provenance, 399
- reachability, 44
- reachable state, 44
- read footprint, 65
- reduction algorithm, 427
- refinement, 307
- reflexive transitive closure, 114
- refutation, 374
- RISC, xx
- rotation, 13
- self-modifying code, 490
- serialization, 38
- shift, 13
- side channel, 492
- sign extension, 18
- signed integer, 15
- signed overflow, 15
- single step, 65
- stack, 267
- stack frame, 267
- state, 30
 - architectural, 35
 - physical representation, 35
- state relation, 330
- store instruction, 86
- stored-program computer, 109
- stuttering, 307, 336
- superoptimization, 353
- tail call, 285
- total correctness, 122
- translation validation, 317

- truncation, 18
- unsigned integer, 15
- vacuous proof, 406
- variant, 120, 438
- vector instruction, 483
- verified compilation, 495
- virtual address, 95
- virtual memory, 95
- word, 6
 - notation, 5
- wraparound, 6
- write footprint, 65
- zero extension, 18

Colophon

This book was designed and typeset with Lua^ATeX from a single-source template: metadata, trim, typography, and edition structure all derive from one configuration file, and every output format—print interior, bleed variant, and ebook—builds from the same LaTeX sources.

The text is set in Libertinus Serif, with Libertinus Sans for titling accents and Libertinus Mono for code, at a base size of 11pt on a 7.0in by 10.0in trim.

Interior architecture: a modular preamble with a four-layer color system, semantic callout boxes, and micro-typographic protrusion tuned per typeface. Citations are managed with biblatex and biber.

Bommarito Press · 2026